SRS: System Information Viewer for Linux

User guide, section-by-section reference, command reference and glossary

I wanted a Linux version of SIV (System Information Viewer) for myself, so I built one and set out to improve on it along the way. SRS is free and open source for the Linux and open-source community. (SRS is an independent project, not affiliated with SIV or its author.)

Contents

1. SRS System Information - User Guide 3
2. SRS Section Reference 79
3. Processors (SRS Section Reference) 98
4. Machine / BIOS / Memory (SRS Section Reference) 127
5. Sensors / Cooling / Power (SRS Section Reference) 172
6. Volumes / Drives (SRS Section Reference) 194
7. Network (SRS Section Reference) 222
8. Linux (SIV's Windows items) (SRS Section Reference) 251
9. Graphics / Monitors / Audio (SRS Section Reference) 305
10. Buses / Devices (part 1 of 2) (SRS Section Reference) 354
11. Buses / Devices (part 2 of 2) (SRS Section Reference) 395
12. Tools (SRS Section Reference) 425
13. SRS (SIV items) (SRS Section Reference) 467
14. SRS - Command-Line Reference 506
15. SRS Glossary 534

SRS System Information - User Guide

Version 2.4.0. Package `srs-sysinfo`. Programs: `srs-gui`, `sysreport-search`, `srs-root`, `srs-sysreport`.

I wanted a Linux version of SIV (System Information Viewer) for myself, so I built one and set out to improve on it along the way. SRS is free and open source for the Linux and open-source community. (SRS is an independent project, not affiliated with SIV or its author.)

> **Credit.** SRS ("System Report Search") is modelled on **SIV64X - System Information Viewer**. SIV is the work of **Ray Hinchliffe, RH Software** (https://rh-software.com). All credit for SIV's design, its item set and its presentation belongs to Ray Hinchliffe.
>
> **No SIV code.** SRS contains **no SIV code, binaries or data files**, and SIV was **not disassembled or decompiled**. SRS was specified only from SIV's observable output (its `-save` text dump, its panels and its printable strings) and written independently in C on Linux data sources (sysfs, procfs, CPUID, hwmon, udev, hwdata). See `EVIDENCE.md` in the source.
>
> **Not affiliated.** SRS is an independent project. It is **not affiliated with, endorsed by or supported by RH Software**. Please do not send SRS questions or bug reports to RH Software. "SIV" and "System Information Viewer" are names of RH Software's product, used here only to say what SRS is modelled on.

Other pages:

- SECTIONS/README.md - **Section Reference**: every one of the 558 items, field by field, what each value means, what is normal and what is not. Start here to *learn*.
- CLI-REFERENCE.md - every command-line command and option.
- GLOSSARY.md - every technical word, in plain English.

Contents

1. What it is, in plain words
2. Who it is for
3. Install
4. First launch
5. Root-only data (srs-root)
6. A tour of the main window
7. The panels and their buttons
8. The Menu
9. Tools
10. Options
11. System tray
12. Keyboard shortcuts

13. Reports and exports
14. Settings and files
15. Reading results and comparing runs
16. Known limits and differences from SIV
17. Privacy: before you share a report
18. Troubleshooting
19. Uninstall
20. FAQ
21. Glossary

---

1. What it is, in plain words

SRS tells you **what is inside your computer and how it is doing right now**.

It lists your processor, memory sticks, motherboard, drives, graphics card, monitors, USB and PCI devices, network adapters, sensors (temperatures, fan speeds, voltages) and a great deal of operating-system detail. Most values update live.

It also has tools:

- **Stress & Burnin** - load every CPU thread and / or the graphics card to check the PC is stable and cool enough.
- **Benchmark** - classic Dhrystone and Whetstone CPU scores.
- **IPI Delay** - how long cores take to signal each other.
- **Latency** - how long memory takes to answer, by block size (shows your cache levels).
- **Device ID Lookup** - turn a number like `1002:7550` into a device name.
- **Find** - search a saved full system report.

Its main window puts everything in one place: an **Overview** that answers "Is my PC OK?", and one page per topic (processor, memory, sensors ...). For SIV users, the **classic view** looks and behaves like SIV, the long-running Windows tool, so they feel at home. It has all 538 of SIV 5.87's report items (558 in total with SRS's own extras):

- **364** show real Linux data.
- **136** are hardware most PCs do not have (Itanium, IPMI, RAID cards...). On a normal PC they say "not present", as SIV would.
- **38** are Windows-only ideas. Most show the Linux equivalent instead (Windows services -> systemd units, drivers -> kernel modules, registry -> `/etc` and `~/.config`, Windows Error Reporting -> `coredumpctl`).

SRS is read-only. It never changes a setting, never writes to hardware or firmware, and makes no network traffic except one `ping` to your router when you open the Ping item (and, only when you ask, the Network Benchmark's internet test and Update hardware lists). The places that change something are the Startup manager (6.9), only when you click its switches or buttons (it turns startup items off and on, and never deletes anything), and, since 2.4, ACPI method evaluation for experts (9.22), which runs a BIOS method you type, only after a one-time warning.

2. Who it is for

- **Beginners** who want to know what hardware they have ("which graphics card? how much RAM? is my RAM at its rated speed?") without opening the case.
- **Enthusiasts and tuners** who watch temperatures, clocks and fan speeds, run a burn-in after a change, and compare before and after.
- **SIV users moving to Linux** who want the same layout and item names (the classic view).
- **Helpers and support people** who need a full text report from someone else's PC.
- **Power users** who script it: everything is also available as plain text from the command line.

3. Install

SRS runs on 64-bit x86 Linux with GTK 3. The command line (`sysreport-search`) also runs without a desktop, for example over SSH.

3.1 Arch Linux and derivatives - AUR

```
yay -S srs-sysinfo
```

or by hand:

```
git clone https://aur.archlinux.org/srs-sysinfo.git
cd srs-sysinfo
makepkg -si
```

3.2 Arch Linux - prebuilt package from the release

```
sudo pacman -U srs-sysinfo-2.4.0-1-x86_64.pkg.tar.zst
```

3.3 Optional extras (recommended)

SRS works on its own. These packages let it show more (Arch names; other distributions use similar names):

Package	What it adds
`libayatana-appindicator`	System tray icon
`polkit`	The password prompt for `srs-root`
`dmidecode`	Memory stick, board and BIOS details, serials (via srs-root)
`smartmontools`, `nvme-cli`	Drive health (SMART) (via srs-root)
`i2c-tools`	Memory SPD decode (via srs-root), and the opt-in DIMM PMIC voltages of Memory Timings (6.12)

Package	What it adds
`ryzen_smu-dkms-git` (AUR)	AMD Ryzen 7000 / 9000: the live memory timings, memory clocks and SoC voltages of Memory Timings (6.12), and the PM table items
`acpi_call-dkms`	Tools > ACPI method evaluation, for experts (9.22)
`pciutils`, `usbutils`	Full PCI config space, `lsusb` in the full report
`lm_sensors`	Sensor labels and the full report's sensor section
`vulkan-tools`, `clinfo`, `mesa-utils`	Vulkan, OpenCL and OpenGL details
`iw`, `bluez-utils`	Wi-Fi link and Bluetooth details
`wireplumber`, `libpulse`	Audio graph details
`inxi`, `lshw`, `hwinfo`	Extra sections in the full system report
`zlib`, `zstd`, `lz4`, `xz`	The compression rows of Everyday Work (9.15); almost every system has them, a missing one only skips its rows
`curl` (libcurl)	The internet speed test of the Network Benchmark (9.16); the `curl` command (or `wget`) for Update hardware lists (9.21)
`vulkan-icd-loader` and your GPU's Vulkan driver (`vulkan-radeon`, `vulkan-intel`, `nvidia-utils`)	The GPU Benchmarks (9.17); almost every desktop has them
`rocm-hip-runtime` (and `hipblas`), `cuda` (or the NVIDIA driver), `opencl-icd-loader` with an OpenCL driver, `clblast`	The ROCm / HIP, CUDA and OpenCL back ends of GPU Burnin (9.18) and the GPU Compute comparison; only if you want them

3.4 Any other distribution - generic tarball

Each release has `srs-sysinfo-<version>-linux-x86_64.tar.gz`, built on an older distribution (Ubuntu 24.04) so it runs on most current ones (glibc 2.38 or newer, GTK 3).

```
tar xzf srs-sysinfo-2.4.0-linux-x86_64.tar.gz
cd srs-sysinfo-2.4.0-linux-x86_64
./usr/bin/srs-gui                       # try it without installing
sudo cp -r usr/bin usr/share /usr/local/     # install for all users
```

The optional boot-time unit (see 5.3) expects `/usr/bin/srs-root`. `INSTALL.txt` in the tarball shows how to point it at `/usr/local/bin` instead.

Run-time needs: glibc 2.38+, GTK 3, GLib, Pango, Cairo, and **hwdata** (the PCI / USB name lists in `/usr/share/hwdata`). Debian/Ubuntu: `sudo apt install hwdata libgtk-3-0`.

3.5 Any distribution - AppImage

Each release also has `srs-sysinfo-<version>-x86_64.AppImage`: the whole program in one file, with GTK and everything else it needs inside. Nothing is installed; you just run the file. It is built on Ubuntu 22.04, so it runs on distributions from about 2022 on (glibc 2.35 or newer).

```
chmod +x srs-sysinfo-2.4.0-x86_64.AppImage
./srs-sysinfo-2.4.0-x86_64.AppImage                    # the window
./srs-sysinfo-2.4.0-x86_64.AppImage --version          # with arguments it is the
command line
./srs-sysinfo-2.4.0-x86_64.AppImage srs-sysreport      # the full system report
```

- **Command line:** every `sysreport-search` command works with the AppImage, for example `./srs-sysinfo-2.4.0-x86_64.AppImage find smart`. A link named `sysreport-search` that points to the AppImage behaves like the real `sysreport-search`.
- **Updates:** the AppImage knows where its updates are. AppImageUpdate (or an AppImage manager that uses it, such as AppImageLauncher) downloads only the parts that changed in the newest release.
- **Run as root** works (see 5.2). **Refresh root data at boot** (5.3) is greyed out: the boot-time service can only be installed by the package for your distribution.
- **Device names:** SRS uses your system's PCI / USB name lists (`hwdata`) when they are installed, and its own copy when they are not.
- **If it does not start** with a message about FUSE: install the `fuse3` package (or `fuse` on older systems), or run it with `--appimage-extract-and-run`.
- **Window theme:** the AppImage uses GTK's own Adwaita theme, light or dark as your desktop asks.
- **Start with the desktop:** an update gives the AppImage a new file name, so tick **Options -> Start with the desktop** again after updating.
- **Licences:** the libraries inside the AppImage (GTK and the rest) come from Ubuntu 22.04 packages. Their licence files are inside it in `usr/share/doc/<package>/copyright`, and `usr/share/doc/srs-sysinfo/BUNDLED-LIBRARIES.txt` lists which library came from which package (see them with `./srs-sysinfo-2.4.0-x86_64.AppImage --appimage-extract`).

3.6 Build from source

You need `gcc`, `make`, `pkg-config` and the GTK 3 development files (Arch `gtk3`; Debian/Ubuntu `libgtk-3-dev`; Fedora `gtk3-devel`).

```
git clone https://github.com/eugenesflanagin-spec/srs-sysinfo.git
cd srs-sysinfo
make native                                        # dist/native/srs-gui and
sysreport-search
sudo make install-system DESTDIR= SYSPREFIX=/usr   # what the Arch package does
```

Other targets:

Target	What it does
`make native`	Build for this machine, with your `CFLAGS` / `LDFLAGS` (the packaging path).
`make install-system DESTDIR=... SYSPREFIX=/usr`	Install programs, menu entries, icon, docs, systemd unit.
`make install`	Per-user install under `~/.local` (also builds the cross targets below, so it needs the cross compilers).

Target	What it does
`make all`	Also i686 and aarch64 command-line builds and the Windows images `SRS64X.exe` / `SRS32X.exe` (needs mingw-w64 and cross gcc).
`make clean`	Delete `dist/`.

Note: `make` / `make all` build into `dist/linux-x86_64/`; only `make native` builds `dist/native/`. If you test from `dist/native/`, rebuild with `make native`.

The optional Windows images (`SRS64X.exe`, `SRS32X.exe`) contain only the device-ID lookup and report search features, not the full system viewer.

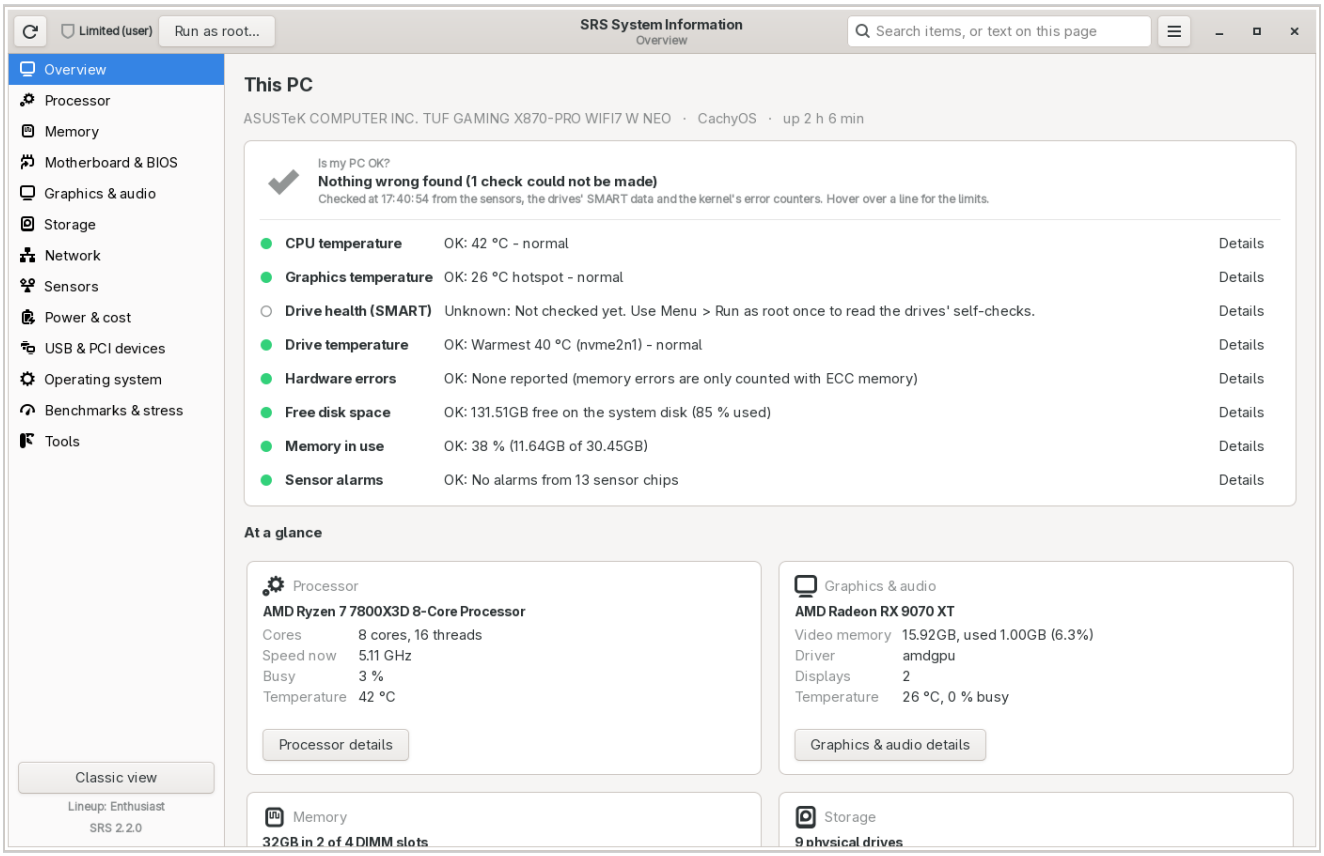
4. First launch

Open your application menu. SRS installs four entries (category System / Monitor):

Menu entry	Opens
SRS System Information	The main window. Right-click it in the menu for shortcuts: Sensors, Processors, Stress & Burnin, Benchmark, Cache & Memory, Storage, OS latency, CPU compute, Everyday work, Network, GPU Benchmark, GPU compute comparison, SRS Pi, Device ID Lookup, Find in System Report, Classic View (16 in all).
SRS Stress & Burnin	The Stress & Burnin tab (CPU and GPU load; nothing starts until you press Start).
SRS Benchmark	The Benchmark, IPI Delay and Latency windows (nothing runs until you press Run).
SRS Full System Report	A terminal that writes a full text report folder (see 13.3).

Or type `srs-gui` in a terminal.

What you see first: the main window. The first time it opens, SRS asks which tabs you want on its pages - **Welcome to SRS** with Basic, Enthusiast or Expert (the default); if you used SRS 2.0, the question is **SRS 2.2: your tabs** with Enthusiast, the 2.0 tabs, already picked. Click OK (or close it: the preselected one is used); you can change it at any time (6.7). On the left is a list of pages; the first one, **Overview**, tells you whether the PC looks healthy and shows its key facts. (The SIV-style window of SRS 1.1 is still there: see Classic view.)



The main window with the Overview page, on the author's PC (light theme).

First things to try:

1. Read **Is my PC OK?** on the Overview. Every line has a green, yellow or red light and says what it means; press **Details** to see the readings behind it.
2. Click **Sensors** on the left for temperatures, fans and voltages.
3. Click **Processor** for the CPU, its cores and caches, and **Memory** for the memory sticks.
4. Type a word such as `temperature` or `BIOS` into the search box at the top and pick an item.
5. Run **Menu -> Run as root (srs-root)...** once, so the items that need root data fill in (next section). Until then the drive health line says "Not checked yet".

5. Root-only data (srs-root)

5.1 In plain words

A few things on Linux can only be read by the administrator (root): memory stick details (SPD), SMBIOS serial numbers and UUID, drive health (SMART), ACPI tables, CPU model-specific registers (MSRs), the full PCI configuration space, and (2.4) the memory controller's live timings, the graphics card's VBIOS image, the GPIO pin states and a few firmware tables.

SRS itself **always runs as your normal user**. Instead, a small helper called `srs-root` reads those things once, as root, and saves them as text files in your own folder `~/.cache/srs/root/`. From then on every panel shows them, without root.

`srs-root` is read-only. It runs: `dmidecode`, `decode-dimms`, `smartctl -x` and `nvme smart-log` for each disk, copies of `/sys/firmware/acpi/tables`, `lspci -vvvnnxxx`, copies of `/proc/ioports` and `/proc/iomem`, `dmesg`, `smbstatus` (if Samba is installed) and reads of a fixed list of AMD and architectural MSRs through `/dev/cpu/N/msr` (it loads the `msr` kernel module for that). Since 2.4 it also reads the items in 5.4. It then gives the files to you (`chown`).

5.2 Three ways to run it

1. **From the window: Menu -> Run as root (srs-root)...** -> OK -> enter your password once. When it finishes, a dialog says "Root data collected into ~/.cache/srs/root" and every open panel refreshes.
2. **From a terminal:** `pkexec srs-root` (or `sudo srs-root`).
3. **At every boot, automatically:** see 5.3.

From the AppImage: way 1 works the same. The administrator account cannot look inside a running AppImage (only you can), so SRS first copies the `srs-root` script into a new folder that only you can open (in `/run/user/<your id>`, or `~/.cache/srs` if that is not possible) and asks for the password to run that copy. The copy is the same read-only script and is deleted as soon as it has finished. Nothing is installed and no system settings are changed.

Run it again after you change hardware (new RAM, new drive) or update the BIOS, so the cache is current. `~/.cache/srs/root/collected.txt` holds the date of the last collection.

USB drives: SMART is skipped for USB-attached disks by default, because SMART pass-through is meaningless on USB sticks and can upset flaky USB enclosures. To include them anyway: `pkexec env SRS_SMART_USB=1 srs-root`.

Running the command line as root: `sudo sysreport-search -save` uses the cache of the user who ran sudo (or collects one first if there is none). You do not need to do this normally.

5.3 Refresh root data at boot

Menu -> Options -> Refresh root data at boot keeps the cache fresh with no prompts.

- Ticking it asks for your password **once** and enables the systemd unit `srs-root@<your user>.service`. It also collects straight away.
- At every boot the unit runs `srs-root --user <you>` as root, at the lowest CPU priority (Nice 19) and idle disk priority, so boot is not slowed. The files are owned by you.
- Unticking it disables the unit (one password prompt). If you cancel the prompt, the tick stays as it was.
- By hand: `sudo systemctl enable --now srs-root@$USER.service` (or `disable`).
- The option is greyed out if the unit file is not installed (`/usr/lib/systemd/system/srs-root@.service` or `/etc/systemd/system/srs-root@.service`). That is always the case with the AppImage: use the package for your distribution if you want this.

5.4 What srs-root reads since 2.4, and the opt-ins

In plain words: 2.4 turns most of the items that used to say "not read" into real data. Some of it needs no root at all; the rest comes from the same one **Run as root**. Two extras are off until you turn them on, because on some machines reading them is not entirely harmless.

No root needed (any time):

- **System CMOS** - the CMOS RAM next to the clock, from the kernel's own read-only copy, with the standard bytes decoded.
- **AMD Fuses** - each core's factory ranking and highest clock (the fuse registers themselves stay unread).
- **Memory Timings**: the memory and fabric clocks, the SoC voltages (with the `ryzen_smu` driver) and every DDR5 module's profiles and temperature (6.12).

With **Run as root** (read-only, files in `~/.cache/srs/root/`):

What	File	Shown on
AMD memory controller registers (Ryzen 7000 / 9000 desktop, through the `ryzen_smu` driver)	`umc.txt`	Memory Timings
Intel memory controller registers (Core 6th gen to Core Ultra, experimental)	`intel-imc.txt`	Memory Timings
The BIOS's AMD AOD table (memory settings), only when the kernel allows it (below)	`aod.bin`, `aod.txt`	Memory Timings
The EBDA pointer and, if there is one, its contents (memory below 1 MB)	`ebda.txt`, `ebda.bin`	System EBDA
The graphics driver's own copy of each AMD VBIOS (the GPU's ROM is not touched)	`vbios-<pci address>.bin`	GPU VBIOS status
The kernel's GPIO table	`gpio.txt`	I/O GPIO
The RAPL energy counters, read twice one second apart (AMD)	`rapl.txt`	MSR Setup (package and core watts)
The UEFI System Table pointers and the firmware-update table (ESRT) details	`efi-systab.txt`, `esrt.txt`	Firmware

How the AMD memory controller is read: the `ryzen_smu` driver has a file (`smn`) where a program writes an address and then reads the value there. Writing the 4-byte address only selects what the driver reads; srs-root never uses the 8-byte form, which would write a register. Each value is read twice, and a value that changed in between is left out.

The AOD table and `iomem=relaxed`. The BIOS's memory settings (ProcODT, drive strengths, the DIMM voltages it set) sit in a part of memory that most Linux kernels keep from every program, root included (kernels built with IO_STRICT_DEVMEM). srs-root reads the table only when the kernel allows it - that is, when the PC was booted with the kernel option `iomem=relaxed`, which weakens that protection. **SRS never asks you to change this.** Without it, Memory Timings says why that part is missing, and everything else on the page works.

The two opt-ins (Menu > Options, both off by default, both ask first; they apply from the next Run as root):

- **Read DIMM voltages from their PMIC chips (SMBus; from the next Run as root)** - each DDR5 module has a small power chip (PMIC). srs-root reads its registers over the memory's SMBus with `i2cdump` (byte reads, nothing written) and Memory Timings shows the voltages it is set to. Why it is off: on some boards the BIOS or the board's controller uses the same bus at the same time without telling Linux, and a collision there can, rarely, upset a sensor or the board's lighting until the next boot. File: `pmic-*.txt`.
- **Read the embedded controller (expert, read-only; from the next Run as root)** - the EC is the board's small helper chip for fans, power buttons, the battery and some sensors. srs-root loads the kernel's `ec_sys` module read-only, copies its 256 registers once and unloads the module again. Why it is off: on a few machines (mostly laptops) reading certain EC registers clears a pending event, such as a battery or key notification, which may then be missed once. File: `ec.bin`; shown on ACPI Embedded Controller.

Nothing above is written to a device. The one 2.4 feature that can change hardware state is ACPI method evaluation (9.22), which is separate and asks first.

6. A tour of the main window

The main window is one window with everything in it. In plain words: **the left side is a list of topics, the right side shows the topic you picked.** Nothing that SRS showed before is gone: every detail is still on the pages, and the SIV-style windows are one click away (6.6).

From top to bottom:

Part	What it is
Title bar	**Refresh** (left; also **F5**) reads the page again. Next to it the **badge** says whether SRS can read everything: **Full access** or **Limited (user)** with a **Run as root...** button (6.10). The **search box** finds any item or text (6.4). The **menu button** (three lines; also **Ctrl+M**) has Save Report, Lineup, Options, Run as root and more (6.5).
Sidebar (left)	The pages: Overview, Processor, Memory, Motherboard & BIOS, Graphics & Audio, Storage, Network, Sensors, Power & Cost, USB & PCI devices, Operating system, Benchmarks & Stress, Tools. **Alt+1** ... **Alt+9**, **Alt+0**, **Alt+-**, **Alt+=** and **Alt+Backspace** jump straight to them, in that order (12). Below the list: **Classic view**, the tab **lineup** in use (6.7) and the SRS version.
Page (right)	What you picked. The title bar's second line names it.

The window follows your desktop's light or dark theme (see **Options -> Theme** in 10).

6.1 The pages and the items on them

Every page groups the items of one topic. "Item" is SRS's word for one table of information, such as **Sensor status** or **SMART drives**; there are 558 of them.

Page	What it covers	Tabs
Overview	Is the PC healthy, and its key facts	-
Processor	The CPU: model, cores and threads, clocks, features, caches	CPU Summary, Processors, CPU Clocks, CPUID Features, Cache Levels, CPUID, Microcode, Speculation Control
Memory	RAM: installed, in use, the sticks and their timings	Memory, DIMMs, SPD (DIMMs), Memory timings, DIMM status/temperatures, Physical memory map, NUMA
Motherboard & BIOS	The board and its firmware	Machine, Hardware summary, Firmware, BIOS / SMBIOS tables, UEFI information, ACPI tables, TPM device
Graphics & Audio	Graphics cards, monitors, sound	Graphics, GPU information, GPU clock speeds, GPU temperatures, Monitors, OpenCL, Audio devices
Storage	Drives, file systems, drive health	Disk drives, Volumes, SMART drives, Partitions, Disk performance, USB disk drives
Network	Adapters, addresses, Wi-Fi, Bluetooth, connections	Network adapters, IP address, Wi-Fi SSIDs, Bluetooth radios, Routing table, TCP V4 ports, TCP/IP counters
Sensors	Temperatures, fans, voltages, power	Sensor status, Hardware status, Cooling status (fans), Power status
Power & Cost	How much power the PC uses, what it costs, history (estimates; see 6.11)	Now, History, Sessions, Price, Setup, Alarms, Sources
USB & PCI devices	Every device in or plugged into the PC	ALL Devices, PCI bus, USB bus, USB tree, HID bus, Thunderbolt
Operating system	Linux itself	General Info (SIV's "Windows" panel; the classic **Linux** button), Drivers, Processes, Startup programs, Installed software, Environment, Users
Benchmarks & Stress	Measure or load the CPU, memory, drives, network and GPU	CPU Benchmarks, Cache & Memory, Storage, OS Latency, CPU Compute, Everyday Work, Network, GPU Benchmark, GPU Burnin, GPU Compute comparison, SRS Pi, Crypto, Prime Sieve, CPU IPI Delay, Memory Latency, Benchmark History (the last results)
Tools	Look-ups, reports and SRS's own items	About SRS, Help

The tabs above are the standard ones (the **Enthusiast** lineup; SRS's default is **Expert**). You can change them on any page at any time (6.7).

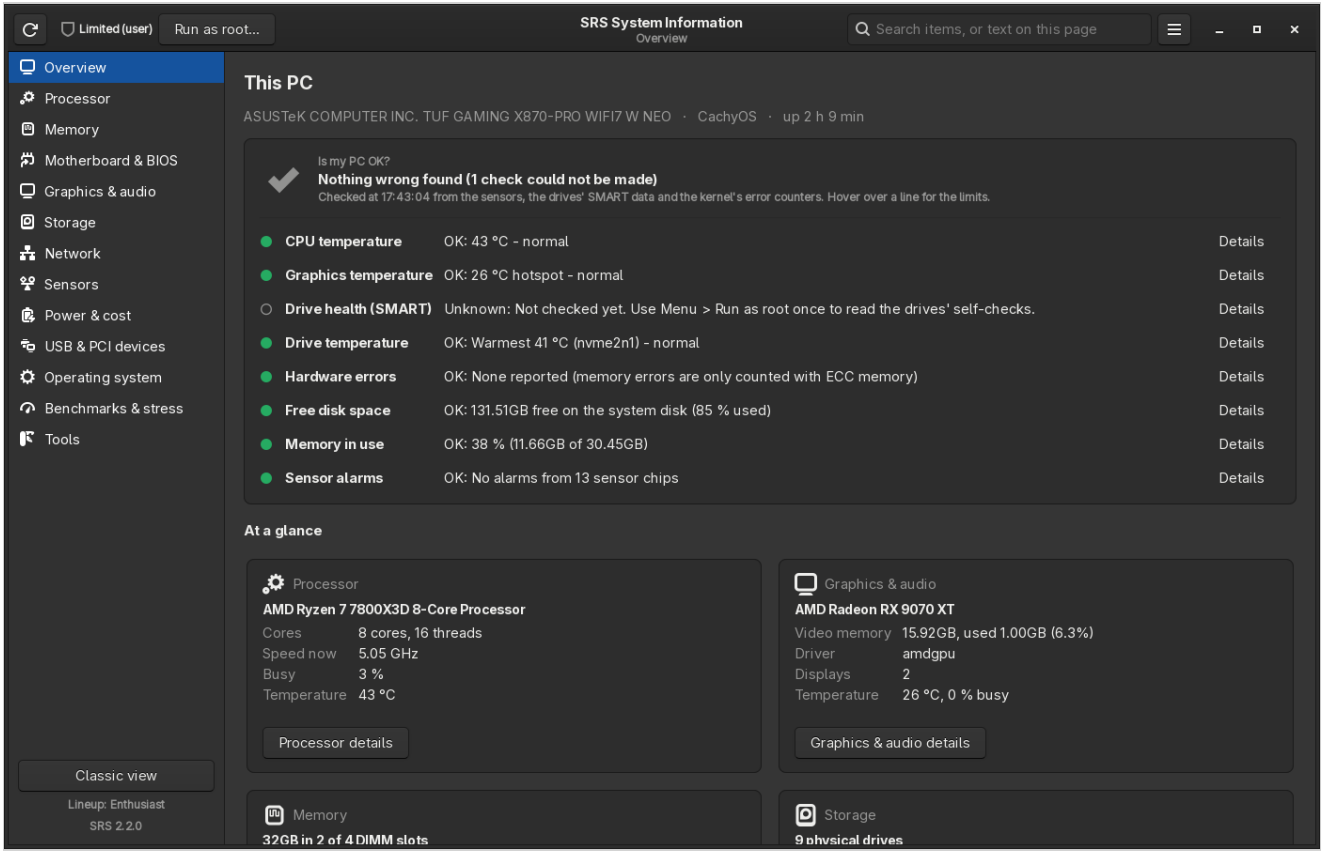
All N items at the right end of the tab row lists **every** item of the page, the expert ones included, in alphabetical order. It is a jump list: type a few letters to filter it (every word must match), then pick an item or press **Enter** for the first one. If the item already has a tab, SRS goes to it ("open" at the right of the row); otherwise it opens as a **temporary tab** (its name in *italics*), which is gone the next time SRS starts unless you keep it (right-click it > **Keep this tab**). On the Processor page the list ends with **Processor CPU-n** and **Cache levels CPU-n** for each CPU (SIV's CPU-n and Cache-n buttons).

Items that cannot show anything on this PC are left out of the list and the tabs: SIV's Windows-only items, hardware SIV checks for that this PC does not have (for example RAID controllers), and - only after

you ask SRS to check (**Options -> Check now**, or **Check daily**, which is off unless you turn it on; 10) - items that came out empty here. The list says how many are hidden; **Options -> Show hidden items** lists them greyed, with the reason ("Windows-only", "not present", "nothing to show on this PC"), and you can still open them.

6.2 Overview: "Is my PC OK?"

The Overview starts with a line about the PC (board, Linux distribution, time since start-up), then **Is my PC OK?**: one line per check, each with a light and a plain-words verdict.



The Overview in the dark theme: every check is green, and the cards under "At a glance" start below.

Light	Means
Green dot, "OK:"	Normal.
Yellow dot, "Check:"	Worth a look, not urgent.
Red dot, "Problem:"	Act on it (the text says what to do).
Hollow grey circle, "Unknown:"	SRS could not check it (for example no sensor, or no root data yet).

Check	Based on	OK / check / problem
CPU temperature	The hottest CPU sensor (k10temp, coretemp, zenpower), against the CPU's own maximum temperature (*limit*, see below)	below limit − 10 °C / up to limit − 3 °C / limit − 2 °C and up
Graphics temperature	The GPU's hotspot (junction) sensor, which runs hotter than the edge, against the card's own critical temperature (*crit*)	below crit − 15 °C / up to crit − 6 °C / crit − 5 °C and up

Check	Based on	OK / check / problem
Drive health (SMART)	Each drive's own self-check, from srs-root's cache	all PASSED / - / any FAILED. "Not checked yet" until srs-root has run once. USB adapters usually cannot pass SMART on.
Drive temperature	Every drive, against its own warning temperature (*warn*)	below warn − 10 °C / up to warn − 1 °C / warn and up
Hardware errors	Kernel taint flag M (the CPU reported a machine check), EDAC counters (ECC memory), HardwareCorrupted in /proc/meminfo	none / corrected ECC errors / a machine check, uncorrected errors or faulty pages taken out of use
Free disk space	The file system mounted at /	below 90 % used / 90-96 % / 97 % and up
Memory in use	Memory used by programs (cache does not count)	below 90 % / 90-94 % / 95 % and up
Sensor alarms	Alarm flags of the hardware monitor chips	none / any flag set / -
CPU throttling	Thermal throttling counters (shown only on CPUs that count them, such as Intel)	never / slowed down at least once / -

Where the limits come from. Each sensor's own limit wherever it publishes one:

- **CPU:** Intel processors report their maximum (coretemp). AMD's driver does not, so for AMD desktop Ryzen SRS uses AMD's specified maximum for the model, for example 89 °C for a Ryzen 7 7800X3D and 95 °C for a Ryzen 9000.
- **Graphics:** the card's own critical temperature for the hotspot sensor, for example 110 °C on a Radeon RX 9070 XT.
- **Drives:** each NVMe or SATA drive's own warning temperature, for example 81.85 °C on a Samsung 980 PRO. Drives are compared one by one, so a cooler drive with a low limit is not hidden behind a warmer one. A drive that only publishes a critical temperature is judged against 5 °C below it.

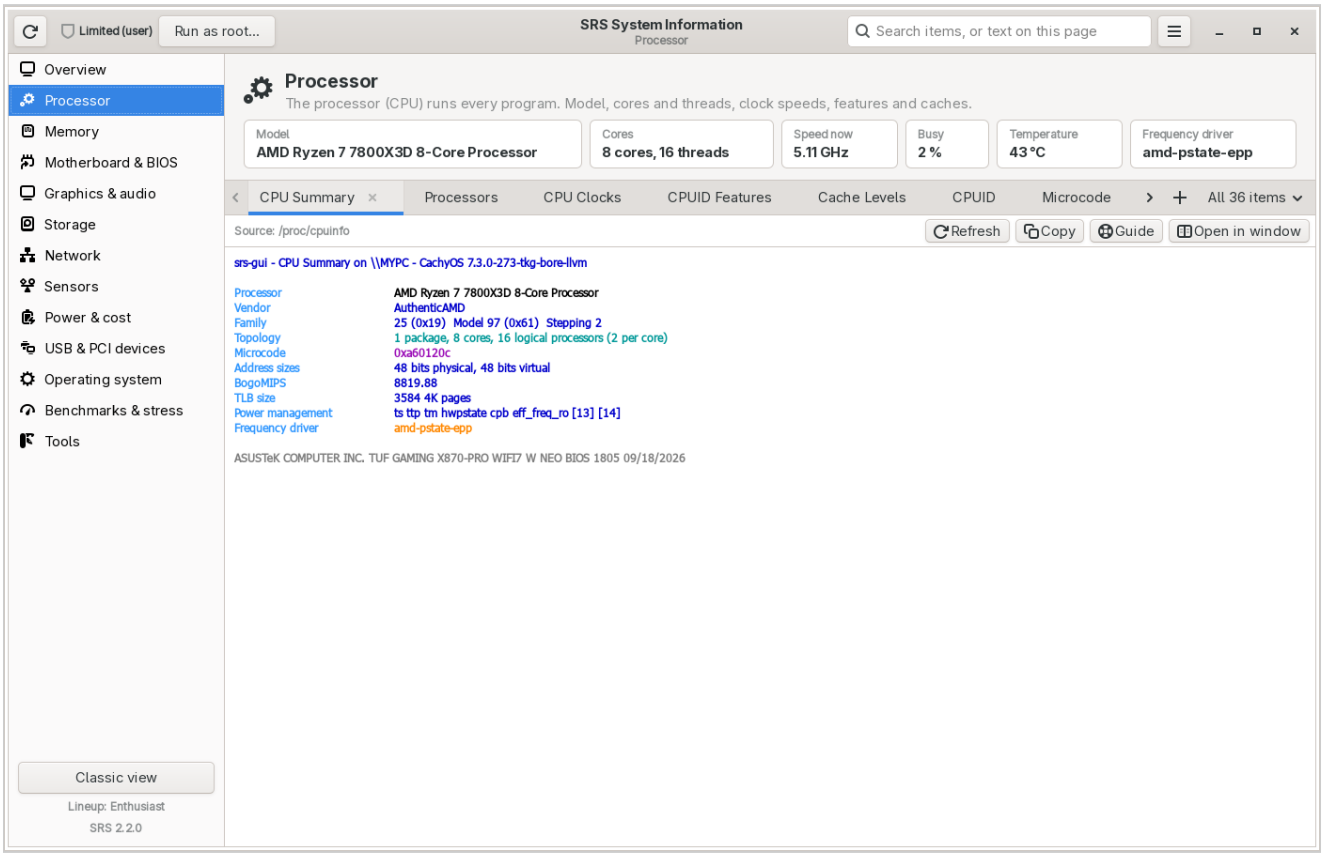
Where no limit is known, a general one is used: CPU 85 / 95 °C, graphics 95 / 105 °C, drives 60 / 70 °C. The tooltip says which one applies.

Hover over a line for its limits; **Details** opens the page and tab with the full readings. The checks repeat every 3 seconds (drive data every 30 seconds); **F5** reads everything again now. They only read what SRS already shows, and change nothing.

Below the checks, **At a glance** has a card for the processor, graphics, memory, storage, operating system and motherboard, with the numbers people look for (model, cores, speed now, memory in use, free space, kernel, BIOS version ...). The button on each card opens its page.

6.3 A page

Each page has three parts:

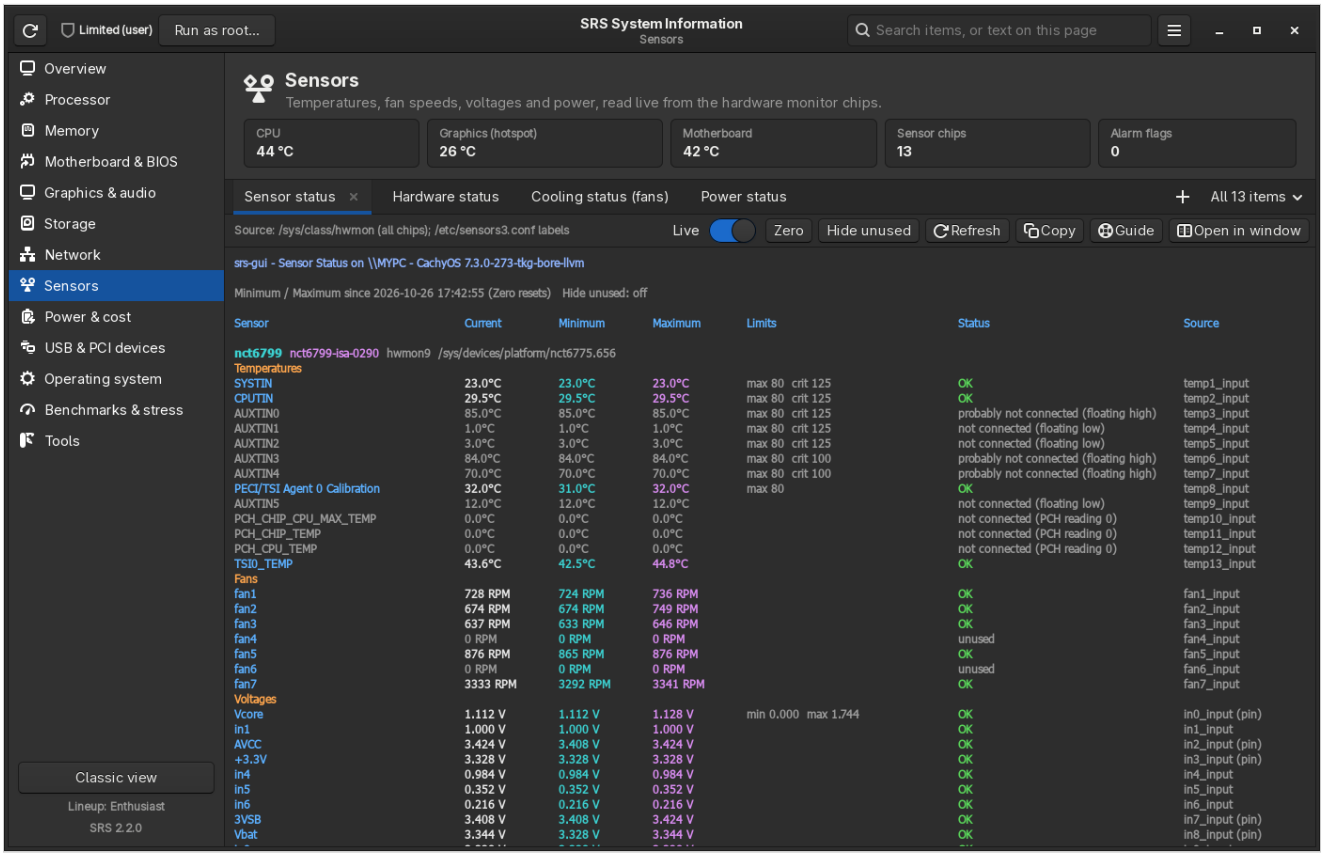


*The Processor page: the heading, the key facts as tiles, then the tabs. **+** adds a tab, and **All 36 items** at the right end of the tab row lists every processor item that can show something on this PC.*

1. **The heading:** the topic and one sentence about what it is.
2. **Key facts** as tiles (for example Model, Cores, Speed now, Busy, Temperature on the Processor page). They update while the page is open; hover over a tile for an explanation and the full value.
3. **The tabs** with the full items - the same text and colours as the classic panels, nothing left out. Above the text:

Control	What it does
Source:	Where the data comes from (sysfs file, tool, srs-root cache). Hover for the item's id.
Needs root data...	Shown when part of the item can only be read as root and srs-root has not run yet.
Live switch	Live items (sensors, clocks, counters ...) refresh by themselves while shown. Turn it off to freeze this tab. **Options -> Update interval** sets the rate for all.
Zero, **Hide unused**	Sensor status only: start minimum / maximum again; hide sensors that read nothing.
Refresh	Read the item again (**F5**).
Reset view	Only on tabs with tables: puts the tables back in the item's own order (6.8). Greyed until you sort.
Copy	Copy all the text, in the item's own order even when a table is sorted. With **Options -> Redact** on, serials, UUIDs, MACs and the computer name are masked in the copy (17).
Guide	This guide at the item's section (**F1**).

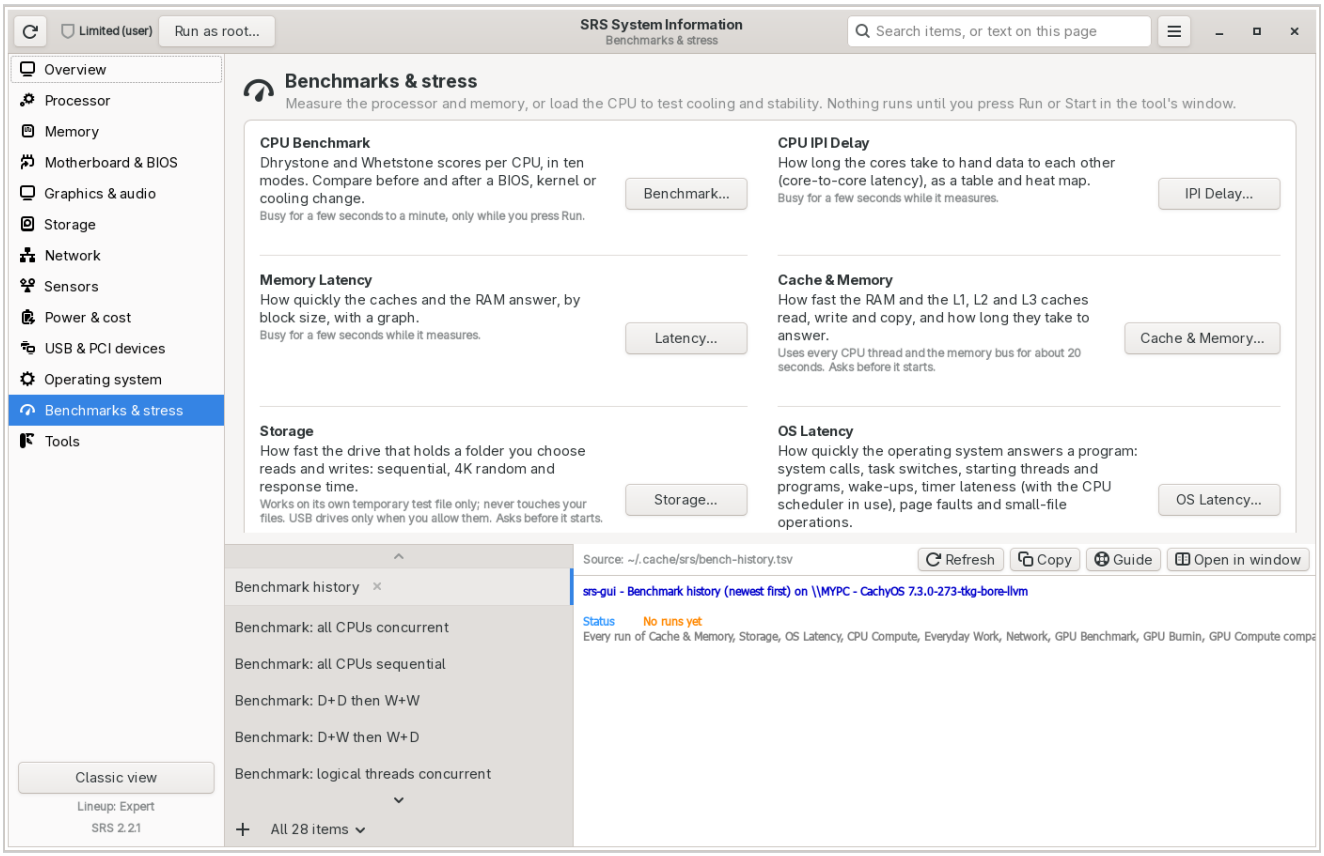
Control	What it does
Open in window	The item as a classic panel with its SIV-style button row.



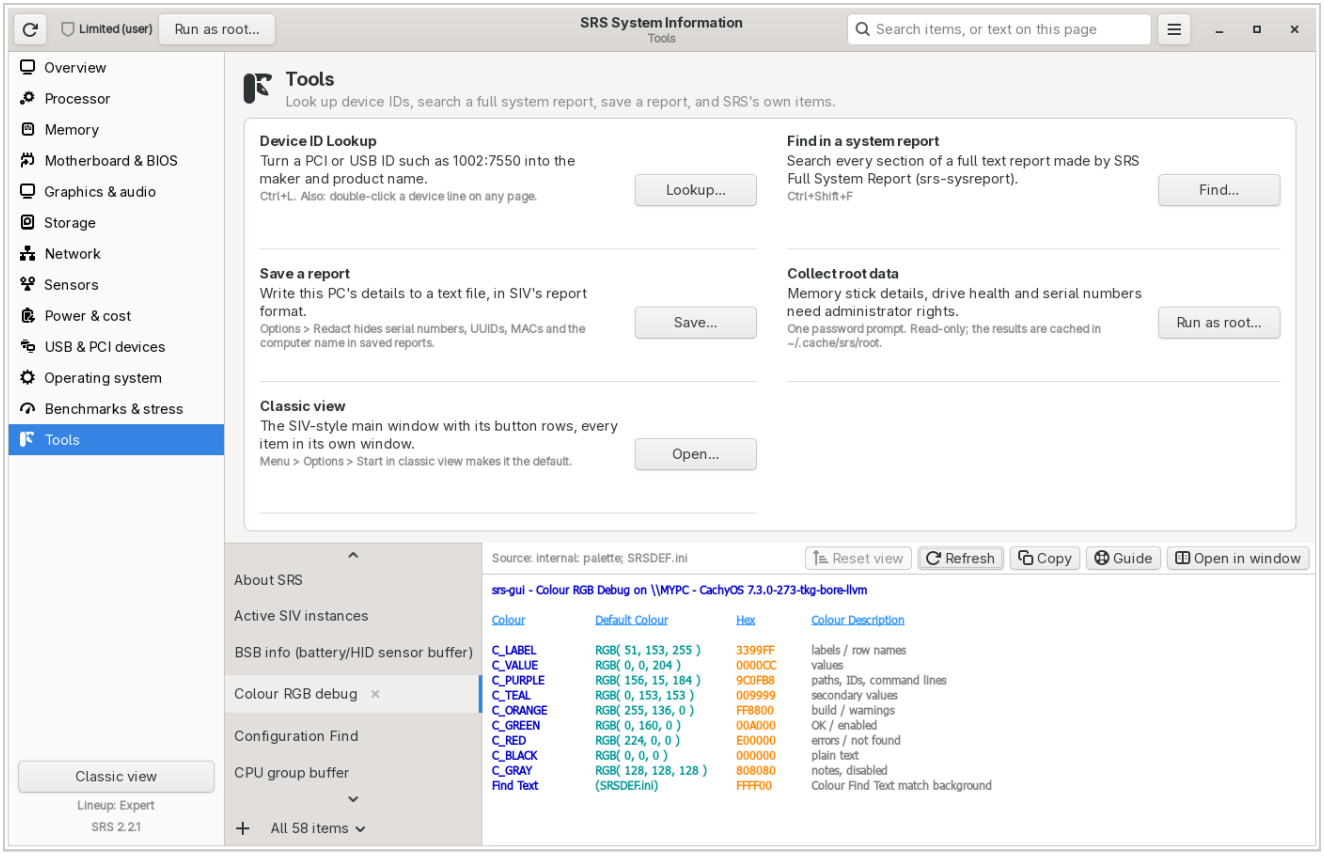
The Sensors page in the dark theme, with the Sensor status tab: the same text and colours as the classic Sensors panel, plus the Live switch and the Zero and Hide unused buttons.

Double-click a line with a PCI or USB ID to look it up (it opens as a tab on the Tools page). Right-click the text for Copy, Select All, the guide link and the Classic Menu. Click a column heading in a table to sort it (6.8); right-click a tab for Keep, Replace with..., Move and Close (6.7).

The **Benchmarks & Stress** page has one tab per tool (2.3.1): Stress & Burnin (CPU and GPU), CPU Benchmarks, Cache & Memory, Storage, OS latency, CPU compute, Everyday work, Network, GPU Benchmark, GPU Compare, SRS Pi, Crypto, Prime Sieve, IPI delay and Memory latency, each with its controls and live results, and compact sections at the top that jump to them (9); nothing runs until you press **Run** or **Start** there. The **Tools** page opens Device ID Lookup, Find in a system report, Save Report, Run as root, the Classic view and (2.4) Update hardware lists (9.21) and ACPI method evaluation (9.22).



The Benchmarks & Stress page (light theme): a button for each tool, with what it measures and how long it keeps the PC busy. Only the first launchers show here; scroll for the rest. The tabs below show the last results.

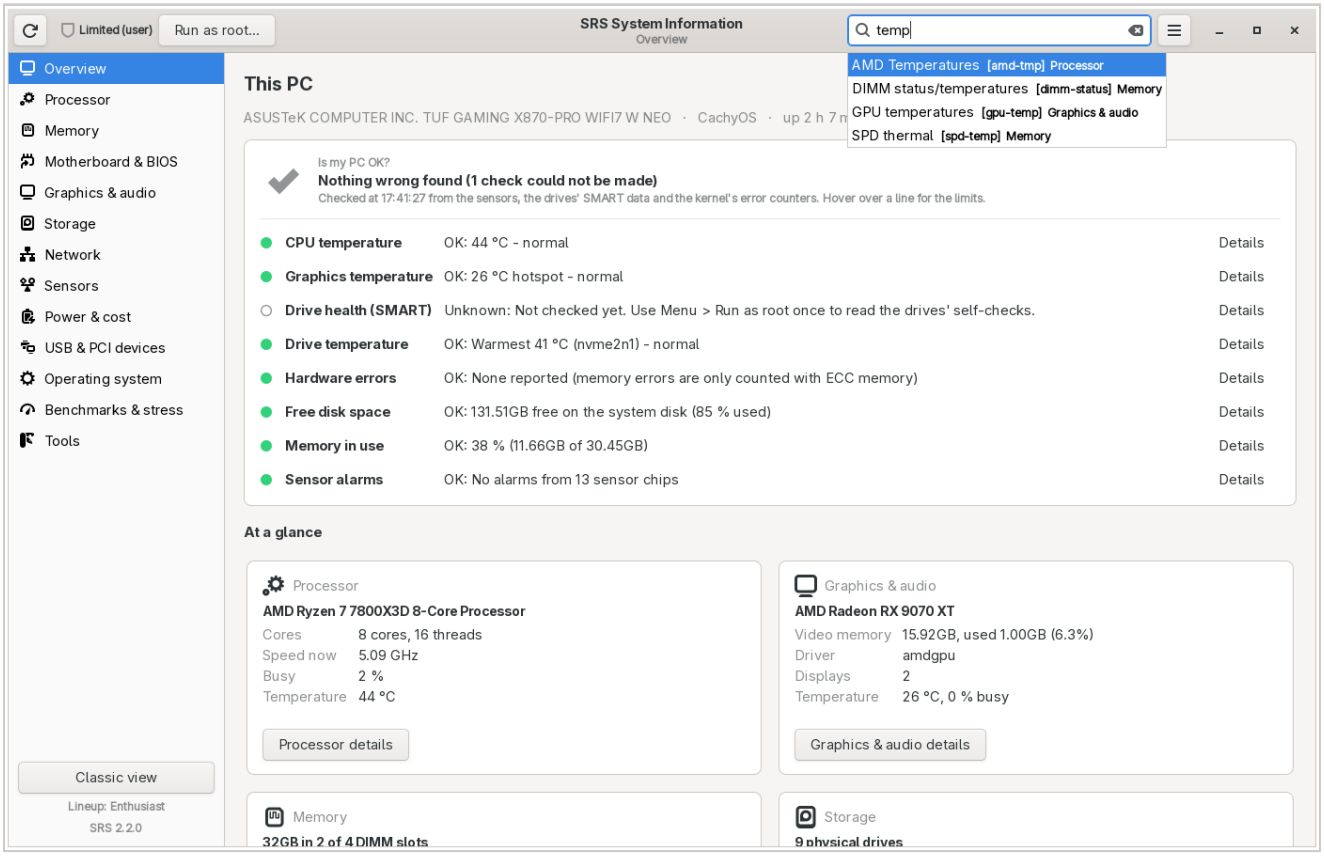


The Tools page, here with the Colour RGB debug tab open below the launchers.

6.4 Search

The search box in the title bar (**Ctrl+F**):

- **Find an item:** type part of a name or id, for example `temperature`, `smart`, `bios` or `usb`, and pick from the list (arrow keys + Enter, or click). The item opens on its page. Page names work too. Every word you type must match, so `cpu clock` finds CPU Clocks.
- **Find text on the current tab:** type the text and press **Enter**. All matches are highlighted and the first one is selected; **F3** / **Shift+F3** go to the next / previous one. The title bar shows how many there are.
- **Item ids with an argument:** type the id and press Enter, for example `processor:3` (CPU-3), `cache:0`, `volumes:/`.
- **Esc** clears the box and the highlights.



Typing `temp` in the search box: each suggestion shows the item's name, its id in brackets and the page it is on.

To search a full text report on disk, use **Find in System Report** (**Ctrl+Shift+F**, see 9.6).

6.5 The menu

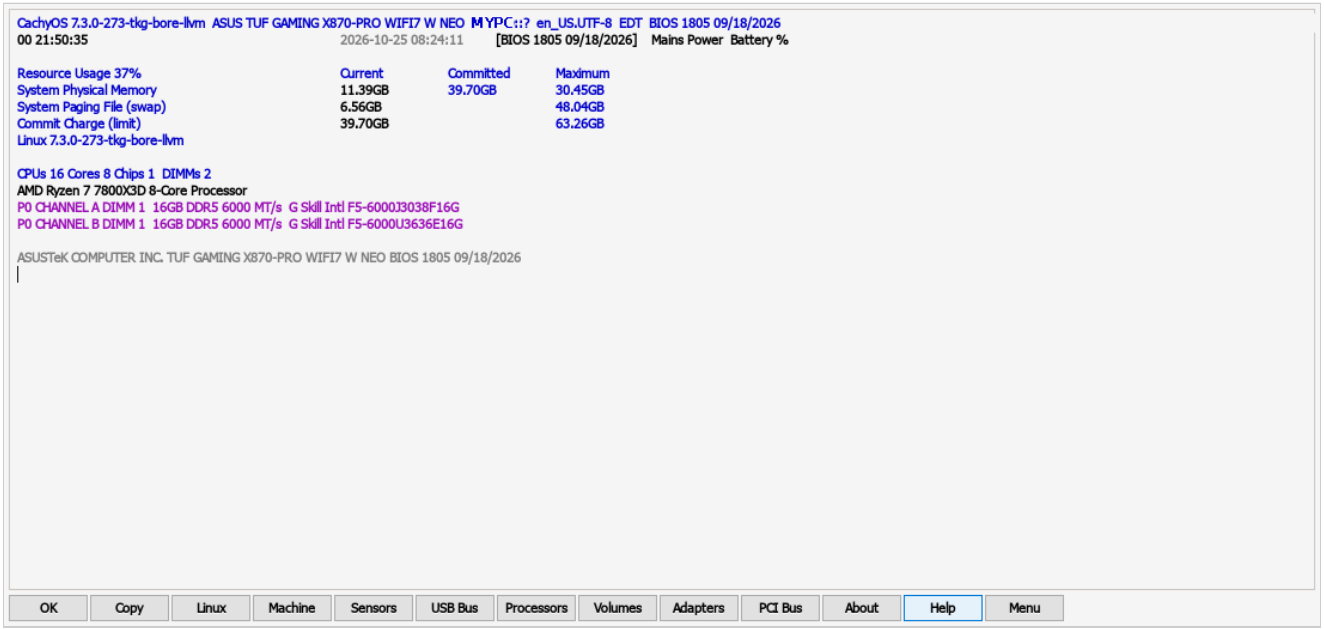
Entry	What it does
Save Report... (Ctrl+S) / **Save All Items...**	Save a text report (13.1).
Copy This Tab	Same as the tab's Copy button.
Lineup	Switch between tab lineups, save, export or import one (6.7).
Options	See 10.

Entry	What it does
Run as root (srs-root)...	Collect the root-only data (5).
Find in System Report... (Ctrl+Shift+F) / **Device ID Lookup...** (Ctrl+L)	The tools of 9.6 and 9.5.
Classic View	Opens the classic main window (6.6).
Classic Menu	The full menu of the classic view: every item by group and every tool (8). Items open as classic panels.
User Guide (F1) / **About SRS**	This guide; version, build and data files.
Quit (Ctrl+Q)	Exit SRS.

6.6 Classic view (the SIV-style windows)

The classic view is SRS's SIV-style interface, exactly as in SRS 1.1: a main window with SIV's button row, and every item in its own panel window with its own buttons (7).

- **Open it** from **Menu -> Classic View**, the **Classic view** button under the sidebar, or the Tools page. Closing it does not close SRS.
- **Start SRS in it:** `srs-gui --gui-classic`, the **Classic View** action of the SRS menu entry (right-click it in your application menu), or tick **Options -> Start in classic view** to make it the default. From the classic Menu, **Open the new main window** brings the sidebar window back; `srs-gui --gui-new` starts with it whatever the option says.



The classic main window on the author's PC (Ryzen 7 7800X3D, ASUS TUF GAMING X870-PRO WIFI7, 2 x 16 GB DDR5-6000).

The classic main window is SRS's version of SIV's main panel: **one screen that tells you what this PC is and how busy it is**. From top to bottom:

Line	Meaning
First line	Operating system and kernel (`CachyOS 7.3.0-...`), motherboard, `host::user`, locale, time zone, BIOS version and date.
Second line	Up-time (days, hours:minutes:seconds), current date and time, BIOS, power source (`Mains Power`) and battery charge (empty on a desktop).
Resource Usage	How much of the memory budget is in use, in percent.
System Physical Memory	RAM in use now, the total "committed" (promised to programs), and the RAM size.
System Paging File (swap)	Swap in use and swap size.
Commit Charge (limit)	Memory promised to programs vs. the kernel's limit (RAM + swap, depending on settings).
Kernel line	The running Linux kernel.
CPUs / Cores / Chips / DIMMs	Logical CPUs (threads), physical cores, CPU packages (sockets) and memory sticks.
CPU name	The processor model, as the CPU reports it.
DIMM lines	One per memory stick: slot, size, type, speed, maker and part number (needs srs-root data).
Last line (grey)	Board maker, model and BIOS.

The classic main window refreshes every 2 seconds (see Options -> Update interval).

The full meaning of every value is in the Section Reference: srs-items.md and machine.md.

The button row

Button	Opens
OK	Closes the window. On the main window this exits SRS. (The window's own close button [X] hides SRS to the tray instead when "Close button hides to tray" is on, see 11.)
Copy	Copies the window's whole text to the clipboard.
Linux	The operating-system panel (SIV's "Windows" button).
Machine	Board, BIOS, SMBIOS and memory sticks.
Sensors	Temperatures, fans, voltages, power.
USB Bus	The USB tree.
Processors	CPUs, cores, threads, caches, clocks.
Volumes	Mounted file systems and disks.
Adapters	Network adapters.
PCI Bus	PCI / PCI Express devices.
About	Version, build, data files, credit and licence.
Help	Help text and command-line qualifiers.
Menu	The full menu of every item and tool (section 8).

Every button label has an underlined letter: press **Alt + that letter** to click it.

6.7 Arranging the tabs and lineups

Each page's tabs are yours to arrange. Nothing you do here changes what SRS reads or shows inside a tab; it only decides which items have a tab and in what order.

On the fly:

To	Do
Add a tab that stays	**+** at the right end of the tab row (or **Ctrl+T**), then pick the item
Look at an item once	**All N items**: it opens as a temporary tab (*italic*), not kept
Keep a temporary tab	Right-click it > **Keep this tab**
Swap a tab for another item	Right-click it > **Replace with...** (same place in the row)
Move a tab	Drag it, or right-click > **Move left** / **Move right**
Close a tab	Its **x** (shown on the tab in view and on temporary tabs), **Ctrl+W**, a middle-click, or right-click > **Close tab**. The item stays in All N items.
Go back to the lineup's tabs for this page	Right-click a tab > **Reset this page's tabs to the lineup**

SRS remembers each page's tabs and their order when it closes. Temporary tabs are not remembered.

Navigating long tab lists (2.3.1). With the tabs in a side list, the mouse wheel over the list goes to the tab above or below (one notch per tab, no wrap-around), so you get from the top tab to the bottom one quickly; the arrow keys, Home and End keep working as before (12). The Benchmarks & Stress page always starts with its tools as tabs (9.0).

Lined-up columns (2.3.1). On every page with columns, each column is as wide as its longest entry, so the columns line up on every row; an entry too long for a sensible width ends in "..." - hover over it for the full text, and Copy, a copied selection and Save Report always give the full text.

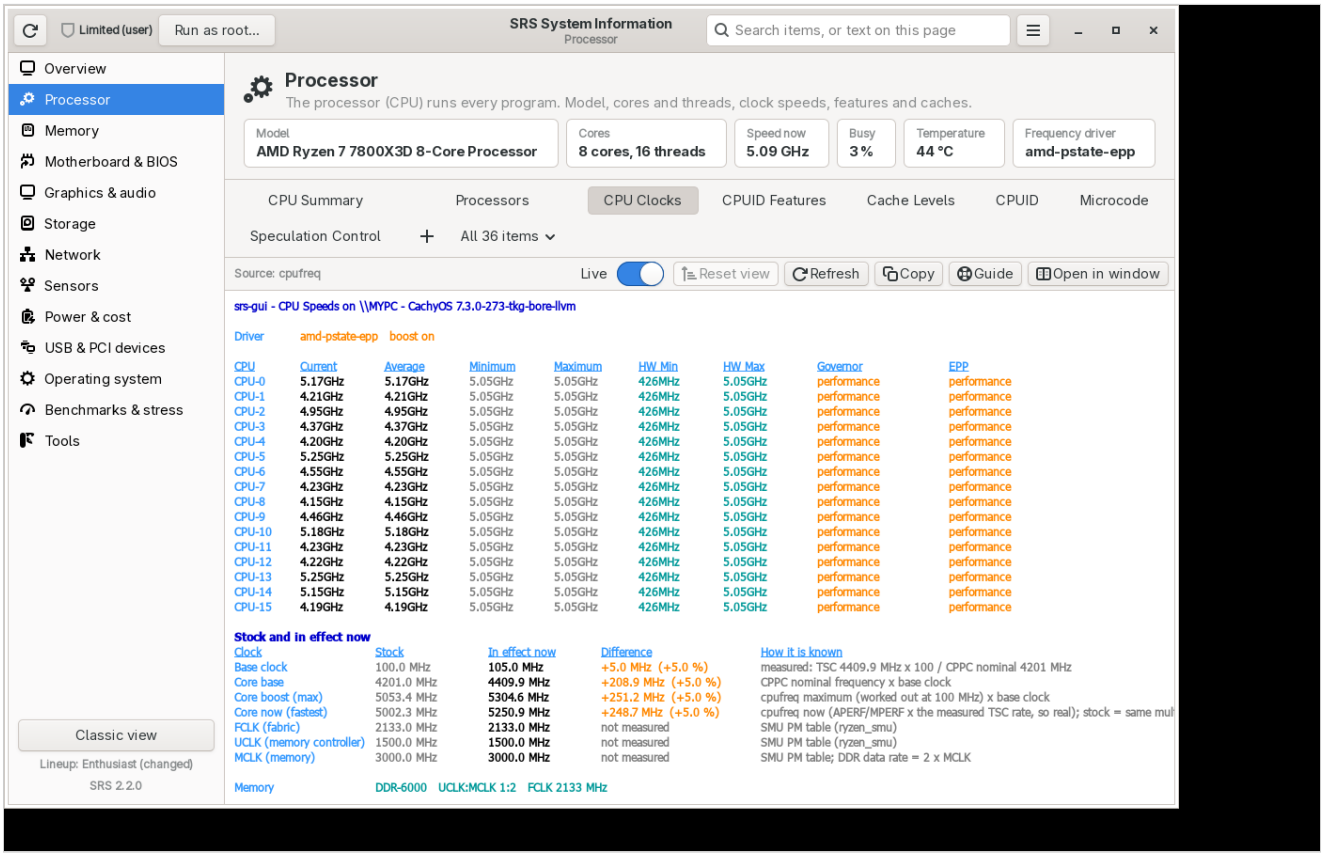
Which tabs a page opens with: Options -> Tabs on each page... (also right-click a tab > **Tabs on this page...**). Pick the page, then one of:

- **The standard tabs only** - the tabs listed in 6.1;
- **The tabs ticked below** - tick any items of the page (the filter box finds them); new ones go at the end;
- **Every item of the page** - one tab per item that can show something here.

Options -> Show every item as a tab (all pages) does the last one for every page at once, whatever each page is set to. With many tabs the **All N items** jump list is still there, and the tab bar can take three shapes (**Options -> Tab bar**):

Tab bar	Looks like
One row that scrolls	The default: one row with arrows when the tabs do not fit (**Ctrl+PageUp / Ctrl+PageDown** step through them)
Wrapped rows	Every tab visible, on as many rows as needed

Tab bar	Looks like
Side list	The tabs in a column on the left of the page, with a scroll bar when needed



The Processor page with wrapped tab rows: every tab visible, + and All N items at the end.

Lineups are saved arrangements: which tabs each page has, their order and the tab bar. **Menu -> Lineup** (also in Options and a tab's right-click menu):

Lineup	Tabs
Basic	A few friendly tabs per page (for example Processor: CPU Summary, CPU Clocks, Cache Levels)
Enthusiast	The standard tabs of 6.1 (the tabs of SRS 2.0).
Expert	Every item of every page as a tab, in a side list. The default.
Your own	**Save as new lineup...** keeps the current tabs under a name you choose

Changes on the fly are allowed at any time without saving: the sidebar then says "Lineup: Expert (changed)". **Save to this lineup** stores them in your own lineup (the three built-in ones stay as they are: use **Save as new lineup...**). **Export lineup...** writes a `.srslineup` file you can share or keep, **Import lineup...** reads one (it gets a new name if one of yours already has it), **Remove this lineup...** removes one of your own, and **Reset to the default lineup** goes back to Expert.

The first time the main window opens it asks which lineup you want (Basic, Enthusiast or Expert) and tells you where to change it later: the menu button (or **Ctrl+M**) -> **Lineup**. It asks only once. (If you start in the classic view, the question waits until you first open the main window.) If you used SRS before 2.2, your pages keep the standard tabs of 2.0 (the Enthusiast lineup, preselected as "the standard tabs, kept") unless you pick another one.

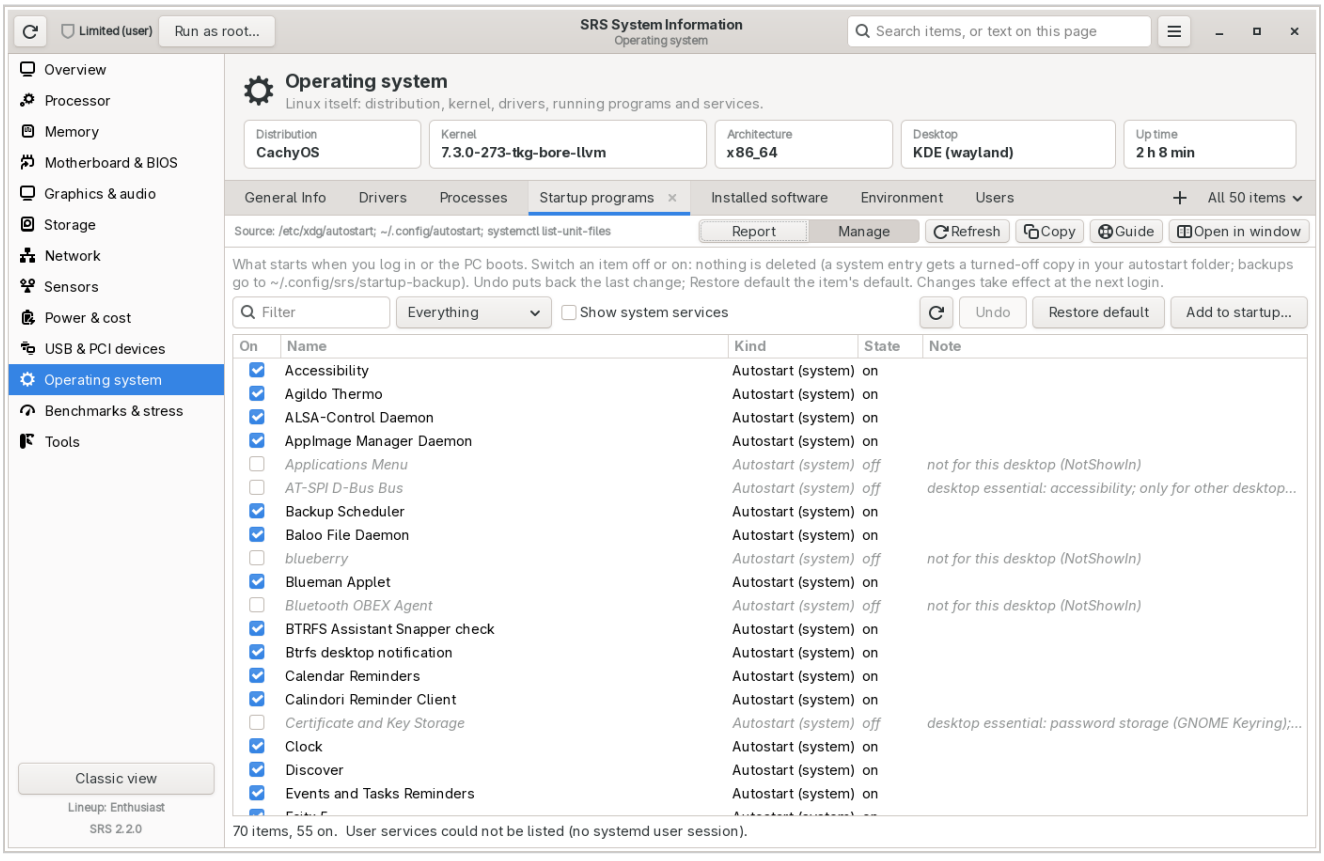
6.8 Sortable tables

Tabs that show a table (a row of blue headings over rows of values, such as Processes, CPU Clocks, Drivers or Installed software) can be sorted: **click a heading**. The first click sorts by that column smallest or A first (`ч` after the heading), the second click largest or Z first (`щ`). The headings are underlined and the mouse pointer becomes a hand over them; hover for a hint.

- Numbers sort by value with their units: 512MB comes before 1.5GB, 900 MHz before 4.41GHz, 9 °C before 45 °C, and RPM, %, V, W and times the same way. Dates sort by date (2026-10-26, 09/18/2026), `[1234]` process numbers by number, and names with numbers in them naturally (CPU-2 before CPU-10).
- Empty cells and `-` always go last.
- Tabs that update by themselves keep the order while they refresh.
- **Reset view** (above the text) puts the item's own order back.
- **Copy** and **Save Report** always give the item's own order, so reports look the same as before; only what you see on screen is sorted.
- **Options -> Sortable tables** turns sorting off (the tables are then plain text, as in 2.0). The classic panels never sort.

6.9 The Startup manager

Operating system -> Startup programs opens on **Report**: the Startup programs item as text, read-only (what Copy and Save Report give). **Manage**, one click away next to it, is the manager: everything that starts when you log in or when the PC boots, with a switch per item.



The Startup manager: autostart entries with their switches, Undo, Restore default and Add to startup.

Column	Meaning
On	Ticked = it starts. Click to turn it off or on. Greyed = it cannot be switched here (see Note).
Name	The program's name. Hover for the file or unit and its command.
Kind	**Autostart (system)** - `/etc/xdg/autostart`, for every user; **Autostart (yours)** - `~/.config/autostart`; **Service (user)** - a systemd user unit; **Service (system)** - a system service (only with **Show system services** ticked).
State	on / off, or the unit's state (enabled, disabled, static, masked ...).
Note	Why it is off or greyed (for example "only for other desktops"), "turned off by SRS (your copy)", "added by SRS", "desktop essential: ...", a unit's default (preset).

Click a column heading to sort; the filter box and the list next to it narrow the list down.

Nothing is ever deleted:

- A **system autostart entry** is turned off with a copy of it in your `~/.config/autostart` that says `Hidden=true` (the desktop uses your copy instead of the system one). Turning it back on moves that copy into `~/.config/srs/startup-backup`.
- **Your own autostart entries** get `Hidden=true` or `Hidden=false`. The first time SRS changes one, the original is kept in the backup folder.
- **Services:** `systemctl --user enable` / `disable` for user units. System services ask first, then use `pkexec systemctl enable` / `disable` (the administrator password); the change applies from the next boot.
- **Undo** puts back the last change made in this session (repeat it for earlier ones).
- **Restore default** puts the selected item back to its default: a system entry loses your copy (moved to the backup folder), your own entry gets its original file back, an entry SRS added is taken out of the list (moved to the backup folder), and a service gets the distribution's default (`systemctl preset`).
- **Add to startup...** starts an installed program (pick it from the list) or any command (run by `sh`) at every login, with an optional delay in seconds. It writes `~/.config/autostart/srs-NAME.desktop`; turn it off or take it out again in this list.

Parts of the desktop ask before they are turned off: Plasma, KWin, KWallet and other password storage, PipeWire and WirePlumber (sound), the desktop portals (file dialogs, screen sharing), the polkit agent (admin password prompts), D-Bus, accessibility and the login screen. You can still turn them off; SRS just says what may stop working. Changes take effect at the next login (system services at the next boot).

6.10 Full access or Limited: the badge

The badge left of the title says what SRS can read right now:

Badge	Means
Full access (green shield)	SRS runs as root, or the root-only data has been collected by srs-root (5). Hover for when it was collected.
Limited (user) (grey shield) and a **Run as root...** button	SRS runs as you, without root data. Drive health (SMART), memory stick details (SPD), serial numbers and BIOS tables, ACPI tables, CPU registers (MSRs) and the kernel log need it; the tooltip lists them. **Run as root...** collects them once with one password prompt (read-only).

With **Options -> Refresh root data at boot** the badge says Full access from the start.

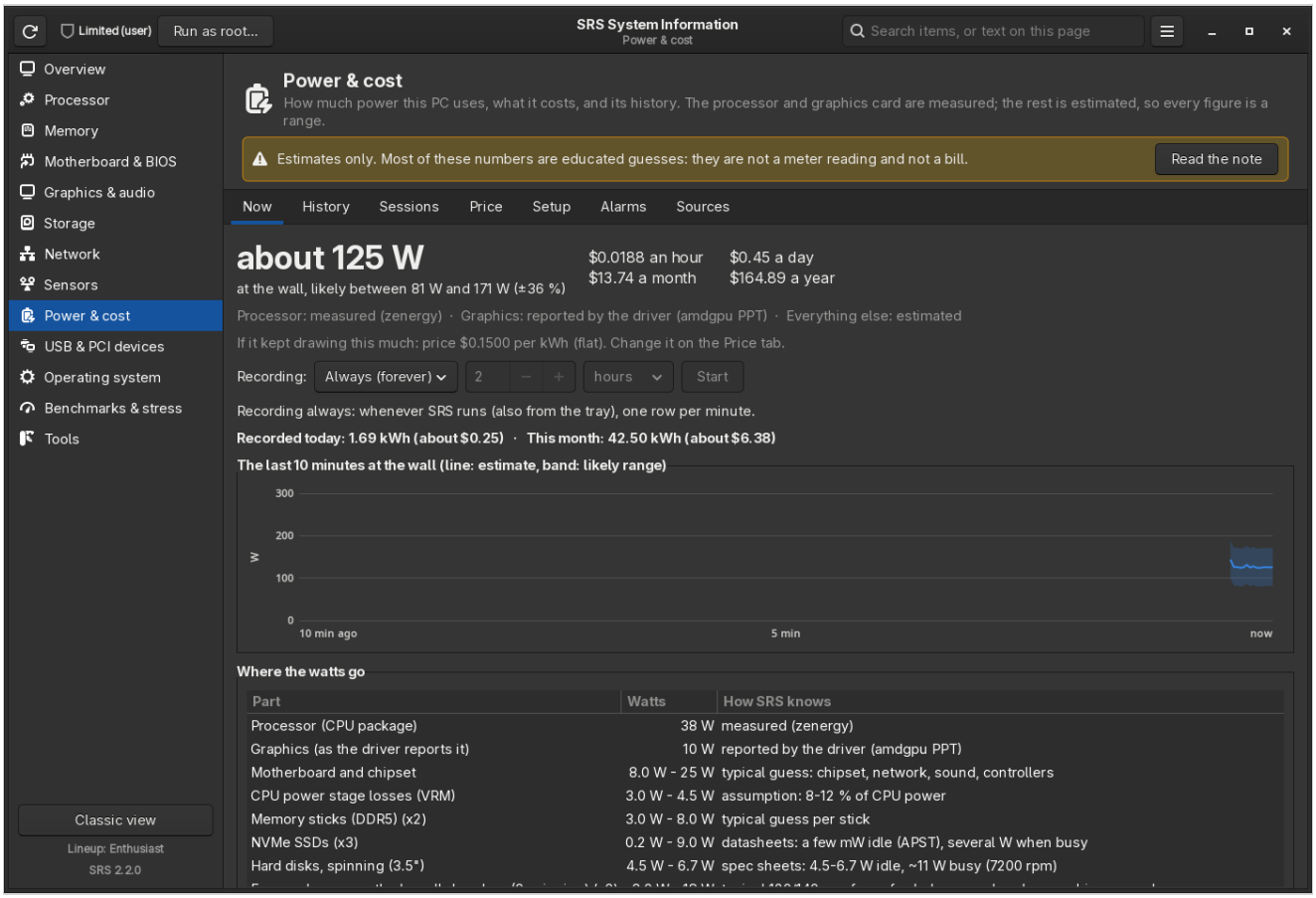
6.11 Power & Cost

> **Estimates only. Please read.** SRS shows how much power your computer **probably** uses and what it **might** cost. These numbers are **estimates**, not a meter reading and not a bill. Only some parts of a PC report their own power (usually the processor and the graphics card), and even those reports can be off. Everything else (motherboard, memory, drives, fans, lights, the power supply's own losses, and your screens) is **guessed** from typical values: that is why SRS shows a range. Prices come from what you type or from public tables that may be out of date or not match your plan. CO_2 figures use grid averages. Alarms can be late, missed or wrong. **Do not use SRS for billing, for safety decisions, or as proof of energy use.** Always check your electricity bill and your supplier. SRS is free software provided "as is", without any warranty (GPL v2.0 or later, sections 11-12); Eugene Flanagin and the SRS contributors are not responsible for any errors, losses, costs or damage that come from using these estimates.

SRS shows this note in full the first time you open the page (press **I understand**), then as a yellow banner at the top of the page (**Read the note** shows it again) and at the top of every CSV export.

In plain words: the page answers "how much power does this PC use, and what does that cost me?". The processor and the graphics card tell Linux how much power they use; SRS adds a low and a high guess for every other part, then allows for the power supply's own losses. The result is a centre figure with a likely range, for example **about 160 W, likely between 116 W and 205 W at the wall**. Recording keeps one line per minute, so you can see days, weeks and months, with the cost at your price.

The page is in the sidebar right under **Sensors** (**Power & Cost**, **Alt+9**). Open it from the command line with `srs-gui --gui-open=power-cost`. The classic view has no Power & Cost page.



Now

- **The big number** is the estimate at the wall, with the likely range under it and the cost per hour, day, month and year **if the PC kept drawing this much** at your price.
- The line under it says how each part is known: **measured** (zenergy, RAPL), **reported by the driver** (amdgpu), or **estimated**.
- **Recording: Off** (nothing is saved; the live figure still works), **For a set time** (choose minutes, hours or days and press **Start**; it stops by itself), or **Always (forever)** (whenever SRS runs, also hidden in the tray; see the background recorder below for when SRS is closed).
- **The last 10 minutes:** the line is the estimate, the shaded band the likely range.
- **Where the watts go:** every part with its watts and where the figure comes from, the power supply's losses, and the total at the wall.

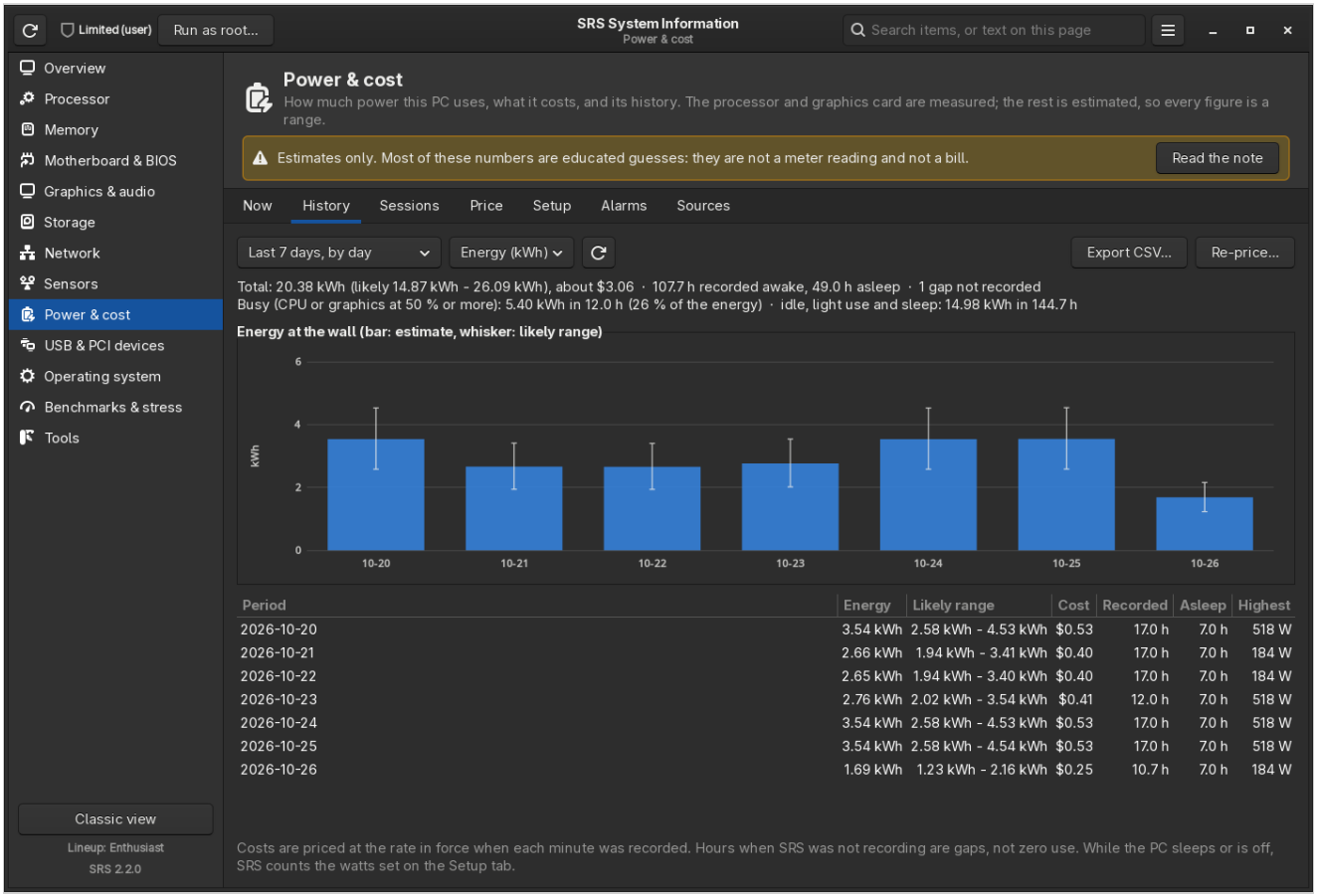
History

Pick a range (last 24 hours by hour; last 7 or 30 days by day; last 12 weeks by week; last 12 months or everything by month) and **Energy** or **Cost**. Bars are the estimate, the whiskers the likely range. Hover over a bar for its numbers. The summary line gives the totals, how long SRS recorded and how long the PC slept, the gaps that were not recorded, CO_2 if you turned it on, and **busy vs idle**: the energy used while the processor or graphics card was busy (at or above the Sessions tab's level, 50 % by default) against the rest.

- **Export CSV...** writes the history for a spreadsheet: one row per minute, hour, day, week, month or year, the shown range or everything, with a decimal point or a decimal comma (then columns are

split by `;`). The file starts with `#` lines: the note above, your power supply, parts list and price. Columns: `period`, `start_utc`, `kwh_estimate`, `kwh_low`, `kwh_high`, `cpu_kwh_measured`, `gpu_kwh_measured`, `cost_<currency>`, `avg_w`, `max_w`, `hours_recorded`, `hours_asleep`, `hours_busy`, `kwh_busy`.

- **Re-price…** costs every recorded minute again with today's price settings. Normally each minute keeps the price that applied when it was recorded (so a time-of-use or tiered price is exact to the minute); use this after correcting a wrong price.



The history picture uses made-up example data.

Sessions

A **session** is a stretch of heavy use, such as a game or a long render: the processor or the graphics card busy at or above a level (50 % by default) for 2 minutes, ending after 3 quiet minutes (all three adjustable). A short pause stays inside the session. Each session shows when it started, how long it lasted, its energy, cost, highest draw and CO_2. These are **the whole PC's** figures while the session lasted: Linux cannot split power by program.

Price

- **Currency:** pick one (its usual symbol, placement and decimals follow), or **Custom** and type your own.
- **Your price:**
  - **Flat:** one price per kWh, from your bill.

    - **Tiered:** up to six tiers ("the first 500 kWh a month at A, then B"). Because the PC is only part of your home's use, you can say how much the rest of the home uses a month; SRS counts that first, so the PC lands in the tier it really adds to. 0 = count the PC first.
    - **Time of use:** four price levels (names and prices editable) and a grid of 24 hours for weekdays and weekends: click an hour to change its level. Each minute is priced at the level of its hour.
    - **Taxes and levies** (%) that are not in the price, and a **standing charge** per day. The standing charge is off in the totals by default, because the PC does not cause it.
- **Typical prices for your area (offline):** if you don't know your price, pick your country and region; SRS fills in an average **from tables built into SRS. Nothing is sent anywhere.** The country is pre-selected from your time zone. For the United States, type a five-digit **ZIP code** and press Enter: SRS picks the state and shows the utility that probably serves it with its average residential price. **Use this price** copies it into "Your price". These are averages for an area, not your plan, and they may be out of date; the date of each table is shown. Your own bill's price is always better.
- **What if:** the cost a year if the PC were left on 24/7 at today's draw and at this month's recorded average, and this month's forecast from the recorded days.
- **CO_2 estimate:** off, or a country's (or the world's) average grid emissions per kWh. Grid averages only; your supplier's mix may be very different.

Setup

- **Power supply (PSU):** pick yours or the nearest **80 PLUS** level at 115 V or 230 V, and type its rating in watts. **I don't know my power supply** uses a wide range. Experts can type an efficiency curve as `load %:efficiency %` pairs (for example from the PSU's Cybenetics report) and press **Use curve**. The line under it shows the load and efficiency right now.
    - The **Montech CENTURY II 1200W** preset is certified **80 PLUS Gold** and **Cybenetics Platinum** (two different test schemes). Its curve is **derived, not measured**: the 80 PLUS Gold minimums, lifted to the Cybenetics Platinum average that HWbusters' review found, and that review's note that efficiency falls **under 70 % at 2 % load** (about 24 W). The exact percentages are only in the review's charts, so the range is kept wide.
    - Below 20 % load (where most PCs idle) the 80 PLUS tests say nothing, so SRS widens the range there.
- **Parts that cannot be measured:** SRS starts from what it finds on this PC (memory sticks, NVMe drives, hard disks, spinning fans, monitors, the motherboard) with typical low and high watts. Double-click a value to change it, untick a row to leave it out, **Add a part...** for typical parts (case fans, AIO cooler and pump, fan hub with lighting, LED strips, monitors, ...), **Reset to SRS's guess...** to start again. **Counts as:**
    - *Watts each*: low and high watts, times the number.
    - *% of CPU power / % of GPU power*: the processor's power stage losses (8-12 %) and the rest of the graphics card (5-30 %: the driver's number leaves out fans, memory and power stages).
    - *CPU / Graphics card (if not measured)*: used only when SRS cannot read that part.
    - *Beside the PC (at the wall)*: monitors and speakers; added to the wall figure, not through the PSU. Monitors are off by default; the energy label's "on mode" power is the best figure.
    - A fan hub on one motherboard header can drive several fans: SRS only sees one header, so set the number of fans yourself.
- **Sleep, off and recording:** the watts the PC draws asleep (default 3 W) and switched off (default 1 W), counted while recording; how often to read the sensors (every 2 s by default; the energy

counters keep exact totals, so 1-10 s is plenty); **Keep recording when SRS is closed** (below); where the files are; **Delete all history...**.

Alarms

Add an alarm... offers: power right now over a limit for some seconds, the average over some minutes over a limit, energy or cost today or this month reaching a limit, and this month heading over a budget (a forecast from the days so far). Limits can be in W, kW or BTU/h (power) or in Wh, kWh, MWh, kJ or MJ (energy); the line under the settings shows the same limit in the other units. An alarm fires **once**: power alarms again only after the power drops clearly (5 %) below the limit, day and month alarms once per day or month. They show as desktop notifications (through the desktop's notification service; no sound from SRS), and on the tab. Energy and cost alarms need Recording on. **Test notification** shows what one looks like.

Sources

What SRS can measure on this PC, in plain words, and the raw readings. **SRS never changes who may read these files.**

Source	What it is	Who can read it
zenergy driver (`Esocket`)	AMD's RAPL energy counter for the CPU package	normal users, by design (the driver limits how often it updates)
amd_energy driver	the same counter (older driver)	normal users
powercap RAPL (`/sys/class/powercap/intel-rapl:*`)	the CPU package counter, Intel and AMD	**root only since Linux 5.10**: energy readings can leak secrets (PLATYPUS, CVE-2020-8694/8695/12912). SRS uses it only if it is readable.
amdgpu `power1_average` (PPT)	the graphics chip's power as the driver reports it	normal users
Intel i915 / xe `energy1_input`, nouveau `power1_input`	graphics card energy or power	normal users
NVIDIA (proprietary driver)	not read yet: the graphics card row of the parts list is used	

If the CPU cannot be measured, install the **zenergy** driver (AMD), or run SRS as root for a quick check; SRS does not ship a rule that opens RAPL to everyone. On AMD the counter is a model inside the chip, not a meter: good for changes, and it may read lower than a wall meter.

Recording, sleep and restarts

- Recording writes one row per minute: seconds recorded, estimate with its low and high, the measured CPU and GPU energy, the cost at the price in force, the highest draw and the busy time. About 40 MB a year.
- Energy counters run round like an odometer (RAPL here after about 65 kJ: about 23 minutes at 47 W). SRS reads them every few seconds and **unwraps** them; a reading after a gap too long to tell one wrap from two is skipped, not guessed. A counter that goes down without wrapping (a driver reload) starts a new baseline.
- **Sleep** is noticed from the clocks (Linux's boot-time clock keeps counting while asleep, the monotonic clock does not); the time asleep is counted at the "asleep" watts. **Shut-down to the next start** is noticed from the boot ID and counted at the "off" watts (up to 31 days). Time when SRS was not running is a **gap**, never made-up data.
- **Keep recording when SRS is closed** writes `~/.config/systemd/user/srs-power-recorder.service` and enables it: it runs `srs-gui --power-recorder` as you (not as root) and records only while Recording is **For a set time** or **Always**. Unticking it stops and removes the service. Only one SRS records at a time (a lock file next to the database); the other shows the same history.

Files

Path	What
`~/.local/share/srs/power.db`	The history (SQLite; readable only by you). `power.db.lock` beside it.
`~/.config/srs/power.ini`	The page's settings: recording, power supply, parts, price, alarms, sessions.
`~/.config/systemd/user/srs-power-recorder.service`	Only when **Keep recording when SRS is closed** is on.
`/usr/share/srs-sysinfo/power-zip-rates.tsv`	The US ZIP code to utility table.

Where the built-in figures come from

Data	Source	Date in SRS 2.4.0	Licence / terms
US average price by state	U.S. Energy Information Administration, *Electric Power Monthly*, Table 5.6.B (residential, year to date)	August 2026 YTD	U.S. government work (public domain); credit EIA
US utility by ZIP code, with its average residential price	NREL, *U.S. Electric Utility Companies and Rates: Look-up by Zip Code (2024)*, from EIA-861 data	2024	Creative Commons Attribution 4.0 (CC BY 4.0); reformatted by SRS

Data	Source	Date in SRS 2.4.0	Licence / terms
Europe, price by country	Eurostat, `nrg_pc_204` (households, 2 500-4 999 kWh a year, all taxes and levies)	2026-S1	Reuse with the source acknowledged (Eurostat copyright notice). SRS shows the figures unchanged. Eurostat is not responsible for SRS's use of them.
Great Britain, price cap	Ofgem, energy price cap (GB average, Direct Debit), unit rate and daily standing charge	1 October to 31 December 2026	Published figures, credited to Ofgem
CO_2 per kWh by country	Ember, *Yearly electricity data* (emissions intensity)	up to 2025	CC BY 4.0
80 PLUS efficiency levels	Wikipedia, "80 Plus" (citing CLEAResult, the certifier)		Facts, credited
Montech CENTURY II 1200W	HWbusters review (Cybenetics Platinum average, under 70 % at 2 % load); PCPartPicker (80 PLUS Gold)	2026-10	Facts, credited

The tables are made by `tools/gen_power_rates.py` from the downloaded files (their addresses are in the script); SRS itself never goes online for them. Australia (AER Default Market Offer) is not included yet: type your price.

Words used on this page

Word	Plain meaning
W (watt)	How fast power is being used *right now*. A 100 W PC uses 100 watts at this moment.
kW (kilowatt)	1,000 W.
Wh (watt-hour)	Energy used *over time*: 100 W for one hour = 100 Wh.
kWh (kilowatt-hour)	1,000 Wh: 1,000 W for an hour, or 100 W for ten hours. **Your bill counts kWh.**
J, kJ, MJ (joule)	Another energy unit: 3,600 J = 1 Wh, 3.6 MJ = 1 kWh. Linux counters count microjoules.
BTU/h	Heat per hour: 1 W ‡ 3.412 BTU/h. All the power a PC uses ends up as heat in the room.
At the wall	What the PC draws from the socket, including the power supply's losses. This is what you pay for.
Inside the PC (DC)	Power after the power supply, used by the parts. Always less than at the wall.
PSU efficiency	The share of the wall power that reaches the parts: 90 % means 10 % becomes heat in the PSU.
80 PLUS / Cybenetics	Two certifications of PSU efficiency. 80 PLUS tests at 20, 50 and 100 % load (Titanium also 10 %); Cybenetics tests many loads, light ones too.
RAPL	The processor's built-in energy counter ("Running Average Power Limit"). Intel measures; AMD models.
PPT	AMD's "Package Power Tracking": the chip's own power figure. For a graphics card it is not the whole card.

Word	Plain meaning
Wrap-around	A counter reaching its top and starting again from zero, like an old car's odometer.
Tiered price	The price per kWh changes after so many kWh a month.
Time of use	The price per kWh depends on the hour and the day.
Standing charge	What you pay per day whatever you use.
Carbon intensity	Grams of CO_2 released per kWh on your grid, on average.

6.12 Memory Timings

In plain words: memory has a speed (like 6000 MT/s) and a long list of small waiting times, the "timings", that say how many clock ticks the memory needs between steps. The BIOS sets them, usually from a profile stored on the module (EXPO on AMD, XMP on Intel). The **Memory Timings** tab shows what the memory really runs at right now, next to what each module's profiles say, so you can check that the BIOS applied your profile and see what changed after you tuned it. Everything on it is only read. New in 2.4.

Open: the **Memory** page -> the **Memory timings** tab (or search "memory timings", item `mem-tweak`); in the classic view it is in the Menu's list of items.

What it needs. On **AMD Ryzen 7000 / 9000 desktop** processors the live timings need the `ryzen_smu` driver (Arch: AUR `ryzen_smu-dkms-git`) and one **Menu > Run as root**. The clocks and voltages need only the driver. The module profiles need nothing: they come from the kernel's `spd5118` driver, without root, on any PC with DDR5. **Intel Core 6th gen to Core Ultra**: live speed and the main timings after Run as root, **experimental** (not checked on a real Intel PC yet - please report what you see and what your BIOS says). Other processors say "not supported yet".

What the parts of the page mean:

- **Clocks and voltages** - the memory speed and its clock (MEMCLK), the memory controller's clock (UCLK) with its ratio to MEMCLK (1:1 is fastest; 1:2 is usual above about 6000 MT/s), the Infinity Fabric clock (FCLK, the link between the cores and the memory controller), and the SoC voltages: VSOC (memory controller and fabric), VDDP (the memory interface), VDDG and VDD MISC.
- **Memory controller** - how many channels have a module, **Gear Down Mode** and **command rate** (1T / 2T: how the controller times its commands; GDM on with 1T is normal for DDR5), power down, bank groups, the refresh mode and Nitro mode.
- **BIOS memory settings (AMD AOD table)** - the electrical settings the BIOS chose: the processor's on-die termination (ProcODT), how hard the CPU and the memory chips drive their signals (drive strengths), the memory chips' own termination (the RTT values), and the DIMM and CPU memory voltages the BIOS set. These are what overclockers tune for stability. They come from a table the BIOS leaves in memory, which most kernels only let programs read when booted with `iomem=relaxed` (5.4); SRS never asks you to change that. They are shown only when the table's own timings match the live ones, so values from a layout that does not fit your BIOS are never shown.
- **Timings** - one row per timing, one column per channel, with the time in nanoseconds for the main ones and a short meaning for each. The first ones are the familiar numbers: **tCL** (CAS latency: clocks from a read command to the first data), **tRCD** (opening a row before a read or write), **tRP** (closing a row), **tRAS** (how long a row stays open), **tRC** (a whole row cycle), and the refresh timings

tREFI (time between refreshes) and **tRFC** (how long a refresh takes). The rest are the "secondary" and "tertiary" timings overclockers tune. Small differences between channels in the PHY and turnaround rows are normal: the memory training sets them per channel.

- **Modules (SPD)** - per module: slot, maker and part number, size and ranks, the maker of the memory chips, when it was made, its temperature, and its profiles: **JEDEC** (the standard speed every board can run), **EXPO** and **XMP** (the faster tested profiles), each with speed, main timings, refresh times and voltages. Compare the profile with the Timings above.

How to read it. Memory speed at the JEDEC speed (4800 MT/s on a 6000 kit) means EXPO / XMP is off in the BIOS. Timings looser than the profile at the same speed mean the BIOS did not apply it (or you changed them). With the PMIC opt-in on, **PMIC set VDD / VDDQ / VPP** shows what each module's power chip is set to, and whether it is locked.

Safety. The page only reads. The memory controller registers are read the way 5.4 describes; the module profiles come from the kernel's own copy of the SPD; the PMIC voltages are an opt-in that asks first. SRS never changes a memory setting.

The register map and the table layouts follow ZenStates-Core / ZenTimings by Ivan Rusanov and memtest86+ by Sam Demeulemeester (facts only, no code copied). Every field is described in the Section Reference.

7. The panels and their buttons

In the classic view (6.6), and from **Open in window** on any tab, each panel opens in its own window with its own button row, like SIV. Every panel has **OK** (close) and **Copy** (copy all text). Live panels refresh on a timer; press **F5** to refresh now.

Every panel also has a **Guide** button (right end of the button row; also **F1**, and **Open guide for this item** in the panel's right-click menu). It opens this guide at that panel's section of the Section Reference: in the PDF viewer if it is Okular, Evince or Papers (they can jump to a section), otherwise the HTML copy of the guide in your web browser. **Menu -> User Guide...** opens the start of the guide. The guide files are `/usr/share/doc/srs-sysinfo/SRS-User-Guide.pdf` and `/usr/share/doc/srs-sysinfo/html/SRS-User-Guide.html`; the menu entry **SRS User Guide (PDF)** opens the PDF.

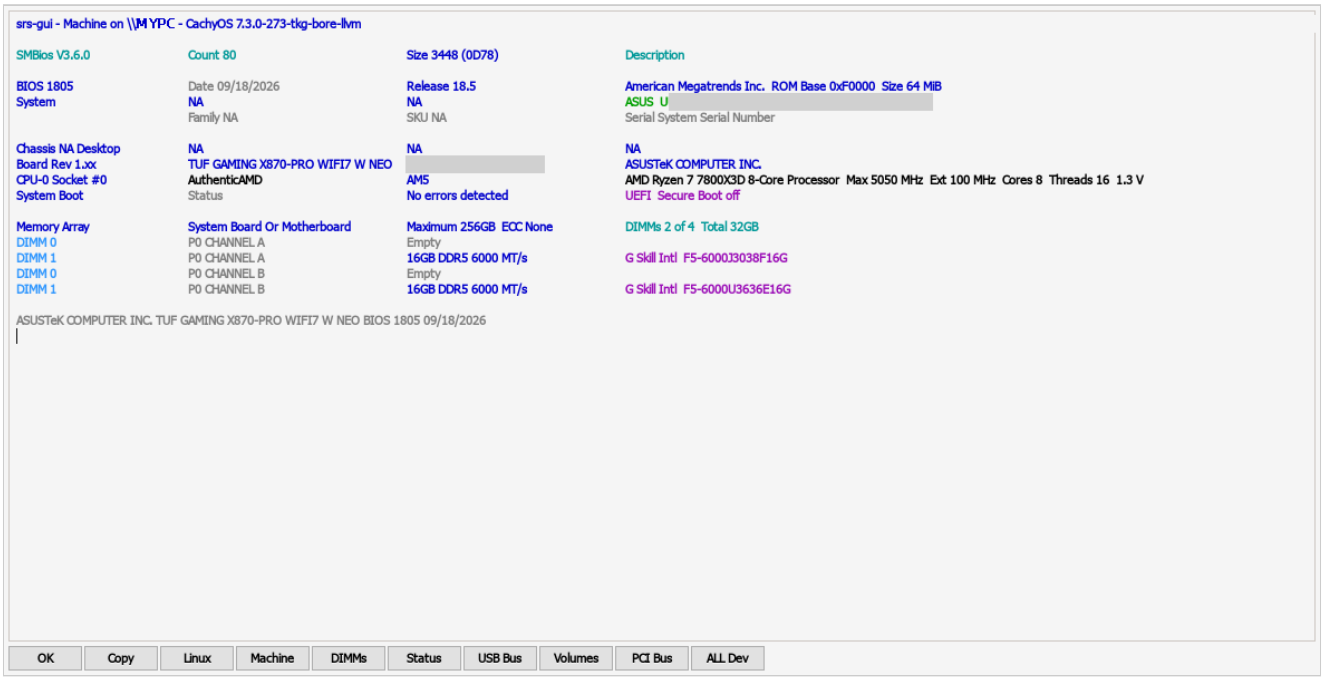
Below is what each panel's extra buttons do. For the meaning of every column and value, follow the link to the Section Reference.

7.1 Linux (SIV's "Windows" panel)

The classic **Linux** button opens the **General Info** item (`windows`; called "Windows (OS) panel" before SRS 2.0). In the main window it is the first tab of the **Operating system** page. The operating system: distribution, kernel, boot, desktop session, directories, kernel command line, per-CPU identification, time zone, up-time, user and locale, displays. Buttons: Linux, Machine, Sensors, USB Bus, Processors, Volumes, PCI Bus, **ALL Dev** (every device), **Graphics**. Reference: linux-os.md, graphics.md.

7.2 Machine

SMBIOS version, BIOS, system, chassis, board, CPU socket, boot status, memory array and every DIMM slot.

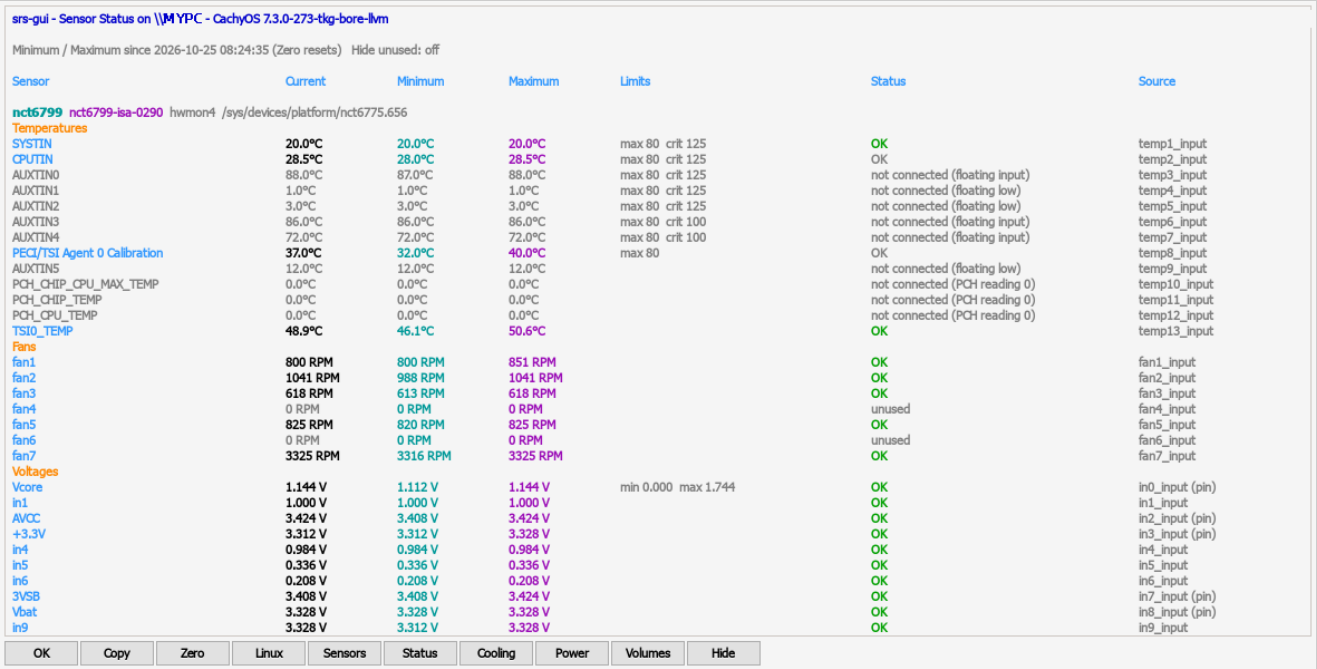


Machine panel (serial number and UUID blanked out in this picture).

Buttons: Linux, Machine, **DIMMs** (memory stick details), **Status** (DIMM temperatures and status), USB Bus, Volumes, PCI Bus, ALL Dev. Reference: machine.md.

7.3 Sensors

Every hardware monitoring chip the kernel knows (Super-I/O, CPU, GPU, drives, memory, others): current, minimum, maximum, limits and a plain status for each reading.



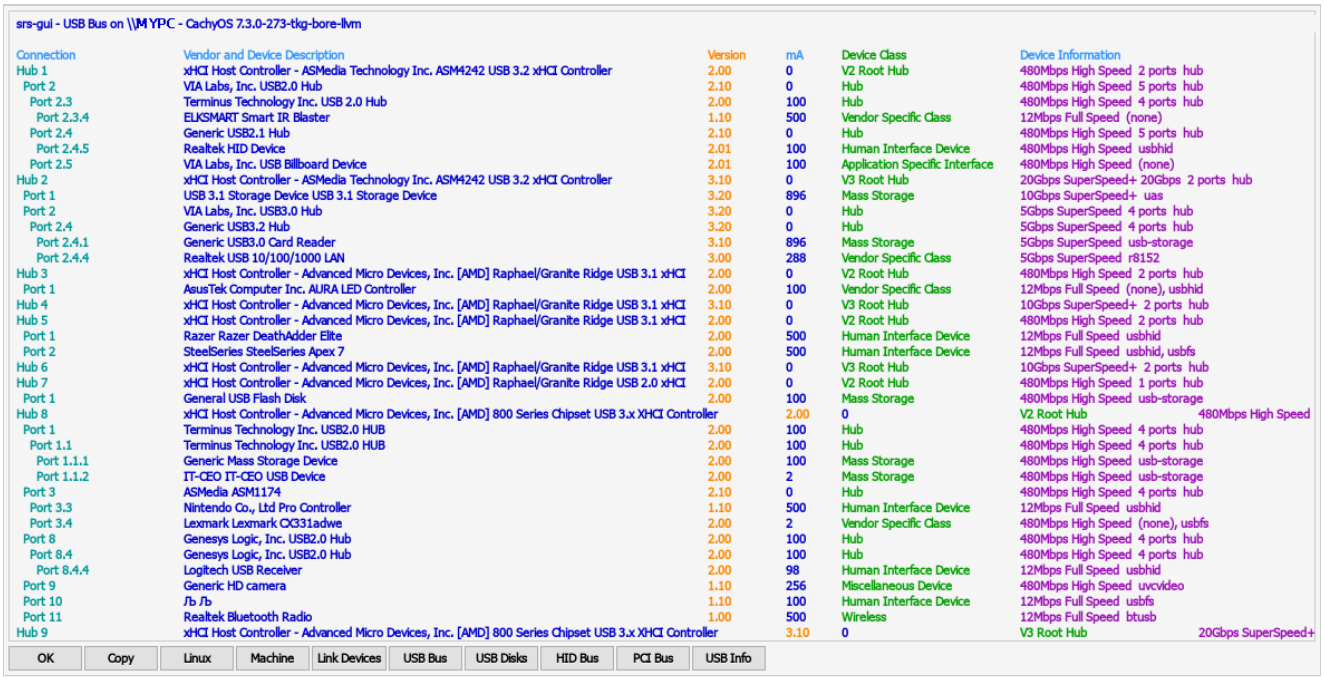
Buttons:

- **Zero** - resets the Minimum and Maximum columns to the current values (the "since" time on the second line resets too).
- **Status** - the hardware status summary.
- **Cooling** - fans and fan control.
- **Power** - power readings (CPU package, GPU, battery...).
- **Hide** - hides sensors that are unused or not connected; press again to show them.
- Linux, Sensors, Volumes.

Some motherboard inputs are not wired to anything and show nonsense values (for example 70-88 C, or 0 C). SRS greys them out and labels them "not connected". The Section Reference explains why: sensors.md.

7.4 USB Bus

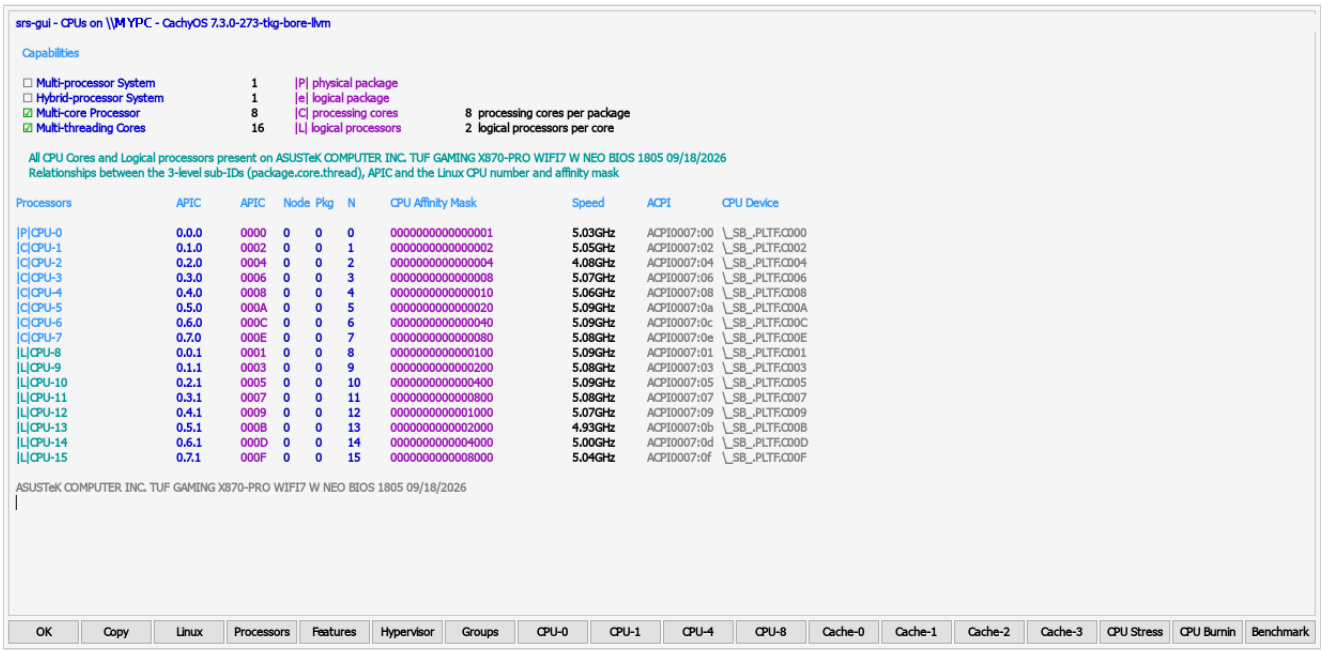
The USB tree: each hub and port, device name, USB version, power draw (mA), class and speed.



Buttons: Linux, Machine, **Link Devices**, USB Bus, **USB Disks**, **HID Bus** (keyboards, mice, game controllers...), PCI Bus, **USB Info**. Reference: buses-1.md, buses-2.md.

7.5 Processors

Capabilities (packages, cores, threads), and for every logical CPU: IDs, APIC ID, affinity mask, current clock, ACPI name.



Buttons: Linux, Processors, **Features** (CPU feature flags), **Hypervisor** (virtualisation), **Groups** (CPU groups / scheduling), **CPU-0, CPU-1, CPU-4, CPU-8** (one CPU's full details), **Cache-0 ... Cache-3** (cache levels), **CPU Stress** and **CPU Burnin** (both open Stress & Burnin), **Benchmark**. Reference: processors.md.

7.6 Volumes

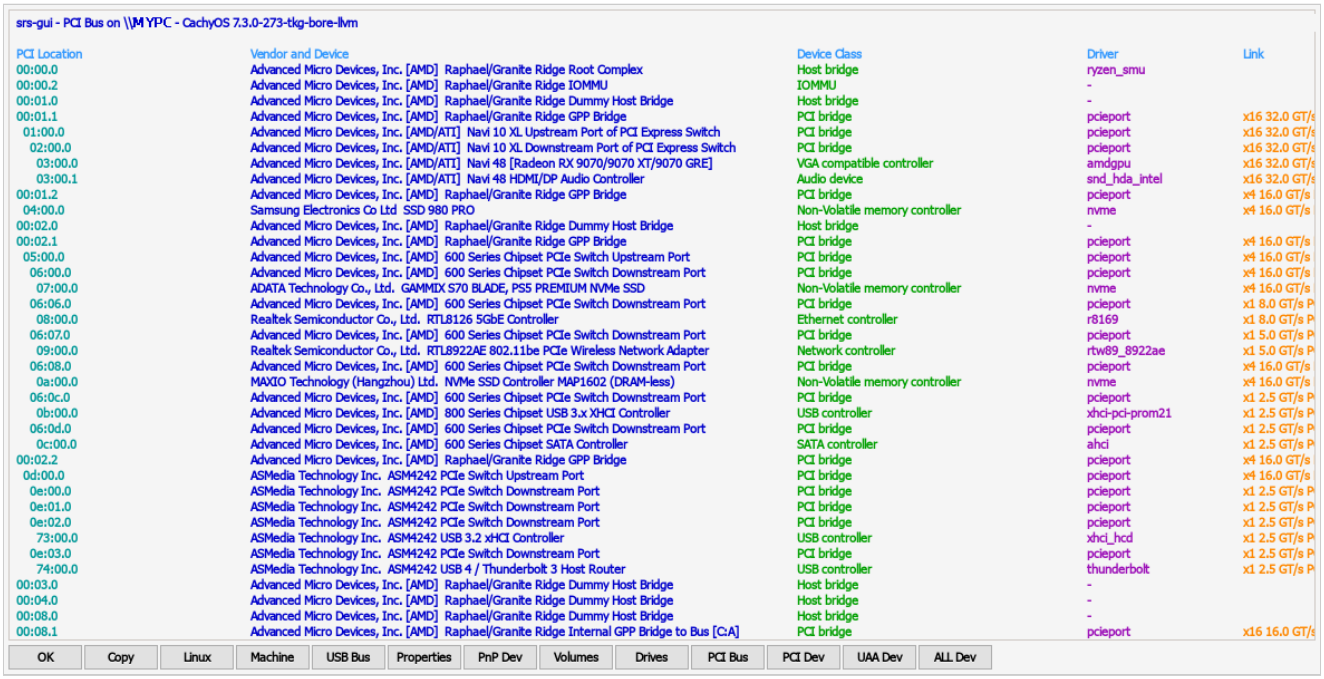
Every mounted file system: label, format, free, total, used percent, cluster size, type and volume ID. Buttons: Linux, Machine, Sensors, USB Bus, Volumes, **Drive /** (details of the root file system), **FS** (file-system statistics), ALL Dev. Reference: storage.md.

7.7 Adapters

Network adapters: name, driver, link, speed, addresses. Buttons: Linux, Adapters, **Counters** (traffic counters), **Interface**, **ARP** (neighbour table), **Routing**, **TCP V4** / **TCP V6** (open ports and connections), **Wi-Fi**, **Bluetooth**, **Show DNS**, **Ping** (pings your default gateway), **Main** (brings the main window to the front). Reference: network.md.

7.8 PCI Bus

Every PCI / PCI Express device: location, vendor and device name, class, driver, and the current PCIe link (width and speed).



Buttons: Linux, Machine, USB Bus, **Properties** (udev properties), **PnP Dev** (ACPI / Plug and Play devices), Volumes, **Drives**, PCI Bus, **PCI Dev**, **UAA Dev** (audio devices), **ALL Dev**. Reference: buses-1.md, buses-2.md.

Tip: in any panel, **double-click a line that contains a PCI ID** like `1002:7550` to open it in Device ID Lookup. (USB IDs open on the PCI bus; switch the lookup to USB by hand.)

7.9 About and Help

About (the item **About SRS**) shows the program and version, build, the data files in use (hwdata `pci.ids`, `usb.ids`, `pnp.ids` with their dates) and where settings live. Then who made SRS ("Created by Eugene Flanagin"), its licence ("Licensed under" GPL-2.0-or-later) and its home page (https://github.com/

eugenesflanagin-spec/srs-sysinfo), and last the statement of origin, credit and disclaimer. In the main window: **Menu -> About SRS**, or the first tab of the **Tools** page. **Help** shows help text and the command-line qualifiers. Their buttons: Linux, DNS Names, Lookup (data files), USB Dev, About, Help; the Lookup panel adds Signature, SRS DLLs and ID Lookup. Reference: srs-items.md.

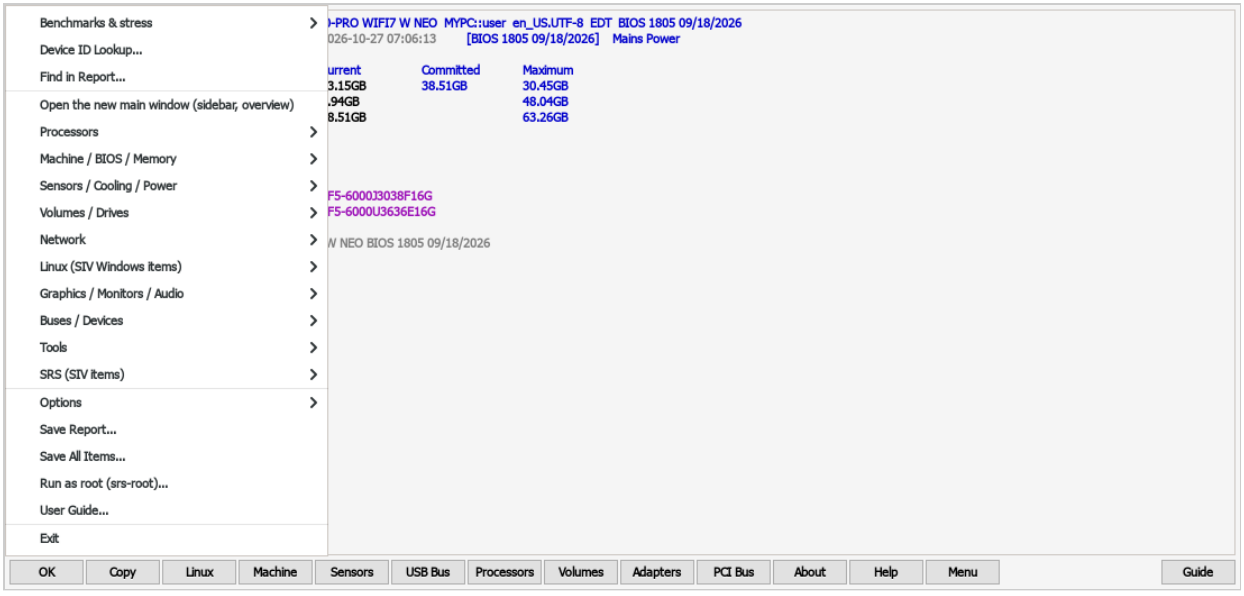
7.10 Every other item

Every other item opens in a window with **OK, Copy, Refresh, Menu, Help**.

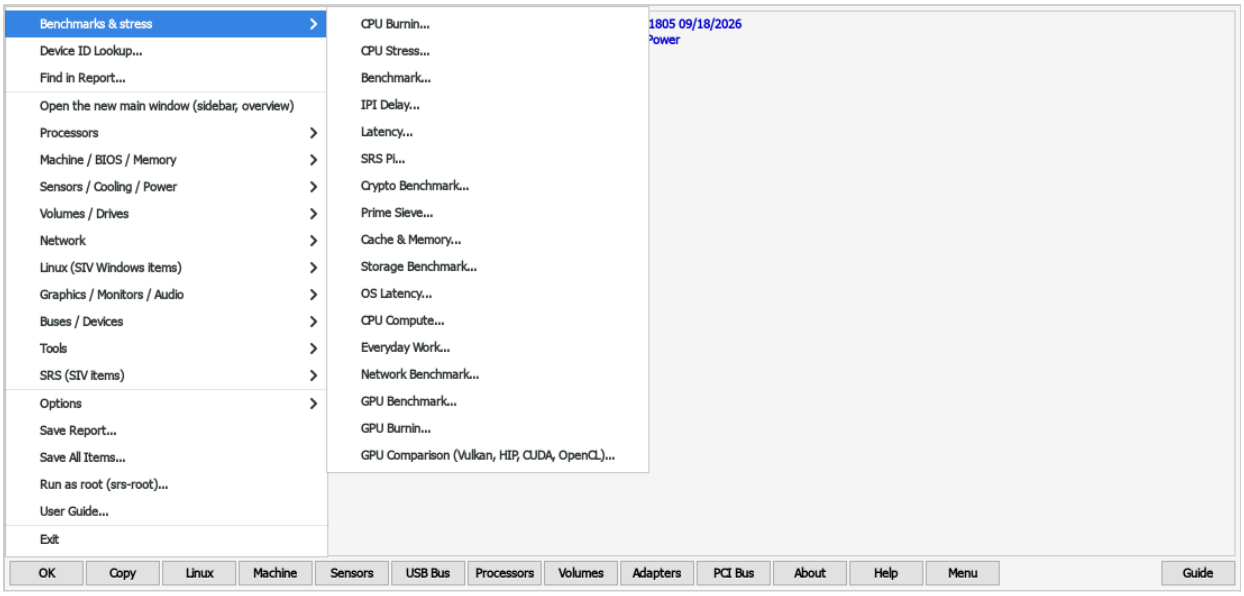
8. The Menu

This is the classic view's Menu. Press **Menu** on the classic main window, press **Ctrl+M** in any classic window, right-click the text of any panel and choose **SRS Menu**, or use **Menu -> Classic Menu** in the main window. It has:

1. **Benchmarks & Stress** (a submenu): Stress & Burnin (CPU / GPU)..., Benchmark..., IPI Delay..., Latency..., SRS Pi..., Crypto Benchmark..., Prime Sieve..., Cache & Memory..., Storage Benchmark..., OS Latency..., CPU Compute..., Everyday Work..., Network Benchmark..., GPU Benchmark... and GPU Comparison...; then **Device ID Lookup...**, **Find in Report...**, and **Classic view** (from the main window) or **Open the new main window** (when SRS started in the classic view). Every entry has its own underlined letter (Alt + letter).



The classic Menu, opened with the Menu button on the classic main window.



The Benchmarks & Stress submenu of the classic Menu.

2. **Every item, grouped** (the same groups as the Section Reference):

   - Processors
   - Machine / BIOS / Memory
   - Sensors / Cooling / Power
   - Volumes / Drives
   - Network
   - Linux (SIV Windows items)
   - Graphics / Monitors / Audio
   - Buses / Devices
   - Tools
   - SRS (SIV items)

   Each entry shows the title and the item id in brackets, e.g. `Cache Levels [cache]`. Items for hardware most PCs lack are marked "(not present)"; Windows-only items are marked "(Windows-only)".

3. **Options** (section 10).

4. **Save Report...**, **Save All Items...** (section 13).

5. **Run as root (srs-root)...** (section 5).

6. **User Guide...**

7. **Exit.**

9. Tools

Nothing in this section ever starts by itself: not at launch, not from the menu, and not from a saved report. You always press Start or Run, and **every benchmark asks before it starts**, saying what it does, about how long it takes, how many CPU threads (or the GPU) it keeps busy and, for SRS Pi, how much

memory it needs. For GPU Burnin, the internet speed test and the heavy runs of the CPU Benchmark, SRS Pi, Crypto and Prime Sieve (all CPU threads at once, SRS Pi from 8 million digits, Prime Sieve from 10 billion) **No** is the default answer, so pressing Enter does not start them. On the command line the benchmarks ask the same question (`Start now? [y/N]`); `--yes` starts them without asking, for scripts (CLI reference).

What has and has not been tested. I build and test SRS on one PC (a Ryzen 7 7800X3D with a Radeon RX 9070 XT, on a wired connection), so anything that needs other hardware may have a bug or two I haven't met yet. If you find one, please open an issue on GitHub with your hardware and the report SRS made, and I'll fix it. The release notes and the CHANGELOG (Known limits) say in detail what was tried; in short, not tried on real hardware so far:

- GPU Benchmarks: Vulkan only on lavapipe (the Mesa software renderer, which runs on the CPU), with small sizes; ROCm/HIP, CUDA on NVIDIA and OpenCL only as far as loading their libraries and finding every function SRS needs - no HIP, NVIDIA CUDA or OpenCL work was run on a GPU. CUDA through ZLUDA: the GPU Compute comparison was run on the author's Radeon RX 9070 XT (2026-10-29), every CUDA row from the built-in kernels (PTX, new in 2.3) without NVRTC; only the cuBLAS FP64 row says "not available" (ZLUDA's cuBLAS has none). GPU Burnin with CUDA was not run on the real card. GPU Burnin's 2.4 matrix multiply ran on the author's RX 9070 XT with ROCm / HIP (about 13.0 TFLOPS FP32) and OpenCL (about 13.4), 0 errors; with Vulkan and CUDA only on lavapipe and a fake CUDA driver so far. Untested: every other real GPU use (AMD, NVIDIA, Intel) and the temperature, power, clock and fan read-outs for NVIDIA and Intel. The built-in CUDA kernels were never run on NVIDIA hardware. The NVIDIA CUDA toolkit install (Options) was tested with fake package managers only.
- Memory Timings (6.12): checked on one Ryzen 7 7800X3D with DDR5-6000 EXPO. The Ryzen 9000 PM table layout (clocks and SoC voltages) is not checked. **Intel memory timings are experimental**: no Intel PC was available, the decoding was checked with made-up register values only. The PMIC and embedded controller opt-ins were not checked on real hardware yet, nor the EBDA dump (the author's UEFI PC has no EBDA).
- ARM (aarch64) and 32-bit (i686) builds: compile-checked and, where noted, self-tested, not used day to day.
- OS Latency: run on Linux 7.x with the scx_lavd scheduler only; other schedulers (BORE, EEVDF) are recognised from the kernel's settings but were not run here.
- CPU Compute: the AVX-512 code has only run on a Zen 4 CPU (Ryzen 7000); the SSE2, AVX2 and plain C code was checked by forcing each one on that same CPU, not on older CPUs.
- Everyday Work: run with the compression libraries of one Arch Linux system (zlib-ng in zlib mode, zstd 1.5, lz4 1.10, xz 5.8); other versions should work but give different speeds. The CRC32C row that uses the CPU's own instruction is in the 64-bit x86 build only.
- Network: run on one wired Ethernet connection only; Wi-Fi, VPN and metered or mobile connections are untested. The internet speed test was run once on that connection, against Cloudflare, with sensible results (download several hundred Mbit/s, upload about 60 Mbit/s; the report showed no IP address, host name or MAC address). It uses Cloudflare's public speed test server, which is not a documented API and may change.

Things to know about the GPU tests:

- A back end that is not installed shows as "not installed" with a one-line hint (for example "install ROCm"); the others still run.
- If your CUDA library is really ZLUDA (CUDA calls translated to an AMD GPU), the column is labelled "CUDA (ZLUDA)".

- Every OpenCL driver is its own column, for example "OpenCL (ROCm)" and "OpenCL (Rusticl)" on the same GPU; CPU drivers (PoCL) are marked CPU.
- Speeds are only comparable between alike SRS builds and the same GPU driver / runtime versions.
- GPU Burnin ends by itself on a driver error, a GPU that stops answering, a piece of work that takes far too long, or a temperature at the limit (default 100 °C). It never changes clocks, voltages or fans. It asks before it starts.
- Opening a GPU window's list of GPUs, or pressing Run, reads the list of GPUs through Vulkan, the way `vulkaninfo` does: the GPUs are named, but no work is sent to them until you answer Yes. ROCm / HIP, CUDA and OpenCL are started only after Yes.

9.0 Use at your own risk

Stress tests and benchmarks push the processor, the graphics card, the memory, the drives and the power supply to their limits. That can bring out faults and cause crashes, freezes, lost or damaged data, or overheating - more so with an overclock or undervolt. Save your work and close other programs first.

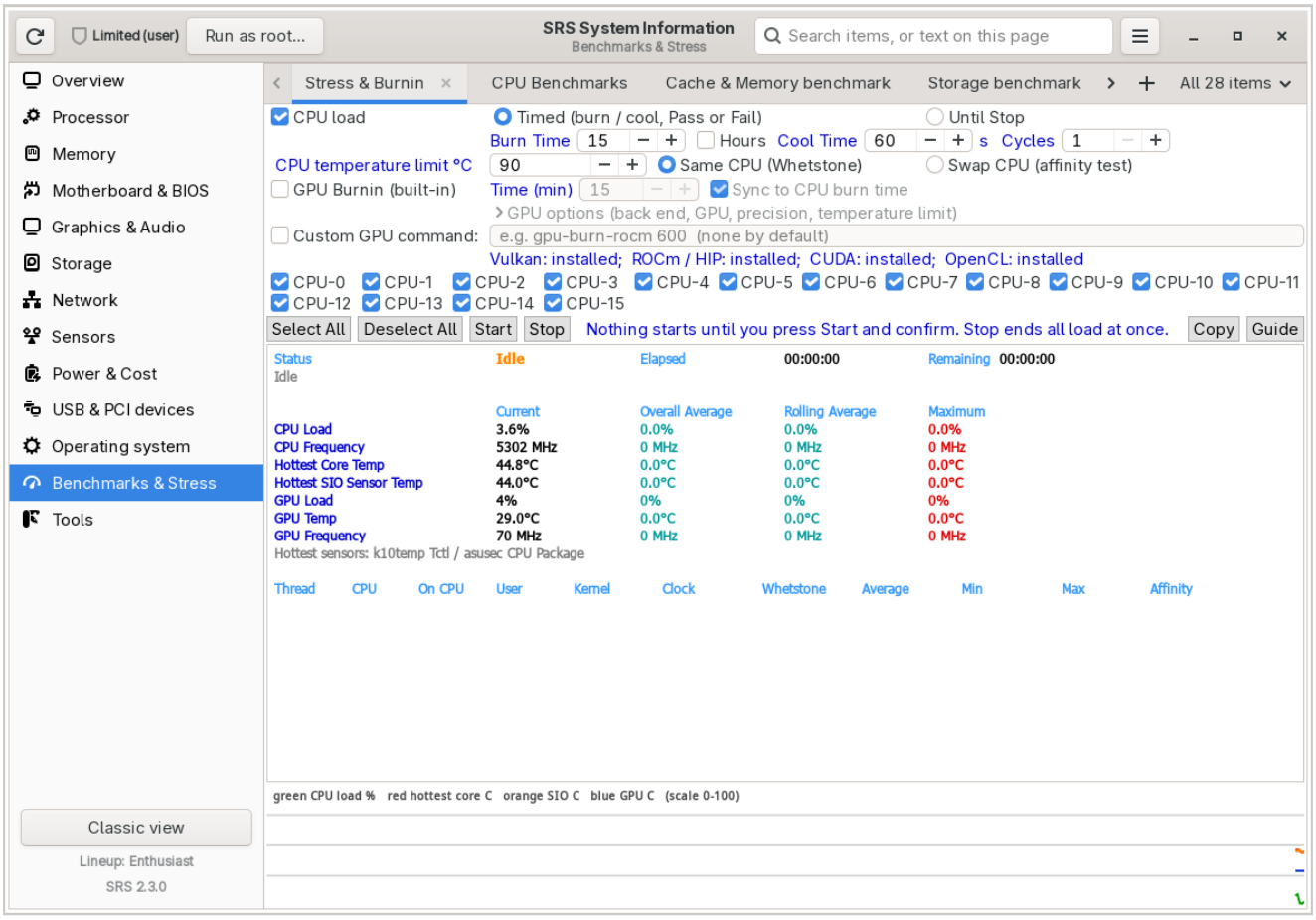
SRS is provided as is, **without any warranty**, as its licence (the GNU General Public License) says. The authors are not responsible for any damage, data loss or other harm that comes from using these tools. This is a plain-English summary, not legal advice; the licence text is what counts.

- **The first time** you press Start or Run in any stress test or benchmark (new main window or Classic View), SRS shows these terms with **I Agree** and **I Disagree**. I Agree is remembered (`Accepted Load Test Terms=1` in `SRSDEF.ini`). I Disagree starts nothing, and every Start / Run asks again until you agree.
- A short notice with a **Terms...** link sits under every stress test and benchmark's controls.
- **Menu > Options > Load test terms (warranty)...** shows them again; I Disagree there withdraws an earlier agreement.
- **Command line:** the start question (`Start now (and agree)? [y/N]`) and `--help` of every load command carry the same notice. `--yes` starts without asking and counts as agreeing; `burn` starts at once (as before) and prints the notice.

Tools as tabs (new main window, 2.3.1). On the **Benchmarks & Stress** page every tool is a tab of its own - Stress & Burnin, CPU Benchmarks, Cache & Memory, Storage, OS latency, CPU compute, Everyday work, Network, GPU Benchmark, GPU Compare, SRS Pi, Crypto, Prime Sieve, IPI delay and Memory latency - with its controls and live results in the page itself, no pop-up window. The small sections at the top of the page (one line each) jump to a tool's tab. The controls are compact and scroll sideways in a narrow window, so the results get most of the height. Only one test runs at a time. Leaving a tab, the page or the window while a test runs asks: **Stop it** or **Keep running** (and stay). Classic View keeps every tool in its own window.

9.1 Stress & Burnin (CPU and GPU)

In plain words: one place to make the processor, the graphics card or both work at 100% so you can see whether the PC stays stable and how hot it gets. Use it after building a PC, changing cooling, or changing BIOS or GPU settings such as overclocks, undervolts (Curve Optimizer) or power limits. Since 2.3.1 this one window replaces the separate CPU Burnin, CPU Stress and GPU Burnin windows.



Open: the **Benchmarks & Stress** page -> the **Stress & Burnin** tab (or its section at the top of the page), the menu entry "SRS Stress & Burnin", the tray menu, or in the classic view Processors -> CPU Burnin / CPU Stress and Menu -> Benchmarks & Stress -> Stress & Burnin (CPU / GPU)... The old entries still work: "CPU Burnin" opens it set to **Timed**, "CPU Stress" to **Until Stop**, "GPU Burnin" with only the GPU ticked. In the new main window it is a tab of the page; Classic View keeps it as its own window.

Before the first run SRS asks once whether you agree to use its stress tests and benchmarks at your own risk (9.0). Until you click **I Agree**, Start asks again and nothing starts.

The three parts, top to bottom:

Control	Meaning	Default
CPU load	Make the ticked CPUs 100% busy. Untick it to test only the graphics card.	on
Timed (burn / cool, Pass or Fail)	CPU Burnin: run for the **Burn Time**, rest for the **Cool Time**, repeat for the **Cycles**, then Pass or Fail.	on
Until Stop	CPU Stress: keep the CPUs busy until you press Stop (safety cap 750 hours).	off
Burn Time + **Hours**	How long each burn lasts (minutes, or hours when Hours is ticked).	15 minutes
Cool Time (s) / **Cycles**	Rest between burns, and how many burn + cool rounds.	60 s / 1
CPU temperature limit C	The run stops at once if the hottest CPU sensor reaches this. 50-110; over 100 asks once (below). Remembered.	90 C

Control	Meaning	Default
Same CPU (Whetstone) / **Swap CPU (affinity test)**	Workers stay on their CPU and give a Whetstone score each, or move between CPUs to check the system places them correctly.	Same CPU
GPU Burnin (built-in)	Load the graphics card too, with SRS's own GPU Burnin (9.18): large matrix multiplications that count every wrong result.	off
Time (min)	How long the GPU part runs, from Start. It always ends with the CPU run at the latest.	15
Sync to CPU stress time / **Sync to CPU burn time**	Run the GPU exactly as long as the CPU load: with Until Stop until you press Stop; with Timed during each burn phase, pausing while the CPUs cool. The Time box is greyed while it is ticked.	on
GPU options (folded)	Back end (Auto = Vulkan, ROCm / HIP, CUDA, OpenCL), which GPU (the first, all, or one from the list), FP32 / FP64, and the GPU temperature limit (50-110, default 100 C). Closed by default; SRS remembers whether you opened it. Even closed, its line shows what it holds, e.g. "GPU options: Auto (Vulkan), FP32, stop at 100 °C" (2.4), so the GPU's own limit is never hidden.	Auto, first GPU, FP32, 100 C
Custom GPU command	Instead of the built-in GPU Burnin, run your own GPU stress program (for example `gpu-burn-rocm 600`). SRS checks the program exists first. It pauses in each cool phase and stops with the run. Ticking it unticks GPU Burnin and the other way round.	empty
CPU-0 ... CPU-n, **Select All / Deselect All**	Which logical CPUs get a worker.	all
Start / Stop	Start (asks first) / stop everything at once.	

The built-in GPU Burnin runs as its own small process (`srs-gui --gpu-burn-child`): if a GPU driver hangs, the SRS window stays usable, Stop still works, and the GPU part ends when SRS ends. Its own messages go to `~/.cache/srs/gpu-load.log`, its result to `~/.cache/srs/bench-gpuburn.txt`.

What you see while it runs, all in the results area under the controls:

- **Status** (Idle / Burning / Cooling / Pass / Fail / Stopped), **Elapsed**, **Remaining**.
- **Current, Overall Average, Rolling Average, Maximum** for CPU Load, CPU Frequency, Hottest Core Temp, Hottest SIO (motherboard) Sensor Temp, GPU Load, GPU Temp and GPU Frequency.
- The **GPU** line: GPU Burnin (built-in) running / finished / stopped / could not start, or the custom command's state (exit code, not found, killed).
- With GPU Burnin: its live page - per GPU the errors found, GFLOPS now and on average, edge / hotspot / memory temperature, power, clock and fan; when it ends, its full result.
- A table per worker thread (CPU, where it runs now, user / kernel time, clock, Whetstone score, **Affinity** ok / FAIL) and a graph: green CPU load %, red hottest core C, orange board sensor C, blue GPU C.

How to read the result:

- **Pass** - the run finished, no worker failed, no temperature limit was reached, and GPU Burnin (when it ran) found no compute errors.

- **Fail** - a CPU worker failed (affinity error), a temperature limit was hit, GPU Burnin found compute errors, or GPU Burnin ended early on a safety rule (GPU temperature, driver error, a GPU that stopped answering). A temperature stop means the cooling cannot keep up at this load. A worker failure, wrong GPU results, or a crash or freeze during the run point to instability (overclock, undervolt or memory settings too aggressive).
- Compare Whetstone scores per CPU: a CPU far below the others may be throttling or busy.

Leaving the tab, the page or the window while a run is on asks whether to **Stop it** or **Keep running** (and stay); closing the tab or the classic window stops it.

Stop limits over 100 C

In plain words: the stop limits are a safety net: the run stops when the CPU or the GPU gets that hot. Since 2.4 both can be set up to 110 °C, for people who know their chips run hot by design (many Ryzen CPUs hold about 89-95 °C under full load; AMD graphics cards' hotspot can reach 100-110 °C) and want the test to keep going there.

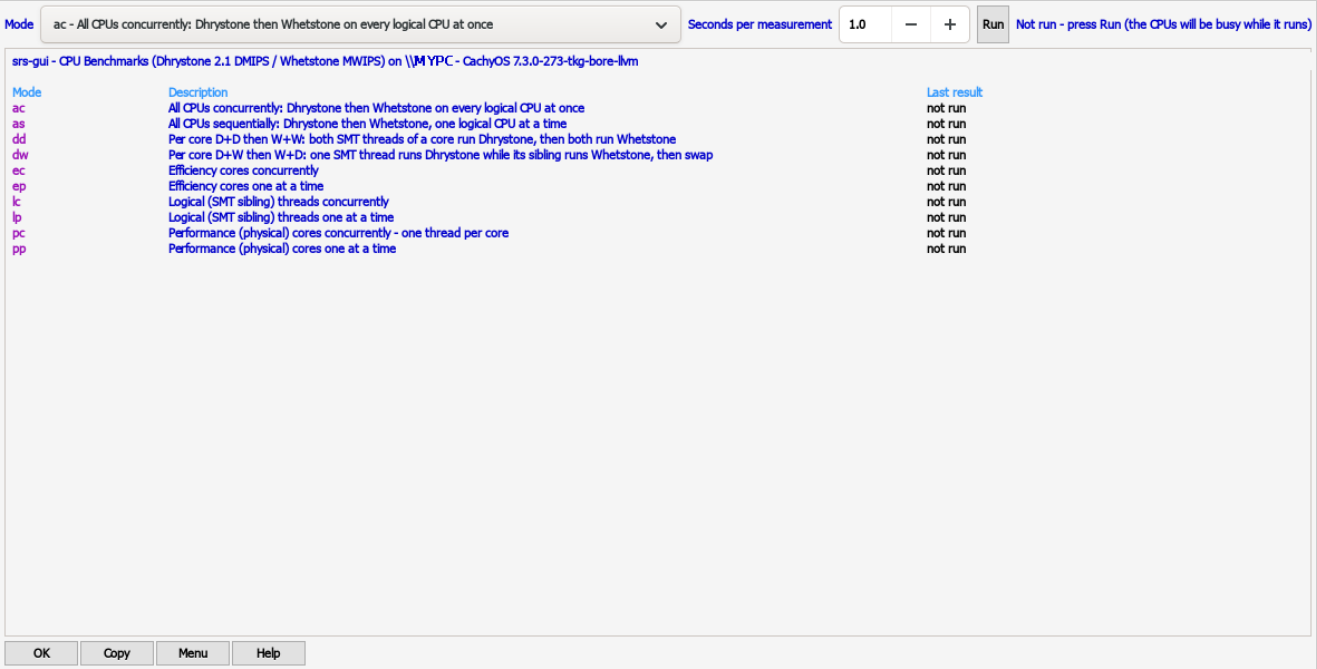
- The first time you set either limit above 100, SRS asks once: **Stop limits over 100 °C: at your own risk**. Above 100 °C many processors and graphics cards are already at or near their own maximum (often 89-110 °C): the chip then slows itself down, and with poor cooling, worn thermal paste or an overclock a fault becomes more likely. The chip's own protection stays on; SRS never changes clocks, voltages or fans.
- **Yes, I accept the risk** is remembered for both limits (`Accepted High Temperature Limits=1` in `SRSDEF.ini`). **No** (the default) puts both limits back to 100.
- A limit over 100 in `SRSDEF.ini` without that yes is read as 100.
- Command line: `burn` and `gpu-burn` take `--limit` 50-100, and 101-110 only together with `--accept-high-limit` (CLI reference).

Saved results. Every run that ends (Pass, Fail or Stopped) is saved to its own file, `~/.cache/srs/burnin-YYYYMMDD-HHMMSS.txt` (the date and time it ended), the same text as **Copy**. To compare two runs: `diff ~/.cache/srs/burnin-20261025-101500.txt ~/.cache/srs/burnin-20261025-113000.txt` or open both in `meld`. SRS never deletes these files.

Full details of every column: tools.md.

9.2 Benchmark (Dhrystone and Whetstone)

In plain words: two classic, small CPU tests. **Dhrystone** measures whole-number (integer) work and reports **DMIPS** (Dhrystone millions of instructions per second). **Whetstone** measures decimal (floating-point) work and reports **MWIPS** (millions of Whetstone instructions per second). Higher is better. They are good for comparing the same PC before and after a change, less good for comparing very different CPUs.



Open: the **Benchmarks & Stress** page -> **Benchmark...**, the menu entry "SRS Benchmark", or in the classic view Processors -> Benchmark or Menu -> Benchmarks & Stress. The page's **CPU Benchmarks** tab shows the last results.

Controls: **Mode**, **Seconds per measurement** (0.1-60, default 1), **Run** (asks first: the mode, about how long it takes and how many CPU threads are busy; for the modes that load many threads at once - `ac`, `pc`, `lc` - **No** is the default answer). Command line: `sysreport-search bench ac` (asks first; `--yes` starts at once). The modes, as in SIV:

Mode	What runs
`ac`	All CPUs concurrently: Dhrystone then Whetstone on every logical CPU at once.
`as`	All CPUs sequentially: one logical CPU at a time.
`dd`	Per core D+D then W+W: both SMT threads of a core run Dhrystone, then both run Whetstone.
`dw`	Per core D+W then W+D: one thread runs Dhrystone while its sibling runs Whetstone, then swap.
`ec` / `ep`	Efficiency cores concurrently / one at a time (hybrid CPUs).
`lc` / `lp`	Logical (SMT sibling) threads concurrently / one at a time.
`pc` / `pp`	Performance (physical) cores concurrently (one thread per core) / one at a time.

The window lists the last result of every mode. Results are kept between sessions.

9.2.1 What hyperthreading (SMT) really does - a worked example

In plain words: most modern CPUs run two "threads" on every core. AMD calls this SMT (simultaneous multithreading), Intel calls it Hyper-Threading. Linux shows each thread as its own CPU, so an 8-core CPU appears as 16 CPUs. The two threads of a core are **not** two cores: they share the same core and take turns using its parts. The benchmark modes let you see exactly what that sharing is worth. This page walks through one real set of results.

The PC. One Ryzen 7 7800X3D: 8 cores, 16 threads. Linux numbers its threads so that CPU-0 to CPU-7 are the first thread of cores 0 to 7, and CPU-8 to CPU-15 are the second thread of the same cores (CPU-0 and CPU-8 share core 0, CPU-1 and CPU-9 share core 1, and so on). Your PC may number them differently; the benchmark uses Linux's own list of which threads share a core.

> These numbers are one PC's example, measured on one day. Your CPU, cooling, BIOS settings and kernel will give different numbers. Some modes ran 5 seconds per measurement and some 20, and differences of 1-2 % between runs are normal noise. What carries over to other PCs is the **pattern**, not the exact scores.

The results (Total of each mode; higher is better):

Mode	What ran	Threads busy at once	Dhrystone (DMIPS)	Whetstone (MWIPS)
`pp`	first thread of each core, one core at a time	1	1,018,068	119,397
`lp`	second thread of each core, one at a time	1	1,012,772	118,632
`pc`	first thread of all 8 cores at once	8	971,275	112,851
`lc`	second thread of all 8 cores at once	8	967,107	112,073
`ac`	all 16 threads at once	16	1,611,893	197,840
`dd`	both threads of one core run the same test, one core at a time	2	1,772,110	214,340
`dw`	one thread runs Dhrystone while its partner runs Whetstone, one core at a time	2	1,899,990	227,781

What the numbers show, step by step:

1. **The "physical" and the "logical" thread are equally fast.** `pp` and `lp` differ by about 0.5 %, and `pc` and `lc` by less than 1 %. That is noise. When a thread has a core to itself, it does not matter whether Linux calls it the first or the second thread of that core: it is the same core. There is no "real" core and "fake" core.

2. **Loading every core at once costs a few percent.** `pc` (8 cores busy) is about 5 % below `pp` (one core busy at a time): Dhrystone -4.6 %, Whetstone -5.5 %. That is not SMT at all. It is boost: a CPU runs its highest clock when only one or two cores are busy, and drops the clock a little when all of them are, to stay inside its power and temperature limits.

3. **The second thread adds a lot of work.** `ac` (all 16 threads) against `pc` (one thread per core, same 8 cores): **+66 % Dhrystone, +75 % Whetstone**. So a second thread did not double the work, but it added two thirds or more. Each thread in `ac` kept about 83 % (Dhrystone) and 88 % (Whetstone) of the speed it had with the core to itself. This happens because one thread alone rarely keeps every part of a core busy; the second thread fills the gaps.

4. **The same gain shows on a single core.** `dd` (both threads of one core, one core at a time) against `pp` (one thread, one core at a time): **+74 % Dhrystone, +80 % Whetstone**. This is a little more than in step 3 because only one core is busy, so it keeps its highest boost clock (`dd` is about 10 % above `ac` for that reason).

5. **Mixed work shares a core better than identical work.** `dw` against `dd`: **+7 % Dhrystone, +6 % Whetstone**. In `dd` both threads want the same parts of the core (both doing whole-number work, then both doing decimal work). In `dw` one thread does whole-number (integer) work while its partner does decimal (floating-point) work. Those use mostly different parts of the core, so they get in each other's way less: each thread kept about 93-95 % of its solo speed.

6. **One core scores a little lower than the rest.** In the `pp`, `pc`, `ac` and `dd` runs, core 1 (CPU-1, and its partner CPU-9) was the slowest, a few percent below the others. On this PC that core handles most of the hardware interrupts (network, disk and USB events), and every interrupt takes a little time away from the benchmark. It is not a faulty core. To see where interrupts land on your PC, look at the counts per CPU in `/proc/interrupts`.

What this means for you:

- Leave SMT on for work that uses many threads at once (compiling, video encoding, rendering, compression): here it gave 66-75 % more total work from the same 8 cores.
- A single thread gains nothing from SMT, and loses nothing either: one thread alone on a core runs at full speed.
- Turning SMT off in the BIOS mostly helps in special cases (some games that are sensitive to timing, or security settings that ask for it). Measure before and after: run `ac` and `pc` with SMT on, then `ac` with SMT off. With SMT off, `ac` should land close to the SMT-on `pc` result.
- To check your own PC, run `pp`, `pc`, `ac`, `dd` and `dw` back to back on an idle PC, two or three times, and compare the same steps as above. The Tools reference explains each mode in detail.

9.2.2 Reading the CPU IPI Delay table - a worked example

In plain words: the IPI Delay tool (Benchmarks & Stress > IPI Delay, section 9.3) times how long one CPU takes to hand a small piece of data to another CPU, for every pair of CPUs. When two threads of a program share data (a lock, a queue, a game engine's job list), this is the delay they pay each time. The result is a big grid of numbers. This page shows how to read it, using one real result.

The PC. One Ryzen 7 7800X3D: one CCD (core complex die, the chip that holds the cores), 8 cores, 16 threads. As in 9.2.1, CPU-0 to CPU-7 are the first thread of cores 0 to 7, and CPU-8 to CPU-15 are the second thread of the same cores (CPU-0 and CPU-8 share core 0, and so on). The test ran 100,000 rounds per pair. Each number is in nanoseconds (ns, billionths of a second): the time for one cache line (a 64-byte block of memory) to go there and back, divided by 2, so it is the one-way time.

> These numbers are one PC's example. Other CPUs, BIOS settings and kernels give different numbers, and a difference of 1-2 ns between runs is normal noise. What carries over is the **pattern**.

The result:

```
From\To 0  1  2  3  4  5  6  7  8  9  10 11 12 13 14 15
CPU-0   -  28 30 29 30 32 34 33 25 28 30 29 30 31 34 33
CPU-1   28 -  30 28 31 28 33 34 28 23 30 27 31 28 32 31
CPU-2   30 30 -  31 29 31 29 31 29 30 23 31 29 32 29 31
CPU-3   29 28 31 -  31 27 30 29 28 28 31 23 31 27 30 29
CPU-4   30 31 29 31 -  29 29 29 29 31 29 32 23 30 29 29
```

```
CPU-5   32 28 31 27 29 -  31 27 29 28 31 27 30 23 32 28
CPU-6   34 33 29 30 29 31 -  31 32 32 30 31 28 32 23 30
CPU-7   33 34 31 29 29 27 31 -  33 31 33 29 29 27 31 24
CPU-8   25 28 29 28 29 29 32 33 -  27 29 29 29 30 32 32
CPU-9   28 23 30 28 31 28 32 31 27 -  31 28 31 28 32 30
CPU-10  30 30 23 31 29 31 30 33 29 31 -  31 29 31 29 32
CPU-11  29 27 31 23 32 27 31 29 29 28 31 -  32 27 30 28
CPU-12  30 31 29 31 23 30 28 29 29 31 29 32 -  28 29 29
CPU-13  31 28 32 27 30 23 32 27 30 28 31 27 28 -  32 27
CPU-14  34 32 29 30 29 32 23 31 32 32 29 30 29 32 -  30
CPU-15  33 31 31 29 29 28 30 24 32 30 32 28 29 27 30 -
```

How to read it:

- Each **row** is the CPU that sends, each **column** is the CPU that receives. Find the row of one CPU and the column of the other: the number where they cross is the delay in ns. Lower is faster.
- The `-` on the diagonal is a CPU paired with itself. That is not measured.
- Each pair is measured once, so the table is a mirror: CPU-3 to CPU-5 is the same number as CPU-5 to CPU-3.

What the numbers show, step by step:

1. **The two threads of one core are the fastest pair in every row: 23-25 ns.** CPU-0 and CPU-8 (25), CPU-1 and CPU-9 (23), and so on up to CPU-7 and CPU-15 (24). Every row has its smallest number in the column of its partner thread (CPU n and CPU n+8). These two threads live on the same core and share its L1 and L2 caches, so the data never has to leave the core.

2. **Every other pair is 27-34 ns.** Two different cores do not share L1 or L2. The data travels through the L3 cache, which all 8 cores on the CCD share. That costs a few nanoseconds more.

3. **Distance on the chip still shows a little.** The highest numbers are core 0 to cores 6 and 7 (CPU-0 to CPU-6: 34, CPU-0 to CPU-7: 33, and the same for their partner threads), and core 1 to cores 6 and 7 (33-34). The cores reach the L3 over a ring, and cores that sit farther apart on that ring take a little longer. The whole spread is only about 7 ns.

4. **The table is even.** Apart from the partner threads, no block of the table stands out. That is what a one-CCD CPU looks like: all cores are on the same die and share one L3.

On a CPU with two CCDs (for example a Ryzen 9 7950X or 9950X) the table typically looks different. Pairs on the same CCD look like the table above, but pairs where one CPU is on the first CCD and the other is on the second CCD show a second block of much higher numbers, typically around 70-80 ns. Those pairs do not share an L3: the data has to cross the Infinity Fabric link between the two dies. The exact numbers depend on the CPU, the memory and fabric clocks, and the BIOS.

What this means for you:

- **Find partner threads.** The smallest number in each row tells you which two CPUs share one core, whatever numbering your PC uses.
- **See the CCD layout.** A block of much higher numbers shows where one CCD ends and the next begins, and which CPUs belong to which CCD.

- **Pin threads that talk to each other a lot.** Games, audio work (a DAW and its plug-ins) and other timing-sensitive programs run smoother when their busy threads stay on one CCD, and the lowest delay is on one core's two threads. Tools such as `taskset` can keep a program on chosen CPUs.
- **Do not expect it to change with tuning.** Curve Optimizer and voltage changes move the clock speed and temperature, but the delay mostly depends on how the chip is built, so this table stays about the same. Run it to learn your CPU's layout, not to measure a tweak.

9.3 IPI Delay

In plain words: measures how long it takes one CPU core to "ping" another and get an answer, for every pair of CPUs. Shows as a table in nanoseconds and a colour heat map (green = fast, red = slow). Cores that share a cache or a chip answer faster; cores on different chips (for example two CCDs on a Ryzen 9) answer slower. Open: the **Benchmarks & Stress** page -> **IPI Delay...**, or Menu -> Benchmarks & Stress -> IPI Delay... in the classic view. Controls: **CPUs** (how many to test), **Rounds per pair** (default 20,000), **Run** (asks first; two CPU threads are busy at a time, for about a second at the defaults). With **CPUs** at 1 there is no pair to measure: nothing runs, the window says "Not started" and the last result is kept. Command line: `sysreport-search ipi` (asks first; `--yes` starts at once; `--cpus 1` exits with 2 and keeps the last result). For a step-by-step reading of one real result (partner threads, the L3, what a two-CCD CPU looks like), see 9.2.2 Reading the CPU IPI Delay table - a worked example.

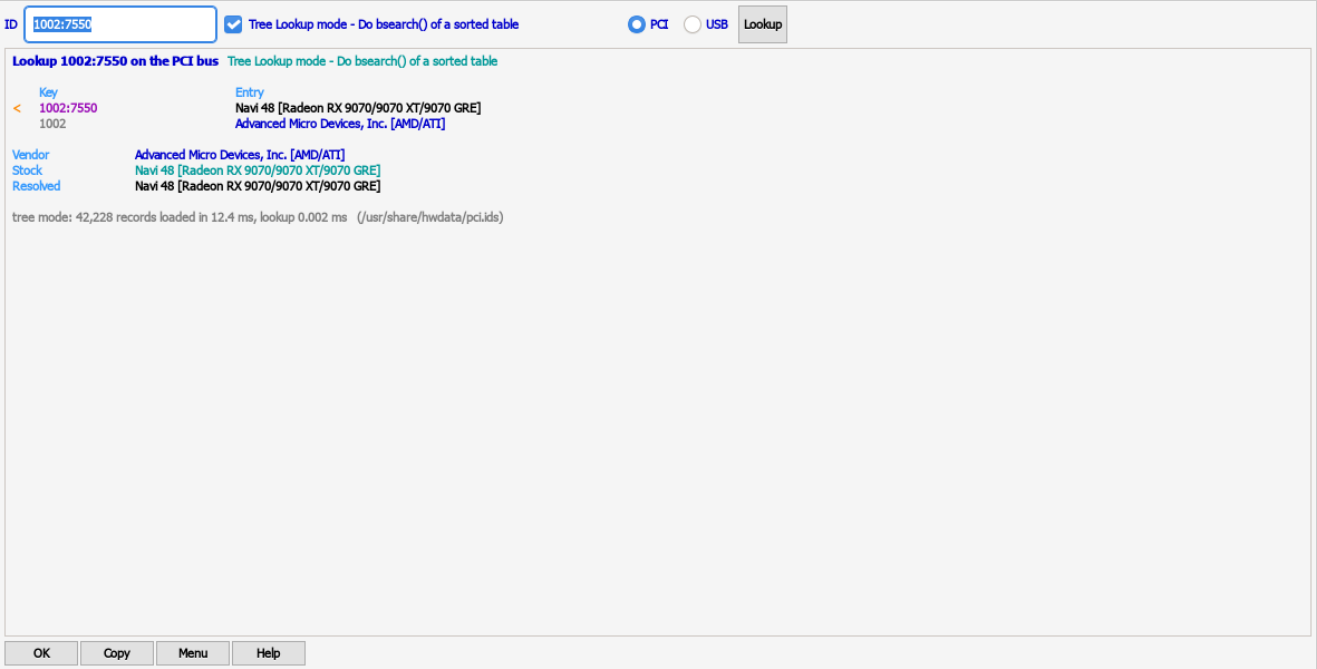
9.4 Latency

In plain words: measures how long one memory read takes, for blocks from tiny to large. Small blocks fit in L1 and answer in about 1 ns; as the block grows past each cache, the time steps up: L2, L3, then RAM. The steps in the graph show you your cache sizes. Open: the **Benchmarks & Stress** page -> **Latency...**, or Menu -> Benchmarks & Stress -> Latency... in the classic view. Controls: **Largest block (MB)** (1-4096, default 256), **Run** (asks first; one CPU thread and that much memory, about 3 seconds at the default). Command line: `sysreport-search latency` (asks first; `--yes` starts at once). Results: a table (block size, time in ns, level) and a curve.

For the **bandwidth** of the RAM and the caches, see 9.11 Cache & Memory. The newer benchmarks - SRS Pi, Crypto, Prime Sieve, the Benchmark History, Cache & Memory, Storage, OS Latency, CPU Compute, Everyday Work, Network and the three GPU tests - are in 9.7 to 9.19; the optional NVIDIA CUDA toolkit is 9.20.

9.5 Device ID Lookup

In plain words: every PCI and USB device has a number pair: **vendor ID : device ID** (for example `1002:7550` = AMD : Radeon RX 9070 series). This tool turns the numbers into names.



Open: **Ctrl+L**, **Menu -> Device ID Lookup...**, the **Tools** page -> **Lookup...**, or double-click an ID on any tab or panel. In the classic view also the Lookup panel's ID Lookup button.

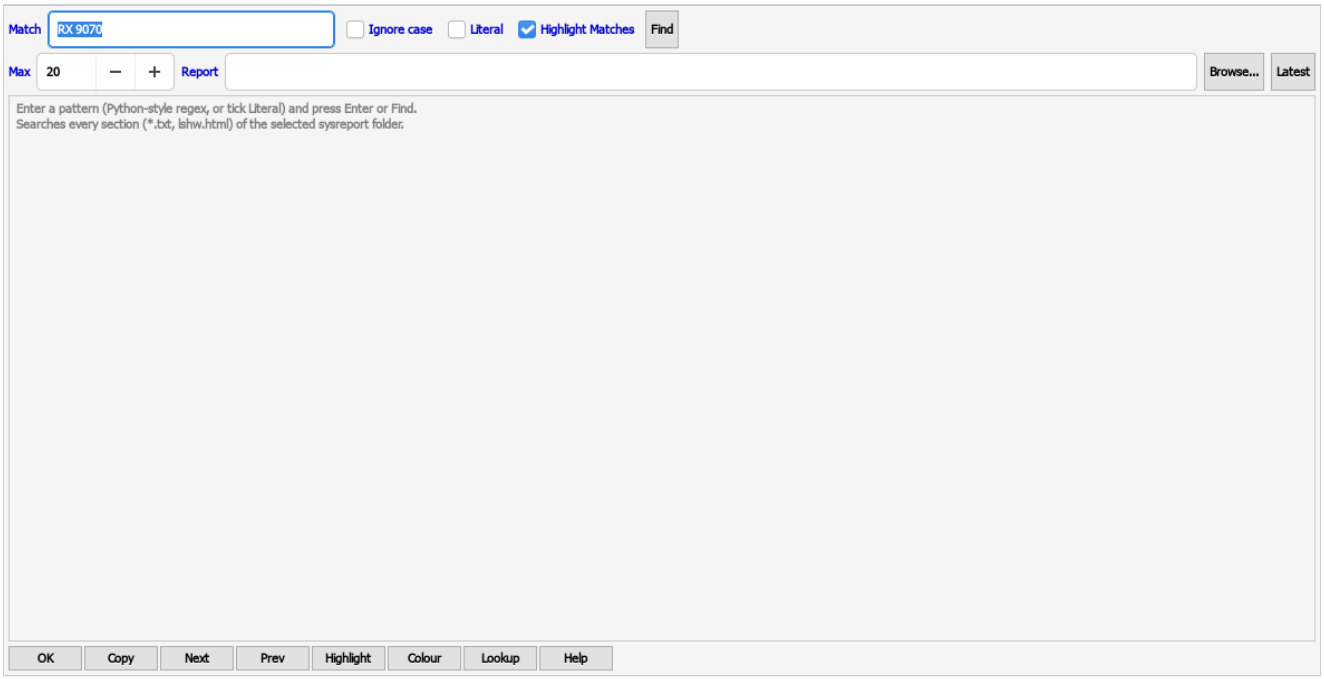
Controls:

- **ID** - type the ID and press Enter or **Lookup**. Formats: `VVVV`, `VVVV:DDDD`, `VVVV:DDDD:SSSS:ssss` (with subsystem vendor and device), and an optional `:RR` revision at the end, e.g. `1002:7550:1EAE:7550:C0`.
- **PCI / USB** - which list to search.
- **Mode** tick box - cycles through three modes, like SIV:
  - **Tree** - "bsearch() of a sorted table": loads the whole hwdata list once, then finds the ID instantly. Tries the most specific key first and falls back step by step.
  - **Scan** - reads the `.ids` file line by line. Slower, same answer; useful to check.
  - **Info** - asks the operating system's own device database (systemd-hwdb) instead of the file.

Output: each key tried (the matching one is marked `<`), then **Vendor**, **Subsystem** (if any), **Stock** (the plain device name) and **Resolved** (the best name found), and how long it took.

9.6 Find (search a full system report)

In plain words: search a saved full system report (made by **SRS Full System Report**, 13.3) for any text, like `grep`, with highlighting.



Open: **Ctrl+Shift+F** or **Menu -> Find in System Report...** in the main window, the **Tools** page -> **Find...**, or the menu entry's "Find in System Report" action. In the classic view: **Ctrl+F** or Menu -> Find in Report....

Controls:

Control	Meaning
Match	What to search for. A regular expression (Python-style) unless Literal is ticked. Press Enter or **Find**.
Ignore case	Upper and lower case count as the same.
Literal	Search for the exact text; characters like `.` `[` `(` have no special meaning.
Highlight Matches	Colour each match (also **Ctrl+H** or the **Highlight** button).
Max	Show at most this many matches per report file (default 20).
Report	The report folder to search. Filled with the newest one in `~/sysreports`.
Browse... / **Latest**	Pick another folder / go back to the newest.
Next / Prev (or **F3 / Shift+F3**)	Jump between matches.
Colour	Choose the highlight colour (saved).
Lookup	Open Device ID Lookup.

If there is no report yet, Find tells you to run SRS Full System Report first.

9.7 SRS Pi

In plain words: SRS works out pi to many decimal places and checks every digit. The time shows how fast one core does heavy number work; a wrong digit means the PC made a mistake (often an unstable overclock or memory). Open: the **Benchmarks & Stress** page -> **SRS Pi...**, or Menu -> Benchmarks & Stress -> SRS Pi... in the classic view. On the command line: `sysreport-search pi --digits 1m`.

Controls: **Digits** (128 thousand to 32 million; each choice shows the time it took on the author's PC), **SuperPi sizes** (tick for SuperPi's power-of-two digit counts: 16K = 16,384 ... 1M = 1,048,576 ... 32M = 33,554,432), **Method** (Chudnovsky = fast, can use threads; Gauss-Legendre / AGM = the method SuperPi uses, one thread), **Threads** (default 1, for a single-thread score), **Run** and **Stop**. **Run asks first**, with the expected time, the threads and the memory it needs; from 8 million digits, or with all threads, it warns about unstable overclocks and **No** is the default answer. On the command line it asks the same; `--yes` starts at once. While it runs the status line shows the time so far and the current step (series, division, square root). Closing the window also stops it.

Times are **SRS Pi** times, not SuperPi times. SuperPi's "1M" is 1,048,576 digits; SRS's 1 million is 1,000,000 - tick **SuperPi sizes** for the same digit counts as SuperPi. 32 million digits takes several minutes and 1-2 GB of memory.

9.8 Crypto

In plain words: how fast the CPU encrypts (AES-128) and fingerprints (SHA-256) data, with and without its built-in crypto instructions (AES-NI, SHA-NI). Open: the **Benchmarks & Stress** page -> **Crypto...**. Command line: `sysreport-search crypto`. Controls: **Threads** (right-hand column; the left column is always one thread), **Seconds per measurement** (default 1; about 8 seconds in all), **Run** (asks first; with all threads - the default - every CPU thread is 100% busy, so **No** is the default answer). The official FIPS test vectors are checked before every run. On the command line it asks the same; `--yes` starts at once.

9.9 Prime Sieve

In plain words: counts the prime numbers up to 100 million, 1 billion, 10 billion or 100 billion and checks the count. Simple integer and cache work that scales with threads. Open: the **Benchmarks & Stress** page -> **Prime Sieve...**. Command line: `sysreport-search prime --limit 1e10`. Controls: **Count primes up to**, **Threads** (default all), **Run** (asks first; from 10 billion or with all threads - the defaults - it warns about unstable overclocks and **No** is the default answer), **Stop**. On the command line it asks the same; `--yes` starts at once.

9.10 Benchmark History

Every SRS Pi, Crypto, Prime Sieve, Cache & Memory, Storage, OS Latency, CPU Compute, Everyday Work, Network, GPU Benchmark, GPU Burnin and GPU Compute comparison run adds a line to `~/.cache/srs/bench-history.tsv` (time, test, settings, result, build). The **Benchmark History** tab on the Benchmarks & Stress page shows it, newest first. Compare only lines with the same settings and the same **Build**: a package built for any x86-64 CPU (`-march=x86-64 -O2`) can score well below a build made for your own CPU (`-march=native -O3`) - for example SRS Pi 1 million digits 3.1 s vs 2.1 s on a 7800X3D.

9.11 Cache & Memory

In plain words: how fast the RAM and the L1, L2 and L3 caches read, write and copy, and how long they take to answer. Use it after a BIOS, EXPO/XMP or memory-timing change. Open: the **Benchmarks & Stress** page -> **Cache & Memory...**, or Menu -> Benchmarks & Stress -> Cache & Memory... in the classic view. Command line: `sysreport-search cachemem`. Controls: **Threads** (default all; the bandwidth

results need all of them), **Seconds per test** (default 1), **Run** (asks first: every thread and the memory bus are busy for about 20 seconds), **Stop**. The status line names the value being measured. A stopped run shows the values it finished and says so.

Close other programs first and **never run another benchmark at the same time** (SRS lets only one run; it cannot stop other programs). The base clock is measured, so boards with a raised reference clock (for example 105 MHz) show the real value; the Infinity Fabric clock needs the `ryzen_smu` driver (Ryzen 7000 desktop); memory speed and type need root data (Tools > Collect root data). The engine is the one from Linux System Audit (LSA) 0.2.0, which is now part of SRS. x86-64 CPUs only.

9.12 Storage

In plain words: how fast the drive that holds a folder you choose reads and writes - big files, many small pieces at once, and one small piece at a time (how snappy it feels) - plus the response time of single reads and writes. It uses its **own temporary test file only**; your files are never touched, and nothing is left behind, even if SRS is stopped or crashes. Open: the **Benchmarks & Stress** page -> **Storage...**, or Menu -> Benchmarks & Stress -> Storage Benchmark... in the classic view. Command line: `sysreport-search storage --dir FOLDER`. Controls: **Folder** (the line below it names the drive, its connection, the file system and the free space - never the serial number), **Test file** (256 MB, 1 GB, 4 GB, 16 GB), **Seconds per test** (default 5), **Standard / Light** (Light uses at most 4 threads), **Allow USB drives** (off by default), **Run** (asks first), **Stop**.

Folders in RAM (`/tmp` on many systems is RAM, not a drive), on the network or on FUSE are refused, as is a drive with too little free space. Drive health (SMART) is on the **Storage** page.

9.13 OS Latency

In plain words: how quickly the operating system answers a program: how long a basic request to the kernel takes (a *system call*), how fast it switches between two programs (a *context switch*), how long it takes to start a thread or a program, how soon a sleeping program wakes up, how late a timer fires, and what a new page of memory or a small file costs. **Lower is better.** Steady numbers (95% and 99% close to the median) mean a smooth, responsive desktop. Open: the **Benchmarks & Stress** page -> **OS Latency...**, or Menu -> Benchmarks & Stress -> OS Latency... in the classic view. Command line: `sysreport-search os-latency`. Controls: **Samples** (1,000-200,000, default 20,000; the slower tests use a tenth or less), **Tick test (s)** (1-30, default 3: how long a 1 ms timer is watched), **Run** (asks first), **Stop**. It takes about 6 seconds at the defaults and keeps only **one or two CPUs** busy, never all of them.

What the numbers mean. Each row is one test, measured many times: **Median** is the typical time, **95%** and **99%** mean 19 in 20 and 99 in 100 tries were this fast or faster, **Worst** and **Best** are the extremes, and the sample count is shown. Times are in ns (billionths of a second), µs (millionths) and ms (thousandths). Below the table SRS names the **CPU scheduler** in use (for example scx_lavd under sched_ext, BORE or EEVDF), the preemption mode, the timer tick (CONFIG_HZ), the clock source, the timer slack, transparent huge pages, the CPUs used and the file system of the test folder (with its mount point; a mount inside your home folder is shown as "your home folder", so your user name is not in the result). Typical values on a current desktop: a system call 35-70 ns, a switch between two programs 1-2 µs, starting a thread about 10 µs, starting a program about 150 µs, a wake-up about 2 µs. Sleeps and the 1 ms tick run about 50 µs late, which is Linux's default timer slack. Hundreds of µs or missed ticks point to deep power-saving states, heavy background work or a busy scheduler.

Safety: it only reads system settings - no root, and no scheduler, priority or timer change. The file test works in its own temporary folder in `~/.cache/srs`, which is removed at the end, after Stop, and (if SRS was killed) on the next run. The result is saved to `~/.cache/srs/bench-os-latency.txt` and added to the Benchmark History. Tested on one PC only - see the note at the start of this chapter.

9.14 CPU Compute

In plain words: how fast the processor does floating-point number crunching - the kind of maths in 3D, video, science and AI programs. Six kinds of work: **matrix multiply** (SGEMM in single and DGEMM in double precision), **peak multiply-add** (the most sums the CPU can do), **FFT** (splitting a signal into frequencies), **N-body** (gravity between 1,024 stars), **Mandelbrot** (a fractal picture) and a small **ray tracer** (a picture with shadows and reflections). Each runs on one thread and then on all threads at once. **Higher is better.** Every answer is checked against a known result: a wrong answer is shown as **WRONG** in red instead of a score, which points to an unstable overclock or undervolt. Open: the **Benchmarks & Stress** page -> **CPU Compute...**, or Menu -> Benchmarks & Stress -> CPU Compute... in the classic view. Command line: `sysreport-search compute`. Controls: **Threads** (default all), **Seconds per test** (default 0.5), **Code** (the best instruction set the CPU has - SSE2, AVX2 or AVX-512 - or a chosen one), **1 thread only** (skips the all-threads column: light load, half the time), **Run** (asks first), **Stop**. It takes about 10 seconds at the defaults.

What the numbers mean. GFLOPS = billions of floating-point operations per second (matrix multiply, multiply-add, FFT, N-body); **Mpixels/s** = millions of picture points per second (Mandelbrot, ray tracer). The left column is one thread, the right column all the chosen threads; the **Code** column says which instruction set was used. FFT and the ray tracer are plain C. The numbers compare PCs and settings with each other; they are lower than a tuned maths library (BLAS) would reach.

Safety: with all threads, every CPU is 100% busy and gets hot, and AVX-512 draws a lot of power: **an unstable overclock or undervolt can crash the PC** under this load. Tick **1 thread only** (command line `--single`) to keep it light. It will not start while CPU Burnin or Stress runs. The result is saved to `~/.cache/srs/bench-compute.txt` and added to the Benchmark History. Tested on one PC only - see the note at the start of this chapter.

9.15 Everyday Work

In plain words: how fast the PC does the work it does all day: **compressing and unpacking files** (with the zlib, zstd, lz4 and xz libraries your system has), **sorting** numbers and words, **checksums and hashes** (CRC32, CRC32C, xxHash64, FNV-1a - the fingerprints used to spot damaged files), **searching and checking text** (finding words, checking JSON and UTF-8 text) and **running a small program in an interpreter** (like a script language does). Each test runs on one thread and on all threads. **Higher is better.** Every result is checked (unpacked data must equal the original, sorted lists must be in order and complete, checksums must match known values), so a wrong answer is reported instead of a fast number. Open: the **Benchmarks & Stress** page -> **Everyday Work...**, or Menu -> Benchmarks & Stress -> Everyday Work... in the classic view. Command line: `sysreport-search work`. Controls: **Threads** (default all), **Seconds per test** (default 0.5), **1 thread only**, the test groups (Compression, Sorting, Checksums and hashes, Text processing, Bytecode interpreter) and the compression libraries (zlib, zstd, lz4, xz) as tick boxes, **Run** (asks first, with the expected time - about half a minute at the defaults), **Stop**.

What the numbers mean. MB/s for compression (million bytes of original data per second, 1 MB = 1,000,000 bytes) with the **ratio** (2.5 = the data became 2.5 times smaller) on text-like, record-like and random data; **Mkeys/s** for sorting (million items per second); **GB/s** for checksums and text (billion bytes per second); **Mops/s** for the interpreter (million instructions per second). A library that is not installed shows "not installed"; the rest still runs. The CRC32C row that uses the CPU's own CRC32 instruction (SSE4.2) is built into the 64-bit x86 version only; the 32-bit (i686) and ARM (aarch64) versions show it as "not built into this image", and a CPU without SSE4.2 as "the CPU has no CRC32C instruction".

Safety: the test data is generated by SRS from fixed seeds (every run and every PC does the same work); none of your files are read or written, only the result is saved. With all threads every CPU is 100% busy, and xz -6 and zstd -19 use about 100 MB of memory per thread (fewer threads are used, with a note, when memory is short). **Stop** is checked between passes: with the largest part size (`--part-kib 16384`) one xz -6 pass can take 10-20 seconds, so "Stopping..." may show for that long (at the default size it is under a second). The result is saved to `~/.cache/srs/bench-work.txt`, and each test group adds a line to the Benchmark History. Tested on one PC only - see the note at the start of this chapter.

9.16 Network

In plain words: two separate tests. **On this computer** sends data between two programs on your own PC through the loopback connection (127.0.0.1): it shows how fast Linux and the CPU can move network data, and how long a message takes to go there and back. Nothing leaves the computer, and the network card is not involved. **Internet speed test** (optional) measures your real connection with Cloudflare's public speed test server (speed.cloudflare.com): how long a name lookup takes (DNS), how long it takes to connect, and your download and upload speed with one and with several connections at once. Open: the **Benchmarks & Stress** page -> **Network...**, or Menu -> Benchmarks & Stress -> Network Benchmark... in the classic view. Command line: `sysreport-search net` (loopback) and `sysreport-search net --internet`. Controls: **On this computer (loopback)** (ticked), **Internet speed test (asks first)** (not ticked), **Streams** (connections at once for the "N streams" tests, default 4), **Seconds per test** (default 5), **IPv4 or IPv6 / IPv4 only / IPv6 only** (internet test), **Run**, **Stop**. The line under the controls names the connection (wired, Wi-Fi, VPN ...), its link speed and whether libcurl is installed - never an address.

Asking first. Run asks before the loopback test (about 20 seconds, a few CPUs busy). The internet test **asks separately**, with No as the default answer, and says which server it contacts, what it measures, how long it takes and **how much data it may use - up to about 1000 MB** with the standard settings (each of the four speed tests stops after 5 seconds or 250 MB; about 250 MB on a 100 Mbit/s line). **Do not run it on a metered or mobile connection.** The server sees your public IP address, as with any web page; SRS does not show or save it. It never runs without that Yes. The speed rows need libcurl (installed on almost every Linux system); without it only the lookup and connect times are measured.

What the numbers mean. Mbit/s = million bits per second (internet speeds are sold in Mbit/s; divide by 8 for MB/s). **Average** counts from the first byte received; **peak** is the best half second. **ms** = thousandths of a second, **µs** = millionths. **Median** = the middle value, **99% under** = all but the slowest 1%, **jitter** = how much the time changes from one try to the next, **lost** = UDP packets that never arrived. Internet results vary with the time of day, Wi-Fi, other traffic and the server.

Safety and privacy: no IP address, MAC address, Wi-Fi network name or host name is shown or saved; only the IP version used. Extra hosts for the connect-time test (`--hosts` on the command line) are shown by name; one typed as a numeric address is shown as "host N (a numeric address)", so it never

lands in the saved result. A speed test server given with a port (`https://host:8443`) is timed on that port too. The result is saved to `~/.cache/srs/bench-net.txt` and added to the Benchmark History. Tested on one PC on a wired connection only - see the note at the start of this chapter.

9.17 GPU Benchmark

In plain words: how fast your graphics card (GPU) does sums and moves data. It runs short pieces of work on the GPU through **Vulkan** (the graphics and compute interface that every modern Linux GPU driver offers - the same one many games use) and checks every answer, so a wrong answer is reported instead of a fast number. **Higher is better**, except for the latency row, where lower is better. Open: the **Benchmarks & Stress** page -> **GPU Benchmark...**, or Menu -> Benchmarks & Stress -> GPU Benchmark... in the classic view. Command line: `sysreport-search gpu`. Controls: **GPU** (Default = the first discrete graphics card; open the list to pick another - SRS reads the list only when you open it: like `vulkaninfo`, opening the list names your GPUs through Vulkan, and no work is sent to a GPU until you answer Yes), **Seconds per test** (default 1), **Memory test size** (0 = auto, 256 MiB on most cards; a bigger size is lowered to a quarter of the video memory and of the PC's memory, and the result says so), **Run** (asks first), **Stop**. The line under the controls says whether Vulkan is installed.

What it measures.

- **FP32, FP16, FP64 multiply-add** - floating-point maths (numbers with a decimal point) at three precisions: FP32 (single precision) is what games and most GPU programs use, FP16 (half precision) is used by AI and some effects, FP64 (double precision) by science programs. In **GFLOPS** - billions of operations per second. Most gaming cards do FP64 at only a small fraction of their FP32 speed; that is normal. A card or driver without FP16 or FP64 says "not supported".
- **INT32 multiply-add** - whole-number maths, in **GIOPS** (billions per second).
- **Video memory read, write and copy** - how fast the GPU uses its own memory, in **GB/s** (1 GB = 1,000,000,000 bytes).
- **Upload and Download** - how fast data moves between the system memory and the graphics card, over the PCIe slot, in GB/s.
- **Dispatch latency** - how long the GPU takes to answer an empty piece of work, in **µs** (millionths of a second); lower is better.

On a GPU built into the processor (integrated graphics) the memory rows measure the system RAM it shares, and the page says so.

Reading the result. Compare runs on the same PC, with alike SRS builds and the same graphics driver version: a driver update alone can change the numbers, and the page says which driver ran. Much lower numbers than last time usually mean heat, a power-saving setting or another program using the GPU. Upload / download well below what the slot should give can mean the card sits in a slot with fewer lanes or its link has dropped to a slower speed. A **WRONG RESULT** means the GPU computed something wrong - usually too much overclock or undervolt, too much heat, or a faulty card; run GPU Burnin to check.

Safety: Run **asks first**, naming the GPU and the time (about 15 seconds at the defaults), and says that the GPU works at full load and that an unstable GPU overclock or undervolt can crash the PC or the display driver under that load. The work is cut into pieces of about 25 ms, one at a time, so the desktop stays usable; a GPU that stops answering ends the run cleanly with the results so far; Stop ends it at once. SRS never changes clocks, voltages or fans, and keeps Vulkan overlays such as MangoHud out of the run. Vulkan is loaded when needed: without it the window says "not available" and the rest of SRS

works as before. The result is saved to `~/.cache/srs/bench-gpu.txt` and added to the Benchmark History. Tested on a software GPU only - see the note at the start of this chapter.

9.18 GPU Burnin

In plain words: a stress test for the graphics card, like the CPU part of Stress & Burnin is for the processor. It keeps the GPU as busy as it can be with large matrix multiplications (big blocks of sums) for as long as you choose, and checks every single result. A healthy card gets the same answer every time; a card that is pushed too far (overclocked, undervolted, too hot, short of power) sooner or later gets one wrong. **0 errors = PASS.** While it runs you see the temperatures, power, clock and fan speed live. Open: since 2.3.1 GPU Burnin is part of **Stress & Burnin** (9.1): tick **GPU Burnin (built-in)**, untick **CPU load** to test only the GPU, and set its time or **Sync to CPU stress time**; the old "GPU Burnin" entries open it that way. Command line (unchanged): `sysreport-search gpu-burn`. Controls (under **GPU options** in Stress & Burnin): **Back end** (see below; Auto = Vulkan), **GPU** (the first GPU, All GPUs, or one from the list; the list always comes from Vulkan, like `vulkaninfo`: it names your GPUs and sends them no work. With ROCm / HIP, CUDA or OpenCL as the back end, SRS starts that runtime only after you answer Yes and then finds the GPU you picked in its own list by its PCI address), **Precision** (FP32, or FP64 for the double-precision units), **Seconds** (default 60; 600-1800 s, 10-30 minutes, checks an overclock or undervolt properly), **Stop at °C** (default 100; up to 110 after the one-time question in 9.1), **Run** (asks first), **Stop**. The line under the controls says which back ends are installed.

The four back ends, in plain words. A program can talk to a GPU in several "languages". SRS can use four of them, and you choose which:

- **Vulkan** - the modern graphics and compute interface that every current GPU driver on Linux speaks (AMD, Intel and NVIDIA). Nothing extra to install. The default, and the right choice for most people.
- **ROCm / HIP** - AMD's own compute platform, used by AI and science programs on AMD cards. Only there if you installed ROCm.
- **CUDA** - NVIDIA's compute platform, used by most AI programs. Needs the NVIDIA driver. If your "CUDA" is really **ZLUDA** (a translation layer that runs CUDA programs on an AMD card), SRS notices and calls it **CUDA (ZLUDA)**. SRS's CUDA tests are built in, so the NVIDIA CUDA toolkit is not needed (9.20).
- **OpenCL** - the older compute interface that works across makers. Every OpenCL driver you have (ROCm, Mesa Rusticl, NVIDIA, PoCL on the CPU ...) shows as its own choice.

A harder load since 2.4. GPU Burnin now uses a faster, "register-blocked" matrix multiply on all four back ends: each thread works out 16 results at once from data kept in its own registers, which keeps the card's arithmetic units much busier. On the author's Radeon RX 9070 XT (FP32, 0 errors): CUDA via ZLUDA 5.3 -> 17.2 TFLOPS, Vulkan 6.2 -> 16.2, OpenCL 3.4 -> 13.4, ROCm / HIP 3.4 -> 13.0 - the card draws more power and gets hotter, as a burn-in should make it. The summing order is unchanged, so every result is still checked the same way. The result's **Kernel** line says which matrix multiply ran; if the new one cannot be used, the older one runs (on HIP then hipBLAS). `SRS_GPUBURN_SIMPLE=1` forces the older one for comparing. The GPU Compute comparison (9.19) keeps the older kernel, so its numbers stay comparable with 2.3.

A back end that is not installed says "not installed" with a hint about what to install; the others still work (the window checks which are installed in the background when it opens, since ROCm's libraries are large). Testing through more than one back end tells you whether a problem is the card or one particular

driver. A software renderer such as **llvmpipe** (Mesa's Vulkan driver that runs on the CPU, installed on many desktops) is never burned as if it were a GPU: Auto, The first GPU and All GPUs skip it, and picking it says it is not a GPU. On the command line, `sysreport-search gpu-burn --list` lists the devices of every back end - that starts each runtime (ROCm / HIP, CUDA, OpenCL) but runs no work on a GPU; `--list --backend vulkan` reads only Vulkan's list.

Reading the result. PASS = no wrong result in the whole run. **FAIL** = at least one wrong result: errors usually mean too much overclock or undervolt, too high a temperature, a weak power supply, or a faulty card or video memory - undo the overclock or undervolt and run it again. **Ended early** = a safety rule stopped it (see below); the page says which. **Stopped** = you pressed Stop (no errors so far). The page also shows the highest edge, hotspot / junction and memory temperatures, power, clock and fan speed, the average speed in GFLOPS, the number of multiplications, the matrix size, the back end and the driver version. "not available" means the card or driver does not report that sensor (for example NVIDIA's own driver gives Linux no temperature). When the card reports **no temperature at all**, the temperature limit cannot work: the live readout says "no temperature sensor: limit not active" and the result says the same, so watch the card yourself (or use a back end and driver that report it).

Safety - what stops it by itself. Run asks first, and **No** is the default answer. The question says that the GPU will draw a lot of power and get hot, that a bad overclock or undervolt can crash the PC or the display driver, and that a weak power supply or poor cooling can make the PC shut down. GPU Burnin then ends by itself when:

- the GPU reaches the **temperature limit** (default 100 °C, 50-110; the hotspot / junction temperature when the card reports one, else the edge temperature);
- the driver reports **any error**, or the GPU **stops answering** (device lost);
- one piece of work takes **more than 2 seconds** (each should take about 20 ms);
- you press **Stop**.

If a GPU driver call never comes back at all (a hung driver), SRS waits 15 seconds after the end, then ends the run with "the GPU driver did not come back" and leaves the GPU alone; restart SRS (or the PC) before the next GPU test; until then the next GPU test, and CPU Burnin, say so ("a GPU call did not return earlier; restart SRS"). With **All GPUs**, a card that cannot start (for example it cannot do FP64) is marked **COULD NOT START** in the table and the others go on; the verdict (PASS / FAIL) comes from the cards that ran, with a note saying how many could not start (if none could start, the run says that); a bigger matrix that does not fit in a small card's memory just keeps the size that worked.

SRS never changes clocks, voltages or fans - it only reads them. GPU Burnin does not start while Stress & Burnin or another benchmark runs, and Stress & Burnin does not start until it has finished (its status line says "not started" and why). The result is saved to `~/.cache/srs/bench-gpuburn.txt` and added to the Benchmark History. The idea of comparing every result with the first one comes from gpu-burn by Ville Timonen; the code is SRS's own. Tested on the author's real card with ROCm / HIP and OpenCL, and with Vulkan and CUDA only on a software GPU and a fake driver - see the note at the start of this chapter.

9.19 GPU Compute: Vulkan vs HIP vs CUDA vs OpenCL

In plain words: the same GPU work, run through every way of programming your graphics card that you have installed (the four back ends explained in 9.18), on the same card, and shown side by side. It tells you which one is fastest on your PC, and how much faster the maker's own tuned library is than a plain program. Open: the **Benchmarks & Stress** page -> **GPU Comparison...**, or Menu -> Benchmarks & Stress -> GPU Comparison (Vulkan, HIP, CUDA, OpenCL)... in the classic view. Command line:

`sysreport-search gpu-compare`. Controls: **GPU** (the card, picked by its Vulkan device; the other back ends find the same card by its PCI address), a tick box per back end (only installed ones can be ticked; the tooltip of a greyed one says what to install), **Seconds per test** (default 1), **Run** (asks first), **Stop**.

Reading the table. Each row is a test, each column a back end:

- **Own kernel** rows: SRS's own plain matrix multiplication - the same work on every back end, so these rows compare the back ends fairly.
- **Vendor library** rows: the maker's tuned library - hipBLAS for ROCm, cuBLAS for CUDA, CLBlast for OpenCL. Vulkan has none, so it says "not available". These rows show what real programs that use the libraries can get.
- **Multiply-add chain**, **video memory copy**, **upload / download** and **kernel launch round trip** (lower is better) work as in the GPU Benchmark.
- `*` marks the fastest in each row. **(85%)** after a number means that column reached 85% of Vulkan's speed in that row (above 100% = faster than Vulkan).
- **CUDA (ZLUDA)**: your CUDA library is really ZLUDA, which runs CUDA programs on an AMD card. Since 2.3 SRS's CUDA kernels are **built in** (pre-compiled as PTX), so every CUDA row runs without the NVIDIA runtime compiler (NVRTC), which ZLUDA does not have. The CUDA column's **software** line says "own kernels built in (PTX)". Only when the graphics driver refuses the built-in kernels do the CUDA rows say "not available" and point to Options -> NVIDIA CUDA toolkit (then please report your driver or ZLUDA version). ZLUDA's cuBLAS has no FP64 matrix multiply, so with ZLUDA the **FP64 matrix multiply, vendor library** row says "not available: cuBLAS on this driver does not support FP64 matrix multiply" - expected, not a fault. On the author's Radeon RX 9070 XT with ZLUDA every other CUDA row ran (2026-10-29: own-kernel FP32 matrix multiply 5377 GFLOPS, cuBLAS FP32 16392, cuBLAS FP16 77046, own-kernel FP64 860 GFLOPS).
- **ROCm / HIP** still needs its runtime compiler (hipRTC, part of ROCm) for its own-kernel rows and, for the error check, its library rows; without it they say so and are not run.
- **FP64 on OpenCL**: some OpenCL drivers (often Mesa's Rusticl) have no double precision. The FP64 rows then say "not available: this OpenCL driver has no FP64 (double precision)".
- Every **OpenCL driver** is its own column, for example "OpenCL (ROCm)" and "OpenCL (Rusticl)" on the same card. Drivers that run on the CPU (PoCL) are marked CPU.
- **not installed** means that back end is not on this PC; the line under the table says what to install. The other columns still run.
- **not run** means that test did not run because the GPU hung or stopped answering earlier in the run: after that, SRS does not touch the GPU again through any back end (a GPU that is having trouble should be left alone), and the line under the table says what happened.

Compare results only between alike SRS builds and the same driver and runtime versions; the page lists each column's version.

Safety: Run asks first, naming the back ends, the GPU and the time (about 20 seconds per back end at the defaults) and that the GPU works at full load; an unstable GPU overclock or undervolt can crash the PC under that load. ROCm / HIP, CUDA and OpenCL start only after your Yes. The work is cut into short pieces, a GPU that fails ends its column cleanly, Stop ends the run at once, and SRS never changes clocks, voltages or fans. The result is saved to `~/.cache/srs/bench-gpucompare.txt` and added to the Benchmark History. HIP, CUDA on NVIDIA and OpenCL were not run on real hardware by the author (CUDA through ZLUDA was, on one card) - see the note at the start of this chapter.

9.20 The NVIDIA CUDA toolkit (optional)

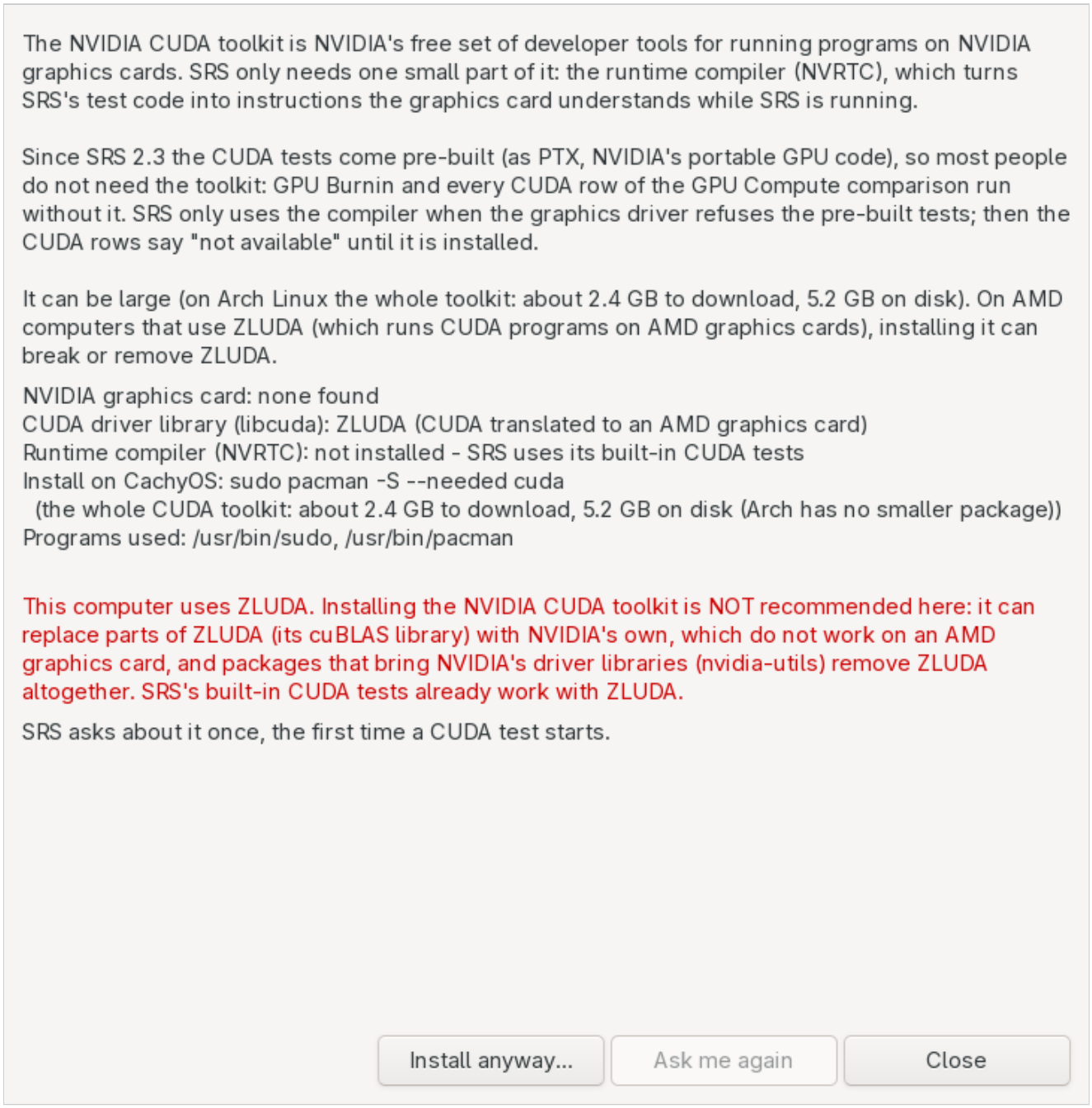
In plain words: the NVIDIA CUDA toolkit is NVIDIA's free set of developer tools for running programs on NVIDIA graphics cards. SRS only needs one small part of it: the **runtime compiler (NVRTC)**, which turns SRS's test code into instructions the graphics card understands while SRS is running. Since SRS 2.3 the CUDA tests come **pre-built**, so most people do not need the toolkit: GPU Burnin and every CUDA row of the GPU Compute comparison run without it. SRS only uses the compiler when the graphics driver refuses the pre-built tests. The toolkit can be large (on Arch Linux about 2.4 GB to download and 5.2 GB on disk), and on AMD computers that use ZLUDA, installing it can break or remove ZLUDA.

When SRS asks. The first time you press **Run** in GPU Burnin (back end CUDA) or the GPU Compute comparison (CUDA ticked) - and only when it matters: there is an NVIDIA card or a CUDA library (also ZLUDA) and NVRTC is not installed. The question explains the above, shows what was found, the exact install command and its size, and has three buttons:

- **Install...** - see "Installing" below. The question still counts as asked: if you then cancel, SRS does not ask again until it is restarted.
- **Not now** - not asked again until SRS is restarted. The run goes on with the built-in tests.
- **Don't ask again** - never asked again (stored as `Ask CUDA Toolkit=0` in SRSDEF.ini).

After **Not now** or **Don't ask again** SRS says once: *You can install it later from Options > NVIDIA CUDA toolkit for GPU tests...*. With ZLUDA the question says the toolkit is **not recommended**, shows the warning in red, and the button reads **Install anyway...**.

Options -> NVIDIA CUDA toolkit for GPU tests... shows the same explanation and the state: NVIDIA card found or not, the CUDA library (NVIDIA's, ZLUDA, or none), NVRTC installed (with its version and file) or not, and the install command for your Linux. Buttons: **Install...** (greyed when NVRTC is already there or your Linux has no command), **Ask me again** (undoes "Don't ask again") and **Close**. After an install the window checks again at once; no restart is needed.



Installing. Nothing is installed without your click, and SRS shows the exact command first:

Linux	Command SRS offers	Size
Arch Linux and Arch-based (CachyOS, EndeavourOS, Manjaro ...)	`sudo pacman -S --needed cuda` (the whole toolkit; Arch has no smaller package)	about 2.4 GB download, 5.2 GB on disk
Debian 13, Ubuntu 24.04	`sudo apt-get install libnvrtc12` (only the compiler library)	about 20 MB download, 70 MB on disk
Debian 12, Ubuntu 22.04	`sudo apt-get install libnvrtc11.2`	similar
Debian / Ubuntu without those	`nvidia-cuda-toolkit` if apt knows it; otherwise instructions: turn on Debian's contrib / non-free or Ubuntu's multiverse, then `sudo apt-get update`	several GB

Linux	Command SRS offers	Size
Fedora, openSUSE, others	no command: instructions to add NVIDIA's CUDA repository from https://developer.nvidia.com/cuda-downloads and install only its `cuda-nvrtc` package	

The Debian package names come from Debian's package archive; the Ubuntu ones are inferred from the CUDA version of Ubuntu's `nvidia-cuda-toolkit` source and were not looked up one by one. That is why SRS first asks apt (`apt-cache show`, read-only) which of them exists, and offers only that one.

SRS opens a **terminal window** (Konsole, GNOME Terminal, Xfce Terminal, kitty, Alacritty, foot, xterm ...) and runs the command there with `sudo`, so you type your password and answer the package manager's questions yourself. Without a terminal program it uses `pkexec` (the system's password window) and the package manager's "no questions" switch. Then nobody answers the questions, so they get their default answers: with pacman's `--noconfirm` that is yes to "Proceed with installation?", the first package where it offers a choice, yes to "Replace X with Y?" and **no** to "remove the conflicting package?" - so a conflict stops the install and nothing is removed without you; apt-get's `-y` answers yes (apt still stops for a held or essential package). With ZLUDA SRS never uses `pkexec`: it needs a terminal window. SRS waits for the result (**Stop waiting** lets you go on), then says whether it worked and checks for NVRTC again.

Only system programs. SRS runs the package manager, `sudo` and `pkexec` only from `/usr/bin`, `/usr/sbin`, `/bin` or `/sbin`, by their full path (the confirmation shows them as *Programs used*). A program of the same name anywhere else - for example in `~/.local/bin`, which is often first in `PATH`, or the current folder - is never used: it could be anything, and it would run as administrator after your password. If they are not in those folders, SRS does not start the install and shows the command to type yourself.

It refuses, with the reason, when: a benchmark or burn-in is running; the package database is locked by another update (pacman's `db.lck`); the computer has no network connection; there is no `sudo` or `pkexec` in a system folder to ask for the password. Afterwards it explains a cancelled password prompt, a refused one, or an error of the package manager (no internet, a locked database, a conflict with an installed package, a "no" answer).

ZLUDA (AMD cards). Installing NVIDIA's toolkit is **not recommended** with ZLUDA: on Arch the `cuda` package puts NVIDIA's own cuBLAS on the library path, where it can take the place of ZLUDA's (and NVIDIA's cuBLAS does not work on an AMD card), and anything that brings NVIDIA's driver libraries (`nvidia-utils`) removes ZLUDA, which conflicts with them. SRS's built-in CUDA tests already work with ZLUDA. If you still go on, SRS asks twice (Cancel is the default) and always uses a terminal window, so you can answer **no** if the package manager offers to remove ZLUDA.

Command line: `sysreport-search cuda-toolkit` shows the same explanation, state and command; `--install` runs it after you type `yes` (in a terminal only; twice with ZLUDA); `--never-ask` / `--ask-again` set the "Don't ask again" choice. The command line never asks on its own: after a CUDA run without NVRTC it prints a one-line note with the menu path and the command (not after "Don't ask again").

GPU words used in 9.17-9.20.

- **GPU** - graphics processing unit: the graphics card, or the graphics part of the processor.
- **Back end** - the "language" a program uses to talk to the GPU: Vulkan, ROCm / HIP, CUDA or OpenCL.

- **Kernel** - a small program that runs on the GPU, many copies at once.
- **Matrix multiplication (GEMM)** - multiplying two big tables of numbers; the core of AI and many science programs, and a hard, steady load for a GPU.
- **FP32 / FP16 / FP64** - numbers with a decimal point stored in 32, 16 or 64 bits (single, half, double precision). **INT32** - 32-bit whole numbers.
- **GFLOPS** - billions of floating-point operations per second; a multiply-add counts as two.
- **Hotspot / junction temperature** - the hottest point measured on the GPU chip, usually 10-20 °C above the **edge** temperature.
- **ZLUDA** - a layer that lets CUDA programs run on AMD graphics cards.
- **CUDA toolkit** - NVIDIA's free developer tools for NVIDIA graphics cards. SRS can use one part of it, NVRTC; optional since 2.3 (9.20).
- **NVRTC** - NVIDIA's runtime compiler: it turns GPU program text into GPU code while a program runs. SRS used it for its CUDA tests before 2.3; now only as a fallback.
- **PTX** - NVIDIA's portable GPU code. SRS 2.3 carries its CUDA tests as PTX, and the graphics driver (or ZLUDA) finishes it for the card when the test starts.
- **Lavapipe** - Mesa's software Vulkan driver that runs on the CPU; used to test SRS without a real GPU.
- **Device lost** - the GPU stopped answering and the driver reset it.

9.21 Update hardware lists

In plain words: SRS turns ID numbers into names ("1002:7550" into "Radeon RX 9070 XT") with three lists that come with Linux: `pci.ids` (PCI devices), `usb.ids` (USB devices) and `pnp.ids` (monitor and PnP device makers). A very new device may be missing from your distribution's copy and show as "unknown". **Update hardware lists** fetches the newest lists from their official sources, only when you ask. New in 2.4.

Open: the **Tools** page -> **Update hardware lists** -> **Update...**; command line `sysreport-search update-lists` (`--list` shows the copies in use, `--yes` downloads without asking).

- The question shows each list, the version in use now and where it comes from: `pci.ids` from the PCI ID Repository (pci-ids.ucw.cz), `usb.ids` from the Linux USB ID list (linux-usb.org), `pnp.ids` from hwdata. **OK** downloads them in the background (with `curl`, or `wget`); a message then says how each one went.
- They go to `~/.local/share/srs/lists` - your own folder, no root, nothing in the system changes.
- Each download is checked first (it must start like the real list and have enough entries); a failed or odd download changes nothing, and the copy in use stays. The previous copy is kept as `.old`.
- From then on SRS uses the **newer** of your copy and the system's (hwdata) copy, by the list's own version (by the number of entries for `pnp.ids`, which has no version). So a later hwdata update that is newer still is used automatically. Pages show the new names the next time they are opened or refreshed; the Lookup item shows which file is in use.
- SRS never downloads these by itself.

9.22 ACPI method evaluation (experts)

In plain words: your BIOS contains small programs, ACPI "methods", that the operating system calls to switch devices on and off, read their status, set fans or power modes and so on. This tool runs **one**

method you type and shows what it returned - useful to experts who debug a BIOS or a laptop. **Unlike the rest of SRS it is not just reading: a method can change hardware state.** New in 2.4.

Open: the **Tools** page -> **ACPI method evaluation** -> **Evaluate...**, or **Menu -> ACPI method evaluation (experts)...**. It needs the `acpi_call` kernel module (Arch: `acpi_call-dkms`) and the root password for every call.

- **The first time** a warning explains the risk: the method runs as root, through `acpi_call`, and can switch devices on or off, change fan, power or performance settings, write to the BIOS settings, or make the PC hang or reboot. Evaluate only methods you know, for example from the ACPI methods item (`acpi-methods`) or your board's documentation. The authors of SRS are not responsible for any damage or data loss (GNU GPL, no warranty). **Yes, I understand** turns the tool on and is remembered (`Accepted ACPI Evaluation=1`); you are asked only once.
- **Method**: an ACPI name path, e.g. `\_SB.PCI0.GPP0._STA`. **Arguments**: none, or up to 7 of: numbers (`1`, `0x1`), simple `"strings"` (letters, digits, space, `. _ -`) and buffers written `b` plus hex bytes (`b0102`).
- **Evaluate** asks for the root password and runs `srs-root --acpi-eval`, which checks the method name and every argument again as root (anything else is refused and nothing runs), loads `acpi_call`, evaluates the method once and shows the **Result** as `acpi_call` returns it.
- **Every call is logged** with the time, the call and the result in `~/.cache/srs/root/acpi-calls.txt`. The **ACPI method evaluation** item (`acpi-eval`; type its name in the search box) shows whether `acpi_call` is loaded and the last 30 calls.

A safe first try is a status method such as a device's `_STA` (it only reports whether the device is present and on). If you are not sure what a method does, do not run it.

10. Options

Menu -> Options (in the main window's menu, the classic Menu and the tray's SRS Menu). All are saved in `~/.config/srs/SRSDEF.ini` and apply at once to every open window.

Option	What it does	Default
Theme -> Light (SIV classic)	SIV's light look and colours.	classic view
Theme -> Dark	Dark panels, SIV's colours brightened so they stay readable, dark graphs.	
Theme -> Follow desktop	Uses your desktop's light/dark setting (via xdg-desktop-portal, set by KDE Plasma and GNOME) and switches live when you change it. Without a portal, it reads `GTK_THEME` or `~/.config/gtk-3.0/settings.ini`.	main window
Update interval	How often live panels refresh: **Default** (each panel's own rate; the classic main window every 2 s; the Overview checks and the key-fact tiles always every 3 s), 0.5, 1, 2, 3, 5 or 10 seconds, or **Paused** (press F5 to refresh by hand). Stress & Burnin always updates every second.	Default
System tray icon	Show the tray icon.	on

Option	What it does	Default
Close button hides to tray	Closing the main window hides it instead of exiting. The first time, a notice says SRS is still running in the tray and how to quit (right-click the tray icon > Exit, or Menu > Quit / Ctrl+Q), with **Don't show this again**.	off
Minimize hides to tray (X11 / XWayland)	Minimising hides it to the tray. Native Wayland apps are not told when they are minimised, so this works only on X11 / XWayland.	off
Start hidden in tray	Start with no window, just the tray icon.	off
Start with the desktop (autostart)	Writes `~/.config/autostart/srs-gui.desktop`, which runs `srs-gui --gui-tray` at login. Untick to remove it.	off
Refresh root data at boot	See 5.3.	off
Redact identifiers in saved reports and copied text	Save Report / Save All Items, the **Copy** button and right-click **Copy** / Ctrl+C mask serial numbers, UUIDs, MAC addresses (vendor half kept), boot / machine IDs and the host name - see 17. The panels still show everything.	off
Start in classic view	Start SRS in the SIV-style main window instead of the window with the sidebar (6.6).	off
Sortable tables in the main window	Click a column heading to sort a table (6.8). Off: tables are plain text as in 2.0.	on
Tab bar	**One row that scrolls**, **Wrapped rows** or **Side list** (6.7). Main window only, like the next five.	One row
Lineup	Basic, Enthusiast, Expert or your own; save, export, import (6.7).	Expert
Tabs on each page...	Per page: the standard tabs, the ticked ones, or every item.	standard
Show every item as a tab (all pages)	Every item that can show something here gets a tab on its page.	off
Show hidden items	List the items that cannot show anything on this PC (Windows-only, not present, empty), greyed, in All N items, and do not hide their tabs.	off
Check daily which items have nothing to show	On: once a day, in the background, SRS reads every item to find the ones with nothing to show on this PC and hides them. Off: only when you choose **Check now**. Windows-only items and hardware that is not present are hidden either way. The network items and Ping are never part of the check.	off
Check now which items have nothing to show	Does that check once, now, in the background.	
NVIDIA CUDA toolkit for GPU tests...	What the optional NVIDIA CUDA toolkit is for, whether it (its runtime compiler, NVRTC) is installed, the install command for your Linux, **Install...** and **Ask me again** - see 9.20.	
Read DIMM voltages from their PMIC chips (SMBus; from the next Run as root)	Run as root also reads the set voltages of each DDR5 module's power chip for Memory Timings. Asks first; see 5.4 (2.4).	off

Option	What it does	Default
Read the embedded controller (expert, read-only; from the next Run as root)	Run as root also copies the embedded controller's 256 registers (ACPI Embedded Controller item). Asks first; see 5.4 (2.4).	off
Load test terms (warranty)...	The "use at your own risk" terms of the stress tests and benchmarks: read them again, agree, or withdraw your agreement - see 9.0.	
Hide to Tray	Hide all SRS windows now.	

Until you pick a Theme, the main window follows the desktop and the classic view starts light.

The tray options are greyed out if the tray library is not installed (the menu then says "(tray needs libayatana-appindicator)").

11. System tray

Like SIV's Icon Tray, SRS can live in the system tray (needs `libayatana-appindicator`; works on KDE Plasma, XFCE, and GNOME with the AppIndicator extension).

- **Tooltip:** CPU temperature and load, GPU temperature and load, updated every 2 seconds.
- **Middle-click:** show / hide SRS.
- **Tray menu:** Show SRS / Hide to Tray, Sensors, Processors, Stress & Burnin (CPU / GPU)..., Benchmark..., Find..., **SRS Menu** (the full menu), Exit. Exit really ends SRS (it asks first while a stress test or benchmark runs). Sensors and Processors open their page in the main window (or their panel in the classic view).
- `srs-gui --gui-tray` starts hidden in the tray.
- Safety: SRS only hides its windows if a tray icon really exists, so it cannot get lost.

12. Keyboard shortcuts

In the main window:

Keys	Action
Alt+1 ... **Alt+9**, **Alt+0**, **Alt+-**, **Alt+=**, **Alt+Backspace**	Go to page 1 to 13, in sidebar order: Overview (Alt+1) ... Sensors (Alt+8), Power & Cost (Alt+9), USB & PCI devices (Alt+0), Operating system (Alt+-), Benchmarks & Stress (Alt+=), Tools (Alt+Backspace)
Ctrl+F	Search box (6.4)
Enter in the search box	Find the text on the current tab
F3 / Shift+F3	Next / previous match on the tab
Esc in the search box	Clear it and the highlights
Ctrl+Shift+F	Find in System Report
Ctrl+H	Highlight matches on / off (Find)
Ctrl+L	Device ID Lookup

Keys	Action
Ctrl+D	ALL Devices tab
Ctrl+S	Save Report
Ctrl+M	Open the menu
Ctrl+T	Add a tab to this page (the **+** button)
Ctrl+W	Close the tab in view (also a middle-click on a tab)
Ctrl+PageUp / Ctrl+PageDown	Previous / next tab, in every tab-bar layout
Mouse wheel over a side tab list	Tabs in a side list (Layout: side): the tab above / below, one notch per tab, stopping at the first and last tab (no wrap-around). Over the page itself the wheel scrolls the page; with tabs on top it does not change tabs. It never starts or stops a test: leaving a running tool's tab still asks first (2.3.1)
Up / Down, Home / End in a tab list or list	GTK's own keys, unchanged: step through the list, first / last. Inside a tool's controls (a spin button, a list box) the arrows change that control, not the tab
Page Up / Page Down in a tab list	10 tabs back / on, stopping at the first / last tab (in a list: a page up / down)
Right-click a tab	Keep, Replace with..., Move, Close, Tabs on this page, Lineup
Click a table heading	Sort by that column (again: the other way)
F5	Read the page again
F1	User guide at the current tab's item (the Overview opens the start)
Shift+F1	Help item
Ctrl+Q	Quit (Menu > Quit). It really ends SRS, also with Close to tray on; a stress test or benchmark still running asks first
Alt + underlined letter	Press that button
Double-click a PCI or USB ID	Look it up

In the classic view:

Keys	Action
Ctrl+M	Open the Menu
Ctrl+F	Find
Ctrl+L	Device ID Lookup
Ctrl+D	ALL Dev (every device)
Ctrl+S	Save Report
Ctrl+H	Highlight matches on / off (Find)
F3 / Shift+F3	Next / previous Find match
F5	Refresh this panel now
F1	Open the user guide at this panel's section
Shift+F1	Help panel

Keys	Action
Esc	Close this panel (not the main window)
Alt + underlined letter	Press that button
Double-click a PCI or USB ID	Look it up (USB IDs from USB items, `VID_xxxx PID_xxxx` and HID names such as `0003:046D:C52B.0001` open a USB lookup)
Right-click panel text	Copy / Select All, plus the SRS Menu

13. Reports and exports

13.1 Save Report and Save All Items (SIV-format text)

- **Menu -> Save Report...** (or **Ctrl+S**) saves SIV's default set of items as text. Default file name: `srs-<hostname>.txt` (`srs-report.txt` when **Options -> Redact identifiers in saved reports** is on).
- **Menu -> Save All Items...** saves every one of the 558 items. Default name: `srs-<hostname>-all.txt`. On the author's PC this is about 52,000 lines and takes about 12 seconds.

The same from the command line (CLI-REFERENCE.md):

```
sysreport-search -save                          # default items to the screen
sysreport-search -save=report.txt               # ... to a file
sysreport-search '-save=[cpuid][sensors]'       # chosen items
sysreport-search '-save=[all]=everything.txt'  # every item to a file
sysreport-search '-save=[all]=share.txt' --redact   # every item, identifiers
masked
```

File layout: a header line with program, version, host, date and the item list, then each item starting with a line `[id] [category]`:

```
srs-gui - System Report Search V2.4.0 MYPC 2026-10-26 08:18:54  [[cache][sensors]]

[cache]  [hardware]

srs-gui - Cache Levels of CPU-0 on \\MYPC - CachyOS 7.3.0-273-tkg-bore-llvm

Cache                Size      Ways      Line        Sets       Shared by
CPUs
L1 Data Cache        32K       8         64          64         2             0,8
...
```

Saving a report **never** starts the burner or a benchmark; benchmark items show their last stored result.

13.2 Copy

Copy on any window puts its whole text on the clipboard, ready to paste into a forum post or chat. With **Redact identifiers** on (17), hexdump lines such as the System CMOS contents are masked too (2.4).

13.3 Full System Report (srs-sysreport)

In plain words: one click writes a folder with the output of many standard Linux tools, plus an `index.html` page that links them, and opens it in your browser. Good for sending to someone who helps you, and it is what **Find** searches.

Run: the menu entry **SRS Full System Report**, or `srs-sysreport` in a terminal.

- Output folder: `~/sysreports/sysreport-YYYYMMDD-HHMMSS/`.
- Without root: `inxi`, `lscpu`, `cpuid`, `sensors`, `lspci -vvnn`, `lsusb -tv`, GPU (DRM) info, `/proc/meminfo`, `lsblk -O`, mounts, kernel and command line, and SRS's own `devices.txt` (every PCI/USB device resolved to a name).
- With **one** `pkexec` password prompt: `dmidecode`, `decode-dimms`, `hwinfo --all`, `lshw -html` and `smartctl -x` for each NVMe and SATA disk. If you cancel the prompt, those are skipped and a note file says so.
- Missing tools just leave a short error in that section's file.

13.4 Device list

`sysreport-search devices` prints every PCI and USB device in the PC with its ID and resolved name. See CLI-REFERENCE.md.

14. Settings and files

Path	What
`~/.config/srs/SRSDEF.ini`	Settings (SRS's version of SIV's SIVDEF file). See below.
`~/.config/autostart/srs-gui.desktop`	Only if "Start with the desktop" is on.
`~/.config/autostart/srs-*.desktop`	Programs you added with **Add to startup** (6.9); also the turned-off copies of system entries (same name as the system file).
`~/.config/srs/startup-backup/`	Everything the Startup manager moved out of the way or changed: `NAME.desktop.YYYYMMDD-HHMMSS` and `NAME.desktop.orig`. Nothing there is used; copy a file back to `~/.config/autostart` to use it again.
`~/.config/srs/lineups/*.srslineup`	Your own tab lineups (6.7), the same format as Export lineup.
`~/.cache/srs/empty-items.txt`	The items that had nothing to show on this PC at the last check (hidden from the tabs and lists). Made only by Options -> **Check now** or **Check daily** (10); delete it to show those items again.

Path	What
`~/.cache/srs/root/`	Root-only data from `srs-root` (`dmidecode.txt`, `spd.txt`, `smart-*.txt`, `nvme-*.txt`, `acpi-tables/`, `pci-all.txt`, `ioports.txt`, `iomem.txt`, `dmesg.txt`, `msrs.txt`, `collected.txt`; since 2.4 `umc.txt`, `intel-imc.txt`, `aod.bin` / `aod.txt`, `rapl.txt`, `ebda.txt` / `ebda.bin`, `vbios-<pci>.bin`, `gpio.txt`, `efi-systab.txt`, `esrt.txt`, with the opt-ins `pmic-*.txt` and `ec.bin`, and the ACPI evaluation log `acpi-calls.txt`; see 5.4).
`~/.local/share/srs/lists/`	`pci.ids`, `usb.ids`, `pnp.ids` downloaded by Update hardware lists, and the previous copies as `.old` (9.21). SRS uses each only while it is newer than the system's.
`~/.cache/srs/gpu-load.log`	Output of the last GPU Load command (Burnin / Stress).
`~/.cache/srs/burnin-YYYYMMDD-HHMMSS.txt`	The summary of each finished Burnin / Stress run (one file per run).
`~/.cache/srs/bench-<mode>.txt`	The last result of each benchmark mode, IPI Delay and Latency.
`~/.cache/srs/bench-pi.txt`, `bench-crypto.txt`, `bench-prime.txt`, `bench-cachemem.txt`, `bench-storage.txt`, `bench-os-latency.txt`, `bench-compute.txt`, `bench-work.txt`, `bench-net.txt`, `bench-gpu.txt`, `bench-gpuburn.txt`, `bench-gpucompare.txt`	The last result of SRS Pi, Crypto, Prime Sieve, Cache & Memory, Storage, OS Latency, CPU Compute, Everyday Work, Network, GPU Benchmark, GPU Burnin and GPU Compute comparison.
`~/.cache/srs/bench-history.tsv`	One line per run of SRS Pi, Crypto, Prime Sieve, Cache & Memory, Storage, OS Latency, CPU Compute, Network, GPU Benchmark, GPU Burnin and GPU Compute comparison, and one per test group of Everyday Work (time, test, settings, result, build).
`~/sysreports/sysreport-*/`	Full system reports.
`/usr/share/hwdata/pci.ids`, `usb.ids`, `pnp.ids`	Device name lists (from the hwdata package, not part of SRS).
`/usr/lib/systemd/system/srs-root@.service`	The boot-time collector unit.
`/usr/share/doc/srs-sysinfo/`	README, CHANGELOG, EVIDENCE, CHECKLIST.tsv (the full item list), this guide as `SRS-User-Guide.pdf`, `html/SRS-User-Guide.html` and the Markdown sources (`USER-GUIDE.md`, `CLI-REFERENCE.md`, `GLOSSARY.md`, `SECTIONS/`, `images/`).
`/usr/share/applications/srs-user-guide.desktop`	The **SRS User Guide (PDF)** menu entry.
`~/.local/share/srs/power.db`, `~/.config/srs/power.ini`	**Power & Cost** history and settings (6.11).

`SRSDEF.ini` keys (section `[SRSDEF]`). You normally change these through the menus; editing by hand is fine while SRS is closed.

Key	Values	Meaning
`Theme`	`Light`, `Dark`, `System`	Options -> Theme (`System` = Follow desktop). Not set: the main window follows the desktop, the classic view is light.
`Update Interval`	milliseconds; `0` = default, `-1` = paused	Options -> Update interval
`Tray Icon`	`1` / `0`	Options -> System tray icon
`Close To Tray`	`1` / `0`	Options -> Close button hides to tray
`Minimize To Tray`	`1` / `0`	Options -> Minimize hides to tray
`Start In Tray`	`1` / `0`	Options -> Start hidden in tray
`GPU Load Command`	text	Stress & Burnin: Custom GPU command
`Stress GPU Burnin`, `Stress GPU Minutes`, `Stress GPU Sync`, `Stress GPU Backend`, `Stress GPU Precision`, `Stress GPU Limit`, `Stress Until Stop`, `Stress GPU Options Open`	numbers	Stress & Burnin: GPU Burnin on, its minutes, Sync, back end (0 Auto, 1 Vulkan, 2 ROCm / HIP, 3 CUDA, 4 OpenCL), FP64, GPU limit C, Until Stop, GPU options open (2.3.1)
`Stress CPU Limit`	50-110	Stress & Burnin: the CPU temperature limit C, remembered (2.4)
`Accepted High Temperature Limits`	`1` / `0`	Stop limits over 100 C accepted once (9.1); without it a limit over 100 is read as 100 (2.4)
`Read DIMM PMIC`	`1` / `0`	Options -> Read DIMM voltages from their PMIC chips (2.4)
`Read EC`	`1` / `0`	Options -> Read the embedded controller (2.4)
`Accepted ACPI Evaluation`	`1` / `0`	The ACPI method evaluation warning accepted (9.22) (2.4)
`Accepted Load Test Terms`	`1` / `0`	The load test terms agreed (the terms' version), `0` = not yet or withdrawn (9.0)
`Tray Close Notice Off`	`1` / `0`	The one-time "still running in the tray" notice was seen
`Colour Find Text`	6 hex digits, e.g. `FFFF00`	Find highlight colour
`Highlight Matches`	`1` / `0`	Find highlighting
`Font Name`	text	Kept for SIV compatibility; not used for drawing
`Redact Reports`	`1` / `0`	Options -> Redact identifiers in saved reports and copied text
`Classic View`	`1` / `0`	Options -> Start in classic view

Key	Values	Meaning
`Sortable Tables`	`1` / `0`	Options -> Sortable tables in the main window
`All Items As Tabs`	`true` / `false`	Options -> Show every item as a tab
`Show Hidden Items`	`true` / `false`	Options -> Show hidden items
`Check Empty Items`	`true` / `false`	Options -> Check daily which items have nothing to show (default `false`)
`Ask CUDA Toolkit`	`1` / `0`	`0` = "Don't ask again" in the NVIDIA CUDA toolkit question; Options -> NVIDIA CUDA toolkit -> **Ask me again** or `sysreport-search cuda-toolkit --ask-again` sets `1` (default `1`)
`SIO Sensors`	e.g. `nct6799:AUXTIN1`	Super-I/O inputs the burner should treat as connected

Section `[Tabs]` holds the main window's tabs as they are now: `Lineup` (the lineup they came from), `Layout` (`row`, `wrap` or `side`), and per page `<page>=id;id;...` and `<page> Every=true` when the page shows every item. Page names: `processor`, `memory`, `board`, `graphics`, `storage`, `network`, `sensors`, `devices`, `os`, `bench`, `tools`.

Environment variables:

Variable	Effect
`SRS_SMART_USB=1`	`srs-root` also reads SMART from USB-attached disks.
`SRS_KWH_COST`, `SRS_CURRENCY`	Electricity price and currency symbol for the kWh Cost item (default `$0.15` per kWh). Example: `SRS_KWH_COST=0.30 SRS_CURRENCY=EUR srs-gui`.
`SRS_OPEN=item,item`	Same as `--gui-open=` (open panels or tools at start).
`SRS_DOC_DIR=folder`	Where the Guide button, F1 and `guide` look for `SRS-User-Guide.pdf` and `html/` (default `/usr/share/doc/srs-sysinfo`, then beside the program).
`GTK_THEME`	Remembered for "Follow desktop"; the Theme option drives SRS's own look.
`SRS_STARTUP_DRY_RUN=1`	The Startup manager never runs `systemctl` or `pkexec`: it only shows the command it would run (autostart files are still written). For trying it out safely.
`SRS_GPUBURN_SIMPLE=1`	GPU Burnin uses the older, simpler matrix multiply instead of the 2.4 one (for comparing) (9.18).
`SRS_RSMU_DIR=folder`	Read the ryzen_smu files (`pm_table`, `codename`, `pm_table_version`) from this folder instead of `/sys/kernel/ryzen_smu_drv` (tests).

15. Reading results and comparing runs

- **Learn what normal looks like on your PC.** Open the **Sensors** page at idle and note the temperatures, fan speeds and voltages. Then you can spot a change later. The Overview's **Is my PC OK?** gives a quick verdict; the Sensors page has the numbers.
- **Minimum / Maximum** columns on the **Sensor status** tab (the Sensors panel in the classic view) record the extremes since SRS started (or since you pressed **Zero**). Press Zero, play a game or run a burn-in, then look at Maximum.
- **Compare two reports:** save `-save=[all]` before and after a change and compare them with any diff tool, for example `diff -u before.txt after.txt | less` or `meld before.txt after.txt`. Expect live values (clocks, temperatures, up-time, counters) to differ every time; look for changes in settings, versions, link speeds, memory speed.
- **Burn-in and benchmarks:** run them on an idle PC, the same way each time, and repeat 2-3 times. A difference smaller than the run-to-run spread is noise. Each burn-in is kept in `~/.cache/srs/burnin-YYYYMMDD-HHMMSS.txt`, so you can put two runs side by side (`meld old.txt new.txt`). Benchmarks keep only the last run of each mode (`~/.cache/srs/bench-<mode>.txt`): copy that file somewhere before you run the mode again if you want to keep it.
- **Watch the PCIe link** (the **PCI bus** tab of **USB & PCI devices**, or the classic PCI Bus panel): a graphics card should normally show its full width (x16) and speed under load; x4 or x8 can mean the wrong slot or a slot that shares lanes.
- **Watch memory speed** (the **Memory** page's DIMMs tab, or Machine in the classic view): if it shows 4800 MT/s for a 6000 MT/s kit, the XMP / EXPO profile is not enabled in the BIOS.

The Section Reference has "Healthy / typical values" and "Unusual values" for every item: SECTIONS/README.md.

16. Known limits and differences from SIV

- **Two views, same data.** The main window and the classic view show the same 558 items with the same text. The classic panels' extra buttons (for example Features, ARP, PnP Dev) are items in their own right: in the main window find them under **All N items** on the page, or with the search box. **Open in window** shows any tab as the classic panel with its button row.
- **"Is my PC OK?" limits.** SRS knows the maximum temperature of Intel CPUs (they report it) and of AMD desktop Ryzen 5000, 7000 and 9000 models (from AMD's specifications). Other AMD CPUs (laptops, Threadripper, older models) and graphics cards that publish no limit are judged against a general limit; the tooltip says which. The checks are a quick guide, not a diagnosis.
- **Linux sources, not Windows ones.** Values come from the Linux kernel and standard tools, so wording and some figures differ from SIV on Windows for the same PC.
- **Root data is a snapshot.** Panels that use `~/.cache/srs/root` show what was true when `srs-root` last ran.
- **No direct hardware access by default.** SRS never touches monitor DDC, raw HID, or the CPU's management firmware (SMU commands), and by default no SMBus / I2C. Some SIV readings that need that are not available. Since 2.4 the exceptions are your choice: the PMIC opt-in reads the memory modules' power chips over SMBus, the embedded controller opt-in loads `ec_sys` read-only (5.4), and ACPI method evaluation runs BIOS code you type (9.22). Memory Timings reads the

AMD memory controller through the `ryzen_smu` driver (an address written to its `smn` file only selects what is read).

- **136 "not present" items** are for hardware most PCs do not have. **38 Windows-only items** show a Linux equivalent or explain why there is none.
- **Benchmarks** are SRS's own implementations of Dhrystone 2.1 and Whetstone. Scores are for comparing runs on the same PC, not for comparing with SIV's own numbers.
- **Minimize to tray** works on X11 / XWayland only.
- **Hybrid (P-core / E-core) CPUs:** E-cores are found from `/sys/devices/cpu_atom/cpus`, which Linux provides for Intel hybrid CPUs. AMD CPUs with mixed core types (Zen 5 / Zen 5c) are not told apart yet, so `ec` / `ep` report "No efficiency cores detected" there.
- **Latency tool level labels** (L1 / L2 / L3 / DRAM) use your CPU's cache sizes from `/sys/devices/system/cpu/cpu0/cache`: a block is labelled with the smallest cache it fits in. Labels are indicative; the steps in the curve are the real boundaries.
- **Notes about fan control, RGB and Thunderbolt** describe what SRS finds on your PC: which fan-control program is running (LACT, CoolerControl, CoreCtrl, fancontrol), whether the kernel exposes LED devices for the GPU, and which Thunderbolt / USB4 controllers and routers exist.
- **Super-I/O "not connected" inputs:** an `AUXTIN` temperature far above the board's own `SYSTIN` / `CPUTIN` readings (more than 25 degrees C) is shown greyed as "probably not connected (floating high)". If you have a probe on that header, give the input a label in `/etc/sensors.d/` and SRS treats it as connected.
- **DDR5 SPD:** `decode-dimms` cannot read DDR5 SPD hubs; the SPD item then shows the SMBIOS memory-device summary instead. Since 2.4 the **Memory Timings** tab reads the DDR5 SPD itself (through the kernel's `spd5118` driver) and shows each module's JEDEC / EXPO / XMP profiles there (6.12).
- **Memory Timings:** live timings on AMD Ryzen 7000 / 9000 desktop only (Intel experimental); the BIOS settings from the AOD table need `iomem=relaxed` on most kernels, which SRS never asks you to set.
- The optional **Windows images** (`SRS64X.exe`, `SRS32X.exe`, source build only) have only the ID lookup and report search features.

17. Privacy: before you share a report

A full report describes your PC in detail. Some items include identifiers you may not want to post publicly:

- Board and system **serial numbers** and the system **UUID** (the Motherboard & BIOS page: Machine and the SMBIOS items).
- **Disk serial numbers**, volume serials and partition IDs (the Storage page).
- **MAC addresses**, IP addresses, Wi-Fi network names (the Network page).
- **Monitor serial numbers** and the GPU unique ID (the Graphics & Audio page).
- USB device serial numbers (Properties, on the USB & PCI devices page).
- The **kernel command line**, which contains your root file system's UUID (General Info and Speculation Control; the classic Linux panel).
- The **boot ID**, machine ID and your PC's host name.

Redact before you share. Save the report with identifiers masked:

```
sysreport-search '-save=[all]=for-forum.txt' --redact
```

or tick **Menu -> Options -> Redact identifiers in saved reports and copied text** and use **Save Report** / **Save All Items**. With the option on, the **Copy** button and right-click **Copy** (or Ctrl+C on a selection) mask the same things, so text you paste into a forum post is safe too. Serial numbers, UUIDs, WWNs, the GPU / Thunderbolt unique IDs, the boot and machine IDs and the host name become `XXXX` (same length, so columns stay aligned); MAC addresses keep their vendor half (`04:7c:16:XX:XX:XX`), which is useful for telling network chips apart. The report's first line ends with `(identifiers redacted)`. Published constants such as GPT partition-type GUIDs are not masked.

The **System CMOS** item (2.4) shows the CMOS RAM, which can hold the BIOS password check value: with Redact on, its bytes (and every hexdump line in copied text) become `XX`. The ACPI evaluation log (`~/.cache/srs/root/acpi-calls.txt`) is not part of any report.

The **Power & Cost** history (`~/.local/share/srs/power.db`) is not part of any report; it says when the PC was in use, so think before sharing an export of it. Nothing on that page goes online.

Not masked: **IP addresses** and **Wi-Fi network names**. Look for them (or use Find) before you post. Or save only the items you need: `sysreport-search '-save=[processors][sensors]=for-forum.txt'`.

18. Troubleshooting

Problem	What to do
The Overview says "Not checked yet" for drive health, or a tab shows **Needs root data...**	Run **Menu -> Run as root (srs-root)...** once (or press **Needs root data...**). The drive health line turns green or red after that.
A line on the Overview is grey ("Unknown")	SRS found nothing to check, for example no CPU or GPU temperature sensor. Look at the Sensors page; see "Sensors page is nearly empty" below.
DIMM lines, serials, SMART or SPD tabs are empty or say to run srs-root	Run **Menu -> Run as root** once (needs `polkit` for the password prompt), or `sudo srs-root`. Install `dmidecode`, `smartmontools`, `nvme-cli`, `i2c-tools` for full data.
Run as root does nothing / "did not complete"	You cancelled the password prompt, or no polkit agent is running (some minimal window managers). Use `sudo srs-root` in a terminal instead.
No tray icon	Install `libayatana-appindicator`. On GNOME, install and enable the AppIndicator extension.
Closing the window does not exit SRS	"Close button hides to tray" is on. Use **Menu -> Quit** (Ctrl+Q; **Exit** in the classic Menu) or the tray's Exit, or untick the option.
SRS opens the old SIV-style window	**Options -> Start in classic view** is on. Untick it, or use **Menu -> Open the new main window** in the classic Menu, or start `srs-gui --gui-new`.
I want the old SIV-style window back	**Menu -> Classic View**, the **Classic view** button under the sidebar, or `srs-gui --gui-classic`. Tick **Options -> Start in classic view** to always start there.

Problem	What to do
SRS starts hidden	"Start hidden in tray" or "Start with the desktop" is on; click the tray icon.
Minimize to tray does nothing	You are on native Wayland. The option works only on X11 / XWayland.
Colours look wrong or half-dark	Pick **Options -> Theme** explicitly (Light or Dark).
Temperatures of 70-90 C or 0 C on some motherboard inputs	Those inputs are not connected to a sensor. SRS greys them out ("not connected"). Ignore them. See sensors.md.
Sensors page (or panel) is nearly empty	The kernel has no driver for your board's monitoring chip. Run `sudo sensors-detect` (from `lm_sensors`) and load the module it suggests.
CPU Burnin stops quickly with Fail	The temperature limit was reached. Check cooling. Raise the limit only if you know your CPU's safe maximum.
"GPU load command not found"	The program in GPU Load command is misspelled or not installed. Fix it, or untick GPU Load command. Its output is in `~/.cache/srs/gpu-load.log`.
Find says there is no report	Run **SRS Full System Report** first.
Device names show "Unknown Device"	Your `hwdata` package is old. Update it (new hardware is added to `pci.ids` / `usb.ids` regularly).
Panels update too often / use CPU	**Options -> Update interval** -> 2 or 5 seconds, or Paused. In the main window you can also turn off the **Live** switch of one tab.
I cannot find an item	Type part of its name or its id into the search box (**Ctrl+F**), or look under **All N items** on the page.
`-save=[...]` gives "no matches found" in zsh	Quote it: `sysreport-search '-save=[cpuid]'` (the brackets are a pattern in zsh).
Ping item hangs a moment	It waits for your router to answer. Normal.
The CUDA rows say "the CUDA driver refused the built-in kernels ... see Options > NVIDIA CUDA toolkit"	Your graphics driver (or ZLUDA version) does not take SRS's pre-built CUDA tests. Install the runtime compiler from **Options -> NVIDIA CUDA toolkit for GPU tests...** (not recommended with ZLUDA: update ZLUDA instead), see 9.20.
SRS asks about the NVIDIA CUDA toolkit	Only once, before the first CUDA run, when it is not installed. **Not now** or **Don't ask again** is fine for most people; the CUDA tests are built in.
The CUDA toolkit install says the package database is locked	Another install or update is running. Let it finish, then use **Install...** again.
OpenCL FP64 rows say "this OpenCL driver has no FP64"	That driver (often Mesa's Rusticl) has no double precision. Normal; another OpenCL driver (for example ROCm's) may have it.

19. Uninstall

- **Arch:** `sudo pacman -R srs-sysinfo`
- **AppImage:** delete the `.AppImage` file.
- **Generic tarball:** remove the files you copied (the tarball's file list), for example from `/usr/local/bin` and `/usr/local/share`, and any unit you placed in `/etc/systemd/system`.

- **Before uninstalling**, if you enabled "Refresh root data at boot": `sudo systemctl disable srs-root@$USER.service`
- **Your data** (optional): `rm -r ~/.config/srs ~/.cache/srs ~/sysreports` and `rm -f ~/.config/autostart/srs-gui.desktop`

20. FAQ

Is this SIV? No. It is an independent Linux program modelled on SIV's layout and items. It contains nothing from SIV and is not connected with RH Software.

Does it need root? No. It always runs as your user. Root is used only by the optional `srs-root` helper, once, to save root-only data for you.

Can it harm my PC? The viewer is read-only. The burn-in and benchmarks load the CPU like any stress tool; they ask first, stop at a temperature limit (default 90 C) and stop when you close the window. The Startup manager changes what starts with the PC only when you click it, asks before touching parts of the desktop, keeps every old file, and has Undo and Restore default.

Why are there items that say "not present"? They are SIV items for hardware most PCs do not have (Itanium, IPMI, RAID cards...). SRS keeps them so the item list matches SIV's.

Why "Linux" where SIV says "Windows"? SIV's "Windows" button opens its operating-system panel. On Linux that is Linux, so the classic view's button says **Linux**. The item itself is called **General Info** (id `windows`, as in SIV); in the main window it is the first tab of the **Operating system** page.

Where is the window from SRS 1.1? It is the **classic view**: **Menu -> Classic View**, the **Classic view** button under the sidebar, or `srs-gui --gui-classic`. Tick **Options -> Start in classic view** to make it the default. Nothing was removed: both views show the same items.

Can I use it over SSH or in a script? Yes. `sysreport-search` needs no desktop. See CLI-REFERENCE.md.

Do I need the NVIDIA CUDA toolkit? Almost certainly not. Since 2.3 SRS's CUDA tests are built in. The toolkit (its runtime compiler, NVRTC) is only a fallback when the graphics driver refuses them. If SRS asked and you said **Not now** or **Don't ask again**, you can install it later from **Options -> NVIDIA CUDA toolkit for GPU tests...**. With ZLUDA, do not install it - see 9.20.

Where do device names come from? From the hwdata project's `pci.ids`, `usb.ids` and `pnp.ids` lists on your system, and the systemd hardware database. They are not part of SRS.

Does it run on ARM? The command line builds for aarch64 (`make all`), and the viewer is plain C. The release packages are x86-64.

21. Glossary

The full A-Z glossary is in GLOSSARY.md. The Section Reference explains each term the first time it uses it.

SRS Section Reference

Back to the User Guide - Glossary I wanted a Linux version of SIV (System Information Viewer) for myself, so I built one and set out to improve on it along the way. SRS is free and open source for the Linux and open-source community. (SRS is an independent project, not affiliated with SIV or its author.) SIV is the work of Ray Hinchliffe, RH Software (rh-software.com).

How to use this reference

This reference explains **every item SRS can show** - all 558 of them - one by one. For each item it says, in this order:

1. **In plain words** - what it is, in one to three short sentences.
2. **What you see** - every column and field, and why it matters.
3. **Healthy / typical values** - what normal looks like, with real examples from the author's PC (AMD Ryzen 7 7800X3D, ASUS TUF GAMING X870-PRO WIFI7, 2 x 16 GB DDR5-6000, Radeon RX 9070 XT).
4. **Unusual values and what they suggest.**
5. **Where Linux gets it** - the file, table or tool the value comes from.
6. **Background** - the idea behind it (what a cache is, what a P-state is, ...), where it helps.

Every technical word is explained the first time a page uses it, and again in the Glossary.

Badges under each title:

- **Linux data** - shows real data from your Linux system.
- **Not present on most PCs** - SIV items for hardware most PCs lack (Itanium, IPMI, RAID cards...). On a normal PC they say so.
- **Windows-only (Linux equivalent shown)** - a Windows idea; SRS shows the nearest Linux equivalent.
- **Needs root data** - some or all of the values come from the `srs-root` cache (`~/.cache/srs/root`, see the User Guide). Without it those values are empty.

How to open an item:

- **In the main window** (SRS 2.0): pick the page in the sidebar (the table below names it). The most used items are tabs; **All N items** at the right end of the tab row lists every item of the page. Or type the item's name or id into the search box at the top (**Ctrl+F**). The **Guide** button (F1) on each tab opens the item's section here.
- **In the classic view:** Menu -> the group -> the item (each menu entry shows the item id in brackets), or the panel buttons named in each group's introduction.
- **From the command line:** `sysreport-search '-save=[id]'`.

Groups

The groups are in the same order as SRS's Menu.

Group	Items	Page	Main-window page
Processors	44	processors.md	Processor
Machine / BIOS / Memory	58	machine.md	Memory; Motherboard & BIOS
Sensors / Cooling / Power	34	sensors.md	Sensors
Volumes / Drives	28	storage.md	Storage
Network	36	network.md	Network
Linux (SIV's Windows items)	78	linux-os.md	Operating system
Graphics / Monitors / Audio	52	graphics.md	Graphics & Audio
Buses / Devices (part 1)	64	buses-1.md	USB & PCI devices
Buses / Devices (part 2)	63	buses-2.md	USB & PCI devices
Tools	34	tools.md	Benchmarks & Stress; Tools
SRS (SIV items)	60	srs-items.md	Tools

Every item

Processors

Item	id	Badge
AMD Ryzen Power Management	`amd-cpu`	Linux data
AMD Fuses	`amd-fus`	Linux data
AMD PM Table	`amd-pmt`	Linux data
AMD PM Table 0	`amd-pmt-0`	Linux data
AMD PM table data @ second package/CCD	`amd-pmt-1`	Not present on most PCs
AMD SMU	`amd-smu`	Linux data - Needs root data
AMD Telemetry 0	`amd-tel-0`	Linux data
AMD SVI telemetry plane @ second die	`amd-tel-1`	Not present on most PCs
AMD Thermal 0	`amd-thm-0`	Linux data
AMD THM registers @ second die	`amd-thm-1`	Not present on most PCs
AMD Temperatures	`amd-tmp`	Linux data
CPU Table	`cpu`	Linux data
CPU Description	`cpu-bits`	Linux data
CPU Clocks	`cpu-clocks`	Linux data
CPU Export	`cpu-export`	Linux data
CPUID	`cpuid`	Linux data
CPUID Leaf 0	`cpuid-000`	Linux data
CPU Idle Levels	`cpu-levels`	Linux data

Item	id	Badge
CPU OC Data	`cpu-oc-data`	Not present on most PCs - Needs root data
CPU Summary	`cpus`	Linux data
CPU SMU	`cpu-smu`	Linux data - Needs root data
CPU Speeds	`cpu-speeds`	Linux data
CPU Timers	`cpu-timer`	Linux data
CPU Tunables	`cpu-tweak`	Linux data - Needs root data
CPU Voltages	`cpu-volts`	Linux data
CPUID Features	`features`	Linux data
Hypervisor	`hypervisor`	Linux data
Microcode	`microcode`	Linux data
MSRs	`msrs`	Linux data - Needs root data
MSR Setup	`msr-setup`	Linux data - Needs root data
IBM/Cyrix MSRs	`msrs-ibm`	Not present on most PCs - Needs root data
NexGen NXOP	`nexgen-nxop`	Not present on most PCs
Nodes	`nodes`	Linux data
NUMA	`numa`	Linux data
NUMA memory speed	`numa-speeds`	Linux data
CPU power states	`power-states`	Linux data
Processor Features Present	`present`	Linux data
Processor	`processor`	Linux data
Processors	`processors`	Linux data
Speculation Control	`speculation`	Linux data
Turbo Boost	`turbo-boost`	Linux data
Xeon PIRs	`xeon-pir`	Not present on most PCs
Cache Levels	`cache`	Linux data
CPU Groups	`group`	Linux data

Machine / BIOS / Memory

Item	id	Badge
ACPI PCI root buses	`acpi-buses`	Linux data
ACPI devices	`acpi-dev`	Linux data
ACPI embedded controller	`acpi-ec`	Linux data - Needs root data
ACPI method evaluation	`acpi-eval`	Linux data - Needs root data
ACPI HAL	`acpi-hal`	Linux data
ACPI interrupts and GPEs	`acpi-int`	Linux data

Item	id	Badge
ACPI IPF (Intel Platform Framework) methods	`acpi-ipf`	Not present on most PCs
ACPI ICD0001 methods	`acpi-itm`	Not present on most PCs
ACPI methods	`acpi-methods`	Linux data - Needs root data
ACPI extension (PDX)	`acpi-pdx`	Linux data
ACPI PCI buses and ports	`acpi-ports`	Linux data
ACPI power resource methods	`acpi-pwr`	Linux data
ACPI SPB (serial peripheral bus) methods	`acpi-spb`	Linux data
ACPI tables	`acpi-tables`	Linux data - Needs root data
ACPI Toshiba test	`acpi-test`	Not present on most PCs
ACPI TFP INT3403 methods	`acpi-tfp`	Not present on most PCs
ACPI TMS INT340E methods	`acpi-tms`	Not present on most PCs
ACPI TOS1900 methods	`acpi-tos`	Not present on most PCs
ACPI TPM	`acpi-tpm`	Linux data
ACPI WMI (PNP0C14) methods	`acpi-wmi`	Linux data
APIC (MADT) summary	`bios-apic`	Linux data - Needs root data
ASPT table summary	`bios-aspt`	Not present on most PCs - Needs root data
DMAR table summary	`bios-dmar`	Not present on most PCs - Needs root data
FACP (FADT) summary	`bios-facp`	Linux data - Needs root data
PCI IRQ routing ($PIR)	`bios-irq`	Not present on most PCs - Needs root data
BIOS maker	`bios-make`	Linux data
MPS table summary	`bios-mps`	Not present on most PCs - Needs root data
MSDM table summary	`bios-msdm`	Not present on most PCs - Needs root data
OEMB table summary	`bios-oemb`	Not present on most PCs - Needs root data
BIOS option ROMs	`bios-option`	Linux data - Needs root data
SDEV table summary	`bios-sdev`	Not present on most PCs - Needs root data
SFI summary	`bios-sfi`	Not present on most PCs
SLIC table summary	`bios-slic`	Not present on most PCs - Needs root data
SLIT table summary	`bios-slit`	Not present on most PCs - Needs root data
SRAT table summary	`bios-srat`	Not present on most PCs - Needs root data
BIOS / SMBIOS tables	`bios-tables`	Linux data - Needs root data
CMOS / RTC status	`cmos-status`	Linux data
Dell BIOS	`dell-bios`	Not present on most PCs
DIMMs	`dimms`	Linux data - Needs root data
DIMM status/temperatures	`dimm-status`	Linux data
Firmware	`firmware`	Linux data

Item	id	Badge
Firmware interface table	`f-w-table`	Not present on most PCs
Hardware summary	`hardware`	Linux data
HECI (Intel ME) status	`heci-status`	Not present on most PCs
Machine	`machine`	Linux data
Machine options	`machine-opt`	Linux data
Memory	`memory`	Linux data
Memory timings	`mem-tweak`	Linux data - Needs root data
Overview	`overview`	Linux data
Physical memory map	`physical`	Linux data
SPD (DIMMs)	`spd`	Linux data
SPD thermal	`spd-temp`	Linux data
System CMOS	`system-cmos`	Linux data
System EBDA	`system-ebda`	Linux data - Needs root data
BIOS IDs	`system-ids`	Linux data - Needs root data
TPM device	`tpm-device`	Linux data
UEFI data	`uefi-data`	Linux data
UEFI information	`uefi-info`	Linux data

Sensors / Cooling / Power

Item	id	Badge
AIO Link export	`aio-export`	Not present on most PCs
ASUS ATK/WMI info	`asus-info`	Linux data - Needs root data
ASUS WMI ACPI methods	`asus-wmi`	Linux data - Needs root data
ASUS ATK0110 debug	`atk-debug`	Not present on most PCs
Battery	`battery`	Not present on most PCs
Cooling status (fans)	`cooling`	Linux data
Sensor formula setup	`formula-setup`	Linux data
Hardware status	`hw-status`	Linux data
IPMI dump	`ipmi-dump`	Not present on most PCs
IPMI status	`ipmi-info`	Not present on most PCs
IPMI time	`ipmi-time`	Not present on most PCs
IPMI wait	`ipmi-wait`	Not present on most PCs
kWh cost	`kwh-cost`	Linux data
LCD panel setup	`lcd-panel`	Not present on most PCs
Super I/O LDNs	`ldn-info`	Linux data - Needs root data

Item	id	Badge
AIO Link devices	`link-devices`	Not present on most PCs
AIO Link dump	`link-dump`	Not present on most PCs
AIO Link fans	`link-fans`	Not present on most PCs
AIO Link LEDs	`link-leds`	Not present on most PCs
AIO Link limits	`link-limits`	Not present on most PCs
AIO Link power	`link-power`	Not present on most PCs
AIO Link setup	`link-setup`	Not present on most PCs
AIO Link status	`link-status`	Not present on most PCs
AIO Link test	`link-test`	Not present on most PCs
AIO Link update	`link-update`	Not present on most PCs
Power status	`power`	Linux data
PWM export	`pwm-export`	Linux data
QST status	`qst-status`	Not present on most PCs
Sensor export	`sen-export`	Linux data
Sensor status	`sensors`	Linux data
SIO debug	`sio-dbg`	Linux data - Needs root data
Synthetic sensors setup	`synthetic`	Linux data
Thermal zones	`thermal-zones`	Not present on most PCs
UPS export	`ups-export`	Not present on most PCs

Volumes / Drives

Item	id	Badge
Disk drives	`drives`	Linux data
Drive tree	`drive-tree`	Linux data
Disk API routines	`dsk-api`	Windows-only (Linux equivalent shown)
FAT statistics	`fat`	Linux data
Drive geometry	`geometry`	Linux data
Hidden disks	`hidden`	Linux data
JMicron JMB RAID	`jmb-raid`	Not present on most PCs
JMicron NVMe RAID	`jms-raid`	Not present on most PCs
Disk mapping	`mapping`	Linux data
LSI MegaRAID	`mega-raid`	Not present on most PCs
NTFS statistics	`ntfs`	Linux data
Partitions	`partitions`	Linux data
Disk performance	`performance`	Linux data

Item	id	Badge
Mount points	`points`	Linux data
CSMI RAID	`raid`	Not present on most PCs
RocketRAID	`rocket-raid`	Not present on most PCs
SCSI devices	`scsi`	Linux data
Samsung NVMe (secnvme)	`secnvme`	Linux data - Needs root data
Disk sector sizes	`sectors`	Linux data
SMART drives	`smart`	Linux data - Needs root data
SMART all drives	`smart-all`	Linux data - Needs root data
SMART drive monitor	`smart-sdm`	Linux data - Needs root data
Storage spaces	`spaces`	Linux data - Needs root data
NVMe (stornvme)	`stornvme`	Linux data - Needs root data
USB disk drives	`usb-disks`	Linux data
Volume list	`vol-list`	Linux data
Volumes	`volumes`	Linux data
Intel VROC	`vroc`	Not present on most PCs

Network

Item	id	Badge
Network adapters	`adapters`	Linux data
ARP table	`arp-table`	Linux data
Bluetooth radios	`bluetooth`	Linux data
Bluetooth IDs	`bluetooth-ids`	Linux data
Bluetooth device tree	`blue-tree`	Linux data
Network browse masters	`browsers`	Linux data
TCP/IP counters	`counters`	Linux data
DNS cache	`dns-cache`	Linux data, Needs root data
DNS names	`dns-names`	Linux data
DNS ports	`dns-ports`	Linux data
DNS usage	`dns-usage`	Linux data
Domains / workgroups	`domains`	Linux data
Firewall	`firewall`	Linux data, Needs root data
Network interface statistics	`interface`	Linux data
IP address	`ip-address`	Linux data
LAN Manager share setup	`lm-setup`	Linux data
MAC addresses	`mac-address`	Linux data

Item	id	Badge
Modem API	`modem-api`	Not present on most PCs
Modems	`modems`	Not present on most PCs
My browser / IP location	`my-browser`	Linux data
My IP address	`my-ip-address`	Linux data
Network cards	`net-cards`	Linux data
Network provider names	`net-names`	Windows-only (Linux equivalent shown)
Network	`network`	Linux data
Network profiles	`profiles`	Linux data
Network protocols	`protocols`	Linux data
Proxy config	`proxy`	Linux data
Routing table	`routing`	Linux data
Servers	`servers`	Linux data
TCP V4 ports	`tcp-ip4-ports`	Linux data
TCP V6 ports	`tcp-ip6-ports`	Linux data
Network transports	`transports`	Linux data
UDP V4 ports	`udp-ip4-ports`	Linux data
UDP V6 ports	`udp-ip6-ports`	Linux data
Wi-Fi SSIDs	`wi-fi`	Linux data
WLAN profiles	`wlan`	Linux data, Needs root data

Linux (SIV's Windows items)

Item	id	Badge
ASUS control service	`acs-srv`	Not present on most PCs
Boot mode options	`boot-mode`	Linux data
Corsair Link 4 service	`cl4-srv`	Not present on most PCs
Shell CLSIDs	`clsid`	Windows-only (Linux equivalent shown)
Code pages	`codepage`	Linux data
TAPI country list	`country`	Linux data
Corsair service	`cue-srv`	Not present on most PCs
Base DLLs	`dll`	Linux data
DLLs by address	`dll-addr`	Linux data
Known DLLs	`dll-known`	Linux data
DLLs by name	`dll-name`	Linux data
Drivers	`drivers`	Linux data
Driver versions	`driver-vsn`	Linux data

Item	id	Badge
E-mail accounts	`e-mail`	Windows-only (Linux equivalent shown)
Environment	`environment`	Linux data
ESU status	`esu-sts`	Windows-only (Linux equivalent shown)
System file cache	`file-cache`	Linux data
Open server files	`files`	Linux data - Needs root data
Feature management overrides	`fmo-sts`	Windows-only (Linux equivalent shown)
Folder paths	`folders`	Linux data
Font families	`fonts`	Linux data
Groups	`groups`	Linux data
Handles	`handle`	Linux data
Process heap	`heap`	Linux data
Registry hives	`hives`	Windows-only (Linux equivalent shown)
Code integrity	`integrity`	Linux data
Key names	`key-names`	Linux data
Locales	`locales`	Linux data
Memory heap	`mem-heap`	Linux data
Memory mapping	`mem-map`	Linux data
System metrics	`metrics`	Windows-only (Linux equivalent shown)
MLang conversions	`mlang`	Windows-only (Linux equivalent shown)
DOS device names	`ms-dos`	Linux data
N2K service	`n2k-srv`	Not present on most PCs
NLS code pages	`nls-info`	Linux data
Paging files	`paging`	Linux data
Credentials	`passwords`	Windows-only (Linux equivalent shown)
Pending file renames	`pending`	Windows-only (Linux equivalent shown)
ProLiant monitor service	`plm-srv`	Not present on most PCs
Windows policies	`policies`	Windows-only (Linux equivalent shown)
Memory pools	`pools`	Linux data
Power policy	`power-policy`	Linux data
Power scheme	`power-scheme`	Linux data
Privileges	`priv`	Linux data
Processes	`processes`	Linux data
Product IDs	`product-ids`	Windows-only (Linux equivalent shown)
Product keys	`products`	Windows-only (Linux equivalent shown)
Font ranges	`ranges`	Linux data

Item	id	Badge
RDP session info	`rdp-info`	Linux data
Remote registry paths	`reg-path`	Windows-only (Linux equivalent shown)
Remote registry service	`reg-srv`	Windows-only (Linux equivalent shown)
Text converter render format	`render-format`	Windows-only (Linux equivalent shown)
Intel RST service	`rst-srv`	Not present on most PCs
Scheduling	`scheduling`	Linux data
Driver services	`services-drv`	Linux data
Win32 services	`services-w32`	Linux data
Sessions	`sessions`	Linux data
Setup directories	`setup`	Linux data
Setup API logging	`setup-api`	Windows-only (Linux equivalent shown)
Driver signing	`signing`	Linux data
Installed software	`software`	Linux data
Startup programs	`startup`	Linux data
System	`system`	Linux data
System timers	`sys-tim`	Linux data
System windows	`sys-win`	Windows-only (Linux equivalent shown)
Thread windows	`tid-win`	Windows-only (Linux equivalent shown)
Time	`time`	Linux data
Times	`times`	Linux data
Users	`users`	Linux data
Driver verifier	`verifier`	Windows-only (Linux equivalent shown)
Virtual memory	`virtual`	Linux data
Visual Studio paths	`vs-paths`	Windows-only (Linux equivalent shown)
Dr Watson / crash debugger	`watson`	Linux data
Error reporting	`wer`	Linux data
Windows CE devices	`win-ce`	Windows-only (Linux equivalent shown)
General Info	`windows`	Linux data
WinSAT	`winsat`	Windows-only (Linux equivalent shown)
Time zones	`zones`	Linux data

Graphics / Monitors / Audio

Item	id	Badge
Audio devices	`audio`	Linux data
Direct3D info	`d3d-info`	Linux data

Item	id	Badge
GPU D3D engine nodes	`d3d-node`	Linux data
DirectX information	`directx`	Linux data
DirectX sound devices	`dxd-aud`	Linux data
DirectX system devices	`dxd-dev`	Linux data
DirectX logical disks	`dxd-dsk`	Linux data
DirectX files	`dxd-fil`	Linux data
DirectShow filters	`dxd-ftr`	Linux data
DirectInput gameports	`dxd-gam`	Not present on most PCs
DirectX GPU displays	`dxd-gpu`	Linux data
DirectX system information	`dxd-inf`	Linux data
DirectInput devices	`dxd-inp`	Linux data
DirectMusic ports	`dxd-mic`	Linux data
DirectPlay adapters	`dxd-ply`	Windows-only (Linux equivalent shown)
DirectInput PS/2 devices	`dxd-ps2`	Not present on most PCs
DirectX summary	`dxd-sum`	Linux data
DirectInput USB root	`dxd-usb`	Linux data
GOverlay status	`goverlay`	Linux data
GPU all details	`gpu-all`	Linux data
GPU API routines	`gpu-api`	Linux data
GPU applications	`gpu-apps`	Linux data
GPU VBIOS status	`gpu-bios`	Linux data
GPU clock speeds	`gpu-clocks`	Linux data
CUDA information	`gpu-cuda`	Not present on most PCs
GPU D3D status	`gpu-d3d`	Linux data
GPU register dump	`gpu-dump`	Linux data - Needs root data
GPU export	`gpu-export`	Linux data
GPU fans	`gpu-fans`	Linux data
GPU generations	`gpu-gens`	Linux data
GPU G-Sync / VRR	`gpu-g-sync`	Linux data
GPU I2C buses	`gpu-i2c`	Linux data
GPU information	`gpu-info`	Linux data
GPU LEDs	`gpu-leds`	Not present on most PCs
GPU mapping	`gpu-map`	Linux data
GPU power status	`gpu-power`	Linux data
GPU registry	`gpu-reg`	Linux data

Item	id	Badge
GPU SCF	`gpu-scf`	Linux data
GPU SLI topology	`gpu-sli`	Not present on most PCs
GPU temperatures	`gpu-temp`	Linux data
GPU tree	`gpu-tree`	Linux data
Graphics	`graphics`	Linux data
Joysticks	`joysticks`	Linux data
Monitor ID database	`mon-info`	Linux data
Monitors	`monitors`	Linux data
Monitor tree	`monitor-tree`	Linux data
OpenCL	`opencl`	Linux data
RSR info	`rsr-info`	Windows-only (Linux equivalent shown)
RTSS status	`rtss-sts`	Not present on most PCs
Sound devices	`sound`	Linux data
UAA (HD Audio) devices	`uaa-dev`	Linux data
Wave devices	`wave`	Linux data

Buses / Devices (part 1)

Item	id	Badge
COM port resource arbiter	`arbiter`	Windows-only (Linux equivalent shown)
Chipsets	`chipsets`	Linux data
System device classes	`class`	Linux data
Remote system classes	`class-remote`	Windows-only (Linux equivalent shown)
Co-device installers	`codevice`	Windows-only (Linux equivalent shown)
Compatible IDs	`compatible`	Linux data
Device IDs	`device-ids`	Linux data
All devices	`devices`	Linux data
Device owners	`dev-owner`	Linux data
Device property keys	`devpropkey`	Linux data
Embedded controller I/O	`embedded`	Linux data - Needs root data
System enum (device tree)	`enum`	Linux data
Remote system enum	`enum-remote`	Windows-only (Linux equivalent shown)
FireWire	`firewire`	Not present on most PCs
Ghost (non-present) devices	`ghost-devices`	Windows-only (Linux equivalent shown)
HID all devices	`hid-all`	Linux data
HID bus	`hid-bus`	Linux data

Item	id	Badge
HID tree	`hid-tree`	Linux data
Device interfaces	`interfaces`	Linux data
Device interfaces (extended)	`interfacex`	Linux data
Interface IDs	`int-ids`	Linux data
Interface names	`int-names`	Linux data
Interface owner	`int-owner`	Linux data
ASUS I/O	`io-asus`	Linux data - Needs root data
I/O port dump	`io-dump`	Linux data - Needs root data
I/O GPIO	`io-gpio`	Linux data - Needs root data
I/O PM2 MMIO	`io-mmio`	Linux data - Needs root data
nForce I/O	`io-nforce`	Not present on most PCs
I/O ports	`io-ports`	Linux data - Needs root data
IrDA ports	`irports`	Not present on most PCs
ISA bus	`isa-bus`	Linux data
MCA bus	`mca-bus`	Not present on most PCs
Chipset MCH	`mch`	Linux data - Needs root data
Chipset MRH	`mrh`	Not present on most PCs
NPU names	`npu-names`	Not present on most PCs
PCI all	`pci-all`	Linux data - Needs root data
PCI BARs	`pci-bars`	Linux data
PCI bus	`pci-bus`	Linux data
PCI devices	`pci-dev`	Linux data
PCI flags	`pci-flag`	Windows-only (Linux equivalent shown)
PCI HAL	`pci-hal`	Linux data - Needs root data
PCI ID database	`pci-info`	Linux data
PCI MMIO	`pci-mmio`	Linux data - Needs root data
PCMCIA all	`pcm-all`	Not present on most PCs
PCMCIA bus	`pcm-bus`	Not present on most PCs
PCMCIA ID database	`pcm-info`	Not present on most PCs
I/O PM1 data	`pm1-area`	Linux data - Needs root data
PMBus	`pmb-bus`	Not present on most PCs
PnP manager	`pnp`	Linux data
PnP devices	`pnp-dev`	Linux data
PnP ID database	`pnp-info`	Linux data
PnP manager details	`pnp-man`	Linux data

Item	id	Badge
COM ports	`ports`	Linux data
Device properties	`properties`	Linux data
Device property	`property`	Linux data
PYXIS 21174	`pyxis`	Not present on most PCs
Smart cards	`scard`	Not present on most PCs
Serial COM ports	`serial`	Linux data
SI USB	`si-usb`	Linux data
SMBus	`smb`	Linux data
SMBus devices	`smb-bus`	Linux data
SMBus status	`smb-sts`	Linux data - Needs root data
SMBus setup	`smbus-setup`	Linux data
System SMBus access	`smbus-system`	Linux data

Buses / Devices (part 2)

Item	id	Badge
System HAL resources	`sys-hal`	Linux data - Needs root data
System resources	`sys-res`	Linux data - Needs root data
Thunderbolt	`thunder`	Not present on most PCs
Bus address translation	`translation`	Linux data
UID database	`uid-info`	Linux data
USB all	`usb-all`	Linux data
USB bus	`usb-bus`	Linux data
USB devices	`usb-dev`	Linux data
USB flags	`usb-flag`	Linux data
USB flags detail	`usb-flags`	Linux data
USB ID database	`usb-info`	Linux data
USB names	`usb-names`	Linux data
USB owner	`usb-owner`	Linux data
USB tree	`usb-tree`	Linux data
WPD tree (portable devices)	`wpd-tree`	Linux data
XMC cards	`xmc`	Not present on most PCs
XMC card 0	`xmc-card-0`	Not present on most PCs
XMC card 1	`xmc-card-1`	Not present on most PCs
XMC card 2	`xmc-card-2`	Not present on most PCs
XMC card 3	`xmc-card-3`	Not present on most PCs

Item	id	Badge
XMC control	`xmc-control`	Not present on most PCs
XMC flash 0	`xmc-flash-0`	Not present on most PCs
XMC flash 1	`xmc-flash-1`	Not present on most PCs
XMC flash 2	`xmc-flash-2`	Not present on most PCs
XMC flash 3	`xmc-flash-3`	Not present on most PCs
XMC FPGA 0	`xmc-fpga-0`	Not present on most PCs
XMC FPGA 1	`xmc-fpga-1`	Not present on most PCs
XMC FPGA 2	`xmc-fpga-2`	Not present on most PCs
XMC FPGA 3	`xmc-fpga-3`	Not present on most PCs
XMC names	`xmc-names`	Not present on most PCs
XMC PCIe 0	`xmc-pcie-0`	Not present on most PCs
XMC PCIe 1	`xmc-pcie-1`	Not present on most PCs
XMC PCIe 2	`xmc-pcie-2`	Not present on most PCs
XMC PCIe 3	`xmc-pcie-3`	Not present on most PCs
XMC setup 0	`xmc-setup-0`	Not present on most PCs
XMC setup 1	`xmc-setup-1`	Not present on most PCs
XMC setup 2	`xmc-setup-2`	Not present on most PCs
XMC setup 3	`xmc-setup-3`	Not present on most PCs
XMC state 0	`xmc-state-0`	Not present on most PCs
XMC state 1	`xmc-state-1`	Not present on most PCs
XMC state 2	`xmc-state-2`	Not present on most PCs
XMC state 3	`xmc-state-3`	Not present on most PCs
XMC video 0	`xmc-video-0`	Not present on most PCs
XMC video 1	`xmc-video-1`	Not present on most PCs
XMC video 2	`xmc-video-2`	Not present on most PCs
XMC video 3	`xmc-video-3`	Not present on most PCs
ZX1 MIO-0	`zx1-mio-0`	Not present on most PCs
ZX1 MIO-1	`zx1-mio-1`	Not present on most PCs
ZX1 MIO-2	`zx1-mio-2`	Not present on most PCs
ZX1 MIO-3	`zx1-mio-3`	Not present on most PCs
ZX1 MIO-4	`zx1-mio-4`	Not present on most PCs
ZX1 MIO-5	`zx1-mio-5`	Not present on most PCs
ZX1 MIO-6	`zx1-mio-6`	Not present on most PCs
ZX1 MIO-7	`zx1-mio-7`	Not present on most PCs
ZX1 MIO-8	`zx1-mio-8`	Not present on most PCs

Item	id	Badge
ZX1 MIO-9	`zx1-mio-9`	Not present on most PCs
ZX1 MIO-A	`zx1-mio-a`	Not present on most PCs
ZX1 MIO-B	`zx1-mio-b`	Not present on most PCs
ZX1 MIO-C	`zx1-mio-c`	Not present on most PCs
ZX1 MIO-D	`zx1-mio-d`	Not present on most PCs
ZX1 MIO-E	`zx1-mio-e`	Not present on most PCs
ZX1 MIO-F	`zx1-mio-f`	Not present on most PCs
ZX1 summary	`zx1-mio-s`	Not present on most PCs

Tools

Item	id	Badge
CPU Benchmarks	`benchmark`	Linux data
Benchmark: all CPUs concurrent	`benchmark-ac`	Linux data
Benchmark: all CPUs sequential	`benchmark-as`	Linux data
Benchmark: D+D then W+W	`benchmark-dd`	Linux data
Benchmark: D+W then W+D	`benchmark-dw`	Linux data
Benchmark: E-cores concurrent	`benchmark-ec`	Not present on most PCs
Benchmark: E-cores one at a time	`benchmark-ep`	Not present on most PCs
Benchmark: logical threads concurrent	`benchmark-lc`	Linux data
Benchmark: logical threads one at a time	`benchmark-lp`	Linux data
Benchmark: physical cores concurrent	`benchmark-pc`	Linux data
Benchmark: physical cores one at a time	`benchmark-pp`	Linux data
BOINC status	`boinc`	Not present on most PCs
BOINC RPC info	`boinc-rpc`	Not present on most PCs
ClearCase	`clear-case`	Not present on most PCs
ClearQuest	`clear-quest`	Not present on most PCs
ClearQuest e-mail	`cq-mail`	Not present on most PCs
CPU IPI Delay	`ipi-delay`	Linux data
IPI Dump	`ipi-dmp`	Linux data
IPI Plots	`ipi-plots`	Linux data
Memory Latency	`latency`	Linux data
Cache & Memory Benchmark	`cache-memory`	Linux data
Storage Benchmark	`storage-bench`	Linux data
OS Latency Benchmark	`os-latency`	Linux data
CPU Compute Benchmark	`cpu-compute`	Linux data

Item	id	Badge
Everyday Work Benchmark	`everyday-work`	Linux data
Network Benchmark	`net-bench`	Linux data
GPU Benchmark	`gpu-bench`	Linux data
GPU Burnin	`gpu-burnin`	Linux data
GPU Compute: Vulkan vs HIP vs CUDA vs OpenCL	`gpu-compare`	Linux data
SRS Pi	`srs-pi`	Linux data
Crypto Benchmark	`crypto-bench`	Linux data
Prime Sieve	`prime-sieve`	Linux data
Benchmark History	`bench-history`	Linux data
Lookup	`lookup`	Linux data
Ping	`ping`	Linux data
Configuration Find	`reg-find`	Linux data
Device ID Lookup	`id-lookup`	Linux data
Find	`find`	Linux data
ALL Devices	`all-dev`	Linux data
Stress & Burnin (CPU and GPU)	`burnin`	Linux data
CPU Stress	`cpu-stress`	Linux data

SRS (SIV items)

Item	id	Badge
About SRS	`about`	Linux data
BSB info (battery/HID sensor buffer)	`bsi`	Linux data
CPU group buffer	`cgb`	Linux data
CSV logging	`csv`	Linux data
CSV titles	`csv-titles`	Linux data
Debug API	`debug-api`	Linux data
Delta/cost settings	`delta`	Linux data
DSB info (DIMM sensor buffer)	`dsb`	Linux data
Emoji glyph test	`emoji`	Linux data
Fake IMCs	`fak-imcs`	Not present on most PCs
Fake MCHBAR	`fak-mchbar`	Linux data - Needs root data
Fake PM tables	`fak-pmts`	Linux data
Fake SPDs	`fak-spds`	Linux data
Filter	`filter`	Linux data
Final processing	`final`	Linux data

Item	id	Badge
GPU info buffer	`gpu`	Linux data
Help	`help`	Linux data
Highlight setup	`highlight`	Linux data
Hotkeys	`hot-keys`	Linux data
SIV image (EXE)	`image-exe`	Linux data
SIV image (SYS driver)	`image-sys`	Windows-only (Linux equivalent shown)
Image tree	`image-tree`	Linux data
Initial processing	`initial`	Linux data
Install	`install`	Linux data
SIV languages	`languages`	Linux data
Lock setup	`lck-setup`	Linux data
Maker icons	`makers`	Linux data
SIV manifest	`manifest`	Windows-only (Linux equivalent shown)
Mapping cache	`map-cache`	Windows-only (Linux equivalent shown)
Map/unmap test	`map-unmap`	Windows-only (Linux equivalent shown)
Menus	`menus`	Linux data
Mutex info	`mutex`	Linux data
PMB info (power monitor buffer)	`pmb`	Linux data
Colour RGB debug	`rgb-debug`	Linux data
SDM info (storage device monitor)	`sdm`	Linux data
Sequence setup	`sequence`	Linux data
SGB end	`sgb-end`	Linux data
SGB info (system/board buffer)	`sgb-ini`	Linux data
Active SIV instances	`siv-active`	Linux data
SIV autorun	`siv-autorun`	Linux data
SIV DLLs	`siv-dll`	Linux data
SIV driver	`siv-drv`	Windows-only (Linux equivalent shown)
SIVDEF.ini	`siv-ini`	Linux data
SIV location / window poke	`siv-location`	Linux data
SIV qualifiers	`siv-qualifiers`	Linux data
SIV qualifier list	`siv-quals`	Linux data
SIV setup	`siv-setup`	Linux data
SIV sizes	`siv-sizes`	Linux data
SIV driver service	`siv-srv`	Windows-only (Linux equivalent shown)
SIV usage history	`siv-usage`	Linux data

Item	id	Badge
SIV use / update	`siv-use`	Linux data
SMB end	`smb-end`	Linux data
SMB info	`smb-ini`	Linux data
Lock states	`states`	Linux data
STC info (thread contexts)	`stc`	Linux data
Tune system	`tune`	Linux data
Tuning	`tuning`	Linux data
TZM info (thermal zone buffer)	`tzm`	Linux data
Words	`words`	Linux data
XXB internal buffers	`xxb`	Linux data

Processors (SRS Section Reference)

Back to the section index - Glossary

Where to find these items (SRS 2.0). In the main window they are on the **Processor** page (Alt+2). Its tabs are CPU Summary, Processors, CPU Clocks, CPUID Features, Cache Levels, CPUID, Microcode and Speculation Control; **All N items** lists the rest, plus **Each CPU** and **Cache levels of each CPU** (SIV's CPU-n and Cache-n buttons). You can also type an item's name or id into the search box (Ctrl+F). The rest of this introduction describes the classic view (Menu -> Classic View), with its Menu and panel buttons.

This group covers everything about the processor (CPU): its name and family, how many cores and threads it has, its caches, the features it supports, its clock speeds, power and idle states, temperatures, voltages and the low-level registers behind them. Open any item from **Menu -> Processors -> item**. The quickest way in is the **Processors** button on the classic main panel. Its row of sub-buttons opens **Features**, **Hypervisor**, **Groups**, **CPU-n** (one processor in detail) and **Cache-n** (the cache table of CPU number n). Most items read normal Linux files and need nothing special. **MSRs** needs root data: run the srs-root helper once (Menu -> Run as root) and SRS shows the cached result. Several AMD items need the optional `ryzen_smu`, `zenergy` or `k10temp` drivers. Many pages start with a title line ("srs-gui - ... on \MYPC - CachyOS ...") and some end with the motherboard name and BIOS version.

Contents

- AMD Ryzen Power Management `amd-cpu`
- AMD Fuses `amd-fus`
- AMD PM Table `amd-pmt`
- AMD PM Table 0 `amd-pmt-0`
- AMD PM table data @ second package/CCD `amd-pmt-1`
- AMD SMU `amd-smu`
- AMD Telemetry 0 `amd-tel-0`
- AMD SVI telemetry plane @ second die `amd-tel-1`
- AMD Thermal 0 `amd-thm-0`
- AMD THM registers @ second die `amd-thm-1`
- AMD Temperatures `amd-tmp`
- CPU Table `cpu`
- CPU Description `cpu-bits`
- CPU Clocks `cpu-clocks`
- CPU Export `cpu-export`
- CPUID `cpuid`
- CPUID Leaf 0 `cpuid-000`
- CPU Idle Levels `cpu-levels`
- CPU OC Data `cpu-oc-data`
- CPU Summary `cpus`
- CPU SMU `cpu-smu`
- CPU Speeds `cpu-speeds`

- CPU Timers `cpu-timer`
- CPU Tunables `cpu-tweak`
- CPU Voltages `cpu-volts`
- CPUID Features `features`
- Hypervisor `hypervisor`
- Microcode `microcode`
- MSRs `msrs`
- MSR Setup `msr-setup`
- IBM/Cyrix MSRs `msrs-ibm`
- NexGen NXOP `nexgen-nxop`
- Nodes `nodes`
- NUMA `numa`
- NUMA memory speed `numa-speeds`
- CPU power states `power-states`
- Processor Features Present `present`
- Processor `processor`
- Processors `processors`
- Speculation Control `speculation`
- Turbo Boost `turbo-boost`
- Xeon PIRs `xeon-pir`
- Cache Levels `cache`
- CPU Groups `group`

AMD Ryzen Power Management

`amd-cpu` - Linux data

In plain words. How the Linux `amd_pstate` driver controls the speed of each core on a modern AMD Ryzen CPU, and which cores the chip itself says are its best ones.

What you see.

- **Mode** - the amd_pstate mode: `active` (the CPU picks its own speed inside limits the OS sets), `guided`, or `passive` (the OS picks the speed). Tells you who is really in charge of the clock.
- A table, one row per logical CPU:

Column	Meaning	Why it matters
CPU	Linux CPU number (CPU-0, CPU-1 ...)	Which thread the row is about.
Highest perf	The top CPPC performance level this core can reach (a unitless number)	Higher means this core can boost higher.
Ranking	The "preferred core" rank the CPU reports	The scheduler puts busy work on the highest-ranked cores first.
Max freq	Highest clock the driver will allow	Your real boost ceiling.
Nonlinear	Lowest non-linear frequency	Below this speed, saving power gets much less efficient.

Column	Meaning	Why it matters
EPP	Energy Performance Preference hint (`performance`, `balance_performance`, `balance_power`, `power`)	Tells the CPU how much to favour speed over power saving.
HW prefcore	Whether the hardware supports preferred-core ranking	If `disabled`, all cores are treated as equal.

Healthy / typical values. On the author's Ryzen 7 7800X3D: Mode `active`, Highest perf 166 on every core, Max freq 5.05GHz, Nonlinear 2.98GHz, EPP `performance`, HW prefcore `enabled`. Rankings run from 166 to 196; CPU-1/CPU-9 and CPU-5/CPU-13 (the two threads of the same core) share the top rank 196.

Unusual values and what they suggest.

- The page says "amd_pstate driver not active": the older `acpi-cpufreq` driver is in use, or the CPU is not an AMD Zen 2 or newer. Add `amd_pstate=active` (or `guided`) to the kernel command line.
- Every Ranking is the same: preferred-core support is off in the BIOS or kernel.
- Max freq well below the advertised boost: boost is off, or a power or temperature limit is set.

Where Linux gets it. `/sys/devices/system/cpu/amd_pstate/status` and `/sys/devices/system/cpu/cpuN/cpufreq/amd_pstate_*` plus `energy_performance_preference`.

Background. CPPC (Collaborative Processor Performance Control) is an ACPI standard. Instead of a few fixed speed steps, the CPU offers a smooth scale of "performance levels" and the OS gives it a target range. The CPU then picks the exact clock many times per second.

AMD Fuses

`amd-fus` - Linux data

In plain words. Fuses are one-time settings burned into the chip at the factory (which cores are on, limits, and so on). The fuse registers themselves are not read, but since 2.4 this page shows the per-core factory values that Linux can read without root: how fast each core can go and AMD's quality ranking of the cores.

What you see.

- **amd_pstate driver** (active, passive, guided), **Preferred cores (kernel)** and **Preferred cores (CPU)** - whether Linux and the CPU use the core ranking.
- **Per-core factory values** - one row per logical CPU: **CPU**, **Core** (`-` when the core id is missing), **Highest perf** (the highest performance level), **Ranking** (the preferred-core ranking), **Max MHz** and **Lowest non-linear MHz**.
- **Best ranked CPUs** - the CPUs with the highest ranking, and the value.
- A plain-English note: the ranking is the factory quality order of the cores, set by AMD when the CPU was made and reported by the firmware; a higher number means the core reaches higher clocks, and Linux uses it to put busy tasks on the best cores ("preferred cores").
- **Fuse registers** - "not read": the physical fuse registers (CCD and core disable, part options) need SMN access.

Healthy / typical values. On the author's 7800X3D: amd_pstate active, preferred cores enabled; highest perf 166 on every core, rankings 166-196, best ranked CPUs 1, 5, 9, 13 (196); max 5053 MHz, lowest non-linear 2983 MHz. Both threads of a core share its ranking.

Unusual values and what they suggest. "Per-core data: not available": the CPU is not run by the amd_pstate driver (older CPUs, or `amd_pstate=disable`). All rankings equal, or preferred cores "disabled": the BIOS or the kernel turned the feature off (`amd_prefcore=disable`), or the CPU does not report it.

Where Linux gets it. `/sys/devices/system/cpu/cpu*/cpufreq/amd_pstate_highest_perf`, `amd_pstate_prefcore_ranking`, `amd_pstate_max_freq`, `amd_pstate_lowest_nonlinear_freq`, `amd_pstate_hw_prefcore`, `/sys/devices/system/cpu/amd_pstate/status` and `prefcore` (no root). The fuse registers would need `/sys/kernel/ryzen_smu_drv/smn` (root).

AMD PM Table

`amd-pmt` - Linux data

In plain words. A raw dump of the power-management table that the CPU's own little control processor (the SMU) keeps up to date: powers, clocks, voltages, temperatures and limits, all as numbers.

What you see.

- A summary line: table size in bytes, number of values, and a reminder that the meaning of each slot depends on the table version (shown in AMD SMU).
- Then rows of four values each:
  - **Offset** (purple, hex) - the byte position in the table, e.g. `0000`, `0010`.
  - **[n]** - the slot number (0, 1, 2 ...).
  - **value** - that slot read as a 32-bit floating-point number. Values that are not real numbers or are huge are shown grey, because they are almost certainly not a float at all.

Healthy / typical values. On the author's 7800X3D: 1704 bytes, 426 values, table version 0x00540104. You can spot patterns: slots around 5.25 look like clock limits in GHz, slots near 1.1-1.5 look like voltages, slots in the 40s-50s change with load and look like temperatures or watts. The values change every time you open the page because the SMU updates them constantly.

Unusual values and what they suggest.

- "ryzen_smu pm_table not available": the `ryzen_smu` kernel module is not installed or not loaded.
- "pm_table not readable as this user": the file permissions on your system only let root read it.
- All zeros: the driver loaded but the SMU has not filled the table yet, or the table version is not supported by the driver.

Where Linux gets it. `/sys/kernel/ryzen_smu_drv/pm_table` (from the out-of-tree `ryzen_smu` driver).

Background. The SMU (System Management Unit) is a small processor inside every Ryzen CPU. It runs its own firmware and makes the real decisions about boost clocks, voltages and power limits. Tools like Ryzen Master read this table to show live data. AMD does not publish the layout, so tools map it by version.

AMD PM Table 0

`amd-pmt-0` - Linux data

In plain words. The same power-management table as AMD PM Table, for the first (and on most desktops the only) CPU package.

What you see. Exactly the same layout as AMD PM Table: summary line, then offset, slot number and float value, four per row. Because the table is live, two openings a moment apart show slightly different numbers (on the author's PC slot [3] read 42.69 in one capture and 55.19 in the next).

Where Linux gets it. `/sys/kernel/ryzen_smu_drv/pm_table`.

AMD PM table data @ second package/CCD

`amd-pmt-1` - Not present on most PCs

In plain words. The power-management table of a second CPU package or die. Desktop Ryzen has one package and the driver gives one table, so there is nothing to show.

What you see. Status `Not present on this system` and **Checked**, the source SRS looked at (`/sys/kernel/ryzen_smu_drv/pm_table (single CCD/package)`). "Not present" is normal on single-socket PCs.

AMD SMU

`amd-smu` - Linux data - Needs root data

In plain words. Version information about the SMU (the CPU's built-in power-management processor) and the `ryzen_smu` driver that talks to it. SRS shows this part without root; the root flag is for the SMU mailbox commands, which SRS never sends.

What you see.

Field	Meaning	Why it matters
Driver version	Version of the `ryzen_smu` driver (`drv_version`)	Old drivers may not know new CPUs.
Codename	The driver's number for your CPU generation (`codename`)	Tells the driver which table layout to use.
SMU firmware version	SMU firmware version (the driver's `version` file), e.g. `4.84.4.0`	Changes with BIOS (AGESA) updates.
MP1 interface version	Version of the MP1 mailbox interface (`mp1_if_version`)	Driver/firmware compatibility.

Field	Meaning	Why it matters
PM table version	Layout version of the PM table, in hex and decimal (`pm_table_version`, a binary 32-bit value)	You need this to know what each PM table slot means.
PM table size	Size of the PM table in bytes (`pm_table_size`, a binary 64-bit value)	Should match the table dump.

A closing note says SRS is read-only and never sends SMU commands.

Healthy / typical values. On the author's 7800X3D: drv_version 0.1.7, codename 20, mp1_if_version 2, pm_table_version 0x00540104 (5505284), pm_table_size 1704. A field shows `-` when the driver does not provide it.

Unusual values and what they suggest. "ryzen_smu driver not loaded": install and load the `ryzen_smu` module (it is not part of the normal kernel).

Where Linux gets it. Files in `/sys/kernel/ryzen_smu_drv/`.

AMD Telemetry 0

`amd-tel-0` - Linux data

In plain words. How much power each core and the whole CPU package is using right now, worked out from the CPU's energy counters.

What you see. A table with one row per energy counter:

- **Plane** - the counter's label: `Ecore000`, `Ecore001` ... (one per physical core) and `Esocket0` (the whole package).
- **Power** - watts, measured by reading the counter twice 200 ms apart.
- **Energy** - total energy counted since the counter started, in kilojoules.

Healthy / typical values. On the author's 7800X3D at light load: cores 2.4-4.7 W each, socket 57.86 W. The socket figure is always larger than the cores added together, because it also counts the cache, memory controller, Infinity Fabric and I/O die.

Unusual values and what they suggest.

- "No zenergy / amd_energy hwmon driver loaded": install the `zenergy` module (or the older `amd_energy`).
- One core much higher than the rest at idle: a background task is pinned to it.
- Socket power high while every core is low: often memory or fabric overclocking, or many devices keeping the I/O die busy.

Where Linux gets it. The `zenergy` or `amd_energy` hwmon driver, `/sys/class/hwmon/hwmonN/energy*_input` (these read AMD's RAPL energy counters).

Background. RAPL (Running Average Power Limit) counters add up energy used, in tiny steps. Power is energy divided by time, so reading the counter twice and dividing by the gap gives watts.

AMD SVI telemetry plane @ second die

`amd-tel-1` - Not present on most PCs

In plain words. Power and voltage readings for a second die. Single-package PCs have no second plane.

What you see. Status `Not present on this system` and **Checked** (the source SRS looked at).

Background. SVI (Serial VID Interface) is the link the CPU uses to tell the motherboard's voltage regulator what voltage to give it, and to read back current and voltage.

AMD Thermal 0

`amd-thm-0` - Linux data

In plain words. The CPU's own temperature sensors, read through the `k10temp` driver.

What you see. One line per sensor the driver offers, label and temperature in degrees Celsius:

- **Tctl** - the control temperature that the CPU and fan curves use. On some models it includes a fixed offset, so it can read higher than the real die.
- **Tdie** - the real die temperature (only on models where it differs from Tctl).
- **Tccd1, Tccd2 ...** - one per CCD (core chiplet), the die that holds the cores.
- Other labels the driver adds are shown as-is; unlabelled ones as `temp1`, `temp2` ...

A note at the end explains Tctl and TccdN.

Healthy / typical values. On the author's 7800X3D (one CCD): Tctl 53.6°C, Tccd1 48.1°C. Idle on a desktop is usually 35-55°C. Under full load an X3D chip normally stays below about 89°C, the point where it starts holding clocks back.

Unusual values and what they suggest.

- Tctl near 90-95°C under load: cooler not seated well, pump failing, or fans not ramping.
- Tctl jumps quickly up and down by 10°C or more: normal for Ryzen at light load (single cores boosting).
- "k10temp not loaded": the module is missing or blocked; some boards also hide it behind another driver.

Where Linux gets it. `k10temp` hwmon: `/sys/class/hwmon/hwmonN/temp*_input` and `temp*_label`.

AMD THM registers @ second die

`amd-thm-1` - Not present on most PCs

In plain words. Thermal registers of a second CPU package. A single-socket PC has only one, so nothing is listed.

What you see. Status `Not present on this system` and **Checked** (`/sys/bus/pci/drivers/k10temp (single package)`).

AMD Temperatures

`amd-tmp` - Linux data

In plain words. The same CPU temperature list as AMD Thermal 0.

What you see. Identical to AMD Thermal 0: Tctl, Tdie (if present) and TccdN in °C, plus the explanatory note. The page refreshes every 2 seconds in the window.

Where Linux gets it. `k10temp` hwmon.

CPU Table

`cpu` - Linux data

In plain words. One line per logical CPU showing where it sits (node, package, core, thread), its IDs, and its current speed settings. The window refreshes it every 2 seconds.

What you see.

Column	Meaning	Why it matters
CPU	Linux CPU number	The name tools like `taskset` use.
Node	NUMA node the CPU belongs to	Matters on multi-node systems (see NUMA).
Pkg	Physical package (socket) number	0 on single-socket PCs.
Die	Die number inside the package	Linux topology die ID.
Core	Physical core number	Two CPUs with the same Core are SMT siblings.
Thread	0 = first thread of the core, 1 = its SMT sibling	Shows which CPU numbers share a core.
Affinity	The CPU's bit in a 64-bit affinity mask, in hex	Useful when setting masks by hand. Shown as zeros above CPU 63.
APIC	The CPU's APIC ID in hex	The hardware's own CPU number.
Current	Current clock speed	Changes all the time with load.
Minimum / Maximum	The lowest / highest clock the OS currently allows	Your active speed limits.
Governor	cpufreq governor (`performance`, `powersave`, `schedutil` ...)	The policy that picks speeds.
EPP	Energy Performance Preference	See AMD Ryzen Power Management.
Ranking	amd_pstate preferred-core rank	Higher = better core.

Column	Meaning	Why it matters
Highest	amd_pstate highest perf level	The core's boost potential.

Empty values show as `-`.

Healthy / typical values. On the author's 7800X3D: CPU-0 to CPU-7 are Thread 0 of cores 0-7 and CPU-8 to CPU-15 are their siblings (Thread 1). APIC IDs are even for first threads (0000, 0002 ...) and odd for siblings. Minimum and Maximum are both 5.05GHz because the owner pinned the minimum to the maximum; on a default setup Minimum is much lower (a few hundred MHz).

Unusual values and what they suggest.

- Minimum = Maximum: someone set a fixed speed (gaming tweak, tuned profile). Higher idle power.
- Maximum far below the CPU's boost speed: boost disabled or a power-saving profile is active.
- Current shows `-`: no cpufreq driver loaded (common in virtual machines).

Where Linux gets it. `/sys/devices/system/cpu/cpuN/topology/`, `cpufreq/`, `/proc/cpuinfo` (apicid).

Background. SMT (Simultaneous Multi-Threading, Intel calls it Hyper-Threading) lets one physical core run two threads at once. Linux shows each thread as its own "CPU".

CPU Description

`cpu-bits` - Linux data

In plain words. SIV shows the CPU description Windows keeps in the registry. On Linux, SRS shows the same kind of facts straight from `/proc/cpuinfo` for one CPU (CPU-0 unless another number is given).

What you see. A label/value list of these `/proc/cpuinfo` fields:

- **model name** - full CPU name. **vendor_id** - `AuthenticAMD` or `GenuineIntel`.
- **cpu family / model / stepping** - the CPU's generation numbers, in decimal.
- **microcode** - microcode revision loaded. **cpu MHz** - clock at the moment of reading.
- **cache size** - the L2 cache per core, as the kernel reports it.
- **physical id / siblings / core id / cpu cores** - package number, threads per package, this core's number, cores per package.
- **apicid / initial apicid** - APIC IDs now and at power-on (normally equal).
- **fpu / fpu_exception / wp** - basic x86 features (always `yes` on modern CPUs).
- **cpuid level** - highest standard CPUID leaf.
- **bugs** - known hardware weaknesses the kernel knows about (see Speculation Control).
- **bogomips** - a rough "busy loop" speed number. It is not a benchmark.
- **TLB size** - number of translation buffer entries (AMD only).
- **clflush size / cache_alignment** - cache line size in bytes.
- **address sizes** - physical and virtual address bits.
- **power management** - power features (`ts` thermal sensor, `hwpstate` hardware P-states, `cpb` core boost, and so on; unknown bits appear as `[13]`, `[14]`).

- **flags** - every CPU feature the kernel found (long list), wrapped to about 90 characters a line (2.4).
- **vmx flags** - Intel VT-x sub-features; left out when empty, as on AMD (2.4).

Healthy / typical values. On the author's 7800X3D: family 25, model 97, stepping 2, microcode 0xa60120c, cache size 1024 KB, 16 siblings, 8 cpu cores, address sizes 48 bits physical / 48 bits virtual, TLB size 3584 4K pages, bogomips 8820.31.

Unusual values and what they suggest. `siblings` equal to `cpu cores`: SMT is off. A `bugs` list that grows after a kernel update is normal: newly found issues get added.

Where Linux gets it. `/proc/cpuinfo`.

CPU Clocks

`cpu-clocks` - Linux data

In plain words. The current, average and allowed clock speed of every CPU. Same page as CPU Speeds; it refreshes every second in the window.

What you see. The same layout as CPU Speeds: the driver/boost line and the per-CPU table (Current, Average, Minimum, Maximum, HW Min, HW Max, Governor, EPP). Below it, since 2.2, **Stock and in effect now**: each clock at stock next to what it is now.

Row	Stock	In effect now
Base clock	100 MHz, the standard reference clock	Measured: the TSC rate x 100 / the CPU's nominal frequency from ACPI CPPC (once per run, 50 ms)
Core base	The CPU's nominal frequency (CPPC)	Nominal x base clock / 100
Core boost (max)	cpufreq's maximum (the kernel works it out at 100 MHz)	Maximum x base clock / 100
Core now (fastest)	The same multiplier at 100 MHz	cpufreq's current clock of the fastest core (already real)
FCLK, UCLK, MCLK	What the SMU reports	The same: these are shown, not measured

Difference gives MHz and percent where SRS measured. FCLK (Infinity Fabric), UCLK (memory controller) and MCLK (memory clock) come from the SMU's PM table through the ryzen_smu driver (read-only), for the Zen 4 desktop layout (Ryzen 7000, ryzen_smu codename 20) and only when the values make sense; elsewhere the row says why it is missing. Software cannot measure these clocks, so SRS does not claim a difference: on AM5 an eCLK drives the cores, and whether anything else follows it depends on the board's eCLK mode. The **Memory** line adds the DDR data rate (2 x MCLK), the UCLK:MCLK ratio (1:1 or 1:2) and FCLK.

Healthy / typical values. With the normal 100 MHz base clock every difference is about 0. The author's PC runs a 105 MHz eCLK: base clock 105.0 MHz (+5 %), core base 4201 -> 4410 MHz, boost 5053 -> 5305 MHz, FCLK 2133 MHz, UCLK 1500 MHz, MCLK 3000 MHz (DDR5-6000 at 1:2).

Where Linux gets it. cpufreq files in `/sys/devices/system/cpu/cpuN/cpufreq/`, the TSC (timed against the system clock), `/sys/devices/system/cpu/cpu0/acpi_cppc/nominal_freq`, and `/sys/kernel/ryzen_smu_drv/pm_table` (floats 70, 74 and 78 of the Zen 4 desktop table).

CPU Export

`cpu-export` - Linux data

In plain words. A tiny set of CPU values that other tools (LCD panels, on-screen displays, scripts) can read from a file SRS writes.

What you see.

- **CPU.Name** - CPU model name.
- **CPU.Clock.Max** - the highest current clock of any CPU at the moment of reading, in MHz.
- **CPU.Count** - number of logical CPUs.
- A note on how to write this to a file: `srs-gui -save=[cpu-export]=FILE`.

Healthy / typical values. On the author's PC: AMD Ryzen 7 7800X3D 8-Core Processor, 5115 MHz, 16.

Where Linux gets it. `/proc/cpuinfo` and `scaling_cur_freq` of every CPU.

CPUID

`cpuid` - Linux data

In plain words. A raw dump of every answer the CPU gives to the CPUID instruction. This is the source for most of the other pages in this group. It is for experts and for comparing two machines.

What you see. A title line (CPU number, CPU name, current speed), then one row per leaf and sub-leaf:

Column	Meaning
Leaf	The CPUID question number, in hex. 00000000-000000xx are standard leaves, 80000000-800000xx are extended leaves.
Sub	Sub-leaf number, for leaves that have several pages (cache, topology, XSAVE ...). Empty extra sub-leaves are skipped.
EAX, EBX, ECX, EDX	The four 32-bit registers of the answer, in hex.
ASCII	The same 16 bytes as text, in register order EAX-EBX-ECX-EDX. Non-printable bytes show as `.`.

The ASCII column makes text answers readable: leaves 80000002-80000004 spell the brand name. In leaf 0 the vendor text looks scrambled ("AuthcAMDenti") because the CPU stores it in the order EBX-EDX-ECX.

Healthy / typical values. On the author's 7800X3D: highest standard leaf 00000010, highest extended leaf 80000028, leaf 1 EAX 00A60F12. Many leaves are all zero: that just means "not used by this CPU".

Unusual values and what they suggest. Leaf 1 ECX bit 31 set (hypervisor bit) means you are inside a virtual machine. Very few leaves: an old CPU or a hypervisor that hides features.

Where Linux gets it. The CPUID instruction itself, run with the thread pinned to the chosen CPU (argument = CPU number).

Background. CPUID is an x86 instruction. Software puts a number (the leaf) in a register and the CPU answers in four registers with facts about itself: vendor, family, features, caches, topology.

CPUID Leaf 0

`cpuid-000` - Linux data

In plain words. The CPU's basic identity card from CPUID: who made it, its name and its exact generation.

What you see.

- **Maximum standard leaf** - highest normal CPUID leaf (hex).
- **Vendor string** - 12-letter maker code: `AuthenticAMD`, `GenuineIntel`, `HygonGenuine` ...
- **Maximum extended leaf** - highest extended leaf (hex).
- **Brand string** - the marketing name, built into the chip.
- **Signature (leaf 1 EAX)** - the raw hex signature, then the decoded family, model and stepping (decimal).

Healthy / typical values. On the author's PC: 00000010, AuthenticAMD, 80000028, "AMD Ryzen 7 7800X3D 8-Core Processor", 00A60F12 = family 25 model 97 stepping 2.

Where Linux gets it. CPUID leaves 0, 1, 80000000 and 80000002-80000004.

Background. Family 25 (19h) is AMD Zen 3 and Zen 4; model 97 (61h) is the Zen 4 "Raphael" desktop chip. Stepping is the silicon revision.

CPU Idle Levels

`cpu-levels` - Linux data

In plain words. The sleep levels (C-states) each core can drop into when it has nothing to do, and how much each level has been used since boot.

What you see.

- **Driver** - the cpuidle driver (`acpi_idle`, `intel_idle`, `amd_idle` ...) and its **governor** (the policy that picks a level: `menu`, `teo`, `ladder`, `haltpoll`).
- A table with one block per CPU, one row per state:

Column	Meaning	Why it matters
CPU	Shown on the first state of each CPU	Group header.
State	State name: POLL, C1, C2, C3 …	Deeper = more power saved, slower to wake.
Description	What the state does (e.g. `ACPI HLT`, `ACPI IOPORT 0x414`)	Shows how the state is entered.
Latency	Time to wake from it, microseconds	Deep states add this delay to the first work after idle.
Residency	Minimum time worth staying in it, microseconds	The governor skips a state if the idle gap is shorter.
Usage	How many times it was entered	Shows which states really get used.
Time	Total time spent in it, seconds	Where the core spends its idle time.

Healthy / typical values. On the author's 7800X3D: driver `acpi_idle`, governor `teo`; states POLL (0us), C1 HLT (0us), C2 (18us latency, 36us residency) and C3 (350us, 700us). Most idle time is in C2 and C3, which is what you want for low idle power.

Unusual values and what they suggest.

- Only POLL and C1: deep C-states are off in the BIOS ("Global C-state Control"), or the kernel was booted with `processor.max_cstate=1` / `idle=poll`. Idle power and temperature go up.
- C3 Usage stays 0: the state is disabled (see CPU power states) or wake-ups are too frequent for it.

Where Linux gets it. `/sys/devices/system/cpu/cpuidle/` and `/sys/devices/system/cpu/cpuN/cpuidle/stateM/`.

Background. A C-state is an idle power level. C0 is "running". C1 stops the clock to the core, deeper ones also cut voltage and flush caches. Waking from a deep state takes longer, which is why latency-sensitive setups sometimes limit them.

CPU OC Data

`cpu-oc-data` - Not present on most PCs - Needs root data

In plain words. Intel CPUs have an "overclocking mailbox" register that reports and sets overclocking limits. AMD CPUs do not have it.

What you see. Status `Not present on this system` and **Reason**: IA32_OC_MAILBOX is an Intel interface; on AMD look at AMD Ryzen Power Management and Turbo Boost instead.

CPU Summary

`cpus` - Linux data

In plain words. A short overview of the processor: name, maker, generation, how many cores and threads, and which speed driver Linux uses.

What you see.

- **Processor** - full model name. **Vendor** - maker code.
- **Family** - family, model and stepping in decimal with hex in brackets.
- **Topology** - packages, cores, logical processors, and threads per core.
- **Microcode** - loaded microcode revision. **Address sizes** - physical/virtual address bits.
- **BogoMIPS** - rough loop speed (not a benchmark). **TLB size** - AMD translation buffer size.
- **Power management** - power feature list from `/proc/cpuinfo`.
- **Frequency driver** - the cpufreq driver: `amd-pstate-epp`, `amd-pstate`, `acpi-cpufreq`, `intel_pstate` ...
- Board name and BIOS line at the bottom.

Healthy / typical values. On the author's PC: AMD Ryzen 7 7800X3D, Family 25 (0x19) Model 97 (0x61) Stepping 2, "1 package, 8 cores, 16 logical processors (2 per core)", Microcode 0xa60120c, Frequency driver amd-pstate-epp.

Unusual values and what they suggest. "1 per core" on a CPU that has SMT: SMT is off in the BIOS or with `nosmt`. Frequency driver `acpi-cpufreq` on a Zen 2+ Ryzen: amd_pstate is not enabled.

Where Linux gets it. `/proc/cpuinfo`, `/sys/devices/system/cpu/cpuN/topology/`, `scaling_driver`.

CPU SMU

`cpu-smu` - Linux data - Needs root data

In plain words. The same SMU version page as AMD SMU. SIV uses this item for SMU mailbox results; SRS shows the read-only version data and never sends mailbox commands.

What you see. Identical to AMD SMU: driver version, codename, SMU firmware version, MP1 interface version, PM table version and size, and the read-only note.

Where Linux gets it. `/sys/kernel/ryzen_smu_drv/`.

CPU Speeds

`cpu-speeds` - Linux data

In plain words. The clock speed of every CPU right now, plus the limits that control it. The window refreshes it every second.

What you see.

- **Driver** - the cpufreq driver and whether **boost** is on, off, or `-` (unknown).

- A per-CPU table:

Column	Meaning	Why it matters
CPU	Linux CPU number	
Current	Clock the kernel last saw	Jumps around with load.
Average	Average clock over a short recent window (needs a recent kernel)	Smoother than Current.
Minimum / Maximum	Allowed range set by the OS (scaling limits)	What policy currently allows.
HW Min / HW Max	Range the hardware supports	The absolute floor and ceiling.
Governor	Speed policy	`performance` = always aim high; `powersave` under amd-pstate-epp still boosts but follows EPP.
EPP	Energy Performance Preference	Speed vs power hint.

Healthy / typical values. On the author's 7800X3D: driver amd-pstate-epp, boost on; Current 4.2-5.1GHz under light load; HW Min 426MHz, HW Max 5.05GHz. Current can read a little above Maximum (5.11GHz vs 5.05GHz) because of rounding and a raised reference clock.

Unusual values and what they suggest.

- Every core stuck at a low Current under load: a thermal or power limit, or the `powersave` governor with `power` EPP.
- HW Max equal to the base clock: boost off in the BIOS.
- All columns `-`: no cpufreq driver (virtual machine, or the driver failed to load).

Since 2.2 the same **Stock and in effect now** table as CPU Clocks follows the per-CPU table (both items share one page).

Where Linux gets it. `/sys/devices/system/cpu/cpuN/cpufreq/` (`scaling_*`, `cpuinfo_*`, `cpuinfo_avg_freq`) and `/sys/devices/system/cpu/cpufreq/boost`.

Background. cpufreq is the Linux part that sets CPU speed. The *driver* talks to the hardware; the *governor* decides which speed to ask for.

CPU Timers

`cpu-timer` - Linux data

In plain words. Which hardware clock Linux uses to keep time, and how fast the CPU's own time counter (the TSC) runs.

What you see.

- **Clock source** - the timer Linux uses now (`tsc`, `hpet`, `acpi_pm`).
- **Available** - all usable timers.
- **TSC frequency** - TSC rate measured by SRS over 50 ms, in MHz.

- **TSC properties** - check boxes: **constant** (same rate at any CPU speed), **nonstop** (keeps counting in deep sleep), **known freq** (the kernel was told the exact rate), **RDTSCP** (the instruction that also returns the CPU number is available).
- **CLOCK_MONOTONIC res** - resolution of the system's steady clock, in nanoseconds.
- **Kernel HZ (USER_HZ)** - the tick unit the kernel uses when reporting times to programs.
- **Clock event** - the device that fires timer interrupts (`lapic`, `hpet` ...).

Healthy / typical values. On the author's PC: clock source `tsc`; available `tsc hpet acpi_pm`; TSC 4409.857 MHz; constant, nonstop and RDTSCP ticked; resolution 1 ns; USER_HZ 100; clock event `lapic`. The 7800X3D's TSC is normally 4.20 GHz; 4.41 GHz here comes from the board's reference clock being raised to 105 MHz (4.20 x 1.05 = 4.41).

Unusual values and what they suggest.

- Clock source `hpet` or `acpi_pm`: the kernel decided the TSC is unstable (look in `dmesg` for "clocksource tsc unstable"). Timing calls get much slower; games can stutter.
- TSC frequency very different from the base clock with no reference-clock change: a virtual machine, or firmware bug.

Where Linux gets it. `/sys/devices/system/clocksource/clocksource0/`, `/sys/devices/system/clockevents/`, `/proc/cpuinfo` flags, and SRS's own timing with the RDTSC instruction.

Background. The TSC (Time Stamp Counter) is a counter inside the CPU that goes up at a fixed rate. Reading it is very cheap, which makes it the best clock for an OS when it is stable.

CPU Tunables

`cpu-tweak` - Linux data - Needs root data

In plain words. A read-only view of the CPU settings you can change in Linux: speed policy, boost, SMT and CPU isolation. SRS only shows them; changing them needs root and a tool such as tuned or cpupower.

What you see. (each line appears only if the file exists)

- **Governor (cpu0)** and **Available governors**.
- **EPP (cpu0)** and **Available EPP** values.
- **Boost** - `on` or `off`.
- **SMT control** - `on`, `off`, `forceoff` or `notsupported`. **SMT active** - 1 if SMT is running.
- **Isolated CPUs** - CPUs kept away from normal tasks (`isolcpus=`), or `none`.
- **nohz_full CPUs** - CPUs that run without the regular timer tick, or `none`.
- **sched_autogroup** - 1 if the scheduler groups tasks by session (better desktop feel under load).
- A note: changes are left to tuned / cpupower / LACT.

Healthy / typical values. On the author's PC: governor `performance` (available: performance, powersave), EPP `performance` (the only option offered while the performance governor is active), boost on, SMT on and active, no isolated CPUs, sched_autogroup 1.

Unusual values and what they suggest. SMT control `forceoff`: the kernel was booted with `nosmt=force`, usually for security. Only `performance` in Available EPP: that is normal while the performance governor is selected; switch to `powersave` to see the full list.

Where Linux gets it. `/sys/devices/system/cpu/cpu0/cpufreq/`, `/sys/devices/system/cpu/smt/`, `/sys/devices/system/cpu/isolated`, `nohz_full`, `/proc/sys/kernel/sched_autogroup_enabled`.

CPU Voltages

`cpu-volts` - Linux data

In plain words. The CPU core voltage as measured by the motherboard's sensor chip.

What you see. One line per voltage input of a supported motherboard sensor chip (Nuvoton `nct*`, ITE `it87*`, ASUS `asus*`, or `zenpower`): chip name, input label, and volts with three decimals. `in0` is shown as "Vcore sense" when it has no label; `in1` is shown too; other unlabelled inputs are skipped. A note says per-core voltage (VID) is not available from Linux drivers, so the board reading is shown.

Healthy / typical values. On the author's PC: nct6799 in0 (Vcore sense) 1.096 V, in1 1.000 V. Ryzen 7000 Vcore usually sits between about 0.9 and 1.35 V and moves constantly with load and boost.

Unusual values and what they suggest.

- Nothing listed: no matching sensor driver is loaded (for many boards you need `nct6775` or `it87`).
- Vcore above about 1.40 V at all-core load: a manual overvolt or aggressive BIOS setting; check heat.
- A reading of 0 or a fixed, unchanging odd value: that input is not wired to Vcore on your board.

Where Linux gets it. Super-I/O hwmon drivers, `/sys/class/hwmon/hwmonN/in*_input` and `in*_label`.

Background. Vcore is the main supply voltage of the CPU cores. The CPU asks the board's voltage regulator for a voltage (VID); the board's sensor measures what actually arrives.

CPUID Features

`features` - Linux data

In plain words. A checklist of 177 well-known CPU features (instruction sets, security and virtualization features), each ticked if CPU-0 reports it.

What you see.

- A count line: "N of 177 features present on CPU-0 (CPU name)".
- Two columns of **Feature** (short name) and **CPUID Feature Description** with a tick box. Present features are in normal colour; missing ones are grey.

The list covers standard leaf 1 (e.g. SSE, SSE2, SSE4.2, AVX, AES, RDRAND), leaf 6 (thermal/turbo), leaf 7 (AVX2, AVX-512 family, BMI1/2, SHA, SMEP/SMAP, CET, speculation controls), and AMD extended leaves (SVM, SSE4A, LM 64-bit mode, NX, 1GB pages, Invariant TSC, CPB boost, NPT, AVIC, SME/SEV).

Healthy / typical values. On the author's 7800X3D: 120 of 177. AVX2 and most AVX-512 parts are present (Zen 4 has AVX-512). Intel-only items (VMX, SGX, HLE, RTM, EIST) and old AMD items (3DNow!, FMA4, XOP) are correctly absent. The leaf-7 IBRS/STIBP/SSBD lines are Intel's bits; AMD reports the same protections in the `AMD_IBRS`, `AMD_STIBP`, `AMD_SSBD` lines, which are ticked.

Unusual values and what they suggest.

- AVX or AVX2 missing on a CPU that should have it: inside a VM with a basic CPU model, or the OS has disabled the AVX state (OSXSAVE unticked).
- HVM ticked: you are running under a hypervisor.
- SVM/VMX present but virtualization tools fail: it is turned off in the BIOS (CPUID still shows it).

Where Linux gets it. The CPUID instruction (leaves 1, 6, 7, 80000001, 80000007, 80000008, 8000000A, 8000001F).

Background. Programs check these bits before using special instructions. If a game says "your CPU does not support AVX2", this page tells you whether that is true.

Hypervisor

`hypervisor` - Linux data

In plain words. Can this CPU run virtual machines well, and is this system itself a virtual machine?

What you see. A list of tick boxes for CPU-0:

- **SLAT** - Second Level Address Translation (Intel EPT or AMD RVI): fast memory handling for VMs.
- **HVM** - ticked if this OS is running under a hypervisor.
- **VT-x** and **EPT** - Intel's virtualization and its page tables.
- **AMD-V** (SVM), **RVI** (nested paging), **AVIC** (virtual interrupt controller) and **SEV** (encrypted VMs).

Then:

- **/dev/kvm** - `present (KVM available)` or `absent`.
- **IOMMU** - number of IOMMU groups, or `disabled / none`.
- **kvm_amd nested** - 1 if VMs inside VMs are allowed (AMD only).
- Board/BIOS line.

Healthy / typical values. On the author's PC: SLAT, AMD-V, RVI and AVIC ticked; SEV and HVM not; /dev/kvm present; 36 IOMMU groups; nested 1. That is a fully working KVM host.

Unusual values and what they suggest.

- AMD-V or VT-x unticked on a CPU that has it: virtualization (SVM Mode / Intel VT) is probably off in the BIOS. On some systems the flag stays ticked even when the BIOS has it off; then `dmesg` shows "disabled by BIOS" when KVM loads.

- /dev/kvm absent while AMD-V is ticked: the `kvm_amd` / `kvm_intel` module is not loaded.
- IOMMU `disabled / none`: IOMMU off in the BIOS or kernel; GPU passthrough to a VM will not work.
- SEV unticked on a desktop Ryzen: normal; SEV is an EPYC server feature.

Where Linux gets it. `/proc/cpuinfo` flags, `/dev/kvm`, `/sys/kernel/iommu_groups/`, `/sys/module/kvm_amd/parameters/nested`.

Background. An IOMMU (Input-Output Memory Management Unit) controls which memory each device may reach. IOMMU groups are the smallest sets of devices that can be handed to a VM together.

Microcode

`microcode` - Linux data

In plain words. The microcode version loaded on each CPU. Microcode is a small firmware patch for the processor, loaded by the BIOS or by Linux at boot.

What you see. A two-column table: **CPU** and **Version** (hex). `-` if the kernel does not report it.

Healthy / typical values. All CPUs show the same version. On the author's PC: 0xa60120c on all 16.

Unusual values and what they suggest.

- Different versions on different CPUs: a failed or partial late update; reboot.
- A version older than your BIOS or distro offers: install your distro's `amd-ucode` / `intel-ucode` package, or update the BIOS. New microcode fixes bugs and security issues.

Where Linux gets it. `/sys/devices/system/cpu/cpuN/microcode/version`.

MSRs

`msrs` - Linux data - Needs root data

In plain words. The raw values of a set of the CPU's internal control registers, for every CPU. Only root can read them, so SRS shows what the srs-root helper saved.

What you see. Without root data: **Status** `Needs root` and **Source** (`/dev/cpu/*/msr`, read by srs-root via Menu > Run as root). With root data: a table with **MSR** (register address, hex), **Name**, and one column per CPU with the 64-bit value in hex (`n/a` if unreadable). The registers are:

Name	What it holds
APIC_BASE	Where the interrupt controller lives; bit 8 marks the boot CPU.
MPERF / APERF	Two counters: fixed-rate and actual-rate cycles. Their ratio is the true average clock.
EFER	Extended features in use (64-bit mode, NX, SVM enabled).

Name	What it holds
HWCR	AMD hardware configuration (boost lock, TSC behaviour ...).
PSTATE_CURRENT_LIMIT / CONTROL / STATUS	Legacy P-state limit, requested and current state.
PSTATE_DEF_0..2	Definitions of the classic P-states (clock multiplier and voltage ID).
HW_PSTATE_STATUS	The hardware's current effective P-state.
CPPC_CAP1 / CPPC_ENABLE / CPPC_REQUEST	CPPC capabilities (highest/nominal/lowest perf), on/off, and the OS request (min/max/EPP).
RAPL_PWR_UNIT	Units for the energy counters.
CORE_ENERGY_STAT / PKG_ENERGY_STAT	Raw energy counters for the core and the whole package.

Under the table, **What the registers are** explains each one in plain English (2.4), and a note points at MSR Setup for the decoded values.

Healthy / typical values. On the author's PC most registers are identical on every CPU (EFER 0000000000009d01, CPPC_ENABLE 1). APIC_BASE differs only on CPU-0 (the boot CPU). MPERF, APERF, energy counters and CPPC_CAP1 differ per CPU, which is expected.

Unusual values and what they suggest. All `n/a`: the `msr` kernel module is not loaded (see MSR Setup), or Secure Boot / kernel lockdown blocks MSR access.

Where Linux gets it. `/dev/cpu/N/msr` (the `msr` kernel module), read by srs-root and cached.

Background. An MSR (Model Specific Register) is a CPU control register reached with the RDMSR and WRMSR instructions. They hold settings and counters that normal programs never see. SRS only reads them.

MSR Setup

`msr-setup` - Linux data - Needs root data

In plain words. How SRS gets at the CPU's model-specific registers (MSRs), and what the ones it read mean, in plain English. Everything comes from the root cache that **Menu > Run as root** writes once (`~/.cache/srs/root/msrs.txt`); SRS never writes any register.

What you see.

- **msr driver** - built into the kernel or a module, and whether `/dev/cpu/N/msr` is there.
- **cpuid driver**, **Running as root now**, **Access** (root only, read through srs-root, never written).
- **MSRs last read** (the time in `collected.txt`), **Cache** (the file), **In the cache** ("18 registers x 16 CPUs" - every raw value is on the **MSRs** item), **Refresh** (Menu > Run as root). No cache: how to get it.
- **Decoded** (CPU 0, raw hex in brackets): each **P-state** (enabled or not, and its MHz from AMD's PStateDef: CpuFid[7:0] / CpuDfsId[13:8] x 200), **Core Performance Boost** (HWCR CpbDis, bit 25), **AMD-V (SVM) in EFER** (SVME bit 12: on = the kernel or a hypervisor has enabled SVM), **CPPC**

(CPPC_ENABLE), the **RAPL** energy and time units, and (2.4) **Package power** and **Core power**: srs-root reads the energy counters twice, one second apart (`rapl.txt`), so the page shows the watts of the whole package and of each CPU while Run as root ran - a little above idle, since Run as root itself was busy. Live power: Power & Cost, Sensors > Power status.

- **CPPC capabilities per CPU** - CPPC_CAP1's highest, nominal, lowest nonlinear and lowest performance for every CPU, highest first with a rank: the CPUs the scheduler prefers (amd_pstate preferred cores).
- Voltages are not shown: the VID encoding of newer AMD CPUs (SVI3) could not be checked.

Healthy / typical values. On the author's 7800X3D (family 19h): P0 4200 MHz (`0x800000004ab248a8`), P1 3000 MHz, P2 not used; boost on; SVM on; CPPC enabled; package 38.4 W, cores 18 W in all while Run as root ran; CPU0 181 / 138 / 98 / 14, the best cores 196.

Unusual values and what they suggest. Boost off: BIOS "Core Performance Boost" or a tool turned it off. No cache: run Menu > Run as root once. `module not loaded`: srs-root loads it; kernel lockdown (Secure Boot) can block it.

Where Linux gets it. `/dev/cpu/N/msr` (read by srs-root as root), `/lib/modules/<kernel>/modules.builtin`, the cache `~/.cache/srs/root/msrs.txt`, `rapl.txt` (AMD: PKG_ENERGY_STAT and CORE_ENERGY_STAT read twice, one second apart) and `collected.txt`.

IBM/Cyrix MSRs

`msrs-ibm` - Not present on most PCs - Needs root data

In plain words. SIV can show the special registers of old IBM and Cyrix x86 CPUs (1990s). Modern AMD and Intel CPUs do not have them.

What you see. Status `Not present on this system` and **Checked** `n/a (non-AMD MSR set)`. "Not present" is the expected result on any modern PC.

NexGen NXOP

`nexgen-nxop` - Not present on most PCs

In plain words. NexGen made x86 CPUs in the mid-1990s (their design became the AMD K6). SIV can show their special NxOP data. No current PC has one.

What you see. Status `Not present on this system` and **Checked** `n/a (NexGen CPU)`.

Nodes

`nodes` - Linux data

In plain words. The same NUMA node table as NUMA.

What you see. Identical to NUMA: Node, CPUs, Memory, Free, Distances.

Where Linux gets it. `/sys/devices/system/node/`.

NUMA

`numa` - Linux data

In plain words. How memory and CPUs are grouped into nodes. Almost every desktop has one node, so all CPUs reach all memory at the same speed.

What you see.

Column	Meaning	Why it matters
Node	Node name (`node0`, `node1` ...)	
CPUs	CPUs in this node	Programs run fastest using memory from their own node.
Memory	RAM attached to this node	Should roughly match your installed RAM split across nodes.
Free	RAM free on this node right now	One full node can slow programs even if total free RAM is fine.
Distances	Relative cost to reach each node (10 = local)	20+ means going through another chip.

Healthy / typical values. On the author's PC: one node, `node0`, CPUs 0-15, 30.45GB memory, 3.44GB free at that moment, distance 10. Low Free RAM is normal on Linux because spare memory is used as cache.

Unusual values and what they suggest. Several nodes on a desktop: the BIOS has a "NPS" or "memory interleave" option set to split memory (some Threadripper/EPYC settings). Memory 0 on a node that has CPUs: a memory channel is empty.

Where Linux gets it. `/sys/devices/system/node/nodeN/cpulist`, `meminfo`, `distance`.

Background. NUMA (Non-Uniform Memory Access) means memory is attached to particular CPUs. Reaching "far" memory takes longer. Multi-socket servers and some large CPUs are NUMA; normal desktops are not.

NUMA memory speed

`numa-speeds` - Linux data

In plain words. SIV measures memory speed per NUMA node. SRS shows the node count and the result of the last memory latency test, and (2.3.1) lets you run that test right here: in the new main window the

tab carries the **Memory Latency** controls (**Largest block**, **Run**), with the same question before it starts, the load test terms and one test at a time. It never runs the test by itself.

What you see.

- **NUMA nodes** - count; with one node it adds "(uniform memory access - every CPU sees the same latency)".
- **Last Latency run** - when the shown run was made (the file's date and time), then its table of **Block** size, **Latency** in nanoseconds and the cache **Level** that block size fits in.
- If no test has been run yet: **Memory latency** "Not measured yet - press Run above" (Classic View: Benchmarks & Stress > Memory Latency).

Healthy / typical values. On the author's 7800X3D: 1 node. Latency about 1-4 ns up to 32KB (L1), about 3 ns up to 1MB (L2), and about 8.6-11 ns from 2MB to 64MB (the 96MB L3).

Unusual values and what they suggest. Latency jumps much earlier than the L3 size: other programs were busy during the run, or the L3 is shared with a busy neighbour. Re-run with the PC idle.

Where Linux gets it. `/sys/devices/system/node/` and the saved file `~/.cache/srs/bench-latency.txt`.

CPU power states

`power-states` - Linux data

In plain words. All the CPU's power levels in one place: performance levels (P-states), idle levels (C-states), and the whole-system sleep modes.

What you see.

Performance states (CPPC) - each line only if the file exists:

- **Highest perf (cpu0)** - top CPPC level (boost).
- **Nominal perf** - the level of the guaranteed base clock.
- **Lowest non-linear perf** - below this, power savings drop off.
- **Lowest perf** - absolute minimum level.
- **Nominal frequency** / **Lowest frequency** - the clocks for nominal and lowest levels.
- **Maximum (boost) frequency** - top clock from cpufreq.
- **Transition latency** - time to change speed, in ns (0 = the hardware does it itself).

Idle states (cpu0) - each C-state with its description and exit latency in microseconds. A disabled state is shown grey with "(disabled)".

System sleep states:

- **Supported** - from `/sys/power/state`: `freeze` (suspend-to-idle), `mem` (suspend to RAM), `disk` (hibernate).
- **Suspend type** - from `/sys/power/mem_sleep`; the one in [brackets] is used: `s2idle` or `deep` (S3).
- **Hibernate mode** - from `/sys/power/disk`; [bracketed] is active.

Healthy / typical values. On the author's 7800X3D: Highest perf 166, Nominal 138, Lowest non-linear 98, Lowest 14, nominal 4.20GHz, lowest 545MHz, boost 5.05GHz, transition 0 ns. Idle states POLL, C1, C2 (18 us), C3 (350 us). Sleep: freeze mem disk, suspend type `[deep]`, hibernate `[platform]`.

Unusual values and what they suggest.

- Suspend type only `s2idle`: the firmware offers no S3. Fine on most new laptops; on desktops it can mean the BIOS S3 option is off.
- `disk` missing from Supported: no swap big enough, or Secure Boot lockdown blocks hibernation.
- A C-state marked disabled: someone turned it off (tuning tools do this for lower latency).

Where Linux gets it. `/sys/devices/system/cpu/cpu0/acpi_cppc/`, `cpufreq/`, `cpuidle/`, `/sys/power/`.

Background. A P-state (performance state) is a speed/voltage level while running. A C-state is an idle level. ACPI S-states (S3 = sleep, S4 = hibernate) are for the whole machine.

Processor Features Present

`present` - Linux data

In plain words. Windows has a function that answers "does this CPU support feature X?" for a fixed list of features. SIV shows that list; SRS shows the same list using the Linux feature flags.

What you see. CPU-0's name, then a table:

- **PF** - the Windows feature number.
- **Windows feature (SIV)** - the Windows name (e.g. `PF_AVX2_INSTRUCTIONS_AVAILABLE`), with a tick if present.
- **Linux flag** - the `/proc/cpuinfo` flag SRS checks for it (e.g. `avx2`). A flag shown with a minus sign (`-fpu`) means "ticked when this flag is missing".

Healthy / typical values. On the author's PC everything is ticked except PF_FLOATING_POINT_PRECISION_ERRATA (the 1994 Pentium FDIV bug), PF_FLOATING_POINT_EMULATED and PF_3DNOW_INSTRUCTIONS_AVAILABLE. That is normal for any modern CPU.

Unusual values and what they suggest. PF_NX_ENABLED unticked: NX (no-execute) is off in the BIOS, which is a security risk. PF_VIRT_FIRMWARE_ENABLED unticked: virtualization may be off in the BIOS (SRS checks the `svm` flag for this line).

Where Linux gets it. `/proc/cpuinfo` flags of CPU 0.

Processor

`processor` - Linux data

In plain words. A short card for each CPU: name, generation, where it sits, its speed, its caches and its main instruction sets. The **CPU-n** panel buttons open one CPU; the menu item shows all.

What you see. For each CPU:

- **CPU-n**, model name, then "Family X Model X Stepping n" (family and model in hex, no `0x`) and **APIC p.c.t** (package.core.thread) with `(core)` for a first thread or `(logical)` for an SMT sibling.
- Current speed, e.g. "4.96GHz Processor".
- For first threads only, one line per cache level: name (L1 Data, L1 Code, L2 Unified, L3 Unified), ways, line size, size, sets, and how many CPUs share it with their numbers. On CPU-0 the size is written as "count x size" (e.g. "8 x 32KB") to show the total number of such caches in the chip.
- For first threads, a line with the vendor and key features in brackets: `MMX` or `MMX+`, `SSE4.2`, `XD` (no-execute), `SVM` or `VMX` (virtualization), `AVX512` or `AVX2`.

Healthy / typical values. On the author's 7800X3D, CPU-0: Family 19 Model 61 Stepping 2 (hex for 25/97), APIC 0.0.0 (core); L1 Data and L1 Code 8 x 32KB, 8-way, 64 sets, shared by CPUs 0 and 8; L2 8 x 1MB, 2048 sets; L3 1 x 96MB, 16-way, shared by 16 CPUs (0-15); `[MMX+ SSE4.2 XD SVM AVX512]`.

Unusual values and what they suggest. An L1/L2 cache "shared by 1 CPU": SMT is off. A smaller L3 than the spec sheet: on multi-CCD chips each CCD has its own L3, so you see one CCD's share.

Where Linux gets it. `/proc/cpuinfo`, `/sys/devices/system/cpu/cpuN/cache/indexM/`, topology, cpufreq.

Processors

`processors` - Linux data

In plain words. The main processor page (the **Processors** panel button). It shows what kind of CPU system you have and lists every logical CPU with its IDs.

What you see.

Capabilities - four tick-box lines with a count and a legend:

- **Multi-processor System** - ticked if there is more than one physical package (|P|).
- **Hybrid-processor System** - mixed big/little cores; SRS always shows this unticked with 1 logical package (|e|).
- **Multi-core Processor** - number of cores (|C|) and cores per package.
- **Multi-threading Cores** - number of logical processors (|L|) and threads per core.

Then a note naming the board, and a table:

Column	Meaning
Processors	CPU name with a marker: `|P|` the first CPU, `|C|` the first thread of a core, `|L|` an SMT sibling.
APIC (first)	package.core.thread ID.
APIC (second)	The hardware APIC ID in hex.

Column	Meaning
Node	NUMA node (always 0 in SRS).
Pkg	Physical package.
N	Linux CPU number.
CPU Affinity Mask	This CPU's bit as a 64-bit hex mask.
Speed	Current clock.
ACPI	The ACPI device name of the CPU (e.g. `ACPI0007:00`).
CPU Device	The ACPI path of the CPU object in the firmware tables (e.g. `\_SB_.PLTF.C000`).

Board/BIOS line at the end.

Healthy / typical values. On the author's 7800X3D: 1 package (unticked multi-processor), 8 cores, 16 logical processors, 2 per core. CPU-0 to CPU-7 are `|P|` / `|C|` (thread 0), CPU-8 to CPU-15 are `|L|`.

Unusual values and what they suggest. Missing ACPI names (`-`): the firmware does not link CPUs to ACPI objects (common in VMs). Fewer CPUs than the spec: some disabled in the BIOS, or `maxcpus=` / `nosmt` on the kernel command line (compare with CPU Groups).

Where Linux gets it. `/sys/devices/system/cpu/cpuN/topology/`, `firmware_node`, `/proc/cpuinfo`, cpufreq.

Background. The APIC (Advanced Programmable Interrupt Controller) ID is the number the hardware uses for each thread. Linux CPU numbers are just the order Linux brought them up.

Speculation Control

`speculation` - Linux data

In plain words. SIV shows the Windows "speculation control" settings. On Linux, SRS shows the kernel's own list of CPU security weaknesses (Spectre, Meltdown and friends) and whether each one is fixed on your machine.

What you see.

- One line per known issue, as the kernel names it (e.g. `spectre_v1`, `spectre_v2`, `meltdown`, `retbleed`, `spec_rstack_overflow`, `tsa`, `vmscape` ...), and the kernel's status text:
  - green **Not affected** - this CPU does not have the weakness.
  - orange **Mitigation: ...** - the weakness exists and the kernel is protecting against it.
  - red **Vulnerable** - the weakness exists and is not protected.
- **Kernel command line** - the boot options, so you can see if mitigations were turned off.

Healthy / typical values. On a default system every line says "Not affected" or "Mitigation: ...". On the author's PC several lines say "Vulnerable" because the owner boots with `mitigations=off` for a little more speed. That is a deliberate choice, not a fault.

Unusual values and what they suggest. "Vulnerable" without `mitigations=off` in the command line: old microcode (see Microcode) or an old kernel. "Vulnerable" on a shared or internet-facing machine is a real risk; on a single-user gaming PC it is a trade-off some people accept.

Where Linux gets it. `/sys/devices/system/cpu/vulnerabilities/*` and `/proc/cmdline`.

Background. Speculative execution means the CPU guesses ahead to run faster. Researchers found ways to leak data through those guesses. Mitigations close the leaks at some cost in speed.

Turbo Boost

`turbo-boost` - Linux data

In plain words. Whether the CPU is allowed to boost above its base clock, and how high it can go.

What you see. (lines appear only when the file exists)

- **CPB (CPUID 80000007 EDX bit 9)** - ticked if the CPU supports Core Performance Boost.
- **cpufreq boost** - `enabled` (green) or `disabled` (red).
- **amd_pstate status** - `active`, `guided` or `passive`.
- **amd_pstate prefcore** - whether preferred-core scheduling is on.
- **Maximum boost frequency** - amd_pstate's top clock.
- **Lowest non-linear frequency** - see AMD Ryzen Power Management.
- **cpuinfo_max_freq** - top clock as cpufreq reports it.

Healthy / typical values. On the author's 7800X3D: CPB ticked, boost enabled, amd_pstate active, prefcore enabled, maximum 5.05GHz, non-linear 2.98GHz, cpuinfo_max_freq 5.05GHz.

Unusual values and what they suggest. Boost disabled: off in the BIOS ("Core Performance Boost"), or turned off by a power profile. Maximum equal to the base clock (4.20GHz on this chip): boost off.

Where Linux gets it. CPUID leaf 80000007, `/sys/devices/system/cpu/cpufreq/boost`, `/sys/devices/system/cpu/amd_pstate/`, `cpu0/cpufreq/`.

Background. "Turbo Boost" is Intel's name and "Core Performance Boost" (CPB) is AMD's for the same idea: run above the base clock when power and temperature allow.

Xeon PIRs

`xeon-pir` - Not present on most PCs

In plain words. Some older Intel Xeon chips had a small Processor Information ROM (PIR) with factory data. Desktop and AMD CPUs do not.

What you see. Status `Not present on this system` and **Checked** `n/a (Intel Xeon only)`.

Cache Levels

`cache` - Linux data

In plain words. A table of the CPU's caches: size, layout and which CPUs share each one. The **Cache-n** panel buttons show the table for CPU number n.

What you see. Title "Cache Levels of CPU-n", then:

Column	Meaning	Why it matters
Cache	Level and type: L1 Data, L1 Code, L2 Unified, L3 Unified	L1 is fastest and smallest, L3 slowest and biggest.
Size	Size of one such cache (K = kilobytes)	Bigger caches hold more of a program's working data.
Ways	Associativity	How many places a line can go; more = fewer collisions.
Line	Line size in bytes	The unit the cache moves data in (64 on modern CPUs).
Sets	Number of sets	Size / (ways x line).
Shared by	How many CPUs share it	2 = shared by the two threads of one core; all = whole chip.
CPUs	Which CPUs share it	

A note points to Benchmarks & Stress > Latency for measured speeds.

Healthy / typical values. On the author's Ryzen 7 7800X3D, CPU-0: L1 Data 32K and L1 Code 32K (8-way, shared by CPUs 0,8), L2 1024K (8-way), L3 98304K (96MB, 16-way, 98304 sets), shared by 16 CPUs (0-15). The huge L3 is the "3D V-Cache" that the X3D name refers to.

Unusual values and what they suggest. No rows: the kernel has no cache info (some VMs). L3 shared by fewer CPUs than you have on a multi-CCD chip: normal, each CCD has its own L3.

Where Linux gets it. `/sys/devices/system/cpu/cpuN/cache/indexM/` (`level`, `type`, `size`, `ways_of_associativity`, `coherency_line_size`, `number_of_sets`, `shared_cpu_list`).

Background. A cache is a small fast memory on the CPU that keeps copies of recently used RAM. L1 is inside each core, L2 is per core, L3 is shared. A cache hit takes a few nanoseconds; going out to RAM takes 60-80 ns or more.

CPU Groups

`group` - Linux data

In plain words. Windows splits large systems into "processor groups" of up to 64 CPUs. Linux has no such groups; SRS shows the Linux CPU sets instead (the **Groups** panel button).

What you see.

- **online** - CPUs running now. **offline** - CPUs switched off (`none` if all are on).
- **possible** - CPUs the kernel could ever bring up. **present** - CPUs physically there.
- **isolated** - CPUs kept away from normal tasks. **nohz_full** - CPUs without the regular timer tick.
- **kernel_max** - the highest CPU number this kernel supports.
- **This process affinity** - the CPUs SRS itself is allowed to run on.

Healthy / typical values. On the author's PC: online, possible and present all 0-15; offline, isolated and nohz_full `none`; kernel_max 15; affinity 0-15.

Unusual values and what they suggest.

- Offline CPUs: turned off by hand (`echo 0 > .../online`), by `nosmt`, or by `maxcpus=`.
- possible larger than present: normal on servers and many desktops (room for hot-plug).
- Process affinity smaller than online: SRS was started under `taskset` or a restricted cgroup.

Where Linux gets it. `/sys/devices/system/cpu/online`, `offline`, `possible`, `present`, `isolated`, `nohz_full`, `kernel_max`, and the process's own `sched_getaffinity`.

Machine / BIOS / Memory (SRS Section Reference)

Back to the section index - Glossary

Where to find these items (SRS 2.0). In the main window they are on two pages: **Memory** (Alt+3: the memory sticks, SPD, timings, memory map, NUMA) and **Motherboard & BIOS** (Alt+4: everything else). Each page's **All N items** lists the rest. You can also type an item's name or id into the search box (Ctrl+F). The rest of this introduction describes the classic view (Menu -> Classic View), with its Menu and panel buttons.

This group describes the computer itself rather than any one add-in card: the motherboard and its firmware (BIOS / UEFI), the memory sticks and how memory is laid out, the ACPI tables and devices the firmware hands to the operating system, the TPM security chip and the CMOS clock. Open it in SRS with **Menu -> Machine / BIOS / Memory -> item**, or use the **Machine** panel button and its sub-buttons **DIMMs** and **Status**. Most items work as a normal user. Items marked "Needs root data" are only complete after you run **Menu > Run as root (srs-root)** once. That helper saves four read-only copies that SRS then reads: `dmidecode.txt` (full SMBIOS data: serials, UUID, CPU socket), `spd.txt` (memory-stick SPD decode), `acpi-tables/` (copies of the ACPI tables) and `iomem.txt` (the I/O memory map). The item Machine options shows which of these sources are present on your PC.

All examples come from the author's PC: an ASUS TUF GAMING X870-PRO WIFI7 W NEO board with an AMD Ryzen 7 7800X3D, 2 x 16 GB DDR5 and BIOS 1805.

A few words first

Some ideas come up again and again in this group. Read these once and the rest is easier.

- **ACPI** (Advanced Configuration and Power Interface) is how the firmware describes the board to the operating system: which devices exist, how to power them on and off, how interrupts are wired. It comes as a set of binary **tables**, each with a four-letter signature such as `DSDT` or `APIC`.
- The **ACPI namespace** is a tree of named objects built from those tables. Paths look like `\_SB_.PCI0.GPP0` ("System Bus, PCI root 0, general purpose port 0"). Each device has a **HID** (hardware ID) such as `PNP0A08` (a PCI Express root bridge) or `AMDI0030` (an AMD GPIO controller).
- **_STA** is the ACPI "status" value of a device. SRS prints it as a hex number plus four letters: **P** present, **E** enabled, **S** shown in the user interface, **F** functioning. `0x0F PESF` means all four are true. `0x00 ----` means the firmware hides the device (normal for unused controllers). A `-` means the device has no _STA method, which the operating system treats as "present and working".
- **SMBIOS** (System Management BIOS), also called **DMI** (Desktop Management Interface), is a second firmware table set. It is a plain inventory: board name, BIOS version, memory slots, serial numbers. Linux shows most of it in `/sys/class/dmi/id`; the serial numbers and some detail are root-only.
- **UEFI** (Unified Extensible Firmware Interface) is the modern replacement for the old PC BIOS. Most people still say "BIOS" for both.

- **srs-root cache**: root-only data that the srs-root helper copied for SRS. SRS itself never runs as root and never writes to hardware (the one exception is ACPI method evaluation, a tool for experts that runs a method you type, through srs-root).

Contents

- ACPI PCI root buses `acpi-buses`
- ACPI devices `acpi-dev`
- ACPI embedded controller `acpi-ec`
- ACPI method evaluation `acpi-eval`
- ACPI HAL `acpi-hal`
- ACPI interrupts and GPEs `acpi-int`
- ACPI IPF (Intel Platform Framework) methods `acpi-ipf`
- ACPI ICD0001 methods `acpi-itm`
- ACPI methods `acpi-methods`
- ACPI extension (PDX) `acpi-pdx`
- ACPI PCI buses and ports `acpi-ports`
- ACPI power resource methods `acpi-pwr`
- ACPI SPB (serial peripheral bus) methods `acpi-spb`
- ACPI tables `acpi-tables`
- ACPI Toshiba test `acpi-test`
- ACPI TFP INT3403 methods `acpi-tfp`
- ACPI TMS INT340E methods `acpi-tms`
- ACPI TOS1900 methods `acpi-tos`
- ACPI TPM `acpi-tpm`
- ACPI WMI (PNP0C14) methods `acpi-wmi`
- APIC (MADT) summary `bios-apic`
- ASPT table summary `bios-aspt`
- DMAR table summary `bios-dmar`
- FACP (FADT) summary `bios-facp`
- PCI IRQ routing ($PIR) `bios-irq`
- BIOS maker `bios-make`
- MPS table summary `bios-mps`
- MSDM table summary `bios-msdm`
- OEMB table summary `bios-oemb`
- BIOS option ROMs `bios-option`
- SDEV table summary `bios-sdev`
- SFI summary `bios-sfi`
- SLIC table summary `bios-slic`
- SLIT table summary `bios-slit`
- SRAT table summary `bios-srat`
- BIOS / SMBIOS tables `bios-tables`
- CMOS / RTC status `cmos-status`
- Dell BIOS `dell-bios`
- DIMMs `dimms`
- DIMM status/temperatures `dimm-status`

- Firmware `firmware`
- Firmware interface table `f-w-table`
- Hardware summary `hardware`
- HECI (Intel ME) status `heci-status`
- Machine `machine`
- Machine options `machine-opt`
- Memory `memory`
- Memory timings `mem-tweak`
- Overview `overview`
- Physical memory map `physical`
- SPD (DIMMs) `spd`
- SPD thermal `spd-temp`
- System CMOS `system-cmos`
- System EBDA `system-ebda`
- BIOS IDs `system-ids`
- TPM device `tpm-device`
- UEFI data `uefi-data`
- UEFI information `uefi-info`

ACPI PCI root buses

`acpi-buses` - Linux data

In plain words. Shows where the PCI Express "tree" of the PC starts, and how every PCI bus hangs off a bridge that the firmware has named in ACPI.

What you see. Two tables.

First table, one row per PCI root bridge (HID `PNP0A08`, `PNP0A03` or `ACPI0016`):

Field	Meaning	Why it matters
Device	ACPI device name, e.g. `PNP0A08:00`	`PNP0A08` = PCI Express root bridge, `PNP0A03` = older PCI root
Path	ACPI namespace path, e.g. `\_SB_.PCI0`	The name the firmware's own code uses for it
UID	Unique ID inside that HID	Tells several root bridges apart (servers often have more than one)
Physical node	The Linux device it became, e.g. `pci0000:00`	Links the ACPI view to the Linux PCI view
Status	`_STA` value and letters	`0x0F PESF` = present and working

Second table, one row per PCI bus Linux knows (from `/sys/class/pci_bus`):

Field	Meaning	Why it matters
PCI Bus	Domain and bus number, e.g. `0000:05`	Bus numbers are what `lspci` addresses start with

Field	Meaning	Why it matters
Bridge	The PCI bridge that creates this bus, or `root`	Shows the parent of each bus
Bridge ACPI path	ACPI name of that bridge, e.g. `\_SB_.PCI0.GPP7`	Ties a bus to a named slot or chipset port
Bus range	Secondary-subordinate bus numbers behind the bridge, e.g. `05-0C`	Everything below that bridge lives in this range

Healthy / typical values. One root bridge `PNP0A08:00 \_SB_.PCI0 ... 0x0F PESF` on a desktop. On the author's board the chipset hangs off `GPP7` (buses `05-0C`), the USB4 controller off `GPP8` (buses `0D-74`, a big range reserved for hot-plugged Thunderbolt devices) and the graphics card off `GPP0` through two extra switch levels (`SWUS`, `SWDS`).

Unusual values and what they suggest.

- A bridge with an empty ACPI path: the firmware did not describe it. Usually harmless, but power management and hot-plug for that branch may then fall back to generic code.
- A very wide bus range (like `0D-74`) on a USB4 / Thunderbolt port is normal: the firmware reserves bus numbers for devices you might plug in later.
- More than one root bridge on a desktop: rare; normal on multi-socket servers.

Where Linux gets it. `/sys/bus/acpi/devices/PNP0A08:*` (path, uid, status, physical_node) and `/sys/class/pci_bus/*` plus `/sys/bus/pci/devices/*/firmware_node` and `secondary_bus_number` / `subordinate_bus_number`.

Background. A **PCI bridge** (shown elsewhere as "PtP", PCI-to-PCI) creates a new bus number behind it. Every slot, M.2 socket and chipset port is really a bridge. The GPU's own switch chips (`SWUS` = switch upstream, `SWDS` = switch downstream) are why a graphics card often appears three bus numbers deep.

ACPI devices

`acpi-dev` - Linux data

In plain words. The full list of devices the firmware describes in ACPI, with their status and which Linux driver picked each one up.

What you see. One row per ACPI device, sorted by name:

Field	Meaning	Why it matters
Device	Linux name, HID plus instance, e.g. `AMDI0030:00`	Unique key for the detail view
HID	Hardware ID, e.g. `PNP0C14` (WMI), `MSFT0101` (TPM 2.0)	Tells you what kind of device it is
_STA	Status hex value + P/E/S/F letters, or `-` if no _STA method	Whether the firmware says it is present and working
Path	ACPI namespace path, e.g. `\_SB_.PCI0.SBRG.EC0_`	The firmware's name for it

Field	Meaning	Why it matters
Physical node / driver	The Linux device and driver bound to it	Empty driver = nothing in Linux claimed it

The footer gives the device count and repeats the meaning of the _STA letters.

Detail view (`acpi-dev:<device>`, e.g. `acpi-dev:PNP0C09:00`): every attribute Linux exposes for that device, when present: `path`, `hid`, `modalias`, `uid`, `adr` (bus address), `status`, `description`, `power_state`, `real_power_state`, `resource_in_use`, then `physical_node`, `driver`, and one `child` line per child device.

Healthy / typical values. On the author's PC: 258 ACPI devices. The 16 `ACPI0007` rows are the CPU threads (`\_SB_.PLTF.C000` to `C00F`, bound to `processor`). `PNP0A08:00` is the PCI root, `PNP0B00:00` the CMOS clock (`rtc_cmos`), `PNP0C09:00` the embedded controller (`acpi-ec`), `MSFT0101:00` the TPM (`tpm_crb_acpi`), `AMDI0030:00` the GPIO controller (`amd_gpio`). Many rows show `0x00 ----`: spare I2C, I3C and UART controllers the board does not use.

Unusual values and what they suggest.

- `0x0B PE-F` (present, enabled, not shown, functioning) is normal for interrupt link devices (`PNP0C0F`, `LNKA`..`LNKH`) and the power button.
- A device you expect (for example a touchpad on a laptop) that shows `0x0F PESF` but has no driver: Linux is missing a driver or a kernel option for it.
- A device with `0x00 ----` that you expected to work: it may be switched off in the BIOS setup.

Where Linux gets it. `/sys/bus/acpi/devices/*` (`hid`, `status`, `path`, `physical_node`, `physical_node/driver`).

Background. HID prefixes tell you who defined the ID: `PNP` and `ACPI` are industry-standard, `LNX` are Linux-internal pseudo-devices (`LNXSYSTM` is the root, `LNXPOWER` a power resource), and vendor IDs such as `AMDI` (AMD), `MSFT` (Microsoft), `RTK` (Realtek) and `ASMT` (ASMedia) are listed on their own in ACPI extension (PDX).

ACPI embedded controller

`acpi-ec` - Linux data - Needs root data

In plain words. The embedded controller (EC) is a small helper microcontroller on the board. On laptops it runs the keyboard, battery and fans; on desktops it is usually much simpler. This item shows whether ACPI describes one and how Linux talks to it, and (2.4, opt-in) a copy of its 256 registers.

What you see.

Field	Meaning	Why it matters
`PNP0C09:00` (first line)	ACPI path of the EC	Confirms an EC is described
_STA	Status of the EC	`-` = no _STA method, treated as present

Field	Meaning	Why it matters
UID	Unique ID	Usually `-` or `0`
Driver	Linux driver	`acpi-ec` is the normal one
ec_storm_threshold	How many EC events in a row before Linux switches to polling	Protects against "GPE storms" from buggy firmware
ec_busy_polling	Whether Linux busy-waits for EC replies (`Y`/`N`)	`Y` is a workaround on some laptops
ec_max_queries	How many EC queries may be outstanding	Tuning value, default 16
ec_sys module	Whether the `ec_sys` debug module is loaded	Needed for an EC register dump
EC register dump	Off: "Menu > Options > Read the embedded controller, then Run as root". On: a hexdump of the 256 registers from the root cache	The raw EC state, for experts

If there is no EC, the item says "No ACPI EC (PNP0C09)".

The register dump is opt-in (2.4). Menu > Options > Read the embedded controller (expert, read-only; from the next Run as root) is off by default and asks first: on a few machines (mostly laptops) reading certain EC registers clears a pending event, such as a battery or key notification, which may then be missed once. When it is on, **Run as root** loads the kernel's `ec_sys` module with `write_support=0` (read-only), copies the 256 registers once into `~/.cache/srs/root/ec.bin`, and unloads the module again if it was not loaded before.

Healthy / typical values. On the author's desktop: `\_SB_.PCI0.SBRG.EC0_`, driver `acpi-ec`, storm threshold 8, busy polling N, max queries 16, `ec_sys` not loaded, register dump off.

Unusual values and what they suggest. No EC on a laptop is very unusual and means battery and lid events will not work. Changed `ec_*` values usually come from a kernel command-line workaround (`acpi.ec_*=`); check your boot loader settings.

Where Linux gets it. `/sys/bus/acpi/devices/PNP0C09:00` and `/sys/module/acpi/parameters/ec_*`; the dump from `/sys/kernel/debug/ec/ec0/io` (root, `ec_sys` module), copied by srs-root into `ec.bin`.

ACPI method evaluation

`acpi-eval` - Linux data - Needs root data

In plain words. ACPI methods are small programs inside the BIOS's ACPI tables. Since 2.4 **Tools > ACPI method evaluation** can run ("evaluate") one that you type, as root, and show its result - for experts. This page shows whether the kernel module that does it is there, where to start, and the last evaluations.

What you see.

- **acpi_call** - whether the third-party `acpi_call` kernel module is loaded (`/proc/acpi/call`). "not loaded" is normal: srs-root loads it when you evaluate a method (Arch package `acpi_call-dkms`).
- **Evaluate a method** - where: Tools > ACPI method evaluation (it runs BIOS code as root and can change hardware state).
- **Method names** - where to find them: ACPI methods (the methods in this PC's DSDT / SSDTs) and ACPI devices (_STA, _HID, _UID, _ADR per device).
- After evaluations: the last 30 lines of the log `~/.cache/srs/root/acpi-calls.txt` (time, the method with its arguments, the result).

How evaluation works. The first time, a warning explains the risk and asks once (**Yes, I understand** / Cancel; `Accepted ACPI Evaluation` in `SRSDEF.ini`). The window then takes a **Method** (an ACPI name path such as `\_SB.PCI0.GPP0._STA`) and up to 7 **Arguments**: numbers (decimal or `0x` hex), simple `"strings"` (letters, digits, space, `.`, `_`, `-`) or buffers written `b` followed by hex bytes (`b0102`). **Evaluate** asks for the root password (pkexec) and runs `srs-root --acpi-eval`, which checks the name and every argument again as root, loads `acpi_call`, evaluates the method once and logs it. The result is shown as `acpi_call` returns it.

Unusual values and what they suggest. "the acpi_call module is not installed": install `acpi_call-dkms` (Arch) or your distribution's acpi_call package. "not an ACPI method path" or "argument not allowed": the input did not pass the checks; nothing was run.

Where Linux gets it. `/proc/acpi/call` (the `acpi_call` module), written only by `srs-root --acpi-eval` for a method you typed; the log `acpi-calls.txt`.

Background. An ACPI **method** is a small program written in AML (ACPI Machine Language) inside the firmware tables. Methods read status (`_STA`) but can also switch devices on or off, change fan, power or performance settings, write BIOS settings, or make the PC hang or reboot. Evaluate only methods you know; this is the one SRS feature that can change hardware state.

ACPI HAL

`acpi-hal` - Linux data

In plain words. A summary of the ACPI layer as Linux sees it: what kind of machine the firmware claims to be, how many tables and devices there are, and the ACPI driver's settings. (HAL, Hardware Abstraction Layer, is the Windows name for this layer.)

What you see.

Field	Meaning	Why it matters
Preferred PM Profile	Machine type the firmware reports: Desktop, Mobile, Workstation, Server, Tablet...	Linux and Windows pick power defaults from it
ACPICA Version	Version of the ACPI interpreter built into the kernel	Newer versions fix firmware quirks
ACPI Tables	Number of tables (including dynamically loaded ones)	See ACPI tables

Field	Meaning	Why it matters
ACPI Devices	Number of ACPI namespace devices	See ACPI devices
SCI Interrupts	Total System Control Interrupts since boot	ACPI's own interrupt line
GPE Events	Total General Purpose Events since boot	Wake and hot-plug signals
Hardware-reduced ACPI	`No (fixed hardware present)` or `Yes / unknown`	Hardware-reduced is used by ARM and some tablets
Firmware Performance (FPDT)	Whether a boot-timing table exists	Used by Firmware
Hotplug …	One line per ACPI hot-plug type (memory, pci_root, processor)	Enabled = Linux will react to hot-add events
acpi.*	Every ACPI driver parameter and its value	Shows kernel command-line tweaks

Healthy / typical values. Author's PC: `1 (Desktop)`, ACPICA 20260408, 30 tables, 258 devices, 3512 SCI interrupts, "No (fixed hardware present)", FPDT present, all three hot-plug types enabled.

Unusual values and what they suggest.

- PM profile `2 (Mobile)` on a desktop board, or `0 (Unspecified)`: lazy firmware; power defaults may be odd.
- SCI and GPE counts that rise by thousands per second: an "interrupt storm", often a buggy GPE. Look at ACPI interrupts and GPEs to find which one.

Where Linux gets it. `/sys/firmware/acpi/pm_profile`, `/sys/firmware/acpi/interrupts/`, `/sys/firmware/acpi/hotplug/`, `/sys/module/acpi/parameters/`.

ACPI interrupts and GPEs

`acpi-int` - Linux data

In plain words. Counts how often each ACPI event has fired since boot: the power button, the real-time clock alarm, and the General Purpose Events (GPEs) that devices use to wake the PC or signal hot-plug.

What you see.

Field	Meaning	Why it matters
Event	Event name: `ff_*` fixed events, `gpeXX` numbered GPEs, `sci`, `gpe_all`, `error`, `sci_not`	Which signal
Count	How many times it fired since boot	Shows busy or stuck events
State	Kernel state words: `EN` enabled, `STS` status bit set, `enabled/disabled`, `masked/unmasked`, `invalid`	Whether Linux is listening to it

Then: how many idle (count 0 and not enabled) GPEs were hidden, and a line `SCI IRQ n: total interrupts (all CPUs)` from `/proc/interrupts`.

Healthy / typical values. Author's PC: `gpe04` 3506, `gpe1B` 6, everything else 0; `sci` and `gpe_all` 3512; `SCI IRQ 9`. `ff_pwr_btn` enabled (the power button works), `ff_rt_clk` disabled (no RTC alarm set). 27 idle GPEs hidden.

Unusual values and what they suggest.

- One GPE with a huge, fast-rising count: a GPE storm. It costs CPU time and battery. The usual fix is a BIOS update or the kernel option `acpi_mask_gpe=0xNN`.
- `error` above 0: the ACPI interpreter hit errors handling events; check `dmesg` for ACPI messages.
- `sci_not` above 0: SCIs arrived that no handler claimed.

Where Linux gets it. `/sys/firmware/acpi/interrupts/*` and the `acpi` line in `/proc/interrupts`.

Background. The **SCI** (System Control Interrupt) is ACPI's single shared interrupt line, usually IRQ 9. Each **GPE** is one bit in the chipset's event register; the firmware's `_Lxx` / `_Exx` methods say what each bit means.

ACPI IPF (Intel Platform Framework) methods

`acpi-ipf` - Not present on most PCs

In plain words. Looks for Intel Platform Framework devices, Intel's newer laptop power and thermal framework (the successor of DPTF, Dynamic Platform and Thermal Framework).

What it would show. One line per matching ACPI device with its path, then **Device** (description), **Probed HIDs** (`INTC1040 INTC1041 INTC1042 INTC1043`) and **Status** (`present` or `Not present on this system`).

What "not present" means. Normal on AMD PCs and on desktops. It only appears on recent Intel laptops.

Where Linux gets it. Name match in `/sys/bus/acpi/devices`.

ACPI ICD0001 methods

`acpi-itm` - Not present on most PCs

In plain words. Looks for an ACPI device with ID `ICD0001`, a thermal / management device found on a few OEM systems.

What it would show. Matching devices with their ACPI path, then Device, Probed HIDs (`ICD0001`) and Status.

What "not present" means. Normal for nearly every PC.

Where Linux gets it. Name match in `/sys/bus/acpi/devices`.

ACPI methods

`acpi-methods` - Linux data - Needs root data

In plain words. Lists every method (small firmware program) defined in the main ACPI code tables, the DSDT and the SSDTs.

What you see.

Field	Meaning	Why it matters
Table	Which table holds the method: `DSDT`, `SSDT1`...	Tells you which firmware module defines it
Method	Method name, relative to its enclosing Scope, e.g. `_PRT`, `_STA`, `_CRS`, `WMBC`	Names starting with `_` are standard ACPI; others are vendor-private
Args	Number of arguments (0-7)	Part of the method's definition
Flags	`Serialized` or `NotSerialized`, plus `Sync` if a sync level is set	Serialized methods cannot run twice at once

The footer gives the total. `acpi-methods:<SIG>` (e.g. `acpi-methods:SSDT6`) shows one table only.

Healthy / typical values. Hundreds to a few thousand rows. On the author's PC the DSDT alone has many `_PRT` (PCI interrupt routing), `_STA`, `_CRS` (current resources), `_PRS` / `_SRS` / `_DIS` (interrupt link handling), and vendor methods such as `WMBC` (an ASUS WMI method), `_PTS` / `_WAK` (prepare to sleep / wake).

Unusual values and what they suggest. The list is made by scanning the table bytes for the AML "Method" opcode, so an odd row (for example a standard name with an unexpected argument count) can be a scan match inside data rather than a real method. For exact work use `acpidump` and `iasl -d`. If the item says it needs the cached tables, run srs-root.

Where Linux gets it. The srs-root copies of `/sys/firmware/acpi/tables/DSDT` and `SSDT*` (root-only on Linux).

Background. Common standard methods: `_STA` status, `_CRS` resources, `_PRW` wake capability, `_PSx` / `_PRx` power states, `_OSC` operating-system capability handshake, `_PTS` and `_WAK` sleep and wake hooks, `_Lxx` GPE handlers.

ACPI extension (PDX)

`acpi-pdx` - Linux data

In plain words. Lists the vendor-specific device IDs in the ACPI namespace, that is, every device whose HID is not a standard `PNP`, `LNX` or `ACPI` one. These are the parts that need vendor drivers.

What you see.

Field	Meaning	Why it matters
HID	Vendor hardware ID, e.g. `AMDI0010`	First letters name the vendor

Field	Meaning	Why it matters
Count	How many devices share that HID	e.g. 4 I2C controllers
First path	ACPI path of the first one	Where it sits
Driver	Linux driver of the first one, or `-`	`-` = unclaimed (often disabled anyway)

A note explains the prefixes: AMDI = AMD, MSFT = Microsoft, RTK = Realtek, ASMT = ASMedia.

Healthy / typical values. Author's PC: `AMDI0010` x4 (I2C, `i2c_designware`), `AMDI0030` (GPIO, `amd_gpio`), `AMDI0101` (`amd_x3d_vcache`, the X3D cache-preference driver), `AMDIF031` (`pt-gpio`), `MSFT0101` (TPM, `tpm_crb_acpi`), `RTK5439` and `RTK5452` (Realtek USB-C / USB4 power-delivery chips on the I2C bus).

Unusual values and what they suggest. A vendor HID with no driver may simply be disabled in firmware (check its _STA in ACPI devices). If it is `0x0F PESF` and has no driver, a feature (sensor, button, power profile) may not work until a newer kernel adds support.

Where Linux gets it. `/sys/bus/acpi/devices`.

ACPI PCI buses and ports

`acpi-ports` - Linux data

In plain words. Every PCI device that the firmware also describes in ACPI, matched to its ACPI name. This tells you which physical port or slot a device is behind.

What you see.

Field	Meaning	Why it matters
PCI	PCI address bus:device.function, e.g. `03:00.0`	Same as `lspci`
Class	PCI class code; `0604 PtP` marks a PCI-to-PCI bridge (a port)	0300 = display, 0108 = NVMe, 0C03 = USB, 0200 = Ethernet
ID	Vendor:device ID, e.g. `1002:7550`	Identifies the chip
ACPI node	Linux name of the ACPI companion, e.g. `device:3e`	Links to ACPI devices
ACPI path	The firmware's name, e.g. `\_SB_.PCI0.GPP0.SWUS.SWDS.VGA_`	Shows the chain of ports down to the device
Driver	Linux driver bound to the PCI device	`pcieport` for ports

Footer: number of PCI functions with an ACPI companion.

Healthy / typical values. Author's PC: 45 functions. The graphics card is `\_SB_.PCI0.GPP0.SWUS.SWDS.VGA_` (`amdgpu`); NVMe drives sit at `...GPP7.UP00.DP00.NV00` and `...DP40.EP00`; the 5 GbE network chip at `...DP30.EP00` (`r8169`); Wi-Fi at `...DP38.EP00`.

Unusual values and what they suggest. A PCI device missing from this list has no ACPI companion. That is normal for some devices behind the chipset, but can mean ACPI power management (D3cold,

wake) will not work for it. Devices behind a port marked `GPP7.UP00` share the chipset link to the CPU; heavy traffic on several of them at once shares that bandwidth.

Where Linux gets it. `/sys/bus/pci/devices/*/firmware_node` plus `class`, `vendor`, `device` and `driver`.

ACPI power resource methods

`acpi-pwr` - Linux data

In plain words. Shows the ACPI "power resources" (switchable power rails the firmware can turn on and off) and the ACPI power state of each device that supports power management.

What you see. Two tables.

Field (power resources)	Meaning
Resource	`LNXPOWER:nn`
Path	ACPI path of the power resource, e.g. `\_SB_.PCI0.GPP8.PWRS`
In use	`on` if a device is currently using it, else `off`
Status	_STA of the resource (`0x01 P---` = on)

Field (devices)	Meaning
Device	ACPI device name
Path	ACPI path
power_state	State Linux believes: `D0` (fully on) to `D3hot` / `D3cold` (off)
real_power_state	State read back from the hardware, or `-`

Footer: number of devices with ACPI power management (`_PSx` / `_PRx` methods).

Healthy / typical values. Author's PC: 14 power resources; the USB4 branch (`GPP8...`) all `on`, the graphics-card branch resources (`M237`) `off`. The I2C controller sits in `D3hot` (idle, powered down), the USB4 ports in `D0`.

Unusual values and what they suggest.

- A device stuck in `D0` that should idle (for example an unused USB4 controller) costs power. Check runtime power management settings (`power/control` in sysfs, or `powertop`).
- `power_state` and `real_power_state` disagreeing: the device did not reach the state Linux asked for; a common cause of failed suspend.

Where Linux gets it. `/sys/bus/acpi/devices/LNXPOWER:*` (`path`, `resource_in_use`, `status`) and `/sys/bus/acpi/devices/*/power_state`, `real_power_state`.

Background. D-states are device power states: D0 on, D1/D2 light sleep (rare), D3hot off but still powered, D3cold power fully removed.

ACPI SPB (serial peripheral bus) methods

`acpi-spb` - Linux data

In plain words. Lists the low-speed serial buses on the board (I2C, I3C, UART serial ports, SPI) and the small devices on them, such as USB-C controllers or memory temperature sensors.

What you see. First table, ACPI serial-bus controllers and serial devices:

Field	Meaning
Device	ACPI device (`AMDI0010` I2C, `AMDI0015` I3C, `AMDI0020` UART, `PNP0500`/`PNP0501` serial port, `RTK*` Realtek devices...)
Path	ACPI path
Driver	Linux driver or `-`
Physical node	Linux device it became

Second table, every I2C adapter and client Linux knows:

Field	Meaning
I2C	Linux name: `i2c-N` for a bus, `N-00AA` for a device at address 0xAA on bus N
Adapter / client name	e.g. `SMBus PIIX4 adapter port 0`, `spd5118`, `AMDGPU DM i2c hw bus 0`
ACPI path	ACPI companion if any

Then one line per SPI device, and a note that SRS lists names only and never talks on these buses.

Healthy / typical values. Author's PC: one active I2C controller `\_SB_.I2CA` (`i2c_designware`) with two Realtek chips on it (`U4PD`, `TYPC`, the USB4 / USB-C power controllers); one active serial port `PNP0501:00` (`serial`); the chipset SMBus (`SMBus PIIX4`, three ports) with two `spd5118` clients at `1-0051` and `1-0053`, which are the two DDR5 sticks' temperature sensors; and many `AMDGPU` buses that belong to the graphics card's display outputs.

Unusual values and what they suggest. On DDR5 systems, no `spd5118` clients means the memory temperature sensors are not visible (the driver is not loaded, or the board blocks SMBus access). See DIMM status/temperatures.

Where Linux gets it. `/sys/bus/acpi/devices`, `/sys/bus/i2c/devices/*/name`, `/sys/bus/spi/devices`.

Background. I2C / SMBus (System Management Bus) is a two-wire bus for slow chips: sensors, memory SPD chips, RGB controllers. Each chip has a 7-bit address; DDR5 memory sticks answer at 0x50-0x57.

ACPI tables

`acpi-tables` - Linux data - Needs root data

In plain words. Lists every ACPI table the firmware provides, with its header: size, version and who built it.

What you see.

Field	Meaning	Why it matters
Table	Signature, e.g. `DSDT`, `SSDT3`, `APIC` (Linux numbers repeated tables)	What the table is
Length	Size in bytes	
Rev	Table revision	Newer revisions add fields
Check	Checksum byte	All bytes of a valid table add up to 0
OEM	OEM ID, e.g. `ALASKA` (AMI's placeholder), `AMD`	Who supplied the table
OEM Table	OEM table ID, e.g. `A M I`, `AmdTable`, `CPMCMN`	Often names the firmware module
OEM Rev	OEM revision	
Creator	Tool that compiled it: `AMI`, `INTL` (Intel's iasl compiler), `MSFT`, `AMD`	
Creator Rev	Version of that tool	

Footer: total tables. Without the srs-root cache only the name and file size are shown. `acpi-tables:<SIG>` (e.g. `acpi-tables:HPET`) shows the parsed header of one table plus a hex dump of up to 4 KB.

Healthy / typical values. Author's PC: 30 tables, among them `APIC` (CPU interrupt controllers), `DSDT` (88990 bytes, the main ACPI code), `FACP`, `FACS`, `HPET` (timer), `IVRS` (AMD IOMMU), `MCFG` (PCIe configuration space), 16 `SSDT`s from AMD modules, `TPM2`, `VFCT` (a copy of the graphics card's video BIOS), `WPBT`, `WSMT`.

Unusual values and what they suggest.

- `WPBT` (Windows Platform Binary Table) lets firmware hand Windows a program to run at startup (on ASUS boards, the Armoury Crate installer). Linux ignores it. You can usually turn it off in the BIOS setup ("Download Armoury Crate app").
- `FACS` has no standard header, so its OEM and revision columns are not meaningful.
- A missing `MCFG` or `HPET` on a modern PC is rare and worth a BIOS update.

Where Linux gets it. `/sys/firmware/acpi/tables/` (root-only to read; SRS uses the srs-root copies in `acpi-tables/`). Same data as `acpidump`.

Background. DSDT (Differentiated System Description Table) holds the main ACPI code; **SSDTs** (Secondary System Description Tables) add more, often one per firmware module (CPU, USB4, memory tools).

ACPI Toshiba test

`acpi-test` - Not present on most PCs

In plain words. Looks for any ACPI device whose ID starts with `TOS`, which Toshiba (now Dynabook) laptops use.

What it would show. Matching devices with their ACPI path, then Device ("Toshiba laptop ACPI devices"), Probed HIDs (`TOS`) and Status.

What "not present" means. You do not have a Toshiba laptop. Normal.

Where Linux gets it. Name match in `/sys/bus/acpi/devices`.

ACPI TFP INT3403 methods

`acpi-tfp` - Not present on most PCs

In plain words. Looks for Intel DPTF (Dynamic Platform and Thermal Framework) thermal "participants", the temperature sensors and cooling devices of Intel laptops.

What it would show. Matching devices with their ACPI path, then Device ("Intel DPTF thermal participant"), Probed HIDs (`INT3403 INTC1043`) and Status.

What "not present" means. Normal on AMD PCs and desktops.

Where Linux gets it. Name match in `/sys/bus/acpi/devices`.

ACPI TMS INT340E methods

`acpi-tms` - Not present on most PCs

In plain words. Looks for the Intel DPTF `INT340E` device (part of Intel's laptop thermal management).

What it would show. Matching devices with their ACPI path, then Device, Probed HIDs (`INT340E`) and Status.

What "not present" means. Normal on AMD PCs and desktops.

Where Linux gets it. Name match in `/sys/bus/acpi/devices`.

ACPI TOS1900 methods

`acpi-tos` - Not present on most PCs

In plain words. Looks for the Toshiba laptop "extras" device that controls hotkeys, backlight and fan on Toshiba / Dynabook laptops.

What it would show. Matching devices with their ACPI path, then Device ("Toshiba laptop extras"), Probed HIDs (`TOS1900 TOS6200 TOS6207 TOS6208`) and Status.

What "not present" means. Not a Toshiba laptop. Normal.

Where Linux gets it. Name match in `/sys/bus/acpi/devices`.

ACPI TPM

`acpi-tpm` - Linux data

In plain words. Shows how the firmware describes the TPM (Trusted Platform Module, the security chip used for disk encryption keys and Secure Boot measurements).

What you see. For each TPM device in ACPI (`MSFT0101` = TPM 2.0, `PNP0C31` / `IFX0102` = TPM 1.2):

- **Path**, **HID**, **UID**, **_STA**, **Driver** - as in ACPI devices.

Then the decoded **TPM2 table** (when the srs-root cache has it):

Field	Meaning
Signature … Creator Revision	The standard ACPI table header (see ACPI tables)
Platform Class	`0 (Client)` for PCs, `1 (Server)`
Control Area	Memory address where commands are exchanged with the TPM
Start Method	How the OS tells the TPM to start a command: `2 ACPI Start`, `6 TIS 1.3 / FIFO`, `7 Command Response Buffer`, `8 CRB with ACPI Start`, and ARM / I2C variants

Without the cache only the table size is shown. Without a TPM: "No ACPI TPM device" and / or "No TPM2 ACPI table".

Healthy / typical values. Author's PC: `MSFT0101:00`, `\_SB_.TPM_`, `0x0F PESF`, `tpm_crb_acpi`; TPM2 table 76 bytes, Platform Class 0 (Client), Start Method 2 ACPI Start. That is the usual pattern for AMD's firmware TPM.

Unusual values and what they suggest. No `MSFT0101` device on a board that has a TPM: it is switched off in the BIOS setup (on AMD boards look for "fTPM" or "Firmware TPM"). Windows 11 requires it.

Where Linux gets it. `/sys/bus/acpi/devices/MSFT0101:*` and the srs-root copy of `/sys/firmware/acpi/tables/TPM2`.

ACPI WMI (PNP0C14) methods

`acpi-wmi` - Linux data

In plain words. WMI (Windows Management Instrumentation) is a way for firmware to offer vendor features (fan modes, RGB, sensor readings, keyboard lights) through ACPI. This item lists the WMI interfaces the firmware offers and which Linux drivers use them.

What you see.

Field	Meaning	Why it matters
GUID	The 128-bit ID of the WMI interface	Each feature has its own GUID
Obj	ACPI object id (the `WQxx`/`WMxx` suffix) or notify id	Which firmware object implements it
Inst	Instance count	
Exp	"Expensive" flag (1 = slow to query)	
Set	Whether the data block can be written (`-` for method / event blocks)	
Driver	Linux driver bound to it, or `-`	e.g. `wmi-bmof`, `asus-wmi`, `dell-wmi`
Description	Name SRS knows for a few common GUIDs	e.g. "Binary MOF (WMI class descriptions)", "ASUS management (ATK / asus-wmi)"

Then **WMI mapper devices**: every `PNP0C14` ACPI device with its path. A note says SRS never invokes WMI methods.

Healthy / typical values. Author's PC: six WMI mapper devices (`\AWW0`, `\ALCW`, `\RMTW`, `\AMZW`, `\WFDV`, `\AOD_`), several Binary MOF blocks bound to `wmi-bmof`, and the ASUS management GUID with no driver bound (normal on ASUS desktops; `asus-wmi` is mainly for laptops).

Unusual values and what they suggest. A vendor GUID with no driver means that feature is not used by Linux. That is common. Projects such as `asus-wmi-sensors` or vendor-specific modules may add support for some of them.

Where Linux gets it. `/sys/bus/wmi/devices/*` (`guid`, `object_id` / `notify_id`, `instance_count`, `expensive`, `setable`, `driver`).

Background. Binary MOF is a compressed description of the vendor's WMI classes; tools can decode it to learn what methods exist.

APIC (MADT) summary

`bios-apic` - Linux data - Needs root data

In plain words. Decodes the MADT (Multiple APIC Description Table, signature `APIC`). It tells the operating system how many CPU threads exist and how interrupts are wired to them.

What you see. The standard ACPI header (Signature, Length, Revision, Checksum, OEM ID, OEM Table ID, OEM Revision, Creator ID, Creator Revision), then:

- **Local APIC Address** - memory address of each CPU's local interrupt controller (normally `0xFEE00000`).
- **Flags** - `PCAT_COMPAT (dual 8259)` means the old PC interrupt controllers also exist.

Then one row per entry:

Entry	ID	UID/Addr	GSI/Flags	Detail
Local APIC	APIC ID of a CPU thread	ACPI processor UID	flags	`Enabled`, `Online capable` or `Disabled`
Local x2APIC	x2APIC ID (for more than 255 threads)	UID	flags	Enabled / Disabled
I/O APIC	I/O APIC ID	its address	`GSI n` = first interrupt number it handles	
Int Override	`IRQ n` (legacy ISA number)	`bus 0`	`GSI n` it is remapped to	`flags` (polarity / trigger)
Local APIC NMI	`all` or a CPU UID		`flags`	which `LINTn` pin carries the NMI
LAPIC Addr Override, NMI Source, x2APIC NMI	rarely used entry types			
Other	unknown type			`type n len n`

Footer: counts of Local APIC, x2APIC, I/O APIC, Overrides and LAPIC NMI entries. Without the srs-root cache the item shows only "present, N bytes".

Healthy / typical values. Author's PC (8 cores, 16 threads): 32 Local APIC entries, 16 `Enabled` (IDs 0-15) and 16 `Disabled` placeholders; 2 I/O APICs at `FEC00000` (GSI 0) and `FEC01000` (GSI 24); overrides IRQ 0 -> GSI 2 (the timer) and IRQ 9 -> GSI 9 (the ACPI SCI).

Unusual values and what they suggest.

- Fewer enabled entries than CPU threads you expect: SMT or some cores are disabled in the BIOS.
- `Disabled` placeholders are normal. One firmware image supports several CPU models, so it lists slots for the biggest one (here 16 cores / 32 threads).

Where Linux gets it. The srs-root copy of `/sys/firmware/acpi/tables/APIC`.

Background. An **APIC** (Advanced Programmable Interrupt Controller) delivers interrupts to CPUs. Each thread has a **local APIC**; the **I/O APIC** collects device interrupts and maps them to **GSIs** (Global System Interrupts).

ASPT table summary

`bios-aspt` - Not present on most PCs - Needs root data

In plain words. Looks for the ASPT (AMD Secure Processor Table), which tells the OS where the AMD security processor's mailbox is.

What it would show. If present: the ACPI header and a hex dump (up to 1 KB) from the srs-root cache. If absent: `ASPT not present in /sys/firmware/acpi/tables`, its Purpose, and **Tables present** (count and list of all tables).

What "not present" means. Normal on most desktop boards, including the author's. The AMD security processor is still there; see HECI (Intel ME) status.

Where Linux gets it. `/sys/firmware/acpi/tables`.

DMAR table summary

`bios-dmar` - Not present on most PCs - Needs root data

In plain words. The DMAR (DMA Remapping) table describes Intel's IOMMU, called VT-d. AMD systems use a different table, IVRS, so on AMD this item decodes IVRS instead.

What you see. If DMAR is absent: `DMAR not present`, its Purpose ("Intel VT-d DMA remapping; AMD platforms use IVRS instead") and the Tables present line. If DMAR is present: its header and a hex dump.

If an IVRS table exists, a second block follows:

- The ACPI header of IVRS.
- **IVinfo** - capability word: virtual address bits (VA), physical address bits (PA), guest virtual address bits (GVA), and "EFR supported" (extended feature register).
- One row per entry: **Type** (`IVHD 10h/11h/40h` = IOMMU hardware definition, `IVMD` = memory definition), **Flags**, **DevID** (PCI ID of the IOMMU), **Segment**, **Base** (register address).

Healthy / typical values. Author's PC: no DMAR (AMD). IVRS present: `VA 64 bits, PA 48 bits, GVA 2 bits, EFR supported`, two IVHD entries (types 10h and 11h) for one IOMMU at `00000000F7600000`.

Unusual values and what they suggest. No IVRS (on AMD) or no DMAR (on Intel): the IOMMU is switched off in the BIOS setup ("IOMMU" / "SVM" / "VT-d"). You need it for passing a GPU through to a virtual machine and for DMA protection of Thunderbolt / USB4 ports.

Where Linux gets it. `/sys/firmware/acpi/tables` and the srs-root copy of `IVRS`.

Background. An **IOMMU** (Input-Output Memory Management Unit) controls which memory each device may read and write with DMA (direct memory access). It isolates devices from each other and from the OS.

FACP (FADT) summary

`bios-facp` - Linux data - Needs root data

In plain words. Decodes the FADT (Fixed ACPI Description Table, signature `FACP`). It is the "master" table: it points to the other key tables and lists the fixed power-management hardware.

What you see. The ACPI header, then:

Field	Meaning
FIRMWARE_CTRL (FACS)	Address of the FACS table (used for sleep / wake)
DSDT / X_DSDT	32-bit and 64-bit address of the DSDT
Preferred PM Profile	Desktop, Mobile, Server...
SCI Interrupt	IRQ used by ACPI (usually 9)
SMI Command Port	I/O port used to switch between firmware and OS control, with the enable / disable values
PM1a Event / Control	I/O ports of the main power-management registers
PM Timer Block	ACPI timer port, length, and whether the timer is 24-bit or 32-bit
GPE0 Block / GPE1 Block	I/O ports of the GPE registers (see ACPI interrupts and GPEs)
C2 / C3 Latency	Advertised wake-up time of legacy CPU idle states, in microseconds (values above 100 / 1000 mean "not supported")
RTC Day/Month Alarm, Century	CMOS register indexes for these RTC fields
FADT Minor Version	Sub-version of the table
Reset Register	How the OS can reset the PC: address space, address and value
Hypervisor Vendor	Set only when running inside a virtual machine

Then **Flags** (hex) and a tick list of the 24 FADT feature flags (WBINVD, PROC_C1, SLP_BUTTON, RTC_S4, TMR_VAL_EXT, RESET_REG_SUP, PCI_EXP_WAK, USE_PLATFORM_CLOCK, HW_REDUCED_ACPI, LOW_POWER_S0_IDLE_CAPABLE and others). `[=]` = set, `[_]` = clear.

Then **IA-PC Boot Architecture** (hex) and its six flags: LEGACY_DEVICES, 8042 (PS/2 keyboard controller), VGA Not Present, MSI Not Supported, PCIe ASPM Controls, CMOS RTC Not Present.

Without the cache, only the PM profile from sysfs is shown.

Healthy / typical values. Author's PC: profile 1 (Desktop), SCI 9, SMI port 0xB2, PM timer 32-bit, C2/C3 latency 101 / 1001 us (that is, "not used": modern CPUs use their own idle states), reset by writing 0xBE to port 0xB2. Flags 0x0003C5A5 include RESET_REG_SUP, PCI_EXP_WAK and REMOTE_POWER_ON_CAPABLE. Boot architecture 0x0011: LEGACY_DEVICES and PCIe ASPM Controls set, 8042 clear (no PS/2 controller is advertised).

Unusual values and what they suggest.

- `LOW_POWER_S0_IDLE_CAPABLE` set: the board prefers "Modern Standby" (s2idle) over classic S3 sleep. Common on laptops; if suspend misbehaves, check `/sys/power/mem_sleep`.

- `HW_REDUCED_ACPI` set on a normal PC: unusual; fixed hardware (power button, PM timer) is then missing.
- `PCIe ASPM Controls` set means the firmware asks the OS not to change PCIe link power saving (ASPM). Linux respects that unless told otherwise (`pcie_aspm=force`).

Where Linux gets it. The srs-root copy of `/sys/firmware/acpi/tables/FACP`; fallback `/sys/firmware/acpi/pm_profile`.

PCI IRQ routing ($PIR)

`bios-irq` - Not present on most PCs - Needs root data

In plain words. Old PCs had a BIOS table called `$PIR` that said which interrupt each PCI slot used. UEFI boards no longer have it; ACPI `_PRT` methods and the MADT do the job.

What you see.

- **Status** - "Not present on this system".
- **Structure** - "$PIR PCI IRQ routing table".
- **Reason** - UEFI routes PCI interrupts through ACPI `_PRT`; scanning for `$PIR` would need `/dev/mem`, which SRS never opens.
- **Firmware** - "UEFI x86-64 (ACPI MADT/_PRT replace the legacy tables)" or "Legacy BIOS".
- **Linux routing** - how Linux actually routes interrupts: number of IO-APIC lines and PCI-MSI / MSI-X vectors in `/proc/interrupts`.

Healthy / typical values. Author's PC: 5 IO-APIC lines and 79 MSI / MSI-X vectors. On modern systems almost every device uses MSI (message-signalled interrupts).

Unusual values and what they suggest. Many devices on IO-APIC lines and few MSI vectors can mean MSI is disabled (`pci=nomsi`) or a driver does not support it. Shared IO-APIC lines are slower than MSI.

Where Linux gets it. `/proc/interrupts`.

BIOS maker

`bios-make` - Linux data

In plain words. Who made the firmware, which version and date it is, and what features it says it supports.

What you see.

Field	Meaning
BIOS Vendor	Firmware maker, e.g. American Megatrends Inc.
Family	Product family SRS recognises: AMI (Aptio V UEFI), Phoenix, Insyde H2O

Field	Meaning
BIOS Version	The version you compare with the board maker's download page
BIOS Date	Release date
BIOS Release	Major.minor release number from SMBIOS
Board Maker / Board	Motherboard maker and model
Firmware Type	UEFI or Legacy BIOS (how this boot started)
Address, Runtime Size, ROM Size, BIOS Revision, Firmware Revision	From the srs-root cache, when listed by SMBIOS
Characteristics	Tick list of the BIOS feature flags from SMBIOS type 0

Without the cache: "ROM Size / Address (needs root)".

Healthy / typical values. Author's PC: American Megatrends Inc., AMI (Aptio V UEFI), version 1805, 09/18/2026, release 18.5, UEFI, ROM 64 MiB, runtime 64 KiB at 0xF0000. Characteristics include "UEFI is supported", "ACPI is supported", "Firmware is upgradeable".

Unusual values and what they suggest. The characteristics list still shows ancient features (5.25" floppy, NEC 9800 floppy, CGA video): firmware makers keep these flags for compatibility. They do not mean the hardware exists. A BIOS date years older than the board's latest download means a BIOS update is available; updates often fix memory stability, CPU support and security issues.

Where Linux gets it. `/sys/class/dmi/id/bios_*`, `board_*`; `/sys/firmware/efi`; SMBIOS type 0 from the srs-root `dmidecode.txt`.

MPS table summary

`bios-mps` - Not present on most PCs - Needs root data

In plain words. The Intel MultiProcessor Specification (MPS) table was how 1990s PCs listed their CPUs. ACPI's MADT replaced it.

What you see. Status ("Not present on this system"), Structure ("Intel MultiProcessor Specification (*MP* floating pointer)"), Reason (superseded by the ACPI MADT; scanning needs `/dev/mem`, which SRS never opens), Firmware type.

What "not present" means. Normal on every UEFI PC. See APIC (MADT) summary for the modern equivalent.

MSDM table summary

`bios-msdm` - Not present on most PCs - Needs root data

In plain words. The MSDM (Microsoft Data Management) table holds an embedded Windows product key on PCs sold with Windows pre-installed.

What it would show. If present: ACPI header and hex dump from the srs-root cache. If absent: `MSDM not present`, its Purpose, and the Tables present list.

What "not present" means. The PC (or motherboard) was sold without an OEM Windows licence. Normal for self-built PCs.

Where Linux gets it. `/sys/firmware/acpi/tables`.

OEMB table summary

`bios-oemb` - Not present on most PCs - Needs root data

In plain words. OEMB is an AMI OEM-specific table found on some older AMI BIOS boards.

What it would show. If present: header and hex dump. If absent: `OEMB not present`, Purpose, Tables present.

What "not present" means. Normal on current boards.

Where Linux gets it. `/sys/firmware/acpi/tables`.

BIOS option ROMs

`bios-option` - Linux data - Needs root data

In plain words. Lists the PCI devices that carry their own firmware ROM (an "option ROM"), such as a graphics card's video BIOS or a storage controller's boot code.

What you see.

Field	Meaning
PCI	PCI address
Class	PCI class code (030000 = VGA, 010601 = AHCI SATA...)
ID	Vendor:device ID
ROM size	Size of the expansion ROM window
Driver	Linux driver

Footer: number of devices with an expansion ROM, and a note that the ROM images themselves are not read (the ROM must be switched on by a write first). For an AMD graphics card, GPU VBIOS status shows the driver's own copy of its image (2.4).

Healthy / typical values. Author's PC: the graphics card `03:00.0` (128 KB, `amdgpu`) and the SATA controller `0c:00.0` (512 KB, `ahci`).

Unusual values and what they suggest. If you turn off CSM (legacy boot support), the firmware uses only the EFI part of each option ROM. An old card without an EFI option ROM then shows no picture before Linux starts, or cannot boot.

Where Linux gets it. `/sys/bus/pci/devices/*/rom` (the file size only).

SDEV table summary

`bios-sdev` - Not present on most PCs - Needs root data

In plain words. The SDEV (Secure Devices) table lists devices that must only be used by the secure (virtualisation-protected) part of Windows, such as some fingerprint readers.

What it would show. If present: header and hex dump. If absent: `SDEV not present`, Purpose, Tables present.

What "not present" means. Normal on desktops.

Where Linux gets it. `/sys/firmware/acpi/tables`.

SFI summary

`bios-sfi` - Not present on most PCs

In plain words. SFI (Simple Firmware Interface) was a cut-down ACPI replacement for Intel's Moorestown / Medfield phone and tablet chips. Linux removed support in version 5.12.

What you see. Status ("Not present on this system"), Structure, Reason ("Intel MID only; removed from Linux 5.12"), Firmware type.

What "not present" means. Always the case on normal PCs.

SLIC table summary

`bios-slic` - Not present on most PCs - Needs root data

In plain words. The SLIC (Software Licensing) table was used for OEM activation of Windows Vista and 7 on brand-name PCs.

What it would show. If present: header and hex dump. If absent: `SLIC not present`, Purpose, Tables present.

What "not present" means. Normal on self-built and most current PCs.

Where Linux gets it. `/sys/firmware/acpi/tables`.

SLIT table summary

`bios-slit` - Not present on most PCs - Needs root data

In plain words. The SLIT (System Locality Information Table) gives the relative "distance" between NUMA nodes, that is, how much slower memory on another CPU socket is.

What it would show. If present: header and hex dump. If absent: `SLIT not present`, Purpose, Tables present.

What "not present" means. The PC has a single NUMA node (one CPU socket, one memory pool). Normal for desktops. Multi-socket servers and some workstation CPUs have it.

Where Linux gets it. `/sys/firmware/acpi/tables`. `numactl --hardware` shows the distances Linux uses.

Background. NUMA (Non-Uniform Memory Access): on big systems each CPU has "near" memory and "far" memory. The OS tries to keep a program's memory near the CPU running it.

SRAT table summary

`bios-srat` - Not present on most PCs - Needs root data

In plain words. The SRAT (System Resource Affinity Table) says which CPUs and which memory ranges belong to which NUMA node.

What it would show. If present: header and hex dump. If absent: `SRAT not present`, Purpose, Tables present.

What "not present" means. Single-node system. Normal for desktops. Some boards add it only when a "NUMA nodes per socket" BIOS option is enabled.

Where Linux gets it. `/sys/firmware/acpi/tables`.

BIOS / SMBIOS tables

`bios-tables` - Linux data - Needs root data

In plain words. An overview of the SMBIOS inventory tables: version, size, and how many records of each type the firmware provides.

What you see. Header row:

Field	Meaning
BIOS	SMBIOS version, e.g. `SMBios V3.6.0` (only `V3.x` / `V2.x` without root)
Anchor	Entry-point signature: `"_SM3_"` (64-bit, SMBIOS 3) or `"_SM_" + "_DMI_"` (32-bit, SMBIOS 2)

Field	Meaning
EP Size	Size of the entry point: 24 bytes (SMBIOS 3) or 31 (SMBIOS 2)
Table	Total size of the structure table in bytes
Structures	Number of records (needs root)

Then **Table address** (memory address of the table), and a list: **Type**, **Name**, **Count**, one row per SMBIOS record type present (types 128 and up are shown as "OEM-specific"; unknown standard types as `-`).

Healthy / typical values. Author's PC: SMBIOS 3.6.0, `_SM3_`, 24-byte entry point, 3448-byte table, 80 structures. Types include 0 BIOS Information, 1 System, 2 Base Board, 4 Processor, 7 Cache (3), 8 Port Connector (36), 9 System Slots (7), 17 Memory Device (4, one per slot), 43 TPM Device, 45 Firmware Inventory.

Unusual values and what they suggest. Type 17 count lower than your number of slots: the firmware hides empty slots, so slot positions in DIMMs may not match the board printing. SMBIOS 2.x on a new board is very unusual.

Where Linux gets it. `/sys/firmware/dmi/tables/smbios_entry_point` and `DMI` (sizes); type counts from the srs-root `dmidecode.txt`.

Background. SMBIOS record **types**: each describes one kind of thing (type 4 a CPU socket, type 17 a memory slot, type 9 an expansion slot). `dmidecode -t <type>` prints one type.

CMOS / RTC status

`cmos-status` - Linux data

In plain words. The real-time clock (RTC) keeps the time while the PC is off, powered by the coin battery on the board. This item shows the clock, its alarm, its settings and the battery check.

What you see. First, every line the kernel's RTC driver reports:

Field	Meaning
rtc_time / rtc_date	Time and date stored in the clock chip
alrm_time / alrm_date	Wake-up alarm setting
alarm_IRQ, alrm_pending	Whether the alarm is armed / has fired
update IRQ enabled, periodic IRQ enabled, periodic IRQ frequency, max user IRQ frequency	Timer interrupt settings of the chip
24hr, BCD	Clock format (24-hour, binary-coded decimal)
periodic_IRQ, update_IRQ, HPET_emulated, DST_enable, periodic_freq	More chip flags
batt_status	Coin-battery check: `okay` (green) or a warning (red)

Then the `/sys/class/rtc/rtc0` block: **name** (driver and ACPI device), **date**, **time**, **since_epoch** (seconds since 1970), **hctosys** (1 = Linux set its clock from this RTC at boot), **max_user_freq**, **wakealarm** and **offset** when set, then:

- **RTC - system clock** - difference between the RTC and Linux's clock in seconds (orange if more than 2 s).
- **RTC time scale** - whether the RTC holds UTC or local time (from `/etc/adjtime`).

Healthy / typical values. Author's PC: `rtc_cmos PNP0B00:00`, 24-hour BCD, batt_status `okay`, hctosys 1, difference -1 s, time scale UTC.

Unusual values and what they suggest.

- `batt_status` not `okay`, or a clock that is wrong after every power cut: the CR2032 coin cell is flat. A flat cell can also reset BIOS settings.
- A difference of many seconds or hours: time sync (NTP) is off, or Windows and Linux disagree on UTC versus local time on a dual-boot PC. Linux expects UTC.
- `hctosys 0`: Linux did not read its clock from this RTC.

Where Linux gets it. `/proc/driver/rtc`, `/sys/class/rtc/rtc0/*`, `/etc/adjtime`.

Background. CMOS (originally the chip technology) is the old name for the RTC chip and its small battery-backed memory, which once held all BIOS settings.

Dell BIOS

`dell-bios` - Not present on most PCs

In plain words. Dell PCs have extra BIOS interfaces (for fans, sensors, settings). This item checks for them.

What you see. System Vendor. On a Dell: **dell-smbios** and **dell_smm_hwmon**, each `loaded` or `not loaded`. Otherwise **Status**: "Not a Dell system: Dell BIOS SMI data does not apply".

What "not present" means. Not a Dell. On a Dell where both show `not loaded`, load the `dell_smm_hwmon` module to get fan and temperature readings.

Where Linux gets it. `/sys/class/dmi/id/sys_vendor`, `/sys/bus/platform/devices/dell-smbios.0`, `/sys/module/dell_smm_hwmon`.

DIMMs

`dimms` - Linux data - Needs root data

In plain words. One line per memory slot: what is in it, how big, how fast it is rated, how fast it is actually running, who made it, and its temperature. Open it with the **DIMMs** panel button.

What you see.

Field	Meaning	Why it matters
Slot	Slot name from the firmware, e.g. `DIMM 1`	Match it to the board printing
Bank	Channel name, e.g. `P0 CHANNEL A`	Which memory channel the slot is on
Size	Capacity of the stick, or `Empty`	
Type	DDR4, DDR5…	
Form	DIMM (desktop) or SODIMM (laptop)	
Rated	Base speed the stick reports (JEDEC)	The "safe default" speed
Configured	Speed the memory is running at now	Shows whether EXPO / XMP is active
Rank	Number of ranks (1 or 2)	Dual-rank is a little faster, harder to overclock
Manufacturer	Module maker	
Part Number	Model code of the stick	Use it to look up the kit's spec
Serial	Module serial number	Tells identical sticks apart
SPD	The `spd5118` sensor device paired with this slot, e.g. `1-0051`	
Temp	Current stick temperature from that sensor	

Footer: "N of M slots populated, X installed", the data source, and a note that the SPD pairing is inferred (slot index i is matched to the sensor at SMBus address 0x50 + i).

Detail view (`dimms:<slot index>`, e.g. `dimms:1`): every SMBIOS field the system has for that slot (from udev: locator, bank, size, type, type detail, form factor, speeds, manufacturer, module manufacturer ID, part number, serial, rank, configured voltage, memory technology…; or, from the srs-root cache: Locator, Bank Locator, Size, Type, Type Detail, Form Factor, Speed, Configured Memory Speed, Manufacturer, Part Number, Serial Number, Rank, Minimum / Maximum / Configured Voltage, Total Width, Data Width), plus a line for the paired SPD5118 sensor and its temperature.

Healthy / typical values. Author's PC: 2 of 4 slots used, 32 GB. Both sticks in the recommended slots (`DIMM 1` of channel A and channel B, the "A2 / B2" slots on most boards): 16 GB DDR5 DIMM, rank 1, rated 4800 MT/s, configured 6000 MT/s (EXPO profile active), G.Skill, temperatures 30-31 °C.

Unusual values and what they suggest.

- **Configured equals Rated** (for example 4800 on a 6000 kit): EXPO / XMP is off in the BIOS.
- **Two sticks in the same channel** (both "CHANNEL A"): the memory runs single-channel, roughly half the bandwidth. Move one stick to the other channel.
- **Different part numbers** on the sticks: a mixed pair. It can work, but mixed kits are a common cause of instability at EXPO / XMP speeds. Matched kits are safer.
- **Serial all zeros** on a stick: some modules simply do not program a serial. Harmless by itself.
- Size shown smaller than expected or a stick missing: poor seating, a dead stick, or a slot disabled by the board.

Where Linux gets it. SMBIOS type 17 as imported by systemd-udevd into `/run/udev/data/+dmi:id` (readable without root), else the srs-root `dmidecode.txt`; temperatures from the `spd5118` hwmon driver. It is marked "needs root" because the fallback path and full detail come from the cache; on most systemd distributions the table is complete without it.

Background. MT/s (mega-transfers per second) is the data rate. DDR memory moves data twice per clock, so 6000 MT/s is a 3000 MHz clock. Marketing often writes "6000 MHz", which is not strictly right.

DIMM status/temperatures

`dimm-status` - Linux data

In plain words. Live temperatures and limits of the thermal sensors on DDR5 memory sticks. This is what the **Status** panel sub-button shows. It refreshes every 2 seconds.

What you see.

Field	Meaning
Sensor	Linux hwmon name, e.g. `hwmon7`
Device	I2C device of the sensor, e.g. `1-0051` (bus 1, address 0x51)
Slot	Compact slot label, e.g. `A1` = channel A, DIMM 1 (inferred from the address)
Temp	Current temperature
Min / Max / Crit / LCrit	Limits programmed into the sensor: low warning, high warning, critical high, critical low
Alarms	Which limit alarms are active (`min`, `max`, `crit`, `lcrit`), or `none` (green)
Enabled	Whether the sensor is enabled

A note says only the hwmon values are read, never the SPD memory chip itself. If there are no sensors: "No spd5118 hwmon sensors" with a hint that the `spd5118` driver must bind to SMBus addresses 0x50-0x57.

Healthy / typical values. Author's PC at idle: 31.0 °C and 30.5 °C; limits 0 / 55 / 85 / 0 °C; alarms none. Under heavy load DDR5 typically runs 40-55 °C.

Unusual values and what they suggest.

- Above about 55 °C: the stick reaches its warning limit. DDR5 then refreshes more often (a little slower), and errors become more likely when overclocked. Add airflow over the memory.
- One stick much hotter than the other: it may be under a CPU cooler's exhaust, or have no heat spreader contact.
- No sensors on DDR5: the `spd5118` module is not loaded, or the board's SMBus is locked. DDR4 sticks usually have no sensor.

Where Linux gets it. `/sys/class/hwmon/hwmon*/` where `name` is `spd5118`: `temp1_input`, `temp1_min`, `temp1_max`, `temp1_crit`, `temp1_lcrit`, `temp1_*_alarm`, `temp1_enable`.

Background. Every DDR5 stick has an **SPD hub** chip (spd5118) that holds the stick's description (the SPD data) and contains a temperature sensor.

Firmware

`firmware` - Linux data

In plain words. How the PC's firmware started Linux: UEFI or legacy BIOS, Secure Boot state, firmware update records, and how long the firmware took to boot.

What you see.

Field	Meaning
Firmware Type	`UEFI`, or "Legacy BIOS (no /sys/firmware/efi)"
Platform Size	64-bit or 32-bit UEFI
Firmware Vendor Pointer, Runtime Services, Configuration Table	Memory addresses of UEFI structures (for debugging)
System Table	The UEFI System Table's pointers to the ACPI (`ACPI20`, `ACPI`) and SMBIOS (`SMBIOS3`, `SMBIOS`) tables (root only; from the srs-root cache `efi-systab.txt`, 2.4)
Secure Boot	Enabled / Disabled / Unknown
ESRT Entries	Number of firmware-update resources (EFI System Resource Table). With the root cache (`esrt.txt`, 2.4), per entry: what it updates (for example "system firmware (the BIOS)"), its **Firmware class** GUID, **Version**, **Lowest allowed** version and the **Last update** attempt with its result. An AMI-style hex version is shown as the BIOS version it encodes: "BIOS 1805 (stored as 0x1805 = 6149)"
UEFI Variables	Number of variables stored in firmware
BIOS Vendor / BIOS Version	From SMBIOS

Firmware Boot Performance (ACPI FPDT), when the board provides it:

Field	Meaning
Firmware start	When firmware timing started
Boot loader load	When the firmware started loading the boot loader
ExitBootServices	When the OS took over from the firmware (start -> end)
Time in firmware + loader	Total time from power-on timing start to OS hand-over

Healthy / typical values. Author's PC: UEFI, 64-bit, Secure Boot Disabled, 1 ESRT entry (system firmware, BIOS 1805, lowest allowed 1805, last update: success), 150 variables. Typical firmware time is 5-20 s; boards with DDR5 that re-train memory at every boot (or right after a BIOS settings change) can take much longer. The boot-loader-to-hand-over part also includes any time spent in the boot menu.

Unusual values and what they suggest.

- A very long "Boot loader load" time: memory training. On AMD AM5 boards enable "Memory Context Restore" in the BIOS so training is skipped on normal boots (if your memory is stable with it).
- Large gap between Boot loader load and ExitBootServices: time spent in the boot loader menu or its timeout.
- An ESRT **Last update** that is not "success": the last firmware update through UEFI capsules (for example `fwupd`) failed; the BIOS kept its old version.
- **Lowest allowed** equal to **Version**: the BIOS cannot be downgraded through capsule updates (anti-rollback).
- ESRT 0: the firmware does not offer capsule updates, so `fwupd` cannot update it.

Where Linux gets it. `/sys/firmware/efi/` (`fw_platform_size`, `fw_vendor`, `runtime`, `config_table`, `systab`, `esrt/`, `efivars/`), `/sys/firmware/acpi/fpdt/boot/*_ns`.

Firmware interface table

`f-w-table` - Not present on most PCs

In plain words. The Firmware Interface Table described firmware components on Intel Itanium (IA-64) servers.

What you see. Status ("Not present on this system"), Structure ("IA-64 (Itanium) firmware interface table"), Reason ("IA-64 only"), Firmware type.

What "not present" means. Always the case on x86 PCs.

Hardware summary

`hardware` - Linux data

In plain words. A one-page summary of the main parts of the PC.

What you see.

Field	Meaning
Processor	CPU model name
Logical CPUs	Number of CPU threads
System	System maker and product name (`NA` if the firmware left a placeholder)
Motherboard	Board maker and model
BIOS	Vendor, version, date, and `(UEFI)` or `(Legacy)`
Memory	Total, type, speed, slots used, and the first stick's maker and part number
TPM	TPM 2.0, TPM 1.2 or none

Field	Meaning
Secure Boot	Enabled / Disabled / "n/a (legacy BIOS)"

Then a device list from `lspci`: **Class**, **Address**, **Device** for graphics, audio, network, NVMe, SATA, RAID, USB and Thunderbolt controllers.

Healthy / typical values. Author's PC: AMD Ryzen 7 7800X3D, 16 logical CPUs, ASUS TUF GAMING X870-PRO WIFI7 W NEO, BIOS 1805 (UEFI), 32GB DDR5 6000 MT/s in 2 of 4 DIMMs, TPM 2.0, Secure Boot Disabled; Radeon RX 9070 series graphics, three NVMe controllers, RTL8126 5GbE, RTL8922AE Wi-Fi 7, ASM4242 USB4.

Unusual values and what they suggest. "System ASUS NA": self-built boards often leave the system product name as a placeholder. That is normal. A missing controller (for example no network) usually means it is disabled in the BIOS setup.

Where Linux gets it. `/sys/class/dmi/id`, `/sys/class/tpm/tpm0`, efivars, `lspci -mm`.

HECI (Intel ME) status

`heci-status` - Not present on most PCs

In plain words. HECI (Host Embedded Controller Interface) is how the OS talks to the Intel Management Engine, a separate processor inside Intel chipsets. AMD has its own equivalent, the PSP (Platform Security Processor). This item reports either.

What you see.

- **Intel MEI / HECI** - `present (/sys/class/mei)` or "Not present on this system".
- **AMD PSP / CCP** - the PCI address of the AMD security processor bound to the `ccp` driver, or "no ccp-bound device".

Healthy / typical values. Author's PC (AMD): no Intel MEI; AMD PSP at `0000:75:00.2` (ccp).

What "not present" means. On an AMD PC, Intel MEI is never present. On an Intel PC with no MEI, the `mei_me` driver is not loaded or the ME interface is hidden in the BIOS.

Where Linux gets it. `/sys/class/mei`, `/sys/bus/pci/drivers/ccp`.

Background. The **PSP** runs the firmware TPM (fTPM) and memory encryption on AMD systems. It starts before the main CPU.

Machine

`machine` - Linux data

In plain words. The main page of this group, modelled on SIV's Machine panel: the SMBIOS identity of the PC in a compact grid, from BIOS to CPU socket to memory slots. The **Machine** panel button opens it.

What you see. Rows of four columns:

Row	Contents
SMBios	Version, record count ("Count (needs root)" without the cache), table size in decimal and hex
BIOS	Version, date, release, vendor, plus ROM base address and ROM size (cache)
System	Product name, version, vendor and UUID (needs root if not readable)
(second line)	Family, SKU, system serial number
Chassis	Version and chassis type (Desktop, Tower, Laptop, Mini PC...), vendor, serial, asset tag
Board	Revision, model, board serial, maker
CPU-0 Socket #0	CPU vendor ID (`AuthenticAMD` / `GenuineIntel`), socket name (AM5, LGA1700...), model, max speed, external clock, cores, threads, voltage
System Boot	SMBIOS boot status ("No errors detected"), and `UEFI Secure Boot on/off` or `Legacy BIOS`
Memory Array	Location, maximum capacity, ECC type, DIMMs used of total, total installed
One row per slot	Slot, bank, then size / type / configured speed and maker / part, or `Empty`

Placeholder strings the firmware leaves ("To be filled by O.E.M.", "Default string", "System Product Name"...) are shown as `NA`. If the record count, serials or socket are missing, the page ends with the "Needs root" hint.

Healthy / typical values. Author's PC: SMBios V3.6.0, Count 80, size 3448 (0D78); BIOS 1805 release 18.5, ROM 64 MiB; System `NA NA ASUS` (placeholders); Chassis Desktop; Board Rev 1.xx TUF GAMING X870-PRO WIFI7 W NEO, serial XXXX; socket AM5, Max 5050 MHz, Ext 100 MHz, 8 cores, 16 threads, 1.3 V; System Boot "No errors detected", UEFI Secure Boot off; Memory Array maximum 256GB, ECC None, 2 of 4 DIMMs, 32GB.

Unusual values and what they suggest.

- **Ext** (external clock) not 100 MHz: base clock (BCLK) overclocking is active.
- **ECC** other than None on a desktop with normal memory is unusual; with ECC sticks installed, "None" means ECC is not enabled.
- **System Boot** status other than "No errors detected": the firmware logged a problem at the last boot (for example a memory training failure or a watchdog reset).

Where Linux gets it. `/sys/class/dmi/id/*`, `/run/udev/data/+dmi:id`, `/proc/cpuinfo`, efivars, plus SMBIOS types 0, 1, 4, 16, 32 from the srs-root `dmidecode.txt`.

Machine options

`machine-opt` - Linux data

In plain words. A checklist of the data sources this group can use on your PC, so you can see at a glance why an item is empty.

What you see. One tick line per source, `[=]` available, `[_]` not available, with its path:

- DMI identification (unprivileged) - `/sys/class/dmi/id/`
- SMBIOS memory devices via udev - `/run/udev/data/+dmi:id`
- dmidecode cache (serials, UUID, socket) - `~/.cache/srs/root/dmidecode.txt`
- SPD decode cache - `~/.cache/srs/root/spd.txt`
- ACPI table copies - `~/.cache/srs/root/acpi-tables/`
- I/O memory map cache - `~/.cache/srs/root/iomem.txt`
- spd5118 DIMM thermal sensors - hwmon devices named `spd5118`
- EDAC memory controllers - `/sys/devices/system/edac/mc`
- UEFI variables (read-only) - `/sys/firmware/efi/efivars`
- TPM - `/sys/class/tpm/tpm0`
- CMOS RTC - `/proc/driver/rtc`
- ACPI WMI bus - `/sys/bus/wmi/devices`
- Three lines that are always off by design: direct SMBus / SPD EEPROM reads, ACPI method evaluation / EC writes, and `/dev/mem`, `/dev/nvram`, `/dev/port`.

Healthy / typical values. Author's PC: everything on except EDAC (non-ECC memory) and the three always-off lines.

Unusual values and what they suggest. Cache lines off: run **Menu > Run as root (srs-root)**. EDAC off with ECC memory installed: load the `amd64_edac` (AMD) or `ie31200_edac` / `igen6_edac` (Intel) module, or ECC is disabled in the BIOS.

Where Linux gets it. Checks that each path exists.

Memory

`memory` - Linux data

In plain words. How much memory is in use, how much is free, and the detailed counters Linux keeps. Refreshes every second.

What you see. First a usage table:

Row	Used	Available	Total	Load
System Physical Memory	RAM in use (total minus "available")	RAM available to programs	Usable RAM	Percent used
System Paging File (swap)	Swap in use	Free swap	Swap size	Percent used
Commit Charge	Memory programs have asked for (`Committed_AS`)	Commit limit minus that	Commit limit	Percent of the limit

Then the `/proc/meminfo` counters: MemFree, Buffers, Cached, SwapCached, Active, Inactive, Dirty, AnonPages, Mapped, Shmem, Slab, SReclaimable, KernelStack, PageTables, VmallocUsed, Percpu, HardwareCorrupted (red if not 0), AnonHugePages, HugePages_Total, Hugepagesize, Zswap, Zswapped, DirectMap4k / 2M / 1G.

Then:

- **Installed (SMBIOS)** - capacity and slots used, from the firmware.
- **Memory Speed** - type, configured speed and rated speed.
- **Memory Blocks** - online and total memory blocks, block size, online total.
- **Auto Online Blocks** - policy for hot-added memory.
- **EDAC** - number of memory controllers with error reporting and, for the first one, its name and counts of corrected (CE) and uncorrected (UE) errors; or "no memory controllers registered".

Healthy / typical values. Author's PC, desktop in use: 10.60 GB used of 30.45 GB (35%), swap 6.39 GB of 48.04 GB used, commit charge 39.57 GB of a 63.26 GB limit; Cached 16.38 GB; HardwareCorrupted 0B; installed 32GB, DDR5 6000 MT/s (rated 4800); 251 of 251 blocks online.

Unusual values and what they suggest.

- **Total well below installed** (for example 24 GB of 32 GB): memory reserved for integrated graphics, or a stick not detected. A small gap (here 30.45 of 32 GB) is normal: firmware, the kernel and reserved areas take some.
- **Cached** large and **MemFree** small: normal. Linux uses spare RAM as disk cache and gives it back when needed. Look at **Available**, not Free.
- **HardwareCorrupted** above 0: the kernel took pages offline because of memory errors. Test your RAM (memtest86+) and lower memory overclocks.
- **Commit Charge** close to the limit: programs may fail to get memory; add swap or RAM.
- **EDAC CE** rising: correctable errors on ECC memory, an early warning of a failing stick.

Where Linux gets it. `/proc/meminfo`, `/sys/devices/system/memory/`, `/sys/devices/system/edac/mc/`, SMBIOS type 17.

Background. Swap (Windows: paging file) is disk or compressed memory (zram, zswap) used when RAM is full. **Commit charge** is how much memory programs have reserved, not how much they actually touch.

Memory timings

`mem-tweak` - Linux data - Needs root data

In plain words. The page header says **Memory Timings**. It shows what your memory really runs at: its speed and clocks, the timings the memory controller uses right now, the BIOS's memory settings, and what each module says it can do (its JEDEC, EXPO and XMP profiles), so you can compare "what the profile says" with "what the BIOS set". Everything on it is only read. New in 2.4; before, it showed only the speed.

Which PCs. Live timings: **AMD Ryzen 7000 / 9000 desktop** processors (family 19h models 60h-7Fh, family 1Ah models 40h-4Fh), with the `ryzen_smu` driver installed (AUR `ryzen_smu-dkms-git`) and one **Menu > Run as root**. **Intel Core 6th gen to Core Ultra**: experimental (see below). The module profiles work on any PC with DDR5 modules that the kernel's `spd5118` driver has found, without root. Other processors say "not supported yet".

What you see.

Clocks and voltages (from the ryzen_smu PM table, no root):

Field	Meaning
Memory speed	Data rate in MT/s (two transfers per MEMCLK cycle)
MEMCLK	The memory's own clock
UCLK	The memory controller's clock, with its ratio to MEMCLK: 1:1 is fastest, 1:2 is usual above about 6000 MT/s
FCLK	Infinity Fabric clock: the link between the CPU cores and the memory controller
VSOC	SoC voltage: memory controller and fabric
VDDP	Memory PHY (the controller's I/O circuits) voltage
VDDG IOD / CCD	Fabric link voltages; "not in this PM table version" when the table has none
VDD MISC	Voltage of other SoC parts

Memory controller (live): **Channels with a DIMM**, **Gear Down Mode (GDM)** (Enabled / Disabled), **Command rate** (1T / 2T), **Power down**, **Bank groups**, **Refresh mode** (normal, fine granularity, per bank or mixed) and **Nitro mode (Rx / Tx / Ctrl)**.

BIOS memory settings (AMD AOD table), only when the kernel lets srs-root read the table (see below):

Field	Meaning
ProcODT	The processor's on-die termination resistance (ohms): how the CPU's memory pins absorb signal reflections
Proc data drive strength, DRAM data drive strength, CAD bus drive strength	How hard the CPU, the memory chips and the command / address bus drive their signals (ohms; lower = stronger)
RttNomWr / RttNomRd, RttWr, RttPark / RttParkDqs	The memory chips' own termination settings (Off, or RZQ/n with its ohms)
DIMM VDD / VDDQ / VPP (set)	The module voltages the BIOS set
CPU VDDIO (memory)	The CPU's memory I/O voltage
Source	"the BIOS's AOD table; its timings match the live ones above"

When the table cannot be read, **Status** says why: "not readable on this kernel: the table sits in memory that the kernel keeps from programs" (kernels built with IO_STRICT_DEVMEM, which is most, block it unless booted with `iomem=relaxed`, which weakens that protection; SRS does not ask you to change it, and the timings do not need it), "not read yet (Menu > Run as root), or this BIOS has no AOD table", or "its layout is not known for this processor / BIOS yet (please report it)". The values are shown only when the table's own timings match the live ones, so a layout that does not fit a BIOS is never shown.

Timings - one row per timing, one column per channel (A, B ...), the **Time** in nanoseconds for the ones where that is meaningful, and **What it is** in plain English. In memory controller clocks: tCL, tRCDRD, tRCDWR, tRP, tRAS, tRC (the main ones), tRRDS / tRRDL, tFAW, tWTRS / tWTRL, tWR, tRTP, tCWL, tREFI (time between refreshes, in microseconds), tRFC / tRFC2 / tRFCsb (refresh cycle times), the read-to-read and write-to-write turnarounds (tRDRDSCL, tRDRDSC, tRDRDSD, tRDRDDD, tWRWRSCL, tWRWRSC, tWRWRSD, tWRWRDD), tRDWR, tWRRD, tCKE, tXP, tCPDED, tPD, tCSH, tMOD, tMRD, tPPD, tSTAG, tSTAGsb and the PHY delays tPHYWRD, tPHYWRL, tPHYRDL. A channel that differs from the others is a normal result of memory training for the PHY and turnaround values.

Modules (SPD) - per DDR5 module, no root: **Slot**, **Module** (maker and part number), **Size** (size, ranks, chip size and width), **DRAM chips** (chip maker), **Made** (year and week), **Temperature**, and (opt-in) **PMIC set VDD / VDDQ / VPP** and **PMIC mode**. Then a profile table: **JEDEC**, **EXPO 1-2** and **XMP 1-3** with speed, CL-RCD-RP-RAS-RC, tRFC1 / 2 / sb, VDD / VDDQ / VPP and the profile's name.

DIMM voltages from their PMIC chips (opt-in). Each DDR5 module has a small power chip (PMIC). **Menu > Options > Read DIMM voltages from their PMIC chips (SMBus; from the next Run as root)** is off by default and asks first: on some boards the BIOS or the board's controller uses the same bus at the same time without telling Linux, and a collision there can, rarely, upset a sensor or the board's lighting until the next boot. When it is on, Run as root reads each PMIC's registers 0x00-0x3F with `i2cdump` (byte reads through the kernel's SMBus driver, no `-f`, nothing written) and the page shows the voltages it is set to (JESD301-2 voltage codes; "high-voltage mode" for overclocking PMICs, told apart by the module's profile voltage) and whether it is **secure (locked)** or **programmable (unlocked)**. The live voltage meter is not read: it needs a write to the PMIC.

Intel (experimental). srs-root runs `sysreport-search intel-imc-dump` as root: it takes the memory controller's register window (MCHBAR) address from the host bridge, maps it read-only and reads the timing registers with aligned 32-bit reads; the window is never switched on. The page shows **Memory controller**, **Memory speed** (MT/s and type), **Channels with memory** and **tCL-tRCD-tRP-tRAS**, with the status "experimental: Intel timings are not checked on a real PC yet - please report what you see (and what the BIOS says)". Covered: Skylake to Comet Lake, Rocket Lake, Alder / Raptor Lake, Meteor / Arrow Lake.

Healthy / typical values. Author's PC (Ryzen 7 7800X3D, 2 x 16 GB DDR5-6000 EXPO): 6000 MT/s, MEMCLK 3000 MHz, UCLK 1500 MHz (1:2), FCLK 2133 MHz, VSOC 1.248 V, VDDP 0.998 V; 2 channels, GDM Enabled, 1T; timings 30-38-38-96, tRC 146, tRFC 884 (294.67 ns), tREFI 65535; ProcODT 48 ohms, DIMM VDD / VDDQ / VPP set 1.435 / 1.435 / 1.900 V (with `iomem=relaxed`); modules G.Skill F5-6000J3038F16G (SK Hynix chips, EXPO and XMP 6000 30-38-38-96) and F5-6000U3636E16G (Samsung, XMP 6000 36-36-36-76), 28-29 °C.

Unusual values and what they suggest.

- Memory speed at the JEDEC speed (4800 MT/s on a 6000 kit): EXPO / XMP is off in the BIOS.
- Timings far looser than the profile at the same speed: the BIOS did not apply the profile, or you set them by hand.
- "the ryzen_smu driver is not loaded": install `ryzen_smu` (AUR `ryzen_smu-dkms-git`), then Menu > Run as root.
- "this processor's memory controller is not supported yet": only Ryzen 7000 / 9000 desktop so far.
- A module temperature above about 55 °C at idle: poor airflow over the memory.

Where Linux gets it. The srs-root cache `umc.txt` (AMD memory controller registers read through `/sys/kernel/ryzen_smu_drv/smn`: an address written there only selects what the driver reads, 4 bytes, never the 8-byte form that writes; each value read twice under a lock), `aod.bin` / `aod.txt` (the AOD table, `/dev/mem`, only when the kernel allows it), `pmic-*.txt` (opt-in), `intel-imc.txt` (Intel); `/sys/kernel/ryzen_smu_drv/pm_table` (clocks and voltages, no root); `/sys/bus/i2c/drivers/spd5118/*/eeprom` and the `spd5118` hwmon temperature (no root).

Background. SMN (System Management Network) is AMD's internal register bus. **UMC** is the Unified Memory Controller. **AOD** (AMD Overclocking Data) is a table the BIOS leaves in memory with its memory settings. **SPD** is the description each module carries; **EXPO** (AMD) and **XMP** (Intel) are tested faster

profiles stored in it. The register map and table layouts follow ZenStates-Core / ZenTimings by Ivan Rusanov and memtest86+ by Sam Demeulemeester (facts only, no code copied).

Overview

`overview` - Linux data

In plain words. SIV's classic overview header: one screen with the OS, board, user, uptime, power source, memory use, CPU and memory sticks. Refreshes every second.

What you see.

- **First line** - OS name and kernel, board maker and model, `host::user`, language (`LANG`), time zone, BIOS version and date.
- **Second line** - uptime as `DD HH:MM:SS` (days first), current date and time, `[BIOS version date]`, and the power source: `Mains Power` or `Battery Power`, followed by `Battery N%` for each battery the system reports.
- **Resource table** - "Resource Usage N%" (physical memory load), then three rows with **Current**, **Committed** and **Maximum**: System Physical Memory (RAM in use, total commit charge, usable RAM), System Paging File (swap in use, swap size), Commit Charge (limit) (committed memory, commit limit).
- **Linux** kernel release.
- **CPUs / Cores / Chips / DIMMs** - logical CPUs, physical cores, CPU packages, sticks installed.
- CPU model name.
- One line per installed stick: bank, slot, size, type, speed, maker and part number.

Healthy / typical values. Author's PC: `CachyOS 7.3.0-... ASUS TUF GAMING X870-PRO WIFI7 W NEO MYPC::user en_US.UTF-8 EDT BIOS 1805 09/18/2026`; uptime 21 h 46 min; Mains Power; Resource Usage 35%; CPUs 16 Cores 8 Chips 1 DIMMs 2.

Unusual values and what they suggest. The Committed figure on the physical-memory row is the total commit charge, so it can be larger than the RAM size; that is normal when swap exists. A "Battery" entry on a desktop usually comes from a wireless mouse, keyboard or UPS (uninterruptible power supply) that reports a battery to Linux.

Where Linux gets it. `/etc/os-release`, `/sys/class/dmi/id`, `/proc/uptime`, `/sys/class/power_supply`, `/proc/meminfo`, `/sys/devices/system/cpu/*/topology`, SMBIOS type 17.

Physical memory map

`physical` - Linux data

In plain words. The memory map the firmware gave Linux at boot: which address ranges are usable RAM and which are reserved for firmware, ACPI or devices.

What you see.

Field	Meaning
#	Entry number
Start / End	First and last physical address (hex)
Size	Length of the range
Type	`System RAM`, `Reserved`, `ACPI Tables` (reclaimable after boot), `ACPI Non-volatile Storage` (kept for firmware), `Persistent Memory`, `Unusable memory` …

Then **Total** rows: size and count of entries per type. Then, with the srs-root cache, the top-level `/proc/iomem` list: the same address space with owners, including PCI windows, the IOMMU, I/O APICs, HPET and the graphics card's memory window. Without it, a note that resource owners need root.

Healthy / typical values. Author's PC: 23 entries; System RAM total 31.10 GB in 8 ranges, Reserved 13.68 GB (that is address space, mostly not real RAM), ACPI NVS 32.06 MB, ACPI Tables 8.82 MB. The biggest RAM range is 0x100000000-0x87DEFFFFF (29.97 GB above 4 GB). The iomem list shows `PCI ECAM` at 0xE0000000 (256 MB of PCIe configuration space) and a large window at the top for the graphics card (Resizable BAR).

Unusual values and what they suggest.

- System RAM total far below installed RAM: a stick is not detected, or memory is set aside (integrated graphics, `memmap=` kernel option, crash kernel).
- `Unusable memory` or `Bad RAM` entries: the firmware marked memory as faulty.
- Reserved address space is not lost RAM: most of it is device windows and the "PCI hole" below 4 GB, which modern systems remap above 4 GB.

Where Linux gets it. `/sys/firmware/memmap/*` (start, end, type); owners from the srs-root `iomem.txt` (copy of `/proc/iomem`, whose addresses are hidden from normal users).

Background. This is the **E820** / UEFI memory map. Physical addresses are also used by devices, so the address space is bigger than the installed RAM.

SPD (DIMMs)

`spd` - Linux data

In plain words. The SPD (Serial Presence Detect) is the small description each memory stick carries: maker, size, speeds, timings, and XMP / EXPO overclock profiles. This item shows the decoded SPD.

What you see. Two possible forms:

- **With the srs-root cache `spd.txt`**: the text output of `decode-dimms` (from i2c-tools), unchanged. A full decode lists per stick the module maker, manufacture date, size and organisation, supported speeds and CAS latencies, primary timings, and any XMP / EXPO profiles.
- **Without it**: an SMBIOS-based summary per installed stick: bank and slot, Module Manufacturer, Manufacturer ID (JEDEC), Part Number, Size, Type (type, form factor, detail), Rated Speed (SPD/JEDEC), Configured Speed, Ranks, SPD Hub (the `spd5118` device). Two notes explain that the full

decode comes from the cache and that SRS never reads the SPD chip directly. Then the "Needs root" hint.

When the cached `decode-dimms` run decoded no DIMMs ("Number of SDRAM DIMMs detected and decoded: 0", normal with DDR5), SRS says so and shows the SMBIOS summary instead.

Healthy / typical values. A full decode shows one block per stick. On the author's DDR5 PC `decode-dimms` decodes 0 DIMMs, so the page shows the SMBIOS summary of both sticks.

For DDR5 the Memory timings item (2.4) shows each module's maker, part number and JEDEC / EXPO / XMP profiles from the kernel's `spd5118` copy of the SPD, without root.

Unusual values and what they suggest. "0 DIMMs decoded" with DDR5: `decode-dimms` needs the `spd5118` driver's `eeprom` file and a version of i2c-tools that understands DDR5. Updating i2c-tools, then running srs-root again, usually fixes it. Rated speed and part number are also in DIMMs.

Where Linux gets it. srs-root cache `spd.txt` (output of `decode-dimms`, which reads `/sys/bus/i2c/drivers/spd5118/*/eeprom`); fallback SMBIOS type 17.

Background. XMP (Intel Extreme Memory Profile) and **EXPO** (AMD Extended Profiles for Overclocking) are tested faster settings stored in the SPD. The BIOS applies them when you enable the profile.

SPD thermal

`spd-temp` - Linux data

In plain words. The same DDR5 stick temperature table as DIMM status/temperatures, under the SPD name. Refreshes every 2 seconds.

What you see. Sensor, Device, Slot, Temp, Min, Max, Crit, LCrit, Alarms, Enabled, exactly as described for DIMM status.

Healthy / typical values. Author's PC: A1 31.0 °C, B1 30.5 °C, limits 0 / 55 / 85 / 0 °C, no alarms.

Where Linux gets it. `spd5118` hwmon attributes `temp1_*`.

System CMOS

`system-cmos` - Linux data

In plain words. The CMOS RAM is a small battery-backed memory next to the clock, where the BIOS keeps a few settings. Since 2.4 SRS shows its contents, read from the kernel's own copy (no root, no direct hardware access), and decodes the standard bytes.

What you see.

- **RTC driver** - e.g. `rtc_cmos PNP0B00:00`.
- **CMOS battery** - `okay` (green) or the warning the driver reports (red).

- **CMOS RAM** - size and range, e.g. "114 bytes, addresses 0x0E - 0x7F", and where it came from.
- **Standard IBM AT bytes** - 0x0E diagnostic status, 0x0F shutdown status, 0x10 floppy drives, 0x15-0x16 base memory, 0x17-0x18 extended memory (the AT field; it cannot show more than 64 MB), 0x2E-0x2F checksum (and whether it matches), 0x32 century.
- **CMOS RAM contents** - a hexdump with CMOS addresses. With **Redact Reports** on, the bytes are masked as `XX` in reports and copied text, because CMOS can hold the BIOS password check value.
- Notes: the bytes not listed belong to the BIOS maker and are not decoded; the clock registers are on CMOS / RTC status.

Healthy / typical values. Author's PC: `rtc_cmos PNP0B00:00`, battery okay, 114 bytes; no diagnostic errors, normal shutdown status, no floppy drives, base / extended memory 0 KB and the AT area all zero (a UEFI BIOS does not use it), century 20.

Unusual values and what they suggest. Battery not okay: replace the coin cell (usually a CR2032) before BIOS settings and the clock get lost. A diagnostic status other than 0x00: the BIOS flagged a problem at the last start (on old PCs: clock or checksum errors). "not available: the kernel has no cmos_nvram0 nvmem device": the `rtc_cmos` driver did not register it.

Where Linux gets it. `/sys/bus/nvmem/devices/cmos_nvram0/nvmem` (the `rtc_cmos` driver's read-only copy), `/sys/class/rtc/rtc0/name`, `/proc/driver/rtc`.

System EBDA

`system-ebda` - Linux data - Needs root data

In plain words. The EBDA (Extended BIOS Data Area) is a small block just below 640 KB that legacy BIOSes used for their own data. Since 2.4 srs-root reads the pointer to it and, if there is one, its contents; UEFI PCs usually have none.

What you see.

- **EBDA pointer (0x40E)** - the segment the BIOS Data Area points to, e.g. "0000: no EBDA (normal on UEFI firmware)", with how it was read (once by srs-root, from the first megabyte of memory, read-only). With an EBDA: its address, **size** and a hexdump of its contents (at most 64 KiB).
- Without root data: "not read yet: needs the root cache (Menu > Run as root)". When the kernel has no `/dev/mem`, the page says so.
- **Conventional memory** - the first memory-map entry (start - end, type).
- **EBDA (implied)** - either "none reserved: the firmware map gives RAM up to 0x9FFFF", or the reserved range between the end of that RAM and 0x9FFFF.
- **Firmware** - "UEFI (EBDA kept only for CSM/legacy compatibility)" or "Legacy BIOS".
- With the srs-root cache: the `/proc/iomem` lines below 1 MB (Reserved, System RAM, PCI Bus, System ROM).

Healthy / typical values. Author's PC: pointer 0000 (no EBDA), conventional memory 0x0 - 0x9FFFF System RAM, UEFI; iomem shows the System ROM at 0xF0000-0xFFFFF.

Unusual values and what they suggest. Nothing to fix here. An EBDA on a UEFI PC means CSM (legacy boot support) is active. "invalid pointer (at or above 0xA0000), not read": the pointer is not a real EBDA address; SRS does not read there.

Where Linux gets it. srs-root cache `ebda.txt` / `ebda.bin` (`/dev/mem` below 1 MB, which kernels with STRICT_DEVMEM still allow, read-only), `/sys/firmware/memmap/0/*`, srs-root `iomem.txt`.

BIOS IDs

`system-ids` - Linux data - Needs root data

In plain words. The identity strings in SMBIOS: UUID, serial numbers, asset tags, and the memory sticks' serials. Useful for warranty claims and asset lists. Keep them private when you share reports.

What you see.

Field	Meaning
System UUID	Unique ID of this machine (green)
System Serial / Board Serial / Chassis Serial	Serial numbers from SMBIOS types 1, 2, 3
System SKU / System Family	Product variant and family
Board Asset Tag / Chassis Asset Tag	Inventory tags (often blank)
DIMM Serial *channel slot*	Serial of each installed memory stick
DMI modalias	One-line summary Linux uses to match drivers to the board (`bvn` BIOS vendor, `bvr` version, `bd` date, `svn` system vendor, `pn` product name, `rvn` board vendor, `rn` board name, `rvr` board revision, `cvn` chassis vendor...)

Fields that need root and are not in the cache show "(needs root)", followed by the root hint.

Healthy / typical values. Author's PC: a UUID, Board Serial XXXX, and placeholder strings for the rest ("System Serial Number", "Default string", "SKU", "To be filled by O.E.M."), which is typical for a retail motherboard. DIMM serials: one real serial and one `00000000`.

Unusual values and what they suggest. Placeholder strings are normal on self-built PCs. On a brand-name PC they can mean the board was replaced and the service centre did not program the IDs, which can break OEM Windows activation or warranty look-ups. An all-`FF` or all-`0` UUID means the firmware did not set one.

Where Linux gets it. `/sys/class/dmi/id/*` (serials and UUID are root-only there), else the srs-root `dmidecode.txt`; DIMM serials from SMBIOS type 17; `/sys/class/dmi/id/modalias`.

TPM device

`tpm-device` - Linux data

In plain words. The TPM (Trusted Platform Module) as Linux sees it: version, driver, how it is connected, and its measurement banks.

What you see.

Field	Meaning
TPM Version	TPM 2.0 or TPM 1.2
Driver	e.g. `tpm_crb_acpi`, `tpm_tis`
Interface	`CRB (Command Response Buffer)` or `TIS/FIFO`
Implementation	On AMD with CRB: "firmware TPM (AMD fTPM in the PSP, likely)"
ACPI Node / ACPI Path / ACPI Status	The ACPI device behind it
Physical Presence Interface	PPI version (how the OS asks the firmware to clear or enable the TPM)
PCR Banks	Hash algorithms with PCR banks, e.g. `sha256`
Device Nodes	`/dev/tpm0` and `/dev/tpmrm0` (the resource-managed node)
ACPI TPM2 table	Size of the TPM2 table

PCR values are not read by default. `tpm-device:pcr` adds the 24 sha256 PCR values (PCR 0 to PCR 23). If there is no TPM: "No TPM (/sys/class/tpm empty)".

Healthy / typical values. Author's PC: TPM 2.0, `tpm_crb_acpi`, CRB, AMD fTPM, `MSFT0101:00`, `\_SB_.TPM_`, `0x0F PESF`, PPI 1.3, bank sha256, `/dev/tpm0 /dev/tpmrm0`, TPM2 table 76 bytes.

Unusual values and what they suggest.

- No TPM: enable fTPM (AMD) or PTT (Intel) in the BIOS setup.
- Only a `sha1` bank: old firmware or TPM 1.2; some disk encryption tools want sha256.
- PCR values that change after a BIOS update or settings change are expected. If your disk encryption is sealed to PCRs, it will then ask for the recovery key.

Where Linux gets it. `/sys/class/tpm/tpm0/*` (`tpm_version_major`, `device/driver`, `device/firmware_node`, `ppi/version`, `pcr-*`), `/sys/firmware/acpi/tables/TPM2` size.

Background. PCRs (Platform Configuration Registers) hold hashes of everything that ran during boot (firmware, boot loader, kernel). Disk encryption can be "sealed" so the key is only released when the PCRs match.

UEFI data

`uefi-data` - Linux data

In plain words. Lists every UEFI variable stored in the firmware's flash: boot entries, Secure Boot keys, and the board maker's own settings. Names and sizes only.

What you see. Variables grouped by vendor GUID (the 128-bit ID of whoever defined them). Each GUID line may carry a known label: "EFI Global Variable" (the standard boot variables such as `BootOrder`, `Boot0000`, `SecureBoot`, `PK`, `KEK`), "Image Security Database" (`db` allowed signatures, `dbx` revoked

signatures), "systemd Boot Loader Interface" (variables set by systemd-boot or GRUB). Under each GUID: **Variable** name and **Size** in bytes.

Detail view (`uefi-data:<Name-GUID>`, the full file name): **Attributes** in hex with flags **NV** (non-volatile, survives power-off), **BS** (boot-services access), **RT** (runtime access, visible to the OS), **TA** (time-based authenticated, i.e. signed), the size, and a hex dump of up to 4 KB. Read-only.

Healthy / typical values. Author's PC: 159 variables. The standard group has `Boot0000`, `Boot0001`, `BootCurrent`, `BootOrder`, `KEK`, `ConIn`, `ConOut`; `db` is 11440 bytes and `dbx` 22314 bytes. Many vendor groups hold ASUS and AMD BIOS settings (`Setup`, `AmdSetup...`, `AsusQFanSetupData`, `MyFav`, `StdDefaults`...).

Unusual values and what they suggest.

- A very large number of variables or a nearly full variable store can make firmware updates or new boot entries fail. Old `Boot####` entries can be removed with `efibootmgr`.
- An empty `dbx` (or a very small one) means the revocation list was never updated, which matters if you use Secure Boot.

Where Linux gets it. `/sys/firmware/efi/efivars/` (each file is 4 bytes of attributes then the data).

UEFI information

`uefi-info` - Linux data

In plain words. The key UEFI boot settings: Secure Boot mode, boot menu timeout, language, which boot entry started this session, and the boot order.

What you see.

Field	Meaning
UEFI Platform	64-bit or 32-bit UEFI
SecureBoot	1 (on) / 0 (off)
SetupMode	1 = no Platform Key installed, Secure Boot keys can be changed freely
AuditMode / DeployedMode	Extra Secure Boot modes, when the firmware supports them
Timeout	Firmware boot menu timeout in seconds
PlatformLang	Firmware language
OsIndicationsSupported	Hex mask of OS-to-firmware requests the firmware supports; SRS names BootToFwUI (reboot into setup), FileCapsule and CapsuleResultVar (firmware updates)
Boot Loader / Firmware Info / Firmware Type / Kernel Stub	Strings from the systemd Boot Loader Interface, when the boot loader sets them
BootCurrent	Boot entry used for this boot (green)
BootOrder	Boot entries in the order the firmware tries them

Field	Meaning
Boot Entries	One line per entry in BootOrder: `Boot####`, `*` if active, and its description

If the PC did not boot via UEFI: "Not booted via UEFI (/sys/firmware/efi absent)".

Healthy / typical values. Author's PC: 64-bit, SecureBoot 0 (off), SetupMode 0, timeout 1 s, en-US, Boot Loader GRUB 2.16, BootCurrent Boot0001 ("UEFI OS", the Linux entry), BootOrder 0001,0000,001D,001C,001E,001F, with a Windows Boot Manager entry and several USB-stick entries.

Unusual values and what they suggest.

- **SetupMode 1**: no Platform Key is installed. Secure Boot cannot be enforced until keys are enrolled (normal right after "Clear Secure Boot keys").
- **BootCurrent** not the first entry in BootOrder: the PC booted from a fallback, a one-time boot (boot menu) or the first entry failed.
- Stale entries (removed drives, old distributions) can be cleaned with `efibootmgr -b XXXX -B`.

Where Linux gets it. `/sys/firmware/efi/fw_platform_size` and efivars `SecureBoot`, `SetupMode`, `AuditMode`, `DeployedMode`, `Timeout`, `PlatformLang`, `OsIndicationsSupported`, `BootCurrent`, `BootOrder`, `Boot####`, and the systemd `Loader*` / `StubInfo` variables.

Sensors / Cooling / Power (SRS Section Reference)

Back to the section index - Glossary

Where to find these items (SRS 2.0). In the main window they are on the **Sensors** page (Alt+8). Its tabs are Sensor status (with Zero and Hide unused), Hardware status, Cooling status (fans) and Power status; **All N items** lists the rest. The Overview's **Is my PC OK?** sums up the temperatures and alarms. You can also type an item's name or id into the search box (Ctrl+F). The rest of this introduction describes the classic view (Menu -> Classic View), with its Menu and panel buttons.

This group shows what your PC can measure about itself: temperatures, fan speeds, voltages, power use and batteries. It also covers the chips and firmware that supply those readings (the Super I/O chip, ASUS WMI, IPMI on servers) and the export and "setup" items that SIV offers for its own sensor tools. Open it from **Menu -> Sensors / Cooling / Power -> item**, or use the **Sensors** panel button. The panel has sub-buttons: **Zero** (reset the Minimum / Maximum columns), **Status** (Sensor status), **Cooling** (Cooling status), **Power** (Power status) and **Hide** (hide unused sensors). Everything here is read only: SRS never changes a fan speed, a limit or a register. Four items are only complete with root data from **srs-root** (Menu -> Run as root): ASUS ATK/WMI info, ASUS WMI ACPI methods, Super I/O LDNs and SIO debug.

All examples come from the author's PC: an ASUS TUF GAMING X870-PRO WIFI7 W NEO board (Nuvoton NCT6799 Super I/O chip), an AMD Ryzen 7 7800X3D, a Radeon RX 9070 XT, DDR5 memory, three NVMe drives and one SATA hard disk.

Contents

- AIO Link export `aio-export`
- ASUS ATK/WMI info `asus-info`
- ASUS WMI ACPI methods `asus-wmi`
- ASUS ATK0110 debug `atk-debug`
- Battery `battery`
- Cooling status (fans) `cooling`
- Sensor formula setup `formula-setup`
- Hardware status `hw-status`
- IPMI dump `ipmi-dump`
- IPMI status `ipmi-info`
- IPMI time `ipmi-time`
- IPMI wait `ipmi-wait`
- kWh cost `kwh-cost`
- LCD panel setup `lcd-panel`
- Super I/O LDNs `ldn-info`
- AIO Link devices `link-devices`
- AIO Link dump `link-dump`

- AIO Link fans `link-fans`
- AIO Link LEDs `link-leds`
- AIO Link limits `link-limits`
- AIO Link power `link-power`
- AIO Link setup `link-setup`
- AIO Link status `link-status`
- AIO Link test `link-test`
- AIO Link update `link-update`
- Power status `power`
- PWM export `pwm-export`
- QST status `qst-status`
- Sensor export `sen-export`
- Sensor status `sensors`
- SIO debug `sio-dbg`
- Synthetic sensors setup `synthetic`
- Thermal zones `thermal-zones`
- UPS export `ups-export`

AIO Link export

`aio-export` - Not present on most PCs

In plain words. An AIO (all-in-one) liquid cooler or a fan/RGB hub can report its own pump speed, liquid temperature and fan speeds over USB. SIV can read some of these hubs directly. SRS only checks whether Linux already has a kernel driver for one, and tells you if it found any.

What you see. This item, and all eleven "AIO Link ..." items below, print the same short report:

- Status line - "No AIO Link Devices found on this system", or one line per hub found (`hwmonN` - driver name). When a hub is found, a note says its readings are part of Sensor status and Cooling status.
- Checked - which kernel drivers were looked for: Corsair (Commander Pro `corsair_cpro`, Corsair PSUs `corsair-psu`), NZXT (`nzxt-kraken2`, `nzxt-kraken3`, `nzxt-smart2`), Aquacomputer (`d5next`, `quadro`, `octo`, `highflownext`), ASUS ROG Ryujin and Gigabyte Waterforce.
- A note that USB HID cooler devices belong to CoolerControl or liquidctl. SRS never opens `hidraw` (the raw USB HID device files), so it cannot clash with those tools.

Healthy / typical values. On the author's PC: "No AIO Link Devices found". That is normal for any cooler without a Linux hwmon driver, and for an AIO whose pump is plugged into a motherboard header. In that case the pump shows up as an ordinary fan in Cooling status.

Unusual values and what they suggest. You own a supported hub but nothing is found: the driver module is not loaded, or another tool has claimed the device. Check `lsmod` for the driver name.

Where Linux gets it. `/sys/class/hwmon/hwmon*/name`, matched against the list of cooler drivers.

Background. Corsair iCUE LINK is Corsair's single-cable system for fans, pumps and lights. SIV talks to such hubs itself. On Linux the usual tools are liquidctl and CoolerControl, which drive the hub over USB HID. SRS stays out of their way on purpose.

ASUS ATK/WMI info

`asus-info` - Linux data - Needs root data

In plain words. ASUS boards hide some sensor and control functions behind firmware interfaces called ATK and ASUS WMI. This item shows which of those interfaces your board has, and whether any Linux ASUS sensor driver supports your exact board.

What you see.

Field	Meaning	Why it matters
Board vendor / Board / BIOS	From the DMI (Desktop Management Interface) tables	Driver support is decided per board name
ASUS WMI GUID	`97845ED0-...` (the ATK / ASUSWMI interface). Green if present	If present, the ASUS WMI interface exists in firmware
ASUS sensor drivers	One line per driver: `asus_ec_sensors`, `asus_wmi_sensors`, `asus_atk0110`, `asus_wmi`, `asus_nb_wmi`. Each says loaded / not loaded, then either "supports this board" or why not	Tells you if an extra ASUS driver could add sensors
ASUS WMI blocks	Table of WMI blocks that belong to ASUS firmware devices: GUID, Obj, Inst, ACPI parent, Driver, Description	Shows what ASUS put in the firmware
ACPI tables / DSDT / SSDT	Where the copied ACPI tables are, and which ones were copied	Needed if you want to read the firmware code yourself

The reasons a driver line can give:

- "N supported boards, this board is not listed" - the driver has a list of boards; yours is not on it.
- "binds by ACPI device id, not by board (DMI)" - the driver loads on a firmware device, not a board name (true for `asus_atk0110`).
- "module not built for this kernel" - your kernel has no such module file.
- "modinfo failed" - SRS could not read the module's info.

WMI table columns (same as in ASUS WMI ACPI methods):

- GUID - the 128-bit name of the interface.
- Obj - the two-letter object id. `MO` is common for a data block holding a binary MOF (a description of the interface). `ev D2` means an event, with notify id `D2`.
- Inst - how many instances the block has.
- ACPI parent - the `PNP0C14:NN` firmware device and its UID (for example `AsusMbSwInterface`).
- Driver - the Linux driver bound to the block, or `-` for none.
- Description - a name SRS knows for that GUID, if any.

Healthy / typical values. On the author's TUF GAMING X870-PRO WIFI7 W NEO (BIOS 1805): the ASUS WMI GUID is present on `PNP0C14:02 AsusMbSwInterface`; `asus_ec_sensors` says "60 supported boards, this board is not listed" and `asus_wmi_sensors` says "16 supported boards, this board is not listed". That is fine: board sensors come from the Super I/O chip instead (see Sensor status). Only the `wmi-bmof` driver is bound.

Unusual values and what they suggest.

- "supports this board" but "not loaded": you could get extra readings (often chipset, VRM or water temperatures) by loading that module.
- ACPI tables "Needs root: Menu > Run as root (srs-root)": the copy of the tables has not been made yet.

Where Linux gets it. `/sys/class/dmi/id/*`, `/sys/bus/wmi/devices`, `modinfo -F alias` (the `dmi:` board patterns), `/sys/module`, and the srs-root copy of `/sys/firmware/acpi/tables`.

Background. WMI (Windows Management Instrumentation) is a Windows interface. Board makers put WMI "blocks" inside the ACPI (Advanced Configuration and Power Interface) firmware so Windows tools can call them. Linux can see these blocks through the `wmi` bus. A driver must exist for each one before it does anything useful.

ASUS WMI ACPI methods

`asus-wmi` - Linux data - Needs root data

In plain words. A full list of every WMI firmware device and WMI block on the board, not only the ASUS ones. SRS lists them but does not run them.

What you see.

- ACPI WMI devices (PNP0C14) - one line per WMI device: device name, UID (the firmware's own name for it, for example `AsusLedControlWmi`) and ACPI path (for example `\ALCW`).
- WMI GUID blocks - the same table as in ASUS ATK/WMI info, but for all blocks.
- A note: "WMI methods are not evaluated". Calling firmware methods from user space could change settings, so SRS never does it.
- ACPI tables / DSDT / SSDT - as in ASUS ATK/WMI info.

Healthy / typical values. The author's board has six WMI devices (`PNP0C14:00` to `:05`), including `AsusLedControlWmi` (RGB lights), `AsusMbSwInterface` (the main ASUS WMI interface), `AsusMiniZipWmi` and `AsusWifiOptionWmi`, plus 14 GUID blocks. Most have no Linux driver (`-`). That is normal.

Unusual values and what they suggest. "No WMI devices (/sys/bus/wmi/devices empty)": the `wmi` kernel module is not loaded, or the firmware has no WMI at all (common on older or server boards).

Where Linux gets it. `/sys/bus/acpi/devices/PNP0C14*` (uid, path), `/sys/bus/wmi/devices`, and the srs-root ACPI table copy.

Background. `PNP0C14` is the standard ACPI id for a "WMI mapper" device. Each one holds a `_WDG` table that lists its GUID blocks. The method code lives in the DSDT and SSDT tables (the main and extra ACPI code tables).

ASUS ATK0110 debug

`atk-debug` - Not present on most PCs

In plain words. Older ASUS boards (roughly up to the early 2010s) gave sensor readings through a firmware device called ATK0110. This item checks for it.

What you see.

- ACPI device - the ATK0110 device name if found, or "No ATK0110 ACPI device (used by older ASUS boards only)".
- asus_atk0110 module - loaded / not loaded.
- Checked - `/sys/bus/acpi/devices/ATK0110*`.

Healthy / typical values. Modern boards, like the author's X870, have no ATK0110. "Not present" is the expected answer.

Where Linux gets it. `/sys/bus/acpi/devices`, `/sys/module/asus_atk0110`.

Battery

`battery` - Not present on most PCs

In plain words. Lists every battery or UPS that Linux knows about. On a desktop that is often only the batteries inside wireless controllers, mice and headsets.

What you see. One block per battery, named after the Linux power-supply entry. Each block shows only the fields the driver provides:

Field	Meaning
type	`Battery` or `UPS`
scope	`System` (powers the PC) or `Device` (powers a peripheral, like a game pad)
online	1 if on external power
status	Charging, Discharging, Full, Not charging
capacity	Charge level in %
capacity_level	A rough level (Full, Normal, Low, Critical) when no % is given
health	Good, Overheat, Dead and so on
technology	Li-ion, Li-poly ...
manufacturer / model_name	Who made it

Field	Meaning
cycle_count	Charge cycles so far (battery wear)
voltage_now, current_now, power_now	Live V, A, W
energy_now / energy_full / energy_full_design	Wh now, Wh when full today, Wh when new
charge_now / charge_full / charge_full_design	the same in mAh, for drivers that report charge

The serial number is never shown. A last line, Checked, says where SRS looked.

Healthy / typical values. The author's desktop has no system battery. It shows one Nintendo Switch controller battery: type Battery, scope Device, status Discharging, capacity_level Full.

Unusual values and what they suggest. On a laptop, `energy_full` well below `energy_full_design` means a worn battery (for example 70% of design = 30% wear). Health other than Good is worth acting on.

Where Linux gets it. `/sys/class/power_supply/*` (entries of type Battery or UPS).

Cooling status (fans)

`cooling` - Linux data

In plain words. One line per fan header: how fast the fan spins, how hard the board drives it, who is in control, which temperature it follows and the fan curve the chip uses.

What you see.

Column	Meaning	Why it matters
Fan	Fan name (driver label if any, else `fanN`, or `pwmN` for an output with no speed sensor)	Matches the header number in the chip, not always the board's printed name
Current	Speed now, in RPM (revolutions per minute)	The main health sign
Minimum / Maximum	Lowest / highest speed SRS has seen since start or since **Zero**	Shows whether the fan ramps up
Target	Target speed, if the chip uses one	Shown for non-Super-I/O chips, and for Super I/O fans only in Fan Speed Cruise mode
Duty	How hard the header drives the fan: % and the raw value 0-255 in brackets	100% (255) is full power
Mode	`PWM` (4-pin signal control) or `DC` (3-pin voltage control)	A 4-pin fan on a DC header still works, but control is coarser
Control	The chip's control mode number and name (see below), or "firmware / driver" when the driver gives no mode	Who decides the fan speed

Column	Meaning	Why it matters
Temperature source	The temperature input the fan follows: number and label	If this is a wrong or floating input, the fan reacts to nothing
Curve	Fan curve points as `temperature:duty`	What the board will do as things heat up

Control mode names for Nuvoton (NCT) and Winbond (W83) chips: 0 off (full speed), 1 manual (software), 2 Thermal Cruise, 3 Fan Speed Cruise, 4 Smart Fan III, 5 Smart Fan IV. Other chips: 0 full speed / off, 1 manual (software), 2 and up automatic (hardware). The curve is only shown for automatic modes.

Fans reading 0 RPM are greyed. With **Hide** on, they are left out.

Under the table, **Thermal cooling devices** groups the kernel's cooling devices by type and state, for example "16 devices Processor state 0/3" (16 CPU threads, throttle step 0 of 3, so no throttling).

Healthy / typical values. On the author's NCT6799, all seven headers run in mode 5 Smart Fan IV and follow input 8 "PECI/TSI Agent 0 Calibration" (the CPU temperature as the chip sees it). fan1 runs at 874 RPM at 45% duty with the curve `20°C:35% 30°C:35% 40°C:35% 45°C:50% 125°C:100%`. fan7 runs at 3325 RPM at 100% - a pump-style reading. The Radeon's own fan runs at 553 RPM, 15% duty, with a Target of 553 RPM, controlled by its firmware.

Unusual values and what they suggest.

- 0 RPM with duty above 0%: nothing is plugged into that header, the fan has a zero-RPM mode, or the fan is stuck. On the author's PC, fan4 (68%) and fan6 (100%) read 0 RPM because nothing is plugged in.
- A fan stuck at 100% with a flat curve (`0°C:100% 100°C:100% ...`): the BIOS is set to full speed for that header. Often chosen for pumps.
- Speed never changes under load: the temperature source may be an unconnected input, or the curve is flat.
- Very low RPM at high duty on a fan that used to be faster: dust, a worn bearing, or a fan that needs more starting voltage.
- Mode `DC` on a 4-pin fan: switch the header to PWM in the BIOS for finer control.

Where Linux gets it. `/sys/class/hwmon/hwmon*/fanN_input`, `pwmN`, `pwmN_mode`, `pwmN_enable`, `pwmN_temp_sel`, `pwmN_auto_pointN_temp/pwm`, `fanN_target`; `/sys/class/thermal/cooling_device*`.

Background. PWM (Pulse Width Modulation) switches the fan's power on and off very fast. The "duty" is the share of time it is on. A fan curve is a list of points: at this temperature, use this duty. The chip draws straight lines between the points.

Sensor formula setup

`formula-setup` - Linux data

In plain words. Shows any rename, formula or "ignore" rule from your lm-sensors config that applies to the sensor chips in your PC. SIV keeps its own formula table; on Linux the same job is done by `/etc/sensors3.conf` and `/etc/sensors.d/*`.

What you see. A table with one row for each matching rule:

Column	Meaning
SIV Device Name	The chip name (for example `nct6799`)
User Name	The new label from a `label` line
Formula	A `compute` formula (read, write), or `set = value (applied only by sensors -s)`, or `ignore`
Units	Units of that sensor (°C, RPM, V, W, J, A, %, Hz)
SIV Device ID	`hwmonN/feature`, for example `hwmon4/in4`
Source	File and line number, for example `sensors3.conf:233`

If no rule matches, SRS says so and lists every chip with its full lm-sensors name, so you know what name to write in a `chip` line.

Healthy / typical values. On the author's PC: "No label / compute / set / ignore statements match the chips present", followed by the 13 chips (nct6799-isa-0290, k10temp-pci-00c3, ...). That is normal: the stock `sensors3.conf` has no section for the NCT6799.

Unusual values and what they suggest. A rule you wrote does not show: the `chip` pattern does not match the name shown here. Copy the name from this list.

Where Linux gets it. `/etc/sensors3.conf` (or `/etc/sensors.conf`) and `/etc/sensors.d/*`, matched to `/sys/class/hwmon` chips. SRS only reads these files.

Background. In lm-sensors config, `label in4 "+12V"` renames a reading, `compute in4 @*12, @/12` scales it (the `@` is the raw value; the first formula converts the reading, the second converts back for writing limits), `ignore` hides it, and `set` writes a limit when you run `sensors -s`.

Hardware status

`hw-status` - Linux data

In plain words. A one-screen summary of every sensor chip: how many readings of each type it has, how many are not connected, and how many are in alarm. Plus the case-open switch and the list of loaded monitoring drivers.

What you see.

Column	Meaning
Chip	Driver name of the chip (`nct6799`, `k10temp`, `amdgpu` ...)
Sensors name	Full lm-sensors name, for example `nct6799-isa-0290`

Column	Meaning
Device	The kernel device it hangs off (PCI address, platform device, NVMe name ...)
Temp / Fans / Volts / Power / PWM	How many readings of each type
Other	Energy, current, humidity and clock readings together
NC	Readings judged "not connected" (see Sensor status)
Alarms	Readings with an alarm flag set (red if any, green if none)

Below the table:

- `<chip> intrusionN` - the chassis intrusion switch: "Closed", or "Intrusion latched (set since last clear, not live; ...)".
- `<chip> beep` - whether the chip's alarm beeper is Enabled or Disabled.
- Monitoring modules - which known sensor drivers are loaded.

Healthy / typical values. The author's PC has 13 chips. The NCT6799 has 13 temperatures, 7 fans, 18 voltages and 7 PWM outputs, with 9 not connected and 0 alarms. k10temp has 2 temperatures, zenergy 9 energy readings, amdgpu 3 temperatures, 1 fan, 1 voltage, 1 power, 1 PWM and 2 clocks. Each NVMe drive has 3 temperatures. Modules: `nct6775 nct6775_core k10temp zenergy drivetemp spd5118 amdgpu ryzen_smu`.

Unusual values and what they suggest.

- Alarms above 0: open Sensor status to see which reading.
- "Intrusion latched": the case-open flag was set at some point and nobody cleared it. It is not live. It is often set after a CMOS reset or on boards with no intrusion header wired. On the author's PC intrusion0 is latched and intrusion1 is Closed.
- A chip you expect is missing: its driver is not loaded (for example no `nct6775`, so no board fans).

Where Linux gets it. `/sys/class/hwmon/hwmon*` (all attributes), `intrusionN_alarm`, `beep_enable`, `/proc/modules`.

Background. "hwmon" is the Linux hardware-monitoring framework. Every sensor driver puts its readings in a folder `/sys/class/hwmon/hwmonN` using the same file names, so any tool can read any chip.

IPMI dump

`ipmi-dump` - Not present on most PCs

In plain words. IPMI (Intelligent Platform Management Interface) is a server feature. A small separate computer on the board, the BMC (Baseboard Management Controller), watches sensors and lets admins manage the machine remotely. In SIV this item dumps the BMC's records. SRS only checks if a BMC is there.

What you see. (All four IPMI items print the same check.)

- Status - "No IPMI BMC found on this system", or "IPMI device present (not opened)".
- /dev/ipmi0 - present / absent (the device file used to talk to the BMC).
- /sys/class/ipmi - present / absent.
- ipmi_si / ipmi_devintf - the two kernel modules, loaded / not loaded.
- ipmitool - installed / not installed.

Healthy / typical values. Desktop boards, like the author's, have no BMC: everything reads absent.

Unusual values and what they suggest. On a server where you expect a BMC: load `ipmi_si` and `ipmi_devintf`, then use `ipmitool sdr` (sensor records) or `ipmitool sel list` (event log). SRS does not open the device itself.

Where Linux gets it. `/dev/ipmi0`, `/dev/ipmi/0`, `/sys/class/ipmi`, `/sys/module/ipmi_*`.

IPMI status

`ipmi-info` - Not present on most PCs

In plain words. In SIV: the BMC's identity and health. In SRS: the same "is there a BMC" check as IPMI dump. "Not present" means the board has no server management controller, which is normal for desktops.

IPMI time

`ipmi-time` - Not present on most PCs

In plain words. In SIV: the BMC's own clock, used to time-stamp its event log. In SRS: the same presence check as IPMI dump. On a server, `ipmitool sel time get` shows that clock.

IPMI wait

`ipmi-wait` - Not present on most PCs

In plain words. In SIV: timing of requests to the BMC. In SRS: the same presence check as IPMI dump. Nothing to wait for when there is no BMC.

---

kWh cost

`kwh-cost` - Linux data

In plain words. Turns the power your parts are drawing right now into money: per hour, per day and per year, as if the PC ran like this all the time.

What you see.

- Rate - price per kWh (kilowatt-hour). Default $0.1500. Set your own with the environment variables `SRS_KWH_COST` (for example `0.32`) and `SRS_CURRENCY` (for example `€`).
- A table: Component, Power (W), per Hour, per Day, per Year.
  - CPU line: the RAPL package power (`CPU package-0 (RAPL)`), or if there is none, the zenergy `Esocket` reading.
  - One line for each other hwmon power sensor (the GPU shows as `GPU PPT`).
- Total Power (measured) - the sum, with its cost.
- A note that only parts with power sensors are counted.

Healthy / typical values. On the author's PC at light load: CPU 49.60 W and GPU 36.00 W, total 85.60 W, which is $0.31 per day or $112.48 per year at $0.15/kWh.

Unusual values and what they suggest. The total is always lower than the wall socket reading. The board, drives, fans, RAM and the power supply's own losses are not measured. A wall power meter shows the real figure; expect it to be clearly higher than this total.

Where Linux gets it. `/sys/class/powercap/*/energy_uj` (RAPL), hwmon `energy*_input` (zenergy) and `power*_input` / `power*_average` (for example amdgpu). Power is worked out from two energy readings about 0.2 s apart.

Background. 1 kWh is 1000 W used for one hour. A PC drawing 100 W all day uses 2.4 kWh per day.

LCD panel setup

`lcd-panel` - Not present on most PCs

In plain words. SIV can show its readings on small external screens, such as LCDSysInfo USB panels or the screen on some AIO coolers. SRS does not drive such screens.

What you see. "No supported external LCD panel (LCDSysInfo / AIO LCD) found" and a note that case and AIO displays are driven over USB HID by their own tools and are not probed.

Healthy / typical values. This message is the normal answer on every Linux PC.

Super I/O LDNs

`ldn-info` - Linux data - Needs root data

In plain words. The Super I/O is a small chip on the motherboard that reads fans, voltages and board temperatures. It is split into "logical devices". This item shows which chip you have, where its hardware-monitor part lives, and what the kernel said when it found it.

What you see.

- Chip - the Super I/O driver name, for example `nct6799`.
- Platform device - the kernel device, for example `nct6775.656`. The number is the base address in decimal (656 = 0x290).
- HWM base (LDN 0x0B) - the I/O address of the hardware-monitor logical device, for example `0x0290`.
- hwmon - the sysfs folder, for example `/sys/class/hwmon/hwmon4`.
- Driver - `nct6775 + nct6775_core (loaded)` when the Nuvoton driver is in use.
- I/O ports - lines from the I/O port map that mention the chip or its address. Without root data the addresses are hidden (shown as zeros), so SRS uses the srs-root copy.
- Kernel log - the driver's start-up message (from the srs-root copy of the kernel log).
- A note that the LDN configuration registers are not read: SRS never does raw port I/O.

Healthy / typical values. On the author's PC: chip nct6799 at HWM base 0x0290; I/O ports `0290-029f : PNP0C02:03`; kernel log `nct6775: Found NCT6796D-S/NCT6799D-R or compatible chip at 0x2e:0x290`.

Unusual values and what they suggest.

- "No Super-I/O hardware monitor driver bound": no fan or board readings at all. Load `nct6775` (Nuvoton) or `it87` (ITE). On some boards the firmware reserves the address range and the driver refuses to load; the common workaround is the kernel option `acpi_enforce_resources=lax`, used at your own risk.
- "Needs root: Menu > Run as root (srs-root)" under I/O ports or Kernel log: run srs-root once.

Where Linux gets it. `/sys/class/hwmon/*/device` (platform device), `/proc/ioports` or the srs-root `ioports.txt`, the srs-root `dmesg.txt`, `/sys/module/nct6775`.

Background. The chip is reached through two "config" ports, 0x2E/0x2F (or 0x4E/0x4F). Each logical device (serial port, keyboard, hardware monitor ...) has a number, the LDN (Logical Device Number). On Nuvoton chips the hardware monitor is LDN 0x0B. The kernel log's `0x2e:0x290` means "config port 0x2E, hardware monitor at 0x290". A `PNP0C02` entry is an ACPI "motherboard resources" device that reserves the range.

AIO Link devices

`link-devices` - Not present on most PCs

In plain words. In SIV: the list of devices on an AIO / fan hub chain. In SRS: the same cooler driver check as AIO Link export. If a supported hub is found it is listed here as `hwmonN - driver name`.

AIO Link dump

`link-dump` - Not present on most PCs

In plain words. In SIV: a raw data dump from the hub. SRS never reads raw USB data from coolers, so it shows the same driver check as AIO Link export.

AIO Link fans

`link-fans` - Not present on most PCs

In plain words. In SIV: fan speeds reported by the hub. In SRS: the same check as AIO Link export. Fans of a hub with a Linux driver appear in Cooling status.

AIO Link LEDs

`link-leds` - Not present on most PCs

In plain words. In SIV: the hub's RGB lighting channels. In SRS: the same check as AIO Link export. For lighting on Linux, tools like OpenRGB or liquidctl are used.

AIO Link limits

`link-limits` - Not present on most PCs

In plain words. In SIV: warning limits stored in the hub. In SRS: the same check as AIO Link export. Limits of a driver-supported hub show in the Limits column of Sensor status.

AIO Link power

`link-power` - Not present on most PCs

In plain words. In SIV: power drawn through the hub (and, for Corsair PSUs, PSU rails). In SRS: the same check as AIO Link export. A `corsair-psu` driver's readings would show in Power status.

AIO Link setup

`link-setup` - Not present on most PCs

In plain words. In SIV: settings for the hub link. SRS has nothing to set up and shows the same check as AIO Link export.

AIO Link status

`link-status` - Not present on most PCs

In plain words. In SIV: live status of each linked device. In SRS: the same check as AIO Link export.

AIO Link test

`link-test` - Not present on most PCs

In plain words. In SIV: a communication test with the hub. SRS never talks to the hub directly, so it shows the same check as AIO Link export.

AIO Link update

`link-update` - Not present on most PCs

In plain words. In SIV: hub firmware details. SRS does not touch cooler firmware and shows the same check as AIO Link export. Use the maker's tool to update hub firmware.

Power status

`power` - Linux data

In plain words. Every power and energy reading in the PC: the CPU's own energy counters, power readings from drivers such as the GPU, and any batteries or other power supplies.

What you see. Three blocks:

1. **RAPL energy (powercap)** - one row per energy zone: zone name and folder, power now (W), total energy counted so far (J, joules), and any power limits the zone reports (`name N W`).
2. **hwmon power and energy** - one row per hwmon power, energy or current reading: chip and label, value, then either total energy (J) or the sysfs file name. GPU power rows also show `cap` (current power limit), `max` (highest limit allowed) and `default` (factory limit).
3. **Power supplies** - every entry in `/sys/class/power_supply`, with the same fields as Battery (mains adapters too, not only batteries).

Healthy / typical values. On the author's 7800X3D near idle: package-0 52.77 W, core 1.91 W. zenergy shows each core at about 2-4 W and Esocket0 at 52.22 W. The Radeon shows `amdgpu PPT 35.00 W cap 374 W max 374 W default 304 W` - the power limit has been raised from the 304 W default to the 374 W maximum. The only power supply is a game controller battery.

Unusual values and what they suggest.

- Package power far above core power at idle is normal on Ryzen desktop chips: the I/O die, memory controller and cache use a lot of the package power.
- `cap` lower than `default` on a GPU: something (a tuning tool or a profile) has lowered the power limit, which can cut performance.
- "No powercap energy zones": the `intel_rapl` powercap driver is not loaded (it is also used for AMD).
- A watt value of `-`: the counter could not be read twice in time; open the item again.

Where Linux gets it. `/sys/class/powercap/*/energy_uj` and `constraint_N_*`; hwmon `power*_average`, `power*_input`, `power*_cap*`, `energy*_input`, `curr*_input`; `/sys/class/power_supply`.

Background. RAPL (Running Average Power Limit) is a CPU feature that counts the energy used. The counter only goes up; power is the change in energy divided by the time between two readings. Linux names the zones `intel-rapl` even on AMD CPUs. zenergy is an extra (out-of-tree) driver that reads the same AMD counters per core. PPT (Package Power Tracking) is AMD's name for the power the GPU or CPU is allowed to draw.

PWM export

`pwm-export` - Linux data

In plain words. A CSV (comma-separated values) snapshot of every fan control output, ready to paste into a spreadsheet.

What you see. A header line, then one line per PWM output:

Column	Meaning
hwmon	The hwmon folder (`hwmon4`)
chip	Full chip name (`nct6799-isa-0290`)
pwm	Output name (`pwm1`)
raw	Raw drive value 0-255
duty_pct	Duty in %
enable	Control mode number (see Cooling status)
mode	1 = PWM, 0 = DC
temp_sel	Number of the temperature input the fan follows
fan_rpm	Speed of the fan with the same number

`-1` means the driver does not provide that value. A note says values are read only.

Healthy / typical values. From the author's PC: `hwmon4,nct6799-isa-0290,pwm1,133,52.2,5,1,8,955` (52% duty, Smart Fan IV, PWM mode, following input 8, 955 RPM). The GPU line is `hwmon9,amdgpu-pci-0300,pwm1,40,15.7,-1,-1,-1,554`: the amdgpu driver gives no mode or source here.

Where Linux gets it. hwmon `pwmN`, `pwmN_enable`, `pwmN_mode`, `pwmN_temp_sel`, `fanN_input`.

QST status

`qst-status` - Not present on most PCs

In plain words. QST (Intel Quiet System Technology) was fan control built into the Intel management engine on some Intel desktop boards around 2007-2010. SIV can read it. SRS reports "Not present on this system" (Checked: n/a (Intel QST)). Modern boards, and all AMD boards, use the Super I/O or the embedded controller instead.

Sensor export

`sen-export` - Linux data

In plain words. A CSV snapshot of every sensor reading, for spreadsheets or logs.

What you see. Header `hwmon,chip,feature,label,value,unit,minimum,maximum,status`, then one line per reading:

- hwmon, chip - where it comes from.
- feature - the hwmon name (`temp3`, `fan1`, `in0`, `energy9` ...).
- label - the name in quotes (driver label, lm-sensors label, a standard pin name, or the feature name).
- value - with 3 decimals, after any lm-sensors formula. Energy readings are exported as watts.
- unit - C, RPM, V, W, A, %, MHz.
- minimum, maximum - lowest and highest values seen since start or since **Zero**.
- status - `ok`, `alarm`, or `not-connected` (a floating or unwired input; these lines are greyed).

Healthy / typical values. From the author's PC: `hwmon5,k10temp-pci-00c3,temp1,"Tctl",56.375,C,56.375,57.625,ok` and `hwmon4,nct6799-isa-0290,temp3,"AUXTIN0",86.000,C,86.000,86.000,not-connected`.

Unusual values and what they suggest. Filter out `not-connected` lines before you graph or average anything. They are not real temperatures (see Sensor status).

Where Linux gets it. All of `/sys/class/hwmon/hwmon*`.

Sensor status

`sensors` - Linux data

In plain words. The main sensor page. Every temperature, fan, voltage, power, energy, clock and PWM reading from every chip, with the lowest and highest values seen, the chip's limits and a plain status. It refreshes about once a second.

What you see.

A top line: "Minimum / Maximum since (Zero resets) Hide unused: on/off".

Then one block per chip. The chip line shows: driver name, full lm-sensors name, hwmon folder and the kernel device path. Chips are sorted: Super I/O first, then CPU, GPU, memory, drives, everything else. Inside each chip, readings are grouped under Temperatures, Fans, Voltages, Power, Energy, Currents, Humidity, Clocks and PWM duty.

Column	Meaning	Why it matters
Sensor	The label (from the driver, your lm-sensors config, a standard pin name, or just `in4`)	Tells you what it measures
Current	Value now (°C, RPM, V, W, A, %, MHz)	
Minimum / Maximum	Lowest / highest seen since SRS started or since **Zero**	Catches spikes you were not watching
Limits	The chip's own limits: `min`, `max`, `crit` (critical); `cap` for power; `total N J` for energy	What the chip or driver treats as too high
Status	See the list below	The quick answer
Source	The sysfs file read, plus `(compute)` if an lm-sensors formula was applied, or `(pin)` for a standard pin name	Lets you check the raw value yourself

Status values, checked in this order:

- `not connected (...)` - the input is not wired to anything. Greyed. The reasons are explained below.
- `ignored (sensors.conf)` - your lm-sensors config says to ignore it. Greyed.
- `ALARM` (red) - the chip has set an alarm flag for this reading.
- `CRITICAL` (red) - at or above the `crit` limit.
- `HIGH` (orange) - above `max` (not used for power).
- `LOW` (orange) - below `min` (not used for temperatures).
- `unused` (grey) - reads exactly 0, for example an empty fan header.
- `OK` (green).

A voltage or fan alarm is ignored when the chip has no limits set (both 0). Those alarm flags mean nothing.

At the bottom: a count line ("13 chips, 79 sensors, 79 shown, 0 hidden, 9 not connected, 0 in alarm"), plus notes explaining greyed rows and `(pin)` names.

Zero clears the Minimum / Maximum columns and restarts the "since" time. **Hide** removes rows that are not connected, ignored, or reading 0.

Healthy / typical values. From the author's PC, near idle:

- CPU: k10temp Tctl 56.4°C, Tccd1 52.4°C (it has peaked at 57.6°C and 56.5°C). The Super I/O's TSI0_TEMP (56.8°C) is the same CPU temperature, read over the CPU's SB-TSI interface.

- Board: SYSTIN 22.0°C and CPUTIN 30.0°C, both with `max 80 crit 125`.
- Vcore 1.120 V (range 1.096-1.144 V), `+3.3V` 3.312 V, 3VSB 3.408 V, Vbat 3.344 V.
- GPU: edge 25.0°C, junction 29.0°C, memory 50.0°C, PPT 37.00 W (`cap 374`), sclk 311 MHz.
- DDR5 modules (spd5118): 31.0°C and 30.5°C, `max 55 crit 85`.
- NVMe Composite: 34.9, 37.9 and 39.9°C. SATA disk (drivetemp): 38.0°C. USB controller in the chipset (prom21_xhci): 51.9°C. Network chip PHY (r8169): 55.0°C, `max 120`.

Unusual values and what they suggest.

- **AUXTIN reading 66-86°C and never moving.** This confuses many people. See "Not connected inputs" below. On the author's board AUXTIN0 reads 86°C, AUXTIN3 84°C and AUXTIN4 70°C. They are not real and nothing is overheating.
- `PCH_...` temperatures at 0.0°C: AMD boards have no Intel-style PCH (Platform Controller Hub) feeding this chip, so these read 0. SRS marks them "not connected (PCH reading 0)".
- CPU temperature (Tctl) above 90°C under normal load: check the cooler mounting, paste and fan curve. Ryzen 7000 chips are designed to run up to 89-95°C under full load, but not at light load.
- GPU junction more than about 20-25°C above edge under load: possible poor paste or pad contact.
- NVMe Composite near its `max`: the drive will slow itself down (thermal throttle). Add a heatsink or airflow.
- Vcore that never drops at idle: power saving may be off in the BIOS.
- Vbat below about 3.0 V: the CMOS coin cell is getting weak. While the PC is on, some boards feed this input from standby power, so a good reading does not prove the coin cell is fine.
- A plain `inN` voltage with an odd value (0.344 V, 1.472 V): see "Voltages" below.

Not connected inputs (floating AUXTIN). A Super I/O chip has many temperature inputs. Each one is built for a thermistor: a small part whose resistance changes with heat. The chip measures the voltage at the pin and turns it into °C using a fixed table. The board maker decides which inputs get a real thermistor. The rest are left unwired. An unwired pin is "floating": its voltage drifts to some level set by leakage and noise, not by heat. The chip still turns that voltage into a temperature, so you get a number that looks real but is not. Signs of a floating input:

- It stays the same, or nearly so: Minimum = Current = Maximum, even under heavy load.
- It does not match anything nearby (86°C while the board and CPU are at 22-30°C).
- It may be very low (1-12°C, colder than the room) instead of high.
- Its alarm flag may be set because the fake value is above the 80°C `max`. Tools such as `sensors` then show ALARM. SRS shows it greyed as not connected and does not count it as an alarm.

SRS marks these as follows:

- `probably not connected (floating high)` - an AUXTIN input reading more than 25°C above the board's own SYSTIN / CPUTIN readings (a real probe on a board header reads near those). On the author's ASUS TUF GAMING X870-PRO these are AUXTIN0, AUXTIN3 and AUXTIN4 of the NCT6799 (66-86°C while SYSTIN / CPUTIN read 22-29°C).
- `not connected (floating low)` - any AUXTIN input reading below 15°C (AUXTIN1, AUXTIN2, AUXTIN5 here).
- `not connected (implausible)` - any Super I/O temperature at or below 0°C or at or above 120°C.
- `not connected (PCH reading 0)` - PCH inputs reading exactly 0.

If you plug a probe into the board's optional thermistor header (often labelled T_SENSOR), one AUXTIN input will start to follow real temperatures and is no longer greyed. To make sure SRS treats an input as connected, give it a label in `/etc/sensors.d/` (inputs labelled there are never marked floating).

Voltages and `(pin)`. The Nuvoton driver gives voltage inputs no names. SRS names the five that are the same on every NCT67xx chip: in0 Vcore, in2 AVCC, in3 +3.3V, in7 3VSB, in8 Vbat, marked `(pin)`. The other `inN` inputs are wired differently on each board. The chip can only measure small voltages (about 2 V at most), so rails such as +12 V and +5 V reach it through a resistor divider. SRS shows the raw pin voltage and does not apply the divider. An lm-sensors `compute` line can scale it, and the result shows as `(compute)`.

Where Linux gets it. Every `/sys/class/hwmon/hwmon*` folder: `*_input`, `*_label`, `*_min`, `*_max`, `*_crit`, `*_alarm`, `power*_average`, `power*_cap`, `energy*_input` (sampled to watts), `pwmN`; labels, formulas and ignores from `/etc/sensors3.conf` and `/etc/sensors.d/*`.

Background.

- Tctl is the AMD "control" temperature that the CPU uses for fan and boost decisions. Tccd1 is the temperature of the first CCD (core complex die, the chip with the cores).
- "PECI/TSI Agent 0 Calibration" is the CPU temperature as the Super I/O sees it, after the board's own adjustment. Board fan curves usually follow it. SB-TSI is AMD's sideband temperature interface.
- GPU edge is the die's edge sensor, junction is the hottest spot on the die, and mem is the video memory.
- NVMe Composite is the drive's own summary temperature. Sensor 1 and 2 are extra sensors (often the controller and the flash).

SIO debug

`sio-dbg` - Linux data - Needs root data

In plain words. A raw list of every value the Super I/O driver exposes, in the driver's own units. It is the "under the bonnet" view behind Sensor status and Cooling status.

What you see.

- Chip and Source - the chip name and its hwmon folder.
- A note: these are register-derived driver values, read only. Raw register dumps would need `/dev/port` and are not done.
- Then one `name value` line for every file in the folder, sorted by name. Raw units: milli-degrees (`46000` = 46°C), millivolts (`1120` = 1.120 V), RPM, PWM 0-255, times in milliseconds.

The main families:

Attribute	Meaning
`tempN_input`, `_label`	Temperature and its name
`tempN_max`, `_max_hyst`, `_crit`	Warning limit, the value it must fall back below to clear, critical limit

Attribute	Meaning
`tempN_offset`	Correction added by the chip
`tempN_type`	Sensor type; `4` = thermistor
`tempN_alarm`, `_beep`	Alarm flag; whether the alarm beeps
`inN_input`, `_min`, `_max`, `_alarm`, `_beep`	Voltage inputs and their limits
`fanN_input`, `_min`, `_pulses`, `_target`, `_tolerance`, `_alarm`, `_beep`	Fan speed, low limit, tacho pulses per turn, target and allowed error for speed modes
`pwmN`, `_enable`, `_mode`, `_temp_sel`	Drive value, control mode, PWM/DC, temperature source
`pwmN_auto_pointN_temp`, `_pwm`	The fan curve points
`pwmN_floor`, `_start`, `_stop_time`	Lowest duty allowed, duty used to start a stopped fan, delay before stopping
`pwmN_step_up_time`, `_step_down_time`	How fast the duty may change
`pwmN_target_temp`, `_temp_tolerance`, `_crit_temp_tolerance`	Settings for Thermal Cruise mode
`pwmN_weight_*`	A second temperature that can add to the fan duty
`intrusionN_alarm`, `_beep`	Case-open switch
`beep_enable`, `name`	Master beeper switch; chip name

Healthy / typical values. On the author's NCT6799: `temp1_label SYSTIN`, `temp1_input 22000`, `temp1_max 80000`, `temp1_type 4`; `pwm1 131`, `pwm1_enable 5`, `pwm1_temp_sel 8`; `pwm1_auto_point4_temp 45000` with `pwm1_auto_point4_pwm 127`; `fan1_pulses 2`; `in0_input 1120`, `in0_max 1744`.

Unusual values and what they suggest.

- `in10_alarm 1` to `in16_alarm 1` with `_min 0` and `_max 0`: the BIOS set no limits for these inputs, so the chip flags them. These alarms mean nothing, and Sensor status ignores them.
- `temp3_alarm 1` and `temp6_alarm 1`: the floating AUXTIN0 and AUXTIN3 inputs read above their 80°C `max`. Again not real.
- `fanN_target` values look odd unless that fan is in mode 3 (Fan Speed Cruise). The chip only uses them in that mode.
- `fanN_pulses` wrong for your fan (most fans give 2 pulses per turn): the RPM reading will be double or half.

Where Linux gets it. Every readable file in the Super I/O's `/sys/class/hwmon/hwmonN` folder.

Synthetic sensors setup

`synthetic` - Linux data

In plain words. SIV lets you build your own "computed" sensors. On Linux the matching feature is the lm-sensors `compute` line. This item lists every `compute` line in your config and, if the chip is present, shows the raw and computed value side by side.

What you see.

- Config files - which files were read (`/etc/sensors3.conf`, `/etc/sensors.d/*`), or "none".
- A table:

Column	Meaning
Source	File and line, for example `sensors3.conf:233`
Chip match	The chip pattern from the `chip` line (`...` if there are more), or the real chip name if it matches one in your PC
Feature	The reading it applies to (`in6`)
Formula (read, write)	The two formulas; `@` is the raw value
Raw	The raw reading (only for present chips)
Computed	The result, or `n/a` if the formula could not be worked out

- A summary: number of compute statements and the list of chips present.
- A note: greyed rows are for chips not in this PC. Add your own in `/etc/sensors.d/<name>.conf`.

Healthy / typical values. On the author's PC: 19 compute statements from the stock `/etc/sensors3.conf`, all for other chips (pc87365, w83627thf, nct6106, f71808e ...), so all rows are greyed and Raw / Computed are empty.

Unusual values and what they suggest. Your own line shows greyed: the `chip` pattern does not match. Use the full names from the summary line or from Sensor formula setup.

Where Linux gets it. `/etc/sensors3.conf` (or `/etc/sensors.conf`) and `/etc/sensors.d/*`, applied to `/sys/class/hwmon` readings.

Background. Example for a +12 V rail measured through a divider that divides by 12: `chip "nct6799-*"` then `compute in4 @*12, @/12` and `label in4 "+12V"`. Find the right factor in your board's documentation or by comparing with the BIOS hardware monitor.

Thermal zones

`thermal-zones` - Not present on most PCs

In plain words. Thermal zones are temperature areas defined by the firmware (ACPI), mostly on laptops. Each zone has trip points: temperatures where the system slows down, starts a fan, or shuts down.

What you see.

- One row per zone: zone name (`thermal_zone0`), type, temperature, and the governor policy that acts on it.
- Under each zone, its trip points: type (`active`, `passive`, `hot`, `critical`) and temperature.

- If there are none: "No ACPI / thermal_zone* zones exposed by this firmware".
- Cooling devices - how many cooling devices exist (details in Cooling status).
- Checked - `/sys/class/thermal`.

Healthy / typical values. Many desktop boards have no zones, like the author's PC ("No ACPI / thermal_zone* zones", 16 cooling devices). That is normal; the CPU protects itself anyway.

Unusual values and what they suggest. On a laptop, a `critical` trip only a few degrees above the current temperature means the machine is about to shut down to protect itself.

Where Linux gets it. `/sys/class/thermal/thermal_zone*` (type, temp, policy, trip_point_N_type/temp).

UPS export

`ups-export` - Not present on most PCs

In plain words. A UPS (Uninterruptible Power Supply) is a battery box that keeps the PC running in a power cut. SIV can export UPS data. SRS shows which UPS tools are installed and any UPS or battery that Linux reports.

What you see.

- apcupsd (apcaccess) - installed (not queried) / not installed.
- Network UPS Tools (upsc) - installed (not queried) / not installed.
- Battery and UPS entries from `/sys/class/power_supply`, with the same fields as Battery.
- "No UPS data to export".

Healthy / typical values. On the author's PC: apcupsd is installed but not queried, NUT is not installed, and the only entry is a game controller battery.

Unusual values and what they suggest. You have a USB UPS but no entry shows: many UPS units are handled only by apcupsd or NUT and do not appear in `/sys/class/power_supply`. Ask those tools directly (`apcaccess status` or `upsc <name>`).

Where Linux gets it. `/usr/bin/apcaccess`, `/usr/bin/upsc` (presence only), `/sys/class/power_supply`.

Volumes / Drives (SRS Section Reference)

Back to the section index - Glossary

Where to find these items (SRS 2.0). In the main window they are on the **Storage** page (Alt+6). Its tabs are Disk drives, Volumes, SMART drives, Partitions, Disk performance and USB disk drives; **All N items** lists the rest. The Overview checks drive health and drive temperature. You can also type an item's name or id into the search box (Ctrl+F). The rest of this introduction describes the classic view (Menu -> Classic View), with its Menu and panel buttons.

This group covers everything about storage: the physical drives in the PC, how they are split into partitions, which filesystems are on them, where those filesystems are mounted, how busy the drives are, and how healthy they are. Open any item with **Menu -> Volumes / Drives -> item**. The panel button **Volumes** opens the volume list straight away; next to it, **Drive /** shows the full detail of the root filesystem (`volumes:/`) and **FS** shows the filesystem statistics of the root filesystem (`ntfs:/`). Most items read only files the kernel already makes public, so they work as a normal user. The drive health items (SMART drives, SMART all drives, SMART drive monitor) and the NVMe controller items need root data: use **Menu -> Run as root** (`srs-root`) once, and SRS shows the cached results from then on. SMART on USB-attached disks is skipped unless you set `SRS_SMART_USB=1` before running `srs-root`.

A note on units: SRS shows sizes in powers of 1024 but writes them as "GB" and "TB". So a "1 TB" drive (10^12 bytes on the box) shows as 931.51GB. The Drive geometry item shows both styles side by side.

SRS (srs-sysinfo) is a Linux system information viewer inspired by SIV64X (System Information Viewer). SIV is the work of Ray Hinchliffe, RH Software. SRS is not affiliated with or endorsed by RH Software and contains no SIV code or data. Item names follow SIV, which is why some titles name Windows things (NTFS, stornvme, Storage Spaces); SRS shows the nearest Linux equivalent.

Contents

- Disk drives `drives`
- Drive tree `drive-tree`
- Disk API routines `dsk-api`
- FAT statistics `fat`
- Drive geometry `geometry`
- Hidden disks `hidden`
- JMicron JMB RAID `jmb-raid`
- JMicron NVMe RAID `jms-raid`
- Disk mapping `mapping`
- LSI MegaRAID `mega-raid`
- NTFS statistics `ntfs`
- Partitions `partitions`
- Disk performance `performance`
- Mount points `points`
- CSMI RAID `raid`

- RocketRAID `rocket-raid`
- SCSI devices `scsi`
- Samsung NVMe (secnvme) `secnvme`
- Disk sector sizes `sectors`
- SMART drives `smart`
- SMART all drives `smart-all`
- SMART drive monitor `smart-sdm`
- Storage spaces `spaces`
- NVMe (stornvme) `stornvme`
- USB disk drives `usb-disks`
- Volume list `vol-list`
- Volumes `volumes`
- Intel VROC `vroc`

Disk drives

`drives` - Linux data

In plain words. One line per physical drive: what it is, how it is connected, and a short list of its main traits. Optical drives (CD/DVD/Blu-ray) are listed last, after a blank line.

What you see.

Column	Meaning	Why it matters
Drive	Kernel name: `nvme0n1`, `sda`, `sr0`...	The name every other item and Linux tool uses.
Bus	NVMe, SATA, USB, SCSI, MMC (SD/eMMC), VirtIO (virtual machine)	Tells you the connection, and so the speed you can expect.
Location	udev `ID_PATH`: the PCI address and port the drive hangs off, e.g. `pci-0000:0c:00.0-ata-1.0`	Lets you tell two identical drives apart and see which controller each one uses.
Type	Fixed, Removable, CD Drive (also RAM Disk / Virtual, but those are left out here)	"Removable" is a flag the device itself reports; most USB sticks and card readers set it.
Vendor and Product (Serial Number)	Model name, with the serial in brackets	Identify the exact drive for warranty or firmware lookups.
Revision	Firmware version	Check against the maker's site for known-bug fixes.
State	`live` for NVMe controllers, `running` for SATA/USB/SCSI devices	Anything else (`offline`, `dead`, `resetting`) means the kernel has lost or is recovering the drive.

Column	Meaning	Why it matters
Characteristic	Size, media (SSD, HDD, HDD/Flash, Optical, No media), spindle speed in rpm (HDDs that report it), `TRIM` if the drive accepts discard, `WriteCache` if write-back caching is on, `RO` if read-only	A quick health-of-setup check: an SSD without TRIM, or a disk stuck read-only, stands out here.

The last line counts physical drives. Loop devices, zram and device-mapper volumes are left out (see Drive tree for those). The board name and BIOS version follow as a footer.

Healthy / typical values. On the author's PC: three NVMe SSDs (Samsung 980 PRO 1TB, ADATA XPG GAMMIX S70 Blade 2TB, PNY CS2340 500GB), all `live`, all "SSD TRIM WriteCache". One SATA Seagate 4TB disk: `running`, "3.64TB HDD 7200rpm WriteCache". USB sticks and card readers show as Removable with media "HDD/Flash", and a USB DVD writer shows as "1024.00MB Optical".

Unusual values and what they suggest.

- Size `0` with media "SSD" or "No media": the device answered but has no usable storage behind it. For a card reader slot that just means "no card inserted". For a USB enclosure (on the author's PC, a USB 3.1 NVMe enclosure listed as `0 SSD`) it usually means the drive inside is missing, dead, or not detected by the bridge chip.
- An SSD without `TRIM`: the drive or its USB bridge does not pass discard commands. The drive can slow down over time; many USB bridges cause this.
- `RO` on a disk you expect to write to: a write-protect switch, a failing SSD that went read-only to protect your data, or a kernel decision after errors. Check `dmesg`.
- A USB drive marked HDD/Flash: SRS cannot tell spinning disks from flash behind most USB bridges, because they report "rotational" by default.

Where Linux gets it. `/sys/block/<disk>/` (size, removable, queue/rotational, queue/discard_max_bytes, queue/write_cache, device/model, device/rev or firmware_rev, device/state), `/sys/class/nvme/<ctrl>/state`, and the udev database (`/run/udev/data/b<maj>:<min>`) for model, serial, path and rpm.

Background. *TRIM* (called *discard* on Linux) lets the operating system tell an SSD which blocks no longer hold data, so the SSD can erase them ahead of time. *Write-back cache* means the drive confirms a write once it is in its own fast memory, before it is on the flash or platter; it is faster, and safe as long as the system shuts down cleanly or the drive has power-loss protection.

`drives:<disk>` (for example `drives:sda`) shows just that one drive.

Drive tree

`drive-tree` - Linux data

In plain words. A tree that shows each disk, the partitions on it, what filesystem each one holds and where it is mounted. Stacked devices (encryption, LVM, RAID, zram) hang underneath the device they sit on.

What you see.

- **Disk line**: name, size, bus, type, model (serial), and the partition table type (`gpt` or `dos`).
- **Partition lines** (`|--` and `` `-- ``): name, size, filesystem type (`-` if none is known), filesystem label, then every mount point of that partition. Swap in use shows as `[SWAP]`.
- **(whole disk)**: a disk with no partition table but a filesystem or swap directly on it, such as `zram0 swap [SWAP]`.
- **`` `-> `` holder lines**: devices built on top of this one (for example a dm-crypt or LVM volume on a partition), with size, name and mount points. They nest up to six levels.

Unlike Disk drives, this tree also lists zram, loop-backed and device-mapper disks. `drive-tree:<disk>` shows one disk only.

Healthy / typical values. On the author's PC the Samsung NVMe holds `nvme0n1p1 2.05GB vfat EFI /boot/efi` and `nvme0n1p2 929.46GB btrfs LINUX / /srv /root / home /var/cache /var/log /var/tmp`. That one partition has seven mount points because it is btrfs with several subvolumes. A dual-boot disk shows an `ntfs WIN11` partition with no mount point, which is normal when Windows is not in use.

Unusual values and what they suggest.

- A partition with `-` as the filesystem: empty, a filesystem udev does not recognise, or a damaged filesystem. Cross-check with Volume list.
- A data partition you expect to see but which has no mount point: it is simply not mounted (yet).
- A disk with size `0` and no partitions: no media, or a dead drive behind a USB bridge.

Where Linux gets it. `/sys/class/block` (partitions, sizes, `holders/` links), udev `ID_FS_*` and `ID_PART_TABLE_TYPE`, `/proc/self/mountinfo` and `/proc/swaps`.

Background. A *holder* is the kernel's word for a device that uses another device. If you put an encrypted container on `sda2`, the decrypted `dm-0` is a holder of `sda2`.

Disk API routines

`dsk-api` - Windows-only (Linux equivalent shown)

In plain words. On Windows, SIV lists the entry points of a vendor disk toolkit (the JMicron SDK). Linux has no such toolkit; the kernel drivers plus a set of command-line tools do the same jobs. This item shows which of those tools and kernel interfaces exist on your PC.

What you see.

- **Disk tools**: a tick box for each of `smartctl`, `nvme`, `hdparm`, `sdparm`, `sg_inq`, `lsscsi`, `blkid`, `lsblk`, `udisksctl`, `mdadm`, `lvm`, `btrfs`, `skdump`, `fio`, ticked if it is in your PATH (or in /usr/sbin, /sbin). Then "Found N of 14 disk tools in PATH".
- **Kernel interfaces**: tick boxes for SCSI bsg (`/dev/bsg`), NVMe class (`/sys/class/nvme`), SCSI generic (sg), udev database, hwmon (drivetemp/nvme), device-mapper (`/dev/mapper/control`), md RAID (`/proc/mdstat`), btrfs sysfs (`/sys/fs/btrfs`).

Healthy / typical values. The author's PC: 13 of 14 tools found (only `sg_inq` missing), and every kernel interface present except md RAID, because no software RAID is in use.

Unusual values and what they suggest. `smartctl` missing means `srs-root` cannot collect SMART data, so the SMART items stay empty; install `smartmontools`. `nvme` missing means no `nvme smart-log` copy (install `nvme-cli`). An unticked md RAID box only means the md driver is not loaded, which is normal without software RAID.

Where Linux gets it. A PATH search for each tool, and a check that each kernel path exists. Nothing is run.

FAT statistics

`fat` - Linux data

In plain words. Facts about every mounted FAT or exFAT filesystem: which FAT type, how many clusters, how many are free, and the volume serial.

What you see. For each filesystem, a heading "Statistics for () /dev/ ", then:

Field	Meaning	Why it matters
FAT type	FAT12, FAT16 or FAT32 for vfat; the exFAT version (e.g. `1.0`) for exFAT	FAT32 cannot hold files of 4 GiB or more; exFAT can.
Clusters	Total number of clusters (allocation units)	More clusters means more small files fit without waste.
Free clusters	Clusters not in use	Free space = free clusters x cluster size.
Cluster size	Size of one cluster	Every file uses at least one whole cluster.
Serial	Volume serial number (the `XXXX-XXXX` value)	The same number Windows shows; Linux uses it as the UUID of FAT volumes.

If nothing matches, "Status: No mounted FAT/exFAT volumes" is shown. `fat:<mount point or device>` shows one volume, whatever its type.

Healthy / typical values. On the author's PC: the EFI system partition at `/boot/efi` is FAT32 with 4KB clusters and 537,058 clusters, almost all free. A 16 GB USB stick is FAT32 with 8KB clusters. A Ventoy USB stick is exFAT with 32KB clusters and only 51,149 of 953,065 clusters free (it is nearly full of ISO images).

Unusual values and what they suggest. Free clusters near zero: the volume is full. On an EFI partition that can stop kernel or bootloader updates. Very large clusters (64KB or more) on a disk of small files waste a lot of space.

Where Linux gets it. `statvfs()` on the mount point, plus udev `ID_FS_VERSION` and `ID_FS_UUID`. The kernel vfat and exfat drivers export no statistics of their own.

Background. A *cluster* is the smallest piece of space a FAT filesystem hands out. A 1-byte file still takes one whole cluster.

Drive geometry

`geometry` - Linux data

In plain words. Each physical drive's size in bytes, in decimal units (as printed on the box) and in binary units (as most software shows), plus the old cylinder/head/sector figures.

What you see.

Column	Meaning
Physical Drive Description	`/dev/<disk>` plus model and serial
Media	SSD, HDD, HDD/Flash, Optical, No media
Cylinders	Size divided by (255 x 63 x sector size): the legacy "CHS" cylinder count
Tracks	Always 255 (heads per cylinder in the legacy translation)
Sectors	Always 63 (sectors per track in the legacy translation)
Bytes	Logical sector size in bytes (usually 512)
Total Size	Exact size in bytes, with thousands separators
Size (1000)	Size in decimal GB/TB (1 GB = 1,000,000,000 bytes), as drive makers count
Size (1024)	Size in binary units (1 GB here = 1,073,741,824 bytes), as Linux tools count

A **Total** row adds up all drives.

Healthy / typical values. The Samsung 980 PRO "1TB" is 1,000,204,886,016 bytes: 1.00TB in the 1000 column and 931.51GB in the 1024 column. The Seagate "4TB" is 4.00TB / 3.64TB. The author's whole system adds up to 7.63TB decimal (6.94TB binary).

Unusual values and what they suggest. A total size much smaller than the label: a fake-capacity USB stick, a drive limited by a Host Protected Area, or a USB bridge that only sees part of a disk. A size of 0: no media. The 1000 vs 1024 gap itself is normal and not "missing space".

Where Linux gets it. `/sys/block/<disk>/size` (always in 512-byte units) and `/sys/block/<disk>/queue/logical_block_size`.

Background. *CHS* (cylinder-head-sector) is how disks were addressed in the 1980s and 1990s. Modern drives use *LBA* (Logical Block Addressing): the drive is just a numbered list of blocks. The 255/63 figures are a fixed translation kept for old partitioning tools; they say nothing about the real platters or flash chips.

Hidden disks

`hidden` - Linux data

In plain words. Storage devices that exist but do not show up as normal, usable disks.

What you see. Three short lists:

- **Block devices flagged hidden**: block devices with `/sys/block/*/hidden` set to 1. These are NVMe multipath "path" devices: the kernel hides the individual paths and shows one combined disk. Normally "none".
- **Devices present but without media (size 0)**: name, bus, type and model of each device that reports a size of 0.
- **NVMe generic (char) namespaces**: the `/dev/ngXnY` character devices. Every NVMe namespace has one; tools use them to send raw NVMe commands. They are not disks you can mount.

Healthy / typical values. The author's PC shows no hidden devices; two size-0 devices (an empty USB card-reader slot and a USB NVMe enclosure whose drive is not detected); and `/dev/ng0n1 /dev/ng1n1 /dev/ng2n1`, one per NVMe SSD.

Unusual values and what they suggest. A fixed internal drive under "without media": the drive answers on the bus but reports no capacity, which can mean failure, a locked drive, or a firmware fault. A hidden device on a desktop with no multipath setup is rare; check `nvme list-subsys`.

Where Linux gets it. `/sys/block/*/hidden`, `/sys/block/*/size`, and a listing of `/dev/ng*`.

Background. *Multipath* means one drive is reachable over more than one route (common in servers with dual-port NVMe). The kernel joins the routes into one disk so you do not write to the same drive twice by mistake.

JMicron JMB RAID

`jmb-raid` - Not present on most PCs

In plain words. Looks for JMicron storage controllers. On Windows, SIV reads RAID details through JMicron's own toolkit. On Linux there is no JMicron RAID interface, so SRS only reports whether any JMicron hardware is present.

What you see. "Checked: PCI vendor 0x197b, USB vendor 152d (JMicron)", then one line per match (PCI address with ID, class and driver; or USB port with VID:PID and product name), and a Status line: "Not present on this system" or "N device(s) found; Linux manages these with the ahci driver + md/dm".

Healthy / typical values. Most PCs have no JMicron PCI controller. Note that JMicron also makes very common USB-to-SATA and USB-to-NVMe bridge chips (USB vendor 152d), so a USB enclosure shows up here too. On the author's PC a USB 3.1 NVMe enclosure (152d:0583) is listed. That is a bridge, not a RAID controller.

Unusual values and what they suggest. A real JMicron SATA RAID card on Linux runs in plain AHCI mode: the disks appear separately, and any RAID is done with `mdadm` or device-mapper.

Where Linux gets it. `/sys/bus/pci/devices/*/vendor` and `/sys/bus/usb/devices/*/idVendor`.

JMicron NVMe RAID

`jms-raid` - Not present on most PCs

In plain words. The same JMicron check as JMicron JMB RAID, aimed at JMicron's USB bridge chips (the "JMS" series), some of which can join two drives into one inside the enclosure.

What you see. The same layout as JMicron JMB RAID. The Status line says Linux uses "uas/usb-storage (bridge RAID is configured in its firmware)".

Healthy / typical values. The author's PC lists one JMicron bridge (152d:0583, a single-drive USB NVMe enclosure). A single-drive enclosure has no RAID to show.

Unusual values and what they suggest. If a dual-bay JMicron enclosure is set to RAID 0 or 1, Linux only sees the one combined disk the bridge presents. The RAID mode is set with the enclosure's switch or the maker's tool, not from Linux.

Where Linux gets it. PCI and USB vendor IDs in sysfs.

Disk mapping

`mapping` - Linux data

In plain words. For each disk: every stable name Linux gives it, and for each partition its filesystem, label, mount points and IDs. Use it to write reliable `/etc/fstab` entries.

What you see. For each disk, a heading "Disk Drive /dev/ <model (serial)>", then:

- **Symbolic Link**: each `/dev/disk/by-id/...` name (model + serial, `eui.`, `wwn-` and `nvme.` IDs). These names never change between boots.
- **Path**: the `/dev/disk/by-path/...` names (which port the disk is on).
- One line per partition: name, filesystem and label (`<none>` if no label), and mount points (or "Not mounted", or `[SWAP]`). Below it, in grey, the **PARTUUID** (the partition's own ID, from the partition table) and the **UUID** (the filesystem's ID).
- A disk with no partitions gets one **Point** line with its mount points, or "No Volumes".

Healthy / typical values. On the author's PC the Seagate shows two by-id names (a `wwn-0x...` and an `ata-ST4000NM0033-9ZM170_XXXX` name) and two by-path names. A DOS (MBR) partitioned USB stick has short PARTUUIDs like `XXXXXXXX-01`: the 8-digit disk signature plus the partition number.

Unusual values and what they suggest.

- Two disks with the same by-id serial: cheap card readers and clones often share serials; use by-path or the filesystem UUID for those.
- A partition with a UUID but no filesystem name: udev saw something but could not identify it.
- An ISO-style UUID such as `2026-08-19-18-44-21-00`: that is an ISO 9660 (CD image) filesystem, whose "UUID" is its creation date. Normal on live USB sticks.

Where Linux gets it. The udev database (`S:` symlink lines and `ID_PART_ENTRY_UUID`, `ID_FS_UUID`), `/proc/self/mountinfo` and `/proc/swaps`.

Background. `sda` and `nvme0n1` can swap places between boots. A *UUID* (Universally Unique Identifier) or *PARTUUID* stays the same, so it is the safe way to point at a filesystem in `/etc/fstab` or a bootloader.

LSI MegaRAID

`mega-raid` - Not present on most PCs

In plain words. Looks for an LSI / Broadcom (also Avago) MegaRAID hardware RAID card, found in servers and workstations.

What you see. "Checked: PCI vendor 0x1000 (LSI/Broadcom)", any matching PCI devices (address, ID, class, driver), and Status: "Not present on this system" or "N device(s) found; Linux manages these with megaraid_sas + storcli".

Healthy / typical values. "Not present on this system" on the author's PC and on almost every desktop.

Unusual values and what they suggest. If a card is found, the disks behind it are hidden by the card's firmware; use Broadcom's `storcli` tool for array status. Note the check matches any PCI device from vendor 0x1000, which also covers LSI SAS HBAs (non-RAID host adapters).

Where Linux gets it. `/sys/bus/pci/devices/*/vendor`.

NTFS statistics

`ntfs` - Linux data

In plain words. Inside facts about each mounted NTFS volume, followed by the same kind of facts for native Linux filesystems (btrfs, ext2/3/4, XFS, F2FS).

What you see. For each filesystem, a heading "Statistics for () /dev/ ", then fields that depend on the filesystem type.

NTFS (ntfs3 driver):

Field	Meaning	Why it matters
NTFS version	Driver/format version, e.g. `ntfs3.1`	
Cluster size	Bytes per cluster (usually 4096)	
Clusters	Total clusters on the volume	
MFT records	Size of the Master File Table, in records	Each file or folder uses at least one record.

Field	Meaning	Why it matters
MFT records in use	Records holding files	Close to "MFT records" just means the table will grow; NTFS does that by itself.
Dirty (runtime)	`clean` or `dirty`: the driver's current view	
Dirty flag (on disk)	`clean` or `dirty`: the flag stored on the volume	`dirty` means Windows (or the driver) thinks the volume was not closed cleanly.
Label (ntfs3)	Volume label as the driver reads it	

btrfs: btrfs fsid (filesystem ID), Label, Node size (size of a metadata tree block, usually 16384), Sector size, Checksum (algorithm, e.g. `crc32c`), Generation (a counter that goes up at every commit), Features (on-disk features enabled), Devices (count and list), three Allocation lines (data, metadata, system: profile, space set aside, space used, percent, and raw disk use), Global reserve (emergency metadata space), "Device N errors" (write, read, flush, corruption and generation error counters; green when all zero, orange otherwise), and Commit stats (how many commits, and how long they took in ms).

ext2/3/4: the files the kernel exports under `/sys/fs/ext4/<dev>/`: `lifetime_write_kbytes`, `session_write_kbytes`, `errors_count`, `first_error_time`, `last_error_time`, `delayed_allocation_blocks`, `reserved_clusters`, `inode_readahead_blks`, `mb_stream_req`, `journal_task`.

XFS: Log head LSN and Log tail LSN (positions in the journal), then the raw XFS statistics block.

Other filesystems get the note "No filesystem-specific statistics exported by this filesystem driver." `ntfs:<mount point or device>` shows any single filesystem; the **FS** panel button uses `ntfs:/` to show the root filesystem.

Healthy / typical values. On the author's PC the 4TB NTFS data disk has 976,754,175 clusters of 4096 bytes and 5,139,584 MFT records, 4,563,941 in use. The btrfs root shows `data single`, `metadata dup`, `system dup`, all device error counters at 0, and a max commit time of 427 ms.

Unusual values and what they suggest.

- NTFS `dirty`: the volume was not closed cleanly (Windows Fast Startup or hibernation, a crash, or pulling the drive). On the author's PC the 4TB disk shows `dirty`. Fix it by running `chkdsk /f` from Windows (or `ntfsfix` as a lesser fallback). Writing to a dirty volume from Linux is a risk.
- btrfs data allocation at 95% or more used: the filesystem is close to full; btrfs can report "no space" before `df` shows 0. A balance (`btrfs balance start -dusage=50`) frees half-empty chunks.
- Any btrfs device error counter above 0: the disk has returned bad data or failed writes. Check SMART, cables and run `btrfs scrub`. `corruption_errs` means checksums did not match.
- ext4 `errors_count` above 0: the filesystem found damage; run `fsck` when it is unmounted.
- Very long btrfs commit times (several seconds): heavy write load or a slow or failing disk.

Where Linux gets it. `/proc/fs/ntfs3/<dev>/volinfo` and `label`, `/sys/fs/btrfs/<fsid>/`, `/sys/fs/ext4/<dev>/`, `/sys/fs/xfs/<dev>/`. All read-only.

Background. The *MFT* (Master File Table) is NTFS's index of every file. btrfs keeps data and metadata in separate pools of *chunks*; "allocated" is the space turned into chunks, "used" is what is really filled. The *dup* profile stores two copies of metadata on the same disk, which lets btrfs repair a bad metadata block by itself.

Partitions

`partitions` - Linux data

In plain words. The partition table of each disk: where every partition starts, how long it is, what type it claims to be, and whether it is bootable.

What you see. For each partitioned disk, a heading "/dev/ <model (serial)>

Signature ". The table type is `gpt` or `dos` (MBR); the signature is the GPT disk GUID or the 8-digit MBR disk ID. Then one row per partition:

Column	Meaning
#	Partition number
Device	Partition name
Type	Partition type, translated to a name where SRS knows it (EFI System, Microsoft basic data, Microsoft reserved, Windows recovery, Linux filesystem, Linux swap, Linux root (x86-64), Linux LVM, Linux RAID, Linux home, Linux extended boot, BIOS boot, LUKS, Linux /srv; for MBR: FAT16, FAT32, NTFS/exFAT/HPFS, Linux, Linux swap, EFI (FAT), Extended, hidden types and more). Unknown types show the raw GUID or hex code.
Partition Start / about	Start offset in bytes, then rounded
Partition Length / about	Length in bytes, then rounded
Name	GPT partition name (MBR partitions have none)
Hidden	`Yes` for MBR "hidden" type codes (0x10-0x1F range), else `No`
Boot	`ESP` for an EFI System Partition, `Active` for an MBR partition with the boot flag, else `No`

Disks with no partitions are skipped. `partitions:<disk>` shows one disk and says "no partition table" if it has none.

Healthy / typical values. Most partitions start at 1,048,576 bytes (1 MiB), the modern default. On a dual-boot NVMe on the author's PC: `EFI System` (2GB, Boot = ESP), `Microsoft basic data` (Windows), `Linux filesystem`, and a swap partition. A Ventoy USB stick shows an `Active` exFAT partition and a small `EFI (FAT)` one. Partition numbers do not have to follow the order on disk.

Unusual values and what they suggest.

- A type that does not match the content. On the author's Samsung NVMe, both the EFI partition and the btrfs root carry the GPT type GUID for "BIOS boot", so the EFI partition shows Boot = `No` instead of `ESP`. Linux mounts by UUID and does not care, but some firmware and installers look for the EFI System type to find the boot partition. `gdisk` or `sgdisk -t` can fix the type code.

- Type `0x0` on a USB stick: an "isohybrid" live image whose partition entry has no type. Normal for many Linux live ISOs written with `dd`.
- A start that is not a multiple of 1 MiB: see Disk sector sizes for whether it hurts.

Where Linux gets it. `/sys/class/block/<part>/{start,size,partition}` and the udev `ID_PART_TABLE_*` and `ID_PART_ENTRY_*` properties. SRS never reads raw sectors.

Background. *GPT* (GUID Partition Table) is the modern partition format: up to 128 partitions, each with a type GUID, a unique PARTUUID and a name. *MBR* (Master Boot Record, shown as `dos`) is the old format: four primary partitions, a one-byte type code, and an "active" flag for the old BIOS boot method. The *ESP* (EFI System Partition) is the FAT partition the UEFI firmware boots from.

Disk performance

`performance` - Linux data

In plain words. A live view of how much each drive is reading and writing, and how busy it is. In the GUI it refreshes about once a second.

What you see. Each drive gets two rows.

First row	Second row	Meaning
Disk Drive Description and Serial Number	Type, Location	Which drive
Read Size	Write Size	Total data read / written since boot
Read Delta	Write Delta	Read / write speed right now (bytes per second)
Read Count	Write Count	Total read / write requests completed since boot
Read Delta	Write Delta	Requests per second right now
Merge Count	Queue Depth	Merged reads since boot / requests in flight right now
Merge Delta	Busy %	Merged reads per second / share of time the drive had work to do

The last line gives the measured interval. The first time you open the item SRS takes two samples 0.25 s apart; after that the delta is since the previous refresh.

Healthy / typical values. An idle desktop shows most drives at `0/s` and `0.0%` busy. On the author's PC, the Samsung system SSD had read 1.83TB since boot and showed 0.8% busy, while the Seagate HDD showed 68.8% busy with a queue depth of 3 during a short read burst. HDDs reach high busy percentages at low speeds because each seek takes milliseconds.

Unusual values and what they suggest.

- Busy % near 100 for a long time with low speeds: the drive is the bottleneck (often an HDD doing random I/O, or a failing drive retrying reads).
- A large queue depth on a SATA HDD that never drains: the disk cannot keep up.

- Steady writes on an "idle" system: a background job (indexer, backup, swap, a browser cache). Use `iotop` to find the program.
- Busy % on an NVMe SSD can read high even when it has spare capacity, because an SSD works on many requests in parallel. Use speed and queue depth for SSDs.

Where Linux gets it. `/proc/diskstats` (read and write sectors, requests, merges, in-flight count and busy time), sampled twice.

Background. A *merge* happens when the kernel joins two requests for neighbouring blocks into one larger request. *Queue depth* is how many requests are waiting at the drive at once.

Mount points

`points` - Linux data

In plain words. Every place where a filesystem is attached to the folder tree, sorted into real disks, network shares, and virtual (kernel) filesystems.

What you see. Four columns: **Mount Point** (the folder), **Source** (the device, share or pseudo-name), **Type** (filesystem type), **Options** (mount options, cut at 90 characters). Three groups:

- **Block device filesystems**: filesystems on real disks.
- **Network filesystems**: NFS, SMB/CIFS, SSHFS, 9p, Ceph, GlusterFS, rclone, davfs, AFS.
- **Virtual / pseudo filesystems** (grey): `/proc`, `/sys`, tmpfs, cgroups, FUSE mounts, overlay and similar.

The last line gives the totals: "N block, N network, N virtual mount points".

Healthy / typical values. The author's PC has 17 block, 2 network and 33 virtual mount points. The btrfs root partition appears seven times (`/`, `/home`, `/var/log`...), one per subvolume. A second mount of the NTFS data disk at `/srv/newdl-ro` uses `ro` (read-only). Network shares include an NFS v4 export and a CIFS share from a router at `//192.0.2.x/...`.

Unusual values and what they suggest.

- A disk filesystem mounted `ro` that should be writable: the kernel may have switched it to read-only after errors. Check `dmesg`.
- `noatime` or `relatime` are normal and good for SSDs. A plain `atime`-only mount writes on every read.
- The same mount point listed twice: something was mounted over it. Only the last one is visible.

Where Linux gets it. `/proc/self/mountinfo`.

Background. A *bind mount* or *subvolume mount* attaches part of an existing filesystem at a second place. That is why one partition can have many mount points.

CSMI RAID

`raid` - Not present on most PCs

In plain words. On Windows, SIV reads RAID arrays through a CSMI driver. Linux has no CSMI; instead SRS shows the three kinds of Linux RAID: md software RAID, device-mapper, and btrfs multi-device profiles.

What you see.

- **md software RAID (/proc/mdstat)**: the contents of `/proc/mdstat` (arrays, members, state and any rebuild progress), or "md driver not loaded - no md RAID arrays".
- **device-mapper (LVM / dm-raid / dm-crypt)**: one line per dm or md device: name, size, kind (`LVM`, `dm-crypt` or the md level) and the devices under it. Or "no device-mapper devices".
- **btrfs multi-device profiles**: for each btrfs filesystem, its label, device count and the profile of data, metadata and system (single, dup, raid0, raid1, raid1c3, raid1c4, raid10, raid5, raid6).

Healthy / typical values. On the author's PC: no md arrays, no device-mapper devices, and three single-device btrfs filesystems with `data=single metadata=dup system=dup`.

Unusual values and what they suggest. In `/proc/mdstat`, a member list like `[U_]` means one disk of a mirror is missing (the array is *degraded*); `recovery` or `resync` with a percentage means a rebuild is running. A btrfs filesystem with several devices but `data=single` has no redundancy: losing one disk loses data.

Where Linux gets it. `/proc/mdstat`, `/sys/block/dm-*/dm/{name,uuid}`, `/sys/block/md*/md/level`, `/sys/block/*/slaves`, `/sys/fs/btrfs/<fsid>/allocation/`.

Background. *CSMI* (Common Storage Management Interface) is a Windows driver interface some RAID controllers offer. *RAID* (Redundant Array of Independent Disks) joins disks for speed (raid0), safety (raid1, mirrors) or both. Linux does RAID in software through *md* (`mdadm`), *device-mapper* (LVM) or the filesystem itself (btrfs).

RocketRAID

`rocket-raid` - Not present on most PCs

In plain words. Looks for a HighPoint RocketRAID (or other HighPoint) controller card.

What you see. "Checked: PCI vendor 0x1103 (HighPoint)", any matching PCI devices, and Status: "Not present on this system" or "N device(s) found; Linux manages these with the HighPoint rr* driver".

Healthy / typical values. "Not present on this system" on the author's PC and nearly all desktops.

Unusual values and what they suggest. A found card with driver `-` means no driver is bound: HighPoint drivers are usually out-of-tree, so install the vendor package (often a DKMS module).

Where Linux gets it. `/sys/bus/pci/devices/*/vendor`.

SCSI devices

`scsi` - Linux data

In plain words. Everything the Linux SCSI layer manages. That includes SATA disks, USB sticks, USB card readers and optical drives, because Linux talks to all of them using SCSI commands.

What you see.

Column	Meaning
H:C:T:L	Host : Channel : Target : LUN, the SCSI address
Type	Direct-Access (disk), CD/DVD, Sequential (tape), Enclosure, RAID, ZBC (zoned disk) and others
Vendor / Product / Revision	As the device reports over SCSI (SATA disks say `ATA` as vendor and cut the model to 16 characters)
Block	The block device name (`sda`, `sr0`)
Generic	The SCSI generic device (`sg0`...) used by tools such as `smartctl` and `sg_inq`
Host	Host adapter number and its driver (`ahci`, `usb-storage`, `uas`...)
State	`running` is normal

The last line lists every SCSI host and its driver.

Healthy / typical values. The author's PC: the Seagate SATA disk on `host0 ahci`, and USB devices on their own hosts with `usb-storage`. A multi-slot card reader appears as one target with several LUNs (`8:0:0:0` and `8:0:0:1`), one per slot.

Unusual values and what they suggest.

- State `offline` or `blocked`: the kernel stopped talking to the device after errors or a reset.
- A fast USB SSD on `usb-storage` rather than `uas`: the slower BOT protocol is in use, often because the bridge is on a quirks list. This costs speed.
- A host driver shown as `(null)`: the driver does not publish a name in sysfs. The `uas` driver does this; see USB disk drives for the real driver.

Where Linux gets it. `/sys/class/scsi_device/*/device/{type,vendor,model,rev,state,block,scsi_generic}` and `/sys/class/scsi_host/*/proc_name`.

Background. *SCSI* (Small Computer System Interface) began as a parallel cable standard, but its command set lives on: SATA, USB storage and SAS all carry SCSI-style commands. A *LUN* (Logical Unit Number) is one device behind one address, such as one slot of a card reader.

Samsung NVMe (secnvme)

`secnvme` - Linux data - Needs root data

In plain words. The same NVMe controller report as NVMe (stornvme), limited to Samsung controllers. On Windows, "secnvme" is Samsung's own NVMe driver. Linux uses the in-kernel `nvme` driver for all brands, so this is just a Samsung filter.

What you see. For each controller with PCI vendor 0x144d (or "Samsung" in the model), all the fields described under NVMe (stornvme). If none: "No Samsung NVMe controllers found".

Healthy / typical values. The author's Samsung 980 PRO 1TB: PCI ID 144d:a80a, PCIe link "16.0 GT/s PCIe x4 (max 16.0 GT/s PCIe x4)", Composite temperature 37.9C, one namespace of 931.51GB with 512-byte LBAs.

Unusual values and what they suggest. See NVMe (stornvme). The Samsung tool "Magician" is Windows-only; on Linux, firmware updates use Samsung's bootable ISO or `fwupd` where supported.

Where Linux gets it. `/sys/class/nvme/*` and the `srs-root` cache.

Disk sector sizes

`sectors` - Linux data

In plain words. The block sizes each disk reports, its preferred I/O sizes, its I/O scheduler, and whether each partition starts on a well-aligned boundary.

What you see. A table per disk:

Column	Meaning	Why it matters
Logical	Smallest block the OS can address (usually 512, sometimes 4096)	
Physical	Real block size inside the drive (512 or 4096)	A 4096-physical / 512-logical drive ("512e") does slow read-modify-write cycles on badly aligned writes.
HW Sector	Hardware sector size the driver uses	
Min I/O	Smallest efficient I/O	
Optimal I/O	Preferred I/O size (0 = not reported); RAID arrays report the stripe size here	
Align Ofs	Alignment offset in bytes (should be 0)	Non-zero means the drive's first block is shifted.
Discard Gran	Discard (TRIM) granularity; 0 = no discard support	0 on an SSD means TRIM cannot be used.
Max I/O	Largest single request the kernel will send (KB)	
Scheduler	Active I/O scheduler (`none`, `mq-deadline`, `bfq`, `kyber`, or others)	

Then **Partition alignment**: for each partition, its start sector, byte offset, and three checks: **Physical** (start is a multiple of the physical block size), **4K**, and **1MiB**.

Healthy / typical values. On the author's PC all NVMe and SATA drives report 512/512, align offset 0, and every main partition starts at sector 2048 (1 MiB) or another 1 MiB multiple, so all three boxes are ticked. NVMe drives use `none` (or another low-overhead scheduler) and spinning or USB disks use `bfq`. zram reports 4096 for everything, as expected for a RAM-backed device.

Unusual values and what they suggest.

- **4K unticked** on a partition of a 4K-physical drive: every write may touch two physical blocks. This was common with Windows XP-era partitioning (start sector 63). Recreate or move the partition.
- **1MiB unticked but 4K ticked**: fine for performance. It is common on live-USB images (the author's Garuda stick starts its first partition at sector 64).
- **Physical and 4K both unticked**: misaligned. On SSDs and 4K drives this costs speed and extra wear.
- Discard granularity 0 on an SSD: TRIM is not possible on this path (common behind USB bridges).

Where Linux gets it. `/sys/block/<disk>/queue/{logical_block_size,physical_block_size,hw_sector_size,minimum_io_size,optimal_io_size,discard_granularity,max_sectors_kb,scheduler}`, `/sys/block/<disk>/alignment_offset`, and each partition's `start`.

Background. An *I/O scheduler* decides the order in which waiting requests go to the drive. Fast NVMe drives usually do best with `none`; HDDs benefit from `bfq` or `mq-deadline`, which group nearby requests to save seeks.

SMART drives

`smart` - Linux data - Needs root data

In plain words. A health summary of each drive: is SMART available, the temperature, the drive's own pass/fail verdict, and how many hours it has been powered on. The full reports are in SMART all drives.

What you see.

Column	Meaning
Drive	Kernel name
Model (Serial)	Model and serial
Bus	NVMe, SATA, USB...
Size	Capacity
SMART	`NVMe log` for NVMe drives; `Enabled` or `Disabled` for ATA drives (from udev); `unknown (USB bridge)` for USB drives; `-` if unknown
Temperature	Live, from the kernel's hwmon sensor for that drive, or else from the SMART data
Health	The overall verdict: `PASSED` or `FAILED` (from `smartctl`), or for NVMe-only data "OK (no critical warning)" or "CRITICAL WARNING"
Power On	Power-on hours

Below the table: either "Health / attributes: Needs root" (no `srs-root` data yet), or "srs-root cache: N drives". Optical drives and devices without media are left out.

`smart:<dev>` (for example `smart:sda`) shows one drive in full: bus, firmware, every temperature sensor, then for NVMe the controller state, or for ATA drives the feature flags udev knows (SMART, SMART enabled, Security, Security frozen, Power management, APM, Power-up in standby, Write cache, Read look-ahead, HPA, Download microcode, SATA 3Gb/s, SATA 1.5Gb/s) and the WWN, and finally the whole cached `smartctl -x` report.

Healthy / typical values. On the author's PC all four internal drives say `PASSED`. NVMe SSDs run at 35-40C at idle, the Seagate HDD at 38C. Power-on hours: 24,137 for the Samsung SSD (about 2.75 years of running time), 640 for a new PNY SSD, 76,271 for the Seagate (about 8.7 years). The three USB sticks and card readers show `unknown (USB bridge)` and `-` because `srs-root` skips USB disks by default.

Unusual values and what they suggest.

- `FAILED`: the drive itself says it is failing or about to fail. Back up now and replace it.
- `PASSED` does not mean "perfect". The verdict only flips when a normalised value crosses its threshold; a drive can pass while its raw counters (for example reallocated sectors) are rising. Read the attributes in SMART all drives.
- `Disabled`: SMART is switched off in the drive; `smartctl -s on /dev/sdX` turns it on.
- NVMe temperature above about 70C at idle: poor airflow or a missing heatsink. HDDs prefer to stay below about 45-50C.

Where Linux gets it. The `srs-root` cache (`~/.cache/srs/root/smart-<dev>.txt` from `smartctl -x`, and `nvme-<dev>.txt` from `nvme smart-log`), the udev `ID_ATA_FEATURE_SET_*` properties, and the `nvme` / `drivetemp` hwmon sensors.

Background: what SMART is. *SMART* (Self-Monitoring, Analysis and Reporting Technology) is a log the drive keeps about itself: error counts, wear, temperatures, hours. Reading it needs raw access to the drive, which needs root; that is why SRS collects it with `srs-root` and caches it. USB disks are skipped by default because many USB bridges do not pass SMART commands through, and some flaky enclosures disconnect when they get one. Set `SRS_SMART_USB=1` to include them anyway.

Background: how to read ATA (SATA) SMART attributes. SATA and older drives report a table of numbered *attributes*. Each row has these columns (as `smartctl` prints them):

Column	Meaning
ID# / ATTRIBUTE_NAME	The attribute number and its usual name. Numbers are mostly standard; meanings can differ by maker.
FLAGS	Letters: **P** pre-failure (crossing the threshold predicts failure), **O** updated during normal use, **S** speed/performance, **R** error rate, **C** event count, **K** auto-keep.
VALUE	The *normalised* value: the drive's own score, usually starting at 100 (or 200, or 253) and going **down** as things get worse.
WORST	The lowest normalised value ever recorded.
THRESH	The *threshold*: the maker's failure line. If VALUE drops to or below THRESH, that attribute has failed. 0 means "never fails, information only".

Column	Meaning
FAIL	`-` = fine; `FAILING_NOW` = VALUE is at or below THRESH now; `In_the_past` (shown short as `Past`) = WORST once dropped to or below THRESH, but VALUE has since recovered.
RAW_VALUE	The real counter (sectors, hours, degrees, events). Its meaning and format are up to the maker.

So: *normalised* values are the maker's verdict on a common scale; *raw* values are the actual counts. For counters such as reallocated sectors, the raw number is the one to watch, because the normalised value often stays at 100 until things are already bad. For "rate" attributes on Seagate drives (1 Raw Read Error Rate, 7 Seek Error Rate, 195 Hardware ECC Recovered) the raw number is a packed value of several counters and looks huge even on a healthy drive; judge those by the normalised value only.

Attributes that matter most:

ID	Name	What it counts	Worry when
5	Reallocated_Sector_Ct	Bad sectors the drive has replaced with spares	Raw above 0 and growing. A few may be stable for years; a rising count means the surface is failing.
197	Current_Pending_Sector	Sectors that could not be read and wait to be remapped	Any raw above 0. These are unreadable right now.
198	Offline_Uncorrectable	Sectors that failed during the drive's own offline scan	Any raw above 0.
187	Reported_Uncorrect	Errors the drive could not fix with ECC and reported to the host	Raw above 0 means data was lost on a read.
10	Spin_Retry_Count	Spin-ups that needed a retry (HDD)	Raw above 0: motor or power problems.
184	End-to-End_Error	Data corrupted inside the drive between cache and media	Any raw above 0.
188	Command_Timeout	Commands that timed out	Rising counts: cable, power or drive problems. Seagate packs three counters into the raw value, so judge by the normalised value.
199	UDMA_CRC_Error_Count	Data corrupted on the cable between drive and controller	A rising count points to a bad SATA cable or connector, not the disk itself. The count never resets.
9	Power_On_Hours	Hours powered on	Information; compare with the drive's rated life.
12	Power_Cycle_Count	Times powered on	Information.

ID	Name	What it counts	Worry when
4 / 193	Start_Stop_Count / Load_Cycle_Count	Spin-ups / head parking events (HDD)	Load cycles growing by hundreds per day: aggressive power saving (APM) is wearing the drive.
190 / 194	Airflow_Temperature_Cel / Temperature_Celsius	Temperature (raw = degrees C; extra numbers are min/ max)	Above the drive's rated maximum, often 55-60C.
177 / 231 / 233 / 202	Wear_Leveling_Count / SSD_Life_Left / Media_Wearout_Indicator / Percent_Lifetime_Remain	SSD wear (names vary by maker)	Normalised value near the threshold, or "life left" under about 10%.

Example from the author's Seagate 4TB disk. VALUE / WORST / THRESH for 5 Reallocated_Sector_Ct are 100 / 100 / 010 with raw 0: no bad sectors. 197 and 198 are also 0. 190 Airflow_Temperature_Cel is 064 / 041 / 045 and its FAIL column says `Past`: Seagate scores this attribute as 100 minus the temperature, so a WORST of 41 means the disk once reached 59C, above its limit of 55C (the SCT data agrees: lifetime maximum 59C). The disk is fine now at 36C, but it was too hot at some point. 199 UDMA_CRC_Error_Count raw is 19: there were cable glitches in the past; if the number stays at 19 there is nothing to do, if it grows, replace the SATA cable. 1, 7 and 195 have huge raw values that are normal for Seagate.

Background: how to read NVMe SMART. NVMe drives have no attribute table. They give one fixed "SMART / Health Information" log page:

Field	Meaning	Worry when
Critical Warning	A bit field; 0x00 = all good. Bits mean: spare below threshold, temperature over limit, reliability degraded, read-only mode, volatile backup failed.	Anything other than 0x00.
Temperature / Temperature Sensor N	Composite and per-sensor temperatures	Near the "Warning Comp. Temp. Threshold" given in the report.
Available Spare	Spare flash blocks left, as a percentage (starts at 100%)	Falling towards the Available Spare Threshold.
Available Spare Threshold	The level at which the drive raises a critical warning	
Percentage Used	The drive's estimate of how much of its rated write life is used	Near or above 100%. It can go past 100; the drive still works but is past its rating.
Data Units Read / Written	Data moved, in units of 1,000 x 512 bytes (512,000 bytes); smartctl adds the total in TB	Compare written TB with the drive's TBW (terabytes written) rating on its spec sheet.
Host Read / Write Commands	Commands completed	Information.
Controller Busy Time	Minutes the controller was busy	Information.

Field	Meaning	Worry when
Power Cycles / Power On Hours	Power-ups and powered hours	Information.
Unsafe Shutdowns	Times power was lost without a proper shutdown notice	A high share of power cycles points to hard resets or power cuts; usually harmless, but data in flight can be lost.
Media and Data Integrity Errors	Unrecovered data errors	Any value above 0.
Error Information Log Entries	Entries in the error log over the drive's life	Rising numbers; many drives log harmless entries, so check the error log itself.
Warning / Critical Comp. Temperature Time	Minutes spent above the warning / critical temperature	Growing values: improve cooling.
Thermal Temp. N Transition Count / Total Time	How often and how long the drive slowed itself to cool down	Many transitions: the drive throttles under load.

Example from the author's Samsung 980 PRO 1TB. Critical Warning 0x00, Available Spare 100% (threshold 10%), Percentage Used 38%, about 333 TB written, Media and Data Integrity Errors 0, and 7,932 unsafe shutdowns out of 11,989 power cycles. The drive is healthy; it has used about a third of its rated write life. The high unsafe-shutdown share means the PC was often reset or powered off without a clean shutdown.

Self-test logs. Both kinds of drive keep a list of recent self-tests (short and extended). "Completed without error" is good. "Aborted" (by the host, by a command, or by a controller reset) only means the test was interrupted, often by a reboot or by the system going to sleep; run it again. A test that ends with "read failure" and a failing LBA means a real bad sector.

SMART all drives

`smart-all` - Linux data - Needs root data

In plain words. The full SMART report of every drive, one after another. This is the place to read the attributes explained under SMART drives.

What you see. For each drive with media (optical drives skipped):

- A heading with the drive name and model (serial).
- **Bus / type**, **Firmware**, **Temperatures** (every hwmon sensor, for example "Composite 37.9C Sensor 1 37.9C Sensor 2 45.9C" on an NVMe SSD), **SMART** (as in the SMART column of SMART drives).
- **Source**: the cache file the report came from.
- The cached `smartctl -x` output in full: an information section (model, firmware, capacity, versions, power states, temperature thresholds), the SMART data section (health verdict, attributes

or the NVMe health log), error logs, self-test logs, and for SATA drives also SCT temperature history, device statistics and SATA phy event counters.
- For USB disks without `SRS_SMART_USB=1`: "skipped: USB-attached (set SRS_SMART_USB=1 to include)".
- Without `srs-root` data: "SMART data: Needs root".

`smart-all:<dev>` gives the same single-drive view as `smart:<dev>`.

Healthy / typical values. See the examples under SMART drives. For a SATA disk, the SCT section shows "Lifetime Min/Max Temperature" (9/59 C on the author's Seagate) and a 10-minute temperature history. "?" entries in that history are times when the PC was off or asleep. The SATA phy event counters should be 0 apart from a few COMRESET entries after boots.

Unusual values and what they suggest. Non-zero "Command failed due to ICRC error" or R_ERR counters in the SATA phy events point at the cable or port, like attribute 199. "ATA Security is: ... frozen" is normal: the BIOS locks the security commands at boot so nothing can set a drive password.

Where Linux gets it. `~/.cache/srs/root/smart-<dev>.txt`, written by `srs-root` running `smartctl -x /dev/<dev>`; NVMe drives also get `nvme-<dev>.txt` from `nvme smart-log`.

Background. *SCT* (SMART Command Transport) is a newer ATA channel for extra data such as temperature history and error recovery timeouts. *SCT Error Recovery Control* (shown as e.g. "Read: 100 (10.0 seconds)") limits how long the drive retries a bad sector; NAS and RAID disks set it low so the array does not drop the disk.

SMART drive monitor

`smart-sdm` - Linux data - Needs root data

In plain words. A compact live table of drive temperatures, with power-on hours and power cycles. In the GUI it refreshes about every 2 seconds.

What you see.

Column	Meaning	Source
Drive / Model	Kernel name and model	sysfs / udev
Current	Temperature now	hwmon `temp1_input` (live, no root)
Lowest	Lowest since the sensor started	hwmon `temp1_lowest` (drivetemp only)
Highest	Highest since the sensor started	hwmon `temp1_highest` (drivetemp only)
Critical	The drive's critical temperature limit	hwmon `temp1_crit` (NVMe)
Power On	Power-on hours	`srs-root` SMART cache
Cycles	Power cycles	`srs-root` SMART cache

`-` means the value is not available. Without `srs-root` data, a note says power-on hours and cycles need root.

Healthy / typical values. On the author's PC: NVMe drives at 35.9-39.9C with critical limits of 84.8C (Samsung), 109.8C (ADATA) and 94.8C (PNY); the Seagate at 38.0C with lowest 34.0C and highest 41.0C since boot. NVMe drives show `-` for Lowest/Highest because the kernel NVMe sensor does not track them; SATA drives show `-` for Critical because `drivetemp` does not always report one.

Unusual values and what they suggest. Current within about 10C of Critical: the drive is close to throttling. A Highest far above Current on an HDD: a heavy job or poor airflow earlier. USB drives show `-` everywhere: no hwmon sensor and no SMART data through the bridge.

Where Linux gets it. `/sys/class/nvme/<ctrl>/hwmonN/` for NVMe and `/sys/block/<disk>/device/hwmon/hwmonN/` for SATA (the `drivetemp` kernel module), plus the `srs-root` cache.

Background. *hwmon* is the kernel's hardware monitoring interface. The `drivetemp` module reads SATA drive temperatures through SMART without root; load it with `modprobe drivetemp` if SATA temperatures are missing.

Storage spaces

`spaces` - Linux data - Needs root data

In plain words. On Windows, "Storage Spaces" pools several disks into one. The Linux equivalents are btrfs (which can span disks) and device-mapper / LVM / md. This item lists all of them. In practice everything shown here comes from public sysfs files, so it is complete without `srs-root`.

What you see.

- **btrfs filesystems (pooled storage)**: for each btrfs filesystem, a "btrfs " heading and the full btrfs detail described under NTFS statistics: fsid, label, node and sector size, checksum, generation, features, devices, allocation per type with profile, global reserve, per-device error counters, commit stats.
- **device-mapper / LVM / md volumes**: name, size, kind (`LVM`, `dm-crypt`, or the md RAID level) and the underlying devices.
- **Pools**: "N btrfs, N dm/md".

Healthy / typical values. On the author's PC: three btrfs filesystems (LINUX, SteamLibrary, 500GB_SSD), each on one device, with error counters all 0, and no dm/md volumes.

Unusual values and what they suggest. A btrfs filesystem whose Devices count is lower than you set up: a member disk is missing (the filesystem may only mount with `-o degraded`). Non-zero error counters: see NTFS statistics. A dm-crypt line with no mount: an unlocked encrypted container that is not mounted.

Where Linux gets it. `/sys/fs/btrfs/<fsid>/`, `/sys/block/dm-*/dm/`, `/sys/block/md*/md/`.

Background. *LVM* (Logical Volume Manager) and *dm-crypt* (disk encryption) are built on the kernel's *device-mapper*, which maps one virtual block device onto parts of real ones.

NVMe (stornvme)

`stornvme` - Linux data - Needs root data

In plain words. A detailed card for each NVMe SSD controller. On Windows, "stornvme" is Microsoft's NVMe driver; on Linux the in-kernel `nvme` driver handles every brand, and SRS reads what it publishes.

What you see. For each controller (`/dev/nvme0`, `/dev/nvme1`...):

Field	Meaning	Why it matters
Model / Serial number / Firmware	Identity	Firmware updates fix real bugs on many SSDs.
State	`live` is normal	`resetting`, `connecting`, `dead`: the drive is having trouble.
Transport	`pcie` for an internal SSD (`tcp`, `rdma`, `fc` for network NVMe)	
Controller ID	The controller's number inside the subsystem	
Controller type	`io`, `admin`, `discovery`, or `reserved` (older drives that do not report a type)	
Subsystem NQN	NVMe Qualified Name: a unique name for the drive, often with the serial in it	
I/O queues	Number of queues the driver set up (often one per CPU, plus one)	
SQ size	Submission queue depth	
NUMA node	Which memory node the slot belongs to (`-1` on single-node desktops)	
Keep-alive	Keep-alive timeout (0 for local PCIe drives)	
PCI address / PCI ID	Where it sits on the PCI bus, and vendor:device ID	
PCIe link	Current speed and width, with the maximum in brackets; green when they match, orange when lower	The single most useful line here: shows whether the SSD runs at full speed.
Driver	`nvme`	
Temperatures	Every sensor (Composite plus Sensor 1, 2...)	
Namespace nvmeXnY	Size and LBA (block) size of each namespace	

Then either "SMART log available in the srs-root cache, see [smart]" or, if no cache exists, "Identify/SMART: Needs root".

Healthy / typical values. On the author's PC all three SSDs show "16.0 GT/s PCIe x4 (max 16.0 GT/s PCIe x4)" (PCIe 4.0 x4, full speed), state `live`, 17 I/O queues on a 16-thread CPU, and NUMA node -1. The PNY's NQN contains its serial, shown here as `nqn.2014-08.com.00000:nvme:XXXX`.

Unusual values and what they suggest.

- PCIe link lower than max, e.g. "8.0 GT/s x4 (max 16.0 GT/s x4)": a slot that only runs PCIe 3.0, a BIOS setting, or a riser/adapter. "x2" on an x4 drive: the M.2 slot shares lanes with another device (check the board manual). Some drives also drop speed at idle to save power, so check under load.
- Temperatures: NVMe drives start throttling near the warning threshold (shown in SMART all drives).
- State not `live`: see `dmesg` for "nvme ... timeout" or "resetting controller" messages; often a power-saving (APST) problem or failing drive.

Where Linux gets it. `/sys/class/nvme/<ctrl>/{model,serial,firmware_rev,state,transport,cntlid,cntrltype,subsysnqn,queue_count,sqsize,numa_node,kato}`, the PCI device's `current_link_speed`, `current_link_width`, `max_link_speed`, `max_link_width`, and the controller's hwmon sensor.

Background. *NVMe* (Non-Volatile Memory Express) is the protocol fast SSDs use over *PCIe* (PCI Express). *GT/s* (gigatransfers per second) is the per-lane speed: 8.0 = PCIe 3.0, 16.0 = PCIe 4.0, 32.0 = PCIe 5.0. A *namespace* is a slice of the SSD presented as one disk; consumer SSDs have one.

USB disk drives

`usb-disks` - Linux data

In plain words. Every disk connected over USB, with the bridge chip inside the enclosure, its USB speed and the driver in use.

What you see.

Column	Meaning	Why it matters
Disk	Kernel name	
Vendor and Product	What the disk reports (with serial)	
USB Bridge (Manufacturer Product)	What the USB device reports	Shows the real chip or enclosure brand.
VID:PID	USB vendor and product ID	Search this to find the exact bridge chip and known quirks.
Speed	Link speed in Mbit/s: 480M = USB 2.0, 5000M = USB 3.x Gen 1, 10000M = Gen 2, 20000M = Gen 2x2	A fast drive on 480M is stuck at USB 2.0 speed.
Driver	`uas` (USB Attached SCSI, faster) or `usb-storage` (older Bulk-Only Transport)	
Size	Capacity, or "no media"	
Port	USB bus and port path, e.g. `2-2.4.1` = bus 2, port 2, hub port 4, hub port 1	

The last line counts USB disks.

Healthy / typical values. On the author's PC: a USB NVMe enclosure (JMicron 152d:0583) at 10000M on `uas`; a USB 3.0 card reader (05e3:0749) at 5000M with two slots (`sdd`, `sde`); a USB 2.0 flash drive and a DVD writer at 480M on `usb-storage`.

Unusual values and what they suggest.

- A USB 3 drive at 480M: it is in a USB 2.0 port, on a USB 2.0 hub or cable, or the connector is not fully seated.
- A fast SSD enclosure on `usb-storage` instead of `uas`: the kernel turned UAS off for this bridge (a known-quirk list), costing speed but avoiding hangs.
- "no media" on an enclosure that should hold a drive: the drive inside is not detected. On the author's PC the NVMe enclosure shows this.

Where Linux gets it. The sysfs path of each `/sys/block/<disk>`, walked up to the USB device (`idVendor`, `idProduct`, `manufacturer`, `product`, `speed`) and the interface `driver` link. Devices are not opened.

Background. A *USB bridge* is the chip in an enclosure, stick or reader that turns USB into SATA, NVMe or flash commands. Its quality decides speed, TRIM support and whether SMART works at all.

Volume list

`vol-list` - Linux data

In plain words. Every disk and partition with its filesystem type, version, label, UUID, size and mount points: the same idea as `lsblk -f`.

What you see. Columns **Name** (disks at the left, partitions indented), **FSType** (vfat, ntfs, btrfs, ext4, swap, iso9660...), **Version** (filesystem version, e.g. FAT32, FAT16, FAT12, exFAT 1.0, swap 1, "Joliet Extension" for ISO images), **Label**, **UUID**, **Size**, **Mount points** (all of them, or `[SWAP]`). zram and other virtual disks are included.

Healthy / typical values. On the author's PC: `nvme0n1p1 vfat FAT32 EFI 2.05GB /boot/efi`, `nvme1n1p4 swap 1 SWAP 32.81GB [SWAP]`, `zram0 swap 1 zram0 15.23GB [SWAP]`. A small Ventoy EFI partition is FAT16 and a 4MB live-USB EFI partition is FAT12, both normal for their size. NTFS partitions show a 16-hex-digit UUID.

Unusual values and what they suggest. A whole disk showing the same filesystem as its first partition (for example `iso9660` on both `sde` and `sde1`): an isohybrid live image, normal. A partition with an empty FSType: no filesystem udev recognises (or an encrypted or damaged one). Two partitions with the same UUID: a cloned disk; mounting by UUID becomes unpredictable, so change one with `tune2fs -U`, `btrfstune -u` or similar.

Where Linux gets it. `/sys/class/block`, the udev database (`ID_FS_TYPE`, `ID_FS_VERSION`, `ID_FS_LABEL_ENC`, `ID_FS_UUID`), `/proc/self/mountinfo`, `/proc/swaps`. No `blkid` is run.

Volumes

`volumes` - Linux data

In plain words. The main "how full are my drives" table: every mounted disk filesystem and network share, with free space, total space, and percent used. This is what the **Volumes** panel button opens.

What you see.

Column	Meaning
Drive and Path	Mount point
Volume Label	Filesystem label, `<none>`, or the share path for network mounts
Format	Filesystem type; `(R)` means mounted read-only
Quota	Empty (kept for the SIV layout)
Free	Space available to normal users
Total	Filesystem size
Used	Percent of the filesystem in use (total minus all free blocks)
Cluster	Block size the filesystem reports
Drive Type	Fixed, Removable, CD Drive, RAM Disk, Virtual, Network
Volume Serial	FAT serial, NTFS serial (the low 8 hex digits of its ID, as Windows shows it), or the UUID

Network mounts show `-` for sizes: SRS never queries them, because a stalled server would freeze the query. After the table come **Total** rows per drive type: "count (distinct)", where count is the number of mount points and the bracketed number is how many different filesystems they are; Free, Total and Used are added up once per filesystem.

Per-volume detail (`volumes:<mount point or device>`, or the **Drive /** button for `/`): Mount point, Source, Device (with major:minor numbers), Physical disk, Drive type (type, bus, media), Format (with version), Volume label, Volume serial / UUID, Partition (number, GPT name, type GUID), Subvolume / bind root (for btrfs subvolumes and bind mounts, e.g. `/@`), Mount options, Super options (filesystem-level options such as `compress=zstd:3,ssd,discard=async`), Total, Free ("available" and "incl. reserved"), Used (amount and percent), Block / fragment size, Inodes (total and free, `n/a` for filesystems such as btrfs that do not have a fixed count), Max name length, Access (read/write or read-only), followed by the filesystem statistics described under NTFS statistics.

Healthy / typical values. On the author's PC the btrfs root shows 146.84GB free of 929.46GB (83% used) on seven mount points; the totals line "Fixed 14 (5)" means 14 mount points but only 5 different filesystems. The read-only NTFS bind mount shows `ntfs3 (R)`. An ISO live stick shows 100% used with 0 free, which is normal for a read-only ISO 9660 filesystem.

Unusual values and what they suggest.

- Used above 90% on the root filesystem: updates and logs can fail; btrfs gets slow and may report "no space" early.
- Free (available) much lower than "incl. reserved" on ext4: ext4 keeps 5% for root by default; `tune2fs -m 1` lowers that on a big data disk.

- `(R)` on a filesystem you did not mount read-only: the kernel switched it after errors, or NTFS is dirty (see NTFS statistics).

Where Linux gets it. `/proc/self/mountinfo`, `statvfs()` on each local mount point, the udev `ID_FS_*` and `ID_PART_ENTRY_*` properties, and sysfs for the disk under each partition.

Background. An *inode* is a filesystem's record for one file. ext4 creates a fixed number when formatted, so it can run out of inodes before it runs out of space; btrfs and XFS create them as needed. *Reserved space* is kept back for root so the system can still log and recover when users fill a disk.

Intel VROC

`vroc` - Not present on most PCs

In plain words. Looks for Intel VROC (Virtual RAID on CPU), Intel's RAID for NVMe drives attached to the processor, found on workstation and server boards.

What you see. "Checked: PCI vendor 0x8086 class 0x0104 (Intel RAID/VMD)", any matching PCI devices (address, ID, class, driver), and Status: "Not present on this system" or "N device(s) found; Linux manages these with vmd + mdadm (IMSM)".

Healthy / typical values. "Not present on this system" on the author's AMD PC and most desktops.

Unusual values and what they suggest. On Intel laptops with "RAID / Intel RST" mode on in the BIOS, a VMD controller appears and NVMe drives hide behind it. Linux supports it with the `vmd` driver; if Linux cannot see the SSD at all, switching the BIOS storage mode to AHCI/NVMe is the common fix (Windows then needs its drivers adjusted first).

Where Linux gets it. `/sys/bus/pci/devices/*/{vendor,class}`.

Background. *VMD* (Volume Management Device) is the Intel PCIe block VROC and RST use to gather NVMe drives. Linux builds the RAID itself with `mdadm` in Intel's *IMSM* (Intel Matrix Storage Manager) metadata format, so the array also works under Windows.

Network (SRS Section Reference)

Back to the section index - Glossary

Where to find these items (SRS 2.0). In the main window they are on the **Network** page (Alt+7). Its tabs are Network adapters, IP address, Wi-Fi SSIDs, Bluetooth radios, Routing table, TCP V4 ports and TCP/IP counters; **All N items** lists the rest. You can also type an item's name or id into the search box (Ctrl+F). The rest of this introduction describes the classic view (Menu -> Classic View), with its Menu and panel buttons.

This group covers everything about how your PC talks to other machines: network adapters and their addresses, traffic counters, the routing and neighbour tables, open ports, DNS, the firewall, saved connection profiles, Wi-Fi, Bluetooth, and Windows-style file sharing (SMB/Samba). Open it from **Menu -> Network -> item**, or press the **Adapters** panel button. Its sub-buttons jump to Counters, Interface, ARP, Routing, TCP V4, TCP V6, Wi-Fi, Bluetooth, Show DNS, Ping and Main. Every item here only reads local files and asks local tools; nothing is sent on the network. The one exception is the **Ping** button. It opens the Tools group's Ping item, which sends three pings to your default gateway (your router). That is the only network traffic SRS ever makes. Three items need root data (run **srs-root**) to be complete: DNS cache, Firewall and WLAN profiles. The port tables also show more process names under root.

Contents

- Network adapters `adapters`
- ARP table `arp-table`
- Bluetooth radios `bluetooth`
- Bluetooth IDs `bluetooth-ids`
- Bluetooth device tree `blue-tree`
- Network browse masters `browsers`
- TCP/IP counters `counters`
- DNS cache `dns-cache`
- DNS names `dns-names`
- DNS ports `dns-ports`
- DNS usage `dns-usage`
- Domains / workgroups `domains`
- Firewall `firewall`
- Network interface statistics `interface`
- IP address `ip-address`
- LAN Manager share setup `lm-setup`
- MAC addresses `mac-address`
- Modem API `modem-api`
- Modems `modems`
- My browser / IP location `my-browser`
- My IP address `my-ip-address`
- Network cards `net-cards`

- Network provider names `net-names`
- Network `network`
- Network profiles `profiles`
- Network protocols `protocols`
- Proxy config `proxy`
- Routing table `routing`
- Servers `servers`
- TCP V4 ports `tcp-ip4-ports`
- TCP V6 ports `tcp-ip6-ports`
- Network transports `transports`
- UDP V4 ports `udp-ip4-ports`
- UDP V6 ports `udp-ip6-ports`
- Wi-Fi SSIDs `wi-fi`
- WLAN profiles `wlan`

A note on the examples: real addresses from the author's PC have been replaced. The home network is shown with the documentation addresses 192.0.2.x, which are set aside for examples and are never used on a real network (router 192.0.2.1, wired card 192.0.2.20, Wi-Fi 192.0.2.21). A real home network usually uses 192.168.x.x, 10.x.x.x or 172.16-31.x.x. MAC addresses, link-local IPv6 addresses and Wi-Fi names are shown as `XX:XX:...`, `fe80::XXXX` and "HomeWiFi".

Network adapters

`adapters` - Linux data

In plain words. One block per network adapter (real or virtual), with its addresses, how it got them, its DNS servers, gateway and the multicast groups it listens to. This is the Linux version of Windows' `ipconfig /all`.

What you see.

Top line (whole PC):

Field	Meaning	Why it matters
Linux IP Configuration	Your OS name and kernel.	Context.
Host	The computer's host name.	How other machines may know you.
Domain	DNS (Domain Name System) domain or search domain, "-" if none.	Home PCs usually have none.
DNS IP Address	DNS servers from `/etc/resolv.conf`, one per line.	Who turns names into addresses.
Routing / Proxy	"Enabled/Disabled" for IP forwarding, then for a proxy in the environment.	Forwarding on means the PC can pass packets between adapters (Docker and VMs turn it on).

Then one block per adapter (physical first, then virtual, loopback last). Each block has at least three rows:

Column	Meaning	Why it matters
Adapter / Description	Row 1: interface name. Row 2: driver. Under the block: full description (PCI name, USB product, or "Bridge (virtual, driver bridge)").	Tells you which chip and driver handle the port.
Type (Index)	Row 1: Ethernet, Wi-Fi, Bridge, TUN, TAP, veth, Loopback, PPP, etc., plus the kernel's interface index. Row 2: MAC address.	The index is a stable number tools use instead of the name.
Speed	Row 1: link speed (only when up). Row 2: MTU. Row 3: state (up, down, unknown, dormant).	Wrong speed or "down" is the first thing to check when the network is slow or dead.
Adapter Address	Each IPv4 and IPv6 address with its prefix length (for example 192.0.2.20/24).	The address other machines use to reach this adapter.
Prefix Origin	Where the network part came from: DHCP (Dynamic Host Configuration Protocol: the router hands out the address), Manual, RouterAdv (IPv6 router advertisement), WellKnown (loopback, link-local).	Shows if an address was handed out or typed in.
Suffix Origin	Where the host part came from: DHCP, Manual, Link (built from the MAC), Random (privacy address), WellKnown.	"Random" means IPv6 privacy addresses are on.
DAD State	Duplicate Address Detection: Preferred (fine), Tentative (still checking), Duplicate (another device has it), Deprecated (being phased out).	"Duplicate" is a real address conflict.
DNS and Gateway	DNS servers for this adapter (from NetworkManager, else resolv.conf), then GW = default gateway (IPv4, then IPv6).	Only adapters with a default route show a GW.
Multicast Address	Multicast groups the adapter has joined, up to four (then "+N more").	Normal: 224.0.0.1 (all hosts), 224.0.0.251 (mDNS), 239.255.255.250 (SSDP/UPnP: Simple Service Discovery Protocol / Universal Plug and Play), ff02::1.

You can open one adapter on its own with `adapters:<name>`, for example `adapters:em1`.

Healthy / typical values. On the author's PC the 2.5 GbE wired port `em1` (Realtek RTL8126, driver r8169) shows `Ethernet (3)`, `2.5 Gbps`, `MTU 1500`, `up`, a DHCP address 192.0.2.20/24 in state Preferred, a link-local `fe80::XXXX/64` (WellKnown / Link), `DNS 192.0.2.1` and `GW 192.0.2.1`. Virtual adapters (VMware vmnet1/vmnet8, libvirt virbr0, Docker docker0, a veth pair, a TAP device) have Manual addresses and no gateway. Docker bridges and veth report a fake 10.0 Gbps. `lo` is `Loopback (1)` with 127.0.0.1/8 and ::1/128, MTU 65536.

Unusual values and what they suggest.

- Wired link at 100 Mbps or 10 Mbps on a gigabit card: bad cable (a broken pair), old switch port, or a power-saving setting.
- An address starting 169.254.: DHCP failed and Linux gave itself a fallback address.
- DAD State "Duplicate": two devices have the same IP. Check for a static address clashing with the router's DHCP range.

- Two adapters (wired and Wi-Fi) in the same subnet, both with a gateway: works, but the one with the lower route metric wins (see Routing table).
- Physical adapter state "down" with no cable plugged in is normal.

Where Linux gets it. `/sys/class/net/<if>/` (type, operstate, speed, mtu, address), `ip -o addr show`, `nmcli device show`, `/etc/resolv.conf`, `/proc/net/route`, `/proc/net/igmp` and `/proc/net/igmp6`.

Background. A *prefix length* (the "/24") says how many leading bits are the network part. /24 means the first three numbers (192.0.2) are the network and the last number picks the host. *Multicast* is one-to-many traffic: a device joins a group address and receives anything sent to it, which is how printers and media players announce themselves.

ARP table

`arp-table` - Linux data

In plain words. The list of nearby devices your PC has recently talked to on the local network, with their hardware (MAC) addresses. Below it, the services those devices announce over mDNS (printers, speakers, phones).

What you see.

Neighbour table:

Column	Meaning
IP Address	The neighbour's IPv4 or IPv6 address.
Physical Address	Its MAC address ("-" if not resolved).
HW Type	"Ethernet" for IPv4 ARP (Address Resolution Protocol) entries, "IPv6 NDP" (Neighbor Discovery Protocol) for IPv6.
Flags	IPv4: Dynamic (learned), Static (set by hand), Incomplete (asked, no answer yet). IPv6: the kernel state, such as REACHABLE, STALE, DELAY, PROBE, FAILED, PERMANENT.
Adapter	Which of your interfaces it was seen on.

mDNS / DNS-SD services (read from avahi's cache; SRS sends no new queries):

Column	Meaning
1	Your interface the announcement arrived on.
2	IPv4 or IPv6.
3	The service's name (for example a printer's name). Names keep avahi's escape codes, so `\032` is a space and `\064` is "@".
4	Service type, such as "Internet Printer", "AirPlay Remote Video", `_kdeconnect._udp`, `_adb._tcp`.
5	Domain, normally `local`.

Healthy / typical values. On the author's PC the router 192.0.2.1 appears as Dynamic on both `em1` and `wlan0`, plus a few other home devices. IPv6 entries show STALE (seen a while ago, still usable) and the odd FAILED. The mDNS list shows a smart TV ("AirPlay Remote Video"), a Lexmark printer and scanner (Internet Printer, `_uscan._tcp`), KDE Connect and a phone with ADB enabled. The same service usually appears once per interface.

Unusual values and what they suggest.

- The router's IP shows two different MAC addresses over time: possible ARP spoofing (someone pretending to be the router), or you swapped the router.
- Many Incomplete or FAILED entries: devices that went offline, or a scan of the subnet.
- "Neighbour table is empty" on a connected PC: you have not talked to anything yet, or the adapter is down.
- "avahi-daemon cache empty or avahi not running": avahi is off, so no mDNS list.

Where Linux gets it. `/proc/net/arp`, `ip -o -6 neigh show`, and `avahi-browse -a -t -p -c` (cache only).

Background. IP addresses are for routing; on the local cable the hardware needs a MAC address. ARP (IPv4) and NDP (IPv6) ask "who has 192.0.2.1?" and remember the answer for a short time. mDNS (multicast DNS) lets devices announce names and services on the LAN (local area network) without a DNS server.

Bluetooth radios

`bluetooth` - Linux data

In plain words. Your Bluetooth controller(s): what chip it is, which driver runs it, whether it is blocked, and the settings and services the Bluetooth daemon reports.

What you see. Per controller (`hci0`, `hci1`...), from sysfs:

- **Bus** - `usb` or `pci` plus the port. Most built-in Bluetooth is a USB device even on a Wi-Fi card.
- **Driver** - usually `btusb`.
- **USB ID** - vendor:product and revision (USB only).
- **USB device** - manufacturer and product strings.
- **rfkill** - the radio kill switch: soft-blocked (turned off in software) and hard-blocked (a physical switch or BIOS).

Then, from `bluetoothctl show`, one block per controller:

- **Controller** line - its Bluetooth address and type (public/random).
- **Manufacturer, Version** - Bluetooth SIG company ID and the core spec version code (for example 0x0c = Bluetooth 5.3).
- **Name, Alias** - the name other devices see.
- **Class** - the device class code (desktop computer, etc.).
- **Powered, PowerState** - is the radio on.
- **Discoverable, DiscoverableTimeout, Pairable** - can others find and pair with it (timeout 0 = no time limit).

- **Services (UUID)** - the profiles it offers (Audio Sink/Source, A/V Remote Control, Phonebook Access, etc.).
- **Modalias** - the ID of the Bluetooth stack (BlueZ).
- **Discovering** - is a scan running now.
- **Roles** - central (connects to devices) and/or peripheral (devices connect to it).
- **Advertising Features** block - Bluetooth Low Energy advertising limits: instances, what can be included, secondary PHYs (1M, 2M, Coded), min/max transmit power, max advert length.

Healthy / typical values. On the author's PC: `hci0`, Bus `usb 8-11:1.0`, Driver `btusb`, USB ID `13d3:3617`, "Realtek Bluetooth Radio" (the Bluetooth half of the RTL8922AE Wi-Fi 7 card), rfkill "soft-blocked no, hard-blocked no", Powered yes, Version 0x0c (12), roles central and peripheral, TX power -11 to 16 dBm.

Unusual values and what they suggest.

- soft-blocked yes: aeroplane mode or a desktop toggle turned Bluetooth off; `rfkill unblock bluetooth` turns it on.
- hard-blocked yes: a laptop switch or BIOS setting.
- "No Bluetooth controllers in /sys/class/bluetooth": no adapter, it is disabled in BIOS, or firmware failed to load (check `dmesg`).
- "bluetoothctl: no controllers reported": the hardware is there but `bluetoothd` (bluetooth.service) is not running.
- Discoverable yes all the time: anyone nearby can see the PC. Fine at home, less so in public.

Where Linux gets it. `/sys/class/bluetooth/hci*`, `bluetoothctl list` and `bluetoothctl show`.

Background. *rfkill* is the kernel's on/off switch for every radio (Wi-Fi, Bluetooth, mobile). A *UUID* here is a standard number for a Bluetooth profile; the short ones (like 110b = Audio Sink) are defined by the Bluetooth SIG.

Bluetooth IDs

`bluetooth-ids` - Linux data

In plain words. The ID codes of your Bluetooth controller and of every device you have paired.

What you see.

Column	Meaning
Device	`hci0` etc. for a controller, "paired device" for a known device.
Address	"(controller)", or the device's Bluetooth address.
Name	Device name.
Modalias / ID	For a controller, `usb:vVVVVpPPPPdDDDD` (vendor, product, device release). For a device, its `bluetooth:` or `usb:` modalias, if it sends one.
Class / Icon	The device class code and the icon name BlueZ picks (phone, audio-headset, input-mouse...).

A footer counts controllers and known devices.

Healthy / typical values. On the author's PC: `hci0` with `usb:v13D3p3617d0000`, and one paired phone with class `0x005a020c` and icon `phone`.

Unusual values and what they suggest. A device with Modalias "-" just did not send one (common for cheap devices). A long list of old devices you no longer own can be removed in your desktop's Bluetooth settings.

Where Linux gets it. `/sys/class/bluetooth/<hci>/device/modalias`, `bluetoothctl devices`, `bluetoothctl info <address>`. No discovery scan is run.

Background. A *modalias* is a short ID string the kernel uses to pick a driver automatically.

Bluetooth device tree

`blue-tree` - Linux data

In plain words. A simple tree: each controller, and under it the devices it knows, with their status.

What you see.

- Controller lines from `bluetoothctl list` (address, name, `[default]` for the default one).
- One branch per known device: name, address, icon (device type).
- Under a device, its status words: `paired`, `trusted`, and `CONNECTED` if connected right now.

Healthy / typical values. On the author's PC: controller "MYPC [default]" with one phone, "paired trusted" (not connected at that moment).

Unusual values and what they suggest. Paired but not trusted means the device has to be confirmed each time it connects. "bluetoothd not answering; devices unknown" means the controller exists but the Bluetooth service is stopped. "No known devices" means nothing was ever paired.

Where Linux gets it. `bluetoothctl list`, `bluetoothctl devices`, `bluetoothctl info`.

Network browse masters

`browsers` - Linux data

In plain words. Which machine on your LAN keeps the list of Windows-style computers ("Network Neighbourhood"), and how your own Samba is set up to take part.

What you see.

- **nmbd** - is Samba's NetBIOS name daemon running.
- **workgroup, local master, preferred master, domain master, os level** - your Samba settings for the browser election (os level 0 means "never try to win").
- A table of machines in nmbd's browse list that are **Master Browsers**: Name, Workgroup, Type (the server type bits: Workstation, Server, PrintQ, Unix, NT, NT Server, Potential/Backup/Master Browser, Domain Master, DC, Time, and others), Comment.

Healthy / typical values. On the author's PC nmbd is running, workgroup WORKGROUP, all master options No, os level 0. The master browser is the router (a GL.iNet GL-BE9300), with type "Workstation, Server, PrintQ, Unix, NT, NT Server, Master Browser, Domain Master".

Unusual values and what they suggest.

- "browse list not present - nmbd is not running": Samba's name service is off, so there is no list. Normal on PCs that do not share files.
- No master browser at all: Windows PCs may not see each other by name. Modern Windows uses WS-Discovery (Web Services Discovery) instead, so this matters less today.
- Two machines both claiming Master Browser: an election fight, often from two Samba boxes both set to "preferred master = yes".

Where Linux gets it. `testparm -s` for settings, `/var/cache/samba/browse.dat` (nmbd's cached list). SRS sends no NetBIOS queries.

Background. *NetBIOS browsing* is the old Windows way to list computers. One machine per workgroup wins an election and becomes the *master browser*; others ask it for the list.

TCP/IP counters

`counters` - Linux data

In plain words. Live traffic rates for every adapter, then the kernel's protocol counters for IP, ICMP, TCP and UDP since boot.

What you see.

Rates table (refreshes every second in the window):

Column	Meaning
Adapter	Interface name.
Rx /s, Tx /s	Bytes received / sent per second.
Rx pkt/s, Tx pkt/s	Packets per second.
Rx total, Tx total	Totals since the adapter came up.
State	up, down, unknown.

The footer says how long the measuring window was ("rates over the last 0.50 s"). On first open in the window it says "rates appear after the next refresh". In a saved report SRS takes a half-second sample so the rates are real.

Then the `/proc/net/snmp` tables. Each has a heading (Ip, Icmp, IcmpMsg, Tcp, Udp, UdpLite) and one counter per line. Zero counters are greyed. Key ones:

- **Ip**: Forwarding (1 = router mode on, 2 = off), DefaultTTL, InReceives, InHdrErrors, InDiscards, OutRequests, OutNoRoutes, fragment and reassembly counts.
- **Icmp / IcmpMsg**: ping and error messages in and out (InType0 = echo reply, InType3 = destination unreachable, InType8 = echo request).

- **Tcp**: ActiveOpens (connections you started), PassiveOpens (connections to your servers), AttemptFails, EstabResets, CurrEstab (open now), InSegs, OutSegs, RetransSegs (resent), InErrs, OutRsts.
- **Udp**: InDatagrams, NoPorts (sent to a port nobody listens on), InErrors, RcvbufErrors and SndbufErrors (buffer full).

Healthy / typical values. On the author's PC, idle: `em1` about 1.9 KB/s in and 1.2 KB/s out, 14.19 GB received since boot; `lo` about 1 KB/s each way. Tcp CurrEstab 19, RetransSegs 2,142 out of 3.8 million OutSegs (well under 1%). Udp RcvbufErrors 0.

Unusual values and what they suggest.

- RetransSegs above about 1-2% of OutSegs: packet loss somewhere (Wi-Fi interference, bad cable, overloaded router).
- InCsumErrors or InHdrErrors rising: damaged packets, often a bad cable or failing NIC.
- Udp RcvbufErrors rising: a program cannot read fast enough (game servers, streaming); the receive buffer is too small.
- Big AttemptFails compared with ActiveOpens: many connections refused or timed out, for example a blocked port or a dead server.
- Ip Forwarding 1 on a desktop: normal if you run Docker or VMs, otherwise check why.

Where Linux gets it. `/sys/class/net/<if>/statistics/` sampled twice, and `/proc/net/snmp`.

Background. *TCP* (Transmission Control Protocol) is the reliable, ordered connection used by web, SSH and most apps; lost pieces are resent. *UDP* (User Datagram Protocol) just fires packets with no resend, used by DNS, games and voice. *ICMP* (Internet Control Message Protocol) carries ping and error replies.

DNS cache

`dns-cache` - Linux data, Needs root data

In plain words. Is there a local DNS cache on this PC, and what is in it.

What you see.

- One line per known caching program that is running, with its process ID: systemd-resolved, dnsmasq, nscd, unbound, named (BIND), kresd, dnscrypt-proxy, stubby, pdns_recursor, AdGuardHome. "none running" if none.
- If systemd-resolved runs: the cache contents from `resolvectl show-cache`. That command is root-only, so without srs-root you get "Needs root: Menu > Run as root (srs-root)".
- If systemd-resolved does not run: a note that glibc itself does not cache lookups, and that dnsmasq copies are often the ones libvirt or containers start for their own bridges.

Healthy / typical values. On the author's PC, `dnsmasq running (pid ...)` and systemd-resolved not running. Here dnsmasq is a real system cache: `/etc/resolv.conf` points at 127.0.0.1, where dnsmasq listens (see DNS ports). SRS lists cache contents only for systemd-resolved, so the dnsmasq cache itself is not shown.

Unusual values and what they suggest. Two different caches running at once (for example systemd-resolved and dnsmasq both on port 53) can fight; one usually fails to start. No cache at all is fine; lookups just go straight to your router or DNS provider each time.

Where Linux gets it. Process names in `/proc/*/comm`; `resolvectl show-cache` (root).

Background. A *DNS cache* remembers answers ("example.com is at ...") for their time-to-live, so repeated lookups are instant and work for a while even if the upstream server is slow.

DNS names

`dns-names` - Linux data

In plain words. All the names this computer and your user account go by.

What you see.

Row	Meaning
NetBIOS Name	Windows-network name (from Samba, or the host name in capitals, max 15 characters).
DNS Domain	The PC's DNS domain, often empty at home.
DNS Host Name	The host name.
Full DNS Name	Host name plus domain (the FQDN, fully qualified domain name).

The two columns, Computer Name and Computer Physical Name, are the same on Linux (Windows can differ on clusters).

Then the user block:

- **Display Username** - your login name.
- **Legacy Account Name** - `NETBIOSNAME\user`, the Windows-style form.
- **Full Name (GECOS)** - the real name stored in `/etc/passwd` (only if set).
- **User ID** - your numeric UID.

Then `/etc/hostname`, and from `hostnamectl`: StaticHostname, PrettyHostname, Chassis (desktop, laptop, vm...), Deployment and Location, when set.

Healthy / typical values. On the author's PC: NetBIOS Name MYPC, no DNS domain, Full DNS Name MYPC, user `user`, legacy name `MYPC\user`, User ID 1000, Chassis desktop.

Unusual values and what they suggest. `/etc/hostname` different from DNS Host Name: the name was changed without a reboot, or something (DHCP, a VM tool) set a temporary name. A host name longer than 15 characters is cut short in the NetBIOS name, which can confuse Windows shares.

Where Linux gets it. `gethostname()`, `getdomainname()`, `/etc/resolv.conf`, `testparm` (netbios name), `/etc/passwd`, `/etc/hostname`, `hostnamectl --json=short`.

DNS ports

`dns-ports` - Linux data

In plain words. Every socket on a DNS-related port: local DNS servers listening, and your open connections to DNS servers.

What you see.

Column	Meaning
Proto	UDP, UDP6, TCP, TCP6.
Address	The local address (for a listener) or the remote server address.
Port	53 (DNS), 853 (DNS over TLS), 5353 (mDNS), 5355 (LLMNR, Link-Local Multicast Name Resolution).
Service	Name from `/etc/services` (domain, mdns...).
State	Listen, Local, or "To server" (an outgoing query).
Process (PID)	The program, if SRS can see it. Programs of other users (including root) show "-" unless you run srs-root.

Below that: the DNS servers from `/etc/resolv.conf` (each on port 53), resolver `options`, and whether systemd-resolved is running (its stub listens on 127.0.0.53).

Healthy / typical values. On the author's PC dnsmasq listens on 127.0.0.1:53 (UDP and TCP) and on the libvirt and retro-VM bridge addresses; avahi listens on 0.0.0.0:5353 (mDNS). resolv.conf lists 127.0.0.1 with `options timeout:2 attempts:3`.

Unusual values and what they suggest.

- Something listening on port 53 on your LAN address (not 127.x or a VM bridge): your PC is acting as a DNS server for the network. Fine if intended.
- resolv.conf points at 127.0.0.53 but systemd-resolved is "not running": DNS will fail.
- Nothing listening on 127.0.0.1:53 while resolv.conf says 127.0.0.1: your local cache died, so names stop resolving.

Where Linux gets it. `/proc/net/udp`, `udp6`, `tcp`, `tcp6`, process sockets under `/proc/*/fd`, `/etc/resolv.conf`.

DNS usage

`dns-usage` - Linux data

In plain words. How name lookups are set up on this PC, or, with systemd-resolved, its lookup statistics.

What you see. If systemd-resolved runs, the output of `resolvectl statistics`: transactions, cache size, cache hits and misses, DNSSEC verdicts. Otherwise a note, then:

- **Name servers** - how many are in resolv.conf.

- **Search list** - domains tried after short names.
- **Resolver options** - resolv.conf options (timeout, attempts, edns0...).
- **NSS hosts order** - the `hosts:` line of `/etc/nsswitch.conf`, the order Linux tries: files (`/etc/hosts`), mdns, resolve, dns, myhostname, wins...
- **/etc/hosts entries** - number of non-comment lines.
- **/etc/gai.conf** - present, or absent (default IPv6-first rules).

Healthy / typical values. On the author's PC: 1 name server, no search list, `timeout:2 attempts:3`, NSS order `mymachines mdns_minimal resolve files myhostname dns wins`, 9 hosts entries, gai.conf present.

Unusual values and what they suggest. A huge `/etc/hosts` (thousands of lines from an ad blocker) slows every lookup because it is read top to bottom; a local DNS cache is faster for that. With resolved statistics, a low cache-hit rate is normal for browsing many new sites; many DNSSEC "bogus" verdicts suggest a broken upstream server or a clock that is far off.

Where Linux gets it. `resolvectl statistics`, `/etc/resolv.conf`, `/etc/nsswitch.conf`, `/etc/hosts`, `/etc/gai.conf`.

Background. *NSS* (Name Service Switch) is glibc's plug-in list that decides where names are looked up, and in what order.

Domains / workgroups

`domains` - Linux data

In plain words. Which Windows workgroup or domain, Kerberos realm and DNS domain this PC belongs to, plus the workgroups seen on your LAN.

What you see.

- **SMB workgroup** - Samba's workgroup (WORKGROUP is the Windows default).
- **Kerberos realm** - for Active Directory joins.
- **Samba security** - USER (normal standalone), ADS (Active Directory member), etc.
- **NIS domain** - old Unix network login domain, usually "-".
- **DNS domain, DNS search list** - from resolv.conf.
- **AD/realmd join** - "keytab present" if `/etc/krb5.keytab` exists (joined to a domain), else "not joined".
- A table of domains/workgroups in nmbd's browse list: name, workgroup, type, comment (usually the master browser's name).

Healthy / typical values. On the author's PC: workgroup WORKGROUP, security USER, not joined; the LAN list shows WORKGROUP with type "NT, Domain Enum", led by the router.

Unusual values and what they suggest. A workgroup name different from your other PCs means they will not list each other in the old browse view. "keytab present" on a home PC is unusual and means it was once joined to a company domain.

Where Linux gets it. `testparm` (workgroup, realm, security), `getdomainname()`, `/etc/resolv.conf`, `/etc/krb5.keytab`, `/var/cache/samba/browse.dat`.

Firewall

`firewall` - Linux data, Needs root data

In plain words. Is the firewall running, and what each active zone allows in.

What you see.

- **firewalld state** - running, not running, or "not installed".
- **Panic mode** - "yes" means all traffic is being blocked.
- **Default zone** - the zone new interfaces fall into.
- One block per active zone (the heading says "(active)" and "(default)" where true). Each line is a zone setting that has a value: target (ACCEPT, default, %%REJECT%%, DROP), ingress/egress priority, icmp-block-inversion, interfaces, sources, services, ports, protocols, forward, masquerade, forward-ports, rich rules.
- **nftables ruleset** - the raw kernel rules need root (`nft list ruleset`), so this line points you to srs-root.

Healthy / typical values. On the author's PC firewalld is running, panic mode no, default zone `public` (interfaces em1 and wlan0, services dhcpv6-client, kdeconnect, ssh). Extra zones exist for Docker (target ACCEPT), libvirt (dhcp, dns, ssh, tftp), a retro-VM bridge (target %%REJECT%% with only NFS (Network File System), Samba and AFP (Apple Filing Protocol) allowed) and a trusted zone for Waydroid.

Unusual values and what they suggest.

- Your LAN interfaces in zone `trusted` or a zone with target ACCEPT: everything is allowed in. Fine for VM bridges, risky for the adapter that faces the internet or a public Wi-Fi.
- Services open that you do not use (ssh, samba, cockpit): close them to reduce what others can reach.
- "not installed": you may use plain nftables, ufw, or no firewall at all. Check with root.
- Panic mode yes: someone ran `firewall-cmd --panic-on`; the network will look dead.

Where Linux gets it. `firewall-cmd --state`, `--query-panic`, `--list-all-zones` (read-only queries); `nft list ruleset` needs root.

Background. *firewalld* groups rules into *zones*. Each network interface belongs to one zone, and the zone decides which incoming services are allowed. *Masquerade* is NAT (network address translation): sharing one address for many devices behind it.

Network interface statistics

`interface` - Linux data

In plain words. The kernel's error and traffic counters for every adapter since it came up.

What you see.

Column	Meaning
Interface	Name.
Rx Bytes / Rx Packets	Received data and packets.
Rx Err	Received frames with errors (bad checksum, wrong length).
Rx Drop	Received but thrown away (no room in the queue, or unwanted traffic).
Tx Bytes / Tx Packets	Sent.
Tx Err / Tx Drop	Failed or dropped sends.
Multicast	Multicast packets received.
Collisions	Half-duplex collisions; should be 0 on modern full-duplex links.
Carrier chg	How many times the link went up or down.

Healthy / typical values. On the author's PC `em1` shows 14.19 GB and 11.8 million packets in, 0 errors, 0 drops, Carrier chg 2 (link came up at boot). Bridges with no members show some Tx Drop (711 on one bridge), which is harmless: packets sent to an empty bridge go nowhere.

Unusual values and what they suggest.

- Rx Err rising on a wired port: bad cable, bad connector, or electrical noise. Try another cable or port.
- Collisions above 0: the link fell back to half duplex; usually an auto-negotiation problem with an old switch.
- Carrier chg climbing while you watch: the link keeps dropping. Loose cable, power-saving (EEE, Energy-Efficient Ethernet), or a flaky driver.
- Rx Drop on a busy server: the receive ring is too small or the CPU is too slow to keep up.

Where Linux gets it. `/sys/class/net/<if>/statistics/*` and `carrier_changes` (the same numbers as `/proc/net/dev` and `ip -s link`).

IP address

`ip-address` - Linux data

In plain words. Every IP address on every adapter, with its scope, flags and lifetime.

What you see.

Column	Meaning
Adapter	Interface name ("(no addresses)" if it has none).
Address	IPv4 or IPv6 address.
Prefix	Prefix length, such as /24 or /64.
Scope	host (only this PC, like 127.0.0.1), link (only this cable/Wi-Fi, like fe80::), global (routable).

Column	Meaning
Flags	dynamic (from DHCP or SLAAC, stateless address autoconfiguration, where IPv6 builds its own address from router adverts), noprefixroute, temporary (privacy address), stable-privacy, tentative, deprecated, dadfailed, mngtmpaddr... "-" if none.
Lifetime (valid/ preferred)	Seconds until the lease runs out (valid) and until it stops being used for new connections (preferred), or "forever".

Open one adapter with `ip-address:<name>`.

Healthy / typical values. On the author's PC `em1` has 192.0.2.20 /24 global `dynamic noprefixroute` with about 37,900 seconds left (a 12-hour DHCP lease), and a link-local `fe80::XXXX /64` lasting forever. VM bridges have static addresses lasting forever.

Unusual values and what they suggest. A lifetime counting down to near 0 without renewing means DHCP renewals are failing. `dadfailed` means address conflict. No global IPv6 address while your provider supports IPv6: the router is not sending router advertisements, or IPv6 is off.

Where Linux gets it. `ip -o addr show`.

Background. A *DHCP lease* is an address the router lends you for a set time. Your PC renews it halfway through. *Link-local* addresses (fe80:: for IPv6, 169.254. for IPv4) only work on the local segment and are made automatically.

LAN Manager share setup

`lm-setup` - Linux data

In plain words. Your Samba (Windows file sharing) configuration, as Samba itself reads it.

What you see. The effective `smb.conf` from `testparm -s`, section by section:

- **[global]** - server-wide settings: workgroup, server string, security, interfaces and bind interfaces only, hosts allow/deny, min/max protocol, map to guest, logging, printing, usershare limits, wins support, idmap.
- **One section per share** ([homes], [printers], and your own shares) - path, comment, read only, guest ok, guest only, force user, valid users, create mask, directory mask, inherit acls, printable.

Only settings that differ from the defaults are printed. Values of any setting whose name contains TOKEN, KEY, SECRET, PASS, AUTH, COOKIE or PSK are hidden as `****` (except plain Yes/No).

Healthy / typical values. On the author's PC: security USER, standalone server, minimum protocol SMB2 for client and server, Samba bound only to `lo` and the home subnet, `hosts allow` limited to the LAN and 127.0.0.1, `hosts deny = 0.0.0.0/0`. Shares include [homes], several file shares and a printer.

Unusual values and what they suggest.

- `guest ok = Yes` with `read only = No`: anyone on the network can write there without a password. Only safe on a trusted LAN with `hosts allow` set.

- A share with `path = /`: the whole disk is shared. Limit with `valid users` or remove it if not needed.
- `server min protocol = NT1` (SMB1): very old and insecure protocol; only enable for retro machines that need it.
- "Samba is not installed or smb.conf is unreadable": no Samba, nothing shared this way.

Where Linux gets it. `testparm -s --suppress-prompt` (reads `/etc/samba/smb.conf`).

Background. *SMB* (Server Message Block) is the Windows file-sharing protocol; *Samba* is its Linux implementation. "LAN Manager" is the old name from the 1980s Microsoft/IBM network software.

MAC addresses

`mac-address` - Linux data

In plain words. The hardware address of every adapter, whether it is the factory one, and which company made the chip.

What you see.

Column	Meaning
Adapter	Interface name.
MAC Address	The address in use now.
Permanent	The factory address burned into the card (physical adapters only, from `ethtool -P`), "-" for virtual ones.
Assigned	How the current address was set: permanent (factory), random (made up at start), stolen (copied from another device, as bonds do), set by user (changed by software or NetworkManager).
Broadcast	The link broadcast address (ff:ff:ff:ff:ff:ff on Ethernet and Wi-Fi).
Vendor (OUI)	Maker looked up from the first half of the address, or "(locally administered)" for made-up addresses.

Healthy / typical values. On the author's PC the wired port is "permanent" with vendor ASUSTek (the motherboard maker), the Wi-Fi card shows AzureWave Technology (the module maker), the USB adapter shows Realtek, VMware adapters show VMware, and bridges, TAP, Docker and veth are "(locally administered)" with "set by user". `lo` is all zeros.

Unusual values and what they suggest.

- Wi-Fi with Assigned "random" or "set by user" and a current address different from Permanent: MAC randomisation is on (a privacy feature). Normal, but your router will see a "new device" per network.
- Two adapters with the same MAC on one network: they will fight; usually a cloned VM.
- A vendor that does not match the card you bought: the card was rebranded, or the address was changed.

Where Linux gets it. `/sys/class/net/<if>/address`, `addr_assign_type`, `broadcast`; `ethtool -P`; vendor from `/usr/share/hwdata/oui.txt`.

Background. A *MAC address* (Media Access Control) is the 6-byte hardware address used on the local network. The first three bytes are the *OUI* (Organizationally Unique Identifier), bought by the maker. If the second-lowest bit of the first byte is set, the address is *locally administered*: made up by software, not by a factory.

Modem API

`modem-api` - Not present on most PCs

In plain words. On Windows this lists the telephony (TAPI) modem interface. Linux has no such API; SRS shows what ModemManager knows instead. The page title reads "Modems" because it shares its output with the next item.

What you see. Either the `mmcli -L` list of modems (mobile broadband sticks, built-in LTE/5G modules), or "ModemManager reports no modems", or "ModemManager (mmcli) not available".

Healthy / typical values. On the author's PC: "ModemManager reports no modems". This is normal for any desktop without a mobile data module.

Where Linux gets it. `mmcli -L` (ModemManager).

Modems

`modems` - Not present on most PCs

In plain words. Mobile broadband modems (4G/5G USB sticks, built-in WWAN (wireless wide area network) cards) known to ModemManager.

What you see. Each modem line from `mmcli -L` (its path, maker and model), or "ModemManager reports no modems" / "ModemManager (mmcli) not available". Old serial dial-up modems are not listed; Linux has no general way to enumerate those.

Healthy / typical values. "ModemManager reports no modems" on a desktop. That just means there is no mobile modem, not that anything is wrong.

Unusual values and what they suggest. A laptop with a WWAN card that shows no modem: the card may be disabled in BIOS, rfkill-blocked, or need a firmware unlock (FCC lock) before ModemManager will use it.

Where Linux gets it. `mmcli -L` (ModemManager).

My browser / IP location

`my-browser` - Linux data

In plain words. Which web browser opens your links. The Windows version also looks up your public IP's location online; SRS does not, because it makes no internet requests.

What you see.

- **Default web browser** - from `xdg-settings` (a `.desktop` file name).
- **https handler** - the app registered for `https:` links (`xdg-mime`).
- **$BROWSER** - the BROWSER environment variable, used by many terminal programs.
- **IP geolocation** - always "not looked up - srs makes no internet requests".

Healthy / typical values. On the author's PC the default browser and https handler are the same Floorp `.desktop` file, while `$BROWSER` is set to a different browser (firedragon).

Unusual values and what they suggest. Default browser and https handler disagree: some apps open links in one browser and some in the other. Fix it in your desktop's Default Applications settings. A `$BROWSER` that differs, as here, only affects programs that read that variable.

Where Linux gets it. `xdg-settings get default-web-browser`, `xdg-mime query default x-scheme-handler/https`, the environment.

My IP address

`my-ip-address` - Linux data

In plain words. The address your PC uses when it talks to the outside world, and all your routable addresses.

What you see.

- **Outgoing IPv4 source** - the address the kernel would use for a packet to the internet. SRS finds this with a route lookup for the documentation address 192.0.2.1; nothing is sent.
- **Default gateway** - your router address and the adapter it is reached through.
- **Public (WAN) address** - "not looked up - srs makes no internet requests". Your router's status page or any "what is my IP" site shows it. WAN (wide area network) means the internet side of your router.
- **Global-scope addresses** - every global address per adapter, with prefix and flags.

Healthy / typical values. On the author's PC: outgoing source 192.0.2.20 (the wired port), gateway `192.0.2.1 dev em1`, and global addresses on em1, wlan0 and the VM/Docker bridges.

Unusual values and what they suggest. Outgoing source is the Wi-Fi address while a cable is plugged in: the Wi-Fi route has the better metric, so traffic goes over the slower link. Outgoing source on a VPN (virtual private network) interface (tun0, wg0): all traffic goes through the VPN, as intended for a full-tunnel VPN. "-" means no route to the internet.

Where Linux gets it. `ip -o route get 192.0.2.1`, `/proc/net/route`, `ip -o addr show`.

Background. At home your PC has a *private* address (192.168.x.x, 10.x, 172.16-31.x). The router swaps it for your one *public* address on the way out (NAT), so websites never see the private one.

Network cards

`net-cards` - Linux data

In plain words. The physical network hardware: chip name, bus slot, IDs, driver, firmware and link details. Virtual adapters are left out (see Network adapters).

What you see. One block per physical adapter:

- **Description** - PCI device name, or USB maker and product.
- **Bus** - `pci` with the slot address, or `usb` with the port path.
- **PCI ID** - vendor:device and subsystem vendor:device (PCI cards). The subsystem tells you the board maker.
- **PCIe link** - lane count and speed the card is running at now.
- **USB ID** - vendor:product and the USB link speed in Mbps (USB adapters).
- **Driver**, **Driver version**, **Firmware**, **Expansion ROM** (if the card has a boot ROM) - from `ethtool -i`.
- **MAC address**.
- **Link** - state; when up, also speed and duplex.
- **MTU** - with the transmit queue length.
- **Wireless PHY** - for Wi-Fi cards, the radio name (phy0...).

Open one card with `net-cards:<name>`.

Healthy / typical values. On the author's PC:

- `em1`: Realtek RTL8126 5GbE, PCI ID 10ec:8126, subsystem 1043 (ASUS), PCIe link x1 8.0 GT/s, driver r8169, firmware rtl8126a-3, link up 2.5 Gbps full duplex, MTU 1500.
- `wlan0`: Realtek RTL8922AE Wi-Fi 7, PCIe x1 5.0 GT/s, driver rtw89_8922ae, firmware 0.35.80.3, Wireless PHY phy0.
- `enp115s0u2u4u4`: a Realtek USB gigabit adapter (0bda:8153, driver r8152) on a 5000 Mbps USB port, link down.

Unusual values and what they suggest.

- A 5 GbE card linked at 2.5 Gbps: the switch or router port is only 2.5 GbE (as here), or the cable is Cat5. Not a fault.
- PCIe link slower than the card supports (for example 2.5 GT/s on a 10 GbE card): slot shares lanes or is set to an older PCIe generation in BIOS. Can cap throughput on fast cards.
- A USB adapter on a 480 Mbps (USB 2) port: a gigabit adapter will top out around 300 Mbps. Move it to a blue/USB 3 port.
- Half duplex on a modern link: auto-negotiation failed. Check the cable and switch.
- Firmware missing or very old: some features (and bug fixes) need the `linux-firmware` package updated.

Where Linux gets it. `/sys/class/net/<if>/device/` (vendor, device, subsystem IDs, current_link_width, current_link_speed), `lspci -nn`, `ethtool -i`.

Background. *Duplex* is whether both ends can send at the same time (full) or must take turns (half). *MTU* (Maximum Transmission Unit) is the biggest packet the link carries, 1500 bytes on normal Ethernet.

Network provider names

`net-names` - Windows-only (Linux equivalent shown)

In plain words. On Windows this lists "network providers" (the plug-ins that let Explorer open `\\server\share` paths). Linux has no such list, so SRS shows the network file-system tools installed and the network shares mounted right now.

What you see.

- **Network providers available** - which client tools exist: Samba client (smbclient), CIFS kernel mounts (mount.cifs), NFS client, SSHFS, GVfs SMB, KIO (KDE's smb:/ sftp:/ fish:/), CUPS network printing (lpstat). Each with its path.
- **Mounted network file systems** - source, mount point, and type for every cifs, smb3, nfs, nfs4, fuse.sshfs, 9p, davfs, fuse.rclone, fuse.gvfsd-fuse, ceph or glusterfs mount. Any user:password in a source is masked.

Healthy / typical values. On the author's PC all seven tools are present, and three network mounts are active: an NFS4 share, GVfs's FUSE mount at `/run/user/1000/gvfs`, and a CIFS mount of the router's USB disk.

Unusual values and what they suggest. A network mount you did not expect: something (fstab, a file manager, an automounter) mounted it. A mount whose server is gone can make file managers and `df` hang. "none mounted" is normal.

Where Linux gets it. Tool paths on disk; `/proc/self/mounts`.

Network

`network` - Linux data

In plain words. The one-page summary of your networking: host name, adapters, gateway, DNS, forwarding and TCP tuning, then one line per adapter.

What you see.

- **Host name**.
- **Adapters** - total, how many physical and virtual, and how many are up (state "up" or "unknown"; loopback and some virtual adapters always report "unknown").
- **IPv4 default gateway** - router address and adapter.
- **IPv6 default route** - or "none".

- **DNS servers** - from `/etc/resolv.conf`.
- **DNS search** - if set.
- **IP forwarding** - on/off for IPv4 and IPv6.
- **TCP congestion control** - the algorithm that decides how fast TCP sends (cubic, bbr, bbr3...).
- **Default qdisc** - the default queueing discipline (fq, fq_codel, pfifo_fast...).

Then a table: Adapter, Type, State, Speed, MAC Address, IPv4 (all addresses with prefix), Driver. The last line names the motherboard maker, model, BIOS version and BIOS date (for example "ASUSTeK COMPUTER INC. TUF GAMING X870-PRO WIFI7 W NEO BIOS 1805").

Healthy / typical values. On the author's PC: host MYPC, "11 (3 physical, 8 virtual), 7 up", gateway `192.0.2.1 dev em1`, no IPv6 default route, DNS 127.0.0.1 (a local dnsmasq), IPv4 forwarding on (needed by Docker and VMs), TCP congestion control `bbr3`, default qdisc `fq`.

Unusual values and what they suggest.

- IPv4 default gateway "none": no internet route. Check the cable/Wi-Fi and DHCP.
- IPv6 default route "none" while your provider offers IPv6: router advertisements are not arriving or IPv6 is disabled on the router.
- DNS servers "none": names will not resolve at all.
- BBR with a qdisc other than fq on older kernels: BBR worked best with fq pacing; modern kernels pace internally, so this matters less now.

Where Linux gets it. `/sys/class/net`, `ip -o addr show`, `/proc/net/route`, `ip -o -6 route show default`, `/etc/resolv.conf`, `/proc/sys/net/ipv4/ip_forward`, `/proc/sys/net/ipv6/conf/all/forwarding`, `/proc/sys/net/ipv4/tcp_congestion_control`, `/proc/sys/net/core/default_qdisc`.

Background. *Congestion control* is how TCP guesses the safe sending speed. Classic algorithms (cubic) slow down when packets are lost; *BBR* measures bandwidth and delay instead and often does better on lossy or long links. A *qdisc* (queueing discipline) is the kernel's waiting line for outgoing packets; `fq` (fair queue) shares the link fairly between connections.

Network profiles

`profiles` - Linux data

In plain words. Every connection NetworkManager has saved: wired, Wi-Fi, VPN, bridges, Bluetooth tethering.

What you see.

Column	Meaning
Profile	The connection name. For Wi-Fi this is usually the network name.
Type	802-3-ethernet (wired), 802-11-wireless (Wi-Fi), vpn, wireguard, bridge, tun, loopback, bluetooth, gsm...
Device	The adapter using it now, or "-" if not active.
Auto	Does it connect automatically.

Column	Meaning
Last used	When it was last activated, or "never".
UUID	The profile's unique ID, used by `nmcli connection up <uuid>`.

Healthy / typical values. On the author's PC: "Wired connection 3" on em1, the home Wi-Fi profile on wlan0, profiles for lo and two bridges and a TAP device, plus inactive ones: older wired profiles, two other Wi-Fi networks, a VPN profile, and a Bluetooth phone-tethering profile.

Unusual values and what they suggest. Several "Wired connection N" profiles for one port: NetworkManager made a new one each time the adapter or its MAC changed; the old ones can be deleted. A Wi-Fi profile with Auto yes for a public network you no longer visit: your PC will join it whenever it sees that name. Remove it.

Where Linux gets it. `nmcli -t connection show`.

Network protocols

`protocols` - Linux data

In plain words. The socket types the kernel has loaded right now, how many sockets of each are open, and how much memory they use.

What you see.

Column	Meaning
Protocol	TCP, UDP, TCPv6, UDPv6, UNIX, UNIX-STREAM, NETLINK, PACKET, RAW, PING, MPTCP, XDP, MCTP, Bluetooth ones (HCI, L2CAP, SCO, RFCOMM, BNEP), ALG (kernel crypto), and others.
Size	Bytes per socket structure in the kernel.
Sockets	Number open now.
Memory	Memory pages used by this protocol's buffers (-1 = not tracked).
Press	Memory pressure: "no" (fine), "yes" (the kernel is limiting buffers), "NI" (not implemented for this protocol).
Module	Kernel module that provides it ("kernel" if built in).

A footer counts the protocols and the IP protocol numbers defined in `/etc/protocols`.

Healthy / typical values. On the author's PC 22 protocols: about 1,500 UNIX-STREAM sockets (normal for a desktop; apps talk to each other this way), 50 TCP, 53 UDP, 88 NETLINK, and Press "no" for TCP.

Unusual values and what they suggest. Press "yes" for TCP: the system is short of socket memory, often a server with very many connections; raise `net.ipv4.tcp_mem`. Thousands of TCP sockets on a desktop: a program is leaking connections.

Where Linux gets it. `/proc/net/protocols`; `/etc/protocols`.

Background. A *socket* is a program's handle for one network conversation (or local channel). *Netlink* sockets are how tools like `ip` talk to the kernel; *UNIX* sockets are local-only pipes between programs.

Proxy config

`proxy` - Linux data

In plain words. Whether apps are told to send web traffic through a proxy server, and where that setting comes from.

What you see. Three or four blocks:

- **Environment (this process)** - http_proxy, https_proxy, ftp_proxy, all_proxy, no_proxy (both lower and upper case), or "none set (direct connection)".
- **/etc/environment** - any proxy lines there, or "none".
- **KDE (kioslaverc [Proxy Settings])** - ProxyType (No proxy, Manual, PAC script, WPAD auto-detect, System environment) and the other keys in that section, such as httpProxy and NoProxyFor. Secret-looking keys are hidden.
- **GNOME proxy mode** - from gsettings, if available ('none', 'manual', 'auto').

Any `user:password@` in a proxy URL is shown as `****`.

Healthy / typical values. On the author's PC: no environment proxy, none in /etc/environment, KDE ProxyType "No proxy" (an old httpProxy value of `http://127.0.0.1:9099` is still stored but unused), GNOME mode 'none'.

Unusual values and what they suggest. A proxy set that you did not configure: some VPN or "privacy" tools, and some malware, set one to route your traffic. A proxy in KDE but not in the environment means KDE apps use it while terminal tools and many other apps do not.

Where Linux gets it. The process environment, `/etc/environment`, `~/.config/kioslaverc`, `gsettings get org.gnome.system.proxy mode`.

Background. A *proxy* is a middle server that makes web requests for you. A *PAC* (proxy auto-config) script picks the proxy per address; *WPAD* finds that script automatically.

Routing table

`routing` - Linux data

In plain words. The kernel's map of "to reach this network, go this way".

What you see. First, whether IPv4 routing (forwarding between adapters) is Enabled or Disabled. Then the IPv4 table:

Column	Meaning
Destination	Network address, or "default" for everything else.

Column	Meaning
Netmask	Which part is the network (255.255.255.0 = /24).
Gateway	Next router to hand packets to, or "on-link" (the destination is directly on that adapter).
Flags	U = route is up, G = uses a gateway, H = route to a single host, ! = reject route.
Metric	Cost; when two routes match, the lower metric wins.
MTU	Per-route MTU, "-" if not set.
Adapter	Interface used.

Then **IPv6 routes**: each line of `ip -6 route show table all` as text. Normal routes are coloured; `local` and `multicast` housekeeping routes are greyed.

Healthy / typical values. On the author's PC: two default routes via 192.0.2.1, one on em1 with metric 100 and one on wlan0 with metric 600, so the wired port wins. On-link routes for the home subnet and each VM/Docker bridge. IPv6 shows only link-local `fe80::/64` routes (no IPv6 internet).

Unusual values and what they suggest.

- No "default" line: no internet access, only local networks.
- Two default routes with the same metric: traffic may flip between adapters.
- A default route on a VPN adapter (tun0, wg0): all traffic goes through the VPN.
- Two routes for the same subnet on different adapters (for example a VM bridge reusing your home subnet): local traffic can go to the wrong place.

Where Linux gets it. `/proc/net/route`, `/proc/sys/net/ipv4/ip_forward`, `ip -o -6 route show table all`.

Background. A *gateway* is the router that forwards your packets to other networks. A *metric* is a priority number: NetworkManager gives wired links 100 and Wi-Fi 600 so cables are preferred.

Servers

`servers` - Linux data

In plain words. Every service on this PC that is waiting for incoming connections, and the SMB (Windows sharing) servers seen on your LAN.

What you see.

Local listening services:

Column	Meaning
Proto	TCP, TCP6, UDP, UDP6.
Bound Address	0.0.0.0 or `::` = all addresses (reachable from the network, unless the firewall blocks it); 127.0.0.1 or `::1` = this PC only (greyed); a specific address = only that adapter.
Port	Port number.
Service	Standard name from `/etc/services`, blank if none.

Column	Meaning
Process (PID)	The program, if visible. Other users' programs show "-" unless you run srs-root.

SMB servers seen on the LAN: Name, Workgroup, Type and Comment from nmbd's browse list (see Network browse masters).

Healthy / typical values. On the author's PC, a fairly busy home server: ssh (22), dnsmasq (53), rpcbind (111), Samba (139, 445), AFP (548) for retro Macs, CUPS printing (631), NFS (2049, mountd 20048), WS-Discovery (5357), syncthing on 127.0.0.1:8384, and a few local helpers. The router shows up as an SMB server.

Unusual values and what they suggest.

- A service on 0.0.0.0 you did not know about: it is reachable from your network. Check what it is and whether the firewall blocks it.
- A database or admin panel on 0.0.0.0 instead of 127.0.0.1: often a misconfiguration.
- Note that `/etc/services` names are just the registered meaning of the number, not the real program. Port 2049 shows as "shilp" but is NFS, and 8080 is labelled "http-alt" whatever runs there.

Where Linux gets it. `/proc/net/tcp`, `tcp6`, `udp`, `udp6` (listening sockets), `/proc/*/fd` for owners, `/var/cache/samba/browse.dat`.

Background. A *port* is a number (0-65535) that picks which program on a machine gets a packet. A *listening* socket is a program waiting for others to connect.

TCP V4 ports

`tcp-ip4-ports` - Linux data

In plain words. Every IPv4 TCP socket: servers waiting for connections and every open connection, with the program that owns it.

What you see.

Column	Meaning
Local Address / Port / Service	Your side of the socket.
Remote Address / Port / Service	The other side; `0.0.0.0 *` for a listener.
State	Listen, Established, SYN Sent, SYN Received, FIN Wait 1, FIN Wait 2, Time Wait, Closed, Close Wait, Last ACK, Closing, New SYN Recv.
User	Account that owns the socket.
Process (PID)	Program and process ID; "-" for other users' programs without root; "(kernel/time-wait)" for sockets no program owns any more.

Listeners are listed first, sorted by port. The footer counts sockets, listening and other.

Healthy / typical values. On the author's PC 51 sockets: 38 listening, 13 other. For example `127.0.0.1 8384 ... Listen user syncthing (18116)` and outgoing `https` (443) connections in state Established, plus a few in Time Wait.

Unusual values and what they suggest.

- Many Close Wait sockets for one program: that program is not closing connections (a bug); it will slowly run out of sockets.
- Hundreds of Time Wait: normal on a busy web client or server; they clear after about a minute.
- Many SYN Received from different addresses: someone is flooding your server (SYN flood).
- An Established connection to an address you do not recognise, owned by an unknown program: worth looking up.

Where Linux gets it. `/proc/net/tcp`; owners from `/proc/<pid>/fd` socket links.

Background. TCP connections go through *states*: a client sends SYN, the server answers, and the connection is Established. Closing passes through FIN Wait / Close Wait, and the side that closes first waits in Time Wait so late packets are not mistaken for a new connection.

TCP V6 ports

`tcp-ip6-ports` - Linux data

In plain words. The same as TCP V4 ports but for IPv6 TCP sockets.

What you see. The same columns and states as TCP V4 ports, with IPv6 addresses. `::` means "all addresses", `::1` is this PC only. Addresses like `::ffff:192.0.2.20` are IPv4 connections accepted by an IPv6 socket (IPv4-mapped addresses).

Healthy / typical values. On the author's PC 30 sockets: 23 listening, 7 other. For example KDE Connect (`kdeconnectd`) listening on `:: 1716` and connected to a phone over an IPv4-mapped address, syncthing on 22000, and CoolerControl talking to its daemon on `::1`.

Unusual values and what they suggest. As for TCP V4. Also, a service listening only on `::` may still accept IPv4 through mapped addresses, so it can be more reachable than it looks.

Where Linux gets it. `/proc/net/tcp6`; owners from `/proc/<pid>/fd`.

Network transports

`transports` - Linux data

In plain words. A count of sockets in use per transport type, and which address families have socket tables. Windows lists NetBIOS transport bindings here; Linux has none.

What you see.

- From `/proc/net/sockstat` and `sockstat6`: **sockets used**, **TCP** (inuse, orphan, tw = time-wait, alloc, mem), **UDP**, **UDPLITE**, **RAW**, **FRAG** (packets being reassembled), and the IPv6 versions.
- **Address families with registered sockets** - which of unix, packet, netlink, raw, tcp, udp, icmp, sctp, bluetooth, can, vsock, xdp and others have a table in `/proc/net`.
- A note that SMB/NetBIOS on Linux simply runs over TCP 139/445 and UDP 137/138.

Healthy / typical values. On the author's PC: 1,937 sockets used, TCP inuse 50, orphan 0, tw 1, UDP inuse 53, FRAG inuse 0.

Unusual values and what they suggest. A large TCP "orphan" count: connections closed by programs but not yet finished in the kernel; many orphans can mean a DoS (denial-of-service) attack or a buggy server. TCP mem near the limit: see Press in Network protocols.

Where Linux gets it. `/proc/net/sockstat`, `/proc/net/sockstat6`, the `/proc/net` directory.

UDP V4 ports

`udp-ip4-ports` - Linux data

In plain words. Every IPv4 UDP socket, with its owner.

What you see. The same columns as TCP V4 ports. UDP has no real connection, so State is "Listen" (bound and waiting) or "Connected" (fixed to one remote address).

Healthy / typical values. On the author's PC 53 sockets, 51 listening: dnsmasq (53), DHCP server (67, bootps) for the VM bridges, rpcbind (111), Samba's NetBIOS name and datagram services (137, 138) on each address including broadcast addresses, WS-Discovery (3702) on each address plus the multicast group 239.255.255.250, avahi mDNS (5353), and syncthing discovery (21027).

Unusual values and what they suggest. A program listening on 0.0.0.0 with a port you do not recognise. Games and voice apps open random high ports while running; that is normal. Port 1900 (SSDP) or 5353 bound by many apps is also normal.

Where Linux gets it. `/proc/net/udp`; owners from `/proc/<pid>/fd`.

UDP V6 ports

`udp-ip6-ports` - Linux data

In plain words. The same as UDP V4 ports but for IPv6 UDP sockets.

What you see. The same columns. Addresses starting `ff02::` are IPv6 multicast groups (for example `ff02::c` for WS-Discovery); `fe80::` addresses are link-local, one per adapter.

Healthy / typical values. On the author's PC 29 sockets, all listening: wsdd on each adapter's link-local address and `ff02::c`, avahi on `:: 5353`, KDE Connect on 1716, syncthing on 21027 and 22000.

Unusual values and what they suggest. As for UDP V4.

Where Linux gets it. `/proc/net/udp6`; owners from `/proc/<pid>/fd`.

Wi-Fi SSIDs

`wi-fi` - Linux data

In plain words. Your Wi-Fi card's current connection (channel, signal, speed), and the networks NetworkManager last saw. SRS never starts a new scan.

What you see. Per wireless adapter, from `iw dev <if> info`:

- **addr** - its MAC address. **ssid** - the network it is on. **type** - managed (normal client), AP, monitor... **wiphy** - radio number. **channel** - channel number, frequency, width. **txpower** - transmit power.

Then from `iw dev <if> link`:

- **Connected to** - the access point's address, or **Link: Not connected**.
- **SSID**, **freq** - network and frequency in MHz.
- **RX / TX** - bytes and packets on this link.
- **signal** - received strength in dBm.
- **rx bitrate / tx bitrate** - the current data rate and how it is built: width, standard (HT = Wi-Fi 4, VHT = Wi-Fi 5, HE = Wi-Fi 6, EHT = Wi-Fi 7), MCS (coding level), NSS (number of spatial streams), GI (guard interval).
- **bss flags, dtim period, beacon int** - access point timing details.
- On Wi-Fi 7 multi-link connections, extra Link and MLD lines per link.

Then **/proc/net/wireless** - the old wireless-extensions quality line (many modern drivers leave it empty, giving "no associated interfaces").

Then **Visible networks** from NetworkManager's cache:

Column	Meaning
(first)	`*` = the one you are on.
SSID	Network name, "(hidden)" if it does not broadcast one.
BSSID	The access point's radio address.
Chan, Band	Channel and band: 2.4 GHz, 5 GHz or 6 GHz.
Rate	Max rate NetworkManager reports.
Signal, Bars	Strength in percent (green 60% and up, orange 30-59%, red below 30%) and as bars.
Security	WPA2, WPA3, WPA1, "open"...

Healthy / typical values. On the author's PC `wlan0` (Wi-Fi 7 card) is on channel 36, 5180 MHz, 160 MHz wide, txpower 20 dBm, signal -28 dBm (excellent; the router is close), tx bitrate 2882.3 MBit/s using EHT-MCS 13 with 2 streams, security WPA3, 92%. The rx bitrate was only 6 MBit/s because the link was idle at that moment; it rises with traffic.

Unusual values and what they suggest.

- Signal weaker than about -70 dBm: slow and unstable. Move closer, or use 5 GHz only where coverage is good.
- Connected on 2.4 GHz with a 5/6 GHz-capable router nearby: slower; the bands may share one name and the PC chose the far-reaching one.
- 20 MHz channel width on 5 GHz: the router or regulatory setting is limiting width.
- Security "open" or WPA1 on your own network: change it to WPA2 or WPA3.
- "NetworkManager has no cached scan results": no scan has happened yet; open your Wi-Fi menu to trigger one.

Where Linux gets it. `iw dev <if> info`, `iw dev <if> link`, `/proc/net/wireless`, `nmcli device wifi list --rescan no`.

Background. *SSID* (Service Set Identifier) is the network name; *BSSID* is the radio address of one access point. *dBm* is signal power on a log scale: -30 is excellent, -67 is good for video calls, -80 is barely usable. *MCS* (Modulation and Coding Scheme) is the encoding step; higher means faster but needs a cleaner signal.

WLAN profiles

`wlan` - Linux data, Needs root data

In plain words. The saved Wi-Fi networks only, with their name, mode and type of security. Passwords are never shown.

What you see. The same columns as Network profiles, but the last column is **SSID / Security / IPv4**: the network name, then in brackets the mode (infrastructure = normal router, adhoc, ap = hotspot), the key management (none/open, wpa-psk = WPA2 password, sae = WPA3 password, wpa-eap = enterprise login, owe = enhanced open), and the IPv4 method (auto = DHCP, manual, shared, disabled).

The footer says keys are never shown: reading them needs root or a secret agent, and SRS does not ask for them.

Healthy / typical values. On the author's PC three saved Wi-Fi networks: the home network (`[infrastructure, sae, ipv4 auto]`, WPA3), a university profile (`wpa-eap`, enterprise login), and one more WPA3 network last used weeks ago.

Unusual values and what they suggest. A saved network with security "open" and Auto yes: your PC will join any network that copies that name, which is how fake hotspots work. Profiles from long ago can be deleted. `wpa-psk` (WPA2) on your own router is fine, but switch to `sae` (WPA3) if all your devices support it.

Where Linux gets it. `nmcli connection show` and `nmcli connection show <uuid>` (ssid, mode, key-mgmt, ipv4.method). Full profiles, including secrets, live in `/etc/NetworkManager/system-connections`, which is root-only.

Background. *WPA2* and *WPA3* are Wi-Fi security standards. WPA3's *SAE* (Simultaneous Authentication of Equals) protects the password even if someone records your traffic.

Linux (SIV's Windows items) (SRS Section Reference)

Back to the section index - Glossary

Where to find these items (SRS 2.0). In the main window they are on the **Operating system** page (Alt+-). Its first tab is **General Info** (SIV's "Windows" panel; the classic **Linux** button), then Drivers, Processes, Startup programs, Installed software, Environment and Users; **All N items** lists the rest. You can also type an item's name or id into the search box (Ctrl+F). The rest of this introduction describes the classic view (Menu -> Classic View), with its Menu and panel buttons.

This group holds the items that SIV64X (System Information Viewer, by Ray Hinchliffe, RH Software) files under "Windows": the operating system itself, its services, drivers, users, memory, time, fonts, languages and crash handling. SRS (srs-sysinfo) is an independent Linux program. It is not affiliated with or endorsed by RH Software and contains no SIV code or data. Most Windows ideas have a close Linux twin, so each item shows that twin: services become systemd units, drivers become kernel modules, the registry becomes the files in `/etc` and `~/.config`, and Windows Error Reporting (WER) becomes `coredumpctl`. Open an item with **Menu -> Linux (SIV Windows items) -> item**, or press the **Linux** button in the classic main window (SIV's "Windows" button). That button opens the overview panel; its sub-buttons are Machine, Sensors, USB Bus, Processors, Volumes, PCI Bus, ALL Dev and Graphics. Only **Open server files** needs root data from the srs-root helper (Menu -> Run as root) to be complete; **Sessions** and **Product IDs** each have one line that also fills in only after srs-root has run.

How to read the saved text. In a saved report (`sysreport-search '-save=[id]'`), `[=]` is a ticked box and `[_]` is an empty box. The GUI draws them as real check boxes. Some items take an argument after a colon, such as `driver-vsn:amdgpu`. You can type that form in the GUI or save it from the command line, for example `sysreport-search '-save=[signing:amdgpu]'`.

"This process" items. Several items (Environment, the DLL items, Process heap, Memory heap, Memory mapping, Virtual memory, Privileges, Handles, and part of Scheduling) describe SRS's own running process. SIV does the same thing on Windows. They show what any normal program on your desktop gets: its libraries, memory, limits and rights.

Contents

- ASUS control service `acs-srv`
- Boot mode options `boot-mode`
- Corsair Link 4 service `cl4-srv`
- Shell CLSIDs `clsid`
- Code pages `codepage`
- TAPI country list `country`
- Corsair service `cue-srv`
- Base DLLs `dll`
- DLLs by address `dll-addr`

- Known DLLs `dll-known`
- DLLs by name `dll-name`
- Drivers `drivers`
- Driver versions `driver-vsn`
- E-mail accounts `e-mail`
- Environment `environment`
- ESU status `esu-sts`
- System file cache `file-cache`
- Open server files `files`
- Feature management overrides `fmo-sts`
- Folder paths `folders`
- Font families `fonts`
- Groups `groups`
- Handles `handle`
- Process heap `heap`
- Registry hives `hives`
- Code integrity `integrity`
- Key names `key-names`
- Locales `locales`
- Memory heap `mem-heap`
- Memory mapping `mem-map`
- System metrics `metrics`
- MLang conversions `mlang`
- DOS device names `ms-dos`
- N2K service `n2k-srv`
- NLS code pages `nls-info`
- Paging files `paging`
- Credentials `passwords`
- Pending file renames `pending`
- ProLiant monitor service `plm-srv`
- Windows policies `policies`
- Memory pools `pools`
- Power policy `power-policy`
- Power scheme `power-scheme`
- Privileges `priv`
- Processes `processes`
- Product IDs `product-ids`
- Product keys `products`
- Font ranges `ranges`
- RDP session info `rdp-info`
- Remote registry paths `reg-path`
- Remote registry service `reg-srv`
- Text converter render format `render-format`
- Intel RST service `rst-srv`
- Scheduling `scheduling`
- Driver services `services-drv`
- Win32 services `services-w32`
- Sessions `sessions`

- Setup directories `setup`
- Setup API logging `setup-api`
- Driver signing `signing`
- Installed software `software`
- Startup programs `startup`
- System `system`
- System timers `sys-tim`
- System windows `sys-win`
- Thread windows `tid-win`
- Time `time`
- Times `times`
- Users `users`
- Driver verifier `verifier`
- Virtual memory `virtual`
- Visual Studio paths `vs-paths`
- Dr Watson / crash debugger `watson`
- Error reporting `wer`
- Windows CE devices `win-ce`
- General Info `windows`
- WinSAT `winsat`
- Time zones `zones`

ASUS control service

`acs-srv` - Not present on most PCs

In plain words. On Windows, ASUS laptops and some desktops run an "ASUS control service" (Armoury Crate and friends). Linux has no such service. SRS shows the nearest Linux pieces instead.

What you see.

- `Windows service` - always "not applicable on Linux - showing the Linux analogue".
- Unit lines `asusd.service`, `supergfxd.service` - two columns: Active state (`active`, `inactive`, `failed`) and Enabled state (`enabled`, `disabled`, `masked`, `static`, or `not-found` / "not installed"). `asusd` is the asusctl daemon for ASUS ROG/TUF laptops (fan curves, keyboard light, charge limit). `supergfxd` switches laptop graphics modes.
- Module lines `asus_wmi`, `asus_nb_wmi`, `asus_ec_sensors`, `asus_wmi_sensors`, `asus_armoury` - "loaded" or "not loaded". These are kernel drivers that talk to ASUS firmware: hot keys, fan and temperature sensors, BIOS settings.

Healthy / typical values. On the author's ASUS TUF X870 desktop board every module is "not loaded" and `asusd` is `inactive / masked`. That is normal for a desktop. On an ASUS laptop you would expect `asus_nb_wmi` loaded and, if you installed asusctl, `asusd` active.

Unusual values and what they suggest. An ASUS laptop with `asus_nb_wmi` "not loaded": Fn hot keys and fan modes will not work; check that the kernel was built with the driver. `asus_ec_sensors` not

loaded on a supported ROG/TUF desktop board: board sensors (chipset, VRM temperature) will be missing from the Sensors items.

Where Linux gets it. `systemctl is-active` / `is-enabled` for each unit; `/sys/module/<name>` for each module.

Background. A **systemd unit** is Linux's version of a Windows service entry. "Masked" means it has been blocked on purpose and cannot start, even by hand.

Boot mode options

`boot-mode` - Linux data

In plain words. How this PC started: UEFI or old BIOS, Secure Boot on or off, which boot loader, and which "target" (Linux's version of a boot mode) it boots into.

What you see.

- `Firmware` - "UEFI" or "Legacy BIOS (CSM)". CSM (Compatibility Support Module) is the old-BIOS emulation inside UEFI firmware.
- `UEFI platform size` - 64-bit or 32-bit UEFI. Some cheap tablets have 32-bit UEFI on a 64-bit CPU, and that needs a special boot loader.
- `Secure Boot` - the line from `mokutil --sb-state`, e.g. "SecureBoot enabled" / "SecureBoot disabled".
- `Default target` - the systemd target used at boot. `graphical.target` = desktop login; `multi-user.target` = text console only.
- `Kernel command line` - the options the boot loader passed to the kernel.
- `Boot loader (bootctl status)` - the System block from `bootctl`: firmware arch, Secure Boot, TPM2 support, measured boot, "Boot into FW" (can the OS reboot you straight into BIOS setup), platform language, and the boot loader product.
- `Safe-mode equivalents (systemd)` - a fixed cheat sheet. Add `systemd.unit=rescue.target` (like Windows Safe Mode), `emergency.target` (bare root shell, read-only root) or `multi-user.target` (like Safe Mode with Networking, no desktop) to the kernel command line for one boot.

Healthy / typical values. Author's PC: UEFI, 64-bit, SecureBoot disabled, `graphical.target`, boot loader "GRUB 2.16", TPM2 Support yes. The command line starts `BOOT_IMAGE=/@/boot/vmlinuz-linux73-tkg-bore-llvm root=UUID=XXXX rw rootflags=subvol=@ ...` followed by tuning options such as `amd_pstate=active preempt=full`.

Unusual values and what they suggest. "Legacy BIOS (CSM)" on a modern board: the disk was installed in old mode. You cannot use Secure Boot, and Windows 11 dual-boot needs UEFI. `Default target multi-user.target` on a desktop: the PC will boot to a text login. Options you do not recognise on the command line (such as `mitigations=off`) change security or performance. Check the GRUB or systemd-boot config before you copy them.

Where Linux gets it. `/sys/firmware/efi` (present = UEFI), `/sys/firmware/efi/fw_platform_size`, `mokutil --sb-state`, `systemctl get-default`, `/proc/cmdline`, `bootctl status`.

Background. UEFI (Unified Extensible Firmware Interface) replaced the PC BIOS. Secure Boot is a UEFI feature that only runs boot loaders signed with trusted keys. A **systemd target** is a named group of services. Booting into a target is like picking a Windows boot mode.

Corsair Link 4 service

`cl4-srv` - Not present on most PCs

In plain words. Corsair Link 4 was Corsair's Windows service for fans, pumps and power supplies. On Linux, the same hardware is run by CoolerControl, liquidctl or kernel hwmon drivers. SRS shows those. This item prints the same probe as Corsair service.

What you see.

- `Windows service` - "not applicable on Linux - showing the Linux analogue".
- Units `coolercontrold.service`, `coolercontrol-liqctld.service`, `liquidcfg.service` - Active state and Enabled state.
- Modules `corsair_cpro` (Commander Pro), `corsair_psu` (Corsair HXi/RMi power supplies), `nzxt_kraken3`, `nzxt_smart2` - "loaded" / "not loaded".
- Processes `coolercontrold`, `coolercontrol-l` (the liquidctl helper; the kernel cuts process names to 15 characters), `liquidctl` - "running PID n" or "not running".

Healthy / typical values. Author's PC: `coolercontrold.service active enabled`, process `coolercontrold` running. No Corsair or NZXT kernel modules are loaded because no such device is attached.

Unusual values and what they suggest. A Corsair PSU on USB but `corsair_psu` "not loaded": power-supply sensors will be missing. Try `modprobe corsair_psu` or let CoolerControl read it. `coolercontrold` enabled but not running: the daemon crashed. Check `journalctl -u coolercontrold`.

Where Linux gets it. `systemctl is-active` / `is-enabled`, `/sys/module/<name>`, the process list in `/proc`.

Background. hwmon is the kernel's hardware-monitoring framework. Each sensor chip appears under `/sys/class/hwmon`, and tools like CoolerControl read and control it there.

Shell CLSIDs

`clsid` - Windows-only (Linux equivalent shown)

In plain words. In Windows, a CLSID (class ID) is a registry number that ties a file type or shell object to a program. Linux has no CLSIDs. The nearest thing is the default-application list, which says which app opens which file type or link type.

What you see.

- Table `MIME Type / Scheme - Default Handler (.desktop)`: one row per line in the `[Default Applications]` part of `~/.config/mimeapps.list`. Lines starting `x-scheme-handler/` (links such as `mailto:` or `https:`) are tinted differently.
- `System .desktop entries` - how many files are in `/usr/share/applications`.
- `User .desktop entries` - how many are in `~/.local/share/applications`.
- If you have no defaults: "no user defaults in ~/.config/mimeapps.list".

Healthy / typical values. Author's PC: `application/pdf -> okularApplication_pdf.desktop;`, `application/ogg -> vlc.desktop;`, `application/vnd.flatpak -> org.kde.discover.desktop;`. A trailing `;` is normal; the format allows a list.

Unusual values and what they suggest. A handler whose `.desktop` file no longer exists (the app was removed): double-clicking that file type does nothing or asks every time. Fix it in System Settings -> Default Applications or File Associations.

Where Linux gets it. `~/.config/mimeapps.list`; the counts are directory listings of `/usr/share/applications` and `~/.local/share/applications`.

Background. A **MIME type** (Multipurpose Internet Mail Extensions type) is a standard name for a kind of file, such as `image/png`. A **.desktop file** is a small text file that describes an app: its name, icon, command, and the MIME types it can open.

Code pages

`codepage` - Linux data

In plain words. A code page is an old table that maps byte values to letters. Windows still lists dozens of them. Linux uses UTF-8 almost everywhere, but its iconv library can still convert the old ones. This item checks each Windows code page against it.

What you see.

- `Page ID` - the Windows code page number (437, 850, 1252, 65001...).
- `Description` - what it is, e.g. "OEM United States", "ANSI Latin 1", "Mac Roman", "Unicode UTF-8".
- `glibc iconv` - the name glibc uses for it (CP437, IBM037, ISO-8859-15, MACINTOSH...), or `-` if none was found.
- `Supported` - ticked if glibc can convert that page to UTF-8.
- Footer: "N of 70 supported; locale charset X" and the number of glibc charmaps in `/usr/share/i18n/charmaps`.

Healthy / typical values. Author's PC: "58 of 70 supported; locale charset UTF-8", 233 charmaps. The gaps are pages glibc does not ship, such as 720 (Arabic Transparent ASMO) and several Mac pages.

Unusual values and what they suggest. Very few pages supported (say under 10): the glibc `gconv` modules are missing or were trimmed, for example in a minimal container. Old text files and some emulators will then show garbled letters. A locale charset of `ANSI_X3.4-1968` means plain ASCII: no `LANG` is set, so programs cannot print accented letters.

Where Linux gets it. glibc `iconv_open()` tested for each name; `/usr/share/i18n/charmaps`; the charset part of `LC_ALL` / `LC_CTYPE` / `LANG`.

Background. "OEM" pages (437, 850...) are the old DOS character sets. "ANSI" pages (1250-1258) are the old Windows ones. UTF-8 (page 65001) can hold every Unicode character, which is why Linux uses it by default.

TAPI country list

`country` - Linux data

In plain words. On Windows, TAPI (Telephony API) keeps a country list with dialling codes. Linux has no TAPI. SRS lists the ISO 3166 country table from the time-zone database instead, with the number of time zones in each country.

What you see.

- `Code` - the two-letter ISO 3166 country code. Your own country (from your locale) is shown in green.
- `Country` - the country name.
- `Zones` - how many time-zone entries `zone.tab` lists for it.
- `Representative Time Zone` - the first zone listed for that country.
- Footer: "N countries; current locale territory XX".

Healthy / typical values. Author's PC: 249 countries, current territory US. Examples: `AR Argentina 12 America/Argentina/Buenos_Aires`, `AQ Antarctica 10 Antarctica/McMurdo`.

Unusual values and what they suggest. "current locale territory -": your `LANG` has no country part (for example `C.UTF-8`), so programs fall back to default number, date and paper formats. Error "iso3166.tab not readable": the `tzdata` package is missing or damaged.

Where Linux gets it. `/usr/share/zoneinfo/iso3166.tab` and `/usr/share/zoneinfo/zone.tab` (both from tzdata); the territory comes from `LC_ALL` or `LANG`.

Corsair service

`cue-srv` - Not present on most PCs

In plain words. The Windows "Corsair Service" belongs to iCUE (Corsair Utility Engine). Linux has no iCUE. SRS shows the same Linux analogue as Corsair Link 4 service: CoolerControl and liquidctl units, Corsair and NZXT hwmon modules, and their processes.

What you see. Exactly the same lines as `cl4-srv`: three units with Active and Enabled state, four modules loaded / not loaded, three processes running / not running.

Healthy / typical values. Author's PC: `coolercontrold.service active enabled`, process running, no Corsair/NZXT modules loaded.

Unusual values and what they suggest. Corsair RGB keyboards and mice are not covered here. On Linux they are driven by OpenRGB or ckb-next, which this probe does not check.

Where Linux gets it. `systemctl`, `/sys/module`, `/proc`.

Base DLLs

`dll` - Linux data

In plain words. A DLL (dynamic-link library) is Windows' shared code file. The Linux version is the **shared object** (`.so` file). This item lists every file mapped into SRS's own process: the program, its libraries, and some data files.

What you see.

- `Image` - the running program file.
- `Dynamic loader` - the program that loads shared libraries at start, normally `/usr/lib/ld-linux-x86-64.so.2`.
- `glibc` - the C library version.
- Table:
  - `Base` / `End` - lowest and highest memory address the file occupies, in hex.
  - `Size` - End minus Base.
  - `Module` - file name.
  - `Version / Path` - for `name.so.N` files, "VN" is the number after `.so.` (the soname major number), then the full path.
- Footer: "N mapped files (executable, shared libraries, locale/data files)".

Healthy / typical values. Author's PC: glibc 2.44, 9 mapped files. These are `sysreport-search` (1.38MB), `libc.so.6` (V6, 2.25MB), `libm.so.6`, the loader (V2), `/etc/ld.so.cache`, and glibc `gconv` converters such as `UTF-16.so`. The converters appear because other items have already used iconv.

Unusual values and what they suggest. A library from an unexpected place (for example `/home/...` or `/tmp/...` instead of `/usr/lib`): something is setting `LD_LIBRARY_PATH` or `LD_PRELOAD`. Check Known DLLs. The GUI process (`srs-gui`) maps many more files (GTK and friends) than the command-line run.

Where Linux gets it. `/proc/self/maps`, `readlink /proc/self/exe`, `gnu_get_libc_version()`.

Background. The **soname** number (the 6 in `libc.so.6`) changes only when a library breaks compatibility. It is not the library's release version.

DLLs by address

`dll-addr` - Linux data

In plain words. The same table as Base DLLs, in memory-address order and without the header lines.

What you see. Base, End, Size, Module, Version / Path, and the "N mapped files" footer, exactly as in `dll`.

Healthy / typical values. The program itself sits low (around `6023822F3000` on the author's PC) and the libraries sit high (`7A58...`). The exact numbers change on every run.

Unusual values and what they suggest. Addresses that are the same on every run would mean ASLR is off; see `randomize_va_space` in Virtual memory.

Where Linux gets it. `/proc/self/maps`.

Background. ASLR (Address Space Layout Randomization) loads programs and libraries at random addresses each time. That makes many attacks harder.

Known DLLs

`dll-known` - Linux data

In plain words. Windows keeps a list of "Known DLLs" that always load from the system folder. Linux has a cache of every library the loader knows about, plus the list of folders it searches. This item shows both.

What you see.

- `ld.so.cache` - the first line of `ldconfig -p`, e.g. "12521 libs found in cache `/etc/ld.so.cache'".
- `LD_LIBRARY_PATH` - extra library folders set in your environment, or `<not set>`.
- `LD_PRELOAD` - libraries forced into every program, or `<not set>`.
- `/etc/ld.so.preload` - shown only if the file exists. It forces libraries into every program system-wide.
- `Search directories` - each folder from `/etc/ld.so.conf` and `/etc/ld.so.conf.d/*.conf`, with the file that added it.
- `Core system libraries (ldconfig -p)` - cache entries for key libraries (libc, libm, libstdc++, libz, libssl, libsystemd, libdrm, libGL, libEGL, libvulkan, libX11, libwayland-client, libasound, libpulse...), both 64-bit (`libc6,x86-64`) and 32-bit (`libc6`).

Healthy / typical values. Author's PC: 12521 libraries, both variables not set. Extra folders come from installed packages, e.g. `/opt/rocm/lib` (rocm.conf), `/usr/lib32` (lib32-glibc.conf).

Unusual values and what they suggest. `LD_PRELOAD` set when you did not set it, or a `/etc/ld.so.preload` you do not recognise: something injects code into every program. That is sometimes a tool (a game overlay, a memory allocator) and sometimes malware. Find out which package owns it. A newly installed library that is missing from the cache: run `sudo ldconfig`.

Where Linux gets it. `ldconfig -p`, `/etc/ld.so.conf`, `/etc/ld.so.conf.d/`, `/etc/ld.so.preload`, environment variables.

DLLs by name

`dll-name` - Linux data

In plain words. The same table as Base DLLs, sorted by file name (A-Z, ignoring case).

What you see. Base, End, Size, Module, Version / Path, and the footer. Author's PC: `gconv-modules.cache`, `ISO8859-13.so`, `ISO8859-15.so`, `ld-linux-x86-64.so.2`, `ld.so.cache`, `libc.so.6`, `libm.so.6`, `sysreport-search`, `UTF-16.so`.

Healthy / typical values / unusual values. As for Base DLLs. Sorting by name makes it easy to spot two copies of the same library from different folders.

Where Linux gets it. `/proc/self/maps`.

Drivers

`drivers` - Linux data

In plain words. Windows drivers become Linux **kernel modules**: pieces of the kernel that are loaded on demand. This item lists every loaded module, the same list as `lsmod`.

What you see.

- `Module Name` - the module name (A-Z).
- `Size` - memory the module uses.
- `Used` - how many other modules or users hold it.
- `State` - `Live` (normal), `Loading` or `Unloading` (anything other than Live is shown in orange).
- `Taint` - letters from the kernel if this module "taints" the kernel: `O` = out-of-tree (built outside the kernel source), `E` = unsigned, `P` = proprietary licence. Blank = clean.
- `Used By` - the modules that depend on it.
- Footer: "N loadable modules loaded, M built into kernel (modules.builtin)".

Healthy / typical values. Author's PC: 237 loaded, 256 built in. `amdgpu` is 15.32MB with Used 61. `bluetooth` is used by `btusb`, `btrtl`, `btintel` and others. `zenergy` shows `OE` (an out-of-tree, unsigned energy-sensor module).

Unusual values and what they suggest. A module stuck in `Unloading`: something still holds it, and a reboot may be needed. `P` on a module: a closed-source driver (for example the NVIDIA one) is loaded, and kernel developers will ignore bug reports from that kernel. Your GPU driver missing from the list (and not built in): the desktop is running on a fallback driver with no 3D acceleration.

Where Linux gets it. `/proc/modules`; the built-in count is the line count of `/usr/lib/modules/<release>/modules.builtin`.

Background. A **built-in** driver is compiled into the kernel file itself. It is always present and never shows in `/proc/modules`.

Driver versions

`driver-vsn` - Linux data

In plain words. Every module the kernel knows about, loaded or built in, with whatever version facts it publishes.

What you see.

- `Module` - name, from `/sys/module`.
- `Type` - `loadable` (a separate `.ko` file) or `built-in`.
- `Version` - the module's own version string, if its author set one (most in-tree modules do not, so `-` is normal).
- `Source Version` - `srcversion`, a checksum of the module's source. It changes whenever the code changes.
- `Refs` - reference count for loadable modules.
- `Parameters` - how many tunable parameters it has under `/sys/module/<name>/parameters`.
- Footer note: "Open as driver-vsn: for full modinfo output".
- With an argument, e.g. `driver-vsn:zram`, the item prints the full `modinfo` output instead: file name, licence, author, description, dependencies, `intree`, `vermagic` (the kernel it was built for), signer, hash algorithm, and each `parm:` with its description.

Healthy / typical values. Author's PC: 390 lines. `ahci` is built in, version 3.0, 4 parameters. `amdgpu` is loadable with 95 parameters. `zenergy` is loadable with version 1.0. `driver-vsn:zram` shows `description: Compressed RAM Block Device`, `intree: Y`, `signer: Build time autogenerated kernel key`.

Unusual values and what they suggest. In `modinfo`, a `vermagic` that names a different kernel than the running one: the module was built for another kernel and will refuse to load. Usually a DKMS (Dynamic Kernel Module Support) rebuild is missing. `intree` absent: an out-of-tree module (DKMS or a vendor package).

Where Linux gets it. `/sys/module/<name>/{version,srcversion,refcnt,initstate,parameters}`; `modinfo <name>` for the argument form.

E-mail accounts

`e-mail` - Windows-only (Linux equivalent shown)

In plain words. SIV lists Windows mail accounts. SRS never reads account data. It only reports which mail programs have a profile on this Linux account, and which app opens `mailto:` links.

What you see.

- `Thunderbird` - the path of each `profiles.ini` it finds (normal install, Flatpak, Snap), then one `Profile` line per profile name.
- `Evolution` - "N source definitions in ~/.config/evolution/sources".
- `KMail / Akonadi` - the Akonadi data folder, if present.

- "No Thunderbird/Evolution/KMail profiles found" if none.
- `mailto: handler` - the `.desktop` file that opens e-mail links.

Healthy / typical values. Author's PC: Evolution with 10 source definitions, the KMail/Akonadi folder present, and a `mailto:` handler set (for example `thunderbird.desktop` on a typical system).

Unusual values and what they suggest. `mailto: handler -`: clicking an e-mail link in a browser does nothing. Set a default mail client in System Settings.

Where Linux gets it. `~/.thunderbird/profiles.ini` (and the Flatpak and Snap copies), `~/.config/evolution/sources`, `~/.local/share/akonadi`, `xdg-mime query default x-scheme-handler/mailto`.

Environment

`environment` - Linux data

In plain words. Environment variables are named text settings that every program inherits from the one that started it, such as `PATH`, `HOME` and `LANG`. Windows and Linux both have them.

What you see.

- `Environment Variable Name` - sorted A-Z.
- `Size` - length of the value in characters.
- `Environment Variable Expansion` - the value. Values whose names contain TOKEN, KEY, SECRET, PASS, AUTH, COOKIE, CREDENTIAL or SESSION_ID are replaced by "******** (masked)".
- `/etc/environment` - a second block with the system-wide settings from that file (comment lines skipped), in the same three columns.
- Footer: "N process variables; values of names matching ... are masked".

Healthy / typical values. You should see `HOME=/home/user`, `LANG=en_US.UTF-8`, `PATH=/usr/local/bin:/usr/bin...`, `XDG_CURRENT_DESKTOP=KDE`, `XDG_SESSION_TYPE=wayland`. The author's PC also has tuning variables such as `AMD_VULKAN_ICD=RADV` and `CC=aocc-clang`.

Unusual values and what they suggest. `PATH` missing `/usr/bin`: many commands will "not be found". A very long `PATH` with repeated folders: a shell start-up file adds the same folder every time it runs. `LD_PRELOAD` or `LD_LIBRARY_PATH` set globally: see Known DLLs. If SRS was started from a menu, it shows the desktop session's variables. Started from a terminal, it shows that shell's variables.

Where Linux gets it. The process environment (`/proc/self/environ`); `/etc/environment`.

ESU status

`esu-sts` - Windows-only (Linux equivalent shown)

In plain words. ESU (Extended Security Updates) is Microsoft's paid extra support for old Windows versions. The Linux question is: "does my distribution still get updates, and when did I last sync them?"

What you see.

- `Distribution` - `PRETTY_NAME` from `/etc/os-release`.
- `Support end` - `SUPPORT_END` if the distribution sets one, else "none (rolling release)".
- `Home`, `Support` - the distribution's web links, if set.
- `Repository databases (last sync)` - each pacman repository database with the date it was last downloaded.

Healthy / typical values. Author's PC: CachyOS, "none (rolling release)", most databases synced on 2026-10-23 or 2026-10-24.

Unusual values and what they suggest. A database that is months old (on the author's PC two old third-party repositories are from 2025-04-12): that repository has stopped publishing or was removed upstream. Consider dropping it from `/etc/pacman.conf`. A `Support end` date in the past on a fixed-release distribution: you are no longer getting security fixes, so upgrade to the next release.

Where Linux gets it. `/etc/os-release`; file times of `/var/lib/pacman/sync/*.db`.

System file cache

`file-cache` - Linux data

In plain words. Linux keeps recently used file data in spare RAM (the **page cache**) so it can be read again without touching the disk. Windows calls this the system file cache. This item shows how big the cache is, how busy it is, and the rules for writing changed data back to disk.

What you see. In the GUI it refreshes every second.

- `Page cache (/proc/meminfo)` table, each with a readable size and raw kB:
  - `Cached` - file data held in RAM.
  - `Buffers` - raw disk blocks (metadata) held in RAM.
  - `Active(file)` / `Inactive(file)` - recently used vs. older cache. Inactive pages are freed first when memory runs short.
  - `Dirty` - changed data not yet written to disk.
  - `Writeback` - data being written right now.
  - `Mapped` - cached files that programs have mapped into memory (libraries, for example).
  - `Shmem` - shared memory and tmpfs (RAM disks).
  - `Unevictable` - pages that may not be dropped (locked memory).
  - `FileHugePages` - cache held in large 2 MB pages.
- `Counters (/proc/vmstat)` - totals since boot: `pgpgin`/`pgpgout` (data read from / written to disk, in kB), `pgfault`/`pgmajfault` (page faults; "major" means the disk had to be read), `workingset_refault_file` (cache pages dropped and then needed again), `workingset_activate_file`, `pgsteal_kswapd`/`pgsteal_direct` (pages reclaimed by the background thread vs. by a program that had to wait), `drop_pagecache` (times someone ran "drop caches").
- `Writeback policy (/proc/sys/vm)` - `dirty_ratio`, `dirty_background_ratio`, `dirty_bytes`, `dirty_background_bytes`, `dirty_expire_centisecs`, `dirty_writeback_centisecs`, `vfs_cache_pressure`.

- `Page Size` - normally 4096 bytes.

Healthy / typical values. Author's PC (32 GB RAM): Cached 17.76GB, Dirty 3.75MB, Writeback 0B. Writeback limits are set in bytes (`dirty_bytes 268435456` = 256 MB, background 64 MB), `vfs_cache_pressure 30`. A big `Cached` value is good. It is free RAM doing useful work and is handed back as soon as programs need it.

Unusual values and what they suggest. `Dirty` staying in the gigabytes: a slow disk (USB stick, network share) is behind on writes, and pulling it out now loses data. `pgsteal_direct` rising fast while a game stutters: programs are waiting for memory to be freed, so RAM is short. A high `workingset_refault_file` with a small cache: the cache is too small for your workload.

Where Linux gets it. `/proc/meminfo`, `/proc/vmstat`, `/proc/sys/vm/*`, `sysconf(_SC_PAGESIZE)`.

Background. Dirty pages are changed in RAM but not yet saved to disk. The `dirty_*` settings decide how much may pile up before the kernel starts writing (background) and before programs are made to wait.

Open server files

`files` - Linux data - Needs root data

In plain words. On Windows this lists files that other PCs have open on your shared folders. On Linux, SRS shows file locks (who is holding which file), files open over Samba (SMB, Windows-style file sharing) and NFS (Network File System) exports.

What you see.

- `File locks (lslocks)` - columns `COMMAND`, `PID`, `TYPE` (`FLOCK`, `POSIX`, `OFDLCK` - three kinds of Linux file lock), `MODE` (`READ`/`WRITE`), `M` (1 = mandatory lock), `PATH`. Up to 400 lines.
- `SMB open files (smbstatus -L)` - comes from the srs-root cache. Without it you see "Needs root: Menu > Run as root (srs-root)".
- `NFS exports (/var/lib/nfs/etab)` - each exported folder with its options, or "none / not readable".

Healthy / typical values. Author's PC: locks held by `kwin_wayland`, `krunner`, `akonadi`, `libvirtd`, `dockerd` and Flatpak's `bwrap`. A virtual machine holds `OFDLCK` read locks on its disk image. Samba reports "No locked files".

Unusual values and what they suggest. A program can't open a file ("resource busy", "database is locked"): look here for who holds it. A lock on a path ending "(deleted)": the file was removed while still open, so its disk space is not freed until that process exits. Long paths may be cut short with "..." by `lslocks`.

Where Linux gets it. `lslocks`, `smbstatus -L` (run by srs-root), `/var/lib/nfs/etab`.

Feature management overrides

`fmo-sts` - Windows-only (Linux equivalent shown)

In plain words. Windows uses hidden "feature management" switches to turn new features on or off. Linux has no such system, and SRS shows no equivalent.

What you see. A short stub: `Status - Windows-only item in SIV`, `Linux equivalent - none`. That is the whole item, and nothing is wrong.

Folder paths

`folders` - Linux data

In plain words. Windows has "known folders" (AppData, Documents, Startup...). Linux has the XDG (freedesktop.org Cross-Desktop Group) folders. This item shows each one, with the Windows-style name next to the Linux variable, and ticks the ones that exist.

What you see. `Name` - tick (exists) + `Folder Path`:

- `PROFILE (HOME)`, `APPDATA (XDG_CONFIG_HOME)` = `~/.config`, `LOCAL_APPDATA (XDG_DATA_HOME)` = `~/.local/share`, `STATE (XDG_STATE_HOME)` = `~/.local/state`, `INTERNET_CACHE (XDG_CACHE_HOME)` = `~/.cache`, `RUNTIME (XDG_RUNTIME_DIR)` = `/run/user/<uid>`.
- Your user folders from `~/.config/user-dirs.dirs`: DESKTOP, DOCUMENTS, DOWNLOAD, MUSIC, PICTURES, PUBLICSHARE, TEMPLATES, VIDEOS, plus any extra ones.
- Fixed ones: FONTS (`~/.local/share/fonts`), PROGRAMS (`~/.local/share/applications`), STARTUP (`~/.config/autostart`), TRASH, RECENT (`recently-used.xbel`), COMMON_PROGRAMS (`/usr/share/applications`), COMMON_STARTUP (`/etc/xdg/autostart`), COMMON_APPDATA (`/etc/xdg`), COMMON_FONTS (`/usr/share/fonts`), PROGRAM_FILES (`/usr/bin`), PROGRAM_FILES_OPT (`/opt`), SYSTEM (`/usr/lib`), TEMP (`$TMPDIR` or `/tmp`).
- `XDG_DATA_DIRS`, `XDG_CONFIG_DIRS` - the system search lists.

Healthy / typical values. Author's PC: every path ticked; RUNTIME is `/run/user/1000`. DOCUMENTS, DOWNLOAD, MUSIC, PICTURES and VIDEOS all point to `/home/user/`, and there is an extra PROJECTS folder.

Unusual values and what they suggest. An empty box next to a user folder: `user-dirs.dirs` points to a folder that was deleted or renamed. Apps may save into your home folder instead. Run `xdg-user-dirs-update` or fix the file. No RUNTIME line: you are not in a proper login session (for example `su` without `-l`), so many desktop apps will fail.

Where Linux gets it. Environment variables, `~/.config/user-dirs.dirs`, fixed paths checked with `stat`.

Font families

`fonts` - Linux data

In plain words. Every installed font family, and which writing systems (character sets) each one covers. On Windows SIV lists GDI (Graphics Device Interface) character sets. On Linux, SRS asks fontconfig which fonts support each language.

What you see.

- `Unique N font families`.
- One line per character set with how many families cover it: `WER Western (:lang=en)`, `CER Central European (pl)`, `CRK Cyrillic (ru)`, `GRK Greek (el)`, `TRK Turkish (tr)`, `BTC Baltic (lt)`, `HBW Hebrew (he)`, `ARC Arabic (ar)`, `VMN Vietnamese (vi)`, `THI Thai (th)`, `JAP Japanese (ja)`, `HGL Hangul (ko)`, `CGB Chinese GB2312 (zh-cn)`, `CB5 Chinese BIG5 (zh-tw)`.
- A grid: one row per family (A-Z), with the code of each set it fully supports.

Healthy / typical values. Author's PC: 2141 families; 728 cover Western, 32 Japanese, 10 Thai. `AR PL New Kai` covers Cyrillic, Japanese and both Chinese sets. `3270 Nerd Font` covers the six European sets. Icon fonts (`octicons`, `icomoon`) cover none, which is normal.

Unusual values and what they suggest. JAP, HGL, CGB or CB5 at 0: you have no CJK (Chinese, Japanese, Korean) font, so those scripts show as empty boxes. Install Noto Sans CJK. A family listed with no sets is usually a symbol, icon or decorative font.

Where Linux gets it. `fc-list : family` and `fc-list :lang=<code> family` (fontconfig).

Background. fontconfig is the Linux library that finds fonts and decides which font to use for each character. "Covers a language" means the font has every letter fontconfig expects for it.

Groups

`groups` - Linux data

In plain words. Linux groups give a set of users shared rights, such as `audio` for sound devices or `wheel` for admin rights. Windows has local groups in the same way.

What you see.

- `Group Name`, `GID` (group ID number), `Members` (users listed for that group).
- Groups the SRS process belongs to are meant to show in green.
- Footer: "N groups; green = groups of the current process (n)".

Healthy / typical values. Author's PC: 286 groups. `root 0`, `wheel 998 user` (wheel members may use sudo), `audio 995` with several service accounts, and `nobody 65534` with no members.

Unusual values and what they suggest. Your user missing from `wheel` (or `sudo` on Debian-style systems): you cannot run admin commands. Your user missing from `video`, `input` or `uucp` when a

device needs it: "permission denied" on GPUs, game controllers or serial adapters. Group changes take effect only after you log out and back in.

Where Linux gets it. `/etc/group`; the process's own groups from `getgroups()`.

Background. A user's main group is in `/etc/passwd`. `/etc/group` lists only the extra (supplementary) memberships, so a group can look empty and still be someone's main group.

Handles

`handle` - Linux data

In plain words. A Windows handle is a ticket to an open object. The Linux version is a **file descriptor**: a small number a process uses for each open file, pipe, socket or device. This item shows system limits, SRS's own descriptors, and the processes holding the most.

What you see.

- `System file-nr (alloc free max)` - descriptors in use system-wide, a legacy "free" figure (always 0 on modern kernels), and the system maximum.
- `file-max` - system-wide maximum.
- `nr_open (per process max)` - the highest per-process limit allowed.
- `inode-nr (alloc free)` - in-memory file records allocated and free.
- `RLIMIT_NOFILE (soft hard)` - SRS's own open-file limit.
- `This process (/proc/self/fd)` - each descriptor number and what it points to (0 = input, 1 = output, 2 = errors).
- `Open handles per process - top 30 of N readable` - process name, PID, descriptor count. Only processes you may read are counted. Without root, that means your own.

Healthy / typical values. Author's PC: `31933 0 524288` in use, soft/hard limit 1048576. Top holders: `dbus-broker` 341, `systemd` 248, `plasmashell` 145, `pipewire` 135.

Unusual values and what they suggest. One process with tens of thousands and rising: a **descriptor leak**. That program will eventually fail with "Too many open files". The first number near `file-max`: the whole system is about to run out. A low soft limit (1024) with games or Wine/Proton: some titles need far more. Esync needs a high `RLIMIT_NOFILE`.

Where Linux gets it. `/proc/sys/fs/{file-nr,file-max,nr_open,inode-nr}`, `getrlimit(RLIMIT_NOFILE)`, `/proc/self/fd`, `/proc/<pid>/fd` for every PID.

Process heap

`heap` - Linux data

In plain words. The **heap** is the pool of memory a program asks for while it runs (`malloc` in C). This item shows SRS's own heap as seen by glibc's allocator.

What you see.

- `Arena (sbrk heap)` - size of the main heap area.
- `mmap'd blocks` - big allocations served straight from the kernel, and how many.
- `In use (allocated)` - bytes the program is actually using.
- `Free (in arena)` - free space inside the heap and how many free chunks.
- `Fastbin free` - small freed blocks kept for quick reuse.
- `Releasable (top)` - free space at the top that could be handed back to the system.
- `[heap] mapping (/proc/self/smaps)` - the heap's address range, then `Size`, `Rss` (in RAM), `Pss` (proportional share), `Pss_Dirty`, `Private_Clean`, `Private_Dirty`, `Anonymous`, `Private_Hugetlb`, `Swap`.

Healthy / typical values. Author's PC, command-line run: arena 12.23MB, 9.88MB in use, 2.35MB free in 113 chunks, 2 mmap'd blocks of 18.01MB total. The heap mapping is 12528 kB with 6392 kB resident.

Unusual values and what they suggest. For SRS itself these numbers only matter to developers. A `Free` figure much larger than `In use` means fragmentation (memory freed in small scattered pieces).

Where Linux gets it. glibc `mallinfo2()`; the `[heap]` entry in `/proc/self/smaps`.

Registry hives

`hives` - Windows-only (Linux equivalent shown)

In plain words. Windows keeps settings in registry hive files. Linux keeps them mostly in plain text files under `/etc` (system) and `~/.config` (yours). A few binary databases act like hives, and SRS lists those, with their sizes and dates.

What you see.

- Table `Database` - `Size` - `Modified`:
  - `~/.config/dconf/user` - GNOME/GTK settings database (dconf).
  - `/usr/share/glib-2.0/schemas/gschemas.compiled` - compiled list of all GTK/GNOME setting keys.
  - `/etc/ld.so.cache` - library cache (see Known DLLs).
  - `/usr/lib/udev/hwdb.bin`, `/etc/udev/hwdb.bin` - the hardware database (device names, quirks).
  - Files in `/etc/dconf/db` - system-wide dconf defaults.
- `KDE config files (~/.config/*rc)` - how many KDE-style `...rc` text files you have and their total size.

Healthy / typical values. Author's PC: dconf user 66.07KB, hwdb.bin 13.87MB, 145 KDE rc files totalling 519.95KB. Dates match the last system update.

Unusual values and what they suggest. `hwdb.bin` much older than the last udev update: the database wasn't rebuilt, so new devices may get generic names. Run `sudo systemd-hwdb update`. A dconf file of several MB: an app is storing too much there.

Where Linux gets it. `stat` of the listed files; directory listings of `/etc/dconf/db` and `~/.config`.

Code integrity

`integrity` - Linux data

In plain words. Windows Code Integrity decides which drivers may load. On Linux, the same job is shared by module signing, kernel lockdown, Secure Boot, Linux Security Modules and hardening switches. The kernel also keeps a **taint** record of anything unusual that has happened since boot.

What you see.

- `Kernel lockdown` - `[none] integrity confidentiality`, or "lockdown LSM not built/active" (an LSM, Linux Security Module, is a pluggable security layer in the kernel). Lockdown stops even root from changing the running kernel.
- `Secure Boot (mokutil)` - enabled / disabled.
- `module.sig_enforce` - `Y` = only signed modules may load; `N` = unsigned ones load but taint the kernel.
- `kernel.modules_disabled` - 1 = no more modules can be loaded until reboot.
- `kernel.kexec_load_disabled` - 1 = no booting another kernel without a reboot.
- `kernel.kptr_restrict` - hides kernel addresses (0 shown, 1 hidden from non-root, 2 hidden from everyone).
- `kernel.dmesg_restrict` - 1 = only root can read the kernel log.
- `kernel.perf_event_paranoid` - how much performance monitoring normal users may do (higher = stricter).
- `kernel.unprivileged_bpf_disabled` - whether normal users can load BPF (Berkeley Packet Filter) programs, small sandboxed programs that run inside the kernel.
- `kernel.yama.ptrace_scope` - who may attach a debugger to a process (1 = only to your own child processes).
- `kernel.randomize_va_space` - ASLR (2 = full).
- `Active LSMs` - the security modules running.
- `TPM version` - 2 for a TPM (Trusted Platform Module) 2.0 security chip.
- `IMA` - Integrity Measurement Architecture, present / not active.
- Taint table: `Tainted N (0xHEX)`, then one box per bit 0-18 with its letter and meaning (P proprietary module, F forced load, S out-of-spec system, R forced unload, M machine check, B bad page, U userspace request, D recent oops, A ACPI (Advanced Configuration and Power Interface) table override, W warning, C staging driver, I firmware workaround, O out-of-tree module, E unsigned module, L soft lockup, K live patch, X distro, T struct randomisation, N test run).

Healthy / typical values. Author's PC: lockdown not active, SecureBoot disabled, `sig_enforce N`, `kptr_restrict 2`, `ptrace_scope 1`, LSMs `capability,landlock,yama,bpf`, TPM 2. Tainted 12356 (0x3044) = bits S, U, O, E: out-of-tree unsigned modules, a userspace-set flag, and the "out of spec" flag that an overclocked or tuned system can trigger. A stock distro kernel with no extra modules shows `Tainted 0`.

Unusual values and what they suggest. Bit `M` (machine check) set: the CPU reported a hardware error. Look in `journalctl -k` for "Machine check". This often points to unstable overclocking or memory. Bit `D` or `W`: the kernel hit a bug or warning since boot. Bit `L`: a CPU got stuck for over 20 seconds. Any of these is worth checking in the kernel log before blaming an app.

Where Linux gets it. `/sys/kernel/security/lockdown`, `mokutil --sb-state`, `/sys/module/module/parameters/sig_enforce`, `/proc/sys/kernel/*`, `/sys/kernel/security/lsm`, `/sys/class/tpm/tpm0/tpm_version_major`, `/sys/kernel/security/ima`, `/proc/sys/kernel/tainted`.

Background. A **taint flag** never clears until reboot. It does not mean the kernel is broken. It records conditions that make crash reports harder to trust.

Key names

`key-names` - Linux data

In plain words. The name of each keyboard key code 0-127, as the Linux kernel numbers them. SIV shows Windows scan-code names. For the main keys, the Linux codes match the classic PC scan codes (set 1).

What you see.

- A 4-column grid of `Scan` (code number) and `Key Name` (from the kernel's `KEY_` list: `ESC`, `A`, `LEFTSHIFT`, `KPENTER`, `VOLUMEUP`...). Codes with no name are blank.
- A note with your console keymap and X11 layout, e.g. "VC Keymap: us; X11 Layout: us".
- A note that the letters you actually get depend on the active keyboard layout (XKB, the X Keyboard Extension).

Healthy / typical values. `1 ESC`, `2-11` = the 1-0 number row, `28 ENTER`, `57 SPACE`, `85` blank, `125 LEFTMETA` (the Windows key). Japanese keys appear at 85-95 (`ZENKAKUHANKAKU`, `HENKAN`, `KATAKANAHIRAGANA`, `MUHENKAN`).

Unusual values and what they suggest. Error "input-event-codes.h not found": the `linux-api-headers` package (or your distro's kernel headers) is missing. The table cannot be built without it.

Where Linux gets it. `/usr/include/linux/input-event-codes.h`; `localectl status` for the layout note.

Background. A key code is the number for a physical key. The **layout** (US, UK, German...) turns that number into a letter. That is why the same key can type `y` or `z`.

Locales

`locales` - Linux data

In plain words. A **locale** is a bundle of language and regional rules: language, date and number format, currency, sorting, paper size. This item shows your current settings and every locale compiled on this PC.

What you see.

- `Current settings` - the `localectl status` lines (System Locale, VC Keymap, X11 Layout/Model/Options). Then each of `LANG`, `LANGUAGE`, `LC_ALL`, `LC_CTYPE`, `LC_NUMERIC`, `LC_TIME`, `LC_COLLATE`, `LC_MONETARY`, `LC_MESSAGES`, `LC_PAPER`, `LC_MEASUREMENT` that is set in the environment ("(empty)" if set to nothing).
- Table:
  - `Locale` - the name, e.g. `en_US.utf8`. Your current one is green.
  - `Description` - its title.
  - `Code` - language-country, e.g. `en-US`.
  - `Code Set` - character set (`UTF-8`, `ISO-8859-1`, `BIG5`...).
  - `Country` - territory name.
- Footer: "Showing N compiled locales (locale -a -v)".

Healthy / typical values. Author's PC: System Locale `LANG=C.UTF-8`, session `LANG en_US.UTF-8`, 502 compiled locales (nearly all of glibc's list, in both legacy and `.utf8` forms). A typical install has only two or three: `C.utf8`, `POSIX`, and your own language.

Unusual values and what they suggest. Your session `LANG` not in the table: the locale was never generated. Programs print "Cannot set LC_ALL" warnings and fall back to English/ASCII. Uncomment it in `/etc/locale.gen` and run `locale-gen`. `LC_ALL` set in your environment: it overrides every other setting, which is useful for testing and confusing if you forget it is there.

Where Linux gets it. `localectl status`, environment variables, `locale -a -v`.

Memory heap

`mem-heap` - Linux data

In plain words. A full memory summary for SRS's own process: how much it has reserved, how much is really in RAM, and its stack.

What you see.

- `/proc/self/status` - `VmPeak` (largest address space so far), `VmSize` (current address space), `VmLck`/`VmPin` (locked memory), `VmHWM` (peak RAM use), `VmRSS` (RAM in use now) split into `RssAnon` (program data), `RssFile` (mapped files) and `RssShmem`, then `VmData`, `VmStk` (stack), `VmExe` (program code), `VmLib` (library code), `VmPTE` (page tables), `VmSwap`, `HugetlbPages`, `Threads`.
- `/proc/self/smaps_rollup` - totals for the whole process: `Rss`, `Pss`, `Pss_Dirty`, `Pss_Anon`, `Pss_File`, `Shared_*`, `Private_*`, `Referenced`, `Anonymous`, `Swap` and so on.
- `Thread stacks / anonymous [stack] mapping` - the main stack's address range and its Size, Rss, Pss, Private, Anonymous and Swap lines.

Healthy / typical values. Author's PC, command-line run: VmSize 50056 kB, VmRSS 23636 kB (19108 anon + 4528 file), 1 thread, VmSwap 0. Stack 128 kB dirty.

Unusual values and what they suggest. `VmSize` far bigger than `VmRSS` is normal. Address space is only reserved, not used. A `VmSwap` above 0 for an active program means the system was short of RAM at some point.

Where Linux gets it. `/proc/self/status`, `/proc/self/smaps_rollup`, `/proc/self/smaps`.

Background. RSS (Resident Set Size) is RAM really in use. **PSS** (Proportional Set Size) splits shared pages fairly between the processes that share them. Adding up PSS for all processes gives a true total.

Memory mapping

`mem-map` - Linux data

In plain words. The raw memory map of SRS's own process: every region of its address space and what is in it.

What you see. One line per region: `Address Range` (start-end in hex), `Perm` (`r` read, `w` write, `x` execute, `p` private / `s` shared), `Offset` into the file, `Dev` (device major:minor), `Inode` (file number), and `Path` (file name, or `[heap]`, `[stack]`, `[vdso]`, or blank for anonymous memory).

Healthy / typical values. Each library appears as 4-5 lines: read-only header, `r-xp` code, read-only data, and `rw-p` writable data. Example: `/usr/lib/libc.so.6` with one `r-xp` region of about 1.6 MB.

Unusual values and what they suggest. A region that is both writable and executable (`rwxp`) in a normal program is a security smell. JIT (just-in-time) compilers (browsers, Java) use them legitimately; SRS does not.

Where Linux gets it. `/proc/self/maps`.

System metrics

`metrics` - Windows-only (Linux equivalent shown)

In plain words. Windows "system metrics" are screen and input facts (screen size, mouse buttons, border widths). Linux desktops keep these in the compositor, so SRS shows scaling variables, input-device counts and the KDE screen setup.

What you see.

- Scaling and cursor variables, if set: `XCURSOR_THEME`, `XCURSOR_SIZE`, `GDK_SCALE`, `GDK_DPI_SCALE`, `QT_SCALE_FACTOR`, `QT_SCREEN_SCALE_FACTORS`, `QT_AUTO_SCREEN_SCALE_FACTOR`, `PLASMA_USE_QT_SCALING`.
- `Pointer devices` - input devices with a mouse handler (mice, touchpads, some receivers).
- `Keyboards (with LEDs)` - devices with keyboard and LED handlers.
- `Outputs (kscreen-doctor -o)` - for each monitor: output number, connector name and an ID, enabled/connected, priority, connector type, all modes (`*` = current, `!` = preferred), geometry,

scale, rotation, overscan, VRR (variable refresh rate), RGB range, HDR (High Dynamic Range), wide colour gamut, ICC (International Color Consortium) colour profile, brightness control.

Healthy / typical values. Author's PC: 4 pointer devices, 3 keyboards. `HDMI-A-1` enabled at `1920x1080@60.00*!`, Scale 1, Vrr incapable, RgbRange Full.

Unusual values and what they suggest. `RgbRange: Limited` on a PC monitor: blacks look grey. Set it to Full in the display settings. `Vrr: incapable` on a FreeSync monitor: FreeSync is off in the monitor's menu, or the cable or port does not support it. "kscreen-doctor not available": you are not in a KDE session, so only the first lines appear.

Where Linux gets it. Environment variables; `/proc/bus/input/devices`; `kscreen-doctor -o`.

MLang conversions

`mlang` - Windows-only (Linux equivalent shown)

In plain words. MLang is a Windows library for converting text between code pages. On Linux, glibc iconv does that job. This item tests each Windows code page from Code pages for three conversions.

What you see.

- Header names your locale charset (the "ACP", Active Code Page).
- `Identifier` - code page number.
- Three boxes: can convert to UTF-16, to UTF-8, to your locale charset.
- `Codepage Description` in angle brackets, and the `iconv name`.
- Note explaining the three boxes.

Healthy / typical values. Author's PC: ACP is UTF-8, and nearly every page shows `[=][=][=]`. Page 720 shows `[_][_][_]` because glibc has no converter for it.

Unusual values and what they suggest. A page with a UTF-8 tick but no ACP tick: your locale charset is not UTF-8, so that text may lose characters on your screen.

Where Linux gets it. glibc `iconv_open()`.

DOS device names

`ms-dos` - Linux data

In plain words. Windows maps DOS names like `C:` to internal devices. Linux has no drive letters. Instead, udev makes stable names under `/dev/disk/` that point to the real device (`/dev/nvme0n1p2`, `/dev/sda1`...). This item lists them all.

What you see. One block per folder that exists, each line `name -> /dev/<device>`:

- `/dev/disk/by-label` - filesystem labels you gave the partitions.

- `/dev/disk/by-uuid` - filesystem UUIDs (Universally Unique Identifiers; what `/etc/fstab` usually uses).
- `/dev/disk/by-partlabel`, `/dev/disk/by-partuuid` - GPT (GUID Partition Table) partition names and IDs.
- `/dev/disk/by-id` - model and serial-based names.
- `/dev/disk/by-path` - names based on the physical port or bus position.

Healthy / typical values. Author's PC by-label: `LINUX -> /dev/nvme0n1p2`, `EFI -> /dev/nvme1n1p2`, `SWAP -> /dev/nvme1n1p4`, `WIN11 -> /dev/nvme1n1p1`, `Ventoy -> /dev/sdd1`, `zram0 -> /dev/zram0`.

Unusual values and what they suggest. Two partitions with the same UUID (common after cloning a disk): the system may mount the wrong one at boot. Change one with `tune2fs -U` or your filesystem's tool. A disk missing from `by-id` but present in `/dev`: it is behind a USB bridge or RAID controller that hides the serial.

Where Linux gets it. Symlinks in `/dev/disk/*`, created by udev.

Background. Device names like `sda` or `nvme1n1` can change between boots when disks are added or removed. The `by-uuid` and `by-label` names do not change, so use those in `/etc/fstab`.

N2K service

`n2k-srv` - Not present on most PCs

In plain words. SIV checks for a Windows vendor service called N2K. It has no Linux counterpart, so SRS prints a short stub.

What you see. `Status - Not present on this system` and `Checked - n/a (Windows vendor service)`. "Not present" is the expected result on Linux. Nothing is wrong.

NLS code pages

`nls-info` - Linux data

In plain words. NLS (National Language Support) is the Windows name for its code-page tables. The Linux version is the set of glibc **charmaps**: definitions of every character set glibc knows, used when building locales.

What you see.

- `ACP (locale charset)` - your current charset.
- `Charmap directory` - `/usr/share/i18n/charmaps`.
- `Charmaps` - how many.
- `Compiled locales` - where compiled locales live (`/usr/lib/locale/locale-archive`).
- A 4-column list of every charmap name, each with a box ticked if iconv can also convert it to UTF-8 under that same name.

Healthy / typical values. Author's PC: ACP UTF-8, 233 charmaps. Most are ticked (`BIG5`, `CP1252`, `EUC-JP`, `GB18030`, `SHIFT_JIS`, `UTF-8` ...). A few rare ones are not (`BS_VIEWDATA`, `SAMI`, `T.101-G2`).

Unusual values and what they suggest. Charmaps 0: the glibc locale source files were removed (some "slim" setups do this). `locale-gen` then cannot build new locales.

Where Linux gets it. Directory listing of `/usr/share/i18n/charmaps`; `iconv_open()`.

Paging files

`paging` - Linux data

In plain words. The Windows paging file is Linux **swap**: disk space (or compressed RAM) used when real RAM is full. In the GUI this item refreshes every 2 seconds.

What you see.

- Table `Swap Name`, `Type` (`partition` or `file`), `Size`, `Used`, `Priority` (higher is used first). "no swap active" if none.
- `Swap Total`, `Swap Free`, `Swap Cached` (pages in both RAM and swap), `Zswap pool` (compressed cache in RAM in front of disk swap).
- `Pages swapped in / out` since boot.
- `swappiness` (0-200: how willing the kernel is to swap rather than drop file cache), `page-cluster` (pages read per swap-in, as a power of 2), `watermark_scale_factor`, `min_free_kbytes` (RAM the kernel keeps free for emergencies).
- One line per zram device: disk size, compression algorithm, data stored, compressed size, real memory used, and compression ratio.
- `zswap.*` - each zswap setting (enabled, compressor, pool limit...).

Healthy / typical values. Author's PC: `/dev/zram0` 15.23GB at priority 100 (6.39GB used) and a 32.81GB disk swap partition at priority -1 (unused). Swappiness 150, page-cluster 0. zram uses zstd, ratio 3.63 (6.21GB of data in 1.75GB of RAM). zswap is off, which is right when zram is in use.

Unusual values and what they suggest. Disk swap in heavy use and "pages swapped in" climbing while you play: you are short of RAM, which causes stutter. Close programs or add RAM. zswap and zram both enabled: memory gets compressed twice for no gain. No swap at all: when RAM fills, the out-of-memory killer ends programs with no warning. Hibernation needs a disk swap at least as large as the RAM in use.

Where Linux gets it. `/proc/swaps`, `/proc/meminfo`, `/proc/vmstat`, `/proc/sys/vm/*`, `/sys/block/zram*/{comp_algorithm,disksize,mm_stat}`, `/sys/module/zswap/parameters`.

Background. zram is a RAM disk that compresses what is written to it. Used as swap, it lets more fit in memory. Swap **priority** decides the order: the higher one fills first.

Credentials

`passwords` - Windows-only (Linux equivalent shown)

In plain words. Windows has one Credential Manager. Linux has several **credential stores** (password keyrings). SRS only checks which ones exist and which agents are running. It never opens or reads them.

What you see.

- `Stores present` - a box and path for each: KWallet (KDE), GNOME Keyring, `pass`, GnuPG, SSH keys, KeePassXC config, Git credentials (`~/.git-credentials`).
- `Secret service / agent processes` - a box and PID for `kwalletd6`, `kwalletd5`, `gnome-keyring-daemon` (shown cut to `gnome-keyring-d`), `keepassxc`, `ssh-agent`, `gpg-agent`, counting only your own processes.

Healthy / typical values. Author's PC: KWallet, GNOME Keyring folder, GnuPG and SSH present. `kwalletd6` and `ssh-agent` running.

Unusual values and what they suggest. `~/.git-credentials` present: Git is storing passwords or tokens there as plain text. A keyring helper is safer. No secret-service process running on a desktop: browsers and chat apps keep asking for passwords, or save them unencrypted.

Where Linux gets it. Existence checks under `$HOME`; the process list in `/proc`.

Pending file renames

`pending` - Windows-only (Linux equivalent shown)

In plain words. Windows can queue file renames and deletes for the next reboot (often after an update). Linux replaces files right away. SRS answers the real question behind it: "is an update half-done, or does the system need a reboot or a config merge?"

What you see.

- `pacman lock` - "none", or `/var/lib/pacman/db.lck` present (a package transaction is running or crashed).
- `Running kernel modules` - the modules folder for the running kernel, or "missing - kernel was upgraded/removed, reboot pending".
- `Last full upgrade` - the last "starting full system upgrade" line from the pacman log.
- `Last pacman transactions` - the last 15 installed / upgraded / removed lines.
- `Unmerged config files` - every `.pacnew` / `.pacsave` under `/etc`.
- Note: updates are not checked, because that needs the network.

Healthy / typical values. Author's PC: no lock, modules folder present, last upgrade 2026-10-24, one unmerged file: `/etc/pacman.d/endeavouros-mirrorlist.pacnew`.

Unusual values and what they suggest. "missing - ... reboot pending" in red: the running kernel's modules were removed by an upgrade. New USB devices or filesystems may fail until you reboot. A lock file with no pacman running: a past update was interrupted. Make sure no pacman is running, then delete the lock and re-run the update. `.pacnew` files: the package shipped a new default config. Compare and merge it (for example with `pacdiff`), otherwise you may miss new settings.

Where Linux gets it. `/var/lib/pacman/db.lck`, `/usr/lib/modules/<release>`, `/var/log/pacman.log`, `find /etc -name '*.pacnew' -o -name '*.pacsave'`.

ProLiant monitor service

`plm-srv` - Not present on most PCs

In plain words. SIV checks for HP's ProLiant server monitoring service. Only HP ProLiant servers have it, and it has no Linux counterpart in SRS.

What you see. `Status - Not present on this system`, `Checked - n/a (HP ProLiant)`. That is the expected result on a desktop.

Windows policies

`policies` - Windows-only (Linux equivalent shown)

In plain words. Windows Group Policy sets rules for what users may do. On Linux the nearest pieces are **polkit** rules (who may do admin actions from the desktop without a password) and PAM (Pluggable Authentication Modules) resource limits.

What you see.

- `/etc/polkit-1/rules.d` - your local polkit rule files.
- `/usr/share/polkit-1/rules.d` - rule files shipped by packages.
- `/etc/security/limits.d` - extra limit files.
- Each folder shows "(empty)" or "not readable (root-only directory)" if so.
- `polkit action files` - how many actions apps have registered.
- `/etc/security/limits.conf` - each active (non-comment) line, or "(only comments)".

Healthy / typical values. Author's PC: local rules such as `10-udisks2.rules` and `49-nopasswd_global.rules`. Package rules include `50-default.rules` and `50-libvirt.rules`.

Unusual values and what they suggest. A rule like `49-nopasswd_global.rules`: admin actions are allowed without a password for some users. That is handy but weaker security, so make sure you meant it. A `limits.conf` line with a low `nofile` or `memlock` value: games, audio or Wine can fail.

Where Linux gets it. Directory listings of the polkit folders, `/usr/share/polkit-1/actions`; `/etc/security/limits.conf`.

Memory pools

`pools` - Linux data

In plain words. Windows reports paged and non-paged kernel pools. Linux has no pools by that name, so SRS shows the matching kernel and system memory totals, plus process, thread and handle counts. In the GUI it refreshes every second, and the Δ (delta) columns show the change since the last refresh.

What you see. Header: "Δ-time" since the last refresh and system up-time. Then a 3 x 3 grid of Description / Current / Δ:

- `Kernel Total` - Slab + KernelStack + PageTables (memory the kernel uses for itself).
- `Kernel Reclaimable` - slab memory the kernel can free on demand (mostly caches).
- `Kernel Unreclaim` - slab memory it cannot free.
- `Physical Total` - RAM the kernel manages.
- `Physical Available` - RAM that can be given to programs without swapping.
- `System Cache` - file cache (Cached + Buffers).
- `Commit Total` - memory promised to programs (Committed_AS).
- `Commit Limit` - the commit ceiling (RAM share + swap).
- `Shared (shmem)` - shared memory and tmpfs.
- Then `Process Count`, `Thread Count`, `Handle Count` (open file handles system-wide).

Healthy / typical values. Author's PC after 21 h: Kernel Total 3.66GB (2.56GB reclaimable), Physical 30.45GB with 19.82GB available, Commit 39.58GB of 63.26GB, 841 processes, 2194 threads, 31968 handles. A one-shot saved report always shows Δ 0 and Δ-time 00 00:00:00.

Unusual values and what they suggest. `Kernel Unreclaim` growing by hundreds of MB over hours: a kernel memory leak, often in a driver. `slabtop` shows which cache is growing. Commit Total above Commit Limit is allowed with default overcommit. But if Physical Available is also low, the out-of-memory killer is near.

Where Linux gets it. `/proc/meminfo`, `/proc/loadavg` (thread count), `/proc` (process count), `/proc/sys/fs/file-nr`.

Background. The **slab** is the kernel's own allocator for small objects (file records, network buffers). "Reclaimable" slab is cache it can drop when memory is needed.

Power policy

`power-policy` - Linux data

In plain words. The Windows power plan settings become the Linux CPU-frequency driver and governor, CPU idle settings, sleep modes and device power saving.

What you see.

- `CPU frequency (cpu0)`: `Scaling driver` (e.g. `amd-pstate-epp`, `intel_pstate`, `acpi-cpufreq`), `Governor`, `Available governors`, `Energy performance preference` (EPP), `Available EPP`, `Scaling min / max (kHz)` (the allowed range), `Hardware max (kHz)`.
- `Platform` (each shown only if present): `amd_pstate status` and `prefcore`, `Boost` (1 = turbo allowed), `intel_pstate status`, `cpuidle driver` and `governor`, `ACPI platform_profile` and its choices (laptop power modes), `Sleep states` (`freeze` = s2idle, `mem` = suspend, `disk` = hibernate), `mem_sleep` (which suspend is used, in brackets), `Hibernate (disk)` mode, `pm_async`, `PCIe ASPM policy` (Active State Power Management for PCI Express links), `USB autosuspend (s)` (-1 = off), `NMI watchdog` (a Non-Maskable Interrupt timer that detects hard CPU lockups), `laptop_mode`, `Wakeup sources armed`.

- `Power supplies` - each power-supply device with type, status and charge %, or "none (desktop on mains power, no battery)".

Healthy / typical values. Author's PC (tuned for gaming): `amd-pstate-epp`, governor and EPP `performance`, min and max both 5053000 kHz (the minimum is pinned to maximum), Boost 1, cpuidle `acpi_idle` with `teo`, `mem_sleep s2idle [deep]`, USB autosuspend -1. A wireless controller's battery appears as a power supply. A default desktop would show governor `powersave` with EPP `balance_performance` and a low minimum such as 400000-600000 kHz.

Unusual values and what they suggest. `Scaling max` well below `Hardware max`: something caps the CPU (a power tool, thermal daemon, or a laptop on battery). `Boost 0`: turbo is off, and single-thread speed will be much lower. `mem_sleep [s2idle]` on a desktop that wakes up warm or drains power: try `deep` if the firmware offers it. Driver `acpi-cpufreq` on a modern Ryzen: amd-pstate is not active. Check the BIOS CPPC (Collaborative Processor Performance Control) setting.

Where Linux gets it. `/sys/devices/system/cpu/cpu0/cpufreq/*`, `/sys/devices/system/cpu/{amd_pstate,intel_pstate,cpuidle,cpufreq}`, `/sys/firmware/acpi/platform_profile*`, `/sys/power/*`, `/sys/module/{pcie_aspm,usbcore}/parameters`, `/proc/sys/kernel/nmi_watchdog`, `/proc/sys/vm/laptop_mode`, `/sys/class/power_supply`.

Background. The **governor** decides how fast the CPU should run. **EPP** (Energy Performance Preference) is a hint to the CPU's own firmware about speed versus power. With amd-pstate-epp or intel_pstate in active mode, EPP does most of the work.

Power scheme

`power-scheme` - Linux data

In plain words. The Windows "active power scheme" becomes the profile chosen in power-profiles-daemon or tuned, and the governor and EPP actually set on every CPU.

What you see.

- `power-profiles-daemon` - current profile (`performance`, `balanced`, `power-saver`) or "not running".
- `tuned` - "Current active profile: ..." or "not running".
- `ACPI platform_profile` - if the firmware has one.
- `Governor` - one line per distinct value: "X on N of M CPUs".
- `EPP` - the same for energy preference.
- `Available profiles (powerprofilesctl list)` - the daemon's list. The active profile has `*`, and each shows the driver behind it.

Healthy / typical values. Author's PC: power-profiles-daemon `performance`, tuned `cachyos-gaming`, governor and EPP `performance` on 16 of 16 CPUs. Here tuned is the driver behind each power-profiles-daemon profile.

Unusual values and what they suggest. Two governor lines (for example `performance on 8 of 16` and `powersave on 8 of 16`): two tools are fighting, or a script set only some cores. Pick one power

manager. Both daemons "not running": nothing manages power, and you get whatever the kernel default is.

Where Linux gets it. `powerprofilesctl get` / `list`, `tuned-adm active`, `/sys/firmware/acpi/platform_profile`, `/sys/devices/system/cpu/cpu*/cpufreq/{scaling_governor,energy_performance_preference}`.

Privileges

`priv` - Linux data

In plain words. Windows privileges (SeDebugPrivilege and so on) correspond to Linux **capabilities**: split-up pieces of root's power. A normal program has none of them in effect. This item shows SRS's own.

What you see.

- `User ... (uid euid gid egid)` - real and effective user and group IDs, and the program file.
- `NoNewPrivs` (1 = this process can never gain rights, even through setuid programs) and `Seccomp` (0 off, 1 strict, 2 filter = a system-call sandbox).
- Table of all 41 capabilities with five boxes:
  - `Effect` (effective) - in force right now.
  - `Permit` (permitted) - may be switched on.
  - `Inherit` (inheritable) - passed to programs it starts.
  - `Bound` (bounding set) - the upper limit; anything not here can never be gained.
  - `Ambient` - kept across starting normal programs.
- Capabilities in effect are shown in green.

Healthy / typical values. Author's PC as user: uid/euid 1000, NoNewPrivs 0, Seccomp 0. Every capability is ticked only under `Bound`, so the process has no special rights.

Unusual values and what they suggest. `Effect` ticks while running as a normal user: SRS was given file capabilities or started by a privileged service. Inside a Flatpak or container you may see `NoNewPrivs 1`, `Seccomp 2` and a reduced bounding set. That is the sandbox working.

Where Linux gets it. `/proc/self/status` (`CapEff`, `CapPrm`, `CapInh`, `CapBnd`, `CapAmb`, `NoNewPrivs`, `Seccomp`); `getuid()` and friends.

Background. Capabilities let a program do one root-only thing without full root. For example, `CAP_NET_BIND_SERVICE` lets a web server use port 80.

Processes

`processes` - Linux data

In plain words. Every running process, like Task Manager's Details tab.

What you see. One row per process, sorted by name:

- `Parent` - the PID of the process that started it, in brackets.
- `PID` - process ID.
- `Threads` - thread count.
- `State` - `R` running, `S` sleeping, `D` waiting on disk/device (uninterruptible), `Z` zombie (finished, parent hasn't collected it), `T` stopped, `I` idle kernel thread.
- `RSS` - RAM in use.
- `Account Name` - owner. root is shown in red.
- `Image Name` - program file name (or the process name if the program file can't be read).
- `Image Filename` - full program path, or `-` if hidden (other users' processes when not root, and kernel threads). The header also shows "Returned N of <pid_max>". pid_max is the highest PID number allowed, not a process limit.

Healthy / typical values. Author's PC: 841 processes. For example `akonadiserver` with 28 threads and 13.00MB, `Xwayland` with 8 threads and 10.94MB. Names in parentheses such as `(sd-pam)` are processes that renamed themselves.

Unusual values and what they suggest. Many `D` processes: something is stuck waiting on a disk, USB device or network share. A failing drive or dropped share is likely. Lots of `Z` zombies from one parent: that parent is buggy. A program running from `/tmp` or a hidden folder that you do not recognise: investigate.

Where Linux gets it. `/proc/<pid>/stat`, owner of `/proc/<pid>`, `readlink /proc/<pid>/exe`, `/proc/sys/kernel/pid_max`.

Product IDs

`product-ids` - Windows-only (Linux equivalent shown)

In plain words. Windows has a product ID tied to its licence. Linux has no licence keys, but it does have IDs for the installation, the current boot, and the hardware.

What you see.

- `Machine ID` - the unique ID of this installation (from `/etc/machine-id`). SRS shows only the first 8 characters and masks the rest.
- `Boot ID` - a random ID made fresh on every boot.
- `OS ID`, `Build ID`, `Image ID` - from `/etc/os-release`, where set.
- `System Vendor`, `Product Name`, `Product SKU` (Stock Keeping Unit, a model variant code) - from the firmware's DMI (Desktop Management Interface) tables.
- `Product Serial` - always masked. Without root data the line reads `Product Serial / UUID - Needs root: Menu > Run as root (srs-root)`.
- Note: "Linux has no product activation keys".

Healthy / typical values. Author's PC: OS ID `cachyos`, Build ID `rolling`, vendor `ASUS`, Product Name "System Product Name" and SKU "SKU". Those last two are the board maker's unfilled placeholder text, which is common on retail desktop boards.

Unusual values and what they suggest. Two PCs with the same Machine ID (common when a disk image was cloned): DHCP (Dynamic Host Configuration Protocol) servers, journald and some licence tools treat them as one machine. Make a new one with `systemd-machine-id-setup` after deleting the old file. Do not post your full machine ID online.

Where Linux gets it. `/etc/machine-id`, `/proc/sys/kernel/random/boot_id`, `/etc/os-release`, `/sys/class/dmi/id/*`.

Product keys

`products` - Windows-only (Linux equivalent shown)

In plain words. SIV can show Windows and Office product keys. Linux has no product keys, so SRS shows a stub.

What you see. `Status - Windows-only item in SIV`, `Linux equivalent - none`. That is expected.

Font ranges

`ranges` - Linux data

In plain words. Which Unicode characters one font contains, as blocks of code points. By default it checks your monospace (terminal) font.

What you see.

- `Font` - the family fontconfig picked; `File` - the font file; `Pattern` - what was asked for (default `monospace`).
- Table:
  - `#` - row number.
  - `Start`, `End` - first and last code point, e.g. `U+0020` to `U+007E`.
  - `Glyphs` - how many code points the range covers (End - Start + 1).
  - `Characters` - a sample: the first printable character of the range.
- Footer: "N ranges, M characters supported. Open as ranges: for another font".
- Argument form: `ranges:DejaVu Sans` checks that font. Letters, digits, spaces and `_ . : - +` are allowed in the name.

Healthy / typical values. Author's PC: monospace is FiraCode Nerd Font Mono, 173 ranges, 12438 characters. It starts with `U+0020-U+007E` (95, basic Latin, sample `!`) and `U+00A0-U+017E` (223, sample `¡`), and ends with a 6896-character private-use block of Nerd Font icons. `ranges:DejaVu Sans` has `U+00A0-U+02E9` (586) as its second range.

Unusual values and what they suggest. A terminal font with only 1-3 ranges: box-drawing, accents or icons will fall back to another font and may look misaligned. Error "fc-match returned nothing": fontconfig is broken or has no fonts.

Where Linux gets it. `fc-match -f '%{family[0]} %{file} %{charset}' <pattern>`.

Background. A **code point** is Unicode's number for a character (`U+0041` is "A"). A font holds drawings (glyphs) for only some code points. Fontconfig fills the gaps from other fonts.

RDP session info

`rdp-info` - Linux data

In plain words. On Windows this shows Remote Desktop (RDP, Remote Desktop Protocol) sessions. On Linux, every login (local or remote) is a systemd-logind session. SRS lists them and checks the common remote-desktop and remote-login services.

What you see.

- Table per session: `SID` (session ID), `User Name`, `Type` (`wayland`, `x11`, `tty`, `unspecified`), `Class` (`user` = a real login, `greeter` = login screen, `manager` = the per-user service manager), `State` (`active`, `online`, `closing`), `Remote` (yes/no), `Remote Host`, `Service` (what created it: `sddm`, `sshd`, `login`, `systemd-user`), `TTY`.
- `Remote desktop / remote login services` - "active-state / enabled-state" for `sshd`, `xrdp`, `gnome-remote-desktop`, `wayvnc`, `x11vnc`, `vncserver@:1`, and KDE's `krdp` user service. Active ones are shown in orange so you notice them.

Healthy / typical values. Author's PC: session 1 `user wayland user active no - sddm-autologin tty3`, session 2 `user unspecified manager active no - systemd-user`. `sshd.service active / enabled`; all others `inactive / not-found`.

Unusual values and what they suggest. A session with `Remote yes` and a host you do not know: someone is logged in from another machine. Check it, and end it with `loginctl terminate-session <SID>` if needed. A remote-desktop service active that you never set up: turn it off.

Where Linux gets it. `loginctl list-sessions`, `loginctl show-session <id> -p Name -p Type ...`, `systemctl is-active` / `is-enabled`.

Remote registry paths

`reg-path` - Windows-only (Linux equivalent shown)

In plain words. SIV lists which registry paths remote users may read. Linux has no registry and no remote registry, so SRS shows a stub.

What you see. `Status - Windows-only item in SIV`, `Linux equivalent - none`. For remote administration services, see Remote registry service.

Remote registry service

`reg-srv` - Windows-only (Linux equivalent shown)

In plain words. The Windows Remote Registry service lets another PC read and change your settings. On Linux, the same risk comes from remote login and remote admin services. SRS checks the common ones.

What you see. Table `Unit`, `Active`, `Enabled` for `sshd.service`, `sshd.socket`, `cockpit.socket` (web admin console), `webmin.service`, `smb.service` (Samba file sharing), `nfs-server.service`, `libvirtd.service` (virtual machines, which can be managed remotely). Active units are shown in orange. Missing ones say "not installed" or `not-found`.

Healthy / typical values. Author's PC: `sshd`, `cockpit.socket`, `smb`, `nfs-server` and `libvirtd` are all active and enabled. That is a lot for a desktop, but each was chosen on purpose.

Unusual values and what they suggest. Anything active that you did not set up yourself. Each one is a door into the PC. Disable unused ones with `systemctl disable --now <unit>`, and check your firewall for the rest.

Where Linux gets it. `systemctl is-active` / `is-enabled`.

Text converter render format

`render-format` - Windows-only (Linux equivalent shown)

In plain words. SIV lists the Windows text-converter render formats (an Office/WordPad document-conversion feature). Linux has nothing comparable, so SRS shows a stub.

What you see. `Status - Windows-only item in SIV`, `Linux equivalent - none`. That is expected.

Intel RST service

`rst-srv` - Not present on most PCs

In plain words. Intel RST (Rapid Storage Technology) is Intel's Windows RAID (Redundant Array of Independent Disks) and NVMe (Non-Volatile Memory Express) caching driver and service. On Linux, the same disks are run by the md RAID driver (`mdadm`, which understands Intel's IMSM, Intel Matrix Storage Manager, format), and Intel VMD (Volume Management Device) by the `vmd` module. This item checks for those.

What you see.

- `Windows service` - "not applicable on Linux - showing the Linux analogue".
- Units `mdmonitor.service`, `mdadm.service` - active and enabled state.
- Modules `md_mod`, `raid0`, `raid1`, `raid456`, `dm_raid`, `vmd` - loaded / not loaded.
- `/proc/mdstat` - its contents (arrays and their health), or "not present".

Healthy / typical values. Author's PC (AMD, no RAID): all modules not loaded, `mdmonitor.service inactive static`, `/proc/mdstat` not present. On an Intel laptop with VMD turned on in the BIOS, `vmd` must be loaded or the NVMe drive is invisible.

Unusual values and what they suggest. In `/proc/mdstat`, `[U_]` instead of `[UU]`: one disk of a mirror has failed or dropped out. Replace it soon. An NVMe drive not found by the Linux installer on an Intel PC: VMD/RST is on in the BIOS. Switch the storage mode to AHCI (Advanced Host Controller Interface), or make sure `vmd` is in the initramfs.

Where Linux gets it. `systemctl`, `/sys/module/<name>`, `/proc/mdstat`.

Scheduling

`scheduling` - Linux data

In plain words. How the kernel shares CPU time between programs. Windows has one fixed scheduler with a few tweaks. Linux has many settings, can run alternative schedulers, and on newer kernels can load one as a BPF program (sched_ext).

What you see.

- `Kernel build` - the kernel's build string (shows the preemption model, e.g. `PREEMPT_DYNAMIC`).

- `sched_ext state` - `enabled` + the loaded scheduler's name, or `disabled`; then `switch_all` (1 = all tasks use it), `enable_seq` (how many times one was loaded), `nr_rejected`. If the kernel lacks it: "not available (CONFIG_SCHED_CLASS_EXT off)".
- `scx_loader.service / scx.service` - whether a sched_ext loader service is active.
- `Scheduler sysctls` - every `/proc/sys/kernel/sched_*` setting, plus `timer_migration` and `numa_balancing`. Which ones appear depends on the kernel and its patches.
- `This process`: `Policy` (SCHED_OTHER normal, SCHED_FIFO/RR real-time, SCHED_BATCH, SCHED_IDLE, SCHED_EXT), `Nice` (-20 high priority to 19 low), `Time slice (sched_rr_get_interval)`, `Autogroup` (the group and its nice value), `preempt=` from the command line.

Healthy / typical values. Author's PC: sched_ext enabled with `lavd_1.1.2...` (LAVD, the Latency-criticality Aware Virtual Deadline scheduler), switch_all 1, nr_rejected 0, `scx_loader active`. Sysctls include `sched_autogroup_enabled 1` and the BORE (Burst-Oriented Response Enhancer) patch set (`sched_bore 1`, `sched_burst_*`). This process: SCHED_OTHER, nice -4 (set by a priority daemon), `preempt=full`. A stock kernel shows sched_ext disabled and fewer `sched_*` entries. The time slice reads 0.000 ms when a sched_ext scheduler handles normal tasks. Under the default scheduler it shows a few ms.

Unusual values and what they suggest. `nr_rejected` above 0: the sched_ext scheduler refused tasks. `enable_seq` climbing while you are not switching schedulers: the BPF scheduler keeps crashing and the kernel falls back to the default. Check `journalctl -k` for "sched_ext". Programs at SCHED_FIFO that you did not set: a misbehaving one can freeze the desktop.

Where Linux gets it. `uname`, `/sys/kernel/sched_ext/*`, `systemctl is-active`, `/proc/sys/kernel/sched_*`, `sched_getscheduler()`, `getpriority()`, `sched_rr_get_interval()`, `/proc/self/autogroup`, `/proc/cmdline`.

Background. Nice is a process's politeness: higher nice means it gives way to others. **Preemption** decides how quickly the kernel can interrupt a running task. `full` gives the lowest latency (good for games and audio).

Driver services

`services-drv` - Linux data

In plain words. Windows lists kernel drivers as "driver services" that are running or stopped. On Linux, each bus (PCI, USB, HID = Human Interface Device, I2C = a two-wire chip-to-chip bus...) has registered drivers. A driver is "Running" when it is bound to at least one device.

What you see.

- `Bus` - the bus type (`pci`, `usb`, `hid`, `i2c`, `platform`, `hdaudio`, `cpu`...).
- `Driver Name`.
- `Devices` - how many devices it is bound to.
- `State` - `Running` (1 or more devices) or `Stopped` (registered, no device).
- `Module` - the module that provides it, or `(built-in)`.
- Footer: "N drivers, M bound to devices (Running)".

Healthy / typical values. Author's PC: 191 drivers, 63 running. `cpu processor 16 Running (built-in)`, `hid hid-generic 8 Running`, `hid razermouse 3 Running`, `hdaudio snd_hda_codec_alc882 1 Running`. Many built-in drivers for chips this PC does not have show `0 Stopped`, which is normal.

Unusual values and what they suggest. The driver for a device you own shows `Stopped` and the device does not work: it is bound to another driver, or the driver failed to probe it. Check `lspci -k` / `lsusb -t` and `journalctl -k`. Two drivers for the same hardware both registered (e.g. `nouveau` and `nvidia`): only one should be bound.

Where Linux gets it. `/sys/bus/*/drivers/*` (bound devices are the links inside), `/sys/bus/*/drivers/*/module`.

Win32 services

`services-w32` - Linux data

In plain words. Windows services are systemd **service units** on Linux. There are two sets: system services (run as root or service accounts) and user services (run inside your login).

What you see. Two tables, `System services` then `User services`, each with:

- `Service Name` - the unit name.
- `Load` - `loaded`, `not-found` (referenced but no file), `masked` (blocked).
- `Active` - `active`, `inactive`, `failed` (red), `activating`.
- `State` - the detail: `running`, `exited` (a one-shot job that finished), `dead`, `failed`.
- `Description`.
- Footer for each: "N units, A active, F failed".

Healthy / typical values. Author's PC: 291 system units (123 active, 1 failed), 156 user units (97 active, 0 failed). Examples: `bluetooth.service loaded active running`, `alsa-restore.service loaded active exited`, `auditd.service masked inactive dead`.

Unusual values and what they suggest. Any `failed` unit: read why with `systemctl status <unit>` and `journalctl -u <unit> -b`. On the author's PC a scheduled rootkit-check job is failed. `not-found` units: something still refers to a service that was uninstalled. Usually harmless, but it can slow boot if it is waited for.

Where Linux gets it. `systemctl list-units --type=service --all` (and `--user`).

Background. `inactive dead` is normal for a service that only starts on demand or on a timer. It is not an error.

Sessions

`sessions` - Linux data

In plain words. Who is logged in, how, and from where: local logins, SSH (Secure Shell) connections and Samba (Windows file sharing) clients.

What you see.

- `Login sessions (loginctl list-sessions)` - `Session`, `UID`, `User`, `Seat` (`seat0` = this PC's screen and keyboard), `Leader` (PID that started it), `Class`, then `TTY / Idle / Since`.
- `Users with sessions (loginctl list-users)` - UID, name, linger (yes = their services keep running after logout), state.
- `Inbound SSH connections` - established TCP connections to port 22, or "none".
- `SMB sessions (smbstatus -b)` - from the srs-root cache, or "Needs root: Menu > Run as root (srs-root)".

Healthy / typical values. Author's PC: session 1 on `seat0` `tty3` (the desktop), session 2 with class `manager` (the user service manager), user 1000 `no active`, no SSH connections. Samba shows its version and empty client tables.

Unusual values and what they suggest. An SSH connection or SMB client you do not recognise. Several stale `closing` sessions for your user: a program is still running after a logout, often a leftover background app.

Where Linux gets it. `loginctl`, `ss -Htn state established '( sport = :22 )'`, `smbstatus -b` (run by srs-root).

Setup directories

`setup` - Linux data

In plain words. Windows lists its setup and install folders. The Linux version: what is in `/boot`, where the EFI System Partition is mounted, and the package manager's own folders.

What you see.

- `Boot Dir /boot`, then each kernel (`vmlinuz-*`), initramfs (`initramfs-*`), microcode (`*-ucode.img`), and the `grub`, `EFI` or `loader` folders.
- `EFI System Partition` - where the ESP (EFI System Partition) is mounted (`/efi`, `/boot/efi` or `/boot`), or "not mounted".
- `Shared Dir` - `/usr/share`.
- pacman settings from `/etc/pacman.conf`, if set: `RootDir`, `DBPath`, `CacheDir`, `LogFile`, `GPGDir`, `HookDir`, `Architecture`, `ParallelDownloads`.
- `Installed packages` - count.
- `Last full upgrade` - from the pacman log.

Healthy / typical values. Author's PC: `/boot` holds three kernels with their initramfs and fallback images, plus `amd-ucode.img` and `grub`. ESP on `/boot/efi`, Architecture `auto`, ParallelDownloads 10, 13538 packages.

Unusual values and what they suggest. An initramfs older than its kernel, or a kernel with no initramfs: the image was not rebuilt after an update, and that kernel may not boot. Run `mkinitcpio -P` (or your distro's tool). No microcode image on a GRUB system: CPU bug fixes from AMD or Intel are not loaded early. ESP "not mounted": boot-loader updates cannot be written.

Where Linux gets it. Directory listing of `/boot`, `/proc/mounts`, `/etc/pacman.conf`, `pacman -Qq`, `/var/log/pacman.log`.

Setup API logging

`setup-api` - Windows-only (Linux equivalent shown)

In plain words. Windows SetupAPI logs every device-driver install. On Linux, udev handles new devices using rule files, and logs to the journal. SRS shows its log level and rule counts.

What you see.

- `udev_log (/etc/udev/udev.conf)` - the log level, or "default (info)".
- `/etc/udev/rules.d` - your local rule files (count).
- `/usr/lib/udev/rules.d` - rule files from packages (count).
- `/run/udev/rules.d` - temporary rules made at runtime (count).
- `udevadm version`.

Healthy / typical values. Author's PC: default log level, 41 local rules, 290 package rules, 1 runtime rule, udev 262.

Unusual values and what they suggest. Many local rules: each one can rename, re-permission or run a script for a device. If a device behaves oddly, look here first. To watch devices being set up live, use `udevadm monitor`.

Where Linux gets it. `/etc/udev/udev.conf`, directory counts, `udevadm --version`.

Driver signing

`signing` - Linux data

In plain words. Like Windows driver signing: are kernel modules signed, and does the kernel insist on it?

What you see.

- `module.sig_enforce` - `Y` (unsigned modules refused), `N` (allowed, but taint the kernel), or `-` (signing not built into the kernel).
- `Loaded modules` - count.
- `Unsigned (E)`, `Out-of-tree (O)`, `Proprietary (P)` - how many loaded modules carry each taint letter.
- `Tainting modules` - each module with its taint letters.
- `Kernel signing configuration` - the `CONFIG_MODULE_SIG*`, `CONFIG_SYSTEM_TRUSTED_KEYS` and `CONFIG_MODULE_HASHES` lines from the running kernel's config.
- Argument form `signing:<module>`: `Module`, `Signer` (or "(unsigned)"), `Hash` algorithm.

Healthy / typical values. Author's PC: `sig_enforce N`, 237 modules, 10 unsigned, 10 out-of-tree, 0 proprietary. The tainting ones are out-of-tree extras such as `ryzen_smu`, `vboxdrv`, `vmmon`, `zenergy`, each `OE`. Kernel config: `CONFIG_MODULE_SIG=y`, `CONFIG_MODULE_SIG_ALL=y`, SHA-512. `signing:amdgpu` shows `Signer: Build time autogenerated kernel key`, `Hash: sha512`.

Unusual values and what they suggest. With Secure Boot on and lockdown active, unsigned modules (DKMS builds such as VirtualBox) will not load. Sign them with your own MOK (Machine Owner Key), or turn enforcement off. `Proprietary (P)` above 0: a closed driver is loaded.

Where Linux gets it. `/sys/module/module/parameters/sig_enforce`, `/proc/modules` (taint column), `/proc/config.gz`, `modinfo -F signer` / `sig_hashalgo`.

Background. "Build time autogenerated kernel key" means the kernel builder made a one-time key and signed all of that kernel's own modules with it. Modules built later (DKMS) cannot be signed with that key.

Installed software

`software` - Linux data

In plain words. Windows "Programs and Features" becomes the package manager's list. SRS lists every pacman package and every Flatpak app.

What you see.

- Table `Package`, `Version`, `Origin`. `repository` = from a configured repo. `foreign (AUR/local)` = not in any configured repo (built from the AUR, the Arch User Repository, installed from a file, or dropped from its repo).
- Footer: "N pacman packages (E explicitly installed, F foreign)".
- `Flatpak (flatpak list)` - application ID, version, branch, installation (`system` or `user`), or "none".

Healthy / typical values. Author's PC: "13538 pacman packages (7211 explicitly installed, 3585 foreign)", and a long Flatpak list (`... stable system`). A typical desktop has 800-1500 packages.

Unusual values and what they suggest. A package marked foreign that you thought came from the repos: it was dropped from the repo and will never get updates, so look for a replacement. A large foreign count: each AUR package has to be rebuilt by you when its libraries change.

Where Linux gets it. `pacman -Q`, `pacman -Qqm` (foreign), `pacman -Qqe` (explicit), `flatpak list`.

Startup programs

`startup` - Linux data

In plain words. Like the Windows Startup folder and Task Manager's Startup tab: what starts when you log in, and which systemd units are enabled at boot.

What you see.

- `XDG autostart (user) ~/.config/autostart` and `XDG autostart (system) /etc/xdg/autostart` - each `.desktop` file with a box (ticked = will start), the file name, the app name, and its command. Notes in brackets:
  - `[KDE;GNOME;...]` - `OnlyShowIn`: it starts only on those desktops. The box is empty if yours is not listed.
  - `[overridden by user file]` - a file with the same name in your own folder replaces the system one.
  - The box is also empty for `Hidden=true` or `X-GNOME-Autostart-enabled=false`.
- `Plasma login scripts ~/.config/plasma-workspace/env` - scripts KDE runs before the desktop starts, or "(none)".
- `Enabled systemd units (system)` and `(user)` - every enabled unit file with its state and preset.

Healthy / typical values. Author's PC: user autostart for CoolerControl, conky, JamesDSP, SRS's own tray (`srs-gui --gui-tray`) and others. System autostart includes `baloo_file [KDE;GNOME;Unity;XFCE]` (ticked under KDE) and `at-spi-dbus-bus [GNOME;Unity;]` (empty under KDE). User timers include `arch-update.timer`.

Unusual values and what they suggest. A ticked entry you do not recognise: check its command. Malware and adware often hide in `~/.config/autostart`. Two entries starting the same tray app: you get two icons. Slow logins: look for `sleep N &&` commands and heavy apps here.

Turning items off and on (2.2). In the main window the Startup programs tab opens on **Report**: this item as text, as above. **Manage**, next to it, is every entry with a switch, Undo, Restore default and Add to startup. Nothing is deleted. See the user guide, section 6.9.

Where Linux gets it. `~/.config/autostart/*.desktop` (or `$XDG_CONFIG_HOME/autostart`), `/etc/xdg/autostart/*.desktop`, `~/.config/plasma-workspace/env`, `systemctl list-unit-files --state=enabled` (and `--user`).

System

`system` - Linux data

In plain words. The overview panel: what Linux you run, on what hardware, since when, and how the desktop is set up. It is the Linux version of SIV's Windows panel. General Info shows the same page: the classic main window's **Linux** button opens it, and in the main window it is the first tab of the **Operating system** page.

What you see.

- Top block (four lines):
  - Distribution name, version (or "Build rolling"), kernel release and architecture.
  - `ID` and `ID_LIKE` from os-release, the kernel build string, and the `Hostname`.
  - `InstallDate` - the oldest believable install time, with its Unix time and where it came from: the pacman install date of the `filesystem` or `base` package, the `/etc/machine-id` file time, or the first line of the pacman log, whichever is oldest.
  - `Product Type Linux`, `Init` (the systemd version, or "unknown (not systemd)") and `Chassis` (Desktop, Laptop, Notebook, Mini Tower... from SMBIOS, the System Management BIOS tables).
- `Linux Desktop Session` - desktop name, session type (`wayland`/`x11`), and the `WAYLAND_DISPLAY` / `DISPLAY` values.
- `Root Directory /`, `System Directory /usr/bin`, `Library Directory /usr/lib`, `Boot Directory /boot` - the Linux counterparts of `C:\`, `System32` and so on.
- `Kernel Command Line`.
- One line per logical CPU: `[CPU-n vendor microcode]`, model name, maximum clock, and Family / Model / Stepping.
- Power line: "Mains Power" or "Battery Power", and "Battery Present" or "No Battery". Then `Time Zone` with abbreviation and offset from UTC (Coordinated Universal Time), the `Clocksource`, and the timer `Resolution`.
- `Up-time` (days hh:mm:ss), `Booted` (date and time), `Load` (the raw `/proc/loadavg`: 1, 5 and 15-minute load averages, running/total threads, last PID).
- `User` (name and uid), `Locale`, `Shell`.
- `System BIOS Date` - BIOS date, vendor, version, and `Firmware UEFI` or `Legacy BIOS`.
- One block per connected monitor: `Primary Desktop Display #1` / `Desktop Display #n` with connector name, `enabled`/`disabled`, the preferred mode, and its `Kernel driver`.
- Footer: the board's maker, model and BIOS.

Healthy / typical values. Author's PC: "CachyOS Build rolling Kernel 7.3.0-273-tkg-bore-llvm x86_64", ID `cachyos` like `arch`, Hostname MYPC, installed 2026-03-09, systemd 262, Chassis Desktop. KDE on Wayland. 16 lines of `AMD Ryzen 7 7800X3D 8-Core Processor 5.05GHz (Family 25 Model 97 Stepping 2)` with microcode `0xa60120c`. Time zone America/New_York (EDT -0400), clocksource tsc, resolution 1 ns. BIOS 09/18/2026 American Megatrends 1805, UEFI. Displays `DP-3` 2560x1440 and `HDMI-A-1` 1920x1080, both on `amdgpu`.

Unusual values and what they suggest.

- Microcode differs between CPU lines: a microcode update reached only some cores. This is rare; update the BIOS or the `*-ucode` package.
- Maximum clock far below the CPU's rated boost: boost is off or the CPU frequency driver did not load. See Power policy.
- `Clocksource hpet` or `acpi_pm` instead of `tsc`: the kernel judged the TSC unstable. Timing and game frame pacing get worse. See System timers.
- "Battery Present" on a desktop: a wireless mouse, headset or controller reports its battery as a power supply. That is harmless.
- A monitor's `Kernel driver` reads `simpledrm` or `efifb`: the real GPU driver did not load.

Where Linux gets it. `/etc/os-release`, `uname()`, `gethostname()`, `/var/lib/pacman/local/*/desc`, `/etc/machine-id` time, `/var/log/pacman.log`, `systemctl --version`, `/sys/class/dmi/id/{chassis_type,bios_*}`, environment, `/proc/cmdline`, `/proc/cpuinfo`, `/sys/devices/system/cpu/cpu*/cpufreq/cpuinfo_max_freq`, `/sys/class/power_supply`, `/etc/localtime`, `/proc/uptime`, `/proc/stat` (btime), `/proc/loadavg`, `/sys/class/drm/card*-*/{status,enabled,modes}`.

Background. Load average is the average number of tasks running or waiting to run. On a 16-thread CPU, a load of 3.6 means the CPU is mostly idle. A load steadily above 16 means work is queuing.

System timers

`sys-tim` - Linux data

In plain words. Which hardware clock the kernel uses to keep time, how fine its timers are, and how the timer tick is set up. Windows exposes a little of this (the timer resolution); Linux shows it all.

What you see.

- `Current clocksource` (green) and `Available clocksources` - e.g. `tsc hpet acpi_pm`.
- `hrtimer resolution` - high-resolution timer precision (1 ns is normal).
- `Coarse (tick) resolution` - one timer tick (1.000 ms = 1000 Hz kernel; 4.000 ms = 250 Hz).
- `timer_migration` - 1 = timers may move to busy CPUs so idle ones can sleep.
- `TSC flags` - which of `tsc`, `constant_tsc` (rate does not change with CPU speed), `nonstop_tsc` (keeps counting in deep sleep), `tsc_deadline_timer` the CPU reports.
- `Clock event devices` - the per-CPU timer device (e.g. "lapic 16 CPU clockevent devices"), then the `broadcast` device used when CPUs sleep deeply.
- `Kernel timer configuration` - `CONFIG_HZ`, `CONFIG_NO_HZ*`, `CONFIG_HIGH_RES_TIMERS`, `CONFIG_TICK_*`, `CONFIG_HPET*` lines from the running kernel.
- `nohz / tsc on cmdline` - shows the kernel command line if it contains `nohz` or `tsc=` options, otherwise `<none>`.

Healthy / typical values. Author's PC: clocksource `tsc` (available `tsc hpet acpi_pm`), 1 ns hrtimers, 1.000 ms tick (`CONFIG_HZ=1000`), flags `tsc constant_tsc nonstop_tsc`, `lapic` on all 16 CPUs, broadcast `hpet`, `NO_HZ_IDLE`. The command line contains `tsc=reliable clocksource=tsc`.

Unusual values and what they suggest. Current clocksource `hpet` on a modern CPU: the kernel found the TSC unreliable (look for "clocksource: Marking clocksource 'tsc' as unstable" in `journalctl -k`). Reading the time becomes much slower, and games and benchmarks suffer. Overclocking or BIOS bugs can cause it. Forcing `tsc=reliable` hides the symptom but not the cause.

Where Linux gets it. `/sys/devices/system/clocksource/clocksource0/*`, `clock_getres()`, `/proc/sys/kernel/timer_migration`, `/proc/cpuinfo` flags, `/sys/devices/system/clockevents/*`, `/proc/config.gz`, `/proc/cmdline`.

Background. The **TSC** (Time Stamp Counter) is a counter inside the CPU that ticks at a fixed rate. It is the fastest clock to read. HPET (High Precision Event Timer) and ACPI PM timer are slower chipset clocks kept as backups.

System windows

`sys-win` - Windows-only (Linux equivalent shown)

In plain words. SIV lists every top-level window Windows knows about. On Linux under X11 or XWayland, SRS can list windows with `wmctrl`. Native Wayland apps cannot be listed: Wayland deliberately gives programs no global window list.

What you see.

- A header row and one line per window from `wmctrl -l -G -x`: `Window ID`, `Desk` (virtual desktop, -1 = all), `X`, `Y`, `W`, `H` (position and size), `Class` (app class), `Host`, `Title`.
- Note: native Wayland clients are not listed.
- Without `DISPLAY` set: a note that wmctrl needs X11/XWayland.

Healthy / typical values. Author's PC (KDE Wayland): one window, conky's (`Conky.Conky`), because conky is the only X11 app running. Everything else is native Wayland.

Unusual values and what they suggest. "wmctrl returned nothing" with X11 apps open: the `wmctrl` package is not installed.

Where Linux gets it. `wmctrl -l -G -x`.

Thread windows

`tid-win` - Windows-only (Linux equivalent shown)

In plain words. SIV lists windows by the thread that owns them. On Linux, SRS lists X11/XWayland windows with the process that owns them.

What you see. `Window ID`, `PID`, `Process` (the process name from `/proc/<pid>/comm`), `Title`. The same Wayland limit as System windows applies.

Healthy / typical values. Author's PC: `0x01600002 0 - conky (MYPC)`. PID 0 and `-` mean the window did not publish its process ID (`_NET_WM_PID`). Some lightweight apps, conky among them, don't.

Where Linux gets it. `wmctrl -l -p`, `/proc/<pid>/comm`.

Time

`time` - Linux data

In plain words. The current time in every useful form, time-zone and NTP (Network Time Protocol) sync status, and the motherboard's real-time clock (RTC).

What you see.

- `Local Time`, `UTC`, `Epoch` (seconds since 1970-01-01 UTC).
- `Time Zone`, `Abbreviation / Offset` (e.g. `EDT UTC-0400 daylight saving time`).
- `Day of year / ISO week`.
- `timedatectl show` - `Timezone`, `LocalRTC` (no = the RTC keeps UTC), `CanNTP`, `NTP` (sync on), `NTPSynchronized`, `TimeUSec`, `RTCTimeUSec`.
- `Real-time clock (/sys/class/rtc/rtc0)` - `name`, `date`, `time`, `since_epoch`, `hctosys` (1 = system time was set from this clock at boot), `max_user_freq`.
- `System Up-time`, `Boot Time`.

Healthy / typical values. Author's PC: America/New_York, EDT, `LocalRTC no`, `NTP yes`, `NTPSynchronized yes`, RTC `rtc_cmos`, with RTC time and UTC matching to the second.

Unusual values and what they suggest. `LocalRTC yes`: the clock stores local time, usually for Windows dual-boot. It causes one-hour jumps at daylight-saving changes. Setting Windows to UTC instead is better. RTC time far from UTC right after boot, or a date in 2000 or so: the CMOS battery is flat. `NTPSynchronized no` for a long time: no network time. Check `systemd-timesyncd` or chrony.

Where Linux gets it. `time()`, `localtime_r()`, `/etc/localtime` or `$TZ`, `timedatectl show`, `/sys/class/rtc/rtc0/*`, `/proc/uptime`, `/proc/stat`.

Times

`times` - Linux data

In plain words. All of Linux's clocks side by side, and how well network time sync is doing.

What you see.

- `Clock` table, each with value in seconds and resolution:
  - `CLOCK_REALTIME` - wall-clock time (can jump when set).
  - `CLOCK_MONOTONIC` - time since boot, never jumps, stops during suspend.
  - `CLOCK_MONOTONIC_RAW` - the same without NTP speed correction.
  - `CLOCK_BOOTTIME` - like monotonic, but keeps counting during suspend.
  - `CLOCK_TAI` - International Atomic Time (no leap seconds).
  - `CLOCK_PROCESS_CPUTIME_ID` / `CLOCK_THREAD_CPUTIME_ID` - CPU time used by SRS.
  - `CLOCK_REALTIME_COARSE` / `CLOCK_MONOTONIC_COARSE` - cheaper, tick-resolution versions.
- `Time spent suspended` - BOOTTIME minus MONOTONIC.
- `/proc/uptime (up idle)` - seconds up, and total idle CPU-seconds across all CPUs.
- `This process user / system` - CPU time SRS has spent in its own code and in the kernel.
- `Time synchronisation` - every `timedatectl show-timesync` property (configured, fallback and current NTP servers, poll intervals, the last NTP message), then the `timesync-status` summary: server, poll interval, leap, stratum, precision, offset, delay, jitter, packet count. If timesyncd is not used: "systemd-timesyncd not running".

Healthy / typical values. Author's PC: all fine clocks at 1 ns, coarse ones at 1000000 ns (1 ms). 0 s suspended. Server `time.cloudflare.com`, stratum 3, offset +113us, jitter 883us.

Unusual values and what they suggest. NTP offset of many milliseconds that keeps growing: the sync server is unreachable or the PC's clock crystal drifts badly. MONOTONIC_RAW running ahead of or behind MONOTONIC by a lot over a day: NTP is correcting a fast or slow crystal, which is normal in small amounts.

Where Linux gets it. `clock_gettime()` / `clock_getres()`, `/proc/uptime`, `getrusage()`, `timedatectl show-timesync --all`, `timedatectl timesync-status`.

Background. NTP **stratum** is how far a server is from a reference clock. Stratum 1 is connected to an atomic or GPS clock, and stratum 3 is two steps away.

Users

`users` - Linux data

In plain words. The accounts on this PC: real people first, then the system accounts that services run as, then the account rules.

What you see.

- Table of human accounts (UID at or above `UID_MIN`, not `nobody`): `User Name` (yours is green), `UID`, `GID` (main group), `Full Name` (first field of the comment), `Home`, `Shell`.
- `System accounts (N, UID < min or nobody)` - a comma list of every service account.
- `Account policy (/etc/login.defs)` - `UID_MIN`, `UID_MAX`, `SYS_UID_MAX`, `GID_MIN`, `PASS_MAX_DAYS` / `PASS_MIN_DAYS` / `PASS_WARN_AGE` (password ageing), `ENCRYPT_METHOD` (password hash type), `UMASK` (default permissions for new files), `HOME_MODE`, `USERGROUPS_ENAB`.
- Note: `/etc/shadow` (the password hashes) is never read.

Healthy / typical values. Author's PC: `user 1000 1000 ... /home/user /bin/fish`, one test account, and 215 system accounts (`root`, `daemon`, `systemd-*`, `polkitd`, `sddm`...). Policy: `UID_MIN 1000`, `ENCRYPT_METHOD YESCRYPT`, `UMASK 022`, `HOME_MODE 0700`.

Unusual values and what they suggest. A human-range account you did not create: check where it came from (some packages create one). A system account with a real login shell (`/bin/bash` instead of `/usr/bin/nologin`) is a common hardening issue, but this table only shows shells for human accounts. `ENCRYPT_METHOD MD5` or `DES`: weak password hashing. Change it to `YESCRYPT` or `SHA512`.

Where Linux gets it. `/etc/passwd`, `/etc/login.defs`.

Background. A **UID** (user ID) is the number the kernel really uses for an account. The name is just a label. UIDs below 1000 are reserved for system services on most distributions.

Driver verifier

`verifier` - Windows-only (Linux equivalent shown)

In plain words. Windows Driver Verifier stress-tests drivers to catch bugs. Linux has no single switch for this. Its checkers are kernel build options (KASAN, UBSAN, KFENCE, lockdep...), and the results end up in the taint flags. SRS shows both.

What you see.

- The same taint table as Code integrity: `Tainted N (0xHEX)` and bits 0-18.
- `Debug/checker options built into the running kernel` - any of `CONFIG_KASAN` (memory-error detector), `UBSAN` (undefined-behaviour checks), `KFENCE` (low-cost sampling memory checker), `DEBUG_PAGEALLOC`, `PROVE_LOCKING/LOCKDEP` (lock-order checks), `DEBUG_KMEMLEAK`, `SLUB_DEBUG`, `DEBUG_LIST`, `DEBUG_OBJECTS`, `FAULT_INJECTION`, `KCSAN` (data-race detector), `DEBUG_VM`, `HARDENED_USERCOPY`, `INIT_ON_ALLOC_DEFAULT_ON`, `INIT_ON_FREE_DEFAULT_ON`.
- `kfence.sample_interval` - if KFENCE is present (ms between samples; 0 = off).

- `slub_debug on cmdline` - yes / no.

Healthy / typical values. Author's PC: Tainted 0x3044 (S, U, O, E), only `CONFIG_SLUB_DEBUG=y` built in (available but off unless turned on at boot), `slub_debug` no. Distribution kernels usually also have KFENCE and HARDENED_USERCOPY.

Unusual values and what they suggest. If you are chasing a driver crash, a kernel with KASAN or `slub_debug` turned on will catch the bad access at the moment it happens. These options slow the system, so use them only for debugging.

Where Linux gets it. `/proc/sys/kernel/tainted`, `/proc/config.gz`, `/sys/module/kfence/parameters/sample_interval`, `/proc/cmdline`.

Virtual memory

`virtual` - Linux data

In plain words. The system's virtual-memory rules (how much memory may be promised, overcommit, huge pages) and SRS's own address-space use and limits.

What you see.

- `System`:
  - `Commit Limit`, `Committed` - the commit ceiling and what has been promised so far.
  - `Vmalloc Total`, `Vmalloc Used` - kernel virtual address space (the total is huge on 64-bit; used is the real figure).
  - `Anonymous Pages`, `Anon Huge Pages` - program memory not backed by files, and how much of it is in 2 MB transparent huge pages.
  - `HugePages Total / Free (size)` - reserved huge pages.
  - `vm.overcommit_memory` (0 heuristic, 1 always allow, 2 strict), `overcommit_ratio`, `overcommit_kbytes`, `max_map_count` (memory regions allowed per process), `mmap_min_addr`, `randomize_va_space` (ASLR, 2 = full), `THP enabled` (`always` / `madvise` / `never` in brackets).
- `This process` - SRS's `Vm*` and `Rss*` lines (see Memory heap), then `RLIMIT_AS`, `RLIMIT_DATA`, `RLIMIT_STACK`, `RLIMIT_MEMLOCK` with soft and hard values.

Healthy / typical values. Author's PC: Commit 39.54GB of 63.26GB, Vmalloc Total 32.00TB, Anon Huge 2.17GB, `overcommit_memory 0`, `max_map_count 2147483642` (raised for games), THP `[always]`, stack limit 8MB, memlock unlimited.

Unusual values and what they suggest. `max_map_count` at the old default 65530: some games (many Proton titles) crash when they run out of regions. Many distributions now ship 1048576 or higher.

`overcommit_memory 2` with a low ratio: programs get "out of memory" errors even with free RAM. A small `RLIMIT_MEMLOCK` (8 MB): some audio, RDMA and emulator tools fail.

Where Linux gets it. `/proc/meminfo`, `/proc/sys/vm/*`, `/proc/sys/kernel/randomize_va_space`, `/sys/kernel/mm/transparent_hugepage/enabled`, `/proc/self/status`, `getrlimit()`.

Background. Overcommit means the kernel promises programs more memory than it really has, betting they will not all use it at once. THP (Transparent Huge Pages) groups memory into 2 MB pages so the CPU spends less time looking up addresses.

Visual Studio paths

`vs-paths` - Windows-only (Linux equivalent shown)

In plain words. SIV shows where Visual Studio is installed. On Linux, SRS shows where each common developer tool is found on your `PATH`, and what it really points to.

What you see. `Tool` - `Path` (first match on `PATH`, or `<not found>`) - `Resolves To` (the real file if `Path` is a link; blank if not). Tools checked: `cc`, `gcc`, `g++`, `clang`, `clang++`, `ld`, `make`, `cmake`, `ninja`, `meson`, `rustc`, `cargo`, `go`, `python3`, `node`, `java`, `dotnet`, `gdb`, `lldb`, `code`, `codium`, `x86_64-w64-mingw32-gcc` (Windows cross-compiler).

Healthy / typical values. Author's PC: `cc -> /usr/bin/gcc`, `clang -> /usr/bin/clang-23`, `rustc` in `~/.cargo/bin` resolving to `rustup`, `python3 -> python3.14`, `java -> /usr/lib/jvm/java-21-openjdk/bin/java`.

Unusual values and what they suggest. A tool found in an unexpected place first (e.g. `~/.local/bin/gcc` or a Homebrew copy) shadows the system one, and builds may use the wrong version. `<not found>` for a tool a build needs: install it or fix `PATH`.

Where Linux gets it. A walk over `$PATH` with `access(X_OK)`, then `realpath()`.

Dr Watson / crash debugger

`watson` - Linux data

In plain words. Dr Watson was Windows' crash catcher. On Linux, when a program crashes the kernel can write a **core dump** (a snapshot of its memory). systemd-coredump normally catches it. This item shows how that is set up.

What you see.

- `core_pattern` - what happens to a core dump. `|/usr/lib/systemd/systemd-coredump ...` means it is piped to systemd-coredump. A plain file name means a `core` file is written in the program's folder.
- `core_pipe_limit`, `core_uses_pid`, `print-fatal-signals` (log every fatal signal), `panic_on_oops` (1 = a kernel bug reboots the PC), `panic` (seconds before auto-reboot after a kernel panic; 0 = never).
- `fs.suid_dumpable` - core dumps for setuid programs (0 off, 1 on, 2 root-only/safe).
- `RLIMIT_CORE (this process)` - maximum core size for SRS (0 = no dumps; unlimited).
- `systemd-coredump.socket` - active or not.
- `coredump.conf` - every active setting from `/etc/systemd/coredump.conf` and its `.conf.d` folders, with the file it came from, or "(defaults only)".

Healthy / typical values. Author's PC: piped to systemd-coredump, `core_uses_pid 1`, `panic 0`, `suid_dumpable 2`, RLIMIT_CORE unlimited, socket active. The main file sets 32G limits, then a drop-in `99-limits.conf` lowers them to `ProcessSizeMax=1G`, `ExternalSizeMax=1G`, `MaxUse=2G`, `KeepFree=10G`. Later files win.

Unusual values and what they suggest. `core_pattern` is plain `core` and `RLIMIT_CORE 0`: crashes leave nothing to debug. `Storage=none`: dumps are logged but not kept. A huge `MaxUse` on a small disk: crash dumps from games can fill `/var`.

Where Linux gets it. `/proc/sys/kernel/{core_pattern,core_pipe_limit,core_uses_pid,print-fatal-signals,panic_on_oops,panic}`, `/proc/sys/fs/suid_dumpable`, `getrlimit(RLIMIT_CORE)`, `systemctl is-active systemd-coredump.socket`, `/etc/systemd/coredump.conf`, `/etc/systemd/coredump.conf.d/`, `/usr/lib/systemd/coredump.conf.d/`.

Error reporting

`wer` - Linux data

In plain words. The Linux counterpart of Windows Error Reporting's crash list: which programs have crashed recently, and how much space the saved dumps use.

What you see.

- `Stored core dumps` - "N files, SIZE in /var/lib/systemd/coredump".

- `Most recent crashes (coredumpctl list -r -n 30)` - newest first: `TIME`, `PID`, `UID`, `GID`, `SIG` (the signal that killed it), `COREFILE` (`present` = dump kept, `missing` = deleted or never stored, `truncated` = cut at the size limit, `none`), `EXE` (the program), `SIZE`.
- "no core dumps recorded (or journal not readable)" if empty.

Healthy / typical values. Author's PC: 8 files, 411.80MB. Recent entries include `plasmashell` (SIGSEGV and SIGABRT, `truncated` at the 1 GB limit), `Xwayland` (SIGABRT, present) and several `kioworker` SIGSEGVs. A healthy desktop has none to a few per month.

Unusual values and what they suggest. One program crashing again and again (e.g. four `kioworker` crashes in five minutes): a real bug or a broken plugin. Run `coredumpctl info <PID>` for its backtrace. `plasmashell` crashes: often a broken widget or a Plasma/Qt version mismatch after a partial update. `truncated` dumps cannot give a full backtrace; raise `ProcessSizeMax` if you need one.

Where Linux gets it. Directory listing of `/var/lib/systemd/coredump`; `coredumpctl list --no-pager -r -n 30`.

Background. SIGSEGV (segmentation fault) means the program touched memory it does not own. **SIGABRT** means the program stopped itself after finding something wrong, often a failed internal check.

Windows CE devices

`win-ce` - Windows-only (Linux equivalent shown)

In plain words. SIV lists Windows CE / Windows Mobile handhelds paired with the PC. These have no Linux counterpart in SRS.

What you see. `Status - Windows-only item in SIV`, `Linux equivalent - none`. That is expected.

General Info

`windows` - Linux data

In plain words. This is the panel SIV shows behind its "Windows" button (called "Windows (OS) panel" before SRS 2.0). In SRS it is the Linux overview, the same page as System. In the main window it is the first tab of the **Operating system** page; in the classic view the main window's **Linux** button opens it.

What you see. Exactly the fields described under System: distribution and kernel, install date, init and chassis, desktop session, system folders, kernel command line, one line per CPU, power and time zone, up-time and load, user and locale, BIOS, and connected monitors with their kernel driver. The panel's sub-buttons (Machine, Sensors, USB Bus, Processors, Volumes, PCI Bus, ALL Dev, Graphics) open the matching items in other groups.

Healthy / typical values / unusual values. As for System.

Where Linux gets it. `/etc/os-release`, `uname()`, `hostname`, sysfs and procfs (see System).

WinSAT

`winsat` - Windows-only (Linux equivalent shown)

In plain words. WinSAT (Windows System Assessment Tool) produced the old "Windows Experience Index" scores. Linux has no built-in equivalent, so SRS shows a stub.

What you see. `Status - Windows-only item in SIV`, `Linux equivalent - none`. To measure performance on Linux, use SRS's own benchmark tools instead.

Time zones

`zones` - Linux data

In plain words. Every time zone in the tz database (tzdata), and which one this PC uses.

What you see.

- `Current` - your zone (from `/etc/localtime` or `$TZ`), in green.
- `timedatectl` - the zone systemd reports.
- Table from `zone1970.tab`:
  - `Countries` - country codes that use the zone (several for shared zones, e.g. `AE,OM,RE,SC,TF`).
  - `Coordinates` - latitude and longitude of the zone's main city, as `+DDMM+DDDMM`.
  - `Time Zone` - the zone name. Yours is green.
  - `Comments` - which area it covers.
- Footer: "N time zones".

Healthy / typical values. Author's PC: Current and timedatectl both `America/New_York`, 312 zones. Argentina alone has 12, such as `America/Argentina/Cordoba` "most areas: CB, CC, CN…".

Unusual values and what they suggest. `Current` and `timedatectl` differ: `$TZ` is set in your environment and overrides the system zone for your programs only. A zone name not in the table (for example `US/Eastern`): it is an old alias. It still works, but `America/New_York` is the modern name.

Where Linux gets it. `/etc/localtime` link, `$TZ`, `timedatectl show -p Timezone --value`, `/usr/share/zoneinfo/zone1970.tab`.

Graphics / Monitors / Audio (SRS Section Reference)

Back to the section index - Glossary

Where to find these items (SRS 2.0). In the main window they are on the **Graphics & Audio** page (Alt+5). Its tabs are Graphics, GPU information, GPU clock speeds, GPU temperatures, Monitors, OpenCL and Audio devices; **All N items** lists the rest. You can also type an item's name or id into the search box (Ctrl+F). The rest of this introduction describes the classic view (Menu -> Classic View), with its Menu and panel buttons.

This group covers what you see and hear: the graphics card (GPU) with its clocks, temperatures, fans, power, firmware and PCIe link, the monitors plugged into it, the 3D and compute software (Vulkan, OpenGL, OpenCL), and sound, MIDI and game-controller devices. Many item names come from Windows (Direct3D, DirectX, dxdiag, RTSS); for those, SRS shows the closest Linux equivalent and says so in the item's title line. Open any item from **Menu -> Graphics / Monitors / Audio -> item**, or click the **Graphics** button on the Linux panel. Only **GPU register dump** (`gpu-dump`) needs root data: it shows the full PCI config space after you run **Menu -> Run as root (srs-root)** once. Everything here is read only: SRS never writes GPU settings, never changes fan speeds and never talks over the monitor's I2C/DDC bus.

The examples come from the author's PC: AMD Ryzen 7 7800X3D, ASUS TUF GAMING X870-PRO WIFI7 board, XFX AMD Radeon RX 9070 XT (amdgpu driver, RDNA 4), an LG ULTRAGEAR 1440p 165 Hz monitor on DisplayPort and a BenQ RL2755 1080p monitor on HDMI.

Contents

- Audio devices `audio`
- Direct3D info `d3d-info`
- GPU D3D engine nodes `d3d-node`
- DirectX information `directx`
- DirectX sound devices `dxd-aud`
- DirectX system devices `dxd-dev`
- DirectX logical disks `dxd-dsk`
- DirectX files `dxd-fil`
- DirectShow filters `dxd-ftr`
- DirectInput gameports `dxd-gam`
- DirectX GPU displays `dxd-gpu`
- DirectX system information `dxd-inf`
- DirectInput devices `dxd-inp`
- DirectMusic ports `dxd-mic`

- DirectPlay adapters `dxd-ply`
- DirectInput PS/2 devices `dxd-ps2`
- DirectX summary `dxd-sum`
- DirectInput USB root `dxd-usb`
- GOverlay status `goverlay`
- GPU all details `gpu-all`
- GPU API routines `gpu-api`
- GPU applications `gpu-apps`
- GPU VBIOS status `gpu-bios`
- GPU clock speeds `gpu-clocks`
- CUDA information `gpu-cuda`
- GPU D3D status `gpu-d3d`
- GPU register dump `gpu-dump`
- GPU export `gpu-export`
- GPU fans `gpu-fans`
- GPU generations `gpu-gens`
- GPU G-Sync / VRR `gpu-g-sync`
- GPU I2C buses `gpu-i2c`
- GPU information `gpu-info`
- GPU LEDs `gpu-leds`
- GPU mapping `gpu-map`
- GPU power status `gpu-power`
- GPU registry `gpu-reg`
- GPU SCF `gpu-scf`
- GPU SLI topology `gpu-sli`
- GPU temperatures `gpu-temp`
- GPU tree `gpu-tree`
- Graphics `graphics`
- Joysticks `joysticks`
- Monitor ID database `mon-info`
- Monitors `monitors`
- Monitor tree `monitor-tree`
- OpenCL `opencl`
- RSR info `rsr-info`
- RTSS status `rtss-sts`
- Sound devices `sound`
- UAA (HD Audio) devices `uaa-dev`
- Wave devices `wave`

Picking a GPU. Items that show one GPU pick the first card using the `amdgpu` driver, or else the first card found. On a PC with more than one GPU you can open an item as `gpu-info:card1` (or `:1`, or part of the PCI slot such as `:03:00`) to choose another one.

Audio devices

`audio` - Linux data

In plain words. A list of the sound cards the kernel found, followed by the PipeWire view of the same hardware: devices, outputs (sinks), inputs (sources) and the programs playing sound right now.

What you see.

Field	Meaning	Why it matters
`cardN`	ALSA card number (ALSA = Advanced Linux Sound Architecture, the kernel sound layer).	`hw:N,...` device names and mixer tools use this number.
Name	The card's long name, e.g. "HD-Audio Generic".	Tells you which chip is which.
Driver `[id]`	Kernel driver (`HDA-Intel` covers almost all HD Audio chips, Intel or not) and the short card ID in brackets.	The short ID is what you use in ALSA configs (`hw:Generic`).
Second line	Where the card lives: memory address and IRQ (interrupt line).	Mostly for debugging. Two cards on the same IRQ is fine today.
PipeWire (`wpctl status`)	The Audio part of `wpctl status`: Devices, Sinks, Sources, Filters, Streams. A `*` marks the default sink or source. `[vol: 0.90]` is the volume (1.00 = 100 %).	This is what applications actually use. If a card is in the ALSA list but missing here, PipeWire or WirePlumber is not using it.

If `wpctl` is missing but `pactl` is installed, SRS shows the first 40 lines of `pactl info` instead.

Healthy / typical values. On the author's PC: `card0 HDA ATI HDMI` (the audio part of the RX 9070 XT, used for sound over HDMI/DisplayPort) and `card1 HD-Audio Generic` (the Realtek ALC1220 on the motherboard). PipeWire lists "Navi 48 HDMI/DP Audio Controller" and "Ryzen HD Audio Controller", and the default sink is a software sink ("JamesDSP Sink") that feeds the real optical output. Streams show programs such as a MIDI synth with `output_FL > ... [active]`.

Unusual values and what they suggest.

- No cards at all: the sound driver did not load (check `dmesg` for `snd_hda_intel`), or the onboard audio is disabled in the BIOS.
- Card present in ALSA but no PipeWire device: WirePlumber may have disabled it (profile "Off"), or PipeWire is not running for your user.
- Sinks empty: no usable output profile; pick one in your desktop's sound settings.
- A stream marked `[paused]` is normal for idle programs (speech services, for example).

Where Linux gets it. `/proc/asound/cards`, then `wpctl status` (PipeWire) or `pactl info`.

Background. Linux sound has two layers. ALSA is the kernel driver layer that talks to the chip. A sound server (today PipeWire, earlier PulseAudio) sits on top, mixes many programs together and routes them to outputs. A *sink* is somewhere sound goes (speakers, HDMI, a virtual effects sink). A *source* is somewhere sound comes from (microphone, line in).

Direct3D info

`d3d-info` - Linux data

In plain words. Direct3D is Windows' 3D graphics interface. Linux uses Vulkan and OpenGL instead, so this item shows which Vulkan and OpenGL drivers are installed and which versions they support.

What you see.

Field	Meaning	Why it matters
Vulkan instance	Version of the Vulkan loader (the library programs talk to first).	Newer loaders support newer features.
Vulkan device N	Each Vulkan device: name, then type (`DISCRETE_GPU`, `INTEGRATED_GPU`, `CPU`...), API version, driver name and driver info.	Shows which driver a game will use.
OpenGL renderer	The OpenGL renderer string (driver, chip, compiler, kernel DRM version).	Confirms you are on the real GPU, not a software renderer.
OpenGL core / compat / ES	Highest OpenGL versions for the core profile, the compatibility profile and OpenGL ES.	Old games need compat; mobile-style apps use ES.
GL video memory	Memory the OpenGL driver reports.	Should match your card's VRAM.
Direct rendering	`Yes` means the GPU draws directly.	`No` means slow software rendering.

If `glxinfo` is missing, SRS falls back to `eglinfo` and shows the EGL version too. If neither is installed it says so (package `mesa-utils`).

Healthy / typical values. On the author's PC: Vulkan instance 1.4.363; device 0 "AMD Radeon RX 9070 XT (RADV GFX1201)", `DISCRETE_GPU`, driver `radv`, Mesa 27.0.0-devel; device 1 "llvmpipe", type `CPU`. OpenGL renderer "AMD Radeon RX 9070 XT (radeonsi, gfx1201, ACO, DRM 3.64, ...)", core 4.6, ES 3.2, video memory 16384MB, direct rendering Yes.

Unusual values and what they suggest.

- Only `llvmpipe` listed, or OpenGL renderer says `llvmpipe`: the GPU driver is not working and everything is drawn by the CPU. Check that `vulkan-radeon` (or your vendor's driver) is installed and that the kernel driver loaded.
- Two GPU entries with the same name: usually two Vulkan drivers installed for one card (for example RADV and AMDVLK). Games may pick the wrong one.
- OpenGL version far below 4.6 on a modern card: old Mesa, or a fallback driver.

Where Linux gets it. `vulkaninfo --summary` (run with implicit layers such as overlays turned off for the query) and `glxinfo -B`, or `eglinfo -B`.

Background. Mesa is the open-source graphics driver collection. On AMD it contains RADV (the Vulkan driver) and radeonsi (the OpenGL driver). `llvmpipe` is Mesa's software renderer. It is always present and is used only as a last resort.

GPU D3D engine nodes

`d3d-node` - Linux data

In plain words. A modern GPU is several engines on one chip: a 3D/compute engine, copy engines, a video engine, a display engine, a security processor and a power-management processor. This item lists each engine and its hardware version.

What you see.

Column	Meaning
Engine	Short name of the hardware block (IP block) from AMD's IP discovery table.
IP version	Major.minor.revision of that block.
Instances	How many copies of the block the chip has.
Function	Plain-English job: `GC` 3D / Compute (graphics core), `SDMA0/1` Copy (system DMA), `VCN` or `UVD` video decode/encode, `VCE` video encode, `JPEG` JPEG decode, `DMU` display (DCN), `MP0` security (PSP), `MP1` power (SMU).

Below the table: **GFX busy** and **Memory busy** in percent (live, refreshed every second in the GUI), and a pointer to GPU applications for per-program engine time.

Healthy / typical values. On the RX 9070 XT: GC 12.0.1, SDMA0 and SDMA1 7.0.1, UVD 5.0.0 (the video engine is reported under the old name UVD), DMU 4.1.0, MP0 and MP1 14.0.3. At the desktop, GFX busy is a few percent (8 % here) and memory busy about 1 %.

Unusual values and what they suggest.

- Empty table: the GPU is not AMD, or it is older than about 2019 and has no IP discovery table.
- GFX busy stuck at 100 % with nothing running: a program is still drawing in the background, or the GPU hung.

Where Linux gets it. `/sys/class/drm/card0/device/ip_discovery/die/0/<block>/0/{major,minor,revision}`, `gpu_busy_percent`, `mem_busy_percent`.

Background. Windows calls these engines "nodes" (3D, Copy, Video Decode...). AMD builds GPUs from reusable blocks called IP blocks, each with its own version. The same video block can appear in several GPU generations, so the block versions tell you more than the marketing name.

DirectX information

`directx` - Linux data

In plain words. DirectX is a Windows-only family of interfaces. This item shows the Linux graphics interface versions instead, then checks which DirectX-to-Vulkan translation layers are installed (these let Windows games run through Wine and Proton).

What you see.

- **Vulkan instance** - loader version.
- **Vulkan device API** - Vulkan version of the first device, with its name.
- **OpenGL** / **OpenGL ES** - core profile and ES versions from `glxinfo`.
- **EGL** - EGL version from `eglinfo` (EGL connects OpenGL to windows on Wayland).
- **OpenCL** - platform version of the first OpenCL platform, from `clinfo`.
- **DirectX** - always "Windows API - on Linux provided by the translation layers below".
- Translation layers, each with `[=]` (found) or `[_]` (not found) and the path that was checked:
  - DXVK (Direct3D 8/9/10/11 to Vulkan) - `/usr/share/dxvk`
  - VKD3D-Proton (Direct3D 12 to Vulkan) - `/usr/share/vkd3d-proton`
  - wine vkd3d (D3D12) - Wine's own `d3d12.dll`
  - WineD3D (D3D to OpenGL/Vulkan) - Wine's `wined3d.dll`
  - DXVK (user), VKD3D-Proton (user) - copies in `~/.local/share`
  - Steam compat tools (Proton) - `~/.local/share/Steam/compatibilitytools.d`

In the GUI the checks show as ticked or empty boxes.

Healthy / typical values. On the author's PC: Vulkan 1.4.363, OpenGL 4.6, OpenGL ES 3.2, EGL 1.5, OpenCL "OpenCL 2.1 AMD-APP (3581.0)". System DXVK, VKD3D-Proton, Wine's vkd3d and WineD3D are all found; the per-user DXVK and VKD3D-Proton folders are not (normal, they are optional).

Unusual values and what they suggest.

- No translation layers at all: Windows games can still run through Steam's Proton (it brings its own copies), but plain Wine will fall back to the slower WineD3D.
- "vulkaninfo not installed": install `vulkan-tools`.

Where Linux gets it. `vulkaninfo --summary`, `glxinfo -B`, `eglinfo -B`, `clinfo`, and file checks on the paths above.

Background. Wine and Proton turn Windows API calls into Linux ones. For graphics, DXVK and VKD3D-Proton translate Direct3D calls into Vulkan calls, which is usually fast enough that games run close to native speed.

DirectX sound devices

`dxd-aud` - Linux data

In plain words. The Linux version of the dxdiag "Sound" tab: every playback and capture device, whether it is in use right now, and the sound server's outputs.

What you see.

- **Playback** and **Capture** lists, one line per ALSA PCM device (PCM = pulse-code modulation, plain digital audio):
  - `hw:card,device` - the ALSA address.
  - Name - e.g. "HDMI 0", "ALC1220 Analog".
  - State - `closed` (not in use) or `open: <format> <channels>ch <rate>` when a program has it open.
- **Sound server sinks** - `pactl list short sinks`: ID, sink name, server module, sample format, channels, rate, state (`RUNNING`, `IDLE`, `SUSPENDED`).
- **Default sink** - where new sound goes.

Healthy / typical values. On the author's PC: four HDMI/DP outputs on card 0 (`hw:0,3`, `hw:0,7`, `hw:0,8`, `hw:0,9`, one per GPU display output), `hw:1,0` ALC1220 Analog, and `hw:1,1` ALC1220 Digital `open: S32_LE 2ch 96000` (the optical output in use at 32-bit, 2 channels, 96 kHz). Capture: ALC1220 Analog and Alt Analog. Default sink `jamesdsp_sink`.

Unusual values and what they suggest.

- Every device `closed` while music is playing: the sound is going somewhere else (Bluetooth, USB headset, or a network sink).
- Open at 44100 or 48000 when you expected 96000: PipeWire is resampling; set the allowed rates in PipeWire's config if you care.
- Sink `SUSPENDED`: idle for a few seconds. Normal; it wakes when sound starts.

Where Linux gets it. `/proc/asound/pcm`, `/proc/asound/cardN/pcmXp/sub0/hw_params` (`p` playback, `c` capture), `pactl list short sinks`, `pactl get-default-sink`.

DirectX system devices

`dxd-dev` - Linux data

In plain words. Every PCI device in the computer, one per line. This is the Linux version of dxdiag's device list.

What you see. The plain output of `lspci -nn` (up to 400 lines). Each line is: bus address (`03:00.0` = bus 3, device 0, function 0), device class with its code (`VGA compatible controller [0300]`), vendor and device name, the vendor:device ID pair in brackets (`[1002:7550]`), and the chip revision (`rev c0`).

Healthy / typical values. On the author's PC the GPU shows as `03:00.0 VGA compatible controller [0300]: ... Navi 48 [Radeon RX 9070/9070 XT/9070 GRE] [1002:7550] (rev c0)`, with its audio function at `03:00.1`. Lines `01:00.0` and `02:00.0` ("Navi 10 XL Upstream/Downstream Port of PCI Express Switch") are a small PCIe switch built into the graphics card itself. Also listed: NVMe drives, the Realtek 5GbE and Wi-Fi 7 chips, USB controllers, the chipset's PCIe switch ports and an ASMedia USB4 controller.

Unusual values and what they suggest.

- A device you installed is missing: it is not seated, not powered, disabled in the BIOS, or its slot shares lanes with another device that is in use.
- A device listed as `Device [xxxx:yyyy]` with no name: the PCI ID database (`hwdata`) is older than the device. Update the package.

Where Linux gets it. `lspci -nn` (pciutils), which reads `/sys/bus/pci/devices` and names devices from `/usr/share/hwdata/pci.ids`.

DirectX logical disks

`dxd-dsk` - Linux data

In plain words. Every disk and partition, its size, filesystem, label and where it is mounted. Like dxdiag's disk list.

What you see. Output of `lsblk -e7 -o NAME,SIZE,TYPE,FSTYPE,LABEL,MOUNTPOINT` (loop devices are left out):

Column	Meaning
NAME	Kernel name, shown as a tree (disk, then its partitions). `sdX` = SATA/USB, `nvmeXnY` = NVMe, `zram0` = compressed RAM swap, `sr0` = optical drive.
SIZE	Size in binary units (G = GiB, T = TiB).
TYPE	`disk`, `part` (partition), `rom` (optical).
FSTYPE	Filesystem: `btrfs`, `ext4`, `ntfs`, `vfat`, `exfat`, `swap`, `iso9660`…
LABEL	Filesystem label.
MOUNTPOINT	Where it is mounted, or `[SWAP]`. Empty = not mounted.

Healthy / typical values. On the author's PC: a 931.5G NVMe with an EFI partition on `/boot/efi` and the Btrfs root, a 1.9T NVMe with Windows, EFI, a Btrfs Steam library and a swap partition, a 3.6T NTFS hard drive, several USB sticks, and `zram0` 15.2G used as swap.

Unusual values and what they suggest.

- A disk showing `0B`: an empty card reader slot or a drive that dropped off the bus. Check `dmesg`.
- A partition with no FSTYPE: unformatted, encrypted (shows `crypto_LUKS` once unlocked tools are installed), or damaged.

Where Linux gets it. `lsblk` (util-linux), which reads `/sys/block` and udev data.

DirectX files

`dxd-fil` - Linux data

In plain words. The graphics runtime files and packages on the system: which Vulkan and OpenCL drivers are registered, which versions of Mesa and friends are installed, and which DirectX translation layers exist.

What you see.

- **Vulkan ICDs** (`/usr/share/vulkan/icd.d`) - one line per JSON file: file name, the driver library it points to, and its Vulkan API version. `i686` files are the 32-bit copies used by older games.
- **OpenCL ICDs** (`/etc/OpenCL/vendors`) - one line per `.icd` file and the library it names.
- **Packages (pacman -Q)** - versions of: mesa, lib32-mesa, vulkan-radeon, vulkan-icd-loader, libdrm, libglvnd, ocl-icd, opencl-mesa, rocm-opencl-runtime, dxvk, vkd3d, vkd3d-proton, linux-firmware. Only packages that are installed (or that another installed package provides) appear. Shown only on systems with `pacman` (Arch and its relatives).
- **Translation layers** - the same `[=]` / `[_]` checks as DirectX information.

Healthy / typical values. On the author's PC: four Vulkan ICDs (`lvp_icd` = llvmpipe software Vulkan and `radeon_icd` = RADV, each in 32-bit and 64-bit), all API 1.4.363. OpenCL ICDs: `amdocl64.icd` (ROCm), `rusticl.icd` (Mesa) and `xilinx.icd`. Packages include a self-built Mesa 27.0 development snapshot (`mesa-tkg-git`), `rocm-opencl-runtime 10.0.0` and `dxvk-mingw-git 3.1.1`.

Unusual values and what they suggest.

- No `radeon_icd` on an AMD system: Vulkan games will fall back to llvmpipe or fail. Install `vulkan-radeon`.
- Missing `i686` ICD: 32-bit games (many older Steam titles) will not find a Vulkan driver. Install `lib32-vulkan-radeon`.
- An ICD pointing at a library that no longer exists: left over from an uninstalled driver. It can slow down or confuse Vulkan start-up.

Where Linux gets it. JSON files in `/usr/share/vulkan/icd.d`, text files in `/etc/OpenCL/vendors`, `pacman -Q`.

Background. An ICD (installable client driver) is a small file that tells the Vulkan or OpenCL loader where a driver library is. The loader reads all of them at start-up and offers every driver it finds to the program.

DirectShow filters

`dxd-ftr` - Linux data

In plain words. DirectShow is Windows' old media framework. The nearest Linux equivalent is GStreamer, so this item lists the GStreamer plugin files that are installed.

What you see. For `/usr/lib/gstreamer-1.0` and (if present) `/usr/lib32/gstreamer-1.0`: one line per plugin file, with the `libgst` prefix and `.so` suffix removed (`audioconvert`, `alsa`, `aom`...), then a count. Files that do not start with `libgst` keep their full name. A note at the end says SRS only lists files: it does not run `gst-inspect-1.0`, which is slow and rebuilds its cache.

Healthy / typical values. On the author's PC: 553 plugins in the 64-bit folder (including many LSP audio-effect plugins) and 106 in the 32-bit folder. A desktop with the usual "good", "bad", "ugly" and "libav" plugin sets has a few hundred.

Unusual values and what they suggest.

- Very few plugins: some media players (GNOME Videos, many GTK apps) will not play common formats. Install `gst-plugins-good`, `gst-plugins-bad`, `gst-plugins-ugly` and `gst-libav`.
- A plugin file exists but a player still fails: the plugin may be blacklisted after a crash. Clearing `~/.cache/gstreamer-1.0` lets GStreamer test it again.

Where Linux gets it. Directory listing of `/usr/lib/gstreamer-1.0` and `/usr/lib32/gstreamer-1.0`.

DirectInput gameports

`dxd-gam` - Not present on most PCs

In plain words. The gameport was the 15-pin joystick socket on old sound cards. Modern PCs do not have one.

What it would show. Each gameport device the kernel found. Otherwise one of:

- "not present (/sys/bus/gameport absent - no gameport driver loaded)" - normal on any PC from the last 20 years.
- "none" - the gameport driver is loaded but found no port.

Where Linux gets it. `/sys/bus/gameport/devices`.

DirectX GPU displays

`dxd-gpu` - Linux data

In plain words. The Linux version of the dxdiag "Display" tab: graphics card name, maker, chip, memory, connected monitors and drivers.

What you see.

Field	Meaning
Card name	Marketing name (from the AMD ID table, or `lspci`).
Manufacturer	GPU vendor and its PCI vendor ID.
Chip type	Chip name and PCI device ID.
DAC type	Always "Integrated digital" (modern cards have no analog output converter).
Device type	PCI class and slot address.
Device key	Windows-style hardware ID: `PCI\VEN_<vendor>&DEV_<device>&SUBSYS_<subsystem device><subsystem vendor>&REV_<revision>`.
Dedicated memory	VRAM size.

Field	Meaning
Shared memory	GTT size (system RAM the GPU may use).
Monitor	One line per connected output: connector name, monitor maker and model (or "(no EDID)").
Driver	Kernel driver.
Shader target	The GPU's compiler target name (e.g. `gfx1201`).

Then **User-space drivers**: the same Vulkan and OpenGL rows as Direct3D info.

Healthy / typical values. On the author's PC: AMD Radeon RX 9070 XT, Navi 48 `[7550]`, device key `PCI\VEN_1002&DEV_7550&SUBSYS_88111EAE&REV_C0` (1EAE = XFX), dedicated memory 15.92GB, shared memory 15.23GB, monitors "DP-3: LG Electronics LG ULTRAGEAR" and "HDMI-A-1: BenQ Corporation BenQ RL2755", driver `amdgpu`, shader target `gfx1201`.

Unusual values and what they suggest.

- Dedicated memory much smaller than the box says (for example 256 MB): the driver is not fully loaded, or you are looking at an integrated GPU.
- Monitor "(no EDID)": a bad cable, adapter or KVM switch that does not pass the monitor's ID data.

Where Linux gets it. `/sys/class/drm/card*/device` (IDs, `mem_info_*`), `/usr/share/libdrm/amdgpu.ids`, `lspci -mm -nn`, connector EDID, `ip_discovery`, `vulkaninfo --summary`, `glxinfo -B`.

DirectX system information

`dxd-inf` - Linux data

In plain words. The Linux version of the dxdiag "System" tab: date and time, computer name, operating system, board, BIOS, processor and memory.

What you see. Current date/time; Computer name (hostname); Operating system; Kernel (name, release, architecture); System manufacturer and System model (from DMI, the firmware's hardware table); BIOS (vendor, board, BIOS version and date); Processor; Memory (total RAM the kernel can use); Page file (swap) - total swap; DirectX version - "n/a on Linux; Vulkan ".

Healthy / typical values. On the author's PC: computer name MYPC, CachyOS with kernel 7.3.0, ASUS TUF GAMING X870-PRO WIFI7 W NEO, BIOS 1805 dated 09/18/2026, AMD Ryzen 7 7800X3D, memory 30.45GB, swap 48.04GB, Vulkan 1.4.363.

Unusual values and what they suggest.

- Memory a lot lower than the RAM you installed (for example 15 GB with 32 GB fitted): a stick is not detected, or memory is reserved for an integrated GPU. A few hundred MB less than installed is normal (firmware and kernel reservations); 30.45 GB of 32 GB is normal.
- System manufacturer "To Be Filled By O.E.M." or empty: the board maker did not fill in the DMI fields. Harmless.

Where Linux gets it. `uname`, `/sys/class/dmi/id`, `/proc/cpuinfo`, `/proc/meminfo` (MemTotal, SwapTotal), `vulkaninfo --summary`.

DirectInput devices

`dxd-inp` - Linux data

In plain words. Every input device the kernel knows: keyboards, mice, game controllers, power buttons, and even the "jack sensing" switches of the sound card.

What you see.

Column	Meaning
Name	Device name as reported by the device or driver. One physical device often appears several times (a keyboard with media keys = keyboard + consumer control + mouse part).
Bus / Vendor / Product	`Bus=0003` USB, `0005` Bluetooth, `0019` built-in platform devices, `0000` virtual; then the USB vendor and product IDs and a version.
Handlers	Device files the kernel made: `eventN` (generic input event device), `kbd` (keyboard handler), `mouseN`, `jsN` (old joystick interface), `leds`, `sysrq`.

The count of input devices is at the bottom.

Healthy / typical values. On the author's PC: 23 devices, including two Power Buttons, a SteelSeries Apex 7 keyboard (four entries), a Razer DeathAdder Elite mouse (three entries), a Logitech mouse, a Nintendo Switch Pro Controller (`event21 js0`) plus its motion sensor "Pro Controller (IMU)", and the HDMI and headphone jack-sense entries of the two sound cards.

Unusual values and what they suggest.

- A controller with no `jsN` handler: old games that use `/dev/input/js*` will not see it, but Steam and SDL games use the event device and will.
- The same device listed twice with different vendor IDs: often a virtual device from a remapping tool (here "OL virtual mouse").

Where Linux gets it. `/proc/bus/input/devices`.

DirectMusic ports

`dxd-mic` - Linux data

In plain words. DirectMusic was Windows' MIDI system. On Linux, MIDI programs connect through the ALSA sequencer. This item lists every sequencer client, its ports and who is connected to whom.

What you see. The ALSA sequencer client list, with memory-pool lines removed:

- `Client N : "Name" [type]` - a program or kernel driver. `Kernel` = built into the kernel, `User` = a program. `Legacy` = MIDI 1.0 (newer clients may say UMP, MIDI 2.0).
- `Port N : "Name" (flags) [direction]` - one MIDI port. Flags: `R`/`r` others can read events from it (it sends MIDI), `W`/`w` others can write to it (it receives MIDI, like a synth), `e` exported (visible to others), `X` duplex. A capital letter means other programs may also subscribe.
- `Connecting To:` / `Connected From:` - live connections, as `client:port`.

If `/proc/asound/seq/clients` is missing, SRS shows `aconnect -l` instead.

Healthy / typical values. On the author's PC: System (timer and announce ports), Midi Through, OSS sequencer, and software synths "MT-32", "Nuked SC55" and "FLUID Synth (Tyroland GS)", plus PipeWire's MIDI bridge clients. "Midi Through Port-0" is connected to the Nuked SC55 client (`Connecting To: 129:0`), so anything sent to Midi Through plays on that synth.

Unusual values and what they suggest.

- Only System and Midi Through: no MIDI hardware and no software synth running. Old games and MIDI players will make no music.
- A USB MIDI keyboard missing: check that `snd_usb_audio` loaded.

Where Linux gets it. `/proc/asound/seq/clients`, or `aconnect -l`.

DirectPlay adapters

`dxd-ply` - Windows-only (Linux equivalent shown)

In plain words. DirectPlay was an old Windows networking layer for multiplayer games. It does not exist on Linux, so SRS lists the network interfaces instead.

What it shows. "DirectPlay: Windows-only (deprecated DirectX networking)", "Nearest: network interfaces", then one line per interface: name, state (`up`, `down`, `unknown`) and hardware (MAC) address, for example `wlan0 up 00:11:22:33:44:55`.

Typical values. Real interfaces (`em1` wired Ethernet, `wlan0` Wi-Fi), `lo` (loopback, state `unknown` is normal) and virtual ones from Docker (`docker0`, `veth...`), virtual machines (`virbr0`, `vmnet1`, `vmnet8`) and VPNs. State `unknown` on virtual adapters is normal.

Where Linux gets it. `/sys/class/net/<name>/operstate` and `/address`. For more detail see the network sections.

DirectInput PS/2 devices

`dxd-ps2` - Not present on most PCs

In plain words. PS/2 is the old round keyboard and mouse socket. Many new boards no longer have one.

What it would show. Each serial-input (serio) port with its description and driver, for example `serio0 i8042 KBD port driver atkbd`. When there are none it shows "no PS/2 ports or devices (/sys/bus/serio/devices empty)". That is normal on boards without a PS/2 socket, such as the author's.

Where Linux gets it. `/sys/bus/serio/devices/*/description` and `driver`.

DirectX summary

`dxd-sum` - Linux data

In plain words. A one-screen summary in the style of dxdiag: OS, CPU, RAM, graphics, monitors, 3D drivers, sound cards and input devices.

What you see. Operating system; Processor; Memory; one **Display adapter** line per GPU (name, VRAM, kernel driver); **Monitors** (how many are connected); the Vulkan and OpenGL rows from Direct3D info; **Sound cards** (ALSA card count); **Input devices** (count from `/proc/bus/input/devices`).

Healthy / typical values. On the author's PC: CachyOS, Ryzen 7 7800X3D, 30.45GB, "AMD Radeon RX 9070 XT 15.92GB (amdgpu)", 2 monitors connected, Vulkan 1.4.363, OpenGL 4.6, 2 sound cards, 23 input devices.

Unusual values and what they suggest. See the individual items. A monitor count lower than the screens you see means a screen is driven by a different GPU or by a USB display adapter.

Where Linux gets it. `/sys/class/drm`, `/proc/meminfo`, `/proc/asound/cards`, `/proc/bus/input/devices`, `vulkaninfo --summary`, `glxinfo -B`.

DirectInput USB root

`dxd-usb` - Linux data

In plain words. The USB tree: each USB controller (root hub), the hubs and devices plugged into it, and the speed each one runs at. Then a list of the input devices that are on USB.

What you see.

- The output of `lsusb -t` (up to 300 lines). Each line: bus and port, device number, interface number (`If 0`), class (`Hub`, `Human Interface Device`, `Mass Storage`, `Video`, `Wireless` ...), the driver bound to it (`usbhid`, `uas`, `usb-storage`, `btusb`, `[none]` = no driver), and the link speed.
- **USB HID input devices** - name and physical path (`usb-<controller PCI address>-<port path>/inputN`) of each input device on USB.

Speeds: `12M` Full Speed (USB 1.1), `480M` High Speed (USB 2.0), `5000M` USB 3.2 Gen 1, `10000M` USB 3.2 Gen 2, `20000M/x2` USB 3.2 Gen 2x2 (two lanes).

Healthy / typical values. On the author's PC: keyboards and mice at 12M (normal, they need no more), a USB 3 flash drive with the `uas` driver at 10000M, a webcam at 480M with `uvcvideo`, a Bluetooth adapter with `btusb`, and root hubs from the CPU, the chipset (`xhci-pci-prom21`) and the ASMedia USB4 controller.

Unusual values and what they suggest.

- A USB 3 drive at `480M`: it is in a USB 2 port, behind a USB 2 hub, or the cable is USB 2 only. Expect about 40 MB/s instead of hundreds.
- A drive on `usb-storage` instead of `uas`: slower and older protocol; the drive or adapter does not support UAS (USB Attached SCSI), or it is blacklisted for bugs.
- `Driver=[none]` on a device you need: no kernel driver matches it; it may need a user-space tool.

Where Linux gets it. `lsusb -t` (usbutils) and `/proc/bus/input/devices`.

GOverlay status

`goverlay` - Linux data

In plain words. GOverlay is a settings program for MangoHud, the in-game overlay that shows frame rate, temperatures and clocks. This item shows whether they are installed, which versions, and the MangoHud settings you are using.

What you see.

- **goverlay**, **mangohud** - installed or not.
- Package versions (on pacman systems) for goverlay, mangohud (and their `-git` builds), lib32-mangohud (32-bit games) and vkbasalt. Only installed ones appear.
- **MANGOHUD env** - the value of the `MANGOHUD` environment variable. `1` means MangoHud is switched on for every Vulkan program you start; "(not set - not injected globally)" means it only runs where you ask for it.
- **goverlay.conf** and **GOverlay version** - the config path and the version recorded in it.
- **MangoHud.conf (active lines)** - every line of `~/.config/MangoHud/MangoHud.conf` that is not empty or a comment (up to 80). A line with `=` shows key and value; a line without `=` is a switch and shows `on`. Values of keys that look secret (token, key, secret, pass, auth, cookie) are shown as `********`.

Healthy / typical values. On the author's PC: goverlay-git 1.9.3, mangohud-git 0.8.4, lib32-mangohud-git, vkbasalt, `MANGOHUD env 1`. The config turns on GPU load, VRAM, core clock, edge/junction/memory temperatures, fan, power and power limit, CPU load and temperature, RAM, frame time graph, FSR and present mode, sets `fps_limit 0` (no limit) and `toggle_hud Shift_R+F12`.

Unusual values and what they suggest.

- mangohud installed but lib32-mangohud missing: the overlay will not appear in 32-bit games.
- `MANGOHUD=1` set globally and a desktop program misbehaves: add it to the `blacklist` line, as the author's config does for several tools.

Where Linux gets it. File checks in `/usr/bin`, `pacman -Q`, the `MANGOHUD` environment variable, `~/.config/goverlay/goverlay.conf`, `~/.config/MangoHud/MangoHud.conf`.

GPU all details

`gpu-all` - Linux data

In plain words. Everything SRS knows about one GPU on one page: information, clocks, temperatures, fans, power, firmware, PCI mapping, and the `lspci -vv` view.

What you see. These headed blocks, in order. Each is described in full in its own item:

1. **Information** - as GPU information.
2. **Clocks** - as GPU clock speeds, including the OverDrive table.
3. **Temperatures** - as GPU temperatures.
4. **Fans** - as GPU fans.
5. **Power** - as GPU power status, plus the `gpu_metrics` header line.
6. **VBIOS / firmware** - as GPU VBIOS status.
7. **Mapping** - as GPU mapping.
8. **lspci -vv (unprivileged)** - up to 200 lines of `lspci -vv -nn` for the GPU: subsystem, control/status flags, interrupt routing, IOMMU group, memory regions (BARs), expansion ROM, kernel driver and module. Without root, `lspci` cannot read the capability list, so it prints "Capabilities: ". For the full list see GPU register dump.
9. A footer line with the motherboard and BIOS version.

Healthy / typical values. See each item. On the author's PC `lspci` shows "Region 0: Memory at ... (64-bit, prefetchable) [size=16G]", which means Resizable BAR is on (see below).

Where Linux gets it. All the sources of the items above, plus `lspci -vv -nn -s <slot>`.

GPU API routines

`gpu-api` - Linux data

In plain words. A check-list of the libraries that programs use to talk to the GPU: the kernel interface, Vulkan, OpenGL, video decoding, OpenCL, ROCm, and CUDA-related ones. Each row says whether the library is installed and where.

What you see. Columns **Library**, **API** (what it is for) and **Path** (or "not found"), with `[=]` / `[_]` (ticked boxes in the GUI):

Library	What it is
`libdrm_amdgpu.so.1`	libdrm amdgpu - the user-space wrapper around the amdgpu kernel interface.
`libdrm.so.2`	libdrm core - talks to the kernel's DRM (Direct Rendering Manager).
`libvulkan.so.1`	Vulkan loader.
`libGL.so.1`	OpenGL through GLVND (the vendor-neutral GL dispatcher).
`libEGL.so.1`	EGL (connects GL to Wayland/X11 windows).

Library	What it is
`libgbm.so.1`	GBM (Generic Buffer Management - lets compositors allocate GPU buffers).
`libOpenCL.so.1`	OpenCL ICD loader.
`libva.so.2`	VA-API (Video Acceleration API - hardware video decode/encode).
`libvdpau.so.1`	VDPAU (older video decode API).
`libamdhip64.so`	ROCm HIP runtime (AMD's CUDA-like compute language).
`libhsa-runtime64.so.1`	ROCm HSA runtime (low-level compute runtime under HIP and OpenCL).
`librocm_smi64.so`	ROCm SMI (System Management Interface - monitoring library).
`libamd_smi.so`	AMD SMI (newer monitoring library).
`libcuda.so.1`	CUDA driver API. On an AMD system this is usually ZLUDA, a layer that runs CUDA programs on AMD GPUs.
`libnvidia-ml.so.1`	NVIDIA NVML (NVIDIA's monitoring library).
`libatiadlxx.so`	AMD ADL - a Windows-only AMD API; on Linux it is expected to be "not found".

Healthy / typical values. On the author's PC every library is found except `libatiadlxx.so`. ROCm libraries live in `/opt/rocm/lib`; the rest in `/usr/lib`.

Unusual values and what they suggest.

- `libva.so.2` missing: browsers and video players cannot use the GPU for video decoding and will use more CPU.
- `libvulkan.so.1` missing: no Vulkan games at all. Install `vulkan-icd-loader`.
- `libnvidia-ml.so.1` present on a PC with no NVIDIA GPU: some package pulled it in as a dependency. Usually harmless.

Where Linux gets it. `ldconfig -p` (the shared-library cache), with a fallback check in `/opt/rocm/lib` and `/usr/lib`.

GPU applications

`gpu-apps` - Linux data

In plain words. Which programs have the GPU open right now, how much video memory each uses, and how much GPU time each has used since it started. Like the GPU columns in Windows Task Manager.

What you see. One row per GPU client (a program can open the GPU more than once, so it can appear on several rows):

Column	Meaning
PID	Process ID.
Process	Program name.
VRAM	Video memory the client holds.
GTT	System memory the client has mapped for the GPU.
GFX s	Seconds of 3D (graphics) engine time used so far.
Compute s	Seconds of compute engine time.
DMA s	Seconds of copy engine time.
Video s	Seconds of video decode plus encode engine time.

The bottom line says how many GPU clients were visible and how many processes could not be inspected. Without root you can only look inside your own processes; system processes (such as `kwin_wayland`, the KDE compositor) are skipped. The list refreshes every two seconds in the GUI.

Healthy / typical values. On the author's PC at the desktop: `plasmashell` with about 377 MB VRAM and 37 s of GFX time, `Xwayland` about 38 MB, `coolercontrol` about 133 MB, `krunner` about 74 MB. Rows with 12KB VRAM and 2MB GTT are small helper contexts and are normal.

Unusual values and what they suggest.

- A program you closed still listed: it is still running in the background.
- A small tool using gigabytes of VRAM: a leak in that program, or it is caching textures. Games and AI models normally use many GB.
- Compute seconds rising with no compute work running: something (a miner, an AI service) is using the GPU.

Where Linux gets it. For every process, the open files in `/proc/<pid>/fd` that point to `/dev/dri/*`, and their `/proc/<pid>/fdinfo/<fd>` statistics (`drm-client-id`, `drm-memory-vram`, `drm-memory-gtt`, `drm-engine-gfx`, `-compute`, `-dma`, `-dec`, `-enc`).

GPU VBIOS status

`gpu-bios` - Linux data

In plain words. The version of the graphics card's own BIOS (VBIOS) and the versions of all the small firmware programs that run inside the GPU. Since 2.4, with the root data, also a look inside the VBIOS image itself (from the driver's own copy).

What you see.

- **VBIOS version** - the card maker's VBIOS part number.
- **VBIOS build** - build number or date.
- **Option ROM** - whether the PCI option ROM is exposed. When the image is cached: "present: the driver's copy of its image is shown below". SRS never reads the ROM itself (that would need a write to switch it on).
- **Firmware versions** - one line per firmware the driver loaded, in hex. Common names on AMD:

Name	What it runs
`asd`	AMD Secure Display / security driver (part of the PSP).
`sos`	PSP secure OS (the security processor's operating system).
`ta_ras`	Trusted app for RAS (reliability: memory error handling).
`smc`	SMU firmware - the power, clock and fan manager.
`dmcub`	Display microcontroller (DCN).
`imu`	Power-up microcontroller for the graphics block.
`me`, `pfp`, `mec`	Command processor parts: micro engine, pre-fetch parser, compute micro engine.
`mes`, `mes_kiq`	Micro Engine Scheduler - schedules work queues on the GPU.
`rlc`	RunList Controller - power gating and context switching for graphics.
`sdma`, `sdma2`	Copy engine firmware.
`vcn`	Video codec firmware.

VBIOS image (the driver's copy) - AMD cards, after **Menu > Run as root** (2.4): srs-root copies the copy of the VBIOS that the `amdgpu` driver keeps in memory (debugfs `amdgpu_vbios`; the GPU is not accessed) into `~/.cache/srs/root/vbios-<pci address>.bin`. The page shows:

Field	Meaning
Size, Cache	Size of the copy and where it is saved
Signature	`55 AA (PCI expansion ROM)` when the image starts like a valid option ROM
Checksum	Whether the first image's bytes add up to 0 (mod 256), as they must
Part number, Build string	The part number and build strings found in the image (for example `113-EXT108832-100`, `ATOMBIOSBK-AMD VER023.008.000.068.000001`)
Image table	One row per image (PCIR structure): offset, size, PCI ID, code type (x86 legacy, UEFI ...) and whether it is the last

When the driver's copy ends before the last image, the page says so: the ROM holds more (usually the UEFI GOP driver), which the driver does not keep.

Healthy / typical values. On the author's PC: VBIOS `113-EXT108832-100`, build `00131175`; image 58880 bytes, 55 AA, checksum valid, one legacy x86 image (the driver's copy ends there); smc `0x00685000`, mes `0x00000097`, vcn `0x0910d021`. The exact values depend on the card and on the `linux-firmware` package.

Unusual values and what they suggest.

- A firmware showing `0x00000000`: that firmware did not load. Check `dmesg | grep amdgpu` for firmware errors and update `linux-firmware`.
- Firmware versions that changed after an update and new GPU problems started: a firmware regression is possible; compare with an older `linux-firmware`.
- VBIOS version different from the maker's latest: you may want a VBIOS update, but only from the card maker, and only if it fixes something you have.
- Signature or checksum not valid: the driver's copy is damaged or not a ROM image; compare with the card maker's VBIOS file before trusting it.

Where Linux gets it. `/sys/class/drm/card0/device/vbios_version`, `vbios_build`, the presence of `rom`, and `/sys/class/drm/card0/device/fw_version/*_fw_version`; the image from `/sys/kernel/debug/dri/<n>/amdgpu_vbios` (root), copied by srs-root into `vbios-<pci>.bin`.

Background. The VBIOS lives on a chip on the card. It brings the card up at power-on (so you see the boot screen) and holds the board's power and clock tables. The other firmware files are loaded by the Linux driver from `/usr/lib/firmware/amdgpu` every boot.

GPU clock speeds

`gpu-clocks` - Linux data

In plain words. How fast each part of the GPU is running right now, and the list of speed steps it can choose from. Updates every second in the GUI.

What you see. Three columns: **Clock**, **Current**, **DPM levels ([active])**.

Row	Meaning
GFX (sclk) sensor	Live shader (core) clock from the hardware monitor.
Memory (mclk) sensor	Live memory clock.
GFX (sclk)	Core clock DPM table.
Memory (mclk)	Memory clock DPM table.
Fabric (fclk)	Internal data fabric clock.
SoC (socclk)	Clock for the chip's support logic.
DCE/DCN (dcefclk)	Display engine clock.
PCIe	PCIe link steps: speed, width and the clock used for that step.
GPU load	Percent of time the graphics engine was busy.

Row	Meaning
Memory load	Percent of time the memory controller was busy.
Performance level	The driver's DPM mode: `auto`, `low`, `high`, `manual`, `profile_*`.

In the DPM level column the active step is in `[brackets]`. A row appears only if the driver offers that file.

In text reports (not the GUI) an **OverDrive table** follows: the raw contents of `pp_od_clk_voltage`, read only. On RDNA 4 it shows `OD_SCLK_OFFSET` (how much the top core clock is raised or lowered), `OD_MCLK` (memory clock range), `OD_VDDGFX_OFFSET` (core voltage offset) and `OD_RANGE` (the limits the driver allows).

Healthy / typical values. On the author's PC at the desktop: core about 910-1014 MHz with levels 500 / [910] / 2400 MHz; memory [875] of 96-1258 MHz; fclk [2105] of 313-2400 MHz; socclk [1280]; dcefclk [243]; PCIe [32.0GT/s, x16]. GPU load 4-8 %. Performance level `manual` because the tuning tool LACT has applied an overclock profile. The OverDrive table shows a +300 MHz core offset, memory up to 1375 MHz and a -110 mV voltage offset (an undervolt). On a stock card the level is `auto` and the offsets are 0.

Unusual values and what they suggest.

- Core clock pinned at the top step at idle: something is using the GPU (check GPU applications), or Performance level is `high`. Wastes power and makes heat.
- Memory clock pinned high at idle with two or more monitors: common with mixed refresh rates. The memory cannot change speed without flicker unless both screens give it time to, so the driver keeps it high. Harmless, but idle power is noticeably higher.
- Core clock low under full game load: the card is power-limited or thermal-limited (see GPU power status and GPU temperatures).
- PCIe stuck at the lowest step under load: the link failed to train at full speed (see GPU information).

Where Linux gets it. `/sys/class/drm/card0/device/pp_dpm_sclk`, `pp_dpm_mclk`, `pp_dpm_fclk`, `pp_dpm_socclk`, `pp_dpm_dcefclk`, `pp_dpm_pcie`, `gpu_busy_percent`, `mem_busy_percent`, `power_dpm_force_performance_level`, `pp_od_clk_voltage`, and hwmon `freq1_input` / `freq2_input`.

Background. DPM (dynamic power management) means the GPU keeps a short list of clock steps and jumps between them many times a second. The firmware (SMU) picks the step from load, temperature and power. The memory clock number is the base clock: GDDR6 moves 16 bits per clock per pin, so 1258 MHz is about 20 Gbps per pin, the RX 9070 XT's rated memory speed.

CUDA information

`gpu-cuda` - Not present on most PCs

In plain words. CUDA is NVIDIA's GPU computing system. It only runs on NVIDIA cards. This item checks for NVIDIA hardware and then shows AMD's nearest equivalent, ROCm with HIP.

What it shows.

- **NVIDIA GPUs** - how many NVIDIA cards were found (vendor ID 10DE).
- **/dev/nvidia0** - present or absent (the NVIDIA driver's device file).
- **nvidia-smi** - installed or not.
- **libcuda.so.1** - whether a CUDA driver library exists in `/usr/lib`. On AMD systems this is often ZLUDA (which lets CUDA programs run on AMD), so SRS says "(ZLUDA or NVIDIA)". SRS's GPU Burnin and GPU Compute comparison need only this library for CUDA: their CUDA kernels are built in (PTX, since 2.3), so the NVIDIA CUDA toolkit (its runtime compiler, `libnvrtc`) is optional - see **Options -> NVIDIA CUDA toolkit for GPU tests...** (User Guide 9.20) and tools.
- **Nearest on AMD: ROCm / HIP** - ROCm install folder, HIP runtime library, `/dev/kfd` (the kernel's compute device; "present (compute enabled)" means compute programs can use the GPU), and **HIP target** (the GPU's compiler name, e.g. `gfx1201`).

What "not present" means. On the author's PC: 0 NVIDIA GPUs, no `/dev/nvidia0`, no `nvidia-smi`; `libcuda.so.1` exists (ZLUDA); ROCm in `/opt/rocm`; `/dev/kfd` present; HIP target `gfx1201`. If `/dev/kfd` is absent on an AMD system, ROCm programs cannot run: check that your user is in the `render` group and that the driver loaded.

Where Linux gets it. PCI vendor IDs in `/sys/class/drm`, file checks for `/dev/nvidia0`, `/dev/kfd`, `/opt/rocm`, `libcuda.so.1`.

GPU D3D status

`gpu-d3d` - Linux data

In plain words. A quick "how busy is the GPU" view per card: 3D load, memory load and memory use, followed by the Vulkan devices.

What you see. For each GPU: card and name; **3D/GFX busy** (percent); **Memory busy** (percent); **Dedicated memory** (VRAM used of total); **Shared memory (GTT)** (GTT used of total). Then the Vulkan rows from Direct3D info.

Healthy / typical values. On the author's PC at the desktop: 3D/GFX busy 8 %, memory busy 0 %, dedicated memory 1.03GB of 15.92GB, shared memory 130.06MB of 15.23GB.

Unusual values and what they suggest.

- Dedicated memory almost full and GTT use climbing in a game: the game wants more VRAM than you have. Textures spill into system RAM over PCIe and you get stutter. Lower texture quality.
- High GTT use at the desktop: some program is keeping large buffers in system memory. Usually harmless.

Where Linux gets it. `gpu_busy_percent`, `mem_busy_percent`, `mem_info_vram_total/used`, `mem_info_gtt_total/used` in `/sys/class/drm/card*/device`, and `vulkaninfo --summary`.

GPU register dump

`gpu-dump` - Linux data - Needs root data

In plain words. A raw hex dump of the GPU's PCI configuration space: the small block of settings every PCI device has (IDs, memory windows, power state, PCIe link settings). This is for experts and bug reports.

What you see.

- A header line: slot address and how many bytes a normal user may read. Without root, Linux only lets you read the first 64 bytes.
- The dump: 16 bytes per line, offset first (`000:`, `010:` ...).
- **Full config space (srs-root cache: lspci -vvvnnxxx)** - when you have run srs-root, the full decoded view of the GPU from the root cache: all capabilities (Power Management, PCI Express link capability/control/status, MSI, Advanced Error Reporting, Resizable BAR and more), up to 400 lines.
- Without the root cache: "Needs root: Menu > Run as root (srs-root)" and a note that the extended config space (bytes 256-4095) and amdgpu debugfs registers need root. GPU memory-mapped (MMIO) registers are never read.

Healthy / typical values. On the author's PC: 64 bytes readable; the first bytes `02 10 50 75` are the vendor ID 1002 and device ID 7550 (stored low byte first). In the root view: `LnkCap: Speed 32GT/s, Width x16` and `LnkSta: Speed 32GT/s, Width x16` (the link runs at its full capability), `ASPM Disabled`, MSI enabled.

Unusual values and what they suggest.

- `LnkSta` speed or width below `LnkCap`: see the PCIe notes in GPU information.
- `DevSta: CorrErr+` repeatedly: the link sees correctable errors. A few are normal; a flood in `dmesg` (AER messages) points to a riser cable, a dirty slot or a too-high PCIe generation setting in the BIOS.

Where Linux gets it. `/sys/class/drm/card0/device/config`, and the srs-root cache file `pci-all.txt` (made with `lspci -vvvnnxxx` as root).

Background. PCI configuration space is 256 bytes (PCI) or 4096 bytes (PCI Express) per device. The first 64 bytes are a standard header. Capabilities follow as a linked list; reading them needs root because some devices react badly to being read.

GPU export

`gpu-export` - Linux data

In plain words. A list of every live GPU sensor, its current value, and the exact file it comes from. Useful when you want to set up your own monitoring tool or script.

What you see. Columns **Sensor** (hwmon name and label), **Value** (converted to °C, RPM, W, mV or MHz) and **Source** (the full sysfs path). Every `*_input` and `*_average` file in the GPU's hwmon folder is listed, then **GPU load** (`gpu_busy_percent`) and **VRAM used** (`mem_info_vram_used`). A note says these sources can be saved with `srs -save` or read by any monitor (MangoHud, CoolerControl...).

Healthy / typical values. On the author's PC (hwmon9): fan1 551 RPM, freq1 sclk 910 MHz, freq2 mclk 875 MHz, in0 vddgfx 641 mV, power1 PPT 35.0 W, temp1 edge 25.0 °C, temp2 junction 28.0 °C, temp3 mem 50.0 °C, GPU load 8 %, VRAM used 1.03GB.

Unusual values and what they suggest. The `hwmonN` number can change between boots. Scripts should find the folder through `/sys/class/drm/card0/device/hwmon/` (as shown here) instead of hard-coding `/sys/class/hwmon/hwmon9`.

Where Linux gets it. `/sys/class/drm/card0/device/hwmon/hwmon*/` and `/sys/class/drm/card0/device/`.

Background. hwmon is the kernel's standard sensor interface. Each driver makes a folder with files such as `temp1_input` (millidegrees C), `fan1_input` (RPM), `power1_average` (microwatts) and `in0_input` (millivolts). Every monitoring tool on Linux reads these same files.

GPU fans

`gpu-fans` - Linux data

In plain words. How fast the graphics card's fans are spinning, how fast the card wants them to spin, and the control mode. Read only; updates every second in the GUI.

What you see.

Field	Meaning
Fan 1 speed	Measured fan speed in RPM.
Fan 1 target	Speed the firmware is aiming for.
Fan 1 range	Lowest and highest supported speed.
PWM duty	Fan drive signal as raw value / max, and percent (PWM = pulse-width modulation).
PWM mode	`0 full speed`, `1 manual` (a program sets the speed), `2 automatic (firmware)`.
Fan enabled	Yes / No.

| Fan-control software | `running: lactd` (or `coolercontrold`, `corectrl`, `fancontrol`, ...) when one of the known fan-control programs is running, else `none found running (driver / firmware fan curve)`. |

A note says SRS only reads and never writes fan or PWM settings. Rows appear only if the driver offers the matching file.

Healthy / typical values. On the author's PC at the desktop: 551 RPM, target 551 RPM, range 0 - 3200 RPM, PWM 38 / 255 (15 %). Many cards stop their fans completely (0 RPM, "zero-RPM mode") below about 50-60 °C; that is normal.

Unusual values and what they suggest.

- 0 RPM while the junction temperature is above about 80 °C: fan failure, unplugged fan cable, or a fan curve set wrong in a tuning tool.
- Speed far below target under load: a worn or blocked fan.
- PWM mode `1 manual` when you never set it: a tuning program (LACT, CoreCtrl, CoolerControl) has taken over the fan.

Where Linux gets it. GPU hwmon files `fan1_input`, `fan1_target`, `fan1_min`, `fan1_max`, `pwm1`, `pwm1_min`, `pwm1_max`, `pwm1_enable`, `fan1_enable`.

GPU generations

`gpu-gens` - Linux data

In plain words. Which GPU generation (architecture) the card is, and the version of every hardware block inside the chip.

What you see.

- **Generation** - family name worked out from the graphics core version: RDNA 4 (Navi 4x), RDNA 3.5 (APU), RDNA 3, RDNA 2, RDNA 1, CDNA (Instinct), GCN 5 (Vega), GCN 3/4 (Polaris/Fiji/Tonga), GCN 2, GCN 1. If there is no IP discovery data: "no ip_discovery data (pre-Navi or non-AMD GPU)".
- **GFX IP** - graphics core version and the matching LLVM compiler target (e.g. `gfx1201`).
- A table of all IP blocks: **IP block** name, **Version**, **Instances** (copies) and **HW ID** (AMD's internal block number).

Some block names: `GC` graphics core, `SDMA0/1` copy engines, `LSDMA` light SDMA, `UVD` video, `DMU` display, `MP0` security processor (PSP), `MP1` power manager (SMU), `UMC` memory controllers, `DF` data fabric, `MMHUB` memory hub, `NBIF` PCIe bus interface, `PCIE` PCIe block, `THM` thermal, `SMUIO` SMU input/output, `HDP` host data path, `OSSSYS` interrupt handling, `CLKA` clock blocks, `FUSE` chip fuses, `ATHUB`, `SYSTEMHUB`, `DFX`, `DFX_DAP`, `DBGU_NBIO` (test and debug blocks).

Healthy / typical values. On the RX 9070 XT: Generation RDNA 4 (Navi 4x), GFX IP 12.0.1 (gfx1201). 27 blocks, including UMC 8.14.0 with 8 instances (8 memory controller channels for the 256-bit GDDR6 bus), CLKA 9 instances, GC 12.0.1, MP1 14.0.3. A few blocks (`DAZ`, `DIO`) report 127.127.63, which is a placeholder value rather than a real version.

Unusual values and what they suggest. The generation is only a lookup of the GC version. A brand-new chip may show "unknown" until SRS learns its version. That does not mean anything is wrong with the card.

Where Linux gets it. `/sys/class/drm/card0/device/ip_discovery/die/0/<block>/<instance>/{major,minor,revision,hw_id}`.

Background. Since RDNA 1 (2019), AMD GPUs carry a table in their VBIOS that lists every hardware block and its version. The driver reads this table instead of keeping a fixed list per chip. The `gfx` name (`gfx1201` = GC 12.0.1) is what compilers such as ROCm and Mesa use to pick the right instruction set.

GPU G-Sync / VRR

`gpu-g-sync` - Linux data

In plain words. Whether each monitor supports variable refresh rate (VRR: FreeSync, G-Sync Compatible, or HDMI VRR), whether it is switched on right now, and the refresh range the monitor reports.

What you see. One row per connected output:

Column	Meaning
Connector	Output name (`DP-3`, `HDMI-A-1`).
Monitor	Monitor name from EDID.
VRR capable	`Yes` if the kernel says this monitor and link can do adaptive sync.
VRR enabled	`Yes` if the compositor has turned VRR on for this output now. "No (compositor off/idle)" means it is off at the moment.
EDID range	Lowest and highest vertical refresh rate in the monitor's EDID.

If `drm_info` is not installed, SRS says so and shows only the EDID ranges (`?` in the VRR columns).

Healthy / typical values. On the author's PC: DP-3 LG ULTRAGEAR, capable Yes, enabled No, range 48 - 165 Hz; HDMI-A-1 BenQ RL2755, capable No, range 50 - 76 Hz. KDE switches VRR on only while a full-screen game runs if Adaptive Sync is set to "Automatic", so "No" at the desktop is normal.

Unusual values and what they suggest.

- Capable `No` on a FreeSync monitor: FreeSync is turned off in the monitor's on-screen menu, the cable or adapter does not support it, or (on HDMI) the driver does not support HDMI VRR for that monitor. DisplayPort is the most reliable.
- Enabled `No` while a full-screen game runs: set Adaptive Sync to "Automatic" or "Always" in the display settings.
- A narrow range such as 48-75 Hz: VRR only helps inside that window. Below it, LFC (low framerate compensation) needs the maximum to be at least about twice the minimum.

Where Linux gets it. `drm_info` (connector property `vrr_capable`, CRTC property `VRR_ENABLED`) and the EDID range-limits block in `/sys/class/drm/card0-*/edid`.

Background. Normally a monitor refreshes at a fixed rate and the GPU must wait for it. With VRR the monitor waits for the GPU instead, so frames are shown as soon as they are ready. That removes tearing without the delay of V-Sync. The kernel says whether a screen *can* do it; the compositor (KWin, Mutter, gamescope) decides when to turn it on, per CRTC. A CRTC is the display engine part that scans out one picture to one or more outputs.

GPU I2C buses

`gpu-i2c` - Linux data

In plain words. I2C is a slow two-wire bus that chips use to talk to each other. A GPU has several: one per monitor output (DDC, used to read the monitor's EDID and to change its brightness), plus buses for its

power manager. This item lists all I2C buses in the system and marks which ones belong to the GPU. It lists names only and never talks to any bus.

What you see.

Column	Meaning
Bus	`i2c-N` name (highlighted when it belongs to the GPU).
Name	Adapter name from the driver.
Owner	For GPU buses: the card, and for DDC buses the output they serve and whether it is connected, e.g. `card0 DDC DP-3 (connected)`. For other buses: the device path they hang off.

Healthy / typical values. On the author's PC: `AMDGPU DM i2c hw bus 0-3` are the DDC lines for DP-1, DP-2, DP-3 and HDMI-A-1; `AMDGPU DM aux hw bus 0-2` are DisplayPort AUX channels; `AMDGPU SMU 0` and `1` belong to the power manager. Non-GPU buses: the chipset's SMBus (`SMBus PIIX4`, used for RAM SPD and sensors) and a DesignWare I2C controller with two Realtek devices on it.

Unusual values and what they suggest.

- No `AMDGPU DM i2c` buses: the display part of the driver is not running (for example a headless compute setup).
- For safety, avoid scanning the GPU's SMU buses with I2C tools. On some cards, DDC/CI tools have hung the GPU's power manager. SRS avoids this by only reading bus names.

Where Linux gets it. `/sys/bus/i2c/devices/i2c-*/name`, the bus's sysfs path, and `/sys/class/drm/card0-*/ddc` links.

GPU information

`gpu-info` - Linux data

In plain words. The GPU's identity card: name, chip, maker of the board, architecture, video memory, VBIOS and PCIe link.

What you see.

Field	Meaning	Why it matters
GPU	Marketing name.	What you bought.
Chip	Chip name and PCI device ID from `lspci`.	Several cards share one chip (Navi 48 is the 9070, 9070 XT and 9070 GRE).

Field	Meaning	Why it matters
Device IDs	Windows-style hardware ID (vendor, device, subsystem, revision).	Matches driver databases and the Windows hardware ID.
Vendor	GPU chip maker.	
Board vendor	Company that built the card, with subsystem vendor:device IDs.	Tells XFX from Sapphire, PowerColor and others; BIOS updates come from them.
Architecture	Generation, graphics core (GC) version and `gfx` target.	See GPU generations.
Kernel driver	Driver and DRM card name.	
VRAM	Total, used (and percent), and memory chip maker.	Running out of VRAM causes stutter.
CPU-visible VRAM	How much VRAM the CPU can reach directly. "(Resizable BAR active)" when it is all of it.	See Background.
GTT	Graphics translation table: system RAM the GPU may map; total and used.	Overflow space when VRAM is full.
VBIOS	VBIOS version.	
PCIe link	Current link speed and width, and the maximum.	A slow link costs performance.
Unique ID	A chip-specific ID number (shown as `XXXX` in this guide).	Tells two identical cards apart.
Boot VGA	Whether the firmware used this card for the boot screen.	
Performance level	DPM mode (`auto`, `manual`...).	`manual` means a tuning tool is in control.
Runtime PM	`active` or `suspended`.	A second GPU may sleep when unused.
Board info	Form factor reported by the driver (`type : cem` = a normal plug-in PCIe card).	

A footer line shows the motherboard and BIOS.

Healthy / typical values. On the author's PC: AMD Radeon RX 9070 XT, Navi 48 `[7550]`, board vendor XFX Limited (1EAE:8811), RDNA 4 GC 12.0.1 gfx1201, VRAM 15.92GB (used 1.03GB, 6.5 %) made by hynix, CPU-visible VRAM 15.92GB (Resizable BAR active), GTT 15.23GB (about half of system RAM, the default), VBIOS 113-EXT108832-100, PCIe link "32.0 GT/s PCIe x16 (max 32.0 GT/s PCIe x16)" (PCIe 5.0 x16, full speed), Boot VGA Yes, Performance level manual, Runtime PM active.

Unusual values and what they suggest.

- **PCIe x8 or x4 on an x16 card**: the card is in a secondary slot, the slot shares lanes with an M.2 drive or another card (see your board manual), or there is dirt in the slot. A riser cable can also drop lanes.
- **PCIe 2.5 GT/s at idle**: normal on some cards, which drop the link speed to save power. Check again under load.

- **Lower speed than max under load** (for example 16 GT/s when both card and board support 32): a riser cable, a BIOS setting forcing a lower generation, or signal problems. For gaming the loss from PCIe 5.0 to 4.0 is very small.
- **CPU-visible VRAM 256 MB**: Resizable BAR (also called Smart Access Memory) is off. Turn on "Above 4G Decoding" and "Re-Size BAR" in the BIOS. It can add a few percent in some games.
- **VRAM vendor** (hynix, samsung, micron): just information. It does not mean one card is worse.
- **Runtime PM** `suspended` on your main GPU while the screen is on: unusual; it should be `active`.

Where Linux gets it. `/sys/class/drm/card0/device/` (`vendor`, `device`, `subsystem_*`, `revision`, `mem_info_vram_total/used/vendor`, `mem_info_vis_vram_total`, `mem_info_gtt_total/used`, `vbios_version`, `current_link_speed/width`, `max_link_speed/width`, `unique_id`, `boot_vga`, `power_dpm_force_performance_level`, `power/runtime_status`, `board_info`), `/usr/share/libdrm/amdgpu.ids`, `lspci -mm -nn`, `ip_discovery`.

Background.

- *VRAM* is the card's own fast memory. *GTT* is ordinary system RAM the GPU can use over PCIe; it is much slower for the GPU.
- *BAR* (base address register) is a window through which the CPU sees a device's memory. Classic BARs are 256 MB. *Resizable BAR* lets the window cover all VRAM, so the CPU can write textures directly instead of through a small moving window.
- *PCIe link speed* is the transfer rate per lane: 2.5 GT/s = PCIe 1, 5 = PCIe 2, 8 = PCIe 3, 16 = PCIe 4, 32 = PCIe 5. *Width* is the number of lanes (x1, x4, x8, x16). Bandwidth is speed times width.
- AMD cards contain a small PCIe switch (the "Navi 10 XL Upstream/Downstream Port" entries in `lspci`). The GPU itself sits behind it, so the link value here is the link inside the card. On most AMD cards it reports the same as the slot link, but `lspci -vv` of the upstream port shows the real slot link if you need to be sure.

GPU LEDs

`gpu-leds` - Not present on most PCs

In plain words. Some graphics cards have RGB lighting that software can control. This item says whether such a controller is present.

What it shows.

- **Board** - GPU vendor:device and subsystem IDs.
- **LED device** - each LED class device (`/sys/class/leds`) that belongs to the GPU, if the kernel exposes any.
- **RGB controller** - `none exposed by the kernel for this GPU` when there is none (the author's PC).

- **Checked** - how it was decided: the GPU's LED class devices. SRS never probes the GPU's I2C buses to look for lighting chips; many GPU RGB controllers sit there and only tools such as OpenRGB reach them.

What "not present" means. No RGB lighting that the GPU controls. A card can still have lights driven from a motherboard ARGB header; those belong to the motherboard, not the GPU. Tools such as OpenRGB handle both.

Where Linux gets it. PCI IDs in `/sys/class/drm/card0/device`; `/sys/class/leds/*/device`.

GPU mapping

`gpu-map` - Linux data

In plain words. Where the GPU sits in the PCI system: its address, the path from the CPU to it, its interrupt, IOMMU group, device files, other functions on the same card, and the memory windows (BARs) it uses.

What you see.

Field	Meaning
Bus/Dev/Func	PCI address (domain🚌device.function).
PCI path	Chain of bridges from the CPU's root port to the GPU.
IOMMU group	The isolation group for this device (matters for passing the GPU to a virtual machine).
IRQ	Interrupt number.
NUMA node	Memory node the device is closest to; `-1` = no NUMA information (normal on single-socket desktops).
Local CPUs	CPUs that are "close" to the device.
DRM nodes	Device files: `cardN` (full access, used by compositors), `renderDN` (render-only, used by games and compute), `controlDN` (old, unused).
hwmon	Name of the GPU's sensor folder.
Function	Other functions at the same slot, e.g. the HDMI/DP audio controller.
Resources	Each BAR: address range, size, Memory or I/O, prefetchable or not. `ROM` = expansion ROM window.

Healthy / typical values. On the author's PC: 0000:03:00.0, path `pci0000:00/0000:00:01.1/0000:01:00.0/0000:02:00.0/0000:03:00.0` (CPU root port, then the card's own switch, then the GPU), IOMMU group 15, IRQ 161, NUMA node -1, local CPUs 0-15, DRM

nodes `card0 controlD64 renderD128`, hwmon9, audio function at 03:00.1. BAR 0 = 16GB prefetchable (all VRAM visible, so Resizable BAR is on), BAR 2 = 256MB (doorbell/registers window), BAR 4 = 256B I/O, BAR 5 = 512KB registers, ROM 128KB.

Unusual values and what they suggest.

- BAR 0 only 256MB: Resizable BAR is off (see GPU information).
- GPU and many other devices in the same IOMMU group: passing just the GPU to a virtual machine will be hard on this board or slot.
- PCI path through the chipset (more bridges, a `600 Series Chipset` port): the card is in a slot wired to the chipset instead of the CPU, which adds latency and limits bandwidth.

Where Linux gets it. `/sys/class/drm/card0/device` (realpath, `iommu_group`, `irq`, `numa_node`, `local_cpulist`, `drm/`, `hwmon/`, `resource`), `/sys/bus/pci/devices`, `lspci -mm -nn`.

Background. An IOMMU (input-output memory management unit) controls which memory each device may reach. Devices that cannot be separated from each other share a group.

GPU power status

`gpu-power` - Linux data

In plain words. How much power the GPU is using now, its power limit, core voltage and power-management state. Updates every second in the GUI.

What you see.

Field	Meaning
Power sensor	Label of the power sensor. `PPT` = package power tracking, the power the firmware counts toward the limit.
Power (average)	Average power in watts.
Power (input)	Instant power in watts (only on drivers that offer it).
Power limit	Current limit, with the default and the allowed range.
vddgfx (in0) / in1	Voltages in millivolts (`vddgfx` = core voltage).
GPU load	Graphics engine busy percent.
DPM state	Overall power state: `battery`, `balanced` or `performance`.
Performance level	DPM mode, as in GPU clock speeds.
Runtime PM	`active` / `suspended`.

Field	Meaning
PCI power state	`D0` = fully on; `D3hot` / `D3cold` = asleep.
gpu_metrics	Only in text reports: the version and size of the driver's metrics block. Only its header is decoded.

Healthy / typical values. On the author's PC at the desktop: PPT 34-35 W, power limit 374 W (default 304 W, range 212 - 374 W), vddgfx 616-701 mV, DPM state performance, Performance level manual, PCI power state D0, gpu_metrics v1.3 120 bytes. The limit is at the top of the range because it has been raised in LACT; stock it would read 304 W.

Unusual values and what they suggest.

- Idle power above about 50 W on one monitor: the memory clock is stuck high (see GPU clock speeds) or a program keeps the GPU busy.
- In a game, power sitting exactly at the limit while clocks drop: the card is power-limited. That is normal for modern GPUs. Raising the limit (if your PSU and cooling allow) or undervolting gives more speed.
- Power limit lower than default: something lowered it (a tuning tool or a power-saving profile).
- `D3cold` on your display GPU: should not happen while screens are on.

Where Linux gets it. GPU hwmon `power1_label`, `power1_average`, `power1_input`, `power1_cap`, `power1_cap_default`, `power1_cap_min`, `power1_cap_max`, `in0_input`, `in1_input` (and labels); `gpu_busy_percent`, `power_dpm_state`, `power_dpm_force_performance_level`, `power/runtime_status`, `power_state`, `gpu_metrics` in `/sys/class/drm/card0/device`.

Background. The SMU (system management unit) is a small processor inside the GPU. It watches power, temperature and current many times a second and lowers clocks when a limit is reached. The power limit is how many watts it lets the card use. The allowed range is set by the card maker in the VBIOS.

GPU registry

`gpu-reg` - Linux data

In plain words. Windows keeps driver settings in the registry. On Linux, the equivalent is how the kernel registered the device and the options the GPU driver module was loaded with.

What you see.

- **uevent** - the device's registration data: `DRIVER`, `PCI_CLASS` (30000 = display controller), `PCI_ID`, `PCI_SUBSYS_ID`, `PCI_SLOT_NAME`, `MODALIAS` (the string udev uses to find the right driver).
- **Module** - driver module name.

- **Module srcversion** - a checksum of the module's source; changes when the driver changes.
- **Kernel** - running kernel version.
- **Module parameters** - every parameter in `/sys/module/amdgpu/parameters` and its current value.

Some useful amdgpu parameters: `ppfeaturemask` (which power features are on; `0xfff7ffff` has the OverDrive bit set, which tuning tools need), `dc` (Display Core, `-1` = auto), `runpm` (runtime power management), `aspm` (PCIe link power saving), `gpu_recovery` (`1` = try to reset the GPU after a hang), `dcfeaturemask` / `dcdebugmask` (display feature and debug bits), `sg_display`, `freesync_video`, `deep_color`, `lockup_timeout`, `vm_*` (GPU virtual memory settings), `pcie_gen_cap` / `pcie_lane_cap` (forced PCIe limits, `0` = none). `-1` almost always means "driver default".

Healthy / typical values. On the author's PC: `DRIVER amdgpu`, `PCI_ID 1002:7550`, `PCI_SUBSYS_ID 1EAE:8811`, kernel 7.3.0, `ppfeaturemask 0xfff7ffff`, `dcfeaturemask 8`, `gpu_recovery 1`, most others `-1` or `0`.

Unusual values and what they suggest.

- A parameter that is not default and you do not remember setting: look in `/etc/modprobe.d/` and the kernel command line (`amdgpu.<name>=`).
- `ppfeaturemask` without the OverDrive bit: tuning tools such as LACT or CoreCtrl cannot change clocks or voltages.

Where Linux gets it. `/sys/class/drm/card0/device/uevent`, `driver/module`, `/sys/module/amdgpu/srcversion`, `uname`, `/sys/module/amdgpu/parameters/*`.

GPU SCF

`gpu-scf` - Linux data

In plain words. SCF stands for Scalable Control Fabric (with the Scalable Data Fabric next to it): AMD's internal network that links the parts of a chip. This item shows the versions of the fabric-related blocks and their clocks.

What you see.

- **Data Fabric IP** - version of the data fabric (`DF`).
- **SMU (MP1) IP** - version of the power manager.
- **NBIF IP** - version of the PCIe bus interface.
- **MMHUB IP** - version of the memory hub.
- DPM rows for **Fabric (fclk)**, **SoC (socclk)** and **PCIe**, as in GPU clock speeds.

Healthy / typical values. On the RX 9070 XT: Data Fabric 4.15.1, SMU 14.0.3, NBIF 6.3.1, MMHUB 4.1.0; fclk [2105] of 313 / 2105 / 2400 MHz; socclk [1280] of 417 / 1280 / 1476 MHz; PCIe [32.0GT/s, x16].

Unusual values and what they suggest. Fabric clock stuck at its lowest step under heavy load would limit memory bandwidth, but this is rare; usually the firmware handles it. Rows missing means the driver does not offer that clock table for this GPU.

Where Linux gets it. `ip_discovery` (`DF`, `MP1`, `NBIF`, `MMHUB`) and `pp_dpm_fclk`, `pp_dpm_socclk`, `pp_dpm_pcie`.

GPU SLI topology

`gpu-sli` - Not present on most PCs

In plain words. SLI (NVIDIA) and CrossFire (AMD) were ways to make two graphics cards share one game. They are dead on modern cards and never existed on Linux in that form.

What it shows. GPUs (count), one line per GPU (card, name, slot), and **SLI / CrossFire**: "not applicable - single GPU", or for more than one GPU, "multiple GPUs present; Linux has no SLI/CrossFire link (PRIME / Vulkan device groups instead)".

What "not present" means. Normal. On Linux, several GPUs work through PRIME (one GPU renders, another displays, common on laptops) or through programs that use several Vulkan devices on their own.

Where Linux gets it. `/sys/class/drm`.

GPU temperatures

`gpu-temp` - Linux data

In plain words. The GPU's temperature sensors with their shutdown limits. Updates every second in the GUI.

What you see. One row per sensor, columns **Sensor**, **Current**, **Critical**, **Hysteresis**, **Emergency** (`-` when the driver gives no value). AMD cards have:

- **edge** - temperature at the edge of the chip. The oldest and lowest reading.

- **junction** - the hottest spot on the chip (also called hotspot). This is what the fan curve and throttling use.
- **mem** - the video memory temperature.

Below the table, if the driver offers it, a note about thermal throttling logging (for example "thermal throttling logging enabled, with interval 60 seconds": the driver writes a line to the kernel log, at most once a minute, when the card slows down because of heat).

Healthy / typical values. On the author's PC at the desktop: edge 25 °C, junction 28-29 °C, mem 50 °C. Critical 110 °C and emergency 115 °C for edge and junction; 108 / 113 °C for memory. Under gaming load, junction 70-90 °C and memory up to about 90 °C are normal for this class of card.

Unusual values and what they suggest.

- Junction more than about 20-25 °C above edge under load: poor contact between cooler and chip (thermal paste or pad problem, or a warped cooler).
- Junction near 100-110 °C: the card is throttling. Check fans, case airflow and dust.
- Memory hotter than about 95-100 °C: memory pads are not touching the cooler well.
- At idle, memory warmer than the chip (as here) is normal: memory does not power down as much.

Where Linux gets it. GPU hwmon `temp*_input`, `temp*_label`, `temp*_crit`, `temp*_crit_hyst`, `temp*_emergency`, and `thermal_throttling_logging` in the device folder.

Background. *Critical* is where the driver warns and the firmware slows the card hard. *Emergency* is where the card switches itself off to protect the chip. *Hysteresis* is how far the temperature must fall before a limit counts as cleared.

GPU tree

`gpu-tree` - Linux data

In plain words. A tree view of every GPU: where it is on the PCI bus, its driver, sensor folder and device files, then each display output and the monitor on it.

What you see. For each card: the card name and GPU name, then branches:

- `PCI` - the device path.
- `driver` - kernel driver.
- `hwmonN` - sensor folder.
- `/dev/dri/...` - device files other than `cardN` (`renderD128`, `controlD64`).

- One branch per output (`DP-1`, `HDMI-A-1`...) with its status (`connected` / `disconnected`) and whether it is enabled (`enabled` / `disabled`). Under connected outputs: the monitor's PnP ID, name, maker and preferred resolution from EDID.

Healthy / typical values. On the author's PC: `card0 AMD Radeon RX 9070 XT`, driver amdgpu, hwmon9, `/dev/dri/controlD64`, `/dev/dri/renderD128`; DP-1 and DP-2 disconnected; DP-3 connected enabled, "GSM LG ULTRAGEAR LG Electronics 2560×1440"; HDMI-A-1 connected enabled, "BNQ BenQ RL2755 BenQ Corporation 1920×1080".

Unusual values and what they suggest.

- `connected disabled`: the monitor is plugged in and detected, but the desktop has it turned off (or it is a mirrored or disabled screen).
- A monitor plugged in but shown `disconnected`: cable, port or monitor input problem.

Where Linux gets it. `/sys/class/drm` (cards and `cardN-<connector>` folders, `status`, `enabled`, `edid`), the device's `drm/` and `hwmon/` folders.

Graphics

`graphics` - Linux data

In plain words. One block per active display: which card drives it, which monitor it is, the connector, the current resolution and refresh rate, and every video mode the monitor offers.

What you see. For each connected display:

- Heading: `Display N`, VRAM size, VBIOS version, GPU name.
- **Graphics Card** - GPU name.
- **Monitor** - maker and model from EDID, with the connector.
- **Graphics Chip** - chip name and device ID.
- **Graphics DAC** - always "Integrated (digital output)".
- **Connector** - connector name, kernel connector ID, and DPMS state (`On`, `Standby`, `Suspend`, `Off`; DPMS = display power management signaling, the monitor's sleep state).
- **Resolution** - current mode, `width×height@refresh`.
- **Unique X of Y** - how many different resolutions are in the mode list, out of how many modes in total.
- **Video Modes** - one row per resolution, with all its refresh rates. `*` marks the monitor's preferred (native) mode.

At the end: the total number of displays, how long the scan took, and where monitor names come from. If `drm_info` is not installed the mode list is replaced by a note saying so. A GPU with nothing connected shows "Monitor none connected".

Healthy / typical values. On the author's PC: Display 0 = LG ULTRAGEAR on DP-3, connector id 159, DPMS On, resolution 2560×1440@164.96; 21 modes, 8 unique resolutions; the native row reads "2560×1440 120.00 * 164.96 143.97 59.95". Display 1 = BenQ RL2755 on HDMI-A-1, 1920×1080@60.00, 31 modes, 14 unique.

Unusual values and what they suggest.

- Current refresh lower than the monitor's maximum (for example 60 Hz on a 165 Hz monitor): change it in display settings. If the high rate is not offered, the cable or port may not have enough bandwidth (older HDMI cables, DisplayPort to HDMI adapters).
- The `*` preferred mode is not the highest rate: normal. Many high-refresh monitors mark a lower rate as preferred for compatibility, as the LG does here (120 Hz).
- Resolution lower than native: the picture will be blurry. Set the native resolution.

Where Linux gets it. `/sys/class/drm/card*-*/` (status, EDID, connector ID, DPMS), `drm_info` (mode list, current mode on each CRTC), `/usr/share/libdrm/amdgpu.ids`, `lspci`, `/usr/share/hwdata/pnp.ids`.

Background. KMS (kernel mode setting) is the part of the kernel's DRM that sets screen modes. A connector is a physical output. Each mode is a resolution plus timing; the same resolution can appear at many refresh rates.

Joysticks

`joysticks` - Linux data

In plain words. Game controllers and joysticks the system can see.

What you see.

- A table of `/dev/input/js*` devices: **Device** path, **Name**, **VID:PID** (USB vendor and product ID) and **Bus** (`USB`, `Bluetooth`, or the raw bus number). If there are none: "No joystick (/dev/input/js*) devices".
- **Game controllers in /proc/bus/input/devices** - every input device that has a `js` handler, with its handlers (`eventN jsN`).
- A note that event devices are only listed, never opened.

Healthy / typical values. On the author's PC: `/dev/input/js0 Pro Controller 057e:2009 USB` (a Nintendo Switch Pro Controller), handlers `event21 js0`.

Unusual values and what they suggest.

- Controller works in Steam but not listed here: Steam Input may be presenting it as a virtual device, or the controller exposes only an event device (no `js`). Modern games use event devices, so this is fine.
- Not listed at all: check the cable, the Bluetooth pairing, or whether the controller needs a driver (some Xbox wireless adapters need `xone`).

Where Linux gets it. `/sys/class/input/js*/device/{name,id/vendor,id/product,id/bustype}` and `/proc/bus/input/devices`.

Monitor ID database

`mon-info` - Linux data

In plain words. Every monitor carries a three-letter maker code (PnP ID) in its EDID, such as `GSM` for LG or `BNQ` for BenQ. This item shows the database that turns those codes into names, the codes of your connected monitors, and lets you search the database.

What you see.

- Database path, its date and how many vendor IDs it holds.
- **Connected monitors** - for each: PnP ID, maker name (or "(unknown vendor)"), product code (PnP ID plus 4 hex digits), monitor name and connector.
- A hint: open the item as `mon-info:<text>` to search, for example `mon-info:GSM` or `mon-info:BenQ`.
- With a search: every matching line (up to 200) as code and name. A few common monitor makers that the official list lacks are built into SRS and marked "(built-in, not in pnp.ids)". "no match" if nothing fits.

Healthy / typical values. On the author's PC: `/usr/share/hwdata/pnp.ids` dated 2026-10-04 with 2557 IDs; GSM LG Electronics GSM7766 LG ULTRAGEAR on DP-3; BNQ BenQ Corporation BNQ7F41 BenQ RL2755 on HDMI-A-1.

Unusual values and what they suggest.

- "(unknown vendor)": the code is not in the database. Update `hwdata`, or the monitor uses an unregistered code (common with cheap no-name monitors).
- "pnp.ids not found": install the `hwdata` package.

Where Linux gets it. `/usr/share/hwdata/pnp.ids` and connector EDID.

Monitors

`monitors` - Linux data

In plain words. Everything each monitor says about itself in its EDID: maker, model, serial number, manufacture date, size, supported refresh range, native mode, plus the current mode and adaptive sync state.

What you see.

First a summary table, one row per connected monitor ("Showing X of Y for \"):

Column	Meaning
PnP ID	Maker code plus product code (e.g. `GSM7766`).
Device Description	Maker name and model.
Device Name	Connector (`DP-3`).
Serial Number	Serial number text from EDID, or `-`.
EDID Version	1.3, 1.4...
Manufacture Year - Week	When the monitor was made.
Monitor Display Size	Width × height in cm.
Monitor Resolution	Native resolution.
Vertical Rates	Supported refresh range in Hz (or the native rate if the EDID has no range).
Horizontal Rates	Supported line-rate range in kHz (or `-`).

Then a detail block per monitor:

Field	Meaning
Heading	Connector, kernel name and connector ID.
Monitor	Model name.
Manufacturer	PnP ID and maker name ("(not in pnp.ids)" if unknown).
Product code	Hex product code.
Serial number	Serial text and the separate numeric serial in hex (shown as `XXXX` in this guide).
Manufactured / Model year	Year and week, or a model year if the maker gave only that.

Field	Meaning
EDID version	Version, checksum `OK` or `BAD`, number of extension blocks.
Extensions	Type of each extension: `CTA-861` (audio, HDR, extra modes for TVs and HDMI), `DisplayID` (newer timing data, common for high refresh rates), `VTB`, `DI`, `LS`, `DPVL`, `Block map`, `Manufacturer`.
Display size	Screen size in cm (rounded).
Image size	Exact picture size in mm, if given.
Input	Digital or Analog; for EDID 1.4 also bits per colour (`bpc`) and interface (DVI, HDMI-a, HDMI-b, MDDI, DisplayPort).
Gamma	Display gamma (typically 2.20).
Preferred mode	Native mode with exact refresh rate and pixel clock.
Vertical rates / Horizontal rates / Max pixel clock	From the range-limits block: refresh range, line-rate range, highest pixel clock.
Text	Free text in the EDID, if any.
Status	Connected or not, enabled or not, DPMS state.
Current mode	Mode in use now (needs `drm_info`).
Adaptive sync	`capable, enabled`, `capable, off now` or `not capable` (needs `drm_info`).

You can open `monitors:DP-3` to show only one output. The last line gives the date and size of the PnP ID database.

Healthy / typical values. On the author's PC:

- LG ULTRAGEAR on DP-3: EDID 1.4, checksum OK, 2 extensions (CTA-861 and DisplayID), made 2023 week 3, 70 × 39 cm (697 × 392 mm image, a 31.5" class screen), digital 10 bpc DisplayPort, gamma 2.20, preferred 2560 × 1440 @ 120.00 Hz (pixel clock 497.75 MHz), vertical 48 - 165 Hz, max pixel clock 800 MHz, current mode 2560×1440@164.96, adaptive sync capable, off now.
- BenQ RL2755 on HDMI-A-1: EDID 1.3, 1 extension (CTA-861), made 2015 week 38, 60 × 34 cm, digital, preferred 1920 × 1080 @ 60.00 Hz (148.50 MHz), vertical 50 - 76 Hz, horizontal 30 - 83 kHz, max pixel clock 170 MHz, adaptive sync not capable.
- Serial number: `XXXX` (each monitor has its own).

Unusual values and what they suggest.

- **checksum BAD**: the EDID got corrupted on the way (bad cable, cheap adapter or KVM switch), or the monitor's EDID chip is faulty. The driver may then fall back to safe low resolutions.
- **"no EDID"** for a connected output: the cable or adapter does not carry the DDC wires, or the monitor is off.
- **Display size 0 × 0 cm**: projectors and some TVs leave it out on purpose.
- **Manufacture year far in the future or before 1990**: a made-up value from the maker. Harmless.
- **Horizontal range with min = max**: some monitors fill in placeholder numbers. Trust the DisplayID or CTA extension data instead.

- **6 bpc on a monitor sold as 8-bit**: the panel may use dithering (FRC).

Where Linux gets it. `/sys/class/drm/card*-*/edid` (decoded by SRS itself), `/usr/share/hwdata/pnp.ids`, and `drm_info` for current mode and VRR.

Background. EDID (Extended Display Identification Data) is a 128-byte block, plus 128-byte extensions, stored in a small chip in the monitor. The GPU reads it over the DDC wires in the cable when the monitor is plugged in. It tells the computer who made the monitor, what modes it can show and its physical size, so the desktop can pick the right resolution and scaling.

Monitor tree

`monitor-tree` - Linux data

In plain words. The same tree as GPU tree, but only the outputs and monitors, without the PCI, driver and device-file branches.

What you see. For each card: card name and GPU name, then each output with `connected`/`disconnected` and `enabled`/`disabled`, and under connected outputs the monitor's PnP ID, name, maker and preferred resolution.

Healthy / typical values. On the author's PC: DP-1 and DP-2 disconnected; DP-3 connected enabled, GSM LG ULTRAGEAR LG Electronics 2560×1440; HDMI-A-1 connected enabled, BNQ BenQ RL2755 BenQ Corporation 1920×1080.

Unusual values and what they suggest. See GPU tree.

Where Linux gets it. `/sys/class/drm` connectors and EDID.

OpenCL

`opencl` - Linux data

In plain words. OpenCL is an open standard for running general computing tasks (not graphics) on the GPU, used by some photo, video, science and crypto programs. This item shows the OpenCL platforms (drivers) and the devices each one offers.

What you see.

- **Platforms** - how many OpenCL platforms are installed.
- **Platform** - one line per platform: name and version.
- Then a block per platform, headed by its name: **Devices** (count), then for each device:

Field	Meaning
Device Type	`CL_DEVICE_TYPE_GPU`, `CPU` or `ACCELERATOR`.
Board name	Marketing name (some drivers only).
Name	Device name (ROCm gives the `gfx` target).
Vendor	Driver vendor.
Max compute units	Number of compute units the driver reports.
Max clock frequency	Highest core clock.
Global memory size	Memory usable by kernels, in bytes.
Max memory allocation	Largest single buffer allowed, in bytes.
Local memory size	Fast on-chip shared memory per work-group, in bytes (64 KB on AMD).
Version / Device OpenCL C version	OpenCL version of the device and of its kernel language.
Driver version	Driver build.
Profile	`FULL_PROFILE` (full standard) or `EMBEDDED_PROFILE` (reduced feature set).

A note at the end names the source (`clinfo`) and the ICD folder. If `clinfo` is missing it says "not installed"; if it prints nothing: "no output (no OpenCL platform?)". Both kinds of `clinfo` are understood (2.4): ROCm's (`/opt/rocm/bin/clinfo`, "Key: value") and the cross-vendor one most distributions ship (`/usr/bin/clinfo`, "Key value"), whose key names are mapped onto the same rows; each platform shows its own version, and clinfo's lines about the NULL platform at the end are skipped.

Healthy / typical values. On the author's PC there are 2 platforms:

- **AMD Accelerated Parallel Processing** (ROCm), OpenCL 2.1 platform: RX 9070 XT, 32 compute units, 2400 MHz, global memory 17095983104 bytes (15.92 GiB), max allocation about 14.5 GB, local memory 65536, name `gfx1201`, OpenCL C 2.0, driver 3581.0, FULL_PROFILE.
- **rusticl** (Mesa), OpenCL 3.1: 64 compute units, global memory 17179869184 bytes, max allocation 2147483647 (2 GiB), OpenCL C 1.2, EMBEDDED_PROFILE.

The two drivers count compute units differently: the RX 9070 XT has 64 compute units, grouped in 32 pairs (WGPs, work-group processors). ROCm reports the pairs, rusticl reports the units.

Unusual values and what they suggest.

- No GPU platform, only a CPU one (for example PoCL): install `rocm-opencl-runtime` (AMD), or enable rusticl.

- Programs fail with "out of resources" on large jobs under rusticl: its 2 GiB maximum allocation is lower; use the ROCm platform for those.
- Platform listed but 0 devices: the driver cannot access the GPU, often because your user is not in the `render` group, or `/dev/kfd` is missing for ROCm.

Where Linux gets it. `clinfo`, which loads every ICD in `/etc/OpenCL/vendors`.

RSR info

`rsr-info` - Windows-only (Linux equivalent shown)

In plain words. RSR (Radeon Super Resolution) is a feature of AMD's Windows driver that runs a game at a lower resolution and upscales it with FSR 1 (FidelityFX Super Resolution). It does not exist in the Linux driver, so SRS shows the Linux tools that do the same job.

What it shows.

- **RSR** - "Windows-only (Adrenalin driver feature)".
- **Linux equivalent** - `gamescope -F fsr` (Valve's game compositor with FSR upscaling), `WINE_FULLSCREEN_FSR` (FSR built into Proton-GE and wine-tkg builds) and vkBasalt CAS (a sharpening filter).
- **gamescope** - installed or not.
- **vkBasalt** - installed or not (the program, or its Vulkan layer file).
- **WINE_FULLSCREEN_FSR** - current value of that environment variable, or "(not set)".

Typical values. On the author's PC: gamescope installed, vkBasalt installed, `WINE_FULLSCREEN_FSR` not set (normal: it is usually set per game in Steam launch options).

Where Linux gets it. File checks in `/usr/bin` and `/usr/share/vulkan/implicit_layer.d`, and the environment.

RTSS status

`rtss-sts` - Not present on most PCs

In plain words. RTSS (RivaTuner Statistics Server) is a Windows tool for on-screen statistics and frame-rate limits. It does not run on Linux. The nearest Linux tool is MangoHud.

What it shows.

- **RTSS** - "RivaTuner Statistics Server is Windows-only - not present".
- **Nearest** - "MangoHud (overlay + fps_limit)".
- **mangohud** - installed or not.
- **MANGOHUD env** - value of the `MANGOHUD` variable (`1` = on for all Vulkan programs), or "(not set)".
- **MangoHud.conf** and **fps_limit** - the config file path and its frame-rate limit (`0` = no limit), when the file exists.

Typical values. On the author's PC: mangohud installed, `MANGOHUD env 1`, `fps_limit 0`. For all MangoHud settings see GOverlay status.

Where Linux gets it. File check for `mangohud`, the environment, `~/.config/MangoHud/MangoHud.conf`.

Sound devices

`sound` - Linux data

In plain words. The sound cards, every ALSA device file they created, and the ALSA driver version.

What you see.

- The card list, as in Audio devices.
- **ALSA device nodes (/proc/asound/devices)** - one line per device: a minor number, `[card-device]`, and the type: `digital audio playback`, `digital audio capture`, `control` (the mixer), `hardware dependent` (the codec's own interface), `sequencer` (MIDI), `timer`.
- A note with the ALSA driver version line.

Healthy / typical values. On the author's PC: card 0 has four playback devices (3, 7, 8, 9: one per GPU output), a hardware-dependent node and a control; card 1 has playback 0 and 1, capture 0 and 2, a hardware-dependent node and a control; plus the global sequencer and timer. ALSA version follows the kernel ("k7.3.0-...").

Unusual values and what they suggest. A card with a control but no playback or capture devices: the codec was not recognised. Check `dmesg` for `snd_hda_codec` messages; a newer kernel or `linux-firmware` may help.

Where Linux gets it. `/proc/asound/cards`, `/proc/asound/devices`, `/proc/asound/version`.

UAA (HD Audio) devices

`uaa-dev` - Linux data

In plain words. UAA (Universal Audio Architecture) is Windows' name for the HD Audio standard. This item shows the audio chips (codecs) on each HD Audio controller, and which monitors are connected to the HDMI/DisplayPort audio outputs.

What you see.

- For each card and codec (`card0 codec#0`...):
  - **Codec** - chip name (e.g. Realtek ALC1220).
  - **Address** - codec slot on the HD Audio link (0-15).
  - **AFG Function Id** - the audio function group; `unsol` says whether the codec can send unsolicited events (used for jack detection).
  - **Vendor Id** - codec vendor and chip ID (`0x10ec` = Realtek, `0x1002` = AMD).
  - **Subsystem Id** - board maker and model ID (`0x1043` = ASUS).
  - **Revision Id** - chip revision.
- `eld#X.Y` lines - for each HDMI/DP audio pin with a monitor present: the monitor name and connection type, from its ELD.
- `hwCxDy` lines - the kernel's codec device nodes: vendor name, chip name and vendor ID.

If none are found: "No HD Audio codecs found".

Healthy / typical values. On the author's PC: card 0 codec "ATI R6xx HDMI" (the GPU's HDMI/DP audio; the name is historic) with ELDs "LG ULTRAGEAR (DisplayPort, ELD)" and "BenQ RL2755 (HDMI, ELD)"; card 1 codec Realtek ALC1220, vendor `0x10ec1168`, subsystem `0x1043891c` (ASUS); nodes `hwC0D0` and `hwC1D0`.

Unusual values and what they suggest.

- No ELD line for a monitor with speakers: the monitor did not report audio support, the output is in a mode without audio, or the cable or adapter drops audio.
- Codec name "Generic" or missing: the kernel does not know this codec model yet.

Where Linux gets it. `/proc/asound/cardN/codec#M`, `/proc/asound/cardN/eld#X.Y`, `/sys/class/sound/hwC*D*/{vendor_name,chip_name,vendor_id}`.

Background. HD Audio (High Definition Audio) splits a sound system into a controller (inside the chipset, CPU or GPU) and one or more codecs (the chips that turn digital into analog and drive the jacks). The GPU's HDMI audio is just another HD Audio controller with its own codec. ELD (EDID-Like Data) is the audio part of the monitor's EDID, passed to the sound driver: speaker layout and supported formats.

Wave devices

`wave` - Linux data

In plain words. "Wave" devices are Windows' name for basic digital audio playback and recording. This item lists every ALSA playback and capture device and whether a program is using it right now.

What you see. Two lists, **Wave out (playback)** and **Wave in (capture)**. Each line: `hw:card,device`, device name, and state: `closed`, or `open: <format> <channels>ch <rate>` with the sample format (e.g. `S32_LE` = signed 32-bit little-endian, `S16_LE` = 16-bit), channel count and sample rate.

Healthy / typical values. On the author's PC: playback HDMI 0-3 (`hw:0,3`, `hw:0,7`, `hw:0,8`, `hw:0,9`) closed, `hw:1,0` ALC1220 Analog closed, `hw:1,1` ALC1220 Digital "open: S32_LE 2ch 96000 (96000/1)" (PipeWire is sending 32-bit, 96 kHz stereo to the optical output); capture `hw:1,0` and `hw:1,2` closed.

Unusual values and what they suggest.

- A device you are not using shown open: some program holds it. With PipeWire, the sound server usually holds the active output open, which is normal.
- A program cannot open a device that is listed here as open: another program has it exclusively. Route through PipeWire instead of using `hw:` directly.

Where Linux gets it. `/proc/asound/pcm` and `/proc/asound/cardN/pcmD{p,c}/sub0/hw_params`.

Buses / Devices (part 1 of 2) (SRS Section Reference)

Back to the section index - Glossary - Part 2

Where to find these items (SRS 2.0). In the main window they are on the **USB & PCI devices** page (Alt+0). Its tabs are ALL Devices, PCI bus, USB bus, USB tree, HID bus and Thunderbolt; **All N items** lists the rest. You can also type an item's name or id into the search box (Ctrl+F). The rest of this introduction describes the classic view (Menu -> Classic View), with its Menu and panel buttons.

This group shows how the parts of your PC are wired together: the PCI Express (PCIe) bus that carries the graphics card and SSDs, the USB and HID (Human Interface Device) devices, the small management buses (SMBus / I2C), the old ISA / LPC (Low Pin Count) bus, serial ports, ACPI (Advanced Configuration and Power Interface) devices, and the I/O port and memory address maps. Open any item with **Menu -> Buses / Devices -> item**. The classic main window also has a **PCI Bus** button (its panel has the sub-buttons Properties, PnP Dev, Drives, PCI Dev, UAA Dev and ALL Dev) and a **USB Bus** button (sub-buttons Link Devices, USB Disks, HID Bus and USB Info). Most items work as a normal user. The items marked "Needs root data" show address ranges or raw register copies that Linux only gives to root. Run **Menu > Run as root (srs-root)** once to fill that cache. Until you do, those items show zeros and an orange "Needs root" line. In many panels you can **double-click a device ID** (a `VVVV:DDDD` pair such as `1022:14D8`) to open it in **Device ID Lookup**.

SRS never opens the risky device files these items are about (`/dev/port`, `/dev/mem`, `/dev/i2c-*`, `/dev/hidraw*`). It also never scans a bus. Everything comes from sysfs, procfs, udev and the srs-root cache.

Contents

- COM port resource arbiter `arbiter`
- Chipsets `chipsets`
- System device classes `class`
- Remote system classes `class-remote`
- Co-device installers `codevice`
- Compatible IDs `compatible`
- Device IDs `device-ids`
- All devices `devices`
- Device owners `dev-owner`
- Device property keys `devpropkey`
- Embedded controller I/O `embedded`

- System enum (device tree) `enum`
- Remote system enum `enum-remote`
- FireWire `firewire`
- Ghost (non-present) devices `ghost-devices`
- HID all devices `hid-all`
- HID bus `hid-bus`
- HID tree `hid-tree`
- Device interfaces `interfaces`
- Device interfaces (extended) `interfacex`
- Interface IDs `int-ids`
- Interface names `int-names`
- Interface owner `int-owner`
- ASUS I/O `io-asus`
- I/O port dump `io-dump`
- I/O GPIO `io-gpio`
- I/O PM2 MMIO `io-mmio`
- nForce I/O `io-nforce`
- I/O ports `io-ports`
- IrDA ports `irports`
- ISA bus `isa-bus`
- MCA bus `mca-bus`
- Chipset MCH `mch`
- Chipset MRH `mrh`
- NPU names `npu-names`
- PCI all `pci-all`
- PCI BARs `pci-bars`
- PCI bus `pci-bus`
- PCI devices `pci-dev`
- PCI flags `pci-flag`
- PCI HAL `pci-hal`
- PCI ID database `pci-info`
- PCI MMIO `pci-mmio`
- PCMCIA all `pcm-all`
- PCMCIA bus `pcm-bus`
- PCMCIA ID database `pcm-info`
- I/O PM1 data `pm1-area`
- PMBus `pmb-bus`
- PnP manager `pnp`
- PnP devices `pnp-dev`
- PnP ID database `pnp-info`
- PnP manager details `pnp-man`
- COM ports `ports`
- Device properties `properties`
- Device property `property`
- PYXIS 21174 `pyxis`
- Smart cards `scard`
- Serial COM ports `serial`
- SI USB `si-usb`

- SMBus `smb`
- SMBus devices `smb-bus`
- SMBus status `smb-sts`
- SMBus setup `smbus-setup`
- System SMBus access `smbus-system`

COM port resource arbiter

`arbiter` - Windows-only (Linux equivalent shown)

In plain words. Windows has an "arbiter" that hands out I/O addresses and IRQs (interrupt lines) to COM (serial) ports. Linux has no such component. SRS shows the serial port list instead, plus the I/O ranges that the serial driver has claimed.

What you see.

- The full Serial COM ports table (Port, I/O, IRQ, UART, Base clock, Device), the placeholder count line and the TTY driver list.
- **COM resources (Linux equivalent of the COM port arbiter)** - every line of the I/O port map that mentions `serial`, e.g. `03f8-03ff serial`. This is the address range the port really uses.

Healthy / typical values. On the author's PC: one real port, `ttyS0` at `0x3F8`, IRQ 4, and the claimed range `03f8-03ff serial`. COM1 has used these values since the IBM PC.

Unusual values and what they suggest. No `serial` range while a port is listed means the driver has not claimed it yet. Two devices on the same range would be a firmware conflict, but that is very rare today.

Where Linux gets it. `/sys/class/tty`, `/proc/tty/drivers`, `/proc/ioports` (addresses need the srs-root cache).

Chipsets

`chipsets` - Linux data

In plain words. This is a short list of the "glue" chips on the motherboard: host bridges, the IOMMU, the chipset's PCIe switch and its ports, the SMBus controller and the LPC bridge.

What you see. One line per device. The label on the left is the role SRS gives it:

Label	Meaning
Host Bridge	PCI class 0600. The link between the CPU and the PCI bus. On AMD this includes the Root Complex, the "Dummy Host Bridges" and the eight Data Fabric functions (00:18.0-7).
IOMMU	Class 0806. The input-output memory management unit. It controls which memory each device may reach by DMA (Direct Memory Access).
ISA/LPC Bridge	Class 0601. It connects the legacy LPC bus (Super I/O chip, TPM, BIOS flash).
SMBus Controller	Class 0C05. The controller for the small management bus that reads DIMM SPD chips and sensors.
Chipset Bridge	A PCI bridge whose name contains "Chipset", such as one port of the chipset's PCIe switch.
Chipset Function	Any other device named "Chipset", such as the chipset's USB or SATA controller.

After the label comes the PCI address, the vendor and device names, `[VVVV:DDDD rev RR]` and the driver if one is bound. The last line is the board name and BIOS version.

Healthy / typical values. On the author's ASUS TUF GAMING X870-PRO WIFI7 W NEO (Ryzen 9000 "Granite Ridge" CPU): the Root Complex `1022:14D8` with driver `ryzen_smu`, the IOMMU `1022:14D9`, and the "600 Series Chipset PCIe Switch Upstream Port" `1022:43F4` with its downstream ports `1022:43F5` (X870 boards use the same Promontory 21 chip). The chipset USB controller `1022:43FC` and SATA controller `1022:43F6` also appear, plus `00:18.3` Data Fabric Function 3 with driver `k10temp` (the CPU temperature driver).

Unusual values and what they suggest. If the SMBus controller has no driver, `piix4_smbus` is not loaded, and RAM and RGB tools will not find their devices. If there are no "Chipset" lines on a board that has a chipset, the PCI ID database is old (see PCI ID database).

Where Linux gets it. `/sys/bus/pci/devices/*/class`, `vendor`, `device`, `revision`, `driver`, with names from `/usr/share/hwdata/pci.ids`.

Background. On a modern AMD desktop the memory controller and most PCIe lanes are in the CPU. The "chipset" is a separate chip joined to the CPU by a PCIe x4 link. It acts like a PCIe switch with extra USB and SATA controllers built in.

System device classes

`class` - Linux data

In plain words. Linux sorts every device into a "class" by what it does (block disks, network, sound, input, hwmon sensors...). This item lists each class and the devices in it.

What you see. One row per directory in `/sys/class`:

- **Class name** - such as `block`, `drm` (graphics), `hwmon`, `net`, `tty`.
- **Count** - how many entries the class has.
- **Members** - their names, cut off with `...` after about 150 characters.

Healthy / typical values. On the author's PC: `block 27` (loop devices, three NVMe drives and their partitions, SATA disks), `cpuid 16` (one per logical CPU), `drm 8` (card0, its four connectors, a writeback connector, renderD128, version), `bluetooth 1 hci0`. Many classes are `0`, which is normal. They exist because the kernel supports them, not because hardware is present.

Unusual values and what they suggest. A count of 0 where you expect hardware, such as `net` without your network card or `drm` without `card0`, means the driver did not load. Check Device owners.

Where Linux gets it. `/sys/class/*`.

Background. A sysfs "class" is a view by function. The same device also appears under its bus (`/sys/bus/pci/...`) and at its place in the device tree (`/sys/devices/...`).

Remote system classes

`class-remote` - Windows-only (Linux equivalent shown)

In plain words. SIV on Windows can read the device classes of another PC over the network, through the remote registry. Linux has nothing like that.

What you see. `Status: Windows-only item in SIV` and `Linux equivalent: none (remote registry); nearest ssh + /sys/class`. To see another Linux PC's classes, log in with `ssh` and look at `/sys/class`, or run SRS there.

Co-device installers

`codevice` - Windows-only (Linux equivalent shown)

In plain words. Windows "co-installers" are small helper programs that run when a driver is installed. The closest thing on Linux is udev rules. These are text files that say what to do when a device appears: set permissions, make symlinks, load firmware, start a service.

What you see. Three folders, each with its file count and then every rule file in it:

- `/usr/lib/udev/rules.d` - rules shipped by packages.
- `/etc/udev/rules.d` - your own rules. A file with the same name here overrides the package one.
- `/run/udev/rules.d` - temporary rules made at run time.

Healthy / typical values. On the author's PC: 290 package rules (e.g. `30-amdgpu-pm.rules`, `10-rtl-sdr.rules`, `01-asrock-LED-controller.rules`) and 1 run-time rule (`30-drm-awaiter.rules`). The number at the front sets the order. Lower numbers run first.

Unusual values and what they suggest. A rule left behind by a removed program can still change device permissions. If a USB gadget works only as root, look here for a rule that should give access to your user and is missing.

Where Linux gets it. A file listing of the three `rules.d` folders.

Compatible IDs

`compatible` - Linux data

In plain words. Every device has a "modalias". This is one long ID string that Linux matches against each driver's list of supported hardware to pick the right driver. It is the Linux version of Windows "Compatible IDs".

What you see. Three groups, one line each:

- **PCI devices** (teal label = PCI address) - e.g. `pci:v00001022d000014D8sv00001043sd00008877bc06sc00i00`.
- **USB interfaces** (label = interface name such as `1-2:1.0`) - e.g. `usb:v2109p2817d0623dc09dsc00dp02ic09isc00ip02in00` (a VIA USB hub).
- **ACPI devices** (label = ACPI name such as `ACPI0007:00`) - e.g. `acpi:ACPI0007:`.

How to read a PCI modalias:

Part	Meaning
`v00001022`	Vendor ID 1022 (AMD)
`d000014D8`	Device ID 14D8

Part	Meaning
`sv00001043`	Subsystem vendor 1043 (ASUS, the board maker)
`sd00008877`	Subsystem device 8877 (the board's own number for this part)
`bc06 sc00 i00`	Base class 06 (bridge), subclass 00 (host), programming interface 00

For USB: `v`=vendor, `p`=product, `d`=device release, `dc/dsc/dp`=device class/subclass/protocol, `ic/isc/ip`=interface class/subclass/protocol, `in`=interface number.

Healthy / typical values. On the author's PC the Radeon RX 9070 XT reads `pci:v00001002d00007550sv00001EAEsd00008811bc03sc00i00`: AMD/ATI chip 7550, built by subsystem vendor 1EAE (XFX), class 03 (display).

Unusual values and what they suggest. A subsystem of `sv00000000sd00000000` is normal for internal bridges. On a plug-in card it means the maker did not program one. You can add a filter after the item name (e.g. `compatible:03:00`) to show only matching devices.

Where Linux gets it. `/sys/bus/pci/devices/*/modalias`, `/sys/bus/usb/devices/*:*/modalias`, `/sys/bus/acpi/devices/*/modalias`.

Device IDs

`device-ids` - Linux data

In plain words. This shows every PCI and USB device's ID, and how SRS turned that ID into a name. It shows which entry in the ID database matched.

What you see.

Column	Meaning
Device	The PCI address (`03:00.0`) or the USB device name (`5-1`, `usb3`).
Lookup Key	For PCI: `Vendor:Device:SubVendor:SubDevice:Revision`. Internal parts with no subsystem use `Vendor:Device:Revision`. For USB: `Vendor:Product`.
Matched	Which step of the lookup chain found a name. For PCI: `rung 0` is the most exact. For USB: `V:D` (vendor and product found), `V` (only the vendor is known) or `none`.
Name	The vendor name, then the device name from the step that matched.

The PCI lookup chain for a full key goes from most to least exact: rung 0 `V:D:S:s:RR` -> rung 1 `V:D:S:s` -> rung 2 `V:D:RR` -> rung 3 `V:D` -> rung 4 `V`. For a short key (no subsystem) it is rung 0 `V:D:RR` -> rung 1 `V:D` -> rung 2 `V`.

Double-click any key to open it in **Device ID Lookup**, which shows every step of the chain side by side.

Healthy / typical values. On the author's PC most PCI rows match at `rung 3` (the generic vendor:device name). The Samsung SSD matches at `rung 1`: key `144D:A80A:144D:A801`, where the subsystem entry gives the retail name "SSD 980 PRO" instead of the chip name. Internal bridges like `1022:14DA:00` match at `rung 1` (their short-key V:D). On the USB side the Razer mouse `1532:005C` is `V:D` "DeathAdder Elite". Some devices are only `V` (the SteelSeries keyboard `1038:1612`) or `none` (`9129:1583`).

Unusual values and what they suggest. `none` on PCI means the vendor is missing from the database. This happens with brand-new hardware or a very old `pci.ids`. Update the `hwdata` package. `V` or `none` on USB is common for cheap gadgets and is harmless. The device's own name strings still show in Device owners.

Where Linux gets it. sysfs `vendor`, `device`, `subsystem_vendor`, `subsystem_device`, `revision` and `idVendor/idProduct`, looked up in `/usr/share/hwdata/pci.ids` and `usb.ids`.

Background. A PCI device carries two pairs of IDs. The **vendor/device ID** names the chip (who made the silicon). The **subsystem vendor/device ID** names the product built around it (who made the card or board). The same Navi 48 chip `1002:7550` shows up with a different subsystem on every card brand. The **revision ID** tells chip steppings apart.

All devices

`devices` - Linux data

In plain words. This is every device udev knows about: real hardware, virtual devices and kernel helpers. It is the Linux equivalent of Device Manager with "show hidden devices" on.

What you see.

Column	Meaning
Device	The device path under `/sys/devices` (the leading `/devices/` is dropped), such as `pci0000:00/0000:00:01.2/0000:04:00.0/nvme/nvme0/nvme0n1`.
Subsystem	The bus or class it belongs to: `pci`, `usb`, `block`, `net`, `acpi`, `workqueue`...
Driver	The bound driver, if any.
Node	Its `/dev` node, if any (e.g. `nvme0n1`, `bus/usb/002/002`).

At the end: `Total of N devices in the udev database`.

Healthy / typical values. On the author's PC: 1583 devices. The list starts with the ACPI tree (`LNXSYSTM:00/...`) and ends with `virtual/workqueue/...` entries. Hundreds of rows with no driver are normal. Most entries are containers, ports or virtual objects.

Unusual values and what they suggest. A real device (PCI or USB) with an empty Driver column has no driver. A missing device that you can see in `lspci` means udev has not processed it. This is rare, and restarting `systemd-udevd` fixes it.

Where Linux gets it. `udevadm info --export-db` (read only).

Device owners

`dev-owner` - Linux data

In plain words. For each PCI and USB device, this shows which driver "owns" it and which kernel module that driver lives in.

What you see.

Column	Meaning
Device	The PCI address or USB device name.
Driver	For PCI, the bound driver or `(none)`. For USB, every interface driver, comma-separated (e.g. `usbhid, usbfs`).
Module	For PCI, the module file the driver comes from, or `built-in` if it is compiled into the kernel. It is blank when there is no driver, and blank for USB.
Description	For PCI, the device name. For USB, the maker and product strings the device reports about itself. Root hubs show `xHCI Host Controller - <controller name>`.

Healthy / typical values. On the author's PC: GPU `03:00.0` -> `amdgpu`, NVMe drives -> `nvme`, the 5GbE card -> `r8169`, PCIe bridges -> `pcieport` (module `pcieportdrv`), USB hubs -> `hub`, the webcam -> `uvcvideo`, the Bluetooth radio -> `btusb`. The IOMMU and "Dummy Host Bridges" show `(none)`. That is normal, because they need no driver.

Unusual values and what they suggest.

- A plug-in card or USB gadget with `(none)` has no driver: it is unsupported, or its module is not loaded.
- `usbfs` means a user-space program (OpenRGB, a game launcher, a printer driver) has claimed the interface directly. This is expected for RGB keyboards and printers.
- Garbled description text (stray symbols) comes from the device's own firmware strings, not from SRS.

Where Linux gets it. `/sys/bus/pci/devices/*/driver`, `.../driver/module`, `/sys/bus/usb/devices/*:*/driver`, plus the USB `manufacturer` and `product` files.

Device property keys

`devpropkey` - Linux data

In plain words. udev stores key=value "properties" for each device. This item lists every property name in use and how many devices have it. Use it to find a key you can then look up with Device property.

What you see. One line per key, A-Z, with `N devices`. The last line is `Total of K keys across D devices`.

Healthy / typical values. On the author's PC: 298 keys across 1583 devices. Common ones: `DEVPATH` (every device), `DEVNAME` (370 devices with a /dev node), `DRIVER` (258), `HID_ID` and `HID_NAME` (18), `BUSNUM/DEVNUM` (40 USB devices).

Where Linux gets it. The `E:` lines of `udevadm info --export-db`.

Background. Keys such as `ID_VENDOR_FROM_DATABASE` come from udev's hardware database (hwdb). Keys such as `ID_SERIAL` and `ID_PATH` come from udev helper programs. Desktop software uses them to decide how to treat a device.

Embedded controller I/O

`embedded` - Linux data - Needs root data

In plain words. The EC (Embedded Controller) is a small microcontroller on the motherboard. It handles things like power buttons, battery and fans on laptops, and some sensors on desktops. This item shows where it lives. It does not read it.

What you see.

- **ACPI device** - `PNP0C09:00 (ACPI Embedded Controller)`, or "No ACPI PNP0C09 device".
- **Namespace path** - its place in the firmware's ACPI tables, e.g. `\_SB_.PCI0.SBRG.EC0_`.
- **Bound to** - the Linux device it is attached to, or `(ACPI core EC driver)`.
- **ECDT table** - `present` if the firmware gives an early EC description table, otherwise `absent (EC described in DSDT only)`.

- **ec_sys debugfs** - whether the `ec_sys` debug module (which lets root read EC RAM) is loaded.
- The EC's I/O ports from the port map: on PCs, `0062` = EC data and `0066` = EC command.

Healthy / typical values. On the author's desktop the EC is at `\_SB_.PCI0.SBRG.EC0_`, there is no ECDT, `ec_sys` is not loaded, and the ports are `0062` and `0066`. That is a standard layout.

Unusual values and what they suggest. No EC at all is fine on many desktops. On a laptop, a missing EC explains missing battery or fan readings. SRS never reads EC RAM, because traffic on 0x62/0x66 can collide with the firmware.

Where Linux gets it. `/sys/bus/acpi/devices/PNP0C09:00`, `/sys/firmware/acpi/tables/ECDT`, `/proc/ioports` (addresses from the srs-root cache).

System enum (device tree)

`enum` - Linux data

In plain words. These are the same devices as All devices, drawn as an indented tree, so you can see what is plugged into what.

What you see. One line per device: its short name, indented by depth, then `[subsystem]`. Top-level entries are in bold. For example:

```
LNXSYSTM:00  [acpi]
  LNXPWRBN:00  [acpi]
  LNXSYBUS:00  [acpi]
    ACPI0010:00  [acpi]
      ACPI0007:00  [acpi]
```

Healthy / typical values. On the author's PC the tree starts with the ACPI root and its 16 processor objects (`ACPI0007:00` to `0f`, one per logical CPU). Then come the PCI tree (`pci0000:00`), platform devices and the virtual devices. There are about 1580 lines.

Unusual values and what they suggest. If a device sits under an unexpected parent (an NVMe drive under the chipset instead of the CPU, say), it is in an M.2 slot wired to the chipset. That slot is slower and shares bandwidth.

Where Linux gets it. `udevadm info --export-db`, sorted by device path.

Remote system enum

`enum-remote` - Windows-only (Linux equivalent shown)

In plain words. SIV on Windows can list another PC's device tree through the remote registry. Linux has no such feature.

What you see. `Status: Windows-only item in SIV` and `Linux equivalent: none (remote registry); nearest ssh + udevadm`. On the other PC, run `udevadm info --export-db` over `ssh`, or run SRS there.

FireWire

`firewire` - Not present on most PCs

In plain words. FireWire (IEEE 1394) is an older high-speed bus, used for DV camcorders and pro audio interfaces. Modern boards do not have it.

What it would show. One `1394 controller` line per FireWire card (PCI class 0C00), and whether `/sys/bus/firewire` exists.

What "not present" means. On the author's PC: `/sys/bus/firewire absent` and "No PCI class 0C00 (FireWire) controller present." Nothing is wrong. You would need an add-in card.

Ghost (non-present) devices

`ghost-devices` - Windows-only (Linux equivalent shown)

In plain words. Windows keeps every device ever plugged in, even after it is gone. These are "ghost devices". Linux forgets a device when it is removed. SRS checks the closest thing: udev records in `/run/udev/data` whose device no longer exists.

What you see. One orange line per stale record: `stale udev record (device not present)`. Or, if there are none: "No stale entries in /run/udev/data: Linux drops records when devices go away." A second note explains the Windows difference.

Healthy / typical values. No stale entries, as on the author's PC.

Unusual values and what they suggest. A few stale records after hot-unplugging can happen and are harmless. They vanish at reboot, because `/run` is in memory.

Where Linux gets it. `/run/udev/data` compared with `/sys/dev`, `/sys/bus` and `/sys/class`.

HID all devices

`hid-all` - Linux data

In plain words. HID (Human Interface Device) is the standard way keyboards, mice, game pads, RGB controllers and security keys talk to the PC. This item gives a full card for each HID device.

What you see. For each device, a title line `<bus>:<VID>:<PID>.<instance> <name>`, then:

Field	Meaning
Bus	The bus number and name (`0003 USB`, `0005 Bluetooth`, `0018 I2C`, `0006 Virtual`…), then `VID_xxxx PID_xxxx`.
usb.ids	For USB devices only: the vendor and product names from the USB ID database.
Driver	The HID driver: `hid-generic`, or a special one like `logitech-djreceiver`, `razermouse`, `nintendo`.
Physical	Where it is attached, e.g. `usb-0000:0b:00.0-8.4.4/input1:1` (controller, port path, interface).
Unique ID	The device's own unique ID or serial, if it reports one (shown only when present).
Parent	The USB interface or other device it hangs off.
Report descriptor	The size in bytes of the descriptor that tells the PC what buttons, axes and LEDs the device has.
hidraw node	The raw access node (`hidrawN`). It is listed but never opened.
Input devices	The `inputN` devices the kernel made from it (the keyboard and mouse events your desktop reads).
Modalias	The driver-matching ID, e.g. `hid:b0003g0102v0000046Dp00004057`.

Healthy / typical values. On the author's PC: a Logitech mouse `046D:4057` behind a Nano Receiver `046D:C52F` (driver `logitech-hidpp-device`, parent `0003:046D:C52F.000F`, report descriptor 192 bytes, Unique ID `XXXX`). Also a Nintendo Switch Pro Controller with driver `nintendo`, and a Razer DeathAdder with driver `razermouse`.

Unusual values and what they suggest.

- A game pad on `hid-generic` instead of its own driver may lose rumble, LEDs or the gyro.

- No `Input devices` on a keyboard or mouse interface is normal for vendor-only interfaces (RGB, macro setup). It is a problem only if typing does not work.
- A tiny report descriptor (a few bytes) can mean a broken or half-initialised device.

Where Linux gets it. `/sys/bus/hid/devices/*` (`uevent`, `report_descriptor`, `hidraw/`, `input/`).

Background. One physical USB device often exposes several HID interfaces. A gaming keyboard can have one for keys, one for media keys and one for lighting, so it shows up several times here.

HID bus

`hid-bus` - Linux data

In plain words. This is a one-line-per-device table of the HID bus. It is the **HID Bus** button in the USB Bus panel.

What you see.

Column	Meaning
HID Device	`bus:VID:PID.instance`, e.g. `0003:1532:005C.0001`.
Bus	USB, Bluetooth, I2C, SPI, Virtual, Host or Other.
Name	The name the device reports.
Driver	The HID driver bound to it.
Nodes	The `hidrawN` node and any `inputN` devices.

The last line is `Total of N HID devices`. You can filter, e.g. `hid-bus:1532`.

Healthy / typical values. On the author's PC: 15 HID devices. The SteelSeries Apex 7 keyboard appears four times (four interfaces), the ASUS AURA LED controller twice, and the Razer DeathAdder three times. There is also a "Virtual FIDO/U2F security Key" `0003:FFFF:FFFF` made by software.

Unusual values and what they suggest. A device you can see on the USB bus but not here is not a HID device, or its HID driver failed. Check the kernel log.

Where Linux gets it. `/sys/bus/hid/devices`.

HID tree

`hid-tree` - Linux data

In plain words. This groups HID devices under the USB device they belong to, so you can see which interfaces of one gadget are HID.

What you see. For each USB device that has HID interfaces, a header line `<usb name> VID:PID <description>`, then one line per HID interface: `|__ <usb interface> <HID device> [driver] <input devices>`. HID devices that are not on USB (Bluetooth, I2C touchpads, virtual) are listed at the end with their bus name.

Healthy / typical values. On the author's PC:

```
5-2  1038:1612 SteelSeries SteelSeries Apex 7
  |__ 5-2:1.0  0003:1038:1612.0006  [hid-generic] input6
  |__ 5-2:1.2  0003:1038:1612.0008  [hid-generic] input7
```

Double-click a `VID:PID` such as `1038:1612` to open it in Device ID Lookup. If the lookup window opens on the PCI bus, switch it to USB.

Where Linux gets it. `/sys/bus/hid/devices` matched to `/sys/bus/usb/devices`.

Device interfaces

`interfaces` - Linux data

In plain words. On Windows, "device interfaces" are the named access points programs open. On Linux the matching idea is the `/sys/class` entries, so this item shows the same table as System device classes (its window title says so too).

What you see. Class name, count, members (see System device classes). For more detail use the extended form Device interfaces (extended).

Where Linux gets it. `/sys/class`.

Device interfaces (extended)

`interfacex` - Linux data

In plain words. This lists every single `/sys/class` entry with its device number, the hardware behind it and its driver.

What you see.

Column	Meaning
class/name	e.g. `drm/card0`, `hidraw/hidraw0`, `block/nvme0n1`.
dev_t	The major:minor device number, if it has a `/dev` node (e.g. `226:0`).
Device	The hardware it belongs to (e.g. PCI `0000:03:00.0`, HID `0003:1532:005C.0001`, `nvme0`).
Driver	That hardware's driver.

Healthy / typical values. On the author's PC: `drm/card0 226:0 0000:03:00.0 amdgpu` and `hidraw/hidraw0 243:0 0003:1532:005C.0001 razermouse`. Many entries (bdi, virtual ones) have empty columns. That is normal.

Where Linux gets it. `/sys/class/*/*/dev`, `.../device`, `.../device/driver`.

Background. A **dev_t** is the pair of numbers the kernel uses to find a device file. The *major* picks the driver (259 = NVMe/block extended, 226 = DRM graphics). The *minor* picks one device.

Interface IDs

`int-ids` - Linux data

In plain words. These are only the `/sys/class` entries that have a device file, with their device number and `/dev` path.

What you see. class/name, dev_t (`major:minor`) and the `/dev/...` node name from udev.

Healthy / typical values. On the author's PC: `block/nvme0n1 259:5 /dev/nvme0n1`, `block/sda 8:0 /dev/sda`. Block disks use major 8 for SATA/SCSI and 259 for NVMe.

Unusual values and what they suggest. An entry with a dev_t but no `/dev` path means udev did not make the node. Programs will not find the device.

Where Linux gets it. `/sys/class/*/*/dev`, plus the `N:` (node) line in the udev database.

Interface names

`int-names` - Linux data

In plain words. This shows the friendly symlink udev made for each `/sys/class` entry, such as `/dev/disk/by-label/EFI`.

What you see. class/name and the first udev symlink for it (relative to `/dev`), or `-` if there is none.

Healthy / typical values. On the author's PC: `block/loop0 disk/by-diskseq/142`, `block/nvme0n1p1 disk/by-path/pci-0000:04:00.0-nvme-1-part/by-label/EFI`. Most non-disk entries show `-`.

Unusual values and what they suggest. A disk with no `by-id` or `by-uuid` names can make `/etc/fstab` entries that use those names fail at boot.

Where Linux gets it. The `S:` (symlink) lines of the udev database.

Interface owner

`int-owner` - Linux data

In plain words. This shows which driver and kernel module own each `/sys/class` entry.

What you see. class/name, Driver (or `-`), Module (or `built-in`), Device (the parent hardware).

Healthy / typical values. On the author's PC: `drm/card0 amdgpu amdgpu 0000:03:00.0`, `input/input25 razermouse razermouse 0003:1532:005C.0001`.

Where Linux gets it. `/sys/class/*/*/device/driver` and `.../driver/module`.

ASUS I/O

`io-asus` - Linux data - Needs root data

In plain words. ASUS boards keep their fan and voltage sensors in a Super I/O chip and, on some models, the EC. This item shows which ASUS-related drivers are present. The sensor values themselves are in the Sensors group.

What you see.

- **Board** - the board vendor and name from SMBIOS (System Management BIOS).
- **hwmonN** - each Nuvoton (`nct...`) or ASUS hwmon chip, with a pointer to `[sensors]`.
- **asus-wmi** and **asus_ec_sensors** - `loaded` or `not loaded`.
- I/O port ranges claimed by `nct`, `asus` or `it87` drivers, or "no matching entries".

Healthy / typical values. On the author's ASUS TUF GAMING X870-PRO WIFI7 W NEO: `hwmon4 nct6799`, with `asus-wmi` and `asus_ec_sensors` not loaded. "No matching entries" in the port map is normal, because the `nct6775` driver's port shows up under the Super I/O name, not under "nct".

Unusual values and what they suggest. No `nct` chip on an ASUS board usually means the `nct6775` module is not loaded, or the board needs `acpi_enforce_resources=lax`. `asus_ec_sensors` adds extra readings (chipset temperature, water flow) on supported ROG/TUF models.

Where Linux gets it. `/sys/class/dmi/id/board_*`, `/sys/class/hwmon/*/name`, `/sys/module/*`, `/proc/ioports` (srs-root cache).

I/O port dump

`io-dump` - Linux data - Needs root data

In plain words. SIV can dump the raw contents of the PC's I/O ports. SRS deliberately does not, because reading some ports can confuse the SMBus or EC. It shows which driver owns which port range instead.

What you see. A note saying why `/dev/port` is not read, then the full I/O port map (the same as I/O ports).

Where Linux gets it. `/proc/ioports` (srs-root cache for real addresses).

I/O GPIO

`io-gpio` - Linux data - Needs root data

In plain words. GPIO (General Purpose Input/Output) pins are simple on/off signals the chipset uses for LEDs, buttons, and enabling or resetting other chips. This item lists the GPIO controllers.

What you see. One line per GPIO chip: `gpiochipN <ACPI device> <name> driver <driver>`. Then, with the root data (2.4), **Pin states**: the kernel's own GPIO table as srs-root copied it - per chip its lines, and for AMD's pin controller one row per pin with interrupt, trigger, wake, pull (Ъup, Ђdown), direction and level (`input ↑`, `output ↓`), debounce and the raw register value. Tabs are expanded so the rows stay aligned. Without root data, a note says how to get it (Menu > Run as root).

Healthy / typical values. On the author's PC: `gpiochip0 AMDI0030:00 AMD GPIO Controller driver amd_gpio` (the FCH (Fusion Controller Hub) pins) and `gpiochip1 AMDIF031:00 AMD GPIO Controller driver pt-gpio` (the chipset's pins); 256 pins on gpiochip0, most of them unused inputs.

Unusual values and what they suggest. "no GPIO chips" on an AMD board means `pinctrl_amd` is not loaded. Some laptops need it for the touchpad interrupt.

Where Linux gets it. `/sys/bus/gpio/devices`; pin states from `/sys/kernel/debug/gpio` (root, read only), copied by srs-root into `gpio.txt`.

I/O PM2 MMIO

`io-mmio` - Linux data - Needs root data

In plain words. AMD chipsets put their power-management, SMBus, GPIO and pin-mux registers in one memory-mapped block at `0xFED80000`. This item shows the memory-map lines around it. MMIO (Memory-Mapped I/O) means hardware registers that look like memory addresses.

What you see. A note (registers are not read; no `/dev/mem`), then every line of the physical memory map that mentions AMDI devices, IOAPIC, HPET, Local APIC, `amd_iommu`, `pnp` or `Reserved`.

Healthy / typical values. On the author's PC: `fec00000-fec003ff IOAPIC 0`, `fed00000-fed003ff HPET 0`, `fed81500-fed818ff AMDI0030:00` (the GPIO block inside the FCH region), `f7600000-f767ffff amd_iommu`, and several `Reserved` ranges for firmware.

Where Linux gets it. `/proc/iomem` (srs-root cache; non-root readers see zeros).

---

nForce I/O

`io-nforce` - Not present on most PCs

In plain words. These would be the I/O registers of NVIDIA nForce chipsets (about 2001-2008). No current PC uses one.

What you see. `Status: Not present on this system`, `Checked: n/a (no NVIDIA nForce chipset)`.

I/O ports

`io-ports` - Linux data - Needs root data

In plain words. x86 PCs have a separate 64K "I/O port" address space, left over from the original PC. This is the map of who owns which ports.

What you see. One line per range: `start-end owner`, indented to show nesting (a range inside a bigger one). Owners are drivers or firmware devices.

Healthy / typical values. From the author's PC, all classic PC values:

Range	Owner
`0020-0021`, `00a0-00a1`	pic1, pic2 (the two old interrupt controllers)
`0040-0043`	timer0 (the PIT, the original PC timer)
`0060`, `0064`	keyboard (the PS/2 controller)
`0062`, `0066`	EC data, EC cmd
`0070-0071`	rtc0 (the CMOS real-time clock)
`0cf8-0cff`	PCI conf1 (the old way to reach PCI configuration)
`0290-029f`	reserved by the firmware (`PNP0C02`) for the Super I/O hardware monitor (the `nct6775.656` device: 656 = 0x290)
`0800-0827`	ACPI PM1, PM timer and GPE blocks
`0b00-0b28`	piix4_smbus

Unusual values and what they suggest. If the addresses show as `0000-0000`, you are not root and there is no srs-root cache. The orange line tells you how to fill it. Two drivers on one range can mean a firmware resource conflict (the kernel log says "resource busy").

Where Linux gets it. `/proc/ioports`.

IrDA ports

`irports` - Not present on most PCs

In plain words. IrDA was infrared data transfer, used on old laptops and phones. Linux removed IrDA support in kernel 4.17.

What you see. `Status: Not present on this system`, `Checked: n/a (no IrDA)`.

ISA bus

`isa-bus` - Linux data

In plain words. The ISA (Industry Standard Architecture) bus is gone, but its job lives on in the LPC bus. LPC is a slim bus that links the chipset to the Super I/O chip, TPM, serial port and BIOS flash. This item shows the LPC bridge and what sits behind it.

What you see.

- **LPC bridge** - the PCI class 0601 device, with its driver (often `(none)`, which is normal).
- **ISA bus devices** - how many old-style ISA drivers are probing (usually 0).
- **LPC devices enumerated by ACPI PnP** - the same resource map as PnP manager.
- **Super I/O** - hwmon chips from Nuvoton (`nct`), ITE (`it8`) or Fintek (`f718`), with their platform device.

Healthy / typical values. On the author's PC: LPC bridge `0000:00:14.3 FCH LPC Bridge`, 0 ISA devices, one PnP device (`PNP0501` COM port at `io 0x3f8-0x3ff`, `irq 4`), and the Super I/O `hwmon4 nct6799 (nct6775.656)`.

Where Linux gets it. `/sys/bus/pci/devices` (class 0601), `/sys/bus/isa/devices`, `/sys/bus/pnp/devices`, `/sys/class/hwmon`.

Background. The **Super I/O** chip handles the slow, old-style jobs: the serial port, PS/2 and fan/voltage monitoring. On most desktop boards it is the chip SRS reads fan speeds from.

MCA bus

`mca-bus` - Not present on most PCs

In plain words. MCA (Micro Channel Architecture) was IBM's PS/2 expansion bus from 1987. Linux dropped it long ago.

What you see. `Status: Not present on this system`, `Checked: n/a (no MicroChannel)`.

Chipset MCH

`mch` - Linux data - Needs root data

In plain words. On older Intel PCs the MCH (Memory Controller Hub, the "northbridge") held the memory controller. AMD has no MCH. The nearest thing is the Root Complex at `00:00.0`, so SRS shows that device in full.

What you see. The full PCI card for `0000:00:00.0`:

Field	Meaning
Location	`[ bus - device - function ]` in SIV style.
IDs	`VEN_ DEV_ SUBSYS_ REV_` in Windows style.
Lookup key	The key used for the name lookup. Double-click it for Device ID Lookup.
Class	The 6-digit class code with its class and sub-class names.
Subsystem	Subsystem vendor:device and the vendor name.
Parent bridge	The upstream bridge, or `root bus`.
Driver	The driver and module.
IRQ, NUMA node, Local CPUs	The interrupt, memory node (-1 = single node) and the CPUs closest to it.
PCIe link, Power state, IOMMU group	Shown when the device has them.
BAR / ROM lines	Its memory windows, if any.
Modalias	The driver-matching ID.

Then a note on why AMD has no MCH, then (with the srs-root cache) the `lspci -vvv` text and the hex dump of its configuration space.

Healthy / typical values. On the author's PC: `1022:14D8`, subsystem `1043:8877 ASUSTeK Computer Inc.`, class `060000 Bridge / Host bridge`, driver `ryzen_smu`, NUMA node `-1`, local CPUs `0-15`, power state `D0`.

Where Linux gets it. `/sys/bus/pci/devices/0000:00:00.0/*`; configuration space from the srs-root cache (`pci-all.txt`).

Background. On Ryzen the memory controllers (UMC, Unified Memory Controller) are part of the CPU's Data Fabric. Their settings are reached through SMN (System Management Network). SRS sends no commands to the CPU's management firmware; since 2.4 srs-root only reads the memory controller's timing registers through the ryzen_smu driver (Ryzen 7000 / 9000 desktop), for the Memory timings item.

Chipset MRH

`mrh` - Not present on most PCs

In plain words. The MRH (Memory Repeater Hub) was an Intel 820/840-era chip (1999-2000) that let RDRAM boards use SDRAM. No current PC has one.

What you see. `Status: Not present on this system`, `Checked: n/a (Intel memory repeater hub)`.

NPU names

`npu-names` - Not present on most PCs

In plain words. An NPU (Neural Processing Unit) is an AI accelerator. AMD Ryzen AI laptop chips have one (XDNA). Linux lists such devices under `/sys/class/accel`.

What you see.

- `/sys/class/accel N devices`.
- `amdxdna loaded / not loaded` (the AMD NPU driver).
- A note: "Desktop Ryzen 9000 (Granite Ridge) has no XDNA NPU."

What "not present" means. 0 devices on a desktop Ryzen is correct. On a Ryzen AI laptop with 0 devices, install or load `amdxdna` (kernel 6.14 or newer).

PCI all

`pci-all` - Linux data - Needs root data

In plain words. This is everything SRS knows about every PCI device, in one long report.

What you see.

- **With the srs-root cache:** "Full configuration space (lspci -vvvnnxxx, srs-root cache)", then for each device the full `lspci -vvv` text (Control/Status flags, capabilities, link status, ASPM, AER) and a hex dump of its configuration space (256 bytes, or 4096 for PCIe).
- **Without it:** the same per-device card as Chipset MCH for every device (Location, IDs, Lookup key, Class, Subsystem, Parent bridge, Driver, IRQ, NUMA node, Local CPUs, PCIe link, Power state, IOMMU group, BARs, Modalias), then an orange line saying extended configuration space needs root.

Healthy / typical values. On the author's PC the root cache holds about 3700 lines for 54 devices. Useful lines in the `lspci` text: `LnkCap` (what the link can do), `LnkSta` (what it is doing now) and `Kernel driver in use`.

Unusual values and what they suggest. `LnkSta` showing `(downgraded)`. Uncorrectable error bits set under AER (Advanced Error Reporting). `DevSta: CorrErr+` that keeps coming back points to a weak link: reseat the card, or set the slot to a lower PCIe generation in the BIOS.

Where Linux gets it. sysfs per device; the root cache holds `lspci -vvvnnxxx` output.

PCI BARs

`pci-bars` - Linux data

In plain words. A BAR (Base Address Register) is a window of addresses that a PCI device claims so the CPU can talk to it. This item lists every window: where it is, how big it is and what kind it is.

What you see.

Column	Meaning
Location	The PCI address.
BAR	`BAR 0-BAR 5`, or `ROM` (the option ROM / VBIOS window).

Column	Meaning
Start / End	The address range (hex).
Type	`Memory` or `I/O`, plus `Prefetch` (safe to read ahead and cache) and `64-bit` (can sit above 4 GB), and the size.
Device	The device name.

At the end: `< 4GB Total` and `> 4GB Total`, the memory BAR space below and above the 4 GB line. You can filter by slot, e.g. `pci-bars:03:00.0`.

Healthy / typical values. On the author's PC the Radeon RX 9070 XT's `BAR 0` is `Memory Prefetch 64-bit 16GB` at `0x1F800000000`. That is the whole 16 GB of VRAM visible to the CPU. Totals: 6.70 MB below 4 GB and 16.25 GB above.

Unusual values and what they suggest. If a modern graphics card's big BAR is only `256MB`, Resizable BAR (Smart Access Memory) is off. To turn it on, enable "Above 4G Decoding" and "Re-Size BAR" in the BIOS. This can help some games. A BAR with no address (all zeros) was not assigned: the kernel ran out of address space for it, and the kernel log will say "can't assign".

Where Linux gets it. `/sys/bus/pci/devices/*/resource` (start, end and flags for each BAR).

PCI bus

`pci-bus` - Linux data

In plain words. This is the main PCI view: every PCI and PCIe device in a tree, with its name, type, driver and link speed. It is the **PCI Bus** button on the classic main window, and the **PCI bus** tab of the **USB & PCI devices** page.

What you see.

Column	Meaning
PCI Location	`bus:device.function`, indented under the bridge it sits behind.
Vendor and Device	Names from the PCI ID database.
Device Class	e.g. `VGA compatible controller`, `Non-Volatile memory controller`, `PCI bridge`.
Driver	The bound driver, or `-`.
Link	For PCIe devices: `xW S GT/s PCIe (max xW S GT/s PCIe)`, the current width and speed, then the maximum. It is blank for devices with no PCIe link.

Footer: `Total of N PCI devices (B bridges) in T seconds using <pci.ids> Date ... Version ...`, then the board and BIOS line.

The panel buttons go to Device properties, PnP devices, PCI devices and the drive, UAA (audio) and all-device views.

Healthy / typical values. On the author's PC (54 devices, 35 bridges):

- GPU `03:00.0` Navi 48 -> `amdgpu`, `x16 32.0 GT/s PCIe (max x16 32.0 GT/s PCIe)`. That is the full PCIe 5.0 x16, through the card's own internal switch (`01:00.0/02:00.0`).
- Samsung 980 PRO `04:00.0` on the CPU's M.2 slot: `x4 16.0 GT/s` (PCIe 4.0 x4, its maximum).
- Chipset uplink `00:02.1` -> `05:00.0`: `x4 16.0 GT/s`. Everything behind the chipset shares this.
- Realtek 5GbE `08:00.0`: `x1 8.0 GT/s`. Wi-Fi 7 `09:00.0`: `x1 5.0 GT/s`.

Unusual values and what they suggest.

- **Current width lower than max** (e.g. `x8 ... (max x16 ...)` on a graphics card): the card is in a slot wired for fewer lanes, the slot shares lanes with an M.2 drive or a second slot, or the contacts are dirty or the card is not fully seated.
- **Current speed lower than max:** often harmless. Graphics cards drop to 2.5 GT/s at idle to save power, so check under load. A bridge port also trains down to the slowest partner: on the author's PC port `06:07.0` reads `x1 5.0 GT/s (max x1 8.0 GT/s)` because the Wi-Fi card behind it tops out at 5.0. A device that never reaches its maximum under load has a signal problem: a riser cable, or a slot forced to a lower generation in the BIOS.
- A device with Driver `-` that should have one (network, storage, graphics) is not supported by your kernel.

Double-click a device row's ID in PCI devices or Device IDs to open Device ID Lookup.

Where Linux gets it. `/sys/bus/pci/devices/*` (`vendor`, `device`, `class`, `driver`, `current_link_speed`, `current_link_width`, `max_link_speed`, `max_link_width`) and `pci.ids`.

Background - PCIe link speed and width. A PCIe link is made of **lanes**. Each lane is a pair of wires in each direction. **Width** (x1, x4, x8, x16) is how many lanes. **Speed** is how fast each lane runs, in GT/s (gigatransfers per second). Each PCIe generation doubles the speed:

Speed	Generation	About per lane	x4	x16
2.5 GT/s	PCIe 1.x	250 MB/s	1 GB/s	4 GB/s
5.0 GT/s	PCIe 2.0	500 MB/s	2 GB/s	8 GB/s
8.0 GT/s	PCIe 3.0	985 MB/s	3.9 GB/s	15.8 GB/s
16.0 GT/s	PCIe 4.0	1.97 GB/s	7.9 GB/s	31.5 GB/s
32.0 GT/s	PCIe 5.0	3.94 GB/s	15.8 GB/s	63 GB/s

The link runs at the best speed and width that **both ends** support. Bridges and switches (upstream and downstream ports) fan one link out to several devices, and those devices share the upstream bandwidth.

PCI devices

`pci-dev` - Linux data

In plain words. This is a flat list of every PCI device with its ID and class code, like `lspci -nn`. Add a slot to see one device in full. It is the **PCI Dev** button in the PCI Bus panel.

What you see.

- List form: `0000:03:00.0 1002:7550 [0300] VGA compatible controller: <vendor> <device>`, which is the full address, vendor:device, the 4-digit class code, and the class and names.
- Per-device form `pci-dev:<slot>` (e.g. `pci-dev:0000:03:00.0` or `pci-dev:03:00.0`): the full card described under Chipset MCH, including **PCIe link** and **IOMMU group**.

Double-click a `1002:7550`-style ID to open Device ID Lookup.

Healthy / typical values. On the author's PC: `0000:04:00.0 144D:A80A [0108] Non-Volatile memory controller: Samsung Electronics Co Ltd SSD 980 PRO` and `0000:00:14.3 1022:790E [0601] ISA bridge: ... FCH LPC Bridge`.

Where Linux gets it. `/sys/bus/pci/devices/*` and `pci.ids`.

Background - PCI address and class code. `0000:03:00.0` is *domain*🚌*device.function*. One chip can hold several **functions**. The GPU is `03:00.0` and its HDMI audio is `03:00.1`. The **class code** says what kind of device it is: `0300` display, `0108` NVMe, `0200` Ethernet, `0280` other network (Wi-Fi), `0403` HD audio, `0604` PCI bridge, `0C03` USB controller.

PCI flags

`pci-flag` - Windows-only (Linux equivalent shown)

In plain words. On Windows, SIV shows the HAL's PCI workaround flags. On Linux the matching controls are kernel command-line options, so SRS lists the PCI, PCIe, IOMMU and ASPM options you booted with.

What you see. One line per option starting with `pci`, `pcie`, or containing `iommu` or `aspm`. Or the note "No pci=/pcie_*/iommu options on the kernel command line".

Healthy / typical values. No options at all is the normal default. On the author's PC: `pcie_aspm.policy=performance` (keeps links out of power saving), `pci=pcie_bus_perf` (sets the largest safe packet size) and `iommu=pt` (pass-through: the IOMMU does not remap the host's own devices, which is a little faster).

Unusual values and what they suggest. `pci=noaer` or `pcie_aspm=off` are often added to silence error floods. They hide the symptom and do not fix the link. `pci=nomsi` is a heavy workaround that can reduce performance.

Where Linux gets it. `/proc/cmdline`.

PCI HAL

`pci-hal` - Linux data - Needs root data

In plain words. This shows the address "windows" that the firmware gave each PCI bus: which slices of memory and I/O space each bridge passes on to the devices behind it.

What you see.

- **Memory windows** - lines of the memory map naming `PCI Bus`, `PCI ECAM`, `MMCONFIG` or `pcie`, up to 3 levels deep.
- **I/O windows** - the same for I/O ports, 2 levels deep, including `0cf8-0cff PCI conf1`.
- **PCI buses** - the count and the list of bus numbers.

Healthy / typical values. On the author's PC: root window `80000000-dfffffff` below 4 GB, a big window `10000000000-1fffffffff` above 4 GB (where the GPU's 16 GB BAR lives, `1f800000000-1fc0fffffff` for buses 01-02), the ECAM block `e0000000-efffffff`, and 21 buses.

Unusual values and what they suggest. No window above 4 GB means "Above 4G Decoding" is off, so Resizable BAR cannot work.

Where Linux gets it. `/proc/iomem`, `/proc/ioports` (srs-root cache), `/sys/class/pci_bus`.

PCI ID database

`pci-info` - Linux data

In plain words. These are facts about the file SRS uses to turn PCI ID numbers into names.

What you see.

Field	Meaning
Database	The file path, normally `/usr/share/hwdata/pci.ids`.
SIV equivalent	`pcidevs.txt` (SIV's own file).
Version / Date	When the list was built.
Records	How many vendor, device and subsystem names it holds.
Load time	How long loading took, and how long this lookup took.
Lookup chain	`V:D:S:s:RR -> V:D:S:s -> V:D:RR -> V:D -> V`, the order SRS tries (see Device IDs).
Class names	How many class, subclass and programming-interface names it holds.

Healthy / typical values. On the author's PC: version `2026.10.01`, 42,228 names, 224 class records, loaded in about 10 ms.

Unusual values and what they suggest. A date more than a year old means new hardware will show as "Device XXXX". Update the `hwdata` package. "ids file not found" means `hwdata` is not installed.

Where Linux gets it. `/usr/share/hwdata/pci.ids`.

PCI MMIO

`pci-mmio` - Linux data - Needs root data

In plain words. PCIe devices have up to 4 KB of configuration space each. The CPU reaches it through one big memory window called ECAM (Enhanced Configuration Access Mechanism). The ACPI MCFG table tells the OS where that window is. This item shows it.

What you see.

- **MCFG table** - `present` or `absent`.
- **ECAM base** - its address, PCI segment and bus range (needs the srs-root cache).
- The `PCI ECAM` / `MMCONFIG` line from the memory map.

Healthy / typical values. On the author's PC: `ECAM base 0xE0000000 segment 0 buses 0-255`, and `e0000000-efffffff PCI ECAM 0000 [bus 00-ff]`. That is 256 MB: 1 MB per bus × 256 buses.

Unusual values and what they suggest. If MCFG is absent, Linux falls back to the old port `0xCF8` method. That method only reaches the first 256 bytes, so PCIe extended capabilities (AER, ReBAR, link details) become invisible.

Where Linux gets it. `/sys/firmware/acpi/tables/MCFG`, `/proc/iomem`.

PCMCIA all

`pcm-all` - Not present on most PCs

In plain words. PCMCIA / CardBus were credit-card-sized expansion cards for laptops from the 1990s and 2000s. ExpressCard and USB replaced them.

What it would show. Whether `/sys/class/pcmcia_socket` exists, and how many CardBus/PCMCIA bridges (PCI class 0605/0607) are on the PCI bus.

What "not present" means. On the author's PC: `absent` and `0`. That is expected on any desktop.

PCMCIA bus

`pcm-bus` - Not present on most PCs

In plain words. This is the same check as PCMCIA all: socket class and bridge count.

What you see. `/sys/class/pcmcia_socket absent` and `CardBus/PCMCIA bridges 0 (PCI class 0605/0607)`.

PCMCIA ID database

`pcm-info` - Not present on most PCs

In plain words. SIV has a name list for PCMCIA cards. With no PCMCIA hardware, SRS does not load one.

What you see. `Status: Not present on this system`, `Checked: n/a (no PCMCIA)`.

I/O PM1 data

`pm1-area` - Linux data - Needs root data

In plain words. ACPI defines a few fixed blocks of I/O ports for power management. The **PM1** blocks handle sleep, wake and the power button. **PM_TMR** is a 3.58 MHz timer. **GPE** (General Purpose Events) are wake and event signals. The firmware's FADT (Fixed ACPI Description Table) says where they are. SRS shows the addresses and does not read the registers.

What you see.

- From the FADT (srs-root cache): `PM1a_EVT_BLK`, `PM1b_EVT_BLK`, `PM1a_CNT_BLK`, `PM1b_CNT_BLK`, `PM2_CNT_BLK`, `PM_TMR_BLK`, `GPE0_BLK`, `GPE1_BLK`, each with its port address and length. Unused blocks show `0x0000` in gray.
- The matching I/O port map lines (`ACPI`, `PM1`, `PM_TMR`, `GPE`).
- **PM profile** - the firmware's "preferred power profile": 1 = Desktop, 2 = Mobile, 3 = Workstation, 4 = Enterprise server, 0 = unspecified.

Healthy / typical values. On the author's PC: `PM1a_EVT_BLK 0x0800 length 4`, `PM1a_CNT_BLK 0x0804 length 2`, `PM2_CNT_BLK 0x00B4 length 1`, `PM_TMR_BLK 0x0808 length 4`, `GPE0_BLK 0x0820 length 8`, the "b" blocks unused, and PM profile `1` (Desktop).

Unusual values and what they suggest. A laptop reporting profile 1, or a desktop reporting 2, can change some power defaults. That is a firmware quirk.

Where Linux gets it. `/sys/firmware/acpi/tables/FACP` (root), `/proc/ioports`, `/sys/firmware/acpi/pm_profile`.

PMBus

`pmb-bus` - Not present on most PCs

In plain words. PMBus is an SMBus-based protocol for "smart" power supplies and voltage regulators that report volts, amps and temperature. Servers have it. A few PSUs expose it over USB instead.

What you see. `Status: Not present on this system`, `Checked: /sys/bus/i2c/devices (no PMBus devices)`.

What "not present" means. No kernel PMBus driver is bound. A USB-linked PSU (Corsair, NZXT) shows up under the USB items and sensor tools instead.

PnP manager

`pnp` - Linux data

In plain words. PnP (Plug and Play) devices here are the motherboard's built-in legacy devices that the firmware describes, such as the COM port. This item shows the resources (ports, IRQ, DMA) each one has.

What you see. For each device: `<pnp name> <ID> <description> (ACPI <firmware node>)`, then its resource lines exactly as Linux reports them: `state = active`, `io 0x3f8-0x3ff`, `irq 4`, `dma disabled`.

Healthy / typical values. On the author's PC there is one device, `00:00 PNP0501 16550A-compatible COM Port (ACPI PNP0501:00)`, active at `io 0x3f8-0x3ff`, `irq 4`. Modern Linux binds most other firmware devices through ACPI directly, so the PnP list is short (see PnP manager details).

Where Linux gets it. `/sys/bus/pnp/devices/*/id`, `.../resources`, `.../firmware_node`.

PnP devices

`pnp-dev` - Linux data

In plain words. These are the same PnP devices as a one-line table. It is the **PnP Dev** button in the PCI Bus panel.

What you see.

Column	Meaning
Device	The PnP device name (`00:00`).
ID	The PnP ID (`PNP0501`).
Description	A built-in name, or the vendor from `pnp.ids`.
Driver	The bound driver (`serial`).
Resources	The active resources in one line (lines that say `disabled` are left out).

Healthy / typical values. `00:00 PNP0501 16550A-compatible COM Port serial io 0x3f8-0x3ff, irq 4`.

Where Linux gets it. `/sys/bus/pnp/devices`.

PnP ID database

`pnp-info` - Linux data

In plain words. This item explains the PnP/ACPI ID names and lists every kind of ACPI device ID found on this PC.

What you see.

- **Database** - `/usr/share/hwdata/pnp.ids`. **SIV equivalent** - `pnpdevs.txt`.
- **Records** - how many 3-letter vendor codes the file has.
- **Built-in** - SRS also knows the standard `PNPxxxx`, `ACPIxxxx` and `LNXxxxx` names itself.
- **IDs on this system** - each distinct ACPI ID with its name.

Healthy / typical values. On the author's PC: 2557 vendor codes. Some IDs: `PNP0C09 ACPI Embedded Controller`, `PNP0A08 PCI Express Root Bus`, `MSFT0101 TPM 2.0`, `AMDI0030 AMD GPIO Controller`, `ACPI0007 Processor`, `LNXVIDEO ACPI Video Bus`.

Unusual values and what they suggest. For IDs SRS does not know by heart, the name is a guess from the first three letters of the ID. That guess comes from the old 3-letter EISA vendor list. Newer ACPI IDs use a 4-letter vendor prefix (`AMDI`, `ASMT`, `RTK`), so for those the guessed vendor can be wrong. Read such names as hints only.

Where Linux gets it. `/usr/share/hwdata/pnp.ids`, `/sys/bus/acpi/devices`.

Background. A PnP ID is 3 letters for the vendor plus 4 hex digits (`PNP0501`). ACPI also allows 4 letters plus 4 hex digits (`AMDI0030`). `PNP` and `ACPI` IDs are industry-standard device types.

PnP manager details

`pnp-man` - Linux data

In plain words. This is every device the firmware's ACPI tables describe, with its path in the ACPI "namespace", whether the firmware says it is present, and what Linux attached it to.

What you see.

Column	Meaning
Device	The ACPI device name (`AMDI0010:00`).
ACPI Path	Its namespace path (`\_SB_.I2CA`).
_STA	The firmware's status value. `0x0F` = present, enabled and working. `0x00 absent` = turned off or not fitted. `-` = no status method, so treated as present.
Description	The name from the built-in list or `pnp.ids`.
Bound to	The Linux device it became (e.g. `cpu0`), if any.

Last line: `Total of N ACPI devices, P present (no _STA = present)`.

Healthy / typical values. On the author's PC: 258 ACPI devices, 229 present. The 16 `ACPI0007` processor objects map to `cpu0-cpu15` (in the order C000 -> cpu0, C001 -> cpu8...: the firmware lists the SMT siblings in turn). `AMDI0010:00` (I2C controller A) is `0x0F`. I2C B-D and I3C controllers are `0x00 absent`, because the board does not wire them.

Unusual values and what they suggest. Many `absent` entries are normal. Boards share one firmware across models. A device you need (touchpad I2C, TPM) showing `absent` means it is turned off in the BIOS.

Where Linux gets it. `/sys/bus/acpi/devices/*/path`, `status`, `physical_node`.

COM ports

`ports` - Linux data

In plain words. This lists the serial (COM) ports. It is the same view as Serial COM ports.

What you see / values. See Serial COM ports.

Device properties

`properties` - Linux data

In plain words. These are all the udev properties of every PCI and USB device: IDs, names from the hardware database, paths, driver. It is the **Properties** button in the PCI Bus panel.

What you see. For each PCI device and each USB device (USB interfaces are skipped), the sysfs path in bold, then one line per property:

- Teal lines from udev's own records: `symlink`, `devnode`, `priority` (symlink priority) and `driver`.
- `E:` properties such as `PCI_ID`, `PCI_SUBSYS_ID`, `PCI_CLASS`, `MODALIAS`, `ID_VENDOR_FROM_DATABASE`, `ID_MODEL_FROM_DATABASE`, `ID_PATH`. USB devices add `BUSNUM`, `DEVNUM`, `ID_VENDOR_ID`, `ID_MODEL_ID`, `ID_REVISION`, `ID_SERIAL` and so on.

Values whose key contains TOKEN, KEY, SECRET, PASS, AUTH or COOKIE are shown as `********`. Use `properties:<name>` for any single device, e.g. `properties:nvme0n1` or `properties:0000:03:00.0`. Double-click a `PCI_ID` value (`1022:14D8`) to open Device ID Lookup.

Healthy / typical values. On the author's PC the Root Complex shows `PCI_ID 1022:14D8`, `PCI_SUBSYS_ID 1043:8877`, `ID_MODEL_FROM_DATABASE Raphael/Granite Ridge Root Complex`, `ID_PATH pci-0000:00:00.0`.

Unusual values and what they suggest. Missing `ID_*_FROM_DATABASE` keys mean udev's hardware database (hwdb) does not know the device. Run `systemd-hwdb update` after updating `hwdata`. Note that USB serial numbers (`ID_SERIAL`) appear in full. Remove them before you share a report.

Where Linux gets it. `udevadm info --export-db`.

Device property

`property` - Linux data

In plain words. This shows one udev property across every device that has it. By default it shows `ID_MODEL_FROM_DATABASE` (the database model name).

What you see. `Property <KEY>`, then one line per device: the device path and the value. At the end: `N devices have <KEY>`. Choose another key with `property:<KEY>`, e.g. `property:ID_PATH` or `property:DRIVER`. Device property keys lists all keys.

Healthy / typical values. On the author's PC 117 devices have `ID_MODEL_FROM_DATABASE`, e.g. `.../0000:03:00.0 Navi 48 [Radeon RX 9070/9070 XT/9070 GRE]`. Child entries (sound cards, network interfaces) repeat their parent's name.

Where Linux gets it. `udevadm info --export-db`.

PYXIS 21174

`pyxis` - Not present on most PCs

In plain words. The 21174 "Pyxis" was the core logic chipset of DEC Alpha workstations (about 1997). SIV can read it on Alpha Windows NT. No x86 PC has one.

What you see. `Status: Not present on this system`, `Checked: n/a (DEC Alpha chipset)`.

Smart cards

`scard` - Not present on most PCs

In plain words. These are smart card readers: CCID (Chip Card Interface Device) USB readers for ID cards, bank cards and some security tokens.

What you see. `Status: Not present on this system`, `Checked: pcsc_scan (no CCID reader; pcscd inactive)`.

What "not present" means. No reader is plugged in, and the `pcscd` smart-card service is not running. With a reader attached, start `pcscd` and run `pcsc_scan`. A FIDO/U2F security key is a HID device, not a smart card (see HID bus).

Serial COM ports

`serial` - Linux data

In plain words. This lists the real serial ports (UARTs, Universal Asynchronous Receiver-Transmitters) and every serial-type TTY driver the kernel has.

What you see.

Column	Meaning
Port	The Linux name (`ttyS0` = COM1, `ttyUSB0` = a USB adapter).

Column	Meaning
I/O	The I/O port base address (`0x3F8`).
IRQ	The interrupt line.
UART	The chip type (`16550A`, `16950`...). USB and other adapters show `usb/other`.
Base clock	The UART's input clock. The standard is 1.8432 MHz, which gives 115200 baud at the maximum.
Device	The device it belongs to (the PnP device `00:00:0.0`).

Then `N real ports; M unconfigured 8250 placeholders (nr_uarts) hidden`. Linux makes empty `ttyS1-ttyS31` slots by default, and SRS hides those. Then comes the **TTY drivers** table from `/proc/tty/drivers`: driver name, `/dev` prefix, major number, minor range and type. The note says per-line statistics need root.

Healthy / typical values. On the author's PC: `ttyS0 0x3F8 4 16550A 1.8432 MHz 00:00:0.0`, with 31 placeholders hidden. The drivers include `usbserial /dev/ttyUSB 188`, `usbch343 /dev/ttyCH343USB 170` (a CH343 USB adapter driver), `rfcomm` (Bluetooth serial) and `serial_8250 /dev/ttyS 4 64-95`.

Unusual values and what they suggest. A port with UART `unknown` is not really there. Many boards have a COM header that only works with a bracket cable fitted. An IRQ of 0 means the port is polled, which is slow. A USB serial adapter appears here only while it is plugged in.

Where Linux gets it. `/sys/class/tty/*` (`type`, `port`, `irq`, `uartclk`, `device`), `/proc/tty/drivers`.

SI USB

`si-usb` - Linux data

In plain words. This lists the USB host controllers. Each controller appears as two "root hubs": one for USB 2.0 devices and one for USB 3.x devices.

What you see. Two lines per root hub:

- `usbN <PCI address> <controller name>`. Software-only controllers show the platform device and "virtual host controller".
- `USB <version> root hub, <speed> Mbps, <P> ports, <D> devices attached, driver <driver>`.

Healthy / typical values. On the author's PC (11 root hubs):

- `usb1/usb2` on `73:00.0` ASMedia ASM4242: USB 2.00 at 480 Mbps and USB 3.10 at 20000 Mbps (20 Gbps, the USB 3.2 Gen 2x2 / USB4 ports).
- `usb3-usb6` on the CPU's two USB 3.1 controllers (`75:00.3`, `75:00.4`): 480 and 10000 Mbps.
- `usb7` on the CPU's USB 2.0 controller `76:00.0`.
- `usb8/usb9` on the chipset controller `0b:00.0`: 12 USB 2.0 ports with 13 devices, and 5 USB 3 ports at 20000 Mbps.
- `usb10/usb11` = `vhci_hcd.0`, the USB/IP virtual controller (USB over the network), with 0 devices.

Unusual values and what they suggest. Too many devices on one controller can run out of bandwidth or endpoints. Webcams and audio interfaces are the usual culprits. Move one to a port on a different `usbN`. A controller that is missing completely may be turned off in the BIOS.

Where Linux gets it. `/sys/bus/usb/devices/usb*` (`version`, `speed`, `maxchild`), with the controller found from the sysfs path.

Background - USB speeds and descriptors.

Speed	Name	First in
1.5 Mbps	Low Speed	USB 1.0 (keyboards, mice)
12 Mbps	Full Speed	USB 1.1
480 Mbps	High Speed	USB 2.0
5000 Mbps	SuperSpeed	USB 3.0 / 3.2 Gen 1
10000 Mbps	SuperSpeed+	USB 3.1 / 3.2 Gen 2
20000 Mbps	SuperSpeed+ 20 Gbps	USB 3.2 Gen 2x2

A USB 3 port is really two buses in one connector. That is why each controller has a 2.0 root hub and a 3.x root hub, and why a slow device plugged into a blue port appears on the 2.0 hub. Every USB device describes itself with **descriptors**. The *device descriptor* gives the vendor ID (VID), product ID (PID), release number (bcdDevice) and class. The *configuration descriptor* gives the power draw. Each *interface descriptor* has a class, subclass and protocol, for example 03 = HID, 08 = mass storage, 0E = video. Each *endpoint descriptor* is one data pipe with its type and packet size. Part 2 of this reference covers the USB device lists, which show all of these.

SMBus

`smb` - Linux data

In plain words. SMBus (System Management Bus) and I2C (Inter-Integrated Circuit) are slow two-wire buses for small chips: RAM SPD (Serial Presence Detect) EEPROMs, temperature sensors, RGB controllers, and the graphics card's monitor (DDC) lines. This item lists every bus adapter.

What you see.

- **Controller** - the PCI SMBus controller (class 0C05) with its revision and driver.
- One line per adapter: `i2c-N <adapter name> <parent device>`. SMBus adapters are in bold.
- A note: names come from sysfs only. No bus is opened or scanned.

Healthy / typical values. On the author's PC: controller `0000:00:14.0 FCH SMBus Controller rev 71 driver piix4_smbus`. Adapters `i2c-1`/`i2c-2` "SMBus PIIX4 adapter port 0/2 at 0b00" and `i2c-3` "port 1 at 0b20" (the RAM slots are on port 0). `i2c-0` is the Synopsys DesignWare I2C, `i2c-4/5` are "AMDGPU SMU", and `i2c-6` to `12` are the GPU's display (DM) and DisplayPort AUX buses.

Unusual values and what they suggest. No PIIX4 adapters means `i2c_piix4` is not loaded. Then RAM SPD, temperature and OpenRGB RAM lighting will not work.

Where Linux gets it. `/sys/bus/i2c/devices/i2c-*/name`, `/sys/bus/pci/devices` (class 0C05).

SMBus devices

`smb-bus` - Linux data

In plain words. These are the chips on the SMBus / I2C buses that a kernel driver has claimed. SRS lists only those. It never probes for others.

What you see.

Column	Meaning
Client	`<bus>-<address>` (e.g. `1-0051` = bus 1, address 0x51), or an ACPI name.
Name	The chip type.
Driver	The bound driver, or `(none)`.
Adapter	The bus it is on and that bus's name.

Last line: `Total of N I2C client devices (bus-addr; 0x50-0x57 = SPD5118 hub on DDR5)`.

Healthy / typical values. On the author's PC: `1-0051` and `1-0053`, both `spd5118`, on `i2c-1 SMBus PIIX4 adapter port 0`. These are the SPD hubs of the two DDR5 sticks (their addresses tell you which slots are filled). There are also two Realtek ACPI devices on the DesignWare I2C bus with no driver.

Unusual values and what they suggest. Fewer `spd5118` entries than DIMMs fitted means the `spd5118` module is not loaded, or a stick's SPD hub did not answer. That is worth a reseat if the memory count is also wrong.

Where Linux gets it. `/sys/bus/i2c/devices/<bus>-<addr>` (`name`, `driver`).

SMBus status

`smb-sts` - Linux data - Needs root data

In plain words. SIV reads the SMBus controller's status registers. SRS only shows where they are, because touching them can collide with the kernel driver and with RGB or sensor programs that use the bus at the same time.

What you see. The I/O port ranges owned by `piix4_smbus` / `smbus`, and a note explaining why the registers are not read.

Healthy / typical values. On the author's PC: `0b00-0b08` and `0b20-0b28`, both `piix4_smbus` (the FCH's two SMBus controllers).

Where Linux gets it. `/proc/ioports` (srs-root cache).

SMBus setup

`smbus-setup` - Linux data

In plain words. This shows how SRS handles SMBus: it never probes, and it trusts only what the kernel has found. It also lists every I2C driver and how many chips each one has claimed.

What you see.

- **Probing** - `disabled`, with the reason (CoolerControl, liquidctl, OpenRGB and the kernel own the bus).
- **Data source** - kernel-made clients in `/sys/bus/i2c/devices/<bus>-<addr>`.
- **I2C drivers** - every loaded I2C driver with `N clients`.

Healthy / typical values. On the author's PC almost every driver has 0 clients. These are built into the kernel for other hardware. `spd5118` has 2 clients (the two DDR5 sticks).

Where Linux gets it. `/sys/bus/i2c/drivers/*`.

System SMBus access

`smbus-system` - Linux data

In plain words. This shows whether programs can reach the SMBus / I2C buses from user space, and who is allowed to.

What you see.

- **i2c-dev module** - `loaded (/dev/i2c-* nodes exist)` or `not loaded`.
- One line per `/dev/i2c-N`: `mode` (permissions), `uid`, `gid`. These are checked with stat only, never opened.
- **Device nodes** - the count.
- **piix4_smbus** - whether the AMD/Intel PIIX4 SMBus driver is loaded.

Healthy / typical values. On the author's PC: 13 nodes, all `mode 0660 uid 0 gid 963`. Root and one group (the `i2c` group that OpenRGB-style packages create) can read and write. Everyone else cannot.

Unusual values and what they suggest. `0666` (anyone can write) lets any program send raw commands to RAM or GPU chips. That is a risk, so prefer a group. `i2c-dev not loaded` explains why OpenRGB or `decode-dimms` find nothing. Load it with `modprobe i2c-dev`.

Where Linux gets it. `/sys/class/i2c-dev`, `stat()` of `/dev/i2c-*`, `/sys/module/i2c_dev`, `/sys/module/i2c_piix4`.

Buses / Devices (part 2 of 2) (SRS Section Reference)

Back to the section index - Glossary

Part 1

Where to find these items (SRS 2.0). In the main window they are on the **USB & PCI devices** page (Alt+0). Its tabs are ALL Devices, PCI bus, USB bus, USB tree, HID bus and Thunderbolt; **All N items** lists the rest. You can also type an item's name or id into the search box (Ctrl+F). The rest of this introduction describes the classic view (Menu -> Classic View), with its Menu and panel buttons.

This is the second half (alphabetical) of the Buses / Devices group. It covers the system resource maps (memory, I/O ports, interrupts), bus address translation, Thunderbolt / USB4, every USB view (bus, tree, devices, names, owners, flags, ID database), portable (MTP/PTP) devices, and two families of items for rare hardware: XMC mezzanine cards and the HP zx1 chipset. Open any item from **Menu -> Buses / Devices -> item**. The panel buttons lead to the busiest ones: **USB Bus** opens USB bus, and its sub-buttons are Link Devices, USB Disks, HID Bus and USB Info (the last opens USB ID database); **PCI Bus** has the sub-buttons Properties, PnP Dev, Drives, PCI Dev, UAA Dev and ALL Dev (see part 1). Only System HAL resources and System resources need root data: without it the kernel hides the real addresses, so start the root helper once with **Menu -> Run as root (srs-root)**. In any panel, double-clicking a device ID written as `VVVV:DDDD` (for example `0bda:8153`) opens Device ID Lookup for that ID.

Contents

- System HAL resources `sys-hal`
- System resources `sys-res`
- Thunderbolt `thunder`
- Bus address translation `translation`
- UID database `uid-info`
- USB all `usb-all`
- USB bus `usb-bus`
- USB devices `usb-dev`
- USB flags `usb-flag`
- USB flags detail `usb-flags`
- USB ID database `usb-info`
- USB names `usb-names`
- USB owner `usb-owner`
- USB tree `usb-tree`

- WPD tree (portable devices) `wpd-tree`
- XMC cards `xmc`
- XMC card 0 `xmc-card-0`
- XMC card 1 `xmc-card-1`
- XMC card 2 `xmc-card-2`
- XMC card 3 `xmc-card-3`
- XMC control `xmc-control`
- XMC flash 0 `xmc-flash-0`
- XMC flash 1 `xmc-flash-1`
- XMC flash 2 `xmc-flash-2`
- XMC flash 3 `xmc-flash-3`
- XMC FPGA 0 `xmc-fpga-0`
- XMC FPGA 1 `xmc-fpga-1`
- XMC FPGA 2 `xmc-fpga-2`
- XMC FPGA 3 `xmc-fpga-3`
- XMC names `xmc-names`
- XMC PCIe 0 `xmc-pcie-0`
- XMC PCIe 1 `xmc-pcie-1`
- XMC PCIe 2 `xmc-pcie-2`
- XMC PCIe 3 `xmc-pcie-3`
- XMC setup 0 `xmc-setup-0`
- XMC setup 1 `xmc-setup-1`
- XMC setup 2 `xmc-setup-2`
- XMC setup 3 `xmc-setup-3`
- XMC state 0 `xmc-state-0`
- XMC state 1 `xmc-state-1`
- XMC state 2 `xmc-state-2`
- XMC state 3 `xmc-state-3`
- XMC video 0 `xmc-video-0`
- XMC video 1 `xmc-video-1`
- XMC video 2 `xmc-video-2`
- XMC video 3 `xmc-video-3`
- ZX1 MIO-0 `zx1-mio-0`
- ZX1 MIO-1 `zx1-mio-1`
- ZX1 MIO-2 `zx1-mio-2`
- ZX1 MIO-3 `zx1-mio-3`
- ZX1 MIO-4 `zx1-mio-4`
- ZX1 MIO-5 `zx1-mio-5`
- ZX1 MIO-6 `zx1-mio-6`
- ZX1 MIO-7 `zx1-mio-7`
- ZX1 MIO-8 `zx1-mio-8`
- ZX1 MIO-9 `zx1-mio-9`
- ZX1 MIO-A `zx1-mio-a`
- ZX1 MIO-B `zx1-mio-b`
- ZX1 MIO-C `zx1-mio-c`
- ZX1 MIO-D `zx1-mio-d`
- ZX1 MIO-E `zx1-mio-e`
- ZX1 MIO-F `zx1-mio-f`

- ZX1 summary `zx1-mio-s`

System HAL resources

`sys-hal` - Linux data - Needs root data

In plain words. A short map of the computer's physical address space: which address ranges are RAM, which belong to the firmware, and which are handed to PCI devices. It also shows the top-level map of the old x86 I/O ports. Only the top level is shown, so it fits on one screen.

What you see. Two lists, each line is `start-end owner`:

Part	What it means	Why it matters
Physical memory map (top level)	Every top-level range from the kernel's memory map. Owners include `System RAM`, `Reserved`, `ACPI Tables`, `ACPI Non-volatile Storage`, `PCI Bus 0000:00` (a window of addresses given to PCI devices), `PCI ECAM` (PCI configuration space), `IOAPIC`, `HPET`, `amd_iommu`, `RAM buffer` and ACPI device names like `PNP0C02:04` (motherboard resources).	Shows how much RAM the firmware gave the operating system and where devices live.
I/O port map (top level)	The 64K x86 I/O port space (0000-ffff). Usually almost all of it belongs to `PCI Bus 0000:00`, plus `PCI conf1` at 0cf8-0cff (the old way to reach PCI configuration).	Rarely matters today; legacy devices (serial ports, PS/2 keyboard, SMBus) still use ports.
Footer	"Addresses from the srs-root cache." when root data was used. Without root data the addresses are zeroed by the kernel, so SRS shows only the names, adds "Addresses are zeroed by the kernel for non-root readers." and the hint "Needs root: Menu > Run as root (srs-root)".	Tells you whether the numbers are real.

Healthy / typical values. On the author's PC (32 GB of RAM) the map starts with small `System RAM` and `Reserved` pieces below 1 MB, has `ACPI Tables` and `ACPI Non-volatile Storage` near 0x6D000000, a big `PCI Bus 0000:00` window from 0x80000000 to 0xDFFFFFFF, `PCI ECAM 0000 [bus 00-ff]` at 0xE0000000, and the last `System RAM` range ends at 0x87DEFFFF (just under 34 GiB). That is normal: about 2 GiB below 4 GiB is used by devices, and the firmware moves ("remaps") that RAM above 4 GiB. A large `PCI Bus` window high above (here 0x10000000000-0x1FFFFFFFFF) holds 64-bit device memory such as a graphics card's VRAM window.

Unusual values and what they suggest.

- Total `System RAM` far below the installed amount: memory remapping off in the BIOS, a stick not detected, or a 32-bit limitation.
- All addresses shown as names only: no root data yet (start srs-root).

- No high (above 4 GiB) `PCI Bus` window: "Above 4G decoding" is off in the BIOS, so Resizable BAR cannot work.

Where Linux gets it. `/proc/iomem` and `/proc/ioports`, depth 0 (top level only). Real addresses need root, so SRS reads the copy saved by srs-root.

Background. CPUs talk to RAM and to devices through the same address space. Device registers mapped into it are called MMIO (memory-mapped I/O). The firmware (UEFI/BIOS) builds this map at boot and reports it to Linux (the "e820" map). "HAL" is the Windows Hardware Abstraction Layer, the name SIV uses for this page.

System resources

`sys-res` - Linux data - Needs root data

In plain words. The full list of hardware resources in use: interrupts, legacy DMA channels, I/O ports and the memory map, one level deeper than System HAL resources.

What you see. Four parts:

Part	What it means	Why it matters
Interrupts (IRQ, total count, controller / devices)	One line per interrupt: the IRQ number (or a short name), the total count summed over all CPUs since boot, the interrupt controller and trigger type, and the device(s) using it.	Shows which devices are busy and whether an interrupt is shared or stuck.
DMA channels	The old ISA DMA channels in use.	Almost always just `4: cascade` today.
I/O ports	The whole I/O port tree, nested by indentation (for example `PNP0C09:00` with `EC data` under it).	Shows legacy devices: timer, PS/2 keyboard, RTC, serial port, ACPI PM blocks, SMBus (`piix4_smbus`).
Memory map	The memory map with the top level plus one level of children (for example `Kernel code` inside `System RAM`, or the sub-windows of each PCI bridge).	Shows where each PCI bridge's address window sits.

Controller names in the interrupt list: `IR-IO-APIC` (I/O APIC, with interrupt remapping by the IOMMU), `IR-PCI-MSI-<address>` and `IR-PCI-MSIX-<address>` (message-signalled interrupts from that PCI device). The text after it, like `2-edge` or `9-fasteoi`, is the hardware line number and trigger mode. Named rows (shown grey) are per-CPU events: `NMI` (non-maskable interrupts), `LOC` (local timer), `RES` (rescheduling), `CAL` (function calls between CPUs), `TLB` (TLB shootdowns), `TRM` (thermal events), `MCE` (machine check exceptions), `MCP` (machine check polls) and so on.

Healthy / typical values. On the author's PC each NVMe drive has 17 MSI-X vectors (`nvme0q0` to `nvme0q16`), one per CPU queue; the busiest single source is the chipset USB controller `xhci_hcd` on 0000:0b:00.0 with about 10 million interrupts after 21 hours. `MCE 0` and `TRM 0` are what you want. `DMA channels` shows only `4: cascade`.

Unusual values and what they suggest.

- `MCE` above 0: the CPU reported a hardware error (memory, cache or bus). Check the kernel log and memory stability.
- `TRM` above 0: the CPU hit a thermal limit.
- One device with a huge count while idle (millions per minute): an "interrupt storm", often a misbehaving USB or GPIO device.
- A device still on `IR-IO-APIC` instead of MSI: older or legacy-mode hardware, and the line may be shared.
- An NVMe drive with only one or two vectors: the driver fell back to fewer queues (rare; can lower performance).

Where Linux gets it. `/proc/interrupts`, `/proc/dma`, `/proc/ioports` (all levels) and `/proc/iomem` (two levels). Addresses need root and come from the srs-root cache.

Background. An interrupt is a signal a device sends to make the CPU stop and handle it. Old PCs had 16 shared lines (IRQs). Modern PCIe devices send MSI / MSI-X "messages" instead, so each device (or each queue) gets its own vector and nothing is shared.

Thunderbolt

`thunder` - Not present on most PCs

In plain words. Shows whether the PC has a Thunderbolt / USB4 host and lists the related PCI devices.

What you see.

- `/sys/bus/thunderbolt` - `present` or `absent (thunderbolt module not loaded)`. Present means the kernel's Thunderbolt/USB4 driver is running.
- `USB4 host` lines - every PCI device whose class is "USB4 host interface" (class code 0C0340), with its PCI address and name.
- `Thunderbolt` lines - other PCI devices with "Thunderbolt" in their name (for example Intel Thunderbolt controllers that are not class 0C0340).
- `0-0`, `1-0`, ... lines - the routers the kernel's thunderbolt driver found (`/sys/bus/thunderbolt/devices`): vendor, name, `USB4` or `Thunderbolt <generation>`, and the NVM firmware version.
- `domainN` lines - each Thunderbolt / USB4 domain and its security level (`none`, `user`, `secure`, `dponly`, `usbonly`).

- If nothing is found: `USB4 controller none enumerated on the PCI bus`, followed by a short note on likely reasons (USB4 disabled in the firmware or not powered up on this boot).

Healthy / typical values. On the author's ASUS X870 board: `/sys/bus/thunderbolt present`, a `USB4 host` at 0000:74:00.0 (ASMedia ASM4242 USB 4 / Thunderbolt 3 Host Router), router `0-0 ASMedia Technology Inc. ASM4242 (USB4, NVM 200111.250708)` and `domain0 security level user`.

Unusual values and what they suggest.

- Board has USB4 ports but nothing is listed: USB4 disabled in the BIOS, or the controller did not come up (a cold power-off sometimes helps).
- Switch ports listed but nothing behind them: normal when no Thunderbolt/USB4 PCIe device (dock, eGPU, NVMe enclosure) is plugged in.

Where Linux gets it. `/sys/bus/thunderbolt` and the PCI device list in `/sys/bus/pci/devices` (class 0C0340, vendor 1B21).

Background. USB4 (and Thunderbolt 3/4, which it is based on) can tunnel PCI Express (PCIe) over the cable. That is why a USB4 chip shows up as a PCIe switch: devices you plug in appear behind its downstream ports as if they were in a slot. Each PCI device is named by its address `domain:bus:device.function`, for example 0000:74:00.0. A PCIe link has a speed (generation: 2.5, 5, 8, 16, 32 GT/s for Gen 1-5) and a width (number of lanes: x1, x4, x16); the tunnel is limited by both the cable and the chip's own link.

Bus address translation

`translation` - Linux data

In plain words. For each PCI device memory or I/O window (a BAR), shows the address the device sees on the bus and the address the CPU uses for it.

What you see.

Column	What it means	Why it matters
Location	PCI address `bus:device.function` (domain 0000 left out).	Which device.
BAR	`BAR 0` to `BAR 5`, or `ROM` (the option ROM). Only BARs that are in use are listed.	A device can have several windows: registers, VRAM, doorbells.
Bus Address	Where the window starts as seen from the PCI bus.	

Column	What it means	Why it matters
CPU Address	Where the CPU finds the same window in its physical address space.	
Space	`Memory` (MMIO) or `I/O` (legacy port space).	

A note at the bottom explains that on x86-64 PCI bus addresses map 1:1 to CPU addresses, so both columns are always the same here; the IOMMU (input-output memory management unit) only translates DMA traffic from devices to RAM.

Healthy / typical values. On the author's PC the graphics card 03:00.0 has `BAR 0` at 0x1F800000000 and `BAR 2` at 0x1FC00000000 (both far above 4 GiB), `BAR 4` as `I/O` at 0xF000, `BAR 5` (registers) at 0xDD300000 and a `ROM`. The 16 GiB gap between BAR 0 and BAR 2 matches the card's 16 GB of VRAM, which suggests Resizable BAR is active. NVMe drives and USB controllers each have one memory BAR below 4 GiB.

Unusual values and what they suggest.

- Graphics card BAR 0 only 256 MB below 4 GiB: Resizable BAR (or "Above 4G decoding") is off.
- A device you expect is missing: it has no assigned resources (BIOS ran out of address space, or the device is disabled).

Where Linux gets it. `/sys/bus/pci/devices/*/resource` (start, end and flags of each BAR).

Background. A BAR (Base Address Register) is how a PCI device asks for a piece of the address space. The firmware or kernel assigns each BAR a start address at boot. On some non-x86 systems the host bridge adds an offset, so bus and CPU addresses differ; that is what Windows "HalTranslateBusAddress" reports.

UID database

`uid-info` - Linux data

In plain words. Information about the USB ID name database SRS uses to turn numbers like `0bda:8153` into names. It shows the same page as USB ID database.

What you see. Same fields as USB ID database: Database, SIV equivalent, Version, Date, Records, Load time, Lookup chain, Class names.

Healthy / typical values. On the author's PC: `/usr/share/hwdata/usb.ids`, version 2026.06.26, 23,964 names.

Where Linux gets it. `/usr/share/hwdata/usb.ids` (the `hwdata` package).

USB all

`usb-all` - Linux data

In plain words. Every USB device, root hub and hub with all its details: IDs, strings, speed, power, interfaces, drivers and endpoints. This is the long "everything" view.

What you see. One block per device. The first line is the kernel name (`usb1`, `1-2.3.4`) and a description. Then:

Field	What it means	Why it matters
Bus / Device	Bus number, device number and port path, e.g. `Bus 001 Device 019 port path 2.3.4`. Root hubs show `port path 0 (root hub)`.	Device numbers change each time you replug; the port path tells you the physical route.
IDs	`VID_xxxx PID_xxxx REV_xxxx`: vendor ID, product ID and the device's release number (bcdDevice).	The IDs identify the chip; drivers match on them.
Database	Vendor and product names from usb.ids, or `(vendor not in usb.ids)`.	Helps when the device's own strings are generic ("Generic", "USB2.0 Hub").
Manufacturer / Product / Serial	Strings the device reports about itself (only shown if present).	Cheap devices often report fake or empty strings.
USB version	The USB version the device claims (bcdUSB) and the speed it actually connected at, e.g. `3.20 10Gbps SuperSpeed+`.	The speed is what matters for performance.
Device class	Class code and name; `00 (Defined at Interface level)` means each interface says its own class.	
Max power	Current the device asked for (bMaxPower).	USB 2 ports supply up to 500 mA, USB 3 up to 900 mA.
Hub ports	Number of downstream ports (hubs only).	
Controller	PCI address and name of the host controller, or the virtual host for USB/IP.	Tells you which chip (CPU, chipset, add-on) the port belongs to.
Removable	`removable`, `fixed` (built in, like an internal LED controller) or `unknown`, from the firmware's port description.	
Quirks	Kernel workarounds active for this device, in hex (shown in orange, only when not 0). See USB flags detail.	

Field	What it means	Why it matters
Interface	One line per interface: name (`1-2:1.0` = configuration 1, interface 0), class/subclass/protocol in hex and their most specific name, number of endpoints, bound driver (or `(none)`) and the interface's own string.	Each function of a device (keyboard, mouse, media keys) is a separate interface.
Endpoint lines	`Endpoint Address 0x81 (in) Interrupt Maximum Packet Size 0x0004 Interval 256ms`: address and direction, transfer type (Control, Interrupt, Bulk, Isoc), largest packet in bytes (hex) and polling interval.	Bulk packet size 0x0400 (1024) means SuperSpeed; 0x0200 (512) High Speed; 0x0040 (64) Full Speed.

A footer line gives totals (USB entities, root hubs, external hubs, devices), the time taken and the usb.ids file, date and version.

Healthy / typical values. On the author's PC, `2-1` is a JMicron JMS583 USB-to-NVMe bridge (`152d:0583`), `USB version 3.20 10Gbps SuperSpeed+`, `Max power 896mA`, interface `08/06/62 USB Attached SCSI 4 endpoints driver uas`. The Razer mouse `5-1` runs at `12Mbps Full Speed` with interface `03/01/02 Mouse` (driver usbhid), which is normal for mice.

Unusual values and what they suggest.

- A USB 3 device (USB version 3.x) at `480Mbps High Speed`: it is on a USB 2 port, a USB 2 cable or hub, or the USB 3 contacts are dirty.
- A storage device with `08/06/50 Bulk-Only` instead of `08/06/62 USB Attached SCSI` (UAS): slower, older protocol, or the kernel disabled UAS with a quirk.
- An interface with `driver (none)`: no Linux driver claimed it. Normal for vendor-specific parts (RGB, IR blasters) that a user program talks to; a problem if it is your keyboard or network adapter.
- Garbage product strings (odd characters): a cheap device with broken string descriptors; harmless.

Where Linux gets it. `/sys/bus/usb/devices/*` (idVendor, idProduct, bcdDevice, speed, version, bMaxPower, strings, quirks, removable, interfaces and their `ep_*` endpoint folders) plus usb.ids for names. Similar to `lsusb -v`.

Background. USB descriptors are small data tables every device sends when plugged in. The device descriptor holds the IDs and class; each interface descriptor describes one function; each endpoint is a one-way data pipe. USB speeds: Low Speed 1.5 Mbps, Full Speed 12 Mbps (USB 1.1), High Speed 480 Mbps (USB 2.0), SuperSpeed 5 Gbps (USB 3.2 Gen 1), SuperSpeed+ 10 Gbps (Gen 2) and 20 Gbps (Gen 2x2).

USB bus

`usb-bus` - Linux data

In plain words. The main USB panel (the **USB Bus** button): one line per root hub, hub and device, indented to show what plugs into what.

What you see.

Column	What it means
Connection	`Hub N` for a root hub (N = bus number), or `Port x.y.z` indented by depth for anything behind it.
Vendor and Device Description	Manufacturer and product strings, or names from usb.ids. Root hubs show the host controller's PCI name.
Version	The USB version the device claims (bcdUSB).
mA	Requested current in milliamps.
Device Class	`V2 Root Hub` / `V3 Root Hub` for root hubs, otherwise the device class (or the first interface's class when the device defines it per interface), e.g. `Hub`, `Mass Storage`, `Human Interface Device`.
Device Information	Connected speed, port count for hubs, and the driver(s) bound to its interfaces.

The footer is the same totals line as USB all (entities, root hubs, external hubs, devices, usb.ids version), followed by the motherboard name and BIOS version.

Healthy / typical values. On the author's PC: 40 USB entities, 11 root hubs, 11 external hubs and 18 devices. Every USB controller makes two buses: a USB 2 one and a USB 3 one (Hub 1 = `V2 Root Hub 480Mbps`, Hub 2 = `V3 Root Hub 20Gbps SuperSpeed+ 20Gbps` on the ASMedia ASM4242). Hubs 10 and 11 are `USB/IP Virtual Host Controller` (vhci_hcd), a software controller for USB over the network. A USB 3 hub appears twice, once on each bus (VIA `USB2.0 Hub` on Hub 1 and `USB3.0 Hub` on Hub 2).

Unusual values and what they suggest.

- A fast device under a `V2 Root Hub` only: it is running at USB 2 speed (see USB all).
- mA total on one bus-powered hub above 500: devices may disconnect; use a powered hub.
- Very deep chains (hub in hub in hub): each extra hub adds latency and can cause dropouts.

Where Linux gets it. `/sys/bus/usb/devices` and usb.ids. Similar to `lsusb -t`.

Background. A root hub is the virtual hub built into each USB host controller (xHCI, eXtensible Host Controller Interface, is the modern controller type). External hubs fan one port out to several. The port path 2.3.4 means "root port 2, then port 3 of that hub, then port 4 of the next hub".

USB devices

`usb-dev` - Linux data

In plain words. A compact one-line-per-device list, like `lsusb`.

What you see. Kernel name (`usb1`, `1-2.3`), `Bus NNN Device NNN`, `ID vvvv:pppp` and the description. A note at the end says you can open one device on its own: `usb-dev:<name>`, for example `usb-dev:1-1` or `usb-dev:usb3`, which shows the full detail block from USB all for just that device. Double-click an `ID vvvv:pppp` value to open Device ID Lookup for it.

Healthy / typical values. On the author's PC: `8-8.4.4 Bus 008 Device 013 ID 046d:c52f Logitech USB Receiver`, `5-2 Bus 005 Device 003 ID 1038:1612 SteelSeries SteelSeries Apex 7`. Root hubs always have `ID 1d6b:0002` (USB 2) or `1d6b:0003` (USB 3), the Linux Foundation's IDs.

Unusual values and what they suggest. A device missing from the list was not enumerated: bad cable, not enough power, or the port is disabled. Asking for a name that does not exist prints "No such USB device".

Where Linux gets it. `/sys/bus/usb/devices` and usb.ids.

USB flags

`usb-flag` - Linux data

In plain words. SIV's Windows "UsbFlags" page (per-device USB settings in the registry). On Linux SRS shows the equivalent: the USB core driver's settings and every per-device quirk the kernel applied. The page is identical to USB flags detail; read that section for the fields.

Where Linux gets it. `/sys/module/usbcore/parameters/*` and `/sys/bus/usb/devices/*/quirks`.

USB flags detail

`usb-flags` - Linux data

In plain words. The settings of the Linux USB core and the list of devices that need special kernel workarounds ("quirks").

What you see. First the usbcore parameters:

Field	What it means	Typical
quirks	Extra quirks added by the user on the kernel command line (`usbcore.quirks=VID:PID:flags`). `(empty)` = none.	`(empty)`
autosuspend	Seconds of idle before a USB device is put to sleep. `-1` = autosuspend off.	`2` (kernel default)
authorized_default	Whether new devices are allowed automatically: `1` or `-1` all, `0` none, `2` internal only.	`1`
old_scheme_first / use_both_schemes	How the kernel first talks to a new device (two enumeration methods).	`N` / `Y`
initial_descriptor_timeout	Milliseconds to wait for a new device's first descriptor.	`5000`
usbfs_memory_mb	Memory limit for programs that talk to USB devices directly (usbfs), in MB.	`16` (kernel default)
nousb	`Y` would disable USB completely.	`N`

Then one line per device with active quirks: kernel name, `quirks 0x...` in hex and the `VID:PID`. If there are none: "No per-device USB quirks active" (with the Windows registry path for comparison). You can double-click the VID:PID to open Device ID Lookup.

Healthy / typical values. On the author's PC `autosuspend -1` (turned off by the distribution's tuning) and `usbfs_memory_mb 256` (raised, common for cameras and capture devices). Four devices have quirks: the VIA hub `2109:2817` and the Realtek network adapter `0BDA:8153` have `0x400`, and two Terminus hubs `1A40:0101` have `0x4000`. These come from the kernel's built-in quirk table, not from the user.

Unusual values and what they suggest.

- Common quirk bits: `0x2` reset-resume (needs reset after sleep), `0x40` delay-init, `0x400` no LPM (Link Power Management off, used for devices that drop out in power-saving), `0x4000` hub slow reset. A device that randomly disconnects may need a quirk added in `quirks`.
- `nousb Y`: USB disabled on the kernel command line.
- `authorized_default 0`: new devices are blocked until authorised (security setups, USBGuard).

Where Linux gets it. `/sys/module/usbcore/parameters/*` and `/sys/bus/usb/devices/*/quirks`. The quirk bits are defined in the kernel header `linux/usb/quirks.h`.

USB ID database

`usb-info` - Linux data

In plain words. Facts about the file that turns USB vendor and product numbers into names (the **USB Info** button).

What you see.

Field	What it means
Database	The path of the file SRS loaded.
SIV equivalent	SIV's own file for the same job (`usbdevs.txt`).
Version / Date	Version and date written in the file's header.
Records	How many names it holds (vendors, devices, subsystems).
Load time	Time to load the file, and time for this lookup.
Lookup chain	Order SRS tries when naming a device: `V:D -> V` = vendor plus device first, then vendor only.
Class names	Number of class / subclass / protocol names (used for columns like "Device Class").

If the file cannot be found SRS prints "ids file not found" and names are left blank.

Healthy / typical values. On the author's PC: `/usr/share/hwdata/usb.ids`, `Version 2026.06.26`, `23,964 names`, `Load time 4.291 ms`, `Class names 141`.

Unusual values and what they suggest. A date more than a year old means new devices show `(vendor not in usb.ids)` or no product name; update the `hwdata` package.

Where Linux gets it. `/usr/share/hwdata/usb.ids` (community list at linux-usb.org).

Background. Every USB vendor buys a 16-bit vendor ID (VID) from the USB Implementers Forum and numbers its products with a 16-bit product ID (PID). The ids file is a volunteer-maintained list of these numbers and names.

USB names

`usb-names` - Linux data

In plain words. Puts the name a device reports about itself next to the name from the ID database, so you can see which is more useful.

What you see.

Column	What it means
Device	Kernel name.
VID:PID	Vendor and product ID (double-click to open Device ID Lookup).
Device strings	`Manufacturer / Product` as reported by the device.
usb.ids name	Vendor and product name from the database, or `-` if the vendor is unknown.

Healthy / typical values. On the author's PC the card reader `05E3:0749` calls itself `Generic / USB3.0 Card Reader`, while the database says `Genesys Logic, Inc. SD Card Reader and Writer`; the Bluetooth adapter `13D3:3617` says `Realtek / Bluetooth Radio` while the database names the module maker `IMC Networks`. Both are normal: the chip maker and the board maker are often different companies.

Unusual values and what they suggest.

- A strange vendor in the database column (for example a famous brand for a no-name device): the maker used someone else's VID, which is common with cheap devices.
- `-` in the last column: vendor not in usb.ids (new, unregistered or fake ID).
- Unreadable device strings: broken string descriptors.

Where Linux gets it. `/sys/bus/usb/devices/*/manufacturer`, `product` and usb.ids.

USB owner

`usb-owner` - Linux data

In plain words. Which driver owns each USB interface, and which kernel module that driver lives in.

What you see.

Column	What it means
Interface	Interface name, e.g. `5-1:1.0`.
Class	Interface class name (Hub, Human Interface Device, Mass Storage, Printer, Video, Wireless, Vendor Specific Class...).
Driver	Bound driver, or `(none)`.
Module	Kernel module that provides the driver, or `built-in` if compiled into the kernel; blank when no driver is bound.

Healthy / typical values. On the author's PC hubs use `hub` from `usbcore`, keyboards and mice `usbhid`, the NVMe enclosure `uas`, the webcam `uvcvideo`, Bluetooth `btusb`, the network adapter `r8152`. Some

interfaces show `usbfs` (module usbcore): a user program opened the device directly, for example the printer's IPP-over-USB interfaces or a keyboard's lighting interface.

Unusual values and what they suggest. `(none)` on an interface that should work (a network adapter, a sound card): the driver module is missing or blacklisted. `usb-storage` instead of `uas` for a fast SSD enclosure: UAS was refused or disabled by a quirk.

Where Linux gets it. `/sys/bus/usb/devices/*:*/driver` and `driver/module` symlinks.

Background. One USB device can have several interfaces, each with its own driver. A gaming mouse, for example, may have a mouse interface and two keyboard interfaces for its macro buttons.

USB tree

`usb-tree` - Linux data

In plain words. The USB layout as a text tree, in the same style as `lsusb -t`.

What you see. For each bus: `/: Bus NNN.Port 1: Dev 1, Class=root_hub, <driver>, <speed>M [<controller>]`. Under it, each device as `|__ Port x.y: Dev N, VVVV:PPPP <description>, <speed>M`, and under each device one line per interface: `If 1.0, Class=..., Driver=...` (`[none]` when no driver). Speeds are in Mbit/s: 1.5, 12, 480, 5000, 10000, 20000. You can double-click a VVVV:PPPP ID to open Device ID Lookup.

Healthy / typical values. On the author's PC `Bus 002 ... 20000M [0000:73:00.0]` is the ASM4242's USB 3 bus, with the JMS583 NVMe bridge at `10000M` and a VIA USB 3 hub at `5000M`; `Bus 010` and `Bus 011` are the USB/IP virtual controller `[vhci_hcd.0]`.

Unusual values and what they suggest. A device shown at a lower speed than its parent hub and than it supports: cable or port problem. A device whose parent is the wrong hub: it is plugged into a different port than you thought.

Where Linux gets it. `/sys/bus/usb/devices`.

WPD tree (portable devices)

`wpd-tree` - Linux data

In plain words. Lists phones, cameras and media players that connect as "portable devices" (MTP or PTP) instead of as a USB drive. WPD (Windows Portable Devices) is the Windows name for this.

What you see. One line per device: kernel name, description, and in brackets how it was detected: `[PTP/MTP still image interface]` (an interface of class 06, Still Image, or an interface named MTP) or `[MTP (udev ID_MTP_DEVICE)]` (udev's device database marks it as MTP). If none: "No MTP/PTP portable devices attached (checked USB interface class 06 and udev ID_MTP_DEVICE)."

Healthy / typical values. The author's PC had no phone or camera attached, so the "No MTP/PTP portable devices attached" line is shown. An Android phone set to "File transfer" would appear here.

Unusual values and what they suggest. A phone plugged in but not listed: it is in "charging only" mode, or the phone is locked (many phones hide MTP until unlocked).

Where Linux gets it. USB interface class 06 and the interface string in `/sys/bus/usb/devices`, plus the udev property `ID_MTP_DEVICE`.

Background. MTP (Media Transfer Protocol) and PTP (Picture Transfer Protocol) let the device keep control of its own storage and hand over files one at a time, instead of exposing a raw disk. On Linux, file managers reach them through gvfs or kio (KDE).

XMC cards

`xmc` - Not present on most PCs

In plain words. The summary page for XMC cards. XMC (Switched Mezzanine Card, standard ANSI/VITA 42) is a small plug-in module used in industrial, military and telecom computers. It sits on a carrier board (VME, VPX, CompactPCI or a PCIe adapter) and talks to it over PCI Express. Many XMC cards carry an FPGA (field-programmable gate array, a chip whose logic is loaded from a file), often from Xilinx (now AMD), plus flash memory and video or signal inputs. SRS lists this as "no Xilinx XMC card".

What you see. On a PC without such a card: `Status Not present on this system` and `Checked n/a (no Xilinx XMC card)`. The other XMC items below are the per-card pages SIV offers for up to four cards (0 to 3) and show the same two lines.

What "not present" means. Nothing is wrong. Desktop and gaming PCs never have XMC cards. On a system with one, the card shows up first as an ordinary PCI device in the PCI views (see part 1), where its link speed and width can be checked.

XMC card 0

`xmc-card-0` - Not present on most PCs

In plain words. The page for XMC card number 0: the identity of the card in that position (which card and model it is).

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC card 1

`xmc-card-1` - Not present on most PCs

In plain words. The page for XMC card number 1: the identity of the card in that position (which card and model it is).

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC card 2

`xmc-card-2` - Not present on most PCs

In plain words. The page for XMC card number 2: the identity of the card in that position (which card and model it is).

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC card 3

`xmc-card-3` - Not present on most PCs

In plain words. The page for XMC card number 3: the identity of the card in that position (which card and model it is).

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC control

`xmc-control` - Not present on most PCs

In plain words. The control page for XMC cards: the shared control settings of the card driver.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC flash 0

`xmc-flash-0` - Not present on most PCs

In plain words. The flash memory page for XMC card 0. The flash chip on such a card stores the FPGA's configuration file and the card's firmware.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC flash 1

`xmc-flash-1` - Not present on most PCs

In plain words. The flash memory page for XMC card 1. The flash chip on such a card stores the FPGA's configuration file and the card's firmware.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC flash 2

`xmc-flash-2` - Not present on most PCs

In plain words. The flash memory page for XMC card 2. The flash chip on such a card stores the FPGA's configuration file and the card's firmware.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC flash 3

`xmc-flash-3` - Not present on most PCs

In plain words. The flash memory page for XMC card 3. The flash chip on such a card stores the FPGA's configuration file and the card's firmware.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC FPGA 0

`xmc-fpga-0` - Not present on most PCs

In plain words. The FPGA page for XMC card 0: the programmable chip at the heart of the card and the design loaded into it.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC FPGA 1

`xmc-fpga-1` - Not present on most PCs

In plain words. The FPGA page for XMC card 1: the programmable chip at the heart of the card and the design loaded into it.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC FPGA 2

`xmc-fpga-2` - Not present on most PCs

In plain words. The FPGA page for XMC card 2: the programmable chip at the heart of the card and the design loaded into it.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC FPGA 3

`xmc-fpga-3` - Not present on most PCs

In plain words. The FPGA page for XMC card 3: the programmable chip at the heart of the card and the design loaded into it.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC names

`xmc-names` - Not present on most PCs

In plain words. The list of names (models) of the XMC cards found.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC PCIe 0

`xmc-pcie-0` - Not present on most PCs

In plain words. The PCI Express page for XMC card 0: how the card is connected to the carrier (link speed in GT/s and width in lanes, x1 to x8). A card running below its rated link loses bandwidth.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC PCIe 1

`xmc-pcie-1` - Not present on most PCs

In plain words. The PCI Express page for XMC card 1: how the card is connected to the carrier (link speed in GT/s and width in lanes, x1 to x8). A card running below its rated link loses bandwidth.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC PCIe 2

`xmc-pcie-2` - Not present on most PCs

In plain words. The PCI Express page for XMC card 2: how the card is connected to the carrier (link speed in GT/s and width in lanes, x1 to x8). A card running below its rated link loses bandwidth.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC PCIe 3

`xmc-pcie-3` - Not present on most PCs

In plain words. The PCI Express page for XMC card 3: how the card is connected to the carrier (link speed in GT/s and width in lanes, x1 to x8). A card running below its rated link loses bandwidth.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC setup 0

`xmc-setup-0` - Not present on most PCs

In plain words. The setup (configuration) page for XMC card 0.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC setup 1

`xmc-setup-1` - Not present on most PCs

In plain words. The setup (configuration) page for XMC card 1.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC setup 2

`xmc-setup-2` - Not present on most PCs

In plain words. The setup (configuration) page for XMC card 2.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC setup 3

`xmc-setup-3` - Not present on most PCs

In plain words. The setup (configuration) page for XMC card 3.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC state 0

`xmc-state-0` - Not present on most PCs

In plain words. The state page for XMC card 0: its current running status.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC state 1

`xmc-state-1` - Not present on most PCs

In plain words. The state page for XMC card 1: its current running status.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC state 2

`xmc-state-2` - Not present on most PCs

In plain words. The state page for XMC card 2: its current running status.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC state 3

`xmc-state-3` - Not present on most PCs

In plain words. The state page for XMC card 3: its current running status.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC video 0

`xmc-video-0` - Not present on most PCs

In plain words. The video page for XMC card 0, for cards that capture or output video.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC video 1

`xmc-video-1` - Not present on most PCs

In plain words. The video page for XMC card 1, for cards that capture or output video.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC video 2

`xmc-video-2` - Not present on most PCs

In plain words. The video page for XMC card 2, for cards that capture or output video.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

XMC video 3

`xmc-video-3` - Not present on most PCs

In plain words. The video page for XMC card 3, for cards that capture or output video.

What you see. `Status Not present on this system` and `Checked n/a (no XMC card)`. See XMC cards for what XMC is.

ZX1 MIO-0

`zx1-mio-0` - Not present on most PCs

In plain words. The first of sixteen register pages for the HP zx1 chipset. The zx1 was HP's chipset for Intel Itanium 2 servers and workstations (for example the HP rx2600 server and zx6000 workstation) in the early 2000s. Its MIO (Memory and I/O controller) is the main chip that links the processors to memory and to the I/O adapters that drive the PCI and AGP buses. SIV can show its registers in pages 0 to F, plus a summary.

What you see. On any ordinary PC: `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`.

What "not present" means. Nothing is wrong. The zx1 only exists in HP Itanium (IA-64) machines. A modern x86-64 PC (Intel or AMD) can never have one, so all ZX1 items always say "not present" there.

ZX1 MIO-1

`zx1-mio-1` - Not present on most PCs

In plain words. Register page 1 of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-2

`zx1-mio-2` - Not present on most PCs

In plain words. Register page 2 of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-3

`zx1-mio-3` - Not present on most PCs

In plain words. Register page 3 of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-4

`zx1-mio-4` - Not present on most PCs

In plain words. Register page 4 of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-5

`zx1-mio-5` - Not present on most PCs

In plain words. Register page 5 of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-6

`zx1-mio-6` - Not present on most PCs

In plain words. Register page 6 of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-7

`zx1-mio-7` - Not present on most PCs

In plain words. Register page 7 of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-8

`zx1-mio-8` - Not present on most PCs

In plain words. Register page 8 of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-9

`zx1-mio-9` - Not present on most PCs

In plain words. Register page 9 of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-A

`zx1-mio-a` - Not present on most PCs

In plain words. Register page A of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-B

`zx1-mio-b` - Not present on most PCs

In plain words. Register page B of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-C

`zx1-mio-c` - Not present on most PCs

In plain words. Register page C of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-D

`zx1-mio-d` - Not present on most PCs

In plain words. Register page D of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-E

`zx1-mio-e` - Not present on most PCs

In plain words. Register page E of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 MIO-F

`zx1-mio-f` - Not present on most PCs

In plain words. Register page F of the HP zx1 Memory and I/O controller (MIO), found only in HP Itanium 2 systems.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for background.

ZX1 summary

`zx1-mio-s` - Not present on most PCs

In plain words. The overview page for the HP zx1 chipset: a summary of the MIO controller pages 0 to F in one view.

What you see. `Status Not present on this system` and `Checked n/a (HP zx1 Itanium chipset)`. See ZX1 MIO-0 for what the zx1 is.

What "not present" means. Expected on every x86-64 PC.

Tools (SRS Section Reference)

Back to the section index - Glossary

Where to find these items (SRS 2.2). In the main window they are on two pages. **Benchmarks & Stress** (Alt+=) has (2.3.1) one tab per tool - Stress & Burnin (CPU and GPU), CPU Benchmarks, Cache & Memory, Storage, OS latency, CPU compute, Everyday work, Network, GPU Benchmark, GPU Compare, SRS Pi, Crypto, Prime Sieve, IPI delay and Memory latency - with its controls and live results, compact sections at the top that jump to them, and the benchmark history tab. **Tools** (Alt+Backspace) has buttons for Device ID Lookup, Find in a system report, Save Report, Run as root, the Classic view and (2.4) Update hardware lists and ACPI method evaluation (User Guide 9.21 and 9.22); its **All N items** lists the other items of this group. In the main window **Ctrl+F** is the search box and **Ctrl+Shift+F** opens Find. You can also type an item's name or id into the search box (Ctrl+F). The rest of this introduction describes the classic view (Menu -> Classic View), with its Menu and panel buttons.

This group holds the things you *do* with SRS rather than the things it simply reads: CPU benchmarks, the Stress & Burnin load test (CPU and GPU), a core-to-core delay test, a memory latency test, and the look-up and search tools (device ID lookup, report search, configuration search, ping). Open the items from **Menu -> Tools -> item** (each entry shows as "Title [id]"). The live tool windows are in **Menu -> Benchmarks & Stress**: Stress & Burnin (CPU / GPU)..., Benchmark..., IPI Delay..., Latency..., SRS Pi..., Crypto Benchmark..., Prime Sieve..., Cache & Memory..., Storage Benchmark..., OS Latency..., CPU Compute..., Everyday Work..., Network Benchmark..., GPU Benchmark... and GPU Comparison (Vulkan, HIP, CUDA, OpenCL)...; Device ID Lookup... and Find in Report... are at the top of the Menu itself. The Processors window also has CPU Stress and CPU Burnin buttons (both open Stress & Burnin) and a Benchmark button, and the keys Ctrl+F (Find), Ctrl+L (Device ID Lookup) and Ctrl+D (ALL Devices) work in every window. None of these items needs root data (srs-root): everything here works as a normal user. Load tests and benchmarks never start by themselves, and every benchmark asks before it starts: what it runs, about how long, how many CPU threads it keeps busy (and, for SRS Pi, the memory it needs). Saving a report (`-save`) only shows the last stored result.

Contents

- CPU Benchmarks `benchmark`
- Benchmark: all CPUs concurrent `benchmark-ac`
- Benchmark: all CPUs sequential `benchmark-as`
- Benchmark: D+D then W+W `benchmark-dd`
- Benchmark: D+W then W+D `benchmark-dw`
- Benchmark: E-cores concurrent `benchmark-ec`
- Benchmark: E-cores one at a time `benchmark-ep`
- Benchmark: logical threads concurrent `benchmark-lc`
- Benchmark: logical threads one at a time `benchmark-lp`

- Benchmark: physical cores concurrent `benchmark-pc`
- Benchmark: physical cores one at a time `benchmark-pp`
- BOINC status `boinc`
- BOINC RPC info `boinc-rpc`
- ClearCase `clear-case`
- ClearQuest `clear-quest`
- ClearQuest e-mail `cq-mail`
- CPU IPI Delay `ipi-delay`
- IPI Dump `ipi-dmp`
- IPI Plots `ipi-plots`
- Memory Latency `latency`
- Cache & Memory Benchmark `cache-memory`
- Storage Benchmark `storage-bench`
- OS Latency Benchmark `os-latency`
- CPU Compute Benchmark `cpu-compute`
- Everyday Work Benchmark `everyday-work`
- Network Benchmark `net-bench`
- GPU Benchmark `gpu-bench`
- GPU Burnin `gpu-burnin`
- GPU Compute: Vulkan vs HIP vs CUDA vs OpenCL `gpu-compare`
- SRS Pi `srs-pi`
- Crypto Benchmark `crypto-bench`
- Prime Sieve `prime-sieve`
- Benchmark History `bench-history`
- Lookup `lookup`
- Ping `ping`
- Configuration Find `reg-find`
- Device ID Lookup `id-lookup`
- Find `find`
- ALL Devices `all-dev`
- Stress & Burnin (CPU and GPU) `burnin`
- CPU Stress `cpu-stress`

CPU Benchmarks

`benchmark` - Linux data

In plain words. A summary table of the ten benchmark modes. It shows the last total score of each mode, or "not run". It does not run anything. In the Benchmark window, **Run asks first**: the mode, about how long it takes and how many CPU threads are busy; the modes that load many threads at once (`ac`, `pc`, `lc`) warn about unstable overclocks and default to **No**. On the command line `sysreport-search bench MODE` asks the same; `--yes` starts at once.

What you see.

Column	Meaning	Why it matters
Mode	Two-letter mode code (ac, as, dd, dw, ec, ep, lc, lp, pc, pp).	The same code is used on the command line: `srs-gui bench ac`.
Description	What the mode does (which CPUs run, and in what order).	Tells you what the score is measuring.
Last result	The `Total` line of that mode's last run: total DMIPS and total MWIPS, or `not run`.	A quick side-by-side view of all modes.

The two scores:

- **DMIPS** (Dhrystone MIPS) - from the Dhrystone 2.1 test, which does whole-number (integer) work: string copies, record and pointer handling, small function calls. 1 DMIPS = 1757 Dhrystone loops per second, the speed of a 1978 DEC VAX 11/780. So "125000 DMIPS" means one thread works about 125,000 times faster than that machine on this test.
- **MWIPS** (Million Whetstone Instructions Per Second) - from the Whetstone test, which does floating-point (decimal) maths: arithmetic, sine/cosine/arctangent, square root, log and exp.

Healthy / typical values. On the author's Ryzen 7 7800X3D (8 cores, 16 threads):

Mode	Total DMIPS	Total MWIPS
ac	1644837	200303
as	2024492	236824
dd	1789518	218525
dw	1888968	230582
lc	983919	116411
lp	1023750	120536
pc	987836	116316
pp	1024949	120486

ec and ep show `not run` on any non-hybrid CPU.

Unusual values and what they suggest.

- The results are stored separately and can be from different days, kernels or BIOS settings. Open the single mode item and check its `Run` line before you compare two modes.
- A total far below an earlier run of the same mode: the CPU was hot and throttling, a background job was busy, or a power plan / BIOS setting changed.

Where Linux gets it. Built-in Dhrystone 2.1 and Whetstone code in SRS. Results are kept in `~/.cache/srs/bench-<mode>.txt`. SMT sibling pairs come from `/sys/devices/system/cpu/cpuN/topology/thread_siblings_list`.

Background. Dhrystone and Whetstone are classic, very small tests. They fit inside the L1 cache, so they measure the core itself (clock speed, pipeline, compiler), not memory or disks. Scores depend a lot on the compiler, so compare SRS with SRS, not with other programs.

For a step-by-step reading of all the modes on one PC (what SMT adds, why all-core runs score a little lower, why mixed work shares a core better), see the User Guide, section 9.2.1 What hyperthreading (SMT) really does - a worked example.

Benchmark: all CPUs concurrent

`benchmark-ac` - Linux data

In plain words. Every logical CPU runs Dhrystone at the same time, then every logical CPU runs Whetstone at the same time. This is the "whole chip, fully loaded" score.

What you see. The layout below is the same for every benchmark mode item.

Field	Meaning	Why it matters
Header	"CPU Benchmark [ac] on \\host - distro kernel".	Shows which kernel the run used.
Mode	The mode description.	What was measured.
Run	Date and time of the run, then "N s per measurement".	How old the result is, and how long each test ran. Longer runs are steadier.
Note	Only shown when the run has a remark.	Extra information about the run.
CPU	`CPU-n`, the logical CPU the test thread was pinned to.	Lets you spot one weak CPU.
Dhrystone	That CPU's score in DMIPS, or `-` if not measured.	Integer speed.
Whetstone	That CPU's score in MWIPS, or `-` if not measured.	Floating-point speed.
Total	Sum of the column.	The overall score for the mode.
Footer	Board maker, model and BIOS version.	Ties the result to a BIOS version.

If a mode has never been run, the item instead shows `Status Not run yet`, `Run it` (Benchmarks & Stress > Benchmark, then Run), `Command line` (for example `srs-gui bench ac`) and the note "Benchmarks are never started by -save; this section shows the last result."

Healthy / typical values. On the author's 7800X3D (5 s per measurement): each CPU scored about 101000-104000 DMIPS and 12300-12700 MWIPS. Total 1644837 DMIPS and 200303 MWIPS. A spread of 2-3 % between CPUs is normal.

Unusual values and what they suggest.

- One CPU 10 % or more below the rest: something else was running on it, or that core is clocked lower (core parking, a per-core undervolt or Curve Optimizer value that is too aggressive and makes the core stretch its clock).
- Scores fall during a long run: heat. Check the cooler and the CPU temperature in Sensors.

Where Linux gets it. Built-in tests. Each thread is pinned to its CPU with CPU affinity (`pthread_setaffinity_np`).

Background. With all 16 threads busy, the two threads of each core share one core. That is why each CPU scores lower here than in the "one at a time" modes. The total is still higher, because SMT (simultaneous multithreading) lets a core do more work with two threads than with one.

Benchmark: all CPUs sequential

`benchmark-as` - Linux data

In plain words. The test runs on one logical CPU at a time, CPU-0 to the last one. Each CPU runs Dhrystone and then Whetstone while all the other CPUs are idle.

What you see. The same layout as all CPUs concurrent.

Healthy / typical values. On the author's 7800X3D (0.1 s per measurement): about 125000-129000 DMIPS and 14400-15100 MWIPS per CPU. Total 2024492 DMIPS.

Unusual values and what they suggest.

- The total here is a sum of single-thread scores. The chip can never reach it in real use, because two threads on one core share it. Compare it with `ac` to see the SMT effect: on the author's PC, 1644837 / 2024492 = 0.81. With all threads busy, each thread keeps 81 % of its solo speed, so one core with two threads does about 1.6 times the work of one thread.
- One or two CPUs clearly faster than the rest: those are the "preferred" (best-binned) cores that boost highest when only one core is busy. That is normal on Ryzen and Intel Turbo Boost Max 3.0.
- Very short runs (0.1 s) are noisy. Use 1 s or more for a fair comparison.

Where Linux gets it. Built-in tests, one pinned thread at a time.

Benchmark: D+D then W+W

`benchmark-dd` - Linux data

In plain words. The test goes core by core. Both threads of one core run Dhrystone together, then both run Whetstone together. Then it moves to the next core.

What you see. The same layout as all CPUs concurrent. The rows come in sibling pairs: CPU-0, CPU-8, CPU-1, CPU-9 and so on. CPU-0 and CPU-8 are the two threads of core 0.

Healthy / typical values. On the author's 7800X3D: about 109000-114000 DMIPS and 13400-13900 MWIPS per thread. Total 1789518 DMIPS and 218525 MWIPS. Both threads of a pair score almost the same.

Unusual values and what they suggest.

- The two threads of a pair score very differently: something else was running on one of them.
- One whole pair is lower than the others: that core is slower (a lower boost or a Curve Optimizer offset), or it is hotter.

Where Linux gets it. Built-in tests. Sibling pairs come from `thread_siblings_list`.

Background. Two threads doing the *same* kind of work fight over the same execution units, so each one slows down.

Benchmark: D+W then W+D

`benchmark-dw` - Linux data

In plain words. Core by core, one thread runs Dhrystone while its sibling runs Whetstone at the same time. Then they swap. Each thread ends up with one Dhrystone and one Whetstone score.

What you see. The same layout and pair order as D+D then W+W.

Healthy / typical values. On the author's 7800X3D: total 1888968 DMIPS and 230582 MWIPS, about 5 % higher than `dd`.

Unusual values and what they suggest. If `dw` is *not* higher than `dd`, the core's integer and floating-point sides are not working in parallel well. This can happen with SMT disabled in the BIOS, or when the scheduler moved threads around during the run.

Where Linux gets it. Built-in tests, sibling pairs from sysfs.

Background. Dhrystone uses the integer units and Whetstone uses the floating-point units. Mixed work shares a core better than identical work. That is the main reason SMT helps real programs.

Benchmark: E-cores concurrent

`benchmark-ec` - Not present on most PCs

In plain words. Meant for hybrid CPUs (for example Intel 12th generation and newer) that mix big Performance cores (P-cores) with small Efficiency cores (E-cores). It would run the test on all E-cores at once.

What you see. On a CPU without E-cores: `Mode Efficiency cores concurrently`, `Status No efficiency cores detected on this CPU` and `Checked /sys/devices/cpu_atom/cpus (Intel hybrid CPUs)`. On an Intel hybrid CPU the mode runs like the other modes, on the E-cores listed in that file.

Healthy / typical values. On the author's Ryzen 7 7800X3D, and on any CPU where all cores are the same type, "No efficiency cores detected" is the normal and correct answer. AMD CPUs that mix Zen 5 and Zen 5c cores are not told apart yet.

Benchmark: E-cores one at a time

`benchmark-ep` - Not present on most PCs

In plain words. The same idea as E-cores concurrent, but on one E-core at a time.

What you see. On a CPU without E-cores: `Mode Efficiency cores one at a time` and `Status No efficiency cores detected on this CPU`; on an Intel hybrid CPU, one result per E-core.

Benchmark: logical threads concurrent

`benchmark-lc` - Linux data

In plain words. Only the *second* thread of each core runs (CPU-8 to CPU-15 on the author's PC), all at the same time. The first threads stay idle.

What you see. The same layout as all CPUs concurrent. Only the second-sibling CPUs are listed.

Healthy / typical values. On the author's 7800X3D: about 121500-123500 DMIPS per thread, total 983919 DMIPS and 116411 MWIPS. That is almost the same as `pc` (987836), which runs the first threads.

Unusual values and what they suggest. If `lc` is much lower than `pc`, the "second" threads are not on their own cores. The topology may be reported wrongly, or another task was using the first threads.

Where Linux gets it. Built-in tests. A CPU counts as the "second" thread when its sibling has a lower number in `thread_siblings_list`.

Background. A logical CPU (logical thread) is what Linux calls each hardware thread. With SMT, one physical core shows up as two logical CPUs. Neither one is "real" or "fake": when its partner is idle, either one gets the whole core.

Benchmark: logical threads one at a time

`benchmark-lp` - Linux data

In plain words. The second thread of each core runs alone, one after another.

What you see. The same layout as all CPUs concurrent.

Healthy / typical values. On the author's 7800X3D: about 124500-130000 DMIPS per thread, total 1023750 DMIPS. Almost the same as `pp` (1024949).

Unusual values and what they suggest. A clear gap between `lp` and `pp` for the same core means one of the two threads was disturbed during its run. Run it again.

Where Linux gets it. Built-in tests.

Benchmark: physical cores concurrent

`benchmark-pc` - Linux data

In plain words. One thread per physical core (the first sibling), all cores at the same time. This is the all-core score without SMT.

What you see. The same layout as all CPUs concurrent. On the author's PC: CPU-0 to CPU-7.

Healthy / typical values. On the author's 7800X3D: about 122000-124000 DMIPS per core, total 987836 DMIPS and 116316 MWIPS.

Unusual values and what they suggest. Compare with `ac`. On the author's PC, 1644837 / 987836 = 1.67, so the second threads add about 67 % on this test. A ratio near 1.0 means SMT is off or not working.

Where Linux gets it. Built-in tests, first sibling of each core.

Benchmark: physical cores one at a time

`benchmark-pp` - Linux data

In plain words. One thread per physical core, one core after another, with the rest of the chip idle. This is the single-core score of each core.

What you see. The same layout as all CPUs concurrent.

Healthy / typical values. On the author's 7800X3D: about 125800-129700 DMIPS per core, total 1024949 DMIPS and 120486 MWIPS.

Unusual values and what they suggest. The ranking of cores here matches the CPU's "preferred core" order. One core well below the others in this mode (with the chip otherwise idle) points to a per-core limit: a Curve Optimizer value that is too negative, or a core that cannot boost.

Where Linux gets it. Built-in tests.

Background. Single-core speed matters most for games and older programs that use one main thread. All-core speed matters for compiling, rendering and encoding.

BOINC status

`boinc` - Not present on most PCs

In plain words. BOINC (Berkeley Open Infrastructure for Network Computing) is a volunteer computing client (SETI@home, Rosetta@home and others). SIV can show the state of a running BOINC client.

What you see. `Status Not present on this system` and `Checked n/a (boinc not installed)`.

What "not present" means. No BOINC client data is shown. If you do not run BOINC, there is nothing to do.

BOINC RPC info

`boinc-rpc` - Not present on most PCs

In plain words. BOINC clients take commands over a small network control channel, RPC (Remote Procedure Call). In SIV this item shows the details of that connection.

What you see. `Status Not present on this system` and `Checked n/a (boinc not installed)`.

ClearCase

`clear-case` - Not present on most PCs

In plain words. IBM Rational ClearCase is a version-control system used in large companies. SIV can report on an installed ClearCase client.

What you see. `Status Not present on this system` and `Checked n/a (not installed)`.

ClearQuest

`clear-quest` - Not present on most PCs

In plain words. IBM Rational ClearQuest is a bug- and change-tracking system that goes with ClearCase.

What you see. `Status Not present on this system` and `Checked n/a (not installed)`.

ClearQuest e-mail

`cq-mail` - Not present on most PCs

In plain words. The e-mail notification settings of a ClearQuest client.

What you see. `Status Not present on this system` and `Checked n/a (ClearQuest not installed)`.

CPU IPI Delay

`ipi-delay` - Linux data

In plain words. It measures how long it takes one CPU to pass a message to another CPU through the cache. Every pair of CPUs is tested, and the result is a table in nanoseconds.

What you see.

Field	Meaning	Why it matters
Header	"CPU IPI Delay (core-to-core cache-line round trip / 2, ns)".	The number is the one-way time: a full there-and-back trip divided by 2.
Rounds per pair	How many there-and-back trips were timed for each pair (GUI default 20000, range 1000-1000000).	More rounds give a steadier average.
From\To	Row = sending CPU, column = receiving CPU.	Find any pair at the crossing point.
`-`	The diagonal (a CPU to itself). Not measured.	
Numbers	One-way delay in ns. Shown in green below 40 ns, normal colour from 40 to 99 ns, orange at 100 ns or more.	Low = the two CPUs are close together on the chip.
Footer	Board and BIOS.	

Each pair is measured once, so the table is a mirror: the CPU-3 -> CPU-5 cell always equals CPU-5 -> CPU-3. If it has never been run, the item shows `Status Not run yet`, `Run it` (Benchmarks & stress > IPI Delay, then Run) and `Command line srs-gui ipi [--cpus N] [--rounds R]`.

The GUI window (Menu -> Benchmarks & Stress -> IPI Delay...) has a CPUs box (test only the first N CPUs), a Rounds per pair box, a Run button (it asks first: two CPU threads busy at a time, about a second at the defaults; `sysreport-search ipi` asks the same, `--yes` starts at once), and a heat map under the text. With CPUs = 1 (or a one-CPU system) there is no pair to measure: nothing runs, the window says "Not started" (`ipi --cpus 1` exits with 2) and the last result is kept. The heat map is coloured from green (fastest) to red (slowest), scaled to the largest value.

Healthy / typical values. On the author's 7800X3D (one CCD, 8 cores, 50000 rounds):

- Sibling pairs (CPU-0 and CPU-8, CPU-1 and CPU-9, ...): 24-25 ns. They share one core and its L1 and L2 caches.
- All other pairs: about 28-39 ns. They meet in the shared L3 cache.
- The table is flat apart from the sibling diagonal, because all 8 cores sit on the same CCD.

Unusual values and what they suggest.

- A block of the table much higher than the rest (often 70-150 ns): the CPU has more than one CCD (core complex die), for example a Ryzen 9 7950X, or more than one socket or NUMA node. Pairs on different dies must go through the Infinity Fabric (or the ring/mesh between tiles) and are much slower. Games and audio work run better when their threads stay on one die.
- One CPU slow to every other CPU: it was busy with other work during the test, or it was in a deep sleep state. Close other programs and run again.
- Siblings no faster than other pairs: SMT may be off, or the CPU numbering is not what you expect.

Where Linux gets it. Built-in test. Two threads are pinned to the two CPUs and bounce one shared cache line (a flag in memory) back and forth with atomic loads and stores. The first third of the rounds is an untimed warm-up. Timing uses `CLOCK_MONOTONIC`. The result is stored in `~/.cache/srs/bench-ipi-delay.txt`.

Background. "IPI" stands for Inter-Processor Interrupt, a signal one CPU sends to another. SRS keeps SIV's name, but it does not send real interrupts. It measures the core-to-core cache-line delay, which is the cost every program pays when threads on two cores share data (locks, queues, game engine job systems).

For a step-by-step reading of one real 16-CPU result (partner threads, the shared L3, what a two-CCD CPU looks like, and what the table is useful for), see the User Guide, section 9.2.2 Reading the CPU IPI Delay table - a worked example.

IPI Dump

`ipi-dmp` - Linux data

In plain words. In SIV this is the raw dump of the IPI measurements. In SRS it prints exactly the same table as CPU IPI Delay.

What you see. The same `Rounds per pair` line and From\To matrix, from the last stored run. See CPU IPI Delay for what every number means.

Where Linux gets it. `~/.cache/srs/bench-ipi-delay.txt` (the last IPI run).

IPI Plots

`ipi-plots` - Linux data

In plain words. In SIV this shows the IPI results as graphs. In a saved report SRS prints the same table as CPU IPI Delay. The picture version is the heat map in the GUI IPI Delay window.

What you see. In a report: the same matrix as CPU IPI Delay. In the GUI: a square grid with one cell per CPU pair, green for the fastest pairs and red for the slowest. The heat map appears after a run in the open program. After a restart you see the stored table but an empty map until you press Run again.

Healthy / typical values. On the author's single-CCD 7800X3D the map is almost one colour, with a darker green diagonal line for the sibling pairs. A dual-CCD CPU shows four clear squares: two fast (inside each die) and two slow (between the dies).

Where Linux gets it. Same test as CPU IPI Delay.

Memory Latency

`latency` - Linux data

In plain words. It measures how long the CPU waits for one piece of data, using blocks of memory that get bigger and bigger. Small blocks fit in the fast caches. Big blocks have to come from RAM. The jumps in the result show where each cache ends.

What you see.

Column	Meaning	Why it matters
Block	Size of the memory block tested: 4KB, 8KB, 16KB, ... doubling up to the largest size (default 256MB; 1-4096 MB in the GUI and on the command line).	Each size lands in a different cache level.
Latency	Average time for one load, in nanoseconds. Best of 5 passes of 2 million loads.	Lower is faster.
Level	L1, L2, L3 or DRAM - a rough label for where a block of that size usually lives.	Helps you read the table.

The last line names the cache sizes used for the Level column, read from `/sys/devices/system/cpu/cpu0/cache` (for example "L1d 32KB, L2 1MB, L3 96MB"): a block is labelled with the smallest cache it fits in, and DRAM above the last level. Labels are indicative; the jumps in the Latency column are the real boundaries (compare Cache Levels, `cache`). If never run, the item shows `Status Not run yet` and `Command line srs-gui latency [--max MB]`.

The GUI window (Menu -> Benchmarks & Stress -> Latency...) has a "Largest block (MB)" box, a Run button (it asks first: one CPU thread and that much memory, about 3 seconds at 256 MB; `sysreport-search latency` asks the same, `--yes` starts at once) and a line graph of latency against block size.

Healthy / typical values. On the author's 7800X3D (L1 data 32K, L2 1024K, L3 96 MB "3D V-Cache"), run up to 64MB:

- 4KB-32KB: about 1-4 ns (L1). The 4KB result, 0.94 ns, is about 4-5 clock cycles at 5 GHz.
- 64KB-1MB: about 3.0-3.3 ns (L2), roughly 14 cycles.
- 2MB-64MB: about 8.6-11 ns (L3). 64MB still reads as L3 because the X3D's L3 is 96 MB.
- Larger blocks (128MB, 256MB) would show the DRAM step. On a typical DDR5 system that is around 60-90 ns.

Small blocks can jump around a little (8KB was 3.74 ns in the same run). That is timing noise: the whole 4-32 KB range is still L1.

Unusual values and what they suggest.

- The L2 to L3 step happens at a smaller block than your L2 size: something else is using the cache, or the run was disturbed.
- DRAM latency well above about 100 ns on DDR5 (or about 80 ns on a tuned DDR4 system): loose memory timings, EXPO/XMP not enabled, or the memory controller running at a lower ratio (for example UCLK at half the memory clock on AM5).
- L3 latency much higher than expected on a multi-die CPU: on most Ryzen chips each CCD has its own L3. A block bigger than one CCD's L3 goes to DRAM, not to the other die's cache.

Where Linux gets it. Built-in "pointer chase": a buffer is filled with links in random order, and each load needs the address from the load before. That defeats prefetching, so every step pays the full delay. Lines are 64 bytes. A 200 ms busy spin first brings the core to full clock. Stored in `~/.cache/srs/bench-latency.txt`.

Background. A cache level is a small, fast memory between the core and RAM. L1 is the smallest and fastest (tens of KB per core), L2 is bigger (about 1-2 MB per core), and L3 is shared by many cores (tens of MB). Data the CPU uses often stays close. Latency (waiting time) matters most for programs that jump around in memory, such as games and databases.

Cache & Memory Benchmark

`cache-memory` - Linux data

In plain words. How fast the RAM and each cache level (L1, L2, L3) can **read**, **write** and **copy** data, and how long each takes to answer one request (**latency**). It shows whether the memory runs as fast as it should after a BIOS, EXPO/XMP or memory-timing change, and what each cache level gives you.

What you see. A table: rows Memory (RAM), L1 cache, L2 cache, L3 cache; columns Read, Write, Copy (MB/s for the RAM, GB/s for the caches; 1 MB = 1,000,000 bytes) and Latency (ns). Below it: threads and vector code used (AVX-512, AVX2 or SSE2), how the RAM write was measured, the processor, the measured clock during the run, the **base clock** (measured, so a board running 105 MHz shows 105), the multiplier, the **Infinity Fabric clock** (AMD Ryzen 7000 desktop with the `ryzen_smu` driver), the memory type, speed and channels (from root data), the memory clock as a multiple of the base clock, and the cache sizes.

Value	How it is measured
Read / Write / Copy	One thread pinned to every chosen logical CPU, each in its own block sized to stay inside that cache level (half of its share of L1 / L2 / L3), or for the RAM a block of at least 4 x L3 (at least 256 MB, default 1 GB). Best of repeated 40-80 ms trials over the measuring time (default 1 s per value). **Copy counts the bytes read plus the bytes written.**
RAM Write on AMD	With CLZERO (whole zeroed cache lines written to memory), one thread per core, in 4 KB turns, over 256 MB or 2.5 x L3 - what the memory itself absorbs. Ordinary streaming stores stop at the core-to-memory link's write limit; `--mem-write stream` shows that figure. Other CPUs use streaming stores.
Latency	One thread, a random chain through every 64-byte line of a block. Caches on ordinary 4 KB pages (L1: half of L1d, L2: half of L2, L3: 8 x L2); the RAM on 2 MB pages over 8 x L3 (at least 256 MB, at most half of free RAM), so page-table walks and L3 hits do not distort it.

Healthy / typical values. Ryzen 7 7800X3D, 2 x 16 GB DDR5-6000, FCLK 2000-2133, all 16 threads: RAM about 63,700 MB/s read, 93,800 MB/s write (CLZERO), 63,600 MB/s copy, 80-82 ns; L1 about 2,300 GB/s read and 0.8 ns; L2 about 1,200 GB/s and 3.7 ns; L3 about 700 GB/s and 11-12 ns. Results vary by a few % from run to run.

Unusual values and what they suggest.

- RAM read far below the kit's rating (DDR5-6000 dual channel: roughly 60,000+ MB/s): EXPO/XMP off, one stick in the wrong slot (single channel), or the memory controller at a lower ratio.
- RAM latency well over 90 ns on DDR5: loose timings, or UCLK at half the memory clock.
- Everything 10-50% low or jumping between runs: other programs are busy. Close them and run again; **never run another benchmark (or a second copy of SRS) at the same time** - both results are spoiled.

Where Linux gets it. Built in. The measuring engine comes unchanged from Linux System Audit (LSA) 0.2.0, the same author's project, now part of SRS. x86-64 CPUs only (it uses x86-64 vector code). The base clock is the TSC rate x 100 / the CPU's ACPI CPPC nominal frequency; the Infinity Fabric clock is read (read-only) from the `ryzen_smu` driver's PM table, layout 0x540104 only; the memory sticks come from the root DMI data (Tools > Collect root data). Stored in `~/.cache/srs/bench-cachemem.txt`; each run is added to the Benchmark History. It asks before it starts, and only one benchmark runs at a time in SRS.

Storage Benchmark

`storage-bench` - Linux data

In plain words. How fast the drive that holds a folder you choose can read and write: big files in one go (sequential), lots of small 4 KB pieces at once (like a busy system), and one small piece at a time (how snappy the PC feels). It works on its **own temporary test file only** - your files are never opened, changed or deleted.

What you see. Six rows - Sequential 1 MiB (Q8T1), 4K random (Q32T16; Q8T4 in the Light profile) and 4K random (Q1T1), each Read and Write - with **Average** (the whole test), **Peak** (the best quarter second) and **IOPS** (requests per second) for the 4K rows. Then the response time of single 4K reads and writes (mean, median, 95% and 99% under, slowest), the drive model and connection (NVMe, SATA, USB; **never the serial number**), the folder and file system, the test file size and I/O mode, and the drive temperature before and after (when the drive has a sensor). Q = requests in flight per thread, T = threads; Q x T requests are in flight at once.

Setting	Choices
Folder	Any folder you can write to on the drive you want to test (default: your home folder)
Test file	256 MB, 1 GB (default), 4 GB, 16 GB
Seconds per test	1-30 (default 5)
Profile	Standard (4K random Q32T16 = 16 threads) or Light (Q8T4 = at most 4 threads)
Allow USB drives	Off by default

Safety.

- The test file is created new (it can never be one of your files) and is **unlinked at once**: it has no name while the test runs and its space is freed the moment the test ends - also after Stop, a crash or a forced kill. Nothing is ever left behind.
- Refused: folders in RAM (tmpfs - `/tmp` on many systems), network and FUSE file systems, read-only media, and anything with less than twice the test size free (+0.5 GB). USB drives only when you tick **Allow USB drives** (some USB enclosures drop off under heavy load). Hard disks get a warning.
- It asks before it starts, showing the drive, the folder and how much it writes.
- Writes count only once they are on the drive (`fdatasync` is inside the timing).

Healthy / typical values. Samsung 980 PRO 1 TB (PCIe 4.0 x4 NVMe), btrfs, 64 MB file, 1 s per test, Light profile: sequential read about 6,500 MB/s, write about 3,200 MB/s; 4K Q8T4 read about 1,400 MB/s (350,000 IOPS); 4K Q1T1 read about 51 MB/s (median 72 us), write about 150 MB/s. A SATA SSD tops out near 550 MB/s sequential; a hard disk manages about 150-250 MB/s sequential and around 1 MB/s (100-200 IOPS) at 4K Q1T1.

Unusual values and what they suggest.

- NVMe sequential read far below its rating: the drive sits in a slower M.2 slot or PCIe lanes are shared (see the PCI pages for the link speed and width), or it is hot and throttling (watch the temperature line).
- Write much lower on a longer test than a short one: the drive's fast SLC write cache filled up; that is normal for consumer SSDs.
- Small test files (256 MB) can fit in the drive's own cache and read very fast; use 1 GB or more for figures to compare.
- "buffered I/O" in the Test file line: the file system refused direct I/O; the page cache was dropped before each read test, but results can still be a little high.

Where Linux gets it. Built in: `O_DIRECT` I/O with Linux native asynchronous I/O (`io_submit`) for real queue depths, incompressible random data; the drive is found through `/proc/self/mountinfo` and `/sys/block` (device-mapper and RAID followed to the first member); the temperature from the drive's hwmon sensor. Drive health (SMART) is on the Storage page. Stored in `~/.cache/srs/bench-storage.txt`; each run is added to the Benchmark History.

OS Latency Benchmark

`os-latency` - Linux data

In plain words. How quickly the operating system answers a program: how long a basic request to the kernel takes, how fast it switches between two programs, how long it takes to start a thread or a program, how soon a sleeping program wakes up, and what a new page of memory or a small file costs. Lower is better. Steady numbers (95% and 99% close to the median) mean a smooth, responsive desktop.

What you see. One row per test with **Median** (typical), **95%** and **99%** (19 in 20 and 99 in 100 samples were this fast or faster), **Worst**, **Best**, the number of samples and how it was measured. The groups are System calls, Context switches, Starting threads and processes, Wake-up latency, Timers, Memory mapping and page faults, and File metadata. Below the table: the CPU scheduler that is running (for example scx_lavd under sched_ext, BORE or EEVDF), the preemption mode, the timer tick (CONFIG_HZ), the clock source, the timer slack, transparent huge pages, the CPUs used and the file system of the test folder with its mount point (a mount inside your home folder is shown as "your home folder", so no user name lands in the result). Times are in ns, µs or ms.

Group (`--only` word)	Rows
System calls (`syscall`)	getpid() as a real system call; clock_gettime through the vDSO and as a system call; read() of 1 byte from /dev/zero; sched_yield() (each sample is a batch of 32, divided)
Context switches (`switch`)	Pipe ping-pong between 2 threads and between 2 processes, on the same CPU and on two cores; the figure is half a round trip
Starting threads and processes (`spawn`)	Thread create + join; fork() + wait (with the program's RAM use, which fork cost grows with); posix_spawn of /bin/true + wait
Wake-up latency (`wake`)	Futex wake on two cores and not pinned; condition-variable signal on two cores; from the wake call until the sleeper runs
Timers (`timer`)	How late a 50 µs and a 1 ms sleep end; how late a periodic 1 ms tick fires, with missed ticks counted
Memory (`memory`)	First-write page fault on a 4 KiB page and on a 2 MiB transparent huge page (skipped when THP is `never`); mmap and munmap of 64 KiB
File metadata (`files`)	Create a small file (open + 100-byte write + close), stat it, delete it

Setting	Choices
Samples	1,000-200,000 in the window (command line 100-1,000,000), default 20,000
Tick test	1-30 s in the window (command line 0.1-60), default 3

Safety. It asks before it starts. It only reads system settings: no root, and no scheduler, priority or timer change. At most two CPUs are busy at a time, for about 6 seconds at the defaults. CPU affinity follows the limits you set (taskset, cpusets). The file test works in its own temporary folder in `~/.cache/srs`, which is removed at the end, after Stop, and (for a killed run) on the next run; a folder that holds anything not made by SRS is left alone.

Healthy / typical values. Ryzen 7 7800X3D, Linux 7.3 with scx_lavd (medians): system call 35 ns, vDSO clock read 16 ns, pipe switch 1.1 µs on one CPU and 2.0 µs between two cores, thread start 10 µs, fork 59 µs, program start 142 µs, futex wake 1.9 µs, 4 KiB page fault 0.4 µs, 2 MiB page fault 19 µs, small file create 1.8 µs (tmpfs). Sleeps and the 1 ms tick run about 50 µs late, which is Linux's default timer slack.

Unusual values and what they suggest.

- Wake-ups, the 99% column or Worst in the hundreds of µs, or missed ticks: deep CPU power-saving states (C-states), heavy background work or a busy scheduler. Outliers like this on idle cores are real wake-up costs, not a fault in SRS.
- Sleeps late by much more than 50 µs: a larger timer slack (some power-saving tools raise it).
- A slow fork: a large program; the row shows its RAM use.
- 2 MiB faults much slower than usual: the kernel had to compact memory to find a huge page; the row says how many faults really got one.

Where Linux gets it. Built in: system calls, pipes, futexes and clocks timed with `clock_gettime`; the scheduler from `/sys/kernel/sched_ext` (sched_ext ops such as scx_lavd), `/proc/sys/kernel/sched_bore`, the kernel config or the kernel version; preemption from `uname` and the kernel command

line; CONFIG_HZ from `/boot/config-<release>` or the kernel headers' `.config` (not from `/proc/config.gz`); the clock source from `/sys/devices/system/clocksource`; THP from `/sys/kernel/mm/transparent_hugepage`. Stored in `~/.cache/srs/bench-os-latency.txt`; each run is added to the Benchmark History. Only one benchmark runs at a time in SRS.

CPU Compute Benchmark

`cpu-compute` - Linux data

In plain words. How fast the processor does floating-point number crunching, the maths in 3D, video, science and AI programs. Higher is better. Every answer is checked: a wrong answer is shown as **WRONG** instead of a score.

What you see. One row per test with the result on **1 thread** and on **N threads** (default all logical CPUs, one pinned thread per CPU, each with its own copy of the work), the unit, the code path used (**Code**) and the check. Below: the self-test, the threads and the measuring time.

Test (`--tests` word)	What it does	Unit
Matrix multiply, single / double (`sgemm`, `dgemm`)	288 x 288 matrices, register-blocked	GFLOPS (2 n^3 per multiply)
Peak multiply-add, single / double (`fma-sp`, `fma-dp`)	12 independent chains of multiply-adds	GFLOPS (a fused multiply-add counts 2)
FFT, single / double (`fft-sp`, `fft-dp`)	4096-point complex radix-2 transform	GFLOPS (5 N log2 N per transform)
N-body (`nbody`)	gravity between 1,024 bodies, double precision	GFLOPS (20 per pair)
Mandelbrot (`mandelbrot`)	640 x 480, up to 255 steps, double precision	Mpixels/s
Ray tracer (`raytrace`)	320 x 240, 4 balls on a floor, shadows, 2 reflections, single precision	Mpixels/s

Setting	Choices
Threads	1 to all logical CPUs (default all)
Seconds per test	0.1-10 in the window (command line 0.1-60), default 0.5
Code	best available (default), plain C, SSE2, AVX2 + FMA, AVX-512; a path the CPU lacks falls back to the best it has
1 thread only	skip the N-thread column

How the answers are checked. Matrix multiply uses inputs whose products and sums are exact, so the result must match an integer reference exactly on every code path. Multiply-add returns every value to

its start exactly. The FFT is compared with a direct transform and an energy check (Parseval). N-body is compared with a separate double-precision reference. The Mandelbrot vector code must give the same picture as plain C. The ray tracer's single-precision picture is compared with a double-precision one. Every timed repeat must give the same output as the first. A failed check shows the row as WRONG (red), the history line says WRONG and the exit code is 1.

Safety. It asks before it starts. With N threads every CPU is 100% busy and gets hot, and AVX-512 draws a lot of power: an unstable overclock or undervolt may crash the PC. **1 thread only** keeps it light. It does not start while Stress & Burnin runs.

Healthy / typical values. Ryzen 7 7800X3D, AVX-512, 2 threads, 0.1 s (a short test run, not a full result): SGEMM about 145 GFLOPS on 1 thread, DGEMM about 77, peak multiply-add single about 160, FFT single about 8, N-body about 29 GFLOPS, Mandelbrot about 44 Mpixels/s, ray tracer about 28 Mpixels/s; the 2-thread column is about double. Plain C SGEMM on the same CPU is about 7 GFLOPS, SSE2 about 47, AVX2 about 121.

Unusual values and what they suggest.

- **WRONG** in any row: the CPU gave a wrong answer - an unstable overclock, undervolt or memory setting. Go back to stock settings and run it again.
- The N-thread column far below threads x the 1-thread figure: thermal or power limits, or other programs are busy.
- AVX-512 not much faster than AVX2: normal on CPUs that run AVX-512 at half width (Zen 4) or lower their clock under AVX-512 load.

Where Linux gets it. Built in: SRS's own code with Intel intrinsics, the code path chosen at run time with `__builtin_cpu_supports`, so a generic x86-64 package still uses AVX-512. On ARM (aarch64) only plain C runs. Stored in `~/.cache/srs/bench-compute.txt`; each run is added to the Benchmark History. Only one benchmark runs at a time in SRS.

Everyday Work Benchmark

`everyday-work` - Linux data

In plain words. How fast the PC does the work it does all day long: compressing and unpacking files, sorting, checksums and hashes, searching and checking text, and running a small program in an interpreter. Higher is better. Every result is checked, so a wrong answer is reported instead of a fast number.

What you see. One row per test with the result on **1 thread** and on **N threads** (default all logical CPUs; one pinned thread per CPU, each with its own copy of the data) and the check. Compression rows also

show the **ratio** on text-like, record-like and random data. Below: the library versions found, the self-test, the data size and the threads.

Group (`--only` word)	Rows	Unit	How each result is checked
Compression (`compress`)	zlib -1 / -6 / -9, zstd -1 / -3 / -19, lz4, xz -1 / -6; compress and decompress	MB/s of original data	full round trip: length and every byte equal the original
Sorting (`sort`)	64-bit integers: C library qsort, radix sort; strings: qsort, multikey quicksort	Mkeys/s	in order, and a sum / hash of the keys shows nothing was lost or doubled
Checksums and hashes (`hash`)	CRC32 (byte table, slicing-by-8), CRC32C (table, and the CPU's SSE4.2 instruction in the 64-bit x86 build; the i686 and aarch64 builds show that row as "not built into this image"), xxHash64, FNV-1a	GB/s	known-answer self-test, then every pass equals a value from the simple version
Text processing (`text`)	substring search (memmem, Boyer-Moore-Horspool), JSON check, UTF-8 check	GB/s	match count, JSON token count and code point count equal what the generator wrote
Bytecode interpreter (`interp`)	switch dispatch, threaded dispatch	Mops/s	prime count and hash equal a plain C reference

Setting	Choices
Threads	1 to all logical CPUs (default all)
Seconds per test	0.1-10 in the window (command line 0.1-60), default 0.5
Test groups / libraries	tick boxes (command line `--only`, `--libs`)
1 thread only	skip the N-thread column
Data size	1024 KB of each kind per thread (command line `--part-kib`, 64-16384); Stop is checked between passes, so at 16384 one xz -6 pass can keep "Stopping..." for 10-20 s

Libraries. The compression libraries are the system's own - `libz.so.1`, `libzstd.so.1`, `liblz4.so.1`, `liblzma.so.5` - loaded when the test runs; SRS needs none of them to start. A missing one shows "not installed" and its rows are skipped.

Safety. It asks before it starts, with the expected time (about half a minute at the defaults; the estimate is rough). The data is generated by SRS from fixed seeds, so every run and every PC does the same work; your files are never read or written. With N threads every CPU is 100% busy; xz -6 and zstd -19 use about 100 MB of memory per thread, and fewer threads are used (with a note) when free memory is short. It does not start while Stress & Burnin runs.

Healthy / typical values. Depend strongly on the CPU and the library versions; compare runs on the same PC and SRS build. Ryzen 7 7800X3D, one thread, generic package build (a short run): lz4 about 1,000 MB/s compress and 7,400 MB/s unpack; zstd -1 about 650 / 2,200 MB/s; zlib -6 about 74 / 1,800 MB/s; zstd -19 and xz -6 about 6-7 MB/s compress; radix sort about 110 Mkeys/s against 17 for qsort; CRC32C with the CPU instruction about 14 GB/s, xxHash64 about 15 GB/s, CRC32 slicing-by-8 about 3 GB/s; the interpreter about 1,250-1,500 Mops/s. Random data does not shrink (ratio about 1.0); text and records shrink about 1.8-4.8 times.

Unusual values and what they suggest.

- A row marked as failed: a wrong result - an unstable overclock, undervolt or memory setting.
- "not installed": the library is missing; install it (`zstd`, `lz4`, `xz` / `liblzma` packages) to get those rows.
- N-thread figures far below threads x the 1-thread figure: memory bandwidth, thermal or power limits, or other programs are busy.

Where Linux gets it. Built in: SRS's own sorting, checksum, text and interpreter code; the compression libraries through `dlopen`; threads pinned with `sched_setaffinity`. Stored in `~/.cache/srs/bench-work.txt`; each test group adds a line (`work-compress`, `work-sort`, `work-hash`, `work-text`, `work-interp`) to the Benchmark History. Only one benchmark runs at a time in SRS.

Network Benchmark

`net-bench` - Linux data

In plain words. Two separate tests. **On this computer** (loopback, 127.0.0.1) sends data between two programs on this PC: how fast Linux and the CPU move network data, and how long a message takes to go there and back. Nothing leaves the computer and no network card is used. **Internet** (optional, asks separately) measures the real connection against Cloudflare's public speed test (speed.cloudflare.com): name lookup (DNS), connect and HTTPS times, and download and upload speed with 1 and N connections.

What you see.

Row	How	Unit
TCP, 1 stream (loopback)	one connection, 128 KiB writes	Mbit/s average and peak
TCP, N streams (loopback)	N connections at once (default 4)	Mbit/s
UDP, 1472-byte packets (loopback)	one sender, one receiver; lost packets counted	Mbit/s received, % lost

Row	How	Unit
TCP / UDP round trip (loopback)	8-byte ping-pong (TCP_NODELAY; UDP with sequence numbers)	median, best, 99% under, jitter
DNS lookup (internet)	first lookup of the server name, repeated (median of 5, usually cached), one made-up name under the server's domain (not cached)	ms
HTTPS request (internet)	3 new connections: TCP connect, TLS handshake, server reply (medians)	ms
Connect time (internet)	10 plain TCP connects to port 443 of each host	median, best, 99%, jitter, lost
Download / upload, 1 and N streams (internet)	`GET /__down?bytes=25000000` / `POST /__up` with 25 MB of random data per request, repeated until the time is up	Mbit/s average and peak

Below: the connection (kind such as wired, Wi-Fi, VPN; link speed, duplex, MTU, Wi-Fi signal), the IP version used, the libcurl version and whether a proxy variable is set (only its name).

Setting	Choices
Parts	On this computer (default on), Internet speed test (default off; asks separately)
Streams	1-32 (default 4)
Seconds per test	1-30 in the window (command line 0.1-60), default 5; round trip tests use at most 1 s
IP version	IPv4 or IPv6 (default), IPv4 only, IPv6 only
Data cap per transfer test	250 MB (command line `--max-mb`, 0 = no cap)

Safety and privacy.

- The loopback test asks before it starts; the internet test **asks separately** (No is the default) and states the server, the hosts, the tests, the time and the data use - up to about 1000 MB with the defaults (4 x 250 MB; about 250 MB on a 100 Mbit/s line). It also says the server sees your public IP address. It never runs without that Yes. Avoid metered or mobile connections.
- Nothing private is shown or saved: no IP address (local or public), MAC address, Wi-Fi network name (SSID), host name or proxy address. Interface names that contain a MAC address (`enx...`, `wlx...`) are shortened. Requests carry a plain `srs-sysinfo` user agent and no cookies; https only, certificates checked. Extra connect-time hosts (`--hosts`) are shown by name; one typed as a numeric address is shown as "host N (a numeric address)". A server URL with a port (`https://host:8443`) has its connect time measured on that port.
- Stop ends the current test within about 0.2 s (loopback) or at libcurl's next progress check (internet); a DNS lookup in progress finishes first (bounded by the system's resolver timeout).

Healthy / typical values. Loopback on a Ryzen 7 7800X3D (Linux 7.3, 0.3 s tests): TCP about 110 Gbit/s on 1 stream and 205 Gbit/s on 2, UDP about 8 Gbit/s with 0-1% loss, TCP and UDP round trip about 7 µs. Internet results depend on your plan: a result close to the speed you pay for is healthy; Wi-Fi is often half of it or less.

Unusual values and what they suggest.

- Loopback far below these figures: a busy CPU, power saving, or firewall / traffic-shaping rules that also apply to loopback.
- Internet download far below your plan: Wi-Fi, other traffic in the house, a VPN, or a busy server; run it again at another time and compare with https://speed.cloudflare.com in a browser.
- High jitter or lost connects: an overloaded router or Wi-Fi interference.
- Speed rows say "the server did not answer 200 OK": the public endpoints changed or are rate limited (they are not a documented API); DNS and connect times still work.

Where Linux gets it. Built in: plain sockets for loopback; `getaddrinfo` for DNS; libcurl (loaded at run time with `dlopen`, optional) for HTTPS and the transfers; the default route from `/proc/net/route` and `/proc/net/ipv6_route`; the interface from `/sys/class/net` and Wi-Fi signal from `/proc/net/wireless`. Setups with only policy routing (some VPNs) are reported as "no network connection". Stored in `~/.cache/srs/bench-net.txt`; each run is added to the Benchmark History. Only one benchmark runs at a time in SRS.

GPU Benchmark

`gpu-bench` - Linux data

In plain words. How fast the graphics card (GPU) computes and moves data, measured through Vulkan, the graphics and compute interface every modern Linux GPU driver offers. It runs short pieces of work on the GPU and checks every answer: a wrong answer is shown as WRONG, never as a speed. **Higher is better**, except for the latency row.

What you see.

Row	What it measures	Unit
FP32 multiply-add	single-precision (32-bit) floating-point maths, the kind games and most GPU programs use	GFLOPS
FP16 multiply-add	half precision (16-bit), used by AI and some game effects; "not supported" when the GPU or driver lacks it	GFLOPS
FP64 multiply-add	double precision (64-bit), used by science programs; most gaming GPUs do it at a small fraction of FP32 speed	GFLOPS
INT32 multiply-add	32-bit whole-number maths	GIOPS
Video memory read / write / copy	how fast the GPU reads and writes its own memory (copy counts the bytes read plus the bytes written)	GB/s
Upload / Download	data moving from system memory to the GPU and back, over PCIe on a graphics card	GB/s

Row	What it measures	Unit
Dispatch latency	the round trip of an empty piece of GPU work: how quickly the GPU answers	µs (median, best, 95% under)

Below: the device and its type (discrete GPU, integrated GPU, software renderer), the driver and its version, the Vulkan version, the device memory, the subgroup (wave / warp) size, the buffer size of the memory tests, how it was timed (GPU timestamps, else the host clock) and the **Build** line. GFLOPS = billions of floating-point operations per second (a fused multiply-add counts as 2); GIOPS the same for whole numbers; 1 GB = 1,000,000,000 bytes; µs = millionths of a second.

Setting	Choices
GPU	Default (the first discrete GPU, else the first device) or a device from the list; the list is read when you open it (through Vulkan, like `vulkaninfo`: the GPUs are named, no work is sent before Yes)
Seconds per test	0.1-2 in the window (command line 0.1-10), default 1
Memory test size	0 = auto (256 MiB, less on small cards), up to 4096 MiB; a size above a quarter of the video memory or of the PC's memory is lowered to that, and the result says so

Safety. It asks before it starts, naming the GPU, the time (about 15 seconds at the defaults) and that the GPU works at full load; an unstable GPU overclock or undervolt can crash the PC or the display driver under that load. The work is cut into pieces of about 25 ms of GPU time, one at a time, each waited for with a 5 s watchdog, so the desktop stays usable. A lost or stuck GPU ends the run cleanly with the results so far. Stop ends it at once. SRS never changes clocks, voltages or fans. Vulkan overlay layers (such as MangoHud) are kept out of the run.

Healthy / typical values. Not measured by SRS on a real GPU yet (only on the software renderer); as a rough guide from the makers' published figures, a current graphics card: FP32 in the tens of thousands of GFLOPS, video memory several hundred GB/s, upload and download 10-25 GB/s on PCIe 4.0 x16 (about half on PCIe 3.0), dispatch latency 5-30 µs. An integrated GPU: FP32 a few hundred to a few thousand GFLOPS, and the memory rows measure the system RAM it shares. The software renderer (lavapipe, which runs on the CPU) is far slower; it is there for testing.

Unusual values and what they suggest.

- FP64 at 1/16 to 1/64 of FP32: normal for gaming GPUs; workstation and data-centre cards are faster.
- Upload / download far below the card's link: the card is in a slot with fewer PCIe lanes or a lower PCIe generation, or the link has dropped to a slower speed (see the PCI items).
- Much lower numbers than last time: heat (the GPU slows down when hot), a power-saving profile, another program using the GPU, or a driver change. Compare only results from alike SRS builds and the same driver version.
- WRONG RESULT: the GPU computed something wrong - usually too much overclock or undervolt, too much heat or a faulty card. Run GPU Burnin to check.

Where Linux gets it. Built in: the Vulkan loader (`libvulkan.so.1`) is opened at run time with `dlopen`, so SRS does not need Vulkan to build or start; without it, or without a Vulkan driver, the window says "not available". The compute programs (shaders) are pre-compiled into SRS. Stored in `~/.cache/srs/bench-gpu.txt`; each run is added to the Benchmark History. Only one benchmark runs at a time in SRS, and none runs while Stress & Burnin runs.

GPU Burnin

`gpu-burnin` - Linux data

In plain words. Since 2.3.1 run from Stress & Burnin (tick **GPU Burnin (built-in)**; untick CPU load for the GPU alone), where it runs as its own process. A stress test for the graphics card, the GPU counterpart of the CPU load. It keeps the GPU fully busy with large matrix multiplications for as long as you choose and checks every result: the first multiplication is the reference and every later one must match it exactly. **0 errors = PASS.** Use it to check a GPU overclock or undervolt, the cooling and the power supply. Inspired by gpu-burn (Ville Timonen); the code is SRS's own.

Back ends. The same test can run through four ways of talking to the GPU:

Back end	What it is
Vulkan	the graphics and compute interface every modern GPU driver on Linux offers (AMD, Intel, NVIDIA); the default
ROCm / HIP	AMD's compute platform (needs ROCm installed)
CUDA	NVIDIA's compute platform (needs the NVIDIA driver); "CUDA (ZLUDA)" when the CUDA library is really ZLUDA running on an AMD GPU
OpenCL	the older cross-vendor compute interface; each installed OpenCL driver (ROCm, Mesa Rusticl, NVIDIA, PoCL ...) is its own entry
Auto	Vulkan when it has a GPU, else the first back end that has one

All four are loaded at run time: a back end that is not installed says "not installed" with a one-line hint, and the others still work. CUDA's own kernels are built into SRS (pre-compiled to PTX, NVIDIA's portable GPU code, since 2.3), so CUDA needs only its driver library - also with ZLUDA, which has no runtime compiler; NVRTC (the NVIDIA CUDA toolkit's runtime compiler) is used only if the driver refuses the built-in kernels. HIP needs its runtime compiler (hipRTC) for both the own-kernel and the matrix-library (hipBLAS) rows, because every result is checked with one of SRS's own kernels; without it those rows say so. OpenCL needs only its driver (the matrix library is CLBlast).

The matrix multiply (2.4). GPU Burnin uses a register-blocked kernel on every back end: each group of threads works on a 64 x 64 tile and each thread on 4 x 4 results, with the same summation order as before, so the results are checked the same way. ROCm's compilers turned the older 16 x 16 tiled kernel into code that reached only about 3.4 TFLOPS FP32 on a Radeon RX 9070 XT; with the new one ROCm /

HIP reached about 13.0 and OpenCL about 13.4 TFLOPS on that card, 0 errors - the card is loaded much harder. If the new kernel cannot be used, GPU Burnin falls back to the older one (on HIP then to hipBLAS). The result's **Kernel** line says which ran. `SRS_GPUBURN_SIMPLE=1` forces the older kernel, for comparing. The GPU Compute comparison keeps the older kernel on every back end, so its numbers stay comparable with 2.3.

The optional NVIDIA CUDA toolkit: **Options -> NVIDIA CUDA toolkit for GPU tests...** or `sysreport-search cuda-toolkit` (see the User Guide, 9.20).

What you see. While it runs, per GPU: the time, the **errors** (green 0, red otherwise), GFLOPS now and on average, multiplications per second and the matrix size, the **edge**, **hotspot / junction** and **memory temperatures**, **power**, **GPU clock** and **fan speed** with the highest values ("not available" when the card or driver does not report one). At the end: PASS / FAIL / Stopped / Ended early, errors found, multiplications, average speed, the highest temperatures, power, clock and fan, the back end, driver / runtime, matrix size and precision.

Setting	Choices
Back end	Auto (Vulkan), Vulkan, ROCm / HIP, CUDA, OpenCL (those not installed say so)
GPU	The first GPU, All GPUs, or one from the list (read through Vulkan when you open it, like `vulkaninfo`; no runtime is started and no work is sent. For HIP, CUDA and OpenCL the runtime starts after Yes and finds the picked GPU by its PCI address)
Precision	FP32 (default) or FP64
Seconds	1 to 86,400 (24 hours), default 60
Stop at °C	50-100, default 100; up to 110 after a one-time risk question (2.4; command line: `--accept-high-limit`)

Safety.

- It asks before it starts (No is the default answer), saying that the GPU draws a lot of power and gets hot, that a bad overclock or undervolt can crash the PC or the display driver, and that a weak power supply or poor cooling can make the PC shut down.
- It **ends by itself** when the GPU reaches the temperature limit (hotspot / junction when the card reports one, else edge), on any driver error, when the GPU stops answering ("device lost"), or when one piece of work takes more than 2 s (each piece should take about 20 ms). Stop works at any time. The work is cut into small pieces so the desktop stays usable.
- It **never changes clocks, voltages or fans**.
- It does not start while Stress & Burnin or another benchmark runs, and Stress & Burnin does not start while it runs.
- It never burns a software renderer (llvmpipe, PoCL on the CPU) as if it were a GPU: Auto, The first GPU and All GPUs skip them, and picking one is refused.
- A card that reports no temperature to SRS cannot be stopped at the limit: the live readout and the result say "no temperature sensor: limit not active".
- A driver call that never comes back (a hung driver) is waited for 15 s after the end; then the run ends with "the GPU driver did not come back", the GPU is left alone, and SRS must be restarted before the next GPU test. With All GPUs, a GPU that cannot start is marked and the others go on.

Healthy / typical values. 0 errors, always. Temperatures: most graphics cards slow down by themselves before their limit (AMD hotspot about 100-110 °C, NVIDIA about 83-90 °C edge); a card that sits at its limit needs better cooling or airflow. Speeds depend on the card; compare only with your own earlier runs on the same driver.

Unusual values and what they suggest.

- Any errors: too much overclock or undervolt, too high a temperature, a weak or overloaded power supply, or a faulty card or video memory. Undo the overclock / undervolt and run again.
- "COULD NOT START" (All GPUs): that card could not open or cannot do the precision; the other cards ran and their result stands.
- A message "a GPU call did not return earlier; restart SRS": a driver call hung in an earlier run; restart SRS (or the PC) before the next GPU test.
- Ended early on a driver error or "device lost": the GPU crashed or was reset by the driver - usually an unstable overclock or undervolt, or the power supply.
- GFLOPS that fall over a run while the temperature rises: the card slows down when hot.
- "not available" sensors: the driver does not report them (NVIDIA's own driver reports no temperature to Linux's hwmon, for example), or a software renderer.

Where Linux gets it. Built in: Vulkan (`libvulkan.so.1`), ROCm / HIP (`libamdhip64`, `libhiprtc`, `libhipblas`), CUDA (`libcuda.so.1`, `libcublas`; `libnvrtc` only as a fallback for the built-in PTX kernels) and OpenCL (`libOpenCL.so.1`, `libclblast`) are opened at run time with `dlopen`. The sensors come from the card's hwmon folder in `/sys/bus/pci/devices/<address>/hwmon` (amdgpu: all of them; i915, xe and nouveau: what they offer). Stored in `~/.cache/srs/bench-gpuburn.txt`; each run is added to the Benchmark History.

GPU Compute: Vulkan vs HIP vs CUDA vs OpenCL

`gpu-compare` - Linux data

In plain words. The same pieces of GPU work, with the same sizes, run through every compute back end you have installed (Vulkan, ROCm / HIP, CUDA, OpenCL) on the **same** graphics card, and shown side by side. It answers "which way of programming this GPU is fastest here, and how much does the vendor's own library add?". Every result is checked; a wrong one is shown as an error, never as a speed.

What you see. Rows = tests, columns = back ends. The fastest in each row has a `*`; every other column also shows its speed as a percentage of Vulkan (above 100% = faster than Vulkan; for the launch round trip, above 100% = quicker).

Row	What it measures	Unit
FP32 / FP16 / FP64 matrix multiply, own kernel	SRS's plain tiled matrix multiply, the same work on every back end	GFLOPS

Row	What it measures	Unit
FP32 / FP16 / FP64 matrix multiply, vendor library	the maker's tuned library: hipBLAS (ROCm), cuBLAS (CUDA), CLBlast (OpenCL); Vulkan has none ("not available")	GFLOPS
FP32 / FP64 multiply-add chain (peak)	long chains of multiply-adds: close to the card's peak	GFLOPS
Video memory copy	copying inside the card's memory	GB/s
Upload / Download	system memory to the card and back	GB/s
Kernel launch round trip	how long an empty piece of work takes to start and finish (lower is better)	µs

Columns: "Vulkan", "ROCm / HIP", "CUDA" - or **"CUDA (ZLUDA)"** when the CUDA library is really ZLUDA translating CUDA calls to an AMD GPU - and one column per OpenCL driver, such as **"OpenCL (ROCm)"** and **"OpenCL (Rusticl)"** on the same card; CPU OpenCL drivers (PoCL) are marked CPU. A back end that is not installed shows "not installed" with a one-line hint. Below: each column's device and driver / runtime version, whether a vendor library was found, the sizes and the **Build** line.

Setting	Choices
GPU	the Vulkan device that names the card (Default = the first discrete GPU); the other back ends use the card at the same PCI address
Back ends	Vulkan, ROCm / HIP, CUDA, OpenCL (only installed ones can be ticked)
Seconds per test	0.1-2 in the window (command line 0.1-10), default 1

Safety. It asks first (the back ends, the GPU, the time, full GPU load, the overclock / undervolt warning). HIP, CUDA and OpenCL start their runtimes only after that Yes. Work is cut into short pieces, each with a watchdog. A GPU that hangs or stops answering ends the run: no later back end touches that GPU again, and the tests that did not run say **not run** (the line under the table says why). Stop works. SRS never changes clocks, voltages or fans.

Healthy / typical values. Measured by the author on real hardware once, for CUDA through ZLUDA only: a Radeon RX 9070 XT with ZLUDA (preview build, no NVRTC), 2026-10-29 - own-kernel FP32 matrix multiply 5377 GFLOPS, cuBLAS FP32 16392, cuBLAS FP16 77046, own-kernel FP64 860; cuBLAS FP64 "not available". The other back ends were measured only on the software renderer. In general the vendor libraries are much faster than a plain kernel for matrix multiply, and the own-kernel rows show how well each driver compiles the same simple code; a healthy result has every row filled in and no errors.

Unusual values and what they suggest.

- One back end far behind the others on the same card: an old or debug runtime, or a driver that does not support that card well.
- CUDA (ZLUDA) rows: since 2.3 they run from SRS's built-in kernels (PTX); the CUDA column's software line says "own kernels built in (PTX), no NVRTC". "not available ... the CUDA driver refused the built-in kernels ... see Options > NVIDIA CUDA toolkit": that driver or ZLUDA version does not

take them, and NVRTC is not installed - install NVRTC (User Guide 9.20) and please report the driver or ZLUDA version.
- "FP64 matrix multiply, vendor library: not available: cuBLAS on this driver does not support FP64 matrix multiply" with ZLUDA: ZLUDA's cuBLAS has no FP64 matrix multiply (it answers "not supported", cuBLAS error 15). Expected, not a fault; every other CUDA row runs. A library that says "not supported" only for a bigger size during the run likewise marks only that row "not available".
- "not available" in the HIP rows: without libhiprtc the own kernels cannot be built, and the library rows need one of them for the error check, so they say "needs the runtime compiler ... for the error check" and are not run. The notes under the table say why each test is "not available".
- FP64 rows "not available: this OpenCL driver has no FP64 (double precision)": that OpenCL driver (often Mesa's Rusticl) reports no double-precision support, so the FP64 own-kernel, CLBlast and multiply-add rows are not tried. A kernel that does not build always shows the OpenCL error code.
- FP16 library rows "not available" on OpenCL: the driver has no `cl_khr_fp16`.

Where Linux gets it. Built in: all four back ends and the libraries are opened at run time with `dlopen` (see GPU Burnin); ZLUDA is recognised by marker strings in the CUDA library file. Stored in `~/.cache/srs/bench-gpucompare.txt`; each run is added to the Benchmark History. Compare results only between alike SRS builds and the same driver and runtime versions.

SRS Pi

`srs-pi` - Linux data

In plain words. SRS works out pi (3.14159...) to many decimal places - from 128 thousand up to 32 million - and then checks every single digit. It is a classic CPU test: the time tells you how fast one core is at heavy number work, and a wrong digit means the computer made a mistake, which usually points to an unstable overclock, too little voltage or bad memory.

What you see.

Line	Meaning	Why it matters
Digits	How many decimal places were worked out (after "3."). 1 million = 1,000,000. With **SuperPi sizes** ticked, powers of two: 1M = 1,048,576, marked "(SuperPi size 1M)".	Bigger takes longer and uses more memory.
Method	Chudnovsky series (fast) or Gauss-Legendre / AGM (the method SuperPi uses).	Only compare runs made with the same method.
Threads	1 = single-thread score. Chudnovsky can split its work over more threads; AGM always uses one.	
Time	Seconds from start to the last digit (the check is not counted).	Lower is faster.
Check	"every digit right" when the SHA-256 checksum of all the digits matches the true digits.	Anything else means a wrong result: do not trust that PC's stability.

Line	Meaning	Why it matters
Last 20 digits	The end of the result, for a quick look.	
Build	Compiler and CPU code level the program was built for.	Results only compare between alike builds.

If never run, the item shows `Status Not run yet` and the command line.

Healthy / typical values. Ryzen 7 7800X3D, package build (generic x86-64, -O2), one thread: 128 thousand digits 0.23 s, 1 million 3.1 s, 2 million 7.7 s, 4 million 17.7 s, 8 million 39.6 s. 1 million on 4 threads: 1.4 s. Gauss-Legendre is about four times slower than Chudnovsky (1 million: 13.2 s). A build made for this CPU (`-march=native -O3`) did 1 million in 2.1 s.

Unusual values and what they suggest.

- Check says "wrong": the hardware made an error. Undo the last overclock or voltage change and run it again; a wrong result at stock settings points to memory (run a memory tester).
- Much slower than usual: another program is busy, the CPU is hot and throttling, or a power-saving profile is active.
- "Not enough memory": 32 million digits needs about 1-2 GB of free RAM (more with more threads).

Safety. It asks before it starts, with the expected time, the threads and the memory it needs. From 8 million digits, or with all threads, it warns that an unstable overclock or undervolt can crash the PC under this load, and **No** is the default answer. Stop (or closing the window) ends it at once. On the command line `sysreport-search pi` asks the same; `--yes` starts at once.

Where Linux gets it. Built in (no outside program): big numbers in base 10^8, multiplied with a number-theoretic transform; Newton steps for division and square roots. The true digits' SHA-256 checksums for 128K, 1M, 2M, 4M, 8M, 16M and 32M digits are built into SRS (computed once with the independent mpmath library). Stored in `~/.cache/srs/bench-pi.txt`; each run is added to the Benchmark History.

Background. SuperPi (1995) made "pi to 1 million digits" a famous overclocking test. SRS Pi is SRS's own program with its own name and timings; its times are **not** SuperPi times. SuperPi's "1M" is 2^20 = 1,048,576 digits; SRS's 1 million is exactly 1,000,000. Tick **SuperPi sizes** to work out the same digit counts SuperPi does (16K = 16,384 up to 32M = 33,554,432), so at least the amount of work matches; each of these sizes has its own built-in checksum too. 1M = 1,048,576 digits took 4.2 s on one thread on the 7800X3D (a little more than 5% over 1,000,000 digits, because the multiplications step up to the next power-of-two size).

Crypto Benchmark

`crypto-bench` - Linux data

In plain words. How fast the CPU can encrypt data (AES-128, used by disk encryption, VPNs and HTTPS) and make fingerprints of data (SHA-256, used by package checks, backups and git). Each is measured twice: with the CPU's own crypto instructions (AES-NI, SHA-NI) and in plain C without them, so you can see how much the hardware helps.

What you see. A table in MB/s (1 MB = 1,000,000 bytes) with one column for one thread and one for the chosen number of threads (default: all), and a Check line.

Row	Meaning
AES-128-CTR, AES-NI	AES-128 in counter mode using the AES instructions, eight blocks at a time.
AES-128-CTR, plain C	The same work with lookup tables in ordinary C code.
SHA-256, SHA-NI	SHA-256 using the SHA extensions.
SHA-256, plain C	SHA-256 in ordinary C code.

A row says "not available on this CPU" when the CPU lacks those instructions.

Healthy / typical values. Ryzen 7 7800X3D, one thread: AES-NI about 15,000 MB/s, plain C about 180 MB/s; SHA-NI about 2,100 MB/s, plain C about 430 MB/s. With four threads the numbers grow about four times. For comparison, `openssl speed` on the same core: AES-128-CTR about 12,600 MB/s, SHA-256 about 2,400 MB/s.

Unusual values and what they suggest. Check not "pass": the CPU gave a wrong answer - treat it like a failed stability test. AES-NI rows missing on a modern CPU: the instructions may be switched off in the BIOS or hidden by a virtual machine.

Safety. It asks before it starts. With all threads (the default) every CPU thread is 100% busy, so it warns about unstable overclocks and **No** is the default answer. On the command line `sysreport-search crypto` asks the same; `--yes` starts at once.

Where Linux gets it. Built in, written from the standards FIPS 197 (AES) and FIPS 180-4 (SHA-256). Before every run the official test vectors are checked, and the hardware results are compared with the plain C results. Stored in `~/.cache/srs/bench-crypto.txt`. Runs about 8 seconds at the default of 1 second per measurement.

Prime Sieve

`prime-sieve` - Linux data

In plain words. Counts all the prime numbers up to a limit (100 million to 100 billion) and checks the count against the known answer. It is simple integer and cache work that splits well over many threads.

What you see. Limit, Threads, Time, Speed (million numbers per second), Primes found, and a Check line ("right" when the count matches the known value, for example 455,052,511 primes below 10 billion).

Healthy / typical values. Ryzen 7 7800X3D, up to 10 billion: 4.5 s on one thread, 1.15 s on four threads. The specialised `primesieve` program does the one-thread run in 0.75 s: SRS uses a plain, easy-to-read sieve, so compare SRS with SRS.

Safety. It asks before it starts. From 10 billion, or with all threads (the defaults), it warns about unstable overclocks and **No** is the default answer. Stop ends it at once. On the command line `sysreport-search prime` asks the same; `--yes` starts at once.

Where Linux gets it. Built in: a segmented sieve of Eratosthenes over the odd numbers, in 128 KB pieces that fit in the L2 cache, shared between threads. Stored in `~/.cache/srs/bench-prime.txt`.

Benchmark History

`bench-history` - Linux data

In plain words. A list of every SRS Pi, Crypto and Prime Sieve run, newest first, so you can compare before and after a change (BIOS setting, kernel, cooler).

What you see. When, Test, Settings (digits, threads, method), Result, and Build. Results in red were wrong.

Where Linux gets it. `~/.cache/srs/bench-history.tsv`, a tab-separated text file (one line per run) that any spreadsheet opens. Only compare lines with the same settings and the same Build: a package built for every x86-64 CPU can be much slower than one built for this CPU.

Lookup

`lookup` - Linux data

In plain words. It lists the name databases SRS uses to turn ID numbers into names (PCI, USB, monitor maker and network card maker), with their dates and versions. It also shows the SRS program itself.

What you see.

Column	Meaning	Why it matters
Description	What the file is for: PCI Device Names, USB Device Names, PNP / Monitor Names, MAC Vendor Names, SRS Program, SRS Kernel Driver.	
File Specification	Full path of the file, for example `/usr/share/hwdata/pci.ids`.	Where to look if names are missing.
Current Identification	"Date ... Version ... (N)". N is the number of record lines in the file. When the file has no version line, the file size in bytes is shown instead, and Date is `?`. For SRS Program it is the SRS version.	An old file means new hardware shows as "Unknown".
Code Page	Text encoding (UTF-8).	
WEB Download Location	Where the newest copy of each file is published.	

Since 2.4 the PCI, USB and PNP rows show the copy SRS really uses: the one **Update hardware lists** (Tools page, or `sysreport-search update-lists`) downloaded into `~/.local/share/srs/lists`, or the system's, whichever is newer (by the list's own version; by entry count for `pnp.ids`). The WEB Download Location is where Update hardware lists fetches it. Under the table a note says the page itself never downloads (SIV's WEB UPD / Latest are not reproduced): Update hardware lists is the one way, and only when you ask. The SRS Kernel Driver row says `none`: SRS loads no kernel driver of its own, and root-only data comes from srs-root.

Healthy / typical values. On the author's PC: pci.ids Version 2026.10.01 (42452 records), usb.ids Version 2026.06.26 (25615), pnp.ids 63479 bytes, oui.txt 6321491 bytes. SRS Program `/usr/bin/sysreport-search`, V2.4.0.

Unusual values and what they suggest.

- `not found`: the hwdata package is missing. Install `hwdata` (Arch) or `hwdata` / `pciutils` / `usbutils` (other distros).
- A pci.ids or usb.ids date more than a year old with very new hardware: run **Update hardware lists** (Tools page), or update the hwdata package, to get names for new devices.

Where Linux gets it. `/usr/share/hwdata/pci.ids`, `usb.ids`, `pnp.ids` and `oui.txt`, or the newer copies in `~/.local/share/srs/lists`, plus `/proc/self/exe` for the program path.

Ping

`ping` - Linux data

In plain words. It sends three small test packets to your router (the default gateway) and shows whether they came back and how fast. In SIV this is a Windows ping. SRS uses the Linux `ping` tool.

What you see.

- `Target` - the IPv4 default gateway, read from the routing table.
- The normal `ping` output: one line per reply with `icmp_seq` (packet number), `ttl` (time to live, how many more router hops the reply could have made) and `time` (round trip in ms).
- The summary: packets sent and received, `% packet loss`, and `rtt min/avg/max/mdev` (fastest, average, slowest, and how much the times vary).
- Error lines in red when the ping fails, `No IPv4 default gateway` when there is no route out, or `ping not available` when the tool is missing.

Healthy / typical values. On the author's wired PC, pinging a home router (for example 192.0.2.1): 3 sent, 3 received, 0% loss, average about 0.3 ms. Wired links to a router are usually below 1 ms. Wi-Fi is usually 1-10 ms.

Unusual values and what they suggest.

- Any packet loss to your own router: a bad cable or port, weak Wi-Fi, or an overloaded router.
- Times of tens of ms on a wired link: the router is busy, or the link fell back to a slow speed.
- 100 % loss but the internet works: the router is set to ignore ping. That is harmless.

Where Linux gets it. `/proc/net/route` for the gateway, then `ping -c 3 -i 0.2 -W 1 <gateway>`.

Configuration Find

`reg-find` - Linux data

In plain words. SIV searches the Windows Registry. Linux keeps its settings in text files instead, so SRS searches the text files in `/etc` (system settings) and `~/.config` (your own settings) for a word or pattern.

What you see.

- With no pattern: `Usage reg-find:PATTERN (regex; searches /etc and ~/.config text files)`.
- With a pattern: one line per hit, `path:line-number` followed by the matching line.
- At the end: `N Matches`, `N Matches (limit 200)` when it stopped at 200 hits, or `No Matches for PATTERN`. A broken pattern gives `Match String [...] is Invalid:` and the reason.

Run it from a report with `-save=[reg-find:PATTERN]`. Example: `-save=[reg-find:HandleLidSwitch]` finds where lid-close behaviour is set.

Healthy / typical values. On the author's PC, `^HandleLidSwitch` gives `No Matches`, because the setting only appears commented out (`#HandleLidSwitch=...`) in `/etc/systemd/logind.conf`. Leave out the `^` to see commented lines too.

Unusual values and what they suggest. Fewer hits than expected: some files under `/etc` can only be read by root and are skipped without a message. Use `sudo grep` for those.

Where Linux gets it. A walk of `/etc` and `~/.config`, up to 4 folders deep. Only regular files up to 256 KB. Files whose name contains "shadow" (password hashes) are skipped. Case is ignored.

Background. A regex (regular expression) is a search pattern. `.` means any character, `^` means "start of line", `a|b` means "a or b".

Device ID Lookup

`id-lookup` - Linux data

In plain words. Type a device's ID numbers and SRS tells you the maker and the device name, from the same databases Linux uses.

What you see.

- With no ID: `Usage id-lookup:VVVV:DDDD[:SSSS:ssss][:RR] or id-lookup:usb:VVVV:PPPP`.
- A list of keys, from the most exact to the least exact. Each key shows its database entry, or `(no entry)`. A `<` marks the key that matched.
- `Vendor` - the maker of the chip (from VVVV).
- `Subsystem` - the maker of the card or board (from SSSS). Only shown for 4-part PCI IDs.
- `Stock` - the plain chip name (VVVV:DDDD), as the chip maker names it.
- `Resolved` - the best name found (the `<` row), or `No Matches`.
- Errors: `bad ID ... expected VVVV[:DDDD[:SSSS:ssss]][:RR]` when the format is wrong, or `cannot find the .ids file`.

ID parts: VVVV = vendor, DDDD = device, SSSS:ssss = subsystem vendor and subsystem device (the card maker's own ID), RR = revision. All are hexadecimal. For USB: VVVV = vendor, PPPP = product.

The GUI window (Menu -> Device ID Lookup..., or Ctrl+L) adds a PCI / USB choice and a three-way mode box. **Tree** does a fast binary search of a sorted table. **Scan** reads the .ids file from top to bottom. **Info** asks the system device database (systemd-hwdb) instead. A footer shows the number of records and the load and lookup times.

Healthy / typical values. On the author's PC, `1002:7550:1EAE:8811:C0` (the graphics card) gives no entry for the three exact keys and matches `1002:7550` = "Navi 48 [Radeon RX 9070/9070 XT/9070 GRE]". Vendor = Advanced Micro Devices, Inc. [AMD/ATI], Subsystem = XFX Limited. `usb:1d6b:0002` resolves to "2.0 root hub" by the Linux Foundation.

Unusual values and what they suggest. Only the vendor matches, or nothing at all: the device is newer than your pci.ids / usb.ids file (see Lookup), or the ID was typed wrongly. Info mode can sometimes still name it, because systemd-hwdb is updated separately.

Where Linux gets it. `/usr/share/hwdata/pci.ids` and `usb.ids`. Info mode uses systemd-hwdb.

Background. Every PCI and USB device reports a vendor ID and a device ID. These are the numbers Linux uses to choose a driver. The name is only a label from a community-kept list.

Find

`find` - Linux data

In plain words. It searches your newest saved system report (made by srs-sysreport) for a word or pattern and shows every matching line, grouped by section file.

What you see.

- With no pattern: `Usage find:PATTERN (regex over the newest ~/sysreports report)`.
- `Searching <report folder> for "PATTERN"`.
- For each file with hits: `== file - N Matches`, then each hit as `line-number: text`, with the matching part highlighted.
- `N Matches in X of Y sections`, or `No Matches for PATTERN`.
- `No report found under ~/sysreports - run srs-sysreport first` when there is no report yet.

From a report (`-save=[find:PATTERN]`) the search is case-sensitive and shows up to 20 hits per file. The GUI window (Menu -> Find in Report..., or Ctrl+F) adds: Ignore case, Literal (search for the exact text, not a regex), Highlight Matches, Max (hits shown per file, then "... N more (raise Max)"), Report (which report folder; Browse... and Latest buttons), Next / Prev buttons and F3 / Shift+F3 to jump between hits, Colour (the highlight colour), and Lookup (opens Device ID Lookup).

Healthy / typical values. On the author's PC, `find:Tctl` found 1 match in `sensors.txt` (line 7, `Tctl: +59.6°C`), "1 Matches in 1 of 16 sections".

Unusual values and what they suggest. The report folder date is old: the search only looks at the newest `sysreport-*` folder. Run srs-sysreport again for fresh data, or pick a folder with Browse.

Where Linux gets it. The newest `~/sysreports/sysreport-*` folder: every `.txt` and `.html` file except `index.html`.

ALL Devices

`all-dev` - Linux data

In plain words. Every PCI and USB device in the PC, each one turned from its ID numbers into a maker and device name. Devices the name lists do not know are marked.

What you see. One block per bus, PCI first, then USB.

- Block header: `[PCI] 54 devices, 42,228 records /usr/share/hwdata/pci.ids` - how many devices were found, how many entries the name list has, and which file was used.

Column	Meaning	Why it matters
Slot	PCI address (domain🚌device.function, e.g. `0000:03:00.0`) or USB port path (e.g. `1-2.4`, or `usb1` for a root hub).	Where the device sits.
ID	The full ID: PCI `vendor:device:subvendor:subdevice:revision`, USB `vendor:product`.	The numbers to search for online.
Matched	The most exact part of the ID that has a name in the list, or `-`.	Shows how well the device is known.
Vendor	The chip maker, or `Unknown Vendor`.	
Resolved	The device name. `Unknown Device` (red) when only the vendor is known. `[subsystem: ...]` is added when the card maker is known but the exact card is not in the list.	

At the end: `N device(s) resolved only to vendor or not at all`. Use `all-dev:pci` or `all-dev:usb` to show one bus only.

Healthy / typical values. On the author's PC: 54 PCI devices, all named. For example `0000:03:00.0` = Navi 48 [Radeon RX 9070/9070 XT/9070 GRE] [subsystem: XFX Limited], and the Samsung SSD 980 PRO matched on all four ID parts. 40 USB devices, with 16 known only by vendor or not at all. That is common for USB: many hubs, receivers and controllers are simply not in usb.ids.

Unusual values and what they suggest.

- Many PCI devices "Unknown Device": your pci.ids is old (see Lookup).
- A device you expect is missing from the list: it is switched off in the BIOS, not seated, or (for USB) not powered or not detected. Check `dmesg` for errors.

Where Linux gets it. PCI devices from `/sys/bus/pci/devices`, USB devices from `/sys/bus/usb/devices`, names from `/usr/share/hwdata/pci.ids` and `usb.ids`.

Stress & Burnin (CPU and GPU)

`burnin` - Linux data

In plain words. A stress test (2.3.1: CPU Burnin, CPU Stress and GPU Burnin in one): it makes the CPUs you choose 100 % busy for a set time (**Timed**, Pass or Fail) or until Stop (**Until Stop**), and can load the graphics card at the same time with the built-in **GPU Burnin** (its own time, or **Sync to CPU stress time**) or a **Custom GPU command**. Use it to check that a new build, cooler, overclock or undervolt is stable. Use at your own risk: the first Start asks you to agree to the load test terms (user guide 9.0). The saved item only shows the current state and sensor readings. It never starts a run.

What you see in a report.

Field	Meaning
State	`Running`, or `Idle (start it from Benchmarks & Stress > Stress & Burnin)`.
Worker threads	How many CPUs can get a worker thread ("16 CPUs available").
Hottest CPU sensor	The hottest CPU temperature right now, with chip and label (k10temp, coretemp or zenpower).
Hottest SIO sensor	The hottest motherboard Super I/O sensor right now.
GPU	Graphics card load %, temperature (junction if available) and core clock.
Command line	`srs-gui burn [--threads N | --cpus 0,2] [--seconds S] [--mode same|swap] [--limit C] [--accept-high-limit]`.
Safety note	Never auto-starts; stops at the temperature limit (default 90 C) and the time cap.

What you see in Stress & Burnin (the Benchmarks & Stress page's first tab; Classic View: Menu -> Benchmarks & Stress -> Stress & Burnin (CPU / GPU)...). The CPU part:

- Settings: Burn Time (0-750, in minutes, or in hours if "Hours" is ticked; default 15 minutes), Cool Time (0-900 seconds idle between burns, default 60), Cycles (1-99 burn/cool rounds, default 1), Temperature limit °C (50-110, default 90, remembered; over 100 only after a one-time question, see below), **Same CPU (Whetstone)** or **Swap CPU (affinity test)**, the GPU part (GPU Burnin (built-

in) with Time and Sync to CPU stress time, GPU options; or a Custom GPU command), and one tick box per CPU with Select All / Deselect All. Start asks for confirmation first. The folded **GPU options** line shows what it holds (2.4), for example "GPU options: Auto (Vulkan), FP32, stop at 100 °C", so the GPU's own stop limit is never hidden.

- **Stop limits over 100 °C (2.4).** Both the CPU and the GPU limit go up to 110 °C. The first time either is set above 100, SRS asks once: "Stop limits over 100 °C: at your own risk" - above 100 °C many processors and graphics cards are already at or near their own maximum (often 89-110 °C), they slow themselves down, and with poor cooling, worn thermal paste or an overclock a fault becomes more likely; the chip's own protection stays on and SRS never changes clocks, voltages or fans. **Yes, I accept the risk** is remembered (`Accepted High Temperature Limits` in `SRSDEF.ini`); **No** puts both limits back to 100. A limit over 100 in `SRSDEF.ini` without that yes loads as 100. On the command line `burn` and `gpu-burn` refuse a `--limit` over 100 unless `--accept-high-limit` is given.
- Status line: Idle / Burning / Cooling / Pass / Fail / Stopped, with Elapsed and Remaining time (hh:mm:ss) and the reason text (for example "Pass - Finished").
- A sensor table with 7 rows (CPU Load, CPU Frequency, Hottest Core Temp, Hottest SIO Sensor Temp, GPU Load, GPU Temp, GPU Frequency) and 4 columns: **Current**, **Overall Average** (since start), **Rolling Average** (recent values count most) and **Maximum**.
- A thread table:

Column	Meaning
Thread	Worker number.
CPU	The CPU this worker was pinned to.
On CPU	The CPU it actually ran on during its last pass.
User / Kernel	How busy that CPU was with program work / kernel work, in %.
Clock	That CPU's current clock in MHz.
Whetstone	Score of the last pass, in MWIPS.
Average / Min / Max	Average, lowest and highest pass score so far.
Affinity	`ok`, `FAIL` (red), or `-` before the first pass ends.

- A graph of the last 10 minutes: green CPU load %, red hottest core °C, orange SIO °C, blue GPU °C, on a 0-100 scale.

Pass and Fail.

- **Fail** - the hottest CPU sensor reached the temperature limit ("Fail - k10temp Tctl 90.0 C reached the 90 C limit"), or a worker ran on the wrong CPU ("Fail - a worker thread ran on the wrong CPU (affinity / APIC check)"). The run stops at once.
- **Pass** - all cycles finished ("Pass - Finished"), or the overall time cap was reached.
- **Stopped** - you pressed Stop or closed the window.
- Command line `srs-gui burn`: exit code 0 = Pass, 1 = Fail, 130 = Stopped (Ctrl+C). It prints one status line per second and a thread table at the end. It also accepts `--cool S` and `--cycles N`.

Saved results. When a run ends (Pass, Fail or Stopped), SRS saves the window's summary - the status, the sensor table and the thread table, the same text the Copy button gives - to `~/.cache/srs/burnin-YYYYMMDD-HHMMSS.txt` (date and time the run ended), and the status line says where. Each run gets its own file, so you can compare runs before and after a change, for example with `diff` or `meld`. The command line `burn` saves its summary the same way. SRS never deletes these files; remove old ones by hand when you no longer need them.

Affinity ok / FAIL. After every pass each worker checks two things: (1) Linux says it is running on the CPU it was pinned to (`sched_getcpu`), and (2) the CPU's own ID number (APIC ID, read with the CPUID instruction) matches the one `/proc/cpuinfo` lists for that CPU. `ok` means both matched every time. `FAIL` means the thread was found somewhere else at least once. That should never happen, so it points to a scheduler problem, a CPU that went offline, or a firmware or topology fault. In **Swap** mode each worker moves to the next selected CPU before every pass. That tests that moving threads between cores works.

Healthy / typical values. On the author's 7800X3D at idle: Tctl 56.0°C, nct6799 CPUTIN 30.0°C, GPU 5 % load, 29.0°C, 1725 MHz. Under a full all-core load, a 7800X3D usually settles around 80-89°C. It is designed to hold near its 89°C limit (its Tjmax), so on this chip the default 90°C cut-off never triggers: the CPU slows itself down first. If you want the Fail cut-off to act, set the limit a few degrees below your own CPU's Tjmax. A good run shows steady Whetstone scores per thread (Min close to Max), all Affinity `ok`, and Pass.

Unusual values and what they suggest.

- Fail on temperature within a few minutes: cooler not seated well, pump not running, or not enough case airflow.
- Min far below Max on some threads: the CPU is throttling (heat or power limit). Watch CPU Frequency drop at the same time.
- A sudden reboot or freeze during the burn: the system is not stable. Undo the latest overclock, undervolt or memory change.
- Hottest SIO sensor reading very high (80°C or more) with a cool case: usually a disconnected sensor input. SRS already ignores AUXTIN, PECI, TSI and PCH inputs. You can turn a real one back on with `SIO Sensors=chip:LABEL` in `~/.config/srs/SRSDEF.ini`.

Where Linux gets it. Load: built-in Whetstone in one pinned thread per CPU. Sensors: `/proc/stat` (load), `/sys/devices/system/cpu/cpuN/cpufreq/scaling_cur_freq` (clock), `/sys/class/hwmon` (CPU and Super I/O temperatures), `/sys/class/drm/cardN/device` (`gpu_busy_percent`, `pp_dpm_sclk`, hwmon temperature). The GPU load command's output goes to `~/.cache/srs/gpu-load.log`. Finished runs are saved to `~/.cache/srs/burnin-YYYYMMDD-HHMMSS.txt`.

Background. Tctl is AMD's "control temperature". It is the value the CPU uses for fan control and throttling. A Super I/O (SIO) chip is the motherboard chip that reads fan speeds, voltages and board temperatures. A burn-in finds weak cooling and unstable settings now, at your desk, instead of in the middle of a game or a long job.

CPU Stress

`cpu-stress` - Linux data

In plain words. Since 2.3.1 this is the **Until Stop** mode of Stress & Burnin: the same load engine with no timer. The CPUs stay 100 % busy until you press Stop (or a safety limit hits). The old "CPU Stress" entries open Stress & Burnin set to Until Stop.

What you see. In a report: exactly the same lines as Stress & Burnin (State, Worker threads, sensors, GPU, Command line, Safety note). In the CPU Stress window (Menu -> Benchmarks & Stress -> CPU Stress...): the same sensor table, thread table, graph and controls as CPU Burnin, but without Burn Time, Cool Time and Cycles.

Pass and Fail. Stress has no "finished" point. It ends as **Stopped** when you press Stop, as **Fail** if the hottest CPU sensor reaches the temperature limit or an affinity check fails, or as **Pass** if the overall time cap (750 hours) is reached. Either way the summary is saved to `~/.cache/srs/burnin-YYYYMMDD-HHMMSS.txt`, as for Stress & Burnin.

Healthy / typical values. As for CPU Burnin: temperatures climb for a few minutes and then level off. Clocks stay steady.

Unusual values and what they suggest. Temperatures that keep rising after 10-15 minutes and never level off: the cooler is saturated (for example an AIO whose liquid keeps heating up). Improve airflow or the fan curve.

Where Linux gets it. The same sources as Stress & Burnin.

SRS (SIV items) (SRS Section Reference)

Back to the section index - Glossary

Where to find these items (SRS 2.0). In the main window they are on the **Tools** page (Alt+Backspace). Its tabs are About SRS and Help; **All N items** lists the rest. **Menu -> About SRS** opens About too. You can also type an item's name or id into the search box (Ctrl+F). The rest of this introduction describes the classic view (Menu -> Classic View), with its Menu and panel buttons.

This group holds the items that describe SRS itself rather than your hardware: its version and credits, help text, settings file, command-line options, program image, threads, and a set of small "buffer" snapshots of live system data. In SIV64X (System Information Viewer, by Ray Hinchliffe, RH Software) these items describe SIV's own program, driver and internal buffers. SRS keeps the same item names so a save file lines up with SIV's, and fills each one with the nearest Linux data. Open them from **Menu -> SRS (SIV items) -> item**. The About and Help panels have their own button row (DNS Names, Lookup, USB Dev, About, Help), and the SRS DLLs panel has the Lookup row (DNS Names, Signature, SRS DLLs, Lookup, ID Lookup). Almost everything here works as a normal user. Only the board serial number in SGB info and the SPD cache line in Fake SPDs use data from the root helper (srs-root); start it with **Menu -> Run as root**.

Contents

- About SRS `about`
- BSB info (battery/HID sensor buffer) `bsi`
- CPU group buffer `cgb`
- CSV logging `csv`
- CSV titles `csv-titles`
- Debug API `debug-api`
- Delta/cost settings `delta`
- DSB info (DIMM sensor buffer) `dsb`
- Emoji glyph test `emoji`
- Fake IMCs `fak-imcs`
- Fake MCHBAR `fak-mchbar`
- Fake PM tables `fak-pmts`
- Fake SPDs `fak-spds`
- Filter `filter`
- Final processing `final`
- GPU info buffer `gpu`
- Help `help`
- Highlight setup `highlight`
- Hotkeys `hot-keys`

- SIV image (EXE) `image-exe`
- SIV image (SYS driver) `image-sys`
- Image tree `image-tree`
- Initial processing `initial`
- Install `install`
- SIV languages `languages`
- Lock setup `lck-setup`
- Maker icons `makers`
- SIV manifest `manifest`
- Mapping cache `map-cache`
- Map/unmap test `map-unmap`
- Menus `menus`
- Mutex info `mutex`
- PMB info (power monitor buffer) `pmb`
- Colour RGB debug `rgb-debug`
- SDM info (storage device monitor) `sdm`
- Sequence setup `sequence`
- SGB end `sgb-end`
- SGB info (system/board buffer) `sgb-ini`
- Active SIV instances `siv-active`
- SIV autorun `siv-autorun`
- SIV DLLs `siv-dll`
- SIV driver `siv-drv`
- SIVDEF.ini `siv-ini`
- SIV location / window poke `siv-location`
- SIV qualifiers `siv-qualifiers`
- SIV qualifier list `siv-quals`
- SIV setup `siv-setup`
- SIV sizes `siv-sizes`
- SIV driver service `siv-srv`
- SIV usage history `siv-usage`
- SIV use / update `siv-use`
- SMB end `smb-end`
- SMB info `smb-ini`
- Lock states `states`
- STC info (thread contexts) `stc`
- Tune system `tune`
- Tuning `tuning`
- TZM info (thermal zone buffer) `tzm`
- Words `words`
- XXB internal buffers `xxb`

How to read these items

- Most items start with a title line: `srs-gui - <what> on \\<host> - <OS>`. On the author's PC that reads `srs-gui - Help on \\MYPC - CachyOS 7.3.0-273-tkg-bore-llvm`.

- `[=]` is a ticked box and `[_]` an empty one. A few items use `|X|` and `|_|` the same way.
- SRS is read-only. None of these items change a setting, write a register or open a device.
- SRS is one program image. Run with no arguments it is the window (srs-gui). Run with arguments it is the command line (sysreport-search). So "this process" in the items below is whichever one made the report.

About SRS

`about` - Linux data

In plain words. Who made this program, which version you have, what it is running on, and which name-database files it uses.

What you see.

Field	What it means	Why it matters
Title line	`srs-gui - System Report Search V<version> <host>::<user>`	Quick check of version and machine.
Program	One image works as the GUI (no arguments) and the CLI (with arguments).	Explains why reports name `sysreport-search`.
Version	Program version, e.g. `V2.4.0`.	Quote this in bug reports.
Program Build	File format, compiler and C runtime the program was built with.	Tells a self-built copy from a packaged one.
Built	Date and time the program was compiled.	Shows how old your copy is.
Image	Full path and size of the running program file.	Shows which copy ran, if you have several.
Running on	Kernel name, release, build string and machine type (from `uname`).	The exact kernel the data came from.
Operating System	Distribution name and kernel.	Context for every other item.
Machine	Board maker, model and BIOS version/date.	Context; BIOS updates change many values.
Processor	CPU brand string.	Context.
PCI Device Names / USB Device Names / PnP / Monitor Names	Path of each name database, then its date and version (from the file header) or the file date.	Old files mean new devices show as "unknown".
Settings	Path of the settings file SRSDEF.ini.	Where your choices are kept.

Field	What it means	Why it matters
Live data	The kinds of source SRS reads.	Reminds you some data needs the root helper.
Created by / Licensed under / Home page	The author (Eugene Flanagin), the licence (GPL-2.0-or-later) and the GitHub page.	Who made it and where to get it.
Why SRS / Credit / Disclaimer	Origin, credit to SIV's author and use-at-your-own-risk note.	Credit information.

Healthy / typical values. On the author's PC: `V2.4.0`, built with gcc 16.2.1 and glibc, image `/usr/bin/sysreport-search (1.38MB)`, Linux 7.3.0 x86_64 on an ASUS TUF GAMING X870-PRO WIFI7 W NEO (BIOS 1805), AMD Ryzen 7 7800X3D. pci.ids version 2026.10.01, usb.ids 2026.06.26. pnp.ids has no version header, so only its file date is shown.

Unusual values and what they suggest.

- A database line says `not found`: the hwdata package is missing, so device names cannot be resolved.
- A database date that is a year or more old: update your distribution's hwdata package.

Where Linux gets it. Built-in version strings, `uname()`, `/etc/os-release`, `/sys/class/dmi/id`, and the header lines of `/usr/share/hwdata/*.ids`.

Background. The `.ids` files are community lists that turn numeric device IDs (like `1002:7550`) into names (like an AMD Radeon card). SRS does not ship its own copy; it uses the system's.

BSB info (battery/HID sensor buffer)

`bsi` - Linux data

In plain words. A list of the batteries and power supplies Linux knows about, plus a count of HID (Human Interface Device: keyboards, mice, game pads) devices.

What you see. One row per entry in `/sys/class/power_supply`:

Column	What it means	Why it matters
#	Row number.	-
Device Name	Kernel name of the supply, e.g. `BAT0`, `AC`, or a wireless controller battery.	Tells you which device it is.
Type	`Battery`, `Mains`, `USB` and so on.	Mains = wall power; Battery = something that drains.

Column	What it means	Why it matters
Status	`Charging`, `Discharging`, `Full`, `Not charging`.	Shows if the battery is being charged.
Capacity	Charge left, in percent.	Low values mean recharge soon.
Model	Model name if the driver reports one.	Identifies the battery pack.

Below the rows: `act N num N` (how many supplies were found) and a count of HID devices registered in `/sys/bus/hid/devices`. SRS only counts them; it never opens them. This panel refreshes every 2 seconds.

Healthy / typical values. A desktop often has no battery at all, or only the batteries of wireless game pads and mice. On the author's PC: one entry, a Nintendo Switch controller battery, type Battery, status Discharging; 15 HID devices. A laptop shows `BAT0` (Battery) and `AC` or `ADP1` (Mains).

Unusual values and what they suggest.

- Capacity or Model blank: the driver does not report it. Common for game pads.
- A laptop with no `BAT0`: battery not detected, or the ACPI battery driver is not loaded.
- `Not charging` at less than 100% on a laptop: often a charge limit set in firmware or by a vendor tool.

Where Linux gets it. `/sys/class/power_supply/*/{type,status,capacity,model_name}` and the entries in `/sys/bus/hid/devices`.

CPU group buffer

`cgb` - Linux data

In plain words. Shows that all your CPUs belong to one group, and which CPUs are online.

What you see.

Column	What it means	Why it matters
CGN	CPU group number (always 0 on Linux), with a tick box.	Windows splits more than 64 CPUs into groups; Linux does not.
Num	How many logical CPUs are online.	Should match your thread count.
Affinity	A 64-bit hex mask, one bit per online CPU.	`FFFF` = CPUs 0-15.
Members	`[CPU-0 to CPU-N]`, the highest online CPU number.	Quick range check.
Online / Possible	The kernel's online list and possible list.	Different lists mean some CPUs are switched off.

Healthy / typical values. On the author's 7800X3D (8 cores, 16 threads): `0 16 000000000000FFFF [CPU-0 to CPU-15] 0-15 / 0-15`.

Unusual values and what they suggest.

- Online smaller than Possible: CPUs were taken offline (for example with `nosmt`, `maxcpus=` on the kernel command line, or by writing 0 to a CPU's `online` file).
- Possible much bigger than your real CPU count: firmware reports spare hot-plug slots. Harmless.

Where Linux gets it. `/sys/devices/system/cpu/online`, `/sys/devices/system/cpu/possible` and `/sys/devices/system/cpu/cpuN/online`.

Background. A *logical CPU* is one hardware thread. With SMT (Simultaneous Multi-Threading) each core runs two, so 8 cores show as 16 CPUs. An *affinity mask* is a bit pattern saying which CPUs a task may run on.

CSV logging

`csv` - Linux data

In plain words. SIV can log sensor data to a CSV (Comma-Separated Values) file. SRS has no separate CSV logger; this item tells you what to use instead.

What you see. Fixed lines: CSV logging (not separate), Save (the `-save` syntax), Example (`srs-gui '-save=[sensors][cpu]=/tmp/log.txt'`), Tabular data (columns are padded with spaces at the panel tab stops), and Environmental log (SIV's `-ENVLOG` / `-EXCEL` sensor log has no equivalent; run `-save` on `[sensors]` instead).

Healthy / typical values. Always the same text.

Where Linux gets it. Built-in text.

Background. To log sensors over time, run a `-save` of `[sensors]` on a timer (for example from a shell loop or a systemd timer) and keep each file.

CSV titles

`csv-titles` - Linux data

In plain words. Describes the layout of a save file, since there is no CSV header to show.

What you see.

- File header - `srs-gui - System Report Search V<version> <host> <YYYY-MM-DD HH:MM:SS> [<items>]`.
- Section header - `[<item>] [<category>]`, e.g. `[gpu] [xxb]`.
- Section body - the item's own column titles, then its rows.
- Separator - two blank lines after each section.

Healthy / typical values. Fixed text. Useful if you write a script that splits a save file into items.

Where Linux gets it. Built-in text.

Debug API

`debug-api` - Linux data

In plain words. Build facts and pointers for people who debug SRS.

What you see.

- Build - compiler and runtime, same as in About.
- Assertions - `on` or `off (NDEBUG)`. Assertions are self-checks compiled into the program.
- Item list - the command `srs-gui --list` prints every item.
- Dump any item - `srs-gui -save=[item]=STDOUT` prints one item.
- Process state - where to look: `[stc]` threads, `[mutex]` locks, `[siv-dll]` modules.

Healthy / typical values. On the author's PC: Assertions `on`.

Where Linux gets it. Compile-time build information.

Delta/cost settings

`delta` - Linux data

In plain words. SIV can turn power use into an energy cost. SRS has no such setting.

What you see. Energy cost (not used) and Nearest (live power is in PMB info and the sensor items).

Where Linux gets it. Built-in text.

DSB info (DIMM sensor buffer)

`dsb` - Linux data

In plain words. The temperature of each memory stick (DIMM) that has a sensor.

What you see. One row per DIMM sensor:

Column	What it means	Why it matters
#	Row number.	-
hwmon	The hwmon device name, e.g. `hwmon7`.	Lets you find the raw files.
Device	The I2C (Inter-Integrated Circuit) bus and address, e.g. `1-0051` = bus 1, address 0x51.	Address 0x50-0x57 maps to a slot.
Current	Temperature now, in degrees C.	The main value.
Maximum	The "high" limit set in the sensor.	Above this the stick may slow itself down.
Critical	The "critical" limit.	Above this is a real problem.

If no sensor is found you see `no DIMM temperature sensors (spd5118 / jc42 hwmon) loaded`. This panel refreshes every 2 seconds.

Healthy / typical values. On the author's two DDR5 sticks: `31.00 C` and `30.50 C` at idle, limits 55 C (max) and 85 C (critical). Idle 30-45 C is normal; 50-60 C under heavy load is common with no airflow over the RAM.

Unusual values and what they suggest.

- Above 55 C at idle: poor case airflow, or a hot GPU blowing on the RAM.
- Above 70-80 C: DDR5 starts to slow itself (it refreshes more often) and errors become more likely. Improve cooling.
- One stick much hotter than the other: it sits closer to a heat source, or its sensor is off.
- No rows on DDR5: load the `spd5118` driver (`modprobe spd5118`). DDR4 sticks with a sensor use `jc42`; many DDR4 sticks have none.

Where Linux gets it. `/sys/class/hwmon/hwmonN/temp1_{input,max,crit}` for hwmon devices named `spd5118` (DDR5) or `jc42` (DDR4 and older).

Background. *hwmon* (hardware monitoring) is the Linux system that exposes sensor chips as files. Each DDR5 stick has a small hub chip on the SMBus (System Management Bus) that holds the SPD (Serial Presence Detect: the stick's own description of its size and speeds) and a temperature sensor.

Emoji glyph test

`emoji` - Linux data

In plain words. Checks that your fonts can draw symbols and emoji, so the display looks right.

What you see.

- Glyphs - a row of sample symbols (smiley, monitor, thermometer, lightning, battery, floppy disk, tick, cross, `°C`, `µs`, arrow).
- Boxes - one ticked and one empty tick box.
- Fonts with emoji coverage - every font that has the smiley emoji (U+1F600), from fontconfig.

Healthy / typical values. All glyphs drawn, and at least one colour emoji font in the list. The author's PC lists Noto Color Emoji, Twemoji, Segoe UI Emoji, DejaVu Sans and others.

Unusual values and what they suggest. Empty boxes or question marks instead of glyphs, or `none found`: install an emoji font such as Noto Color Emoji.

Where Linux gets it. `fc-list :charset=1f600 family`.

Fake IMCs

`fak-imcs` - Not present on most PCs

In plain words. In SIV this replays saved memory-controller data from Intel Xeon servers, for testing. SRS has no replay mode, so this always says "Not present".

What you see. Status (`Not present on this system`), Checked (what it looked for: Intel Xeon ICX IMC fixtures) and your Processor name.

Background. An IMC (Integrated Memory Controller) is the part of the CPU that talks to RAM. "ICX" is Intel's Ice Lake server family. A "fixture" is a saved data dump used to test a program without the real hardware.

Fake MCHBAR

`fak-mchbar` - Linux data - Needs root data

In plain words. In SIV this replays saved Intel memory-controller register dumps, for testing. SRS reads no such registers; this item tells you where memory timing data comes from instead.

What you see. Status (what the SIV feature is), srs-gui (reads no MCHBAR registers, never opens `/dev/mem`, uses no replay files) and Memory timings (come from the [spd] item via the srs-root cache file `spd.txt`, and from the kernel EDAC / amd64 drivers).

Where Linux gets it. Nothing is read. The memory timing data it points to needs the root helper (**Menu -> Run as root**).

Background. MCHBAR (Memory Controller Hub Base Address Register) is the address of a block of Intel memory-controller registers. Reading it needs raw physical memory access, which SRS avoids.

Fake PM tables

`fak-pmts` - Linux data

In plain words. SIV can replay saved AMD power-management tables. SRS has no replay; it tells you if the live table is available.

What you see.

- Status - no replay; the live PM table is in the `[amd-pmt-0]` item.
- pm_table - `present` if the ryzen_smu driver exposes the table, else `ryzen_smu driver not loaded`.
- pm_table_version, pm_table_size - the driver's table version and size files.
- SMU firmware - the firmware version of the SMU, e.g. `4.84.4.0` on the author's PC.

Unusual values and what they suggest. `ryzen_smu driver not loaded`: install and load the out-of-tree `ryzen_smu` module if you want PM table data. Intel PCs never have it.

Where Linux gets it. `/sys/kernel/ryzen_smu_drv/{pm_table,pm_table_version,pm_table_size,version}`.

Background. The SMU (System Management Unit) is a small controller inside AMD Ryzen chips that runs clocks, voltages and power limits. Its PM (power management) table is a live block of numbers: power, temperatures, clocks and limits.

Fake SPDs

`fak-spds` - Linux data

In plain words. SIV can replay saved memory-stick SPD dumps. SRS has no replay; it shows where the real SPD data comes from.

What you see.

- Status - no replay; real SPD data is in the `[spd]` item.
- SPD cache - path of `spd.txt` in the root cache, or `Needs root: Menu > Run as root (srs-root)`.
- One line per DIMM sensor (`spd5118` or `jc42`), with its hwmon name and I2C address, e.g. `spd5118 hwmon7 at i2c 1-0051`.
- `no spd5118 / jc42 hwmon devices` if none are loaded.

Healthy / typical values. On the author's PC: cache present, two `spd5118` sensors at 1-0051 and 1-0053.

Unusual values and what they suggest. `Needs root`: run **Menu -> Run as root** once so the `[spd]` item can show memory timings.

Where Linux gets it. `~/.cache/srs/root/spd.txt` (written by srs-root) and `/sys/class/hwmon`.

Filter

`filter` - Linux data

In plain words. Lists the ways you can narrow what SRS shows.

What you see.

- Find pattern - regular expression or literal text, Ignore case, Max matches.
- Lookup filter - PCI or USB bus; Tree, Scan or Info mode.
- Device panels - PCI Bus, USB Bus and ALL Dev pick the bus.
- -save filter - `[item][item]...` picks which items are written.
- Column filter - none; every column is always shown.

Where Linux gets it. Built-in text.

Background. A *regular expression* (regex) is a search pattern. For example `nvme[0-9]` finds `nvme0`, `nvme1` and so on.

Final processing

`final` - Linux data

In plain words. How long SRS has been running and how much CPU time it used so far.

What you see.

- Elapsed - seconds since the process started.
- CPU time - CPU seconds the process has used.
- Sections - how many items SRS has.
- On exit - what happens at close (save settings in GUI mode, free tables, close panels).

Healthy / typical values. On the author's PC, during a full `[all]` save: Elapsed 3.140 s, CPU time 0.160 s, 558 items. Elapsed is much larger than CPU time because most of the time is spent waiting on files and helper tools, not computing.

Unusual values and what they suggest. In a full save, Elapsed far above 10-20 s means some item waited a long time; often a slow or sleeping disk, or a network tool timing out.

Where Linux gets it. `/proc/uptime`, `/proc/self/stat` (start time) and the process CPU clock.

GPU info buffer

`gpu` - Linux data

In plain words. One line per graphics card: driver, PCI address, load, video memory, PCIe link and current clocks.

What you see.

Column	What it means	Why it matters
GPU	DRM card name, e.g. `card0`.	Matches `/sys/class/drm/card0`.
Driver	Kernel driver, e.g. `amdgpu`, `i915`, `nvidia`.	Wrong or missing driver = no acceleration.
PCI ID / Slot	Vendor-device ID and PCI address, e.g. `1002-7550 0000:03:00.0`.	Identifies the exact chip.
Busy	GPU load in percent.	High at idle means something is rendering.
VRAM Used / Total	Video memory in use and installed.	Near full causes stutter in games.
Link	PCIe link width and speed now, e.g. `x16 32.0 GT/s PCIe`.	Lower than expected costs bandwidth.
Clocks (sclk / mclk)	Current core clock and memory clock (the starred level).	Low at idle, high under load.

Columns the driver does not provide show `-`. Busy, VRAM and Clocks come from amdgpu files, so on other drivers they are usually `-`. This panel refreshes every 2 seconds.

Healthy / typical values. On the author's Radeon RX 9070 XT: `card0 amdgpu 1002-7550 0000:03:00.0 7% 1.03GB / 15.92GB x16 32.0 GT/s PCIe 664Mhz / 875Mhz` on the desktop.

Unusual values and what they suggest.

- Link x8 or x4 on an x16 card: the card is in a lower slot, or that slot shares lanes with an M.2 drive.
- Link 2.5 GT/s at idle: normal power saving; it rises under load. If it stays low under load, check the BIOS PCIe setting.
- Busy stuck near 100% at idle: a program is using the GPU; check with a GPU monitor.

Where Linux gets it. `/sys/class/drm/cardN/device/` files: `driver`, `vendor`, `device`, `gpu_busy_percent`, `mem_info_vram_used`, `mem_info_vram_total`, `current_link_speed`, `current_link_width`, `pp_dpm_sclk`, `pp_dpm_mclk`.

Background. *PCIe link width* is how many lanes the card uses (x16 is full size). *GT/s* (gigatransfers per second) is the speed per lane: 8 = PCIe 3.0, 16 = 4.0, 32 = 5.0. *sclk* is the shader (core) clock and *mclk* the memory clock. *DPM* (Dynamic Power Management) levels are the clock steps the card can pick from.

Help

`help` - Linux data

In plain words. The built-in help: what each main-window button does, the hot-keys, and the command-line options. Also opened with **F1**.

What you see. Three blocks:

- Main window buttons - OK, Copy, PCI Bus / USB Bus, ALL Dev, Lookup, Find, Data Files, About / Help, Menu, Refresh, Next / Prev, Highlight / Colour.
- Hot-keys - Ctrl+F Find, Ctrl+L Lookup, Ctrl+D ALL Dev, F3 / Shift+F3 next / previous match, Ctrl+H highlight on/off, F5 refresh, F1 help, Esc close.
- Command line - `srs-gui`, `-save`, `id`, `find`, `devices`, `burn`, `--list`, `--version` / `-h`.

A last line gives the file locations: settings in `~/.config/srs/SRSDEF.ini`, root cache in `~/.cache/srs/root`, reports in `~/sysreports`.

Where Linux gets it. Built-in text.

Background. The PCI Bus button resolves each device by trying the most exact ID first: vendor:device:subvendor:subdevice:revision, then shorter keys, down to vendor only. That is what `V:D:S:s:RR -> ... -> V` means.

Highlight setup

`highlight` - Linux data

In plain words. Your Find highlight colour, whether highlighting is on, and the panel font.

What you see. Columns: (key), Colour, Find Description, Setting, Reported Value.

- `[F]` row - the Colour Find Text as `#RRGGBB` and as `RGB( r, g, b )`.
- Highlight Matches - tick box and `on` / `off`.
- Panel Font - the Font Name setting.

Below: the settings file path (or `SRSDEF.ini not written yet - defaults shown`), and a note that SIV's CPU/GPU speed highlight colours are the coloured columns of the `[cpu]` and `[gpu-clocks]` items.

Healthy / typical values. Defaults, and the author's values: `#FFFF00` (yellow, `RGB( 255, 255, 0 )`), on, Tahoma.

Where Linux gets it. Keys `Colour Find Text`, `Highlight Matches` and `Font Name` in section `[SRSDEF]` of SRSDEF.ini.

Hotkeys

`hot-keys` - Linux data

In plain words. The keyboard shortcuts of the SRS window.

What you see. A header (Profile, Hotkeys Enabled ticked) and one row per shortcut: letter code, action, key and `panel accelerator (fixed)`. The ten shortcuts are Find (Ctrl+F), Lookup (Ctrl+L), ALL Dev (Ctrl+D), Highlight Matches (Ctrl+H), Next Match (F3), Previous Match (Shift+F3), Refresh (F5), Help (F1), Close Panel (Esc) and Copy Selection (Ctrl+C).

Where Linux gets it. Built-in list.

Background. These keys only work while an SRS window has focus. SIV's global hot-keys (which work from any program and can switch screens or LEDs) have no SRS equivalent.

SIV image (EXE)

`image-exe` - Linux data

In plain words. The internal layout of the SRS program file: its sections.

What you see. The program path and size, then the full `readelf -S -W` table: `[Nr]` section number, Name, Type, Address, Off (file offset), Size, ES (entry size), Flg (flags, explained in the "Key to Flags" at the end), Lk, Inf, Al (alignment). If binutils is missing you see `readelf not available (binutils)`.

Healthy / typical values. About 28 sections. `.text` (the code) is the largest; on the author's 1.38 MB build it is `0x0d333c` bytes (about 845 KB). `.rodata` holds fixed text and tables.

Where Linux gets it. `readelf -S -W /proc/self/exe`.

Background. Linux programs use the ELF (Executable and Linkable Format) file format. A *section* is one named part of the file: `.text` = code, `.data` = initial variables, `.bss` = zero-filled memory reserved at start, `.rodata` = read-only data.

SIV image (SYS driver)

`image-sys` - Windows-only (Linux equivalent shown)

In plain words. SIV on Windows loads its own kernel driver (SIVX64.sys). SRS needs none. Instead this lists the Linux kernel modules SRS reads data from.

What you see.

- Status - no kernel driver; root data comes from the srs-root helper.
- SIV equivalent - SIVX64.sys is not needed.
- One line per module that is present: name, `loaded` (a loadable module) or `built-in` (part of the kernel), its version if it reports one, and `refcnt` (how many users hold it).

Modules checked: k10temp, zenpower, coretemp, amdgpu, i915, nvidia, nct6775, nct6683, nct6687, it87, asus_ec_sensors, asus_wmi_sensors, spd5118, jc42, ee1004, ryzen_smu, msr, cpuid, drivetemp, nvme, acpi_cpufreq, amd_pstate, zenergy. Only those present are listed.

Healthy / typical values. On the author's AMD PC: k10temp, amdgpu (refcnt 61), nct6775, spd5118, ee1004, ryzen_smu (version 0.1.7), msr (built-in), drivetemp, nvme, zenergy.

Unusual values and what they suggest.

- No CPU temperature module (k10temp on AMD, coretemp on Intel): CPU temperatures will be missing.
- No Super-I/O module (nct6775, it87 and so on): board fan speeds and voltages will be missing.
- No `drivetemp`: SATA drive temperatures are not in hwmon.

Where Linux gets it. `/sys/module/<name>/`, `version` and `refcnt`.

Background. A *kernel module* is a driver that can be loaded into the running kernel. A *Super-I/O* chip on the motherboard (Nuvoton, ITE) reads fan speeds, voltages and board temperatures.

Image tree

`image-tree` - Linux data

In plain words. Which shared libraries SRS needs, and which libraries those need.

What you see. The program name and path, then a tree: each library the program needs, the file it was loaded from (or `(not loaded)`), and below it the libraries that one needs. A last line counts the shared objects mapped into the process.

Healthy / typical values. SRS needs very little. On the author's PC: `libm.so.6` and `libc.so.6`, each pulling in `ld-linux-x86-64.so.2`; 6 shared objects mapped.

Unusual values and what they suggest. `(not loaded)` next to a needed library: unusual for a running program; the library may be loaded under another path.

Where Linux gets it. `readelf -d` (the NEEDED entries) and `/proc/self/maps`. SRS does not run `ldd`, since `ldd` can run the program.

Background. A *shared library* (`.so` file) is code many programs use at once, such as the C library `libc.so.6`. `ld-linux` is the loader that links them at start.

Initial processing

`initial` - Linux data

In plain words. The steps SRS takes when the window starts.

What you see. A numbered list: 1 read SRSDEF.ini; 2 load pci.ids and usb.ids; 3 find PCI and USB devices and resolve names; 4 find the newest report in `~/sysreports` for Find; 5 build the item list for the Menu; 6 open the main window and set up the shortcut keys; 7 start refresh timers only for open live panels. A note: with arguments, the GUI is skipped and the command line runs.

Where Linux gets it. Built-in text.

Install

`install` - Linux data

In plain words. Where SRS is installed and whether its menu entry and folders exist.

What you see. Program image path, then a tick box for each:

- Menu entry in `~/.local/share/applications`, `/usr/share/applications` and `/usr/local/share/applications` (`srs-gui.desktop`). If found, the `Exec` line it runs.
- Settings folder `~/.config/srs`.
- Root cache folder `~/.cache/srs/root`.

Healthy / typical values. A packaged install has the `/usr/share/applications` entry with `Exec srs-gui`. On the author's PC that entry, the settings folder and the root cache folder are all ticked.

Unusual values and what they suggest.

- No menu entry ticked: SRS will not show in your app menu; reinstall the package.
- Root cache folder not ticked: the root helper has never run, so root-only items will say "Needs root".

Where Linux gets it. Checks those paths and reads `Exec=` from the `.desktop` file.

SIV languages

`languages` - Linux data

In plain words. SRS is English only, using UTF-8 text. This also shows your language settings.

What you see. UI language (`English only`), Encoding (`UTF-8`), then the values of the environment variables LANG, LC_ALL, LC_MESSAGES, LC_CTYPE and LANGUAGE (`<unset>` if not set).

Healthy / typical values. On the author's PC: `LANG en_US.UTF-8`, the LC_ values unset.

Unusual values and what they suggest. A LANG without `UTF-8` (for example `C` or `POSIX`): symbols such as `°C` and `µs` may print wrongly in a terminal.

Where Linux gets it. The process environment.

Background. A *locale* is the set of language, number and date rules a program uses. `LANG` sets the default; `LC_*` variables override parts of it.

Lock setup

`lck-setup` - Linux data

In plain words. How SRS keeps its own threads from getting in each other's way. In short: it needs almost no locks.

What you see. Instance lock (none; several copies may run), Find (one worker thread, the GUI thread owns all windows), Burn-in (one thread per CPU, pinned, polls a stop flag), Hardware locks (none, because SRS does no SMBus, ISA or PCI config access).

Where Linux gets it. Built-in text.

Background. A *lock* or *mutex* (mutual exclusion) stops two threads using the same thing at the same time. SIV needs global locks because it talks to hardware buses directly; SRS leaves that to the kernel.

Maker icons

`makers` - Linux data

In plain words. The companies that made the PCI devices in your PC, with a count for each.

What you see. Columns: Vendor (4-digit hex PCI vendor ID), Maker (name from pci.ids, or `(unknown)`), PCI functions (how many PCI functions from that vendor). If pci.ids is missing: `pci.ids not found`.

Healthy / typical values. On the author's AMD PC: 1022 AMD (38 functions, the chipset and CPU parts), 1002 AMD/ATI (4, the graphics card and its audio), 1B21 ASMedia (7, USB and SATA controllers), 10EC Realtek (2, network), plus Samsung, ADATA and MAXIO (one each, NVMe drives).

Unusual values and what they suggest. `(unknown)` maker: pci.ids is too old for that device, or it is a very new or rare vendor. Update hwdata.

Where Linux gets it. The PCI devices in sysfs, names from pci.ids.

Background. A *PCI function* is one logical device. One physical chip can show several (a GPU usually shows a graphics function and an audio function). SIV shows maker logos here; SRS shows names.

SIV manifest

`manifest` - Windows-only (Linux equivalent shown)

In plain words. Windows programs carry a "manifest" describing them. The Linux equivalent is the ELF *note* sections, shown here.

What you see. The output of `readelf -n`: for each note section, Owner, Data size and Description. Typical notes:

- `.note.gnu.build-id` - a unique build ID (a long hex number) for this exact build.
- `.note.gnu.property` - CPU instruction-set levels the code needs and uses (x86-64-v2, v3, v4) and features (XMM, YMM, ZMM).
- `.note.ABI-tag` - the oldest Linux kernel ABI the program supports.

Healthy / typical values. On the author's build: build ID `XXXX...`, ABI `OS: Linux, ABI: 6.1.0`.

Unusual values and what they suggest. If "x86 ISA needed" lists `x86-64-v4` (AVX-512 level), that build only runs on CPUs with AVX-512. A copy built that way will crash with "illegal instruction" on an older CPU. Use a build made for a lower level.

Where Linux gets it. `readelf -n -W /proc/self/exe`.

Background. The *ABI* (Application Binary Interface) is the set of low-level rules a program and the kernel agree on. *x86-64-v2/v3/v4* are CPU feature levels: v3 needs AVX2, v4 needs AVX-512.

Mapping cache

`map-cache` - Windows-only (Linux equivalent shown)

In plain words. In SIV this shows how its driver maps physical memory. SRS never maps physical memory, so it just explains that.

What you see. Status (Windows-only feature), srs-gui (no driver, `/dev/mem` is never opened), Nearest Linux (kernel drivers expose registers in sysfs; root-only tables come from the srs-root cache).

Background. *Mapping physical memory* means reading hardware registers by their raw address. It is powerful but risky; on Linux the kernel drivers do it for you.

Map/unmap test

`map-unmap` - Windows-only (Linux equivalent shown)

In plain words. In SIV this times its driver mapping and unmapping physical memory. SRS has nothing to time; it shows the same three lines as Mapping cache.

Menus

`menus` - Linux data

In plain words. The full list of every item SRS has, grouped by category.

What you see. For each category, a line `[category] N items`, then one line per item: `[id]`, title, bucket and group. At the end: `N items in M categories`.

- Bucket `Linux` (green) - real Linux data.
- `Absent` - hardware most PCs do not have.
- `Windows-only` - a Windows feature; the Linux equivalent is shown.
- `(stub)` after an item - the item has no data function yet.

Healthy / typical values. On the author's PC the `[tuning]` category alone has 48 items, and the whole program has 558.

Where Linux gets it. The program's own item list (same as `srs-gui --list`).

Mutex info

`mutex` - Linux data

In plain words. File locks that SRS holds right now. Normally none.

What you see. Columns # (lock number), Class (`POSIX`, `FLOCK`, `OFDLCK`), Mode (`ADVISORY` or `MANDATORY`), Access (`READ` or `WRITE`), PID / Inode (process, device and file, and byte range). Then a count: `N file locks held by this process (/proc/locks)`.

Healthy / typical values. `0 file locks held by this process` on the author's PC.

Where Linux gets it. `/proc/locks`, filtered to this process ID.

Background. A *file lock* tells other programs "I am using this file". The kernel lists every lock in `/proc/locks`.

PMB info (power monitor buffer)

`pmb` - Linux data

In plain words. Every live power (watts) and current (amps) reading the kernel's sensor drivers provide.

What you see.

Column	What it means	Why it matters
#	Row number.	-
Device	hwmon chip name, e.g. `amdgpu`, `zenergy`.	Which part the reading belongs to.
Description	Sensor label (e.g. `PPT`) or the raw name (`power1`).	Which rail or limit.
Current	Power in W (3 decimals) or current in A.	The live value.
Characteristic	`cap N W` if a power cap is set for that sensor.	How close you are to the limit.

The panel refreshes every second.

Healthy / typical values. On the author's PC: `amdgpu PPT 35.000 W cap 374 W` (graphics card on the desktop). A desktop GPU idles at roughly 10-40 W; under game load it runs near the cap.

Unusual values and what they suggest.

- GPU power near the cap at idle: something is rendering, or a monitor setup keeps memory clocks high (common with several screens or high refresh rates).
- A cap well below the card's rated power: a power limit was lowered, by a tuning tool or the BIOS.
- No rows: no hwmon driver reports power; CPU power on AMD needs a module such as `zenergy`.

Where Linux gets it. `/sys/class/hwmon/*/power*_input`, `power*_average`, `curr*_input`, with `*_label` and `power*_cap`.

Background. *PPT* (Package Power Tracking) is AMD's name for total power drawn by the chip package. A *power cap* is the limit the firmware holds it to.

Colour RGB debug

`rgb-debug` - Linux data

In plain words. The colour palette SRS uses for text, with what each colour means.

What you see. Columns Colour (internal name), Default Colour (`RGB( r, g, b )`), Hex and Colour Description. Rows: C_LABEL (labels, light blue 3399FF), C_VALUE (values, dark blue 0000CC), C_PURPLE (paths, IDs, commands, 9C0FB8), C_TEAL (secondary values, 009999), C_ORANGE (build and warnings, FF8800), C_GREEN (OK / enabled, 00A000), C_RED (errors / not found, E00000), C_BLACK (plain text), C_GRAY (notes, disabled, 808080), and Find Text (your Colour Find Text setting).

Where Linux gets it. Built-in palette, plus SRSDEF.ini for Find Text.

Background. Knowing the colours helps you scan a panel: red means missing or error, orange means "changed from default" or a warning, green means OK.

SDM info (storage device monitor)

`sdm` - Linux data

In plain words. Every whole disk the kernel sees, with its size and how much has been read and written since boot.

What you see.

Column	What it means	Why it matters
#	Row number.	-
Device Name	Kernel name: `nvme0n1`, `sda`, `sr0`, `zram0`.	Matches `/dev/<name>`.
Model	Model string, or `-`.	Identifies the drive.
Size	Capacity.	`0B` = no media (empty card reader or USB slot).
Reads	Data read since boot.	Shows which drives are busy.
Writes	Data written since boot.	Heavy writes wear SSDs.
In flight	Requests in progress right now.	Stuck above 0 can mean a hung drive.
Rotational / Removable	Spinning disk or solid state; fixed or removable.	Affects which I/O scheduler suits it.

Partitions, loop and ram devices are skipped. Refreshes every 2 seconds.

Healthy / typical values. On the author's PC: three NVMe SSDs (Samsung 980 PRO 1 TB, ADATA S70 2 TB, PNY CS2340 500 GB), a 4 TB Seagate hard disk (`rotational / fixed`), USB sticks, a DVD writer and `zram0` (15.23 GB compressed RAM swap). In flight `0` on all of them.

Unusual values and what they suggest.

- In flight stays above 0 for many seconds: the drive is stuck or very slow. Check `dmesg` for errors.
- A drive you know is there is missing: cable, power, or the drive has dropped off the bus.
- USB sticks shown as `rotational`: normal. Many USB bridges report this by default, not because the stick spins.
- An optical drive shows a size even with no disc: the kernel keeps the last size; ignore it.

Where Linux gets it. `/sys/class/block/<dev>/{size,stat,removable,queue/rotational,device/model}`.

Background. *zram* is a block device in RAM that stores compressed data, used as fast swap. Size and sector counts are in 512-byte sectors, which SRS converts to bytes.

Sequence setup

`sequence` - Linux data

In plain words. The order in which items appear in a plain `-save` (one with no item list). It is the same order SIV64X uses.

What you see. A numbered list: position, `[id]` and title. It starts `1 [overview]`, `2 [sgb-ini]`, `3 [sdm]`, `4 [dsb]`, `5 [bsi]` and so on.

Healthy / typical values. About 120 items in the default save.

Where Linux gets it. The program's built-in default save list.

Background. Keeping SIV's order means you can compare an SRS save with a SIV save side by side.

SGB end

`sgb-end` - Linux data

In plain words. In SIV this marks the end of the system/board buffer. In SRS it is a marker line plus how long the process has been running.

What you see. SGB end (explains there is no shared buffer; SGB info reads sysfs live) and Elapsed (seconds since start).

Healthy / typical values. On the author's full save: `9.230 seconds since start` at that point.

SGB info (system/board buffer)

`sgb-ini` - Linux data

In plain words. The motherboard, BIOS and chipset IDs, plus a few facts about the SRS process.

What you see.

Field	What it means	Why it matters
Maker Name / Model Name / Board Version	The motherboard maker, model and revision.	Find the right manual and BIOS.
Serial Number	Board serial. Needs root; read from the cached dmidecode output. Else `Needs root: ...`.	Warranty and asset tracking. Keep it private.
System	System maker and product name.	Prebuilt PCs show a product here; self-builds show placeholders.
BIOS Vendor / Version / Date / Release	The firmware maker, version, date and release number.	Check if a newer BIOS exists.
Chassis Type	SMBIOS chassis code (3 = desktop, 10 = notebook, and so on).	What kind of case firmware reports.
MCH (00:00.0)	Vendor-device ID of PCI device 00:00.0, the host bridge.	Identifies the CPU / chipset family.
ACT CPUs	Online CPUs (configured CPUs).	Should match your thread count.
SRS Affinity	Hex mask of CPUs SRS may run on.	Normally all CPUs.
SRS PID	Process ID of this SRS run.	-
SRS Driver	`none`.	SRS uses no kernel driver.
Root cache	Whether `dmidecode.txt` is in the root cache.	If not, the serial cannot be shown.
RAW Command	Path of the running program.	-

Healthy / typical values. On the author's PC: ASUSTeK COMPUTER INC., TUF GAMING X870-PRO WIFI7 W NEO, Rev 1.xx, serial `XXXX`, System `ASUS System Product Name` (a placeholder, normal for a self-built PC), American Megatrends BIOS 1805 dated 09/18/2026, release 18.5, chassis type 3, MCH `1022-14d8` (AMD), ACT CPUs `16 (16)`, affinity `000000000000FFFF`.

Unusual values and what they suggest.

- Fields like "To Be Filled By O.E.M." or "Default string": the board maker left them blank. Harmless.
- SRS Affinity not all ones: SRS was started with `taskset` or inside a restricted cgroup or container.
- ACT CPUs smaller than the number in brackets: some CPUs are offline.

Where Linux gets it. `/sys/class/dmi/id/*` (SMBIOS types 0, 1, 2 and 3), PCI `0000:00:00.0` vendor and device files, `sched_getaffinity()`, and `~/.cache/srs/root/dmidecode.txt` for the serial.

Background. *SMBIOS* (System Management BIOS) is a table the firmware fills with facts about the board, BIOS, memory and case. Linux copies the public fields to `/sys/class/dmi/id`; serial numbers are root-only.

Active SIV instances

`siv-active` - Linux data

In plain words. Every copy of SRS running right now.

What you see. Columns Name (process name), PID, Version (`V<version> (this)` for the copy making the report, `-` for others) and Image Filename (program path, or `(not readable)`). Then a count. Names come from `/proc/<pid>/comm`, which the kernel cuts to 15 characters, so `sysreport-search` shows as `sysreport-searc`.

Healthy / typical values. On the author's PC: the tray copy `srs-gui` and the `sysreport-searc` copy making the save; 2 instances.

Unusual values and what they suggest. Many old copies: earlier windows or tray icons did not close. They are safe to close; SRS holds no driver or hardware lock.

Where Linux gets it. `/proc/*/comm` and `/proc/*/exe`.

SIV autorun

`siv-autorun` - Linux data

In plain words. Whether SRS starts by itself when you log in.

What you see. Two tick-box lines: User autostart (`~/.config/autostart/srs-gui.desktop`) and System autostart (`/etc/xdg/autostart/srs-gui.desktop`), each with its `Exec` command if present. Then `srs-gui starts at login.` or how to enable it.

Healthy / typical values. Off by default. On the author's PC the user entry is on and runs `/usr/bin/srs-gui --gui-tray` (start in the system tray).

Where Linux gets it. The XDG autostart folders.

Background. *XDG autostart* is the desktop standard: any `.desktop` file in an autostart folder is run at login. Delete or rename the file to stop it.

SIV DLLs

`siv-dll` - Linux data

In plain words. The files SRS depends on: name databases, the program itself, the settings file and every shared library it has loaded. Opens with the Lookup button row.

What you see. A table with Description, File Specification (path), Current Identification (version or date), Code Page (text encoding, always UTF-8) and WEB Download Location. Rows:

- PCI Device Names, USB Device Names, PnP / Monitor Names - the `.ids` files, their version and upstream URL.
- srs-gui Program - path, version and size.
- Settings - SRSDEF.ini and when it was last written, or `not written yet`.
- Kernel driver - `none`.

Then "Shared objects mapped into this process": each library name, path, size and file date.

Healthy / typical values. On the author's PC: libc, libm, ld-linux and a few `gconv` modules (character-set converters loaded by the C library, such as CP932.so), all from `/usr/lib`.

Unusual values and what they suggest. A library loaded from an odd folder (your home directory, `/tmp`): something is setting `LD_LIBRARY_PATH` or `LD_PRELOAD`. Check your shell setup.

Where Linux gets it. `/proc/self/maps`, file sizes and dates, and the `.ids` file headers. No network check is made.

Background. On Windows a DLL (Dynamic-Link Library) is a shared code file. The Linux equivalent is a shared object (`.so`).

SIV driver

`siv-drv` - Windows-only (Linux equivalent shown)

In plain words. SIV shows its kernel driver here. SRS has no driver; it shows the root helper's cache instead, which is how SRS gets root-only data.

What you see.

- Kernel driver - none.
- Root helper - srs-root (**Menu -> Run as root**) writes a read-only cache; items never run as root.
- Root cache - the cache folder path.
- One line per cache file: name, size and date. Or `empty - Needs root: Menu > Run as root (srs-root)`.

Healthy / typical values. On the author's PC the cache has about 24 files, all from the same run: `dmidecode.txt`, `dmesg.txt`, `msrs.txt`, `pci-all.txt`, `spd.txt`, `smart-<drive>.txt`, NVMe details and more. Tiny `smart-*.txt` files (55 bytes) belong to drives that report no SMART data, such as card readers.

Unusual values and what they suggest.

- Cache files days or weeks old: root-only items show old data. Run the root helper again.
- `smart-*` files for drives no longer attached: left over from an earlier run. Harmless.

Where Linux gets it. `~/.cache/srs/root/`.

Background. SMART (Self-Monitoring, Analysis and Reporting Technology) is a drive's own health log. Reading it needs root, which is why it comes from the cache.

SIVDEF.ini

`siv-ini` - Linux data

In plain words. The whole SRS settings file, printed as it is.

What you see. Each `[section]` in bold, then each `key value` line. Comment lines (`#` or `;`) show as notes. Last line: `N section(s) in <path>`. If the file does not exist yet: `No Sections Found within the SRSDEF.ini file`, `Status 2 File not found.` and the path.

Healthy / typical values. One `[SRSDEF]` section. On the author's PC: Colour Find Text FFFF00, Highlight Matches 1, Font Name Tahoma, GPU Load Command, Tray Icon 1, Close To Tray 1, Minimize To Tray 1, Start In Tray 1, Theme Dark.

Unusual values and what they suggest. "File not found" on a fresh install: normal until you change a setting in the window.

Where Linux gets it. `$XDG_CONFIG_HOME/srs/SRSDEF.ini`, by default `~/.config/srs/SRSDEF.ini`. SIV keeps its settings in SIVDEF.ini and the Windows registry; SRS uses this file.

SIV location / window poke

`siv-location` - Linux data

In plain words. How SRS sizes and places its windows, and what kind of desktop session it is in.

What you see. Main window (default 1000 x 420), Panels (default 1180 x 560, cascaded on X11), Saved positions (none; SRS does not remember positions), Session (`Wayland`, `X11` or `no display (CLI)`), Desktop (from XDG_CURRENT_DESKTOP, if set), GDK_SCALE (window scale factor) and Window poke (not supported on Linux).

Healthy / typical values. On the author's PC: Wayland, KDE, GDK_SCALE `1 (default)`.

Unusual values and what they suggest. Text far too small or large: set `GDK_SCALE=2` (or 1) before starting SRS on a high-DPI screen.

Where Linux gets it. The environment variables WAYLAND_DISPLAY, DISPLAY, XDG_CURRENT_DESKTOP and GDK_SCALE.

Background. On Wayland the compositor (the desktop) decides where windows go; programs cannot place themselves. "Window poke" in SIV sends messages to another program's window; Linux has no general way to do that.

SIV qualifiers

`siv-qualifiers` - Linux data

In plain words. Every command-line option SRS accepts, with what it does.

What you see. A two-column table, Qualifier and Meaning:

- `-save`, `-save=[item][item]...`, `-save=[...]=file|STDOUT` - write items as text.
- `id ID` with `--bus pci|usb` and `--mode tree|scan|info` - look up a device ID.
- `find PATTERN` with `--report DIR`, `-i`, `-F`, `--max N`, `--color auto|always|never` - search a report.
- `devices` - resolve every local device.
- `burn` - CPU burn-in test, with thread, CPU, time, mode and temperature limit options.
- `--list`, `--version`, `-h, --help`.

Last line: Command Used (the program path).

Healthy / typical values. Fixed list for each version.

Where Linux gets it. Built-in list.

Background. In SIV, options are called *qualifiers*. Defaults are given in brackets, e.g. `--bus pci` and `--mode tree`.

SIV qualifier list

`siv-quals` - Linux data

In plain words. The same options as SIV qualifiers, names only, without sub-options or meanings.

What you see. `-save`, `-save=[item][item]...`, `-save=[...]=file|STDOUT`, `id ID`, `find PATTERN`, `devices`, `burn`, `--list`, `--version`, `-h, --help`.

SIV setup

`siv-setup` - Linux data

In plain words. The main settings SRS knows about, and whether each is set in your file.

What you see. Columns (tick), Name, Data and Meaning:

- `|X|` = the key is in SRSDEF.ini, with its value in quotes. `|_|` = not set, showing `<default ...>`.
- Keys: Colour Find Text (Find match background, RRGGBB), Highlight Matches, Font Name, and SIO Sensors (Super-I/O sensor inputs you have switched back on, e.g. `nct6799:AUXTIN1,...`).

Healthy / typical values. On the author's PC: the first three set (`"FFFF00"`, `"1"`, `"Tahoma"`), SIO Sensors `<default none>`.

Where Linux gets it. Section `[SRSDEF]` of SRSDEF.ini. (SIV keeps these in the Windows registry.)

Background. Super-I/O chips have spare inputs that boards often leave unconnected, so SRS hides some by default. SIO Sensors lets you show chosen ones again.

SIV sizes

`siv-sizes` - Linux data

In plain words. The size in bytes of SRS's main internal data structures. Mainly for developers.

What you see. Columns Structure, Bytes and Use, for: srs_item (one item), srs_out (output sink), srs_db (a loaded `.ids` database), srs_rec (one database record), srs_result (a Lookup result), srs_info (an Info lookup), srs_findres and srs_hit (Find results), srs_dev (one device), and the C types `void *`, `long`, `size_t`. Then Items (how many registered) and Page size.

Healthy / typical values. On a 64-bit PC: pointer, long and size_t are 8 bytes; page size 4096. The author's build: srs_item 64 bytes, 558 items.

Unusual values and what they suggest. Page size 16384 or 65536: normal on some ARM systems.

Where Linux gets it. Compile-time sizes and `sysconf(_SC_PAGESIZE)`.

Background. A memory *page* is the unit the CPU and kernel use to manage memory, usually 4 KB on x86.

SIV driver service

`siv-srv` - Windows-only (Linux equivalent shown)

In plain words. In SIV this controls the Windows service that loads its driver. SRS installs no service.

What you see. Status (Windows-only), srs-gui (no driver, no service; root data via the srs-root helper), Helper start (on demand from **Menu -> Run as root**, with a polkit password prompt; it exits after writing the cache) and Nearest Linux (kernel modules, see SIV image (SYS driver)).

Background. *polkit* is the Linux service that asks for your password before a program may do something as root.

SIV usage history

`siv-usage` - Linux data

In plain words. Which version you have, when the program file was made, and when your settings were last saved.

What you see. Columns Version, Image Date (program file date) and Settings Last Written (SRSDEF.ini date, `-` if none). A note: SRS keeps no usage counter.

Healthy / typical values. On the author's PC: `V2.4.0`, image 2026-10-26, settings 2026-10-26.

Where Linux gets it. File modification times of the program and SRSDEF.ini.

SIV use / update

`siv-use` - Linux data

In plain words. Version facts and how up to date the name databases are. SRS never goes online to check for updates.

What you see. Program (version, local build, no update channel), Update check (not performed; no network access), then one line per database: name, version or date, and the upstream URL. Last line: update the `.ids` files with your distribution's hwdata package (or `update-pciids` / `update-usbids`).

Healthy / typical values. pci.ids a few weeks old at most on a rolling distribution. On the author's PC: pci.ids 2026.10.01, usb.ids 2026.06.26.

Unusual values and what they suggest. Databases many months old: new devices may show as unknown. Run **Update hardware lists** (Tools page, 2.4; it downloads only when you ask), or update the system's hwdata package.

Where Linux gets it. The header lines of the `.ids` files in use: since 2.4 the newer of the copies in `~/.local/share/srs/lists` (from Update hardware lists) and the system's.

SMB end

`smb-end` - Linux data

In plain words. In SIV this marks the end of the SMBus buffer. In SRS it is one marker line: SRS does no SMBus access of its own; see SMB info. (The 2.4 opt-in DIMM PMIC read is done by srs-root with `i2cdump` through the kernel's SMBus driver; see Memory timings.)

SMB info

`smb-ini` - Linux data

In plain words. The I2C / SMBus adapters and devices the kernel has registered, by name only.

What you see. Columns Device (sysfs name), Name (kernel's name for it) and Driver (driver bound to it, or `-`). Adapters are named `i2c-N`; devices on them are named `bus-address`, e.g. `1-0051`. A note: SRS never opens `/dev/i2c-*` or probes addresses.

Healthy / typical values. On the author's AMD PC: `SMBus PIIX4 adapter` ports (the chipset SMBus, where the RAM sits), `AMDGPU DM` buses (one per display output, used to read monitor data), `AMDGPU SMU`, a Synopsys DesignWare adapter, and two `spd5118` devices at 1-0051 and 1-0053 (the RAM sticks).

Unusual values and what they suggest.

- No PIIX4 / i801 adapter: the chipset SMBus driver is not loaded (`i2c_piix4` on AMD, `i2c_i801` on Intel), so RAM sensors cannot appear.
- RAM devices missing while the adapter is there: load `spd5118` (DDR5) or `ee1004` / `jc42` (DDR4).

Where Linux gets it. `/sys/bus/i2c/devices/*/name` and the `driver` link.

Background. *I2C* and *SMBus* are slow two-wire buses that link small chips: RAM SPD chips, sensors, RGB controllers and the monitor's DDC (Display Data Channel) for its name and settings. Each device has a 7-bit address.

Lock states

`states` - Linux data

In plain words. Each thread of SRS and what it is doing right now.

What you see. Columns TID (thread ID), Thread (name), State (one letter) and Wait channel (the kernel function it sleeps in, or `-`).

Healthy / typical values. In a command-line save: one thread, state `R` (running).

Unusual values and what they suggest. A thread in state `D` for a long time: it is stuck waiting on a device (often a disk or USB drive).

Where Linux gets it. `/proc/self/task/*/{comm,stat,wchan}`.

Background. Thread states: `R` running, `S` sleeping (waiting for an event), `D` waiting on hardware and cannot be interrupted, `T` stopped, `Z` zombie (finished but not yet collected).

STC info (thread contexts)

`stc` - Linux data

In plain words. A fuller view of SRS's threads: CPU time used, which CPUs each may run on, and priority.

What you see.

Column	What it means
TID	Thread ID.
Name	Thread name (cut to 15 characters).
State	One-letter state, see Lock states.

Column	What it means
Total CPU	User + system CPU time, as minutes:seconds.
Affinity	CPUs the thread may run on, e.g. `0-15`.
Last CPU	The CPU it last ran on, e.g. `CPU-15`.
Priority / Nice	Kernel priority and nice value.

Then a count of threads. Refreshes every 2 seconds in the window.

Healthy / typical values. On the author's PC during a save: one thread, `R`, `0:00.420`, `0-15`, `CPU-15`, `16 / -4`.

Unusual values and what they suggest. Nice not 0: the program was started with `nice`, or a tool such as a scheduler daemon or `ananicy` changed it.

Where Linux gets it. `/proc/self/task/*/stat` and `status` (`Cpus_allowed_list`).

Background. *Nice* runs from -20 (most favoured) to 19 (least). The kernel *priority* shown is 20 + nice for normal tasks.

Tune system

`tune` - Linux data

In plain words. A list of common kernel tuning values (sysctls), with the kernel default, your current value and a commonly used "tuned" value side by side.

What you see.

Column	What it means
Value Name	The sysctl name, e.g. `vm.swappiness`.
Default	The usual kernel default, in brackets.
Current	Your value now. Green = same as default; orange = changed; grey `<none>` = not present in this kernel.
Other	A value often used by tuning guides and gaming distributions, in brackets.
Path	The `/proc/sys` file it comes from.

The values covered: memory (`vm.swappiness`, `vm.vfs_cache_pressure`, `vm.dirty_*`, `vm.page-cluster`, `vm.max_map_count`, `vm.compaction_proactiveness`, `vm.watermark_boost_factor`,

`vm.overcommit_memory`, `vm.zone_reclaim_mode`), kernel (`nmi_watchdog`, `watchdog`, `sched_autogroup_enabled`, `split_lock_mitigate`, `numa_balancing`, `kptr_restrict`, `dmesg_restrict`, `perf_event_paranoid`, `unprivileged_bpf_disabled`, `printk`, `timer_migration`), files (`fs.inotify.max_user_watches`, `fs.file-max`) and network (`default_qdisc`, `tcp_congestion_control`, `rmem_max`, `wmem_max`, `netdev_max_backlog`, `tcp_fastopen`, `tcp_mtu_probing`). SRS only reads; it offers no Apply button.

Healthy / typical values. There is no single "right" value. On the author's tuned gaming setup: `vm.swappiness 150` (high, normal with zram swap), `vm.max_map_count 2147483642` (needed by some games), `net.core.default_qdisc fq`, `tcp_congestion_control bbr3`, `kernel.split_lock_mitigate 0`.

Unusual values and what they suggest.

- `vm.max_map_count` at 65530 and some games crash or fail to load: raise it (many distributions now set 1048576 or more).
- `vm.swappiness` high without zram: the system may swap to disk too soon.
- `kernel.numa_balancing <none>`: the kernel was built without NUMA balancing. Normal on single-socket kernels.
- `fs.inotify.max_user_watches` at 8192 and an IDE or file sync tool says "too many watches": raise it.

Where Linux gets it. `/proc/sys/<name with dots changed to slashes>`.

Background. A *sysctl* is a kernel setting you can read and change at run time. *Swappiness* sets how eagerly the kernel swaps. *Dirty* values set how much changed data may wait in RAM before it is written to disk. *qdisc* (queueing discipline) and *congestion control* decide how network packets are sent.

Tuning

`tuning` - Linux data

In plain words. The current power and performance profile: CPU frequency driver, governor, boost, CPU scheduler, memory and disk scheduler settings.

What you see. Lines appear only when the source exists:

Field	What it means
tuned profile / tuned mode	The active profile of the `tuned` service, or `tuned not active`.

Field	What it means
cpufreq driver	The CPU frequency driver, e.g. `amd-pstate-epp`, `intel_pstate`, `acpi-cpufreq`.
Governor (CPU-0)	The frequency policy, e.g. `performance`, `powersave`, `schedutil`.
EPP (CPU-0)	Energy Performance Preference hint to the CPU.
amd_pstate	amd-pstate mode: `active`, `passive` or `guided`.
Boost	Turbo / boost clocks enabled or disabled.
Platform profile	ACPI platform profile (laptops): `balanced`, `performance`, `low-power`.
sched_ext	Whether an extensible CPU scheduler is loaded, and which one.
THP enabled / THP defrag	Transparent huge page settings; the value in [brackets] is active.
KSM run	Kernel same-page merging on (1) or off (0).
PCIe ASPM policy	PCIe link power saving policy; the [bracketed] one is active.
vm.swappiness	Same as in Tune system.
I/O scheduler	The disk request schedulers available; the [bracketed] one is active.

Healthy / typical values. On the author's PC: tuned `cachyos-gaming`, `amd-pstate-epp`, governor and EPP `performance`, amd_pstate `active`, boost enabled, sched_ext enabled with `lavd`, THP `[always]`, NVMe drives on `[none]` or `[adios]`, the hard disk and USB drives on `[bfq]`. A plain distribution install usually shows `powersave` / `balance_performance` and `madvise` THP.

Unusual values and what they suggest.

- Boost disabled: the CPU never reaches its rated top speed. Check the BIOS or a power tool.
- `acpi-cpufreq` on a modern Ryzen: amd-pstate is not in use (old kernel, or disabled in BIOS / kernel command line).
- A hard disk on `none`: works, but `bfq` or `mq-deadline` usually suits spinning disks better.

Where Linux gets it. `/etc/tuned/active_profile`, `/sys/devices/system/cpu/cpu0/cpufreq/*`, `/sys/devices/system/cpu/amd_pstate/status`, `/sys/firmware/acpi/platform_profile`, `/sys/kernel/sched_ext/`, `/sys/kernel/mm/transparent_hugepage/`, `/sys/kernel/mm/ksm/run`, `/sys/module/pcie_aspm/parameters/policy`, `/sys/class/block/*/queue/scheduler`.

Background. A *governor* decides how fast the CPU should run. *EPP* is a hint the CPU's own firmware uses to balance speed and power. *Transparent huge pages* use 2 MB memory pages instead of 4 KB, which can speed up large programs. *ASPM* (Active State Power Management) lets PCIe links sleep when idle. An *I/O scheduler* orders disk requests.

TZM info (thermal zone buffer)

`tzm` - Linux data

In plain words. The ACPI thermal zones: temperatures the firmware itself reports, with its trip points.

What you see.

Column	What it means
#	Row number.
Device Name (Zone)	Zone type and number, e.g. `acpitz (0)`, `x86_pkg_temp (1)`.
Current	Temperature now.
Passive	Temperature at which the system starts slowing down to cool.
Critical	Temperature at which the system shuts down.
Policy	The kernel's thermal governor for the zone, e.g. `step_wise`.

Then `act N num N`. Refreshes every 2 seconds.

Healthy / typical values. Many desktops, including the author's, have no thermal zones at all: `act 0 num 0` and a note that CPU and GPU temperatures come from hwmon. Laptops usually have several (`acpitz`, `x86_pkg_temp`, `iwlwifi`).

Unusual values and what they suggest.

- Current near Passive: the system is about to throttle. Clean fans, check the cooler.
- A zone stuck at a fixed value like 27.8 C or 0 C: a dummy zone from the firmware. Ignore it.

Where Linux gets it. `/sys/class/thermal/thermal_zone*/{type,temp,policy,trip_point_*_type,trip_point_*_temp}`.

Background. A *thermal zone* is a firmware-defined area with a sensor. A *trip point* is a temperature where the system acts: *passive* = slow down, *critical* = shut down.

Words

`words` - Linux data

In plain words. The list of interface words SRS uses. SIV uses this to check translations; SRS is English only.

What you see. Language (`English (built in, no translation tables)`) and a comma list of the button and label words: OK, Copy, PCI Bus, USB Bus, ALL Dev, Lookup, Find, Data Files, About, Help, Refresh, Next, Prev, Highlight, Colour, Browse..., Latest, Ignore case, Literal, Highlight Matches, Menu, Run as root, Save.

XXB internal buffers

`xxb` - Linux data

In plain words. An index of the "buffer" items. In SIV these are internal data buffers. In SRS each is a live reading of a Linux source.

What you see. Columns Buffer, Description and srs-gui snapshot source:

Buffer	Description	Source
sgb-ini	System/board buffer	`/sys/class/dmi/id`, PCI 00:00.0, process affinity
sdm	Storage device monitor	`/sys/class/block/*/stat`
dsb	DIMM sensor buffer	hwmon spd5118 / jc42
bsi	Battery / HID sensor buffer	`/sys/class/power_supply`, `/sys/bus/hid/devices`
pmb	Power monitor buffer	hwmon power* / curr*
tzm	Thermal zone buffer	`/sys/class/thermal`
cgb	CPU group buffer	`/sys/devices/system/cpu/online`
cpu	CPU info buffer (in the CPU group)	`/sys/devices/system/cpu`, cpufreq
gpu	GPU info buffer	`/sys/class/drm`
stc	Thread contexts	`/proc/self/task`

A note: SRS keeps no shared buffers; each item reads its source when shown.

SRS - Command-Line Reference

Back to the User Guide - Section Reference - Glossary

I wanted a Linux version of SIV (System Information Viewer) for myself, so I built one and set out to improve on it along the way. SRS is free and open source for the Linux and open-source community. (SRS is an independent project, not affiliated with SIV or its author.) SIV is the work of Ray Hinchliffe, RH Software (rh-software.com).

In plain words

SRS has four programs:

Program	What it is
`srs-gui`	The window. **Given arguments, it behaves exactly like `sysreport-search`** (except arguments starting with `--g`, which are window options).
`sysreport-search`	The command line: text reports, device-ID lookup, report search, burn-in, benchmarks. Needs no desktop.
`srs-root`	The root helper that saves root-only data to `~/.cache/srs/root`.
`srs-sysreport`	Writes a full text report folder to `~/sysreports/`.

So `srs-gui -save` and `sysreport-search -save` do the same thing.

> **Note:** `sysreport-search --help` lists every command, including `-save`, `--list`, `burn`, `bench`, `ipi`, `latency`, `pi`, `crypto`, `prime`, `cachemem`, `storage`, `os-latency`, `compute`, `work`, `net`, `gpu`, `gpu-burn`, `gpu-compare`, `cuda-toolkit` and `update-lists`.

> **zsh users:** quote anything with square brackets: `sysreport-search '-save=[cpuid]'`. Without quotes zsh treats `[...]` as a file pattern and says "no matches found".

Quick list

```
sysreport-search -h | --help
sysreport-search --version
sysreport-search --list
sysreport-search -save[=FILE] [--redact]
sysreport-search '-save=[item][item]...[=FILE]' [--redact]
sysreport-search guide [ITEM] [--print]
sysreport-search id VVVV[:DDDD[:SSSS:ssss]][:RR] [--bus pci|usb] [--mode tree|scan|
info]
sysreport-search find PATTERN [--report DIR] [-i] [-F] [--max N] [--color auto|
always|never]
sysreport-search devices [--bus pci|usb|all]
sysreport-search burn [--threads N | --cpus 0,2,5] [--seconds S] [--mode same|swap]
[--limit C] [--accept-high-limit] [--cool S] [--cycles N]
sysreport-search bench ac|as|dd|dw|ec|ep|lc|lp|pc|pp [--seconds S] [--yes]
sysreport-search ipi [--cpus N] [--rounds R] [--yes]
sysreport-search latency [--max MB] [--yes]
sysreport-search pi [--digits 128k|1m|..|32m|16ki|..|1mi|..|32mi|N] [--threads N]
[--method chudnovsky|agm] [--save-digits FILE] [--yes]
sysreport-search crypto [--threads N] [--seconds S] [--yes]
sysreport-search prime [--limit 1e8|1e9|1e10|1e11|N] [--threads N] [--yes]
sysreport-search storage --dir FOLDER [--size MB] [--seconds S] [--profile
standard|light] [--allow-usb] [--yes]
sysreport-search cachemem [--threads N] [--seconds S] [--isa avx512|avx2|sse2] [--
mem-mib M] [--only mem,l1,l2,l3] [--tests read,write,copy,latency] [--mib] [--mem-
write auto|clzero|stream] [--yes]
sysreport-search os-latency [--samples N] [--tick-seconds S] [--only
syscall,switch,spawn,wake,timer,memory,files] [--yes]
sysreport-search compute [--threads N] [--seconds S] [--tests sgemm,dgemm,...] [--
code auto|c|sse2|avx2|avx512] [--single] [--yes]
sysreport-search work [--threads N] [--seconds S] [--only
compress,sort,hash,text,interp] [--libs zlib,zstd,lz4,xz] [--single] [--part-kib K]
[--yes]
sysreport-search net [--local] [--internet] [--seconds S] [--streams N] [--samples
N] [--max-mb MB] [-4|-6] [--server URL] [--hosts HOST[:PORT],...] [--yes] [--yes-
internet]
sysreport-search gpu [--device N] [--seconds S] [--only
fp32,fp16,fp64,int32,read,write,copy,upload,download,latency] [--mem-mib M] [--
list] [--yes]
sysreport-search gpu-burn [--seconds N] [--backend vulkan|hip|cuda|opencl|auto] [--
device N|all] [--fp32|--fp64] [--limit C] [--accept-high-limit] [--list] [--yes]
sysreport-search gpu-compare [--backends vulkan,hip,cuda,opencl] [--seconds S] [--
device N] [--yes]
sysreport-search cuda-toolkit [--check|--install|--ask-again|--never-ask]
sysreport-search update-lists [--list] [--yes]

srs-gui [--gui-open=WHAT[,WHAT...]] [--gui-tray] [--gui-classic | --gui-new]
srs-gui --power-recorder
srs-root [--user NAME]
srs-root --acpi-eval METHOD [ARG...]
srs-sysreport
```

Information commands (safe, read-only)

`--help, -h`

Prints the usage text: every command, `-save`, `--list`, `guide`, the load commands and the `srs-gui` start options. Exit code 0.

```
usage: sysreport-search [-h] [--version] {id,find,devices,-save,--
list,burn,bench,ipi,latency,pi,crypto,prime,cachemem,storage,os-
latency,compute,work,net,gpu,gpu-burn,gpu-compare,cuda-toolkit,update-lists} ...

SIV-style search for the native system report (srs-sysreport).

  id VVVV[:DDDD[:SSSS:ssss]][:RR] [--bus pci|usb] [--mode tree|scan|info]
        hierarchical device-ID lookup, most specific key first
  find PATTERN [--report DIR] [-i] [-F] [--max N] [--color auto|always|never]
        regex search across a report's sections with highlighted matches
  devices [--bus pci|usb|all]
        resolve every local PCI/USB device

System information items (SIV syntax; quote the argument in zsh/fish, [..] is a
glob):
  -save[=[item][item:arg]...][=FILE|STDOUT] [--redact]
        render items as text (no items = SIV's default set, [all] = every item)
        --redact masks serial numbers, UUIDs, MAC addresses (vendor prefix kept),
        boot/machine IDs and the host name, for reports you share
  --list
        every item: id, category, bucket (linux/absent/windows-only), group, title
  guide [ITEM] [--print]
        open the user guide at ITEM's section (PDF viewer, else the HTML guide);
        --print only shows the section and the command that would run

Benchmarks and Stress (load the CPU, the drive, the network or the GPU; run only
when you ask):
  burn [--threads N | --cpus 0,2,5] [--seconds S] [--mode same|swap] [--limit C]
[--accept-high-limit] [--cool S] [--cycles N]
        CPU burn-in with a temperature limit (default 10 s, limit 90 C)
  bench {ac,as,dd,dw,ec,ep,lc,lp,pc,pp} [--seconds S] [--yes]
        Dhrystone / Whetstone in a SIV benchmark mode (S = 0.1..60 s per
measurement);
        asks first, --yes starts at once
  ipi [--cpus N] [--rounds R] [--yes]
        core-to-core (cache-line ping-pong) latency matrix; asks first, --yes
starts at once
```

```
  latency [--max MB] [--yes]
        memory latency by block size, up to MB (1..4096, default 256); asks first,
--yes starts at once
  pi [--digits 128k|1m|..|32m|16ki|..|1mi|..|32mi|N] [--threads N] [--method
chudnovsky|agm] [--save-digits FILE]
     [--yes]
        SRS Pi: pi to N decimal places (default 1m, 1 thread), every digit checked;
        ki / mi = SuperPi sizes (powers of two): 1mi = 1,048,576 digits, 32mi =
33,554,432;
        asks first (with the time and the memory it needs), --yes starts at once
  crypto [--threads N] [--seconds S] [--yes]
        AES-128 and SHA-256 speed, plain C and with AES-NI / SHA-NI (default all
threads, 1 s);
        asks first, --yes starts at once
  cachemem [--threads N] [--seconds S] [--isa avx512|avx2|sse2] [--mem-mib M] [--
only mem,l1,l2,l3]
           [--tests read,write,copy,latency] [--mib] [--mem-write auto|clzero|
stream] [--yes]
        Cache & Memory Benchmark: Read / Write / Copy and Latency of the RAM and
L1-L3 (all threads,
        ~20 s; asks first, --yes starts at once)
  storage --dir FOLDER [--size MB] [--seconds S] [--profile standard|light] [--
allow-usb] [--yes]
        read / write speed of the drive holding FOLDER, on a temporary test file
(default 1024 MB,
        5 s per test); never touches existing files; asks first, --yes starts at
once
  os-latency [--samples N] [--tick-seconds S] [--only
syscall,switch,spawn,wake,timer,memory,files] [--yes]
        how quickly the OS answers: system calls, context switches, thread /
process start, wake-ups,
        timer lateness (with the CPU scheduler in use), page faults, file create /
stat / unlink
        (default 20000 samples, 3 s tick test; ~6 s on one or two CPUs; asks first,
--yes starts at once)
  compute [--threads N] [--seconds S] [--tests sgemm,dgemm,fma-sp,fma-dp,fft-
sp,fft-dp,nbody,mandelbrot,raytrace]
          [--code auto|c|sse2|avx2|avx512] [--single] [--yes]
        CPU Compute: matrix multiply, peak multiply-add, FFT, N-body, Mandelbrot,
ray tracer in GFLOPS /
        Mpixels/s, 1 thread and N threads (default all), every answer checked; ~10
s, all threads 100%
        busy (--single: one thread); asks first, --yes starts at once
  work [--threads N] [--seconds S] [--only compress,sort,hash,text,interp] [--libs
zlib,zstd,lz4,xz]
       [--single] [--part-kib K] [--yes]
        Everyday work: compression (system zlib / zstd / lz4 / xz), sorting,
checksums, text processing
        and a bytecode interpreter, 1 thread and N threads (default all), every
result checked;
        generated data only; asks first, --yes starts at once
```

```
  net [--local] [--internet] [--seconds S] [--streams N] [--samples N] [--max-mb
MB] [-4|-6]
      [--server URL] [--hosts HOST[:PORT],...] [--yes] [--yes-internet]
        Network: loopback TCP / UDP speed and round trip (default; nothing leaves
the computer);
        --internet adds DNS, connect, HTTPS, download and upload tests against
speed.cloudflare.com
        (up to ~1000 MB of data). Asks first; --yes answers the loopback question,
the internet test
        asks separately and only --yes-internet skips that question
  gpu [--device N] [--seconds S] [--only
fp32,fp16,fp64,int32,read,write,copy,upload,download,latency]
      [--mem-mib M] [--list] [--yes]
        GPU compute (Vulkan): FP32 / FP16 / FP64 / INT32 multiply-add (GFLOPS /
GIOPS), video memory
        read / write / copy and upload / download (GB/s), dispatch latency (us),
every result checked;
        ~15 s at full GPU load; Vulkan is loaded at run time; asks first, --yes
starts at once
  gpu-burn [--seconds N] [--backend vulkan|hip|cuda|opencl|auto] [--device N|all]
[--fp32|--fp64]
           [--limit C] [--accept-high-limit] [--list] [--yes]
        GPU Burnin: repeated matrix multiplications at full GPU load, compute
errors counted
        (0 = PASS), live temperature / power / clock; stops at the temperature
limit (default
        100 C), on a driver error or a GPU that stops answering; never changes
clocks, voltages
        or fans (default 60 s); asks first, --yes starts at once
  gpu-compare [--backends vulkan,hip,cuda,opencl] [--seconds S] [--device N] [--
yes]
        GPU Compute comparison: the same tests (matrix multiply with the own kernel
and the vendor
        library, multiply-add, copy, upload / download, launch time) through every
installed back end
        on the same GPU, side by side: fastest marked, % of Vulkan; asks first, --
yes starts at once
  cuda-toolkit [--check|--install|--ask-again|--never-ask]
        the optional NVIDIA CUDA toolkit (its runtime compiler, NVRTC): whether it
is installed, what it
        is for, the install command for this distribution; --install runs that
command after you type
        yes (needs a terminal; warns and asks twice when ZLUDA is in use). SRS's
CUDA tests are built
        in, so it is optional. --ask-again / --never-ask: whether the GUI may ask
about it
  update-lists [--list] [--yes]
        download fresh pci.ids, usb.ids and pnp.ids from their official sources
into
        ~/.local/share/srs/lists (no root); SRS then uses the newer of these and
the system's copies.
```

```
        Each download is checked first; a failed one leaves the copy in use. --list
shows the copies in
        use and their versions; asks first, --yes downloads at once
  prime [--limit 1e8|1e9|1e10|1e11|N] [--threads N] [--yes]
        count the primes up to the limit, checked against the known count (default
1e9, all threads);
        asks first, --yes starts at once

srs-gui only: --gui-open=ITEM[,ITEM|stressburn|burnin|stress|bench|ipi|latency|pi|
crypto|prime|cachemem|storagebench|oslatency|compute|work|netbench|gpubench|
gpuburn|gpucompare|cudatoolkit|loadterms|sensors|idlookup|find]
        open those panels / tools at start (also SRS_OPEN=...), --gui-tray start in
the tray
        --gui-classic start in the classic SIV-style main window, --gui-new in the
new one
        --gui-open=power-cost the Power & Cost page; --power-recorder record its
history with no window
```

`--gui-scan-empty` (not listed) is internal: the main window runs it in a child process to find the items that have nothing to show on this PC, only on Options > Check now or Check daily; it never runs the network items or Ping.

`--version`

```
sysreport-search 2.4.0 (ELF x86-64, gcc 16.2.1 20260810, glibc (system C runtime))
Inspired by SIV64X (System Information Viewer) by Ray Hinchliffe, RH Software (rh-
software.com),
built to improve on it; free for the Linux and open-source community.
Independent project, not affiliated with SIV or its author. GPL-2.0-or-later.
Home page: https://github.com/eugenesflanagin-spec/srs-sysinfo
```

The part in brackets describes how your copy was built.

`--list`

Lists every item SRS knows (558 lines), one per line:

```
about                tuning    linux         siv      About SRS
acpi-buses           hardware  linux         machine  ACPI PCI root buses
acpi-ipf             hardware  absent        machine  ACPI IPF (Intel Platform
Framework) methods
```

Column	Meaning
1	Item id - use it in `-save=[id]` and `--gui-open=id`
2	Category (SIV's: hardware, tuning, ...)
3	Bucket: `linux` (real Linux data), `absent` (hardware most PCs lack), `windows-only`
4	Group (the Menu group: cpu, machine, sensors, storage, network, os, gpu, bus, tools, siv)
5	Title

What each item shows: Section Reference.

`-save` - text reports (SIV syntax)

Form	What it does
`-save`	SIV's default set of items, to the screen.
`-save=FILE`	Default items, to FILE.
`-save=[id1][id2]`	Chosen items, to the screen.
`-save=[id1][id2]=FILE`	Chosen items, to FILE.
`-save=[all]`	Every item (about 52,000 lines and 12 seconds on the author's PC).
`-save=...=STDOUT`	The file name `STDOUT` means the screen.

- `-SAVE` works too (case does not matter), as in SIV.
- Some items take an argument after a colon, the same as the panel buttons: `[processor:4]` (CPU-4), `[cache:2]`, `[volumes:/]`, `[ntfs:/]`.
- An unknown id prints `No such item` for that item (exit code still 0).
- `--redact` (anywhere on the command line) masks identifiers for reports you share: serial numbers (board, system, chassis, DIMM, disk, NVMe, USB, monitor), UUIDs (file systems, partitions, the system UUID, the root file system in the kernel command line), MAC addresses (the vendor half is kept: `04:7c:16:XX:XX:XX`), WWNs, the GPU and Thunderbolt unique IDs, the boot and machine IDs, and the host name. Letters and digits become `X` and the length is kept, so columns stay aligned; the header says `(identifiers redacted)`. Published constants such as GPT partition-type GUIDs are left as they are. Without `--redact` the report is complete, as in SIV. IP addresses and Wi-Fi network names are not masked.
- Saving never starts the burner or a benchmark. Benchmark items print their last stored result.
- Root-only values come from `~/.cache/srs/root` (run `srs-root` first). Run as root (`sudo sysreport-search -save`) it uses the invoking user's cache, or collects one first.

Examples (all verified):

```
sysreport-search -save | less
sysreport-search '-save=[cpuid][cache]=cpu.txt'
srs-gui '-save=[sensors]'
```

```
sysreport-search '-save=[all]=full-report.txt'
sysreport-search '-save=[all]=for-forum.txt' --redact
```

Output starts with a header and then each item:

```
srs-gui - System Report Search V2.4.0 MYPC 2026-10-26 08:28:53  [[cpuid][cache]]

[cpuid]  [hardware]
...
```

`guide` - open the user guide at an item

```
sysreport-search guide [ITEM] [--print]
```

Opens this guide at ITEM's section (no ITEM: the start). With Okular, Evince or Papers as the default PDF viewer the PDF opens at the section; with any other viewer the HTML copy of the guide (`/usr/share/doc/srs-sysinfo/html/SRS-User-Guide.html`) opens in the web browser at the section. `--print` only prints the section and the command that would run:

```
$ sysreport-search guide sensors --print
Item     sensors
Section  sections-sensors--sensor-status
Command  'okular' '/usr/share/doc/srs-sysinfo/SRS-User-Guide.pdf#sections-sensors--
sensor-status'
```

Exit code 0, 1 when the guide is not installed, 2 for an unknown item. In `srs-gui` the same is the **Guide** button on every panel, **F1** and the panel's right-click menu.

`id` - device-ID lookup

Turns PCI or USB numbers into names, using hwdata's `pci.ids` / `usb.ids` (or systemd-hwdb).

Argument / option	Meaning	Default
`VVVV`	Vendor ID (4 hex digits)	required
`:DDDD`	Device ID	
`:SSSS:ssss`	Subsystem vendor and subsystem device (the card maker's IDs)	
`:RR`	Revision (2 hex digits)	
`--bus pci|usb`	Which list	`pci`

Argument / option	Meaning	Default
`--mode tree|scan|info`	`tree`: load the sorted list once, binary search (fast). `scan`: read the file line by line. `info`: ask systemd-hwdb, the system's device database.	`tree`

Options can be written `--bus usb` or `--bus=usb`.

```
$ sysreport-search id 1002:7550
  < 1002:7550               Navi 48 [Radeon RX 9070/9070 XT/9070 GRE]
    1002                    Advanced Micro Devices, Inc. [AMD/ATI]
Vendor     Advanced Micro Devices, Inc. [AMD/ATI]
Stock      Navi 48 [Radeon RX 9070/9070 XT/9070 GRE]
Resolved   Navi 48 [Radeon RX 9070/9070 XT/9070 GRE]
tree mode: 42,228 records loaded in 9.4 ms, lookup 0.002 ms
```

- Each key tried is listed, most specific first; `<` marks the one that matched. `(no entry)` means that exact key is not in the list.
- **Vendor** - the maker. **Subsystem** - the card maker, when a subsystem ID is given and known (e.g. `1002:7550:1EAE:7550:C0` -> `XFX Limited`). **Stock** - the chip's plain name. **Resolved** - the most specific name found.

```
$ sysreport-search id 046d:c52b --bus usb
  < 046D:C52B               Unifying Receiver
    046D                    Logitech, Inc.
...
$ sysreport-search id 1002:7550 --mode info
Query      pci:v00001002d00007550
Vendor     Advanced Micro Devices, Inc. [AMD/ATI]
Resolved   Navi 48 [Radeon RX 9070/9070 XT/9070 GRE]
info mode (systemd-hwdb) in 5.00 ms
```

To find the IDs of your own devices: `sysreport-search devices`, or `lspci -nn` / `lsusb`.

`devices` - every PCI and USB device, named

Option	Meaning	Default
`--bus pci|usb|all`	Which buses	`all`

```
$ sysreport-search devices --bus usb
[USB] 40 devices, 23,964 records (/usr/share/hwdata/usb.ids)
Slot           ID                          Matched
Vendor                       Resolved
1-2.4          0BDA:5411                   0BDA:5411            Realtek
Semiconductor Corp.  RTS5411 Hub
```

```
1-2.4.5        0BDA:1100                  0BDA                  Realtek
Semiconductor Corp.  Unknown Device
```

- **Slot** - where it is plugged in (PCI address, or USB bus-port path).
- **ID** - its full ID. **Matched** - the most specific key found in the list (`0BDA` alone means only the vendor is known). **Resolved** - the name, or "Unknown Device" when the list does not have it yet (update the `hwdata` package).

`find` - search a full system report

Searches every section file (`*.txt`, `lshw.html`) of a folder made by `srs-sysreport`.

Argument / option	Meaning	Default
`PATTERN`	A regular expression (Python-style), unless `-F`	required
`--report DIR`	Which report folder	the newest in `~/sysreports`
`-i`, `--ignore-case`	Ignore upper/lower case	off
`-F`, `--fixed`	Literal text, not a regular expression	off
`--max N`	Show at most N matches per section	20
`--color auto|always|never`	Highlight matches (auto = only on a terminal)	auto

```
 $ sysreport-search find 'RX 9070' --max 3 --color never
Searching /home/user/sysreports/sysreport-20261017-050453 for "RX 9070"

== devices.txt - 1 Match
     30: 0000:03:00.0   1002:7550:1EAE:8811:C0     1002:7550            Advanced
Micro Devices, Inc. Navi 48 ...

== inxi.txt - 3 Matches
     59:   Device-1: Advanced Micro Devices [AMD/ATI] Navi 48 [Radeon RX 9070/9070
...
7 Matches in 4 of 16 sections
```

Numbers on the left are line numbers in that section file. When a section has more matches than `--max`, a line says `... N more (raise --max)`.

Load and benchmark commands (make the CPU busy)

`burn` starts at once: it is the burn-in itself, and you choose its threads, time and temperature limit on the command line. **Every benchmark asks first** - `bench`, `ipi`, `latency`, `pi`, `crypto`, `prime` and the ones below - on the terminal (stderr): what it runs, about how long, how many CPU threads it keeps busy and (SRS Pi) how much memory it needs, then `Start now? [y/N]`. Only `y` starts it. `--yes` (or `-y`) starts it without the question, for scripts. Without a terminal and without `--yes` it does not start and exits with 2. `--help` (`-h`) prints the command's usage and exits 0; an unknown option or a missing value exits 2 - neither starts anything. The windows ask the same question. Close other heavy programs first. Press **Ctrl+C** to stop. Never run them on a PC you are not allowed to load.

Use at your own risk (2.3.1). These commands push the CPU, GPU, memory, drives and power supply to their limits; SRS comes without any warranty, and its authors are not responsible for any damage (see the licence, GNU GPL v2). The question (`Start now (and agree)? [y/N]`), every load command's `--help` and `burn` print that notice. **`--yes` counts as agreeing to it** (so do answering `y` and starting `burn`). The window asks once with I Agree / I Disagree.

```
$ sysreport-search crypto --threads 1
It checks the official AES and SHA-256 test vectors, then measures AES-128
encryption and SHA-256
speed with the CPU's crypto instructions and in plain C, 1.0 s per measurement:
about 8 seconds.
Each test runs on one CPU thread (light load).

Close other programs for accurate results.
Use at your own risk: SRS comes without any warranty, and its authors are not
responsible for any damage (see the licence, GNU GPL v2).
Start now (and agree)? [y/N] n
crypto: not started
```

`burn` - CPU Burnin from the command line

`burn --help` (new in 2.3.1) prints the options and the notice. In the window this is the CPU part of **Stress & Burnin**; the GPU part is `gpu-burn` below.

Option	Meaning	Default
`--threads N`	Use logical CPUs 0..N-1	all
`--cpus 0,2,5`	Use exactly these logical CPUs	all
`--seconds S`	Burn time per cycle, 1 to 2,700,000 seconds	10
`--mode same|swap`	`same`: each worker stays on its CPU and runs Whetstone. `swap`: workers move between CPUs (affinity test).	same
`--limit C`	Stop if the hottest CPU sensor reaches C degrees Celsius: 50-100, or 101-110 with `--accept-high-limit` (2.4; any other value exits 2 without starting)	90

Option	Meaning	Default
`--accept-high-limit`	Allow a `--limit` of 101-110 °C. Above 100 °C many CPUs and GPUs are already at or near their own maximum; using it means you accept that risk (the window asks the same once, Yes / No)	off
`--cool S`	Rest between cycles, seconds	0
`--cycles N`	Number of burn + cool cycles	1

Output: a start line, then one line per second (example values):

```
    12s  load 100.0%  Tctl 78.5 C  CPUTIN 33.0 C  CPU 5050 MHz  GPU 3% 28.0 C
Burning
```

(elapsed, CPU load, hottest core sensor, hottest motherboard sensor, CPU clock, GPU load and temperature, state), then a table per worker (Thread, CPU, Passes, MWIPS last / average / min / max, Affinity `ok` or `FAIL`), the maxima and the final status. The last line names the file the summary was saved to (`Summary saved to /home/you/.cache/srs/burnin-YYYYMMDD-HHMMSS.txt`, one file per run; the same layout as srs-gui's Burnin window).

Exit codes: **0 = Pass**, **1 = Fail** (affinity error or temperature limit), **130 = stopped** (Ctrl+C), **2 = bad option or nothing to run**.

An unknown option prints the usage line and exits 2 **without starting anything**:

```
$ sysreport-search burn --nope
usage: sysreport-search [-h] [--version] {id,find,devices} ...
sysreport-search: error: burn [--threads N | --cpus 0,2,5] [--seconds S] [--mode
same|swap] [--limit C] [--accept-high-limit] [--cool S] [--cycles N]
```

A limit over 100 °C without `--accept-high-limit` is refused the same way (exit 2, nothing started):

```
$ sysreport-search burn --limit 105
burn: a --limit over 100 C needs --accept-high-limit (above 100 C many CPUs and
GPUs are already at or near their own maximum; using it means you accept that risk)
```

`bench` - Dhrystone and Whetstone

```
sysreport-search bench MODE [--seconds S] [--yes]
```

Mode	What runs
`ac`	All CPUs concurrently: Dhrystone then Whetstone on every logical CPU at once

Mode	What runs
`as`	All CPUs sequentially, one logical CPU at a time
`dd`	Per core: both SMT threads run Dhrystone, then both run Whetstone
`dw`	Per core: one thread Dhrystone while its sibling runs Whetstone, then swap
`ec` / `ep`	Efficiency cores concurrently / one at a time (Intel hybrid CPUs; Linux lists the E-cores in `/sys/devices/cpu_atom/cpus`)
`lc` / `lp`	Logical (SMT sibling) threads concurrently / one at a time
`pc` / `pp`	Performance (physical) cores, one thread per core, concurrently / one at a time (E-cores left out)

`--seconds S` is the time per measurement, 0.1 to 60 (default 1). **Asks first** (the mode, the time and how many CPU threads are busy; the modes that load many threads at once also warn about unstable overclocks); `--yes` starts at once. The result is stored and shown in the window and in `-save` reports. Results: DMIPS (Dhrystone) and MWIPS (Whetstone), per CPU and in total; see tools.md. Exit code 0 when it ran, 1 when there is nothing to run in that mode on this CPU (for example `ec` without efficiency cores; then there is no question either), 2 for a bad mode or option, or when it was not started.

`ipi` - core-to-core delay

```
sysreport-search ipi [--cpus N] [--rounds R] [--yes]
```

`--cpus N` tests CPUs 0..N-1 (default all), `--rounds R` round trips per pair (default 20,000). Two CPU threads are busy at a time, for about a second at the defaults. **Asks first**; `--yes` starts at once. Prints a matrix of nanoseconds, one row and column per CPU. With one CPU (`--cpus 1`, or a one-CPU system) there is no pair to measure: it says so, exits with 2 and keeps the last saved result. See tools.md.

`latency` - memory latency by block size

```
sysreport-search latency [--max MB] [--yes]
```

`--max MB` is the largest block, 1 to 4096 MB (default 256; the window allows the same range). It uses that much memory and one CPU thread, for about 3 seconds at the default. **Asks first**; `--yes` starts at once. Prints block size, nanoseconds per read and the cache level. See tools.md.

`pi` - SRS Pi: pi to many digits, every digit checked

```
sysreport-search pi [--digits 128k|1m|..|32m|16ki|..|1mi|..|32mi|N] [--threads N]
[--method chudnovsky|agm] [--save-digits FILE] [--yes]
```

`--digits` is the number of decimal places (default `1m` = 1,000,000; `k` = thousand, `m` = million; `ki` = 1,024 and `mi` = 1,048,576 for the **SuperPi sizes** 16ki ... 32mi; 100 to 32mi). The listed sizes are checked digit for digit with built-in SHA-256 checksums; other sizes only have their first 100 digits checked. `--threads N` (default 1) splits the Chudnovsky series over N threads; `--method agm` uses Gauss-Legendre (always one thread). `--save-digits FILE` writes the digits (without "3.") to FILE. **Asks first**, with the time, the threads and the memory it needs (up to about 1.6 GB for 32mi digits); from 8 million digits, or with all threads, it also warns about unstable overclocks. `--yes` starts at once. Exit code 0 when every checked digit is right, 1 when a digit is wrong or memory ran out, 2 for a bad option or when it was not started. See tools.md.

`crypto` - AES-128 and SHA-256 speed

```
sysreport-search crypto [--threads N] [--seconds S] [--yes]
```

Measures AES-128-CTR and SHA-256 in MB/s on one thread and on N threads (default all), with AES-NI / SHA-NI when the CPU has them and in plain C. `--seconds S` per measurement, 0.1 to 60 (default 1), about 8 measurements in all. **Asks first** (with all threads it warns that every CPU thread is 100% busy); `--yes` starts at once. The FIPS test vectors are checked first; exit code 1 if they fail, 2 for a bad option or when it was not started. See tools.md.

`prime` - count the primes up to a limit

```
sysreport-search prime [--limit 1e8|1e9|1e10|1e11|N] [--threads N] [--yes]
```

Counts the primes up to `--limit` (default 1e9; any number from 100 to 1e11, for example `5e9` or `1g`) on N threads (default all) and checks the count for the powers of ten 1e7 to 1e11. **Asks first** (from 1e10, or with all threads, it also warns about unstable overclocks); `--yes` starts at once. Exit code 1 when the count is wrong, 2 for a bad option or when it was not started. See tools.md.

`cachemem` - Cache & Memory Benchmark

```
sysreport-search cachemem [--threads N] [--seconds S] [--isa avx512|avx2|sse2] [--
mem-mib M]
                          [--only mem,l1,l2,l3] [--tests read,write,copy,latency]
[--mib]
                          [--mem-write auto|clzero|stream] [--yes]
```

Read / Write / Copy (MB/s, caches in GB/s) and Latency (ns) of the RAM and the L1, L2 and L3 caches. It loads every CPU thread and the memory bus for about 20 seconds and **asks first**; `--yes` (`-y`) starts at once (scripts). Without a terminal and without `--yes` it does not start (exit code 2).

Option	Meaning
`--threads N`	Threads for the bandwidth tests (default all logical CPUs)
`--seconds S`	Measuring time per value, 0.1-60 (default 1)
`--isa`	Force the vector code (default: the best the CPU has)
`--mem-mib M`	RAM block in MiB, all threads together (default 1024, at least 4 x L3)
`--only` / `--tests`	Run only some rows / columns, e.g. `--only l1 --tests read`
`--mib`	Binary units (MiB/s, GiB/s)
`--mem-write`	RAM write method: `auto` (CLZERO on AMD CPUs that have it), `clzero`, `stream`

x86-64 only. Exit code 0 when it ran, 1 if memory could not be allocated, 2 for a bad option or no confirmation. See tools.md.

`storage` - drive read / write speed

```
sysreport-search storage --dir FOLDER [--size MB] [--seconds S] [--profile
standard|light] [--allow-usb] [--yes]
```

Tests the drive that holds FOLDER on a new temporary test file (`--size`, 16-65536 MB, default 1024) that is unlinked at once and gone when the test ends - existing files are never touched. Six tests of `--seconds S` each (0.5-60, default 5): sequential 1 MiB Q8T1, 4K random Q32T16 (`--profile light`: Q8T4) and 4K random Q1T1, read and write; average, peak, IOPS and 4K Q1T1 response times. Refuses RAM (tmpfs), network and FUSE folders and too little free space; USB drives only with `--allow-usb`. **Asks first**; `--yes` starts at once. Exit code 0 when it ran, 1 when it was stopped or hit an I/O error, 2 when it did not start (the message says why). See tools.md.

`os-latency` - how quickly the OS answers

```
sysreport-search os-latency [--samples N] [--tick-seconds S]
                            [--only syscall,switch,spawn,wake,timer,memory,files]
[--yes]
```

Times many single samples of system calls, pipe context switches (threads and processes, same CPU and two cores), thread create, fork and posix_spawn, futex and condition-variable wake-ups, sleep overshoot and a periodic 1 ms tick, page faults (4 KiB and 2 MiB), mmap / munmap, and small-file create / stat / unlink. Each row shows the median, 95%, 99%, worst and best. The CPU scheduler in use (sched_ext such as scx_lavd, BORE, EEVDF), the preemption mode, the timer tick, the clock source and the timer slack are

shown with the result. It keeps one or two CPUs busy for about 6 seconds at the defaults and **asks first**; `--yes` (`-y`) starts at once. Without a terminal and without `--yes` it does not start (exit code 2).

Option	Meaning
`--samples N`	Samples for the fast tests, 100-1,000,000 (default 20,000); the slow tests use a tenth to a fortieth
`--tick-seconds S`	How long the 1 ms tick is watched, 0.1-60 (default 3)
`--only`	Run only some groups, e.g. `--only syscall,switch`

Nothing on the system is changed. The file test uses a temporary folder in `~/.cache/srs` that is always removed. Exit code 0 when it ran, 1 when it was stopped, 2 when it did not start (bad option, not confirmed, not available). Example:

```
$ sysreport-search os-latency --only syscall,timer --yes
```

See tools.md.

`compute` - CPU Compute (floating-point work)

```
sysreport-search compute [--threads N] [--seconds S]
                         [--tests sgemm,dgemm,fma-sp,fma-dp,fft-sp,fft-
dp,nbody,mandelbrot,raytrace]
                         [--code auto|c|sse2|avx2|avx512] [--single] [--yes]
```

Matrix multiply (SGEMM / DGEMM), peak multiply-add, FFT, N-body, Mandelbrot and a small ray tracer, in GFLOPS or Mpixels/s, on 1 thread and on N threads, with the best vector code the CPU has. Every answer is checked; a wrong one is shown as WRONG. With N threads every CPU is 100% busy (AVX-512 where the CPU has it), so it **asks first**; `--yes` (`-y`) starts at once. Without a terminal and without `--yes` it does not start (exit code 2). An unstable overclock or undervolt can crash the PC under this load; `--single` keeps it light.

Option	Meaning
`--threads N`	Threads for the right-hand column (default 0 = all logical CPUs)
`--seconds S`	Measuring time per test and column, 0.1-60 (default 0.5)
`--tests`	Run only some tests, e.g. `--tests dgemm,mandelbrot`
`--code`	Force a code path (a path the CPU lacks falls back to the best it has)
`--single`	Only the 1-thread column (light load, half the time)

Exit code 0 when every check passed, 1 when a check failed (WRONG) or it was stopped, 2 when it did not start (bad option, not confirmed, Stress & Burnin running). Example:

```
$ sysreport-search compute --single --tests sgemm,nbody --yes
```

See tools.md.

`work` - Everyday work

```
sysreport-search work [--threads N] [--seconds S] [--only
compress,sort,hash,text,interp]
                      [--libs zlib,zstd,lz4,xz] [--single] [--part-kib K] [--yes]
```

Compression and decompression with the system's zlib, zstd, lz4 and xz libraries (MB/s and ratio), sorting (Mkeys/s), checksums and hashes (GB/s), text processing (GB/s) and a bytecode interpreter (Mops/s), on 1 thread and on N threads; every result is checked. The data is generated by SRS (your files are never read or written). It keeps every CPU busy and **asks first** with the expected time; `--yes` (`-y`) starts at once. Without a terminal and without `--yes` it does not start (exit code 2). A library that is not installed shows "not installed" and its rows are skipped.

Option	Meaning
`--threads N`	Threads for the right-hand column (default 0 = all logical CPUs)
`--seconds S`	Measuring time per test and column, 0.1-60 (default 0.5)
`--only`	Test groups, e.g. `--only compress,hash`
`--libs`	Compression libraries, e.g. `--libs zstd,lz4`
`--single`	Only the 1-thread column
`--part-kib K`	Size of each generated data part per thread, 64-16384 KiB (default 1024)

Exit code 0 when every check passed, 1 when a check failed or it was stopped, 2 when it did not start (bad option, not confirmed, Stress & Burnin running, not enough memory). Each test group adds a line (`work-compress` … `work-interp`) to the history. Example:

```
$ sysreport-search work --only compress --libs zstd,lz4 --single --yes
```

See tools.md.

`net` - Network Benchmark

```
sysreport-search net [--local] [--internet] [--seconds S] [--streams N] [--samples
N] [--max-mb MB]
```

```
                    [-4|-6] [--server URL] [--hosts HOST[:PORT],...] [--yes] [--
yes-internet]
```

Without options: the loopback test only - TCP throughput with 1 and N streams, UDP throughput and loss, TCP and UDP round trip time between two programs on this PC (127.0.0.1). Nothing leaves the computer. It **asks first**; `--yes` (`-y`) starts the loopback test at once.

`--internet` adds DNS, connect, HTTPS, download and upload tests against `https://speed.cloudflare.com`. Before it starts it prints the server, the hosts, the tests, the time and the data use (up to about 4 x `--max-mb`, 1000 MB with the defaults) and says the server sees your public IP address. **It asks its own question, which `--yes` does not answer**: type `y` at the prompt, or pass `--yes-internet` (scripts). Without a terminal and without `--yes-internet` the internet test does not start (exit code 2). `--internet` alone runs only the internet part; `--local --internet` runs both.

Option	Meaning
`--local`	The loopback test (the default when neither `--local` nor `--internet` is given)
`--internet`	The internet tests (asks separately)
`--seconds S`	Seconds per throughput test, 0.1-60 (default 5); round trip tests use at most 1 s
`--streams N`	Connections for the "N streams" tests, 1-32 (default 4)
`--samples N`	TCP connects per host for the connect-time test, 1-100 (default 10)
`--max-mb MB`	Each transfer test stops after this many MB (default 250, 0 = no cap)
`-4` / `-6`	IPv4 only / IPv6 only
`--server URL`	Another speed test server with the same `/__down` and `/__up` paths (https only; a port in the URL is used for its connect-time row too)
`--hosts H,H`	Extra hosts for the connect-time test (`host` or `host:port`); one typed as a numeric address is printed and saved as "host N (a numeric address)"
`--yes`	Start the loopback test without asking
`--yes-internet`	Start the internet test without asking

Without libcurl the DNS and connect tests still run and the HTTPS and speed rows say "not available". No IP address, MAC address, network name or host name is printed or saved. Exit code 0 when it ran, 1 when it was stopped or a test failed, 2 when it did not start (bad option, not confirmed, no network connection for the internet test). Examples:

```
$ sysreport-search net --yes                       # loopback only
$ sysreport-search net --internet --seconds 3 --max-mb 50   # asks; at most about
200 MB
```

See tools.md.

`gpu` - GPU Benchmark (Vulkan)

```
sysreport-search gpu [--device N] [--seconds S] [--only
fp32,fp16,fp64,int32,read,write,copy,upload,download,latency]
                     [--mem-mib M] [--list] [--yes]
```

Measures the graphics card through Vulkan: FP32, FP16, FP64 and INT32 multiply-add (GFLOPS / GIOPS), video memory read, write and copy, upload and download (GB/s, 1 GB = 10^9 bytes) and the round trip of an empty dispatch (µs). Every result is checked; a wrong one is printed as WRONG RESULT. The Vulkan loader is opened at run time: without it, or without a Vulkan driver, the command says so and exits with 2. It **asks first** (the GPU, the time, full GPU load, and that an unstable overclock or undervolt can crash the PC); `--yes` (`-y`) starts at once. Without a terminal and without `--yes` it does not start (exit code 2).

Option	Meaning
`--device N`	The Vulkan device to test, numbered as `--list` shows them (default: the first discrete GPU, else the first device)
`--seconds S`	GPU time per test, 0.1-10 (default 1)
`--only LIST`	Run only these tests (comma-separated): fp32, fp16, fp64, int32, read, write, copy, upload, download, latency
`--mem-mib M`	Buffer size of the memory and transfer tests in MiB, 1-4096 (default: 256, less on small cards); a size above a quarter of the video memory or of the PC's memory is lowered to that, and the result says so
`--list`	List the Vulkan devices (number, name, type, video memory, driver) and exit; nothing runs on a GPU (like `vulkaninfo`, this creates a Vulkan instance to name the devices)
`--yes`	Start without asking

Exit code 0 when every test ran and was right, 1 when it was stopped, a result was wrong or the GPU failed during the run, 2 when it did not start (bad option, not confirmed, no Vulkan, no such device, Stress & Burnin running). Examples:

```
$ sysreport-search gpu --list
$ sysreport-search gpu --only fp32,read,upload --seconds 0.5
```

See tools.md.

`gpu-burn` - GPU Burnin

```
sysreport-search gpu-burn [--seconds N] [--backend vulkan|hip|cuda|opencl|auto] [--
device N|all]
```

```
                          [--fp32|--fp64] [--limit C] [--accept-high-limit] [--
list] [--yes]
```

Keeps the GPU fully busy with large matrix multiplications and counts every result that differs from the first one (0 = PASS). On a terminal a live line shows the time, GFLOPS, the error count, edge and hotspot temperature, power and clock; the full result follows at the end. It **asks first** (the GPU, the back end, the time, high power and heat, the overclock / undervolt warning, the temperature limit); `--yes` (`-y`) starts at once; without a terminal and without `--yes` it does not start (exit code 2). Before you say yes it starts no HIP, CUDA or OpenCL runtime; to name the GPU in the question it reads only Vulkan's device list (like `vulkaninfo`, no work is sent). It ends by itself at the temperature limit, on a driver error, when the GPU stops answering or when a piece of work takes more than 2 s; Ctrl+C stops it. A driver call that never comes back is waited for 15 s after the end, then the run ends ("the GPU driver did not come back"). A card that reports no temperature is shown with "no temperature sensor: limit not active". **It never changes clocks, voltages or fans.** It never burns a software renderer such as llvmpipe as a GPU: the default and `--device all` skip it, and with only a software renderer in Vulkan it does not start (exit 2).

Option	Meaning
`--seconds N`	How long, 1-86400 (default 60)
`--backend B`	`vulkan`, `hip` (ROCm), `cuda`, `opencl` or `auto` (default: Vulkan when it has a GPU, else the first back end that has one)
`--device N`	The device number in that back end's list (see `--list`), or `all` (default: the first GPU; CPU / software devices are skipped)
`--fp32` / `--fp64`	Precision of the multiplications (default FP32)
`--limit C`	Stop at this GPU temperature in °C, 50-100 (default 100; hotspot / junction when the card reports it, else edge); 101-110 only with `--accept-high-limit` (2.4; before 2.4 the range was 50-120). `--limit-c` is the same
`--accept-high-limit`	Allow a `--limit` of 101-110 °C (above 100 °C many GPUs are already at or near their own maximum; you accept that risk)
`--list`	List the devices of each installed back end (or only `--backend`'s) and exit. This starts each runtime but runs no work
`--yes`	Start without asking

With `--device all`, a card that cannot start is marked in the table and a `note -` line says how many; the exit code comes from the cards that ran (all of them failing to start = 1, ended early). Exit code 0 = PASS, 1 = FAIL (errors found), stopped, or ended early by a safety rule, 2 = not started (bad option, not confirmed, back end not installed, no device, Stress & Burnin running). Examples:

```
$ sysreport-search gpu-burn --list
$ sysreport-search gpu-burn --seconds 600 --limit 95
$ sysreport-search gpu-burn --backend hip --fp64 --seconds 120
$ sysreport-search gpu-burn --seconds 900 --limit 105 --accept-high-limit
```

Since 2.4 every back end uses a register-blocked matrix multiply (64 x 64 tiles, 4 x 4 results per thread) that loads the GPU much harder; the result's `Kernel` line says which kernel ran, and `SRS_GPUBURN_SIMPLE=1` forces the older one.

See tools.md.

`gpu-compare` - GPU Compute: Vulkan vs HIP vs CUDA vs OpenCL

```
sysreport-search gpu-compare [--backends vulkan,hip,cuda,opencl] [--seconds S] [--device N] [--yes]
```

Runs the same tests (matrix multiply FP32 / FP16 / FP64 with SRS's own kernel and with the vendor library - hipBLAS, cuBLAS, CLBlast -, FP32 / FP64 multiply-add chain, video memory copy, upload, download, kernel launch round trip) through every chosen back end that is installed, on the same GPU, and prints one table: `*` = fastest in the row, (N%) = that column compared with Vulkan. CUDA that is really ZLUDA is labelled "CUDA (ZLUDA)"; every OpenCL driver is its own column; a back end that is not installed shows "not installed" and a hint. It **asks first**; `--yes` (`-y`) starts at once; without a terminal and without `--yes` it does not start (exit code 2). HIP, CUDA and OpenCL start their runtimes only after that. Ctrl+C stops it. CUDA runs SRS's built-in kernels (PTX, since 2.3), so it needs no runtime compiler (NVRTC); the CUDA column's software line says so. When CUDA was used and NVRTC is missing, one note at the end gives the `cuda-toolkit` command and the menu path (not after "Don't ask again"). An OpenCL driver without FP64 shows its FP64 rows as "not available: this OpenCL driver has no FP64 (double precision)". When the GPU hangs or stops answering in one column, the run ends there: no later back end touches that GPU, and the tests that did not run say "not run".

Option	Meaning
`--backends L`	Comma-separated: `vulkan`, `hip`, `cuda`, `opencl` (default: every installed one). `--backend B` is the same
`--seconds S`	GPU time per test, 0.1-10 (default 1)
`--device N`	The Vulkan device that fixes "the GPU" (numbers from `gpu --list`); the others use the card at the same PCI address (default: the first discrete GPU)
`--yes`	Start without asking

Exit code 0 when it ran and every result was right, 1 when it was stopped, a result was wrong or a GPU failed, 2 when it did not start (bad option, not confirmed, none of the chosen back ends installed). Example:

```
$ sysreport-search gpu-compare --backends vulkan,hip --seconds 0.5
```

See tools.md.

`cuda-toolkit` - the optional NVIDIA CUDA toolkit

```
sysreport-search cuda-toolkit [--check|--install|--ask-again|--never-ask]
```

In plain words: the NVIDIA CUDA toolkit is NVIDIA's free set of developer tools for NVIDIA graphics cards. SRS can use one small part of it, the runtime compiler (NVRTC), but since 2.3 its CUDA tests are built in, so most people do not need it. This command explains that, shows what is installed and the install command for your Linux, and can run it.

Option	Meaning
`--check`	(default) The explanation; NVIDIA card found or not; the CUDA library (NVIDIA's, ZLUDA, none); NVRTC installed (version, file) or not; the install command and its size (or instructions); a red-flag warning when ZLUDA is in use; whether the GUI will ask
`--install`	Checks first (no install command for this Linux, pacman's database locked, no network, no `sudo` / `pkexec`), needs a terminal, shows the exact command and the full paths of the programs it uses, and runs it with `sudo` only after you type `yes` - with ZLUDA you must type `yes` twice. Then reports the result and looks for NVRTC again
`--ask-again`	Let the GUI ask again before the next CUDA run (`Ask CUDA Toolkit=1` in `~/.config/srs/SRSDEF.ini`)
`--never-ask`	The GUI never asks, and `gpu-burn` / `gpu-compare` print no toolkit note (`Ask CUDA Toolkit=0`, the GUI's "Don't ask again")

Commands it offers: Arch-based `sudo pacman -S --needed cuda` (about 2.4 GB download, 5.2 GB on disk); Debian / Ubuntu `sudo apt-get install libnvrtc12` or `libnvrtc11.2` (about 20 MB), else `nvidia-cuda-toolkit`; Fedora, openSUSE and others: instructions for NVIDIA's CUDA repository (https://developer.nvidia.com/cuda-downloads, package `cuda-nvrtc-<version>`). With ZLUDA, installing the toolkit is not recommended (it can replace ZLUDA's cuBLAS on the library path, and NVIDIA's driver libraries conflict with ZLUDA). The package manager, `sudo` and `pkexec` are only taken from `/usr/bin`, `/usr/sbin`, `/bin` or `/sbin` and run by their full path; a program of the same name elsewhere in `PATH` (for example `~/.local/bin`) is never used, and without them in those folders SRS does not start the install and shows the command to type yourself. The GUI has the same under **Options -> NVIDIA CUDA toolkit for GPU tests...** (User Guide 9.20).

Exit code 0: shown, or installed and NVRTC found; 1: not started (with the reason) or the install failed; 2: not confirmed, no terminal, or a bad option.

`update-lists` - newer hardware name lists (2.4)

```
sysreport-search update-lists [--list] [--yes]
```

In plain words: downloads the newest PCI, USB and monitor maker name lists from their official sources, so new devices get names. Only when you run it; no root.

First it prints each list in use: file, version (or the number of entries for `pnp.ids`, which has no version), path and whether it is yours or the system's. Then it shows the sources - `pci.ids` from https://pci-ids.ucw.cz/v2.2/pci.ids, `usb.ids` from http://www.linux-usb.org/usb.ids, `pnp.ids` from hwdata (https://raw.githubusercontent.com/vcrhonek/hwdata/master/pnp.ids) - and asks `Go on? [y/N]`. The files go to `~/.local/share/srs/lists` (`$XDG_DATA_HOME/srs/lists` when that is set), downloaded with `curl` (or `wget`). Each must look like the real list (its header and enough entries) or the copy in use stays; the previous copy is kept as `.old`. From then on every lookup uses the newer of your copy and the system's (`/usr/share/hwdata`), by the list's own version.

Option	Meaning
`--list`	Only show the copies in use; download nothing
`--yes`	Download without asking

Exit code 0: at least one list was updated or checked; 1: not downloaded (answered no, no terminal, or every download failed); 2: a bad option. The window has the same on the Tools page (User Guide 9.21).

Every `cachemem`, `storage`, `os-latency`, `compute`, `work`, `net`, `gpu`, `gpu-burn`, `gpu-compare`, `pi`, `crypto` and `prime` run is added to `~/.cache/srs/bench-history.tsv`.

`srs-gui` window options

Option	What it does
(none)	Open the main window (the window with the sidebar, or the classic one when **Options -> Start in classic view** is on).
`--gui-open=WHAT[,WHAT...]`	Also open these panels or tools at start. In the main window an item opens on its page (for example `sensors` on the Sensors page); in the classic view it opens as its own panel.
`--gui-tray`	Start hidden in the system tray (used by the autostart entry).
`--gui-classic`	Start in the classic view: SIV's main window with its button row, every item in its own panel (as in SRS 1.1).
`--gui-new`	Start in the main window with the sidebar, even when **Start in classic view** is on.
`--power-recorder`	Record **Power & Cost** history with no window, following the page's Recording setting (Off / For a set time / Always). Used by the optional systemd user service that **Power & Cost -> Setup -> Keep recording when SRS is closed** writes. Only one process records at a time (a lock file beside the database); stop it with Ctrl+C or SIGTERM.
any other argument	Run as the command line (`srs-gui -save` = `sysreport-search -save`).

`WHAT` can be any item id from `--list` (e.g. `sensors`, `processors`, `pci-bus`, `processor:4`), or one of these tool names:

Name	Opens
`stressburn`	Stress & Burnin as last used (2.3.1)
`burnin`	Stress & Burnin set to Timed (CPU Burnin)
`stress`	Stress & Burnin set to Until Stop (CPU Stress)
`gpuburn`	Stress & Burnin with only GPU Burnin ticked
`loadterms`	Options -> Load test terms (warranty)...
`bench`	Benchmark
`ipi`	IPI Delay
`latency`	Latency
`pi`	SRS Pi
`crypto`	Crypto Benchmark
`prime`	Prime Sieve
`cachemem`	Cache & Memory
`storagebench`	Storage Benchmark (`storage` alone is the Storage page's item)
`oslatency`	OS Latency
`compute`	CPU Compute
`work`	Everyday Work
`netbench`	Network Benchmark (`network` alone is the Network page's item)
`gpubench`	GPU Benchmark
`gpucompare`	GPU Compute: Vulkan vs HIP vs CUDA vs OpenCL
`cudatoolkit`	The **Options -> NVIDIA CUDA toolkit for GPU tests...** window
`lists`	Update hardware lists (asks first) (2.4)
`acpieval`	ACPI method evaluation (the one-time warning first, if not accepted yet) (2.4)
`idlookup`	Device ID Lookup
`find`	Find
`menu`	Pops up the Menu
`power-cost`	Shows the Power & Cost page (the item `power` is the expert Power status tab on Sensors)

Opening a tool **never starts it**. Example: `srs-gui --gui-open=sensors,burnin`. The environment variable `SRS_OPEN=sensors,burnin` does the same.

The menu entries use: `--gui-open=stressburn` (SRS Stress & Burnin), `--gui-open=bench,ipi,latency` (SRS Benchmark), and the jump-list actions `sensors`, `processors`, `burnin`, `stress`, `bench`, `cachemem`, `storagebench`, `oslatency`, `compute`, `work`, `netbench`, `gpubench`, `gpuburn`, `gpucompare`, `pi`, `idlookup`, `find`, and **Classic View** (`--gui-classic`).

`srs-root`

```
pkexec srs-root            # or: sudo srs-root
sudo srs-root --user NAME  # collect for another user (used by the boot unit)
pkexec srs-root --acpi-eval METHOD [ARG...]   # evaluate one ACPI method (2.4; what
the window's ACPI method evaluation runs)
```

Collects root-only data into `~/.cache/srs/root/` of the user who started it (via `PKEXEC_UID` or the current user), then gives the files to that user. It prints one line per step:

```
srs-root: dmidecode
srs-root: decode-dimms
srs-root: smart
srs-root: acpi-tables
srs-root: lspci
srs-root: msrs
srs-root: rapl
srs-root: umc
srs-root: aod
srs-root: ebda
srs-root: vbios
srs-root: gpio
srs-root: done -> /home/you/.cache/srs/root
```

Since 2.4 it also collects (each only where it applies; files in `~/.cache/srs/root/`):

File	What
`umc.txt`	AMD Ryzen 7000 / 9000 desktop: memory controller (UMC) registers, read through the ryzen_smu driver's `smn` file (an address written there only selects what is read: 4 bytes, never the 8-byte form that writes; each value read twice under `/run/lock/srs-smn.lock`). Without the driver: a "not read" line
`intel-imc.txt`	Intel Core 6th gen to Core Ultra (experimental): the memory controller's timing registers, read by `sysreport-search intel-imc-dump` (MCHBAR mapped read-only from `/dev/mem`, aligned 32-bit reads; the window is never switched on)
`aod.bin`, `aod.txt`	AMD: the BIOS's AOD table (memory settings), named by an SSDT; read from `/dev/mem` only when the kernel allows it (`iomem=relaxed` on kernels with IO_STRICT_DEVMEM), else `aod.txt` says why
`rapl.txt`	AMD: the package and per-core energy counters, read twice one second apart
`ebda.txt`, `ebda.bin`	The BIOS Data Area's EBDA pointer (0x40E) and, when there is one, the EBDA (below 0xA0000, at most 64 KiB)
`vbios-<pci>.bin`	Each AMD GPU's VBIOS, copied from the driver's memory (debugfs `amdgpu_vbios`); the GPU's ROM is not touched
`gpio.txt`	The kernel's GPIO table (debugfs `gpio`)

File	What
`efi-systab.txt`, `esrt.txt`	The UEFI System Table pointers and the ESRT entries' root-only files
`pmic-*.txt`	Opt-in (Options > Read DIMM voltages from their PMIC chips, `Read DIMM PMIC=1`): each DDR5 PMIC's registers 0x00-0x3F, `i2cdump -y -r 0x00-0x3F BUS ADDR b` (byte reads, no `-f`)
`ec.bin`	Opt-in (Options > Read the embedded controller, `Read EC=1`): the EC's 256 registers via `ec_sys` loaded with `write_support=0`, unloaded again if srs-root loaded it
`acpi-calls.txt`	The log of `--acpi-eval` (time, call, result)

`--acpi-eval METHOD [ARG...]` (2.4) evaluates one ACPI method through the `acpi_call` module and exits; nothing else is collected. METHOD must be an ACPI name path (`\_SB.PCI0.GPP0._STA`, segments of up to 4 characters `A-Z 0-9 _`); each ARG a number (decimal or `0x` hex), a `"string"` (letters, digits, space, `. _ -`, up to 64) or a buffer `b` + hex bytes; at most 7. Anything else exits 2 without running. It loads `acpi_call` (exit 3 if it is not installed), writes the call to `/proc/acpi/call`, prints the result and appends it to `acpi-calls.txt`. **This runs BIOS code and can change hardware state**; the window asks once before it allows it (User Guide 9.22).

- It has no `--help`; any argument other than `--user NAME` or `--acpi-eval` is ignored.
- `--user NAME` works only as root; exit code 2 if not root or the user does not exist.
- Set `SRS_SMART_USB=1` to include USB-attached disks in the SMART collection.
- Tools that are not installed are skipped.

Boot unit: `srs-root@NAME.service` runs `srs-root --user NAME` once per boot (Nice 19, idle I/O). `sudo systemctl enable --now srs-root@$USER.service` / `disable`.

`srs-sysreport`

```
srs-sysreport
```

No options. Writes `~/sysreports/sysreport-YYYYMMDD-HHMMSS/` (one file per tool plus `index.html`), asks once for your password (pkexec) for the root sections, opens `index.html` in your browser and prints:

```
Report written to: /home/you/sysreports/sysreport-20261017-050453
Search it with: sysreport-search find PATTERN   (or: sysreport-search id VVVV:DDDD)
```

Exit codes (sysreport-search)

Code	Meaning
0	Success; `find` found matches; `burn` passed
1	`find` found no matches; lookup or benchmark failed; `burn` failed
2	Usage error (missing or unknown argument, bad value); `find` found no report folder; `burn` had nothing to run; a benchmark was not started (answered No, or no terminal and no `--yes`)
130	`burn` stopped with Ctrl+C

Environment variables

Variable	Effect
`SRS_SMART_USB=1`	`srs-root`: include USB disks in SMART
`SRS_KWH_COST`, `SRS_CURRENCY`	Price per kWh and currency symbol for the kWh Cost item (default `$` 0.15)
`SRS_OPEN`	Same as `--gui-open=`
`SRS_POWER_DB`, `SRS_POWER_INI`	Power & Cost: use another history database / settings file instead of `~/.local/share/srs/power.db` and `~/.config/srs/power.ini` (for tests)
`SRS_DATA_DIR`	Folder holding `power-zip-rates.tsv` for the offline US ZIP price lookup (default: next to the program, then `/usr/share/srs-sysinfo`)
`SRS_DOC_DIR`	Folder holding `SRS-User-Guide.pdf` / `html/` for `guide`, F1 and the Guide button (default `/usr/share/doc/srs-sysinfo`)
`SRS_VULKAN_LIB`	GPU Benchmarks: open this file instead of `libvulkan.so.1` (a name that does not exist = "not installed"; for tests)
`SRS_HIP_LIB`, `SRS_CUDA_LIB`, `SRS_OPENCL_LIB`	GPU Burnin and the GPU Compute comparison: open this file instead of the ROCm / HIP, CUDA or OpenCL library (a name that does not exist = "not installed"; for tests)
`SRS_HIP_RTC`, `SRS_CUDA_RTC`, `SRS_HIP_BLAS`, `SRS_CUDA_BLAS`, `SRS_CLBLAST_LIB`	Same, for the runtime compiler (hipRTC / NVRTC), the GEMM library (hipBLAS / cuBLAS) and CLBlast (for tests: `tests/gpu-stub-test.sh` runs against a fake runtime)
`SRS_CTK_DRY_RUN=1`	`cuda-toolkit --install` and the GUI's toolkit install show what they would run and run nothing (for tests)
`SRS_CTK_TOOL_DIRS`	Only together with `SRS_CTK_DRY_RUN=1`: look for the package manager, `sudo` and `pkexec` in these folders (colon-separated) instead of `/usr/bin`, `/usr/sbin`, `/bin`, `/sbin` (for tests: fake tools that are never run). Ignored otherwise

Variable	Effect
`SRS_OS_RELEASE`, `SRS_PCI_SYSFS`, `SRS_CTK_ROUTE`, `SRS_CTK_PACMAN_LOCK`	NVIDIA CUDA toolkit checks: read this file instead of `/etc/os-release`, this folder instead of `/sys/bus/pci/devices`, this route table instead of `/proc/net/route`, this lock file instead of pacman's `db.lck` (for tests: `tests/cuda-toolkit-test.sh`)
`SRS_GPUBURN_SIMPLE=1`	GPU Burnin uses the older tiled matrix multiply instead of the 2.4 register-blocked one (for comparing)
`SRS_SPD_DIR`	Memory Timings: read the modules' SPD from `<dir>/<device>/eeprom` instead of `/sys/bus/i2c/drivers/spd5118` (for tests)
`XDG_DATA_HOME`	Update hardware lists keeps its copies in `$XDG_DATA_HOME/srs/lists` (default `~/.local/share/srs/lists`)
`SRS_GPU_SOFTWARE_ONLY=1`	GPU Benchmarks: refuse every Vulkan device that is not a software renderer such as lavapipe (for tests)
`SUDO_USER`, `PKEXEC_UID`	Used to find whose root-data cache to use when run as root

SRS Glossary

Back to the User Guide - Section Reference

Plain-English meanings of the technical words used in the SRS documents, A to Z. After each definition, "See" lists the Section Reference pages that use the word most.

SRS is a Linux system information viewer inspired by SIV64X (System Information Viewer) by Ray Hinchliffe, RH Software; it is an independent project, not affiliated with SIV or its author.

0-9

- **80 PLUS** - a certificate for power supply efficiency (Bronze, Gold, Platinum, Titanium ...); see PSU efficiency. *See: User Guide 6.11.*

A

- **ABI (Application Binary Interface)** - the low-level rules a program and the kernel (or libraries) agree on so compiled code can work together. *See: srs-items.*
- **ACPI (Advanced Configuration and Power Interface)** - Advanced Configuration and Power Interface; the firmware tables and code that describe the board's devices, power states and interrupts to the operating system. *See: buses-1, buses-2, linux-os, machine, sensors.*
- **acpi_call** - a third-party kernel module that lets root run (evaluate) one ACPI method and read its result through `/proc/acpi/call`. SRS's ACPI method evaluation uses it. *See: machine, User Guide 9.22.*
- **ACPI namespace** - The tree of named objects in the ACPI tables, e.g. `\_SB_.PCI0.SBRG.EC0_`, which is the firmware's own map of the board's devices. *See: buses-1, machine.*
- **AER (Advanced Error Reporting)** - A PCI Express feature that counts and reports link errors (correctable and uncorrectable) to the kernel. *See: buses-1, graphics.*
- **AES (Advanced Encryption Standard), AES-NI** - the standard way to encrypt data (disk encryption, VPN, HTTPS). AES-NI are CPU instructions that do one AES round in hardware, many times faster than plain code. *See: tools.*
- **Affinity mask** - a bit pattern, one bit per CPU, saying which CPUs a task may run on. `FFFF` = CPUs 0-15. *See: processors, srs-items.*
- **AGM (arithmetic-geometric mean), Gauss-Legendre** - a way to compute pi that doubles the number of correct digits at every step; the method SuperPi uses. *See: tools.*
- **AIO** - All-In-One liquid cooler: a sealed pump, radiator and fans in one unit. *See: tools.*
- **AIO (all-in-one) cooler** - A sealed liquid cooler with pump, radiator and fans in one kit. *See: sensors.*

- **ALSA (Advanced Linux Sound Architecture)** - The kernel's sound driver layer. It talks to the sound chips and gives each card numbered devices such as `hw:1,0`. *See: graphics.*
- **Alt+letter (underlined letter)** - in a menu or window, the underlined letter of an entry: press Alt and that letter to pick it. Every entry of a menu has its own. *See: User Guide 6.6.*
- **amd_pstate** - the Linux driver that controls clock speed on AMD Zen 2 and newer CPUs using CPPC; modes are active, guided and passive. *See: processors.*
- **AML** - ACPI Machine Language; the bytecode in the DSDT and SSDTs that the operating system runs to control devices. *See: machine.*
- **AOD (AMD Overclocking Data) table** - a table the BIOS of AMD boards leaves in memory with its memory settings: ProcODT, drive strengths, RTT values and the voltages it set. Memory Timings reads it when the kernel allows (`iomem=relaxed`). *See: machine, User Guide 6.12.*
- **APIC** - Advanced Programmable Interrupt Controller; each CPU thread has one, and its APIC ID is the hardware's own number for that thread. *See: machine, processors.*
- **APIC ID** - the ID number each logical CPU reports about itself through the CPUID instruction. CPU Burnin uses it to check that a thread really runs where it was pinned. *See: tools.*
- **ARP (Address Resolution Protocol)** - how an IPv4 machine finds the MAC (hardware) address that belongs to an IP address on the local network. *See: network.*
- **ASLR (Address Space Layout Randomization)** - loading programs and libraries at random memory addresses each run, to make attacks harder. *See: linux-os.*
- **ASPM (Active State Power Management)** - lets a PCIe link drop into a low-power state when idle. *See: machine, srs-items.*
- **Associativity (ways)** - how many places in a cache a given piece of memory may be stored; more ways means fewer collisions. *See: processors.*
- **Asynchronous I/O (AIO)** - handing the kernel several read or write requests at once and collecting the answers later; how the storage benchmark keeps Q requests in flight. *See: tools.*
- **ATA** - AT Attachment: the command set used by SATA (and older IDE/PATA) hard disks and SSDs. *See: storage.*
- **At the wall** - what the PC draws from the socket, including the power supply's own losses; what the electricity meter counts and you pay for. *See: User Guide 6.11.*
- **AUR (Arch User Repository)** - community build recipes for Arch-based systems. Packages built from it show as "foreign" in pacman. *See: linux-os.*
- **Autosuspend** - the USB core's habit of putting an idle device to sleep after a set number of seconds; -1 turns it off. *See: buses-2.*
- **AUXTIN** - Auxiliary temperature inputs on a Nuvoton Super I/O chip. The board maker decides which are wired; unwired ones float and show fake values. *See: sensors.*
- **Available Spare** - NVMe health field: the percentage of spare flash blocks an SSD has left (starts at 100%). *See: storage.*
- **AVX / AVX2 / AVX-512** - Advanced Vector Extensions; instruction sets that work on many numbers at once (256 or 512 bits wide: 8 or 16 single-precision numbers per instruction). AVX-512 draws a lot of power under full load. *See: processors, tools.*

B

- **Back end (GPU)** - the way a program talks to the GPU for computing: Vulkan, ROCm / HIP, CUDA or OpenCL. GPU Burnin and the GPU Compute comparison can use each one. *See: tools.*

- **BAR (Base Address Register)** - a register in a PCI device that asks for a window of memory or I/O address space; the firmware or kernel assigns its start address at boot. *See: buses-1, buses-2, graphics.*
- **BBR** - a TCP congestion-control algorithm that measures bandwidth and delay instead of waiting for packet loss. *See: network.*
- **bcdDevice / bcdUSB** - fields in a USB device descriptor: the device's own release number, and the USB version it claims to follow. *See: buses-2.*
- **bfq** - Budget Fair Queueing: a Linux I/O scheduler that suits spinning disks and slow USB devices. *See: storage.*
- **Binary MOF** - a compressed description of a vendor's WMI classes, stored in the firmware. *See: machine.*
- **Binary splitting** - computing a long series by splitting it in halves again and again and joining the halves with big multiplications, so the work stays fast and can be shared between threads. *See: tools.*
- **Bind mount** - Attaching a folder of an existing filesystem at a second place in the folder tree. *See: storage.*
- **BMC (Baseboard Management Controller)** - A small separate computer on server boards that monitors sensors and allows remote management through IPMI. *See: sensors.*
- **BogoMIPS** - a rough number from a busy loop the kernel times at boot; not a real speed benchmark. *See: processors.*
- **BOINC** - Berkeley Open Infrastructure for Network Computing, a volunteer computing client. *See: tools.*
- **Boost (CPB / Turbo Boost)** - running above the base clock when power and temperature allow; CPB (Core Performance Boost) is AMD's name, Turbo Boost is Intel's. *See: processors.*
- **BootOrder** - the UEFI variable listing the boot entries (Boot####) in the order the firmware tries them. *See: machine.*
- **BORE (Burst-Oriented Response Enhancer)** - a change to Linux's EEVDF CPU scheduler that favours tasks which run in short bursts (typically interactive programs), shipped in some gaming kernels. *See: tools.*
- **Boyer-Moore-Horspool** - a fast way to find a word in a text by skipping ahead using the last letter of the word. *See: tools.*
- **Bridge (USB bridge)** - The chip in a USB enclosure, stick or card reader that converts USB to SATA, NVMe or flash commands. *See: buses-1, storage.*
- **BSSID** - the radio (MAC) address of one Wi-Fi access point; one network name (SSID) can have several. *See: network.*
- **btrfs** - A Linux copy-on-write filesystem with checksums, subvolumes, snapshots, compression and built-in multi-disk support. *See: storage.*
- **BTU/h (British thermal units per hour)** - a unit of heat flow: 1 W is about 3.412 BTU/h. All the power a PC uses ends up as heat in the room. *See: User Guide 6.11.*
- **Built-in driver** - a driver compiled into the kernel file itself. It is always present and never appears in `/proc/modules`. *See: linux-os.*
- **Bulk-Only Transport (BOT)** - the older, simpler USB mass-storage protocol (interface 08/06/50); slower than UAS. *See: buses-2.*

C

- **Carbon intensity** - grams of CO2 released per kWh of electricity on a grid, on average over a year. *See: User Guide 6.11.*
- **CLBlast** - a free, tuned matrix-maths library for OpenCL; the "vendor library" SRS uses on OpenCL when it is installed. *See: tools.*
- **Compute error (GPU)** - a GPU result that differs from the right one. GPU Burnin counts them: any error means the card is not stable as set up (overclock, undervolt, heat, power) or faulty. *See: tools.*
- **C-state** - an idle power level of a CPU core; C0 is running, deeper states (C1, C2, C3) save more power but take longer to wake. *See: processors.*
- **Cache** - small fast memory on the CPU that keeps copies of recently used RAM; L1 is smallest and fastest, L3 largest and shared. *See: processors.*
- **Cache level (L1, L2, L3)** - small, fast memories between the core and RAM. L1 is the smallest and fastest; L3 is the largest and is shared by many cores. *See: tools.*
- **Capability (Linux)** - one split-off piece of root's power, such as `CAP_NET_BIND_SERVICE`. It is the Linux counterpart of a Windows privilege. *See: linux-os.*
- **CardBus / PCMCIA** - Old credit-card-sized laptop expansion cards, replaced by ExpressCard and USB. *See: buses-1.*
- **CCD** - Core Complex Die, one chiplet of CPU cores on a Ryzen processor. Each CCD usually has its own L3 cache. *See: processors, sensors, tools.*
- **CCID (Chip Card Interface Device)** - The standard USB class for smart card readers. *See: buses-1.*
- **CEM (Card Electromechanical)** - The PCI Express standard for normal plug-in cards; `board_info type : cem` means a regular desktop graphics card. *See: graphics.*
- **Charmap** - a glibc file that defines a character set; used when building locales. *See: linux-os.*
- **CHS** - Cylinder-Head-Sector: the old way of addressing disk blocks, replaced by LBA. SRS shows the legacy 255-head, 63-sector translation. *See: storage.*
- **Chudnovsky series** - a formula for pi that gives about 14 more digits for every term; the fastest way known to compute many digits. *See: tools.*
- **Class code** - The PCI number that says what kind of device it is, e.g. 0300 display, 0108 NVMe, 0604 PCI bridge. *See: buses-1.*
- **Clock source** - the hardware timer Linux uses to keep time (tsc, hpet, acpi_pm). *See: processors.*
- **Clocksource** - the hardware counter the kernel reads to keep time (TSC, HPET, ACPI PM timer). *See: linux-os.*
- **Cluster** - The smallest unit of space a FAT, exFAT or NTFS filesystem hands out to a file. *See: storage.*
- **CLZERO** - an AMD instruction that writes a whole zeroed 64-byte cache line straight to memory; the Cache & Memory Benchmark uses it to measure how fast the RAM itself can take writes. *See: tools.*
- **CMOS** - the battery-backed clock chip and its small memory on the motherboard; the name comes from the chip technology. *See: machine.*
- **Code page** - an old table that maps byte values to characters (e.g. 437, 1252). UTF-8 has replaced code pages on Linux. *See: linux-os.*
- **Code point** - Unicode's number for one character, written like `U+0041`. *See: linux-os.*
- **COM port** - A serial port. COM1 is traditionally at I/O 0x3F8, IRQ 4 (`ttyS0` on Linux). *See: buses-1.*
- **Commit charge** - the total memory programs have reserved; it can exceed RAM when swap exists. *See: machine.*

- **Compression ratio** - how much smaller data became: 2.5 means the packed data is 2.5 times smaller than the original. *See: tools.*
- **Compute unit (CU)** - A block of shader cores inside the GPU. RDNA GPUs pair two CUs into a WGP. *See: graphics.*
- **Congestion control** - the method TCP uses to guess how fast it can send without overloading the path. *See: network, srs-items.*
- **Context switch** - the kernel stopping one task on a CPU and starting another; the OS Latency benchmark times it with two threads or programs passing a byte back and forth through a pipe. *See: tools.*
- **Core dump** - a snapshot of a crashed program's memory, saved for debugging. *See: linux-os.*
- **CPPC** - Collaborative Processor Performance Control; an ACPI standard where the CPU offers a smooth scale of performance levels and the OS sets a target range. *See: processors.*
- **CPU affinity** - a rule that ties a thread to one or more chosen CPUs, so the scheduler does not move it elsewhere. *See: tools.*
- **cpufreq** - the Linux subsystem that sets CPU clock speed, made of a driver (talks to hardware) and a governor (picks the speed). *See: processors.*
- **CPUID** - an x86 instruction that returns facts about the CPU (vendor, family, features, caches) in four registers, organised by leaf number. *See: processors.*
- **cpuidle** - the Linux subsystem that puts idle CPUs into C-states. *See: processors.*
- **CRB** - Command Response Buffer; the memory-based interface used by firmware TPMs. *See: machine.*
- **CRC32, CRC32C** - checksums that detect damaged data (CRC32 in ZIP and Ethernet, CRC32C in storage and networks); many CPUs have an instruction for CRC32C. *See: tools.*
- **Critical Warning** - NVMe health field: a bit field that is 0x00 when the drive sees no serious problem. *See: storage.*
- **CRTC** - The part of the display engine that reads one picture from memory and sends it to one or more outputs. VRR is switched on per CRTC. *See: graphics.*
- **CSM (Compatibility Support Module)** - Compatibility Support Module; the UEFI option that emulates a legacy BIOS for old operating systems and cards. *See: linux-os, machine.*
- **CSMI** - Common Storage Management Interface: a Windows driver interface some RAID controllers offer for status queries. *See: storage.*
- **CSV (comma-separated values)** - A plain-text table format, one line per row, columns split by commas; opens in any spreadsheet. *See: sensors, srs-items.*
- **CTA-861** - An EDID extension block with TV and HDMI data: extra video modes, audio formats and HDR information. *See: graphics.*
- **cuBLAS** - NVIDIA's tuned matrix-maths library for CUDA. *See: tools.*
- **CUDA** - NVIDIA's GPU computing platform, used by most AI programs; needs the NVIDIA driver. With ZLUDA, CUDA programs can run on AMD cards. *See: tools.*
- **CUDA toolkit (NVIDIA CUDA toolkit)** - NVIDIA's free set of developer tools for running programs on NVIDIA graphics cards. SRS can use one small part of it, NVRTC, as a fallback; since 2.3 it is optional, and with ZLUDA it is not recommended (it can break or remove ZLUDA). Install it later from Options > NVIDIA CUDA toolkit for GPU tests... *See: User Guide 9.20, tools.*
- **Curve Optimizer** - an AMD Ryzen BIOS setting that lowers (or raises) the voltage curve of each core. Too negative a value makes a core unstable or slow. *See: tools.*

D

- **DC power (inside the PC)** - the power after the power supply, used by the parts; always less than the power at the wall. *See: User Guide 6.11.*
- **debugfs** - a kernel file system (`/sys/kernel/debug`) for developer information, readable by root only; srs-root copies the GPU driver's VBIOS copy and the GPIO table from it. *See: graphics, buses-1.*
- **Device lost** - the GPU stopped answering and its driver reset it; SRS ends a GPU test cleanly when it happens. *See: tools.*
- **Drive strength** - how hard a chip drives a signal onto the memory bus, given in ohms (lower = stronger). Set by the BIOS for the processor, the memory chips and the command / address (CAD) bus. *See: machine.*
- **D-states** - device power states: D0 fully on, D3hot off but powered, D3cold power removed. *See: machine.*
- **D0 / D3hot / D3cold** - PCI power states. D0 is fully on; D3hot and D3cold are sleep states (D3cold removes power completely). *See: graphics.*
- **DAD (Duplicate Address Detection)** - a check that no other device already uses an address before the PC starts using it. *See: network.*
- **Data Units** - NVMe counters of data read and written; one unit is 1,000 x 512 bytes (512,000 bytes). *See: storage.*
- **dBm** - signal power on a logarithmic scale; for Wi-Fi, -30 is excellent and -80 is barely usable. *See: network.*
- **dconf** - the binary settings database used by GNOME and GTK apps. *See: linux-os.*
- **DDC (Display Data Channel)** - The I2C wires in a video cable that carry the monitor's EDID and DDC/CI commands such as brightness. *See: graphics, srs-items.*
- **Default gateway** - the router your PC sends traffic to when the destination is not on the local network. *See: tools.*
- **Descriptor (USB)** - a small data table a USB device sends when plugged in, describing its IDs, class, interfaces and endpoints. *See: buses-2.*
- **.desktop file** - a small text file that describes an app (name, icon, command, file types it opens). *See: linux-os.*
- **dev_t** - The major:minor number pair the kernel uses to find a device file in /dev. *See: buses-1.*
- **Device descriptor** - The USB record that gives a device's vendor ID, product ID, release number and class. *See: buses-1.*
- **Device-mapper (dm)** - The Linux kernel layer that maps virtual block devices onto real ones; used by LVM and dm-crypt. *See: storage.*
- **DGEMM, SGEMM** - double- and single-precision general matrix multiply: the standard way to measure floating-point speed, counted as 2 x n^3 operations for n x n matrices. *See: tools.*
- **DHCP (Dynamic Host Configuration Protocol)** - the way a router hands out IP addresses, gateway and DNS settings to devices automatically. *See: network.*
- **DHCP lease** - an IP address the router lends to a device for a set time; the device renews it before it runs out. *See: network.*
- **Dhrystone** - a classic integer (whole-number) CPU benchmark. Its score is given in DMIPS. *See: tools.*
- **DIMM** - Dual In-line Memory Module; a desktop memory stick (SODIMM is the laptop size). *See: machine, srs-items.*
- **Dirty flag** - A filesystem marker meaning it was not closed cleanly and should be checked. *See: storage.*

- **Dirty pages** - file data changed in RAM but not yet written to disk. *See: linux-os.*
- **Discard** - The Linux name for TRIM. *See: storage.*
- **DisplayID** - A newer monitor description format, often added as an EDID extension for high resolutions and refresh rates. *See: graphics.*
- **DKMS (Dynamic Kernel Module Support)** - a system that rebuilds out-of-tree kernel modules automatically for each new kernel. *See: linux-os.*
- **DLL (Dynamic-Link Library)** - a Windows shared code file; the Linux equivalent is a shared object (`.so`). *See: linux-os, srs-items.*
- **dm-crypt** - Linux disk encryption built on device-mapper (usually set up with LUKS). *See: storage.*
- **DMA (Direct Memory Access)** - A device reading or writing memory by itself, without the CPU copying the data. *See: buses-1, buses-2, machine.*
- **DMI (Desktop Management Interface)** - Desktop Management Interface; another name for the SMBIOS inventory, shown by Linux in `/sys/class/dmi/id`. *See: linux-os, machine, sensors.*
- **DMIPS** - Dhrystone MIPS: Dhrystone loops per second divided by 1757, the speed of a DEC VAX 11/780. *See: tools.*
- **DNS (Domain Name System)** - the service that turns names like example.com into IP addresses. *See: network.*
- **DNS cache** - a local store of recent DNS answers so repeated lookups are instant. *See: network.*
- **DNS lookup** - turning a name like speed.cloudflare.com into an address; the first lookup asks the network, a repeat usually comes from a cache. *See: tools.*
- **DPM (dynamic power management)** - The GPU's list of clock and voltage steps; the firmware switches between them many times a second. *See: graphics, srs-items.*
- **DPMS (Display Power Management Signaling)** - The monitor's power state: On, Standby, Suspend or Off. *See: graphics.*
- **DRAM** - Dynamic RAM, the main system memory. It is much slower to reach than the caches. *See: tools.*
- **drivetemp** - A Linux kernel module that reports SATA drive temperatures through hwmon without root. *See: storage.*
- **DRM (Direct Rendering Manager)** - The kernel subsystem for GPUs. It creates `/dev/dri/cardN` and `/dev/dri/renderDN` and the folders in `/sys/class/drm`. *See: graphics.*
- **DSDT** - Differentiated System Description Table; the main ACPI table holding the board's AML code. *See: machine.*
- **DSDT / SSDT** - The main and the extra ACPI code tables; they contain the firmware's methods, including WMI code. *See: sensors.*
- **dup** - A btrfs profile that keeps two copies of a block on the same disk. *See: storage.*
- **Duplex** - whether both ends of a link can send at once (full) or must take turns (half). *See: network.*
- **Duty (duty cycle)** - The share of time a PWM signal is on; 100% means full power to the fan. *See: sensors.*
- **DXVK** - A translation layer that turns Direct3D 8/9/10/11 calls into Vulkan, used by Wine and Proton. *See: graphics.*

E

- **E-core** - Efficiency core, a small, low-power core on hybrid CPUs such as Intel 12th generation and newer. *See: tools.*

- **E820** - the firmware memory map that lists which physical address ranges are RAM or reserved. *See: machine.*
- **e820 map** - the memory map the PC firmware hands to the operating system at boot, marking RAM, reserved and ACPI ranges. *See: buses-2.*
- **EBDA** - Extended BIOS Data Area; a small legacy-BIOS data block just below 640 KB. *See: machine.*
- **EC (Embedded Controller)** - A small microcontroller on the motherboard that handles power buttons, battery, fans and some sensors (ACPI ID PNP0C09). *See: buses-1.*
- **ECAM (Enhanced Configuration Access Mechanism)** - The memory window through which the CPU reads each PCIe device's 4 KB configuration space. *See: buses-1, buses-2.*
- **ECC** - Error-Correcting Code memory; detects and fixes single-bit memory errors. *See: machine.*
- **ECDT** - The ACPI Embedded Controller Boot Resources Table, an optional early description of the EC. *See: buses-1.*
- **EDAC** - Error Detection And Correction; the Linux subsystem that reports ECC memory errors. *See: machine.*
- **EDID (Extended Display Identification Data)** - A 128-byte block (plus extensions) stored in the monitor that describes its maker, model, size and supported modes. *See: graphics.*
- **EEVDF (Earliest Eligible Virtual Deadline First)** - the standard Linux CPU scheduler since Linux 6.6 (it replaced CFS). *See: tools.*
- **EGL** - The interface that connects OpenGL and OpenGL ES to windows on Wayland or X11. *See: graphics.*
- **ELD (EDID-Like Data)** - The audio part of a monitor's EDID, passed to the HDMI/DisplayPort sound driver. *See: graphics.*
- **ELF (Executable and Linkable Format)** - the file format of Linux programs and shared libraries. *See: srs-items.*
- **Embedded controller (EC)** - a small microcontroller on the board for keyboard, battery, fans and buttons (mainly on laptops). *See: machine.*
- **Endpoint (USB)** - a one-way data pipe inside a USB interface; its type is Control, Interrupt, Bulk or Isochronous. *See: buses-1, buses-2.*
- **EPP (Energy Performance Preference)** - Energy Performance Preference; a hint to the CPU about how much to favour speed over power saving. *See: linux-os, processors, srs-items.*
- **ESP (EFI System Partition)** - EFI System Partition: the FAT partition the UEFI firmware loads bootloaders from. *See: linux-os, storage.*
- **ESRT** - EFI System Resource Table; lists the firmware parts that can be updated with UEFI capsules. *See: machine.*
- **exFAT** - Extended FAT: a simple filesystem for flash media that allows files over 4 GiB. *See: storage.*
- **EXPO** - AMD Extended Profiles for Overclocking; faster memory settings stored in the SPD. *See: machine.*
- **EXPO / XMP** - memory profiles (AMD EXPO, Intel XMP) stored on the RAM sticks, which set the rated speed and timings in one step. *See: tools.*

F

- **FADT (Fixed ACPI Description Table)** - Fixed ACPI Description Table (signature FACP); points to the other key tables and lists fixed power-management hardware. *See: buses-1, machine.*
- **Family / Model / Stepping** - numbers from CPUID that identify a CPU's generation, design and silicon revision. *See: processors.*

- **Fan curve** - A list of temperature:duty points the fan controller follows, with straight lines between them. *See: sensors.*
- **FAT** - File Allocation Table: a simple filesystem family (FAT12, FAT16, FAT32) used on USB sticks and EFI partitions. *See: storage.*
- **FCH (Fusion Controller Hub)** - AMD's built-in southbridge logic: SMBus, LPC, GPIO, power management. *See: buses-1.*
- **fclk / socclk / dcefclk** - GPU clocks for the internal data fabric, the chip's support logic, and the display engine. *See: graphics.*
- **FCLK / UCLK / MCLK** - on AMD: Infinity Fabric clock, memory controller clock and memory clock. *See: machine.*
- **FFT (Fast Fourier Transform)** - a fast way to split a signal into its frequencies; used in audio, images and science. Counted as 5 N log2 N operations for N points. *See: tools.*
- **File descriptor** - a small number a process uses for each open file, pipe, socket or device; the Linux version of a Windows handle. *See: linux-os.*
- **File lock** - a marker a program sets so others know it is using a file; listed in `/proc/locks`. *See: srs-items.*
- **firewalld zone** - a named set of firewall rules; each network interface belongs to one zone. *See: network.*
- **FireWire (IEEE 1394)** - An older high-speed bus used for camcorders and pro audio. *See: buses-1.*
- **Fixture** - a saved data dump used to test a program without the real hardware. *See: srs-items.*
- **Floating input** - A sensor pin with nothing connected. Its voltage drifts with leakage and noise, so the reading is meaningless. *See: sensors.*
- **fontconfig** - the Linux library that finds fonts and picks one for each character. *See: linux-os.*
- **FP16 / FP32 / FP64** - numbers with a decimal point stored in 16, 32 or 64 bits: half, single and double precision. Games mostly use FP32; most gaming GPUs are much slower at FP64. *See: tools.*
- **FPDT** - Firmware Performance Data Table; records how long the firmware took to boot. *See: machine.*
- **FPGA (Field-Programmable Gate Array)** - a chip whose logic circuits are loaded from a configuration file, so its function can be changed after manufacture. *See: buses-2.*
- **FQDN (fully qualified domain name)** - a host name plus its domain, such as pc.example.lan. *See: network.*
- **FSR (FidelityFX Super Resolution)** - AMD's upscaling technique: render at a lower resolution, then scale up and sharpen. *See: graphics.*
- **fTPM** - firmware TPM; a TPM implemented inside the CPU's security processor instead of a separate chip. *See: machine.*
- **Function (PCI)** - One logical device inside a PCI chip. A GPU's audio part is function 1 (`03:00.1`). *See: buses-1.*
- **Futex** - "fast user-space mutex": the Linux building block that lets a waiting thread sleep and be woken by another; the OS Latency benchmark times the wake-up. *See: tools.*

G

- **gamescope** - Valve's small game compositor; it can upscale games with FSR (`gamescope -F fsr`). *See: graphics.*
- **Gateway** - the router that forwards your packets to other networks, usually your home router. *See: network.*

- **GBM (Generic Buffer Management)** - A library compositors use to get GPU memory buffers for the screen. *See: graphics.*
- **GC (graphics core)** - The 3D and compute engine of an AMD GPU. Its version gives the `gfx` target name (GC 12.0.1 = `gfx1201`). *See: graphics.*
- **Gear Down Mode (GDM), command rate (1T / 2T)** - two ways the memory controller times its commands to the memory: GDM eases timing at high speeds, 1T / 2T is how many clocks a command takes. GDM on with 1T is normal for DDR5. *See: machine.*
- **GEMM (matrix multiplication)** - multiplying two big tables of numbers (general matrix multiply); the core of AI and many science programs, and a hard, steady load for a GPU. *See: tools.*
- **GFLOPS** - billions of floating-point (decimal-point) operations per second. *See: tools.*
- **GIOPS** - billions of whole-number (integer) operations per second. *See: tools.*
- **GLVND** - The vendor-neutral OpenGL dispatch library that lets several OpenGL drivers be installed side by side. *See: graphics.*
- **Governor (CPU)** - the cpufreq policy that decides which speed to ask for (performance, powersave, schedutil). *See: linux-os, processors, srs-items.*
- **GPE (General Purpose Event)** - General Purpose Event; an ACPI event bit used for wake-up and hot-plug signals. *See: buses-1, machine.*
- **GPIO (General Purpose Input/Output)** - Simple on/off signal pins that chips use for LEDs, buttons, resets and enables. *See: buses-1.*
- **GPT** - GUID Partition Table: the modern partition table format, with type GUIDs, PARTUUIDs and names for each partition. *See: storage.*
- **GSI** - Global System Interrupt; the interrupt number space used by I/O APICs. *See: machine.*
- **GStreamer** - The main Linux media framework; plugins handle file formats, codecs and audio/video effects. *See: graphics.*
- **GT/s (gigatransfers per second)** - Gigatransfers per second: the per-lane speed of a PCIe link (8.0 = PCIe 3.0, 16.0 = PCIe 4.0, 32.0 = PCIe 5.0). *See: buses-1, buses-2, srs-items, storage.*
- **GTT (graphics translation table)** - System RAM that the GPU is allowed to map and use, much slower for the GPU than VRAM. *See: graphics.*
- **GUID** - Globally Unique Identifier; a 128-bit ID, used for WMI interfaces and UEFI variable vendors. *See: machine.*

H

- **H:C:T:L** - Host:Channel:Target:LUN, the address of a device in the Linux SCSI layer. *See: storage.*
- **HAL (Hardware Abstraction Layer)** - the lowest layer of Windows that hides hardware differences; SIV uses the name for its resource-map pages. *See: buses-2.*
- **HD Audio (High Definition Audio)** - The Intel standard used by almost all PC sound chips and by GPU HDMI audio: a controller plus one or more codecs. *See: graphics.*
- **HDD** - Hard disk drive: storage on spinning magnetic platters. *See: storage.*
- **Heap** - the memory a program requests while it runs (`malloc`). *See: linux-os.*
- **HECI** - Host Embedded Controller Interface; the link between the OS and the Intel Management Engine (MEI on Linux). *See: machine.*
- **HID (Human Interface Device)** - The device class and protocol for keyboards, mice, game pads, RGB controllers and security keys. *See: buses-1, buses-2, machine, srs-items.*
- **hidraw** - A Linux device node (`/dev/hidrawN`) that gives programs raw access to a HID device. *See: buses-1.*

- **HIP** - AMD's CUDA-like programming language for GPU computing, part of ROCm. *See: graphics.*
- **hipBLAS** - AMD's tuned matrix-maths library for ROCm / HIP. *See: tools.*
- **hipRTC** - ROCm's runtime compiler, the HIP counterpart of NVRTC. The ROCm / HIP rows of GPU Burnin and the GPU Compute comparison still need it. *See: tools.*
- **Holder** - A Linux block device built on top of another one (for example an encrypted volume on a partition). *See: storage.*
- **Host bridge** - The PCI device that connects the CPU to the PCI bus (class 0600). *See: buses-1.*
- **Hotspot temperature** - the hottest point measured on a GPU chip; AMD calls it junction temperature. Usually 10-20 °C above the edge temperature. *See: tools.*
- **HPA** - Host Protected Area: a part of an ATA disk hidden from the operating system, which makes the disk look smaller. *See: storage.*
- **HPET (High Precision Event Timer)** - High Precision Event Timer; a hardware timer described by its own ACPI table. *See: buses-2, machine.*
- **Hub (USB)** - a device that turns one USB port into several. *See: buses-2.*
- **hwdata** - The package that ships the `pci.ids`, `usb.ids` and `pnp.ids` name lists. *See: buses-1.*
- **hwdb** - udev's hardware database, which adds properties such as vendor and model names to devices. *See: buses-1.*
- **hwmon** - The kernel's standard sensor interface. Each driver makes a folder of files such as `temp1_input` and `fan1_input`. *See: graphics, linux-os, machine, processors, sensors, srs-items, storage.*
- **Hybrid CPU** - a CPU that mixes two kinds of cores (P-cores and E-cores). *See: tools.*

I

- **I/O APIC** - the interrupt controller that collects legacy interrupt lines and routes them to CPUs; "IR-" in front means the IOMMU remaps them. *See: buses-2.*
- **iomem=relaxed, IO_STRICT_DEVMEM** - IO_STRICT_DEVMEM is a kernel build option (on in most distributions) that keeps programs, root too, from reading memory that belongs to devices or the firmware through `/dev/mem`; booting with `iomem=relaxed` turns that protection off. Memory Timings' BIOS settings need it; SRS never asks you to set it. *See: machine, User Guide 5.4.*
- **I/O port** - An address in the separate 64K x86 I/O space, used by legacy devices (keyboard controller, RTC, COM ports, SMBus). *See: buses-1, buses-2.*
- **I/O scheduler** - The kernel part that decides the order in which waiting read/write requests go to a drive. *See: srs-items, storage.*
- **I2C (Inter-Integrated Circuit)** - A slow two-wire bus that chips use to talk to each other. Monitors use it for DDC. *See: buses-1, graphics, srs-items.*
- **I2C / SMBus** - slow two-wire buses for small chips such as sensors and memory SPD hubs; SMBus is the PC variant. *See: machine.*
- **ICD (installable client driver)** - A small file that tells the Vulkan or OpenCL loader where a driver library is. *See: graphics.*
- **ICMP (Internet Control Message Protocol)** - the protocol that carries ping and network error messages. *See: network.*
- **iconv** - the glibc library for converting text between character sets. *See: linux-os.*
- **IMC (Integrated Memory Controller)** - the part of the CPU that talks to RAM. *See: srs-items.*
- **IMSM** - Intel Matrix Storage Manager: the RAID metadata format used by Intel RST and VROC, supported by Linux mdadm. *See: storage.*

- **Infinity Fabric** - AMD's internal link between the CCDs, the I/O die and the memory controller. *See: tools.*
- **initramfs** - a small early-boot file system that loads drivers needed to mount the real root disk. *See: linux-os.*
- **Inode** - A filesystem's record for one file or folder. *See: storage.*
- **Interface (USB)** - One function of a USB device, with its own class and driver. One device can have several. *See: buses-1.*
- **IOMMU (input-output memory management unit)** - Input-Output Memory Management Unit; controls which memory each device can reach, needed for passing devices to virtual machines. *See: buses-1, buses-2, graphics, machine, processors.*
- **IOMMU group** - The smallest set of devices the IOMMU can isolate from each other. It matters for passing a device to a virtual machine. *See: buses-1.*
- **IOPS** - input/output operations per second: how many separate read or write requests a drive answers each second. *See: tools.*
- **IP block** - A reusable hardware building block inside an AMD chip (graphics core, video engine, memory controller…), each with its own version. *See: graphics.*
- **IPI** - Inter-Processor Interrupt, a signal from one CPU to another. SRS's IPI Delay measures the core-to-core cache-line delay instead. *See: tools.*
- **IPMI (Intelligent Platform Management Interface)** - A standard for talking to a server's BMC: sensors, event log, remote power control. *See: sensors.*
- **IrDA** - Infrared data transfer on old laptops and phones. Linux removed support in kernel 4.17. *See: buses-1.*
- **IRQ (Interrupt Request)** - A signal a device uses to get the CPU's attention. Also the number of that interrupt line. *See: buses-1, buses-2, graphics.*
- **ISA (Industry Standard Architecture)** - The original PC expansion bus. Its job now lives on in LPC. *See: buses-1.*
- **isohybrid** - A CD/DVD image (ISO 9660) built so it can also boot when written straight to a USB stick. *See: storage.*
- **Itanium (IA-64)** - Intel and HP's 64-bit server processor family, unrelated to x86-64, discontinued in 2021. *See: buses-2.*
- **IVRS** - I/O Virtualization Reporting Structure; the ACPI table describing AMD's IOMMU (Intel uses DMAR). *See: machine.*

J

- **JEDEC** - the standards body for memory; the JEDEC speed is a stick's standard base speed. *See: machine.*
- **Jitter** - how much a time (round trip, connect) changes from one try to the next. *See: tools.*
- **Joule (J)** - a unit of energy: 3,600 J = 1 Wh. Linux energy counters count microjoules (millionths of a joule). *See: User Guide 6.11.*
- **Junction temperature** - the hottest point measured on a graphics chip, usually higher than its "edge" temperature. *See: tools.*

K

- **k10temp** - the Linux driver that reads AMD CPU temperature sensors (Tctl, Tccd). *See: processors.*
- **Kernel (GPU)** - a small program that runs on the GPU, many copies at once (not the Linux kernel). *See: tools.*
- **Kernel module** - a loadable piece of the kernel, usually a driver; the Linux counterpart of a Windows driver. *See: linux-os, srs-items.*
- **KMS (kernel mode setting)** - The part of DRM that sets screen resolutions and refresh rates. *See: graphics.*
- **KSM (Kernel Same-page Merging)** - the kernel finds identical memory pages and keeps only one copy. *See: srs-items.*
- **KVM** - Kernel-based Virtual Machine; Linux's built-in hypervisor, available through /dev/kvm. *See: processors.*
- **kWh (kilowatt-hour)** - 1000 watts used for one hour; the unit electricity is billed in. *See: sensors.*
- **kW (kilowatt)** - 1,000 watts. *See: User Guide 6.11.*

L

- **LACT** - A Linux tool for AMD GPU overclocking, undervolting and fan control. *See: graphics.*
- **LAN (local area network)** - the devices in your home or office connected to the same router or switch. *See: network.*
- **Lane (PCIe)** - One pair of signal wires in each direction. A link has 1, 2, 4, 8 or 16 lanes. *See: buses-1.*
- **Latency** - waiting time: how long it takes for a request (a memory load, a ping) to be answered. *See: tools.*
- **Lavapipe** - Mesa's software Vulkan driver, which runs on the CPU; SRS's GPU tests use it so that they never touch a real graphics card. *See: tools.*
- **LBA** - Logical Block Addressing: numbering a disk's blocks from 0 upwards, the way all modern drives are addressed. *See: storage.*
- **LDN (Logical Device Number)** - The number of one function block (serial port, hardware monitor ...) inside a Super I/O chip. *See: sensors.*
- **Leaf (CPUID)** - the question number given to the CPUID instruction; some leaves have several sub-leaves. *See: processors.*
- **LFC (low framerate compensation)** - A VRR trick that shows frames twice when the frame rate falls below the monitor's VRR minimum. *See: graphics.*
- **Lineup** - a saved arrangement of the main window's tabs: which items each page shows as tabs, in what order, and the tab bar shape. Basic, Enthusiast and Expert are built in; you can save your own. *See: User Guide 6.7.*
- **Link width (PCIe)** - The number of lanes a PCIe link is using (x1, x4, x8, x16). *See: buses-1.*
- **Link-local address** - an address that only works on the local cable or Wi-Fi (fe80:: for IPv6, 169.254. for IPv4), made automatically. *See: network.*
- **llvmpipe** - Mesa's software renderer. It draws on the CPU and is only a fallback; its Vulkan driver is lavapipe. GPU Burnin never burns it as if it were a GPU. *See: graphics, tools.*

- **lm-sensors** - The standard Linux sensor package; its config files (/etc/sensors3.conf, /etc/sensors.d) rename, scale or hide readings. *See: sensors.*
- **Load average** - the average number of tasks running or waiting for a CPU over 1, 5 and 15 minutes. *See: linux-os.*
- **Locale** - a bundle of language and regional rules: language, date and number format, sorting, currency. *See: linux-os, srs-items.*
- **Locally administered MAC** - a MAC address made up by software rather than assigned by the hardware maker. *See: network.*
- **Logical CPU** - one hardware thread as Linux sees it. With SMT, each physical core shows up as two logical CPUs. *See: srs-items, tools.*
- **Loopback (127.0.0.1)** - the computer's connection to itself; data never reaches a network card. *See: tools.*
- **LPC (Low Pin Count)** - A slim bus from the chipset to the Super I/O chip, TPM, serial port and BIOS flash. *See: buses-1.*
- **LPM (Link Power Management)** - USB power saving that puts the link into low-power states between transfers; some devices need it disabled. *See: buses-2.*
- **LSM (Linux Security Module)** - a pluggable security layer in the kernel, such as Yama, Landlock, AppArmor or SELinux. *See: linux-os.*
- **LUN** - Logical Unit Number: one device behind one SCSI address, such as one slot of a card reader. *See: storage.*
- **LVM** - Logical Volume Manager: Linux tool for flexible volumes spread over one or more disks. *See: storage.*

M

- **MAC address (Media Access Control)** - the 6-byte hardware address of a network adapter. *See: network.*
- **MADT** - Multiple APIC Description Table (signature APIC); lists the CPU threads and interrupt controllers. *See: machine.*
- **Mandelbrot set** - a famous fractal; drawing it is a simple, repeatable floating-point test. Measured in Mpixels/s. *See: tools.*
- **MangoHud** - A Linux in-game overlay that shows frame rate, temperatures, clocks and more, and can limit the frame rate. *See: graphics.*
- **Masquerade** - NAT (network address translation): many devices share one address on the way out. *See: network.*
- **Mbit/s** - million bits per second; 8 bits = 1 byte, so 100 Mbit/s is 12.5 MB/s. Internet plans are sold in Mbit/s. *See: tools.*
- **MBR** - Master Boot Record: the old partition table format (shown as `dos`), with four primary partitions and one-byte type codes. *See: storage.*
- **MCA (Micro Channel Architecture)** - IBM's PS/2 expansion bus from 1987. *See: buses-1.*
- **MCE (Machine Check Exception)** - the CPU reporting a hardware error it detected, for example in cache or memory. *See: buses-2.*
- **MCFG** - The ACPI table that gives the ECAM base address. *See: buses-1.*
- **MCH (Memory Controller Hub)** - The old Intel "northbridge" chip that held the memory controller. AMD CPUs have no MCH. *See: buses-1.*

- **MCHBAR (Memory Controller Hub Base Address Register)** - the address of a block of Intel memory-controller registers. *See: srs-items.*
- **mclk** - The GPU's memory clock. GDDR6 moves 16 bits per clock per pin, so 1258 MHz is about 20 Gbps. *See: graphics.*
- **mclk / sclk** - a GPU's memory clock and shader (core) clock. *See: srs-items.*
- **MCS (Modulation and Coding Scheme)** - the Wi-Fi encoding step; higher is faster but needs a cleaner signal. *See: network.*
- **md** - The Linux software RAID driver, managed with `mdadm`. *See: storage.*
- **mDNS (multicast DNS)** - lets devices announce names and services on the LAN without a DNS server. *See: network.*
- **Memory timings (tCL, tRCD, tRP, tRAS, tRFC ...)** - the waiting times, in memory clock ticks, between the steps of a memory access: tCL from a read command to the data, tRCD to open a row, tRP to close it, tRAS how long it stays open, tRFC how long a refresh takes. Lower is faster, if the memory stays stable. *See: machine, User Guide 6.12.*
- **Mesa** - The open-source graphics driver collection on Linux, including RADV (Vulkan) and radeonsi (OpenGL) for AMD. *See: graphics.*
- **Metric** - a route's priority number; when two routes match, the lower metric wins. *See: network.*
- **MFT** - Master File Table: the NTFS index that holds a record for every file. *See: storage.*
- **Microcode** - a small firmware patch for the processor, loaded by the BIOS or the OS at boot to fix bugs and security issues. *See: processors.*
- **MIDI sequencer (ALSA)** - The kernel service that routes MIDI events between programs, synths and MIDI hardware. *See: graphics.*
- **MIME type (Multipurpose Internet Mail Extensions type)** - a standard name for a kind of file, such as `image/png`. *See: linux-os.*
- **MIO (Memory and I/O controller)** - the main chip of HP's zx1 chipset, linking Itanium 2 processors to memory and I/O. *See: buses-2.*
- **MMIO (Memory-Mapped I/O)** - device registers placed in the physical memory address space so the CPU reaches them with normal reads and writes. *See: buses-1, buses-2.*
- **Modalias** - A device's driver-matching ID string, e.g. `pci:v00001022d000014D8...`. Linux uses it to load the right driver. *See: buses-1.*
- **Module (kernel)** - A driver file loaded into the kernel at run time. "Built-in" means it is compiled into the kernel. *See: buses-1.*
- **Mount point** - The folder where a filesystem is attached to the folder tree. *See: storage.*
- **MRH (Memory Repeater Hub)** - An Intel 820/840-era chip that let RDRAM boards use SDRAM. *See: buses-1.*
- **MSI (message signaled interrupts)** - A way for PCI devices to send interrupts as memory writes instead of dedicated wires. *See: graphics.*
- **MSI / MSI-X (Message Signalled Interrupts)** - PCI Express interrupts sent as a memory write instead of a wire; each device or queue gets its own vector. *See: buses-2, machine.*
- **MSR** - Model Specific Register; an internal CPU control or counter register read with RDMSR, root only on Linux. *See: processors.*
- **MT/s** - mega-transfers per second; the memory data rate, twice the memory clock for DDR. *See: machine.*
- **MTP (Media Transfer Protocol)** - the protocol phones and media players use to share files over USB without exposing a raw disk. *See: buses-2.*
- **MTU (Maximum Transmission Unit)** - the largest packet a link carries, normally 1500 bytes on Ethernet. *See: network.*

- **Multicast** - one-to-many traffic: devices join a group address and receive what is sent to it. *See: network.*
- **Multikey quicksort** - a way of sorting text that compares one letter position at a time. *See: tools.*
- **Multipath** - Reaching one drive over more than one route; Linux combines the routes into one disk. *See: storage.*
- **Mutex (mutual exclusion)** - a lock that stops two threads using the same thing at the same time. *See: srs-items.*
- **MWIPS** - Million Whetstone Instructions Per Second, the score of the Whetstone benchmark. *See: tools.*

N

- **N-body simulation** - working out the gravity between every pair of a group of bodies (stars, planets); a classic floating-point workload. *See: tools.*
- **Namespace** - A slice of an NVMe SSD presented as one disk; consumer SSDs have one. *See: storage.*
- **NDP (Neighbor Discovery Protocol)** - the IPv6 version of ARP. *See: network.*
- **NetBIOS browsing** - the old Windows way of listing computers on a LAN through an elected master browser. *See: network.*
- **Netlink** - the socket type tools like `ip` use to talk to the Linux kernel. *See: network.*
- **Nice** - a process's politeness value (-20 to 19); higher nice means it gives way to other processes. *See: linux-os, srs-items.*
- **NMI (Non-Maskable Interrupt)** - an interrupt the CPU cannot ignore, used for watchdogs, profiling and serious hardware errors. *See: buses-2.*
- **Normalised value** - A SMART attribute's score on the maker's scale (often starting at 100 or 200), which goes down as things get worse. *See: storage.*
- **NPU (Neural Processing Unit)** - An AI accelerator such as AMD XDNA in Ryzen AI laptops. *See: buses-1.*
- **NQN** - NVMe Qualified Name: a unique name an NVMe subsystem reports about itself. *See: storage.*
- **NSS (Name Service Switch)** - glibc's list that decides where names are looked up and in what order. *See: network.*
- **NTFS** - New Technology File System: the main Windows filesystem; Linux reads and writes it with the ntfs3 driver. *See: storage.*
- **NTP (Network Time Protocol)** - the protocol that keeps the clock in sync with time servers over the network. *See: linux-os.*
- **NTT (number-theoretic transform)** - a relative of the FFT that works with whole numbers modulo a prime, used to multiply numbers with millions of digits quickly and exactly. *See: tools.*
- **NUMA** - Non-Uniform Memory Access; a design where memory is attached to particular CPUs, so "far" memory is slower. *See: machine, processors, tools.*
- **NUMA node** - A group of CPUs and memory that are close to each other. Desktops have one node, which Linux shows as -1 for PCI devices. *See: buses-1, graphics, storage.*
- **NVMe** - Non-Volatile Memory Express: the protocol fast SSDs use over PCIe. *See: storage.*
- **NVMe Composite** - An NVMe drive's own summary temperature, used for its thermal throttling. *See: sensors.*

- **NVRTC (NVIDIA runtime compiler)** - the part of the CUDA toolkit that turns GPU program text into GPU code while a program runs (`libnvrtc`). SRS used it for its CUDA tests before 2.3; now only when the driver refuses the built-in PTX. ZLUDA has none. *See: User Guide 9.20, tools.*
- **NX / XD** - No-Execute (AMD) / Execute Disable (Intel); marks memory as data-only so it cannot be run as code. *See: processors.*

O

- **O_DIRECT** - opening a file so reads and writes go straight to the drive, past Linux's page cache in RAM; benchmarks use it so they measure the drive, not the RAM. *See: tools.*
- **ODT, ProcODT (on-die termination)** - a resistance a chip switches on at its own pins so signals on the memory bus do not bounce back; ProcODT is the processor's. Set by the BIOS, in ohms. *See: machine.*
- **OpenCL** - An open standard for running general computing tasks on GPUs and other processors. *See: graphics.*
- **OpenGL** - The older cross-platform 3D graphics interface; still used by many programs and older games. *See: graphics.*
- **Option ROM** - firmware stored on a PCI card (listed as the ROM BAR), for example a graphics card's video BIOS. *See: buses-2, machine.*
- **OS latency** - how long the operating system takes to answer a program: system calls, task switches, wake-ups, timers. Lower and steadier means a more responsive PC. *See: tools.*
- **OUI (Organizationally Unique Identifier)** - the first three bytes of a MAC address, identifying the maker. *See: network.*
- **Overcommit** - the kernel promising programs more memory than it really has, betting they will not all use it at once. *See: linux-os.*
- **OverDrive** - AMD's name for user clock, voltage and power tuning. On Linux it appears as `pp_od_clk_voltage`. *See: graphics.*

P

- **P-core** - Performance core, a big, fast core on a hybrid CPU. *See: tools.*
- **Page fault** - what happens the first time a program touches a page of memory the kernel has not set up yet: the kernel stops the program for a moment and supplies the page. *See: tools.*
- **PLATYPUS** - a 2020 attack that used CPU energy readings to learn secrets (CVE-2020-8694/8695, CVE-2020-12912); since Linux 5.10 the powercap RAPL counters are root-only because of it. *See: User Guide 6.11.*
- **PMIC (power management IC)** - the small power chip on every DDR5 module that makes its voltages (VDD, VDDQ, VPP). SRS can read the voltages it is set to (an opt-in). *See: machine, User Guide 5.4.*
- **P-state** - a performance state: a running speed and voltage level of the CPU. *See: processors.*
- **PAC (proxy auto-config)** - a script that picks which proxy to use for each address. *See: network.*
- **Page** - the unit the CPU and kernel use to manage memory, usually 4 KB on x86. *See: srs-items.*
- **Page cache** - spare RAM the kernel uses to hold recently read or written file data. *See: linux-os.*

- **PARTUUID** - The unique ID of a partition, stored in the partition table (not in the filesystem). *See: storage.*
- **PCH (Platform Controller Hub)** - Intel's chipset; PCH temperature inputs on a Super I/O read 0 on AMD boards. *See: sensors.*
- **PCI address** - a device's location written as domain:bus:device.function, for example 0000:03:00.0. *See: buses-1, buses-2.*
- **PCI bridge (PtP)** - a PCI-to-PCI bridge; each slot, M.2 socket or chipset port is one and creates a new bus number. *See: machine.*
- **PCI function** - one logical PCI device; one chip can show several (for example graphics and audio). *See: srs-items.*
- **PCI ID** - the vendor, device, subsystem and revision numbers a PCI device reports. Linux uses them to pick a driver. *See: tools.*
- **PCI ID database** - The `pci.ids` file that turns vendor and device numbers into names. *See: buses-1.*
- **PCIe (PCI Express)** - The serial point-to-point version of PCI used for graphics cards, NVMe drives and most onboard devices. *See: buses-1, storage.*
- **PCIe link speed and width** - Speed is the transfer rate per lane (2.5, 5, 8, 16, 32 GT/s = PCIe 1 to 5); width is the number of lanes (x1 to x16). *See: graphics.*
- **PCIe link width** - the number of PCI Express lanes in use (x1, x4, x8, x16); bandwidth scales with it. *See: buses-2, srs-items.*
- **PCIe switch** - A chip with one upstream port and several downstream ports that share one link between devices. *See: buses-1.*
- **PCM (pulse-code modulation)** - Plain digital audio. An ALSA PCM device is one playback or recording stream. *See: graphics.*
- **PCR** - Platform Configuration Register; a TPM register holding a hash of what ran during boot. *See: machine.*
- **PECI / SB-TSI** - Side channels that let the Super I/O read the CPU temperature (PECI on Intel, SB-TSI on AMD). *See: sensors.*
- **Percentage Used** - NVMe health field: the drive's estimate of how much of its rated write life is used. *See: storage.*
- **PipeWire** - The modern Linux sound (and video) server. It mixes programs together and routes them to outputs. *See: graphics.*
- **PM profile** - The firmware's preferred power profile: 1 Desktop, 2 Mobile, 3 Workstation, 4 Enterprise server. *See: buses-1.*
- **PM table** - the power-management table the AMD SMU keeps, with live powers, clocks, voltages and temperatures. *See: processors, srs-items.*
- **PM1 block** - The ACPI I/O registers for sleep, wake and the power button. *See: buses-1.*
- **PM_TMR** - The ACPI power-management timer, which runs at 3.58 MHz. *See: buses-1.*
- **PMBus** - An SMBus-based protocol for smart power supplies and voltage regulators. *See: buses-1.*
- **PnP (Plug and Play)** - The firmware's description of built-in legacy devices, with IDs like `PNP0501`. *See: buses-1.*
- **PnP ID** - A 3-letter vendor code plus 4 hex digits (`PNP0501`), or with ACPI a 4-letter code plus 4 hex digits (`AMDI0030`). *See: buses-1, buses-2, graphics.*
- **Pointer chase** - a memory test where each load needs the address from the previous load, so prefetching cannot hide the delay. *See: tools.*
- **polkit** - the Linux service that decides which desktop users may perform admin actions, and whether a password is needed. *See: linux-os, srs-items.*
- **Port** - a number (0-65535) that picks which program on a machine receives a packet. *See: network.*

- **Port path** - the chain of USB port numbers from the root hub to a device, for example 2.3.4. *See: buses-2.*
- **Power cap** - the power limit firmware holds a chip to. *See: srs-items.*
- **PPI** - Physical Presence Interface; how the operating system asks the firmware to clear or enable the TPM at the next boot. *See: machine.*
- **PPT (Package Power Tracking)** - AMD's measured power for a CPU or GPU package, and the limit applied to it. *See: graphics, sensors, srs-items.*
- **Preemption** - how quickly the kernel can interrupt a running task to run another; `full` gives the lowest latency. *See: linux-os.*
- **Preferred core (prefcore)** - a per-core ranking from the CPU telling the scheduler which cores boost best. *See: processors.*
- **Prefetchable** - A BAR whose memory can be read ahead and cached safely, such as graphics memory. *See: buses-1.*
- **Prefix length** - the "/24" or "/64" after an address: how many leading bits are the network part. *See: network.*
- **Proton** - Valve's version of Wine, built into Steam, that runs Windows games on Linux. *See: graphics.*
- **Proxy** - a middle server that makes web requests on your behalf. *See: network.*
- **PSP (Platform Security Processor)** - A small security processor inside AMD chips (the `MP0` block); it loads and checks other firmware. *See: graphics, machine.*
- **PSS (Proportional Set Size)** - a process's RAM use with shared pages split fairly among the processes sharing them. *See: linux-os.*
- **PSU efficiency (80 PLUS, Cybenetics)** - the share of the wall power that reaches the parts; 90 % means 10 % turns into heat in the power supply. 80 PLUS and Cybenetics certify it at set loads. *See: User Guide 6.11.*
- **PTP (Picture Transfer Protocol)** - the older camera file-transfer protocol that MTP is built on; uses USB class 06 (Still Image). *See: buses-2.*
- **PTX (Parallel Thread Execution)** - NVIDIA's portable GPU code, a step between program text and the code of one GPU. SRS 2.3 carries its CUDA tests as PTX; the graphics driver (or ZLUDA) finishes it for the card when a test starts. *See: tools.*
- **PWM (pulse-width modulation)** - A way to control fan speed by switching power on and off quickly; the duty cycle sets the speed. *See: graphics, sensors.*

Q

- **Q1T1, Q8T1, Q32T16** - storage test settings: Q = queue depth (how many requests wait at once: 1 is like most desktop programs, 32 like a busy server), T = threads. *See: tools.*
- **qdisc (queueing discipline)** - the kernel's waiting line for outgoing packets, such as fq or fq_codel. *See: network, srs-items.*
- **QST (Quiet System Technology)** - Intel fan control built into the management engine of some 2007-2010 desktop boards. *See: sensors.*
- **Queue depth (Q), threads (T)** - Q = how many requests one thread keeps waiting at the drive at once, T = how many threads do so; "Q32T16" keeps 512 requests in flight. *See: tools.*
- **Quirk (USB)** - a kernel workaround flag for a device that does not follow the USB standard exactly. *See: buses-2.*

R

- **radeonsi** - Mesa's OpenGL driver for AMD GPUs. *See: graphics.*
- **Radix sort** - sorting numbers by their digits (or bytes) instead of comparing them with each other. *See: tools.*
- **RADV** - Mesa's Vulkan driver for AMD GPUs. *See: graphics.*
- **RAID** - Redundant Array of Independent Disks: joining disks for speed (raid0), safety (raid1 mirrors) or both. *See: storage.*
- **Rank** - a set of memory chips on a stick that is accessed together; sticks are single- or dual-rank. *See: machine.*
- **RAPL (Running Average Power Limit)** - A CPU feature that counts energy used; Linux exposes it under /sys/class/powercap, named intel-rapl even on AMD. *See: processors, sensors.*
- **Raw value** - A SMART attribute's actual counter (sectors, hours, degrees); its format is up to the maker. *See: storage.*
- **Ray tracing** - drawing a picture by following rays of light from the eye into a scene, with shadows and reflections. *See: tools.*
- **Regex** - regular expression, a text search pattern (for example `^` = start of line, `.` = any character). *See: tools.*
- **Register-blocked matrix multiply** - a way of writing a matrix multiply in which each GPU thread works out several results at once from numbers kept in its own registers, so the arithmetic units are kept busy instead of waiting for memory. GPU Burnin uses it since 2.4. *See: tools.*
- **Regular expression (regex)** - a search pattern, e.g. `nvme[0-9]` matches nvme0, nvme1 and so on. *See: srs-items.*
- **Report descriptor** - The HID record that tells the PC which buttons, axes and LEDs a device has. *See: buses-1.*
- **Resizable BAR** - A PCIe feature that lets the CPU see all of the GPU's VRAM at once instead of through a 256 MB window. AMD calls it Smart Access Memory. *See: buses-1, buses-2, graphics, machine.*
- **Revision ID** - The PCI number that tells chip steppings or versions apart. *See: buses-1.*
- **rfkill** - the Linux kernel's on/off switch for radios (Wi-Fi, Bluetooth, mobile data); soft = software, hard = physical switch. *See: network.*
- **ROCm** - AMD's open GPU computing platform (HIP, OpenCL, libraries) for Linux. *See: graphics.*
- **Root complex** - The part of the CPU where PCIe starts. On AMD it is the device at `00:00.0`. *See: buses-1.*
- **Root hub** - The virtual hub that stands for a USB host controller's ports. Each USB 3 controller has a 2.0 root hub and a 3.x root hub. *See: buses-1, buses-2.*
- **Round trip (compression)** - compress, then decompress, then compare with the original; it must be identical. *See: tools.*
- **Round trip time** - the time for a message to go to the other end and come back. *See: tools.*
- **RPC** - Remote Procedure Call, a way for one program to send commands to another over a network connection. *See: tools.*
- **RPM (revolutions per minute)** - Fan or pump speed. *See: sensors.*
- **RSS (Resident Set Size)** - the RAM a process is really using right now. *See: linux-os.*
- **RTC (real-time clock)** - the battery-backed clock on the motherboard that keeps time while the PC is off. *See: linux-os, machine.*

- **RTT (RttNom, RttWr, RttPark)** - the memory chips' own termination settings for different moments (idle, writing, parked), written as Off or RZQ/n (RZQ = 240 ohms). Set by the BIOS. *See: machine.*
- **Runtime PM** - Kernel power management that lets an idle device sleep while the system is running. *See: graphics.*
- **ryzen_smu** - a third-party kernel driver (AUR `ryzen_smu-dkms-git`) that gives Linux AMD Ryzen's PM table and read access to its internal registers (SMN). SRS reads clocks, voltages and, through srs-root, the memory timings through it; it is not part of SRS. *See: processors, machine.*

S

- **SAE (Simultaneous Authentication of Equals)** - the WPA3 password handshake, safer than WPA2's. *See: network.*
- **Samba** - the Linux implementation of Windows file and printer sharing (SMB). *See: network.*
- **SATA** - Serial ATA: the cable and protocol for most 2.5-inch and 3.5-inch drives. *See: storage.*
- **sched_ext** - a Linux feature that lets a CPU scheduler be loaded as a BPF program while the system runs. *See: linux-os, srs-items.*
- **SCI** - System Control Interrupt; ACPI's shared interrupt line, usually IRQ 9. *See: machine.*
- **sclk** - The GPU's shader (core) clock. *See: graphics.*
- **SCSI** - Small Computer System Interface: a command set Linux uses for SATA, USB and SAS storage. *See: storage.*
- **SCT** - SMART Command Transport: an ATA channel for extra data such as temperature history and error recovery limits. *See: storage.*
- **SDMA (system DMA)** - The GPU's copy engines, which move data between memory areas without using the 3D engine. *See: graphics.*
- **Section (ELF)** - one named part of a program file: `.text` code, `.data` variables, `.rodata` fixed data, `.bss` zeroed memory. *See: srs-items.*
- **Secure Boot** - a UEFI feature that only starts boot loaders and kernels signed with trusted keys. *See: linux-os, machine.*
- **Session (busy session)** - in Power & Cost, a stretch of heavy use (a game, a render) found from the CPU and graphics load; its energy is the whole PC's while it lasted. *See: User Guide 6.11.*
- **Setup Mode** - UEFI Secure Boot state with no Platform Key installed, in which keys can be changed freely. *See: machine.*
- **SEV** - Secure Encrypted Virtualization; AMD server feature that encrypts virtual machine memory. *See: processors.*
- **SHA-256, SHA-NI** - SHA-256 makes a 256-bit "fingerprint" of data (package checks, backups, git); SHA-NI are CPU instructions that speed it up. *See: tools.*
- **Shared library** - a `.so` code file many programs use at once, such as `libc.so.6`. *See: srs-items.*
- **Sieve of Eratosthenes** - finding prime numbers by crossing out the multiples of each prime; what remains are the primes. *See: tools.*
- **Sink / source** - In a sound server, a sink is where sound goes (speakers, HDMI); a source is where sound comes from (microphone, line in). *See: graphics.*
- **SLAAC (stateless address autoconfiguration)** - the way an IPv6 device builds its own address from the router's advertisements. *See: network.*
- **Slab** - the kernel's allocator for its own small objects; "reclaimable" slab is cache it can drop. *See: linux-os.*

- **SLAT** - Second Level Address Translation (Intel EPT, AMD RVI/NPT); hardware help that makes virtual machine memory fast. *See: processors.*
- **Slicing-by-8** - a way of computing CRC checksums 8 bytes at a time with lookup tables. *See: tools.*
- **SMART (Self-Monitoring, Analysis and Reporting Technology)** - Self-Monitoring, Analysis and Reporting Technology: the health log a drive keeps about itself. *See: srs-items, storage.*
- **SMB (Server Message Block)** - the Windows file-sharing protocol. *See: network.*
- **SMBIOS (System Management BIOS)** - System Management BIOS; the firmware inventory of board, BIOS, CPU socket and memory slots. *See: buses-1, machine, srs-items.*
- **SMBus (System Management Bus)** - an I2C-based bus on the motherboard that links RAM SPD chips and sensors. *See: buses-1, srs-items.*
- **SMN (System Management Network)** - AMD's internal register network for the Data Fabric and memory controllers. Since 2.4 srs-root reads the memory controller's timing registers on it through the ryzen_smu driver (read only) for Memory Timings. *See: buses-1, machine, processors.*
- **SMT** - Simultaneous Multi-Threading (Intel: Hyper-Threading); one physical core runs two threads, shown as two CPUs. *See: processors, srs-items, tools.*
- **SMT sibling** - the other logical CPU that shares the same physical core. *See: tools.*
- **SMU (System Management Unit)** - System Management Unit; a small processor inside a Ryzen CPU that controls boost, voltage and power limits. *See: graphics, processors, srs-items.*
- **Socket** - a program's handle for one network conversation or local channel. *See: network.*
- **soname** - the version number in a shared library's file name (the 6 in `libc.so.6`); it changes only when compatibility breaks. *See: linux-os.*
- **SPD (Serial Presence Detect)** - Serial Presence Detect; the data on each memory stick describing its size, speeds, timings and profiles. *See: buses-1, machine, srs-items.*
- **spd5118** - the SPD hub chip on DDR5 sticks, and its Linux driver; it also provides a temperature sensor. *See: machine.*
- **Speculative execution** - the CPU guessing ahead to run faster; flaws in it (Spectre, Meltdown) can leak data and are closed by mitigations. *See: processors.*
- **SPIR-V** - the compiled form of a Vulkan compute or graphics program (shader); SRS ships its GPU shaders pre-compiled as SPIR-V. *See: tools.*
- **SQLite** - a small database kept in one file; SRS keeps the Power & Cost history in one. *See: User Guide 6.11.*
- **srs-root** - the SRS root helper, started from Menu > Run as root; it saves root-only data (such as real resource addresses) for SRS to read. *See: buses-2, srs-items.*
- **SSD** - Solid-state drive: storage on flash memory, with no moving parts. *See: storage.*
- **SSDT** - Secondary System Description Table; extra ACPI code tables, often one per firmware module. *See: machine.*
- **SSID (Service Set Identifier)** - a Wi-Fi network's name. *See: network.*
- **_STA** - the ACPI status value of a device; SRS shows it as hex plus letters P (present), E (enabled), S (shown), F (functioning). *See: machine.*
- **Standing charge** - what an electricity supplier charges per day whatever you use. *See: User Guide 6.11.*
- **statvfs** - The system call that returns a mounted filesystem's size, free space and block size. *See: storage.*
- **Stratum (NTP)** - how many steps a time server is from a reference clock; stratum 1 is directly attached to one. *See: linux-os.*
- **Subsystem ID** - The second PCI ID pair, which names the board or card maker and their product built around a chip. *See: buses-1.*

- **Subvolume** - A btrfs folder tree that can be mounted on its own, like a partition inside the filesystem. *See: storage.*
- **Super I/O** - A motherboard chip that reads fans, voltages and board temperatures and drives fan headers (Nuvoton NCT, ITE IT87 ...). *See: buses-1, sensors, tools.*
- **Super-I/O** - a motherboard chip (Nuvoton, ITE and others) that reads fan speeds, voltages and board temperatures. *See: srs-items.*
- **SuperSpeed / SuperSpeed+** - USB 3 speeds: 5 Gbps (SuperSpeed, USB 3.2 Gen 1) and 10 or 20 Gbps (SuperSpeed+, Gen 2 and Gen 2x2). *See: buses-2.*
- **SVI** - Serial VID Interface; the link a CPU uses to ask the board's voltage regulator for a voltage and read it back. *See: processors.*
- **SVM / AMD-V** - AMD's hardware virtualization feature (Secure Virtual Machine). *See: processors.*
- **Swap** - disk space or compressed RAM used when real RAM is full; the Linux version of the Windows paging file. *See: linux-os, machine, storage.*
- **Swappiness** - a kernel setting for how eagerly memory is moved to swap. *See: srs-items.*
- **sysctl** - a kernel setting you can read and change at run time, under `/proc/sys`. *See: srs-items.*
- **sysfs** - The `/sys` folder tree where the kernel shows devices and their settings as small text files. *See: buses-1, graphics, storage.*
- **System call** - a request from a program to the operating system kernel (open a file, read the clock, start a thread); the cost of crossing from the program into the kernel and back. *See: tools.*
- **systemd target** - a named group of systemd units that the system boots into, similar to a Windows boot mode. *See: linux-os.*
- **systemd unit** - a systemd configuration object (service, socket, timer...); a service unit is the Linux version of a Windows service. *See: linux-os.*

T

- **Taint flag** - a kernel record of an unusual condition since boot (unsigned module, hardware error, warning); it clears only on reboot. *See: linux-os.*
- **TBW** - Terabytes written: the amount of data an SSD maker rates the drive to survive. *See: storage.*
- **TCP (Transmission Control Protocol)** - the reliable, ordered connection protocol used by the web, SSH and most apps. *See: network.*
- **TCP states** - the stages a TCP connection goes through, such as Listen, Established, Close Wait and Time Wait. *See: network.*
- **Tctl** - AMD's control temperature, the CPU temperature value used for fan control and throttling. *See: sensors, tools.*
- **Tctl / Tccd** - AMD temperatures: Tctl is the control value used for fans and limits (may include an offset), Tccd is each core die. *See: processors.*
- **Thermal zone** - A firmware-defined temperature area with trip points (passive, active, hot, critical); common on laptops. *See: sensors, srs-items.*
- **Thermistor** - A resistor whose value changes with temperature; most board temperature probes are thermistors. *See: sensors.*
- **THP (Transparent Huge Pages)** - grouping memory into 2 MB pages automatically so the CPU spends less time on address lookups. *See: linux-os, srs-items.*
- **Thread state** - a one-letter code: R running, S sleeping, D waiting on hardware, T stopped, Z zombie. *See: srs-items.*

- **Threaded dispatch** - an interpreter technique where each instruction jumps straight to the next one's code instead of going back through one big switch. *See: tools.*
- **Threshold** - The SMART failure line for an attribute; a normalised value at or below it means the attribute has failed. *See: storage.*
- **Throttling** - the CPU lowering its clock to stay under a temperature or power limit. *See: tools.*
- **Thunderbolt** - Intel's high-speed port that carries PCI Express, DisplayPort and USB over one cable; USB4 is based on it. *See: buses-2.*
- **Tiered price** - an electricity price that changes after so many kWh a month. *See: User Guide 6.11.*
- **Time-of-use price** - an electricity price that depends on the hour and the day. *See: User Guide 6.11.*
- **Timer slack** - how much later than asked Linux may wake a sleeping thread, so that wake-ups can be grouped to save power; 50 µs by default. *See: tools.*
- **Tjmax** - the highest temperature a CPU allows before it throttles itself. *See: tools.*
- **TLB shootdown** - an interrupt one CPU sends others to make them forget cached address translations after memory mappings change. *See: buses-2.*
- **TLS handshake** - setting up the encrypted connection that https uses, before any data flows. *See: tools.*
- **TPM** - Trusted Platform Module; a security device that stores keys and measures the boot process. *See: machine.*
- **Transparent huge pages (THP)** - Linux handing out memory in 2 MiB pages instead of 4 KiB ones when it can, so the CPU needs fewer page-table look-ups. *See: tools.*
- **TRIM** - A command that tells an SSD which blocks no longer hold data, so it can erase them ahead of time. *See: storage.*
- **Trip point** - a temperature where the system acts: passive = slow down, critical = shut down. *See: srs-items.*
- **TSC (Time Stamp Counter)** - Time Stamp Counter; a CPU counter that rises at a fixed rate and is the fastest clock for the OS when stable. *See: linux-os, processors.*
- **TTL** - Time To Live, the number of router hops a network packet may still make. *See: tools.*
- **TTY** - A terminal or serial device in Linux (`ttyS0`, `ttyUSB0`, `pts/0`). *See: buses-1.*

U

- **UART (Universal Asynchronous Receiver-Transmitter)** - The chip behind a serial port, e.g. 16550A. *See: buses-1.*
- **UAS (USB Attached SCSI)** - USB Attached SCSI: the faster protocol for USB storage; the older one is Bulk-Only Transport (`usb-storage`). *See: buses-2, graphics, storage.*
- **udev** - The Linux device manager; it names devices, creates `/dev/disk/by-*` links and keeps a database of device properties. *See: buses-1, linux-os, storage.*
- **UDP (User Datagram Protocol)** - a simple send-and-forget protocol used by DNS, games and voice. *See: network.*
- **UEFI (Unified Extensible Firmware Interface)** - Unified Extensible Firmware Interface; the modern firmware that replaced the PC BIOS. *See: linux-os, machine.*
- **UEFI variable** - a named setting stored in the firmware's flash, such as BootOrder or SecureBoot. *See: machine.*
- **UID (user ID)** - the number the kernel uses for a user account; the name is only a label. *See: linux-os.*

- **UMC (Unified Memory Controller)** - AMD's memory controller, built into the CPU's Data Fabric. *See: buses-1, machine.*
- **UPS (Uninterruptible Power Supply)** - A battery unit that keeps the PC powered during a power cut. *See: sensors.*
- **USB (Universal Serial Bus)** - The common peripheral bus: 1.5 and 12 Mbps (USB 1), 480 Mbps (USB 2), 5/10/20 Gbps (USB 3.x). *See: buses-1.*
- **usb.ids** - the community list that maps USB vendor and product numbers to names, installed by the hwdata package. *See: buses-2.*
- **USB/IP** - a Linux feature that shares USB devices over the network; its virtual host controller is vhci_hcd. *See: buses-1, buses-2.*
- **USB4** - the newest USB standard (up to 40 or 80 Gbps) that can tunnel PCI Express and DisplayPort. *See: buses-2.*
- **usbfs** - the Linux interface that lets a user program talk to a USB device directly instead of through a kernel driver. *See: buses-2.*
- **UUID (Bluetooth service)** - Universally Unique Identifier: an ID stored in a filesystem, used to mount it reliably. *See: network, storage.*

V

- **VA-API (Video Acceleration API)** - The Linux interface for hardware video decoding and encoding. *See: graphics.*
- **VBIOS** - The graphics card's own firmware chip. It starts the card at power-on and holds its power and clock tables. *See: graphics.*
- **VCN / UVD** - AMD's video decode/encode engine. VCN is the current name; some chips still report it as UVD. *See: graphics.*
- **Vcore** - the main supply voltage of the CPU cores, measured by the motherboard's sensor chip. *See: processors.*
- **vDSO** - a small piece of kernel code mapped into every program so that reading the clock does not need a real system call. *See: tools.*
- **Vendor ID** - The 16-bit number that names a chip maker, e.g. 1022 AMD, 1002 AMD/ATI, 10EC Realtek. *See: buses-1.*
- **Vendor library (GPU)** - the GPU maker's own tuned maths library: hipBLAS (AMD ROCm), cuBLAS (NVIDIA CUDA) or CLBlast (OpenCL). *See: tools.*
- **VID** - Voltage Identification; the voltage the CPU requests from the regulator. *See: processors.*
- **VID / PID (Vendor ID / Product ID)** - the two 16-bit numbers that identify a USB device's maker and model, written VVVV:PPPP. *See: buses-2.*
- **VID:PID** - USB vendor ID and product ID, which identify a USB chip or device model. *See: storage.*
- **VKD3D-Proton** - A translation layer that turns Direct3D 12 calls into Vulkan, used by Proton. *See: graphics.*
- **VMD** - Volume Management Device: an Intel PCIe block that gathers NVMe drives for Intel RST and VROC. *See: storage.*
- **VPN (virtual private network)** - an encrypted tunnel that carries your traffic through another network or server. *See: network.*
- **VRAM** - The graphics card's own fast memory. *See: graphics.*
- **VROC** - Virtual RAID on CPU: Intel's RAID for NVMe drives attached to the processor. *See: storage.*

- **VRR (variable refresh rate)** - The monitor waits for the GPU's next frame instead of refreshing at a fixed rate. Sold as FreeSync, G-Sync Compatible or HDMI VRR. *See: graphics.*
- **VT-x / VMX** - Intel's hardware virtualization feature. *See: processors.*
- **Vulkan** - The modern cross-platform graphics and compute interface, used by most new Linux games and by DXVK and VKD3D-Proton. *See: graphics.*

W

- **WAN (wide area network)** - the internet side of your router. *See: network.*
- **WGP (work-group processor)** - Two compute units working as a pair on RDNA GPUs. ROCm OpenCL counts WGPs as compute units. *See: graphics.*
- **Whetstone** - a classic floating-point (decimal maths) CPU benchmark. Its score is given in MWIPS. *See: tools.*
- **Wh (watt-hour)** - energy used over time: 100 W for one hour is 100 Wh; 1,000 Wh is 1 kWh. *See: User Guide 6.11.*
- **Wine** - A compatibility layer that runs Windows programs on Linux. *See: graphics.*
- **WMI** - Windows Management Instrumentation; a way for firmware to offer vendor features through ACPI (PNP0C14 devices). *See: machine, sensors.*
- **WPA2 / WPA3** - Wi-Fi security standards; WPA3 is newer and stronger. *See: network.*
- **WPAD** - Web Proxy Auto-Discovery, a way for a PC to find a PAC script automatically. *See: network.*
- **WPBT** - Windows Platform Binary Table; lets firmware hand Windows a program to run at startup. *See: machine.*
- **WPD (Windows Portable Devices)** - the Windows name for MTP/PTP devices such as phones and cameras. *See: buses-2.*
- **Wrap-around (counter)** - a counter reaching its highest value and starting again from zero, like an odometer; SRS unwraps the CPU energy counter. *See: User Guide 6.11.*
- **Write-back cache** - A drive feature that confirms a write once it is in the drive's own fast memory. *See: storage.*
- **WS-Discovery (Web Services Discovery)** - the method modern Windows uses to find other PCs and printers on the LAN. *See: network.*
- **W (watt)** - how fast power is being used right now. *See: User Guide 6.11.*
- **WWN** - World Wide Name: a globally unique ID some drives report, used for stable device names. *See: storage.*

X

- **x86-64-v2 / v3 / v4** - CPU feature levels; v3 needs AVX2, v4 needs AVX-512. *See: srs-items.*
- **XDG autostart** - the desktop standard: `.desktop` files in an autostart folder run at login. *See: srs-items.*
- **XDG (freedesktop.org Cross-Desktop Group) folders** - the standard per-user folders such as `~/.config`, `~/.local/share` and `~/.cache`. *See: linux-os.*
- **XFS** - A high-performance Linux journaling filesystem. *See: storage.*

- **xHCI** - The USB 3 host controller standard. One xHCI controller also serves USB 2 and USB 1 devices. *See: buses-1, buses-2.*
- **XMC (Switched Mezzanine Card)** - an ANSI/VITA 42 plug-in module for industrial and embedded computers, connected over PCI Express. *See: buses-2.*
- **XMP** - Intel Extreme Memory Profile; faster memory settings stored in the SPD. *See: machine.*
- **xxHash** - a very fast non-cryptographic hash by Yann Collet, used to check data quickly; xxHash64 gives a 64-bit result. *See: tools.*

Z

- **zenergy** - An out-of-tree Linux driver that reads AMD energy counters per core and per socket. *See: sensors.*
- **ZLUDA** - A layer that lets programs written for NVIDIA CUDA run on AMD GPUs. *See: graphics.*
- **zram** - a RAM-based block device that stores compressed data, often used as fast swap. *See: linux-os, srs-items, storage.*
- **Zstandard (zstd), LZ4, xz, zlib** - common compression formats and libraries: lz4 is the fastest, zstd fast with good ratios, zlib (gzip / ZIP) the classic, and xz the smallest but slowest. *See: tools.*
- **zswap** - a compressed cache in RAM that sits in front of disk swap. *See: linux-os.*
- **zx1** - HP's chipset for Itanium 2 servers and workstations. *See: buses-2.*